GAOL v5

NOT Just Another Interval Library

User's manual and reference

Edition 5.0, 6 November 2026

for version 5.0.0 of the library

Jordan Ninin

ENSTA, France

Frédéric Goualard

LS2N, Nantes Université, France

Copyright © 2001 Swiss Federal Institute of Technology, Switzerland
Copyright © 2002 Institut de Recherche en Informatique de Nantes (IRIN), France
Copyright © 2002–2016 LINA UMR CNRS 6241, France
Copyright © 2017–2020 LS2N UMR CNRS 6004, France
Copyright © 2026 ENSTA, France

All trademarks, copyrights and trade names are the property of their respective owners, even if they are not specified below.
Part of the work was done while Frédéric Goualard was a postdoctoral fellow at the *Swiss Federal Institute of Technology, Lausanne, Switzerland*, supported by the *European Research Consortium for Informatics and Mathematics* fellowship programme.
This is edition 5.0 of the documentation of GAOL v5, which follows the manual of GAOL 4 by Frédéric Goualard. It is consistent with version 5.0.0 of the `gaol` library.
GAOL is distributed under the GNU Library General Public License, version 2, and CORE-MATH, whose sources GAOL compiles into its library, under the MIT licence; both are reproduced in Appendix B. The files GAOL v5 adds carry the copyright of ENSTA, France, and are distributed under the same licence as GAOL; the files of GAOL that GAOL v5 changes keep the copyright of their authors.

Permission is granted to make and distribute verbatim copies of this manual provided the copyright notice and this permission notice are preserved on all copies.

GAOL IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THIS SOFTWARE IS WITH YOU. SHOULD THIS SOFTWARE PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

Contributors and acknowledgements

GAOL was initially designed and written by Frédéric Goualard (https://frederic.goualard.net), who was its main implementor from its first version, named `cell` (2001), to GAOL 4.

GAOL v5 is written by Jordan Ninin, at ENSTA, for Codac and IBEX, whose intervals are built upon GAOL. It continues GAOL from its version 4.2.2, and brings the changes GAOL needs to compile and compute right on every system it can.

The function `interval pow(const interval&, const interval&)` was designed by Marc Christie. The code for the multiplication and the division is largely inspired by the one presented by Tim Hickey, Qun Ju and Maarten H. van Emden in *Interval Arithmetic: from Principles to Implementation* [7].

GAOL v5 takes up:

- the patch IBEX applies to GAOL, by Gilles Chabert (initialization of the floating-point unit, rounding direction of `midpoint()` and `operator<<`);
- the fork of GAOL by Fabrice Le Bars, for Visual C++, MinGW and the systems the `configure` of GAOL does not know;
- the CMake build of GAOL in IBEX, by Cyril Bouvier and Gilles Chabert, with the compilation flags of the IBEX fork of Fabrice Le Bars;
- the rounding tests of Codac, which its tests follow.

Its elementary functions are those of CORE-MATH [13], by Alexei Sibidanov, Paul Zimmermann and the other authors of the project (Inria), whose sources are distributed with GAOL and compiled into its library.

The layout and the writing of this manual follow the PhD thesis of Jordan Ninin (Université de Toulouse, 2010). The text of the manual of GAOL 4 by Frédéric Goualard is kept wherever GAOL v5 behaves as GAOL 4.

Abstract

GAOL is a C++ library for arithmetic on floating-point intervals. Its intervals enclose real numbers between two doubles, and every operation returns an interval that contains the exact result, whatever the rounding errors of the computer: this is the containment property, on which interval algorithms rest to prove what they compute. GAOL also provides the *relational* versions of its operations, the reverse functions of IEEE 1788-2015, which interval constraint programming needs. GAOL v5, which this manual describes, continues GAOL as Frédéric Goualard developed it, and is written so that it compiles and computes right on every system it can: Linux, macOS and Windows, on x86, x86-64, 32-bit and 64-bit ARM, POWER (ppc64le), s390x and RISC-V (riscv64).

- Every elementary function that CORE-MATH provides is bounded with it, which is correctly rounded in the rounding direction in effect: the value computed at a bound, rounding upward, is the upper bound, and the double below it the lower one, the tightest bounds. The integer powers, the roots of an order other than 2 and 3 and `compoundm1`, which GAOL computes itself, are the tightest bounds or within a few doubles of them (Chapter 11). The bounds are the same on every architecture and with every compiler, and no mathematical library is to be installed or linked: CORE-MATH is compiled into GAOL.

- GAOL v5 follows IEEE 1788-2015 in the operations GAOL provides: the empty set, the infinite bounds, the domains of the functions, the comparisons and the interval literals. The namespace `gaol_ieee1788` gives each operation of the standard the name and the argument order the standard gives it, and the tightness of every operation is documented, as the standard requires.

- The rounding direction is the business of GAOL: each operation that computes a bound sets the direction upward when it is not, whichever direction the calling code left, and GAOL can be built to restore the direction of the program after each operation; the operations of the AVX-512 path, which that build takes where the processor has the instructions, set no direction, their instructions carrying it.

- Numbers are read exactly, and intervals written in decimal are rounded outward, whatever the C library; written in hexadecimal, an interval is read back as the same doubles.

- Three builds, CMake, autotools and meson, configure GAOL the same way; the tests compare the bounds GAOL computes with their exact values, and a continuous integration builds and tests GAOL on Linux, macOS and Windows, with the compilers and on the processors it supports.

This manual is both a user's guide and a reference. It presents the notions of interval arithmetic GAOL relies on, the installation and the use of the library, each of its operations with examples whose outputs were produced by GAOL v5, the names of the standard, the accuracy of the operations, the way the intervals are computed and measured, the tests and the continuous integration, and what a program written for GAOL 4 has to change.

Keywords: interval arithmetic, floating-point arithmetic, IEEE 1788-2015, correctly rounded elementary functions, CORE-MATH, reverse functions, constraint programming, C++.

Contents

Introduction 1

1 Interval arithmetic and floating-point numbers 5
- 1.1 Floating-point numbers and rounding . . . 6
- 1.2 Intervals . . . 7
- 1.3 Interval arithmetic and the containment property . . . 9
- 1.4 The dependency problem . . . 11
- 1.5 IEEE 1788-2015 . . . 11
- 1.6 Relational (reverse) functions . . . 13
- 1.7 The elementary functions and the mathematical libraries . . . 15
  - 1.7.1 The table maker's dilemma . . . 15
  - 1.7.2 libm, the mathematical library of the system . . . 16
  - 1.7.3 mathlib, the IBM Accurate Portable Mathematical Library . 16
  - 1.7.4 CRlibm, the proofs . . . 17
  - 1.7.5 CORE-MATH, correct rounding made adoptable . . . 18
  - 1.7.6 What separates them . . . 18
  - 1.7.7 What it means for the bounds of GAOL . . . 19
- 1.8 The accuracy modes . . . 20
- 1.9 Discussion . . . 21

2 Installation 23
- 2.1 Supported platforms . . . 23
- 2.2 Getting the software . . . 24
- 2.3 Prerequisites . . . 24
  - 2.3.1 Mandatory tools . . . 24
  - 2.3.2 Compilers and options refused . . . 25
  - 2.3.3 Tools for maintainers . . . 29
  - 2.3.4 Optional tools . . . 30
- 2.4 The three builds . . . 30
- 2.5 Building and installing with CMake . . . 33
  - 2.5.1 The version of GAOL . . . 34
  - 2.5.2 Options . . . 36
  - 2.5.3 What the configuration decides . . . 38
  - 2.5.4 The settings of the developers . . . 40

2.5.5 The archive of the sources and the packages 42
2.6 Building and installing with autotools 44
2.7 Building and installing with meson 46
2.8 What the options are worth 48
2.8.1 GCC for Windows 50
2.8.2 Interprocedural optimization is not offered 53
2.9 Tests, examples, performance and the parser 54
2.10 What is installed 55
2.10.1 The configuration header 56
2.11 Discussion 59

3 Using GAOL **61**
3.1 A first program 61
3.2 The headers 63
3.3 Compiling a program using GAOL 64
3.3.1 From CMake 66
3.3.2 From pkg-config 67
3.3.3 From meson 68
3.4 The namespaces 69
3.5 The powers in the two namespaces 70
3.6 Integers 72
3.7 A result thrown away 73
3.8 Tools of interval algorithms 74
3.9 Errors, in short 76
3.10 Common errors 77
3.10.1 Floating-point arithmetic and rounding 77
3.10.2 Functions of numbers 77
3.10.3 Midpoint and radius 78
3.10.4 The doubles of the program 78
3.10.5 Zero 78
3.10.6 The sign of a zero bound 78
3.10.7 The empty set and the domain of a function 79
3.10.8 Relations, sorting and containers 79
3.10.9 Both namespaces 79
3.10.10 Bisection 80
3.10.11 Newton's method: the division and the relational division . 80
3.10.12 Reading text 81
3.10.13 Threads and the global settings 82
3.10.14 The flags of interval arithmetic 82
3.11 Coming from another library 83
3.12 Discussion 85

4 The rounding direction and the floating-point environment 87
4.1 The rounding direction . . . 88
4.2 What the upward rounding does to the program . . . 90
4.3 Preserving the rounding direction . . . 93
4.4 The AVX-512 path . . . 94
4.5 The NEON intervals . . . 96
4.6 The FPU intervals . . . 98
4.7 The control registers of x86 processors . . . 103
4.8 Flush-to-zero and denormals-are-zero . . . 104
4.9 The floating-point exceptions . . . 107
4.10 Initialization and cleanup . . . 112
4.11 The rounding functions of the FPU . . . 115
4.12 Discussion . . . 115

5 Creating intervals 117
5.1 Constructors . . . 117
5.1.1 The four constructors . . . 118
5.1.2 Intervals read from a string . . . 119
5.1.3 A midpoint and a radius . . . 121
5.1.4 Decimal constants . . . 121
5.1.5 Infinite point intervals . . . 122
5.2 Straight assignment . . . 123
5.3 Assignment combined with an operation . . . 123
5.4 Interval constants . . . 125
5.5 Discussion . . . 126

6 Interval relations 129
6.1 Two groups of relations . . . 129
6.2 Set relations . . . 130
6.3 Certainly relations . . . 134
6.4 The order of IEEE 1788-2015 and the overlap state . . . 136
6.5 Relation symbols . . . 138
6.6 Intervals in containers and algorithms . . . 140
6.7 Interval-specific relations . . . 141
6.8 Discussion . . . 144

7 Interval arithmetic: the operations 147
7.1 Functional and relational operations . . . 147
7.2 Functional arithmetic . . . 148
7.2.1 Basic operations . . . 149
7.2.2 Powers and roots . . . 151
7.2.3 Exponentials and logarithms . . . 153
7.2.4 Trigonometric functions . . . 155
7.2.5 Hyperbolic functions . . . 157

7.2.6 Other functions . . . 158
7.3 Relational arithmetic . . . 160
7.3.1 $(n+1)$-ary relational functions . . . 160
7.3.2 The reverse functions of the standard . . . 168
7.4 Cancellative addition and subtraction . . . 168
7.5 Discussion . . . 169

8 Interval functions **171**
8.1 Numeric functions . . . 171
8.2 Integer functions . . . 177
8.3 Splitting methods . . . 179
8.4 Union and intersection . . . 181
8.5 Discussion . . . 182

9 Input and output **183**
9.1 Reading intervals . . . 183
9.1.1 The input format . . . 184
9.2 Writing intervals . . . 190
9.2.1 Converting intervals to strings . . . 191
9.2.2 The output format . . . 191
9.2.3 The number of digits . . . 194
9.2.4 Example . . . 195
9.3 Discussion . . . 196

10 The names of IEEE 1788-2015 **197**
10.1 The namespace of the standard . . . 197
10.2 Where the standard and GAOL differ . . . 198
10.3 The operations . . . 199
10.4 Reading with the names of the standard . . . 203
10.5 What is not provided . . . 203
10.6 Discussion . . . 204

11 Accuracy of the operations **205**
11.1 The conditions . . . 206
11.2 The tightness of each operation . . . 206
11.2.1 Basic operations . . . 209
11.2.2 Integer and absmax functions . . . 210
11.2.3 Powers, exponentials and logarithms . . . 211
11.2.4 Trigonometric and hyperbolic functions . . . 213
11.2.5 Recommended functions . . . 215
11.2.6 The error functions . . . 218
11.2.7 Reverse functions . . . 219
11.2.8 Set operations and numeric functions . . . 221
11.3 The same bounds on every machine . . . 224

11.4 Discussion . . . 226

12 How GAOL computes its intervals **227**
12.1 The representation of an interval and the kernels . . . 228
12.1.1 The SSE2 and NEON kernels . . . 228
12.1.2 When the build takes the AVX-512 path . . . 230
12.1.3 CORE-MATH is scalar . . . 231
12.2 Where the time of a program goes . . . 231
12.3 The rounding direction, by processor . . . 233
12.4 What the compilers give . . . 235
12.5 GAOL compared with the other libraries . . . 237
12.5.1 Performance . . . 241
12.5.2 The special cases . . . 249
12.5.3 The enclosure of the elementary functions . . . 250
12.6 ITF1788 and the benchmark of Tang et al. . . . 251
12.7 The verdicts on new paths . . . 253
12.7.1 Cheap gains that need no new instruction set . . . 255
12.7.2 What a new path has to show . . . 256
12.7.3 Things to watch . . . 257
12.7.4 What was not measured . . . 258
12.8 Discussion . . . 259

13 Floating-point numbers, errors and debugging **261**
13.1 Floating-point constants . . . 261
13.2 Floating-point functions . . . 262
13.3 Version information . . . 264
13.4 Additional functions . . . 264
13.5 Error handling . . . 265
13.5.1 Exceptions . . . 266
13.5.2 The `std::invalid_argument` exception . . . 268
13.5.3 The `std::domain_error` exception . . . 268
13.5.4 Warnings . . . 268
13.6 Debugging facilities . . . 269
13.7 Discussion . . . 270

14 Tests and continuous integration **271**
14.1 The principles of the tests . . . 271
14.2 The programs of the test suite . . . 274
14.2.1 The arithmetic and the elementary functions . . . 276
14.2.2 The rounding direction and the floating-point environment . . . 278
14.2.3 Reading and writing . . . 281
14.2.4 The functions, the standard and the reverse functions . . . 282
14.2.5 CORE-MATH, compoundm1 and the 128-bit integer . . . 284
14.2.6 The expressions . . . 286

14.2.7 The builds and the compilers . . . 287
14.2.8 The unit tests of GAOL 4 . . . 288
14.2.9 The benchmark . . . 289
14.3 The examples . . . 290
14.4 The continuous integration . . . 295
14.4.1 The speed of a pull request . . . 299
14.4.2 Configurations refused or left out . . . 301
14.4.3 What these platforms showed . . . 304
14.5 The scripts of the repository . . . 305
14.6 Coverage . . . 309
14.7 Discussion . . . 310

15 Moving a program from GAOL 4 to GAOL v5 **313**
15.1 Compiling and linking . . . 314
15.2 Results that change . . . 316
15.3 The differences, by theme . . . 317
15.3.1 The elementary functions and CORE-MATH . . . 318
15.3.2 The rounding direction and the initialization . . . 320
15.3.3 The floating-point exceptions and the flush-to-zero modes . . . 324
15.3.4 The constants and the static objects . . . 326
15.3.5 The kernels: SSE2, NEON, AVX-512 and the FPU . . . 326
15.3.6 The constructors, the empty set and the interval functions . . . 329
15.3.7 The powers and the roots . . . 331
15.3.8 The trigonometric, hyperbolic and other functions . . . 333
15.3.9 The relations . . . 335
15.3.10 Reading and writing intervals . . . 336
15.3.11 The errors . . . 338
15.3.12 The namespaces, the headers and the interface . . . 339
15.3.13 The builds, the compilers and the options . . . 340
15.4 Discussion . . . 343

Conclusion **345**

Bibliography **347**

A Documentation and references **349**
A.1 Documentation on GAOL . . . 349
A.2 References . . . 350
A.3 Reporting bugs . . . 351

B Library copying **353**
B.1 The licence of GAOL . . . 353
B.2 The licence of CORE-MATH . . . 362

Index **365**

List of Tables

1.1 The four mathematical libraries GAOL has been built with 19

2.1 The options of the three builds 36
2.2 The settings of the developers 40
2.3 The options of the meson build 47
2.4 The options on x86: nanoseconds per operation, Intel i7-1185G7, Clang 21.1, the best of nine runs 48
2.5 The options on 64-bit ARM (Apple M2, AppleClang 21.0) 49
2.6 GCC for Windows on x86-64: nanoseconds per operation (MSYS2 UCRT64, GCC 16.2) 51
2.7 GCC for Windows on 32-bit x86: nanoseconds per operation and instructions of the objects 51
2.8 The targets of the three builds 54
2.9 The macros of `gaol/gaol_configuration.h` 57

3.1 The power of `gaol` and of `gaol_ieee1788` on some arguments . . . 71
3.2 The names of GAOL v5, of `gaol_ieee1788`, and of IBEX and Codac 83
3.3 The names of GAOL v5, of C-XSC and of Boost.Interval 84
3.4 The meaning of the symbols `<`, `<=` and `==` in each library 84

4.1 What the program computes while GAOL is initialized and after `cleanup()` 91
4.2 The cost of `GAOL_PRESERVE_ROUNDING`, with and without the AVX-512 path 94
4.3 The FPU intervals on an Apple M2 (`GAOL_SIMD=OFF`) 100
4.4 The FPU intervals on the processors of the continuous integration . 101

5.1 The interval constants, static methods of the class `interval` 126

6.1 The relations of IEEE 1788-2015 and the methods of GAOL v5 . . . 131
6.2 The 13 states of Allen returned by `overlap()` 137

7.1 The reverse functions of IEEE 1788-2015 and the relational functions of GAOL 168

10.1 The operations of IEEE 1788-2015 in `gaol_ieee1788`, and their names in GAOL 200

11.1 The tightness of each operation of `interval` 207

12.1 What each build of GAOL computes with vector instructions 229
12.2 The SSE2 intervals against the AVX-512 path 231
12.3 Where the time of the demonstration programs goes 232
12.4 The rounding direction on each processor family 234
12.5 The six implementations compared 237
12.6 The comparison in short 239
12.7 The operations of the benchmark of `doc/compare/performance.md`. 241
12.8 GAOL v5 against GAOL 4.2.3, in nanoseconds per operation 242
12.9 Time per operation of the six libraries, in nanoseconds 244
12.10 The long lines of arithmetic operators between 8 intervals 247
12.11 Time per evaluation of a line, in nanoseconds 247
12.12 The same lines on a 64-bit ARM processor (Apple M2) 248
12.13 The verdicts on the tracks considered for a path of GAOL (from `doc/simd.md`). 253

14.1 The programs of the test suite 274
14.2 The examples of `examples/` 291
14.3 The platforms of the continuous integration 295
14.4 The configurations refused or left out 301
14.5 The scripts of the generated files, the manual and the coverage . . . 306
14.6 The audit of the three builds and the checks of the continuous integration 306
14.7 The scripts of the tests (`tests/`) and their tools (`tests/tools/`) . . 308
14.8 The coverage of the tests on 4 November 2026 310

Introduction

Computers compute with floating-point numbers, and a floating-point number is rarely the real number a program means: 0.1 has no finite binary expansion, the quotient 1/3 is rounded, and the rounding errors of a long computation add up in ways that are hard to follow. Interval arithmetic answers this difficulty by computing with pairs of floating-point numbers: an *interval* $[a, b]$ stands for every real number between a and b, and each operation on intervals returns an interval that contains all the results the operation can take on the reals it encloses. A program that computes with intervals proves, at the end of its computation, that the exact result lies within the interval it obtained. This is the containment property, which the first chapter defines, and which every operation of GAOL honours.

GAOL[1] is a C++ library to perform arithmetic with floating-point intervals. Its development was initiated at the *Swiss Federal Institute of Technology*, Lausanne, Switzerland, while Frédéric Goualard was a postdoctoral fellow supported by the *Swiss National Science Foundation*. It started as a limited version of *Jail* (now *halloween*), a templated C++ interval library developed during Goualard's PhD. To our knowledge, a unique feature of GAOL among the C++ interval arithmetic libraries available when it was written is the implementation of *relational arithmetic operators*, which interval constraint programming requires (Chapter 7). Hence the pun of the name: GAOL is *not* JAIL (*Just Another Interval Library*). The standard for interval arithmetic, IEEE 1788-2015 [9], has since required such operators, as its *reverse functions*.

GAOL v5, written by Jordan Ninin at ENSTA, continues GAOL as Frédéric Goualard developed it, from its version 4.2.2. It was written for Codac and IBEX, whose intervals are built upon GAOL, and brings the changes GAOL needs to compile and compute right on every system it can: Linux, macOS and Windows, on x86, x86-64, 32-bit and 64-bit ARM, POWER (ppc64le), s390x and RISC-V (riscv64). What it brings, and this manual describes, is the following.

- **Correctly rounded elementary functions.** Every elementary function that CORE-MATH [13] provides is bounded with it, which is correctly rounded in the rounding direction in effect: the value GAOL computes at a bound, rounding upward, *is* the upper bound, and the double below it the lower one,

[1] For those readers who are not native English speakers, "gaol" should be pronounced jāl, like the word "jail", of which it is a, chiefly British, variant.

the tightest bounds. The integer powers, the roots of an order other than 2 and 3 and `compoundm1`, which GAOL computes itself, are the tightest bounds or within a few doubles of them (Chapter 11). The bounds are the same on every architecture and with every compiler, and there is no mathematical library to install, to find or to link along with GAOL: CORE-MATH is compiled into it.

- **IEEE 1788-2015.** GAOL v5 follows the standard for interval arithmetic in the operations GAOL provides: the empty set, the infinite bounds, the domains of the functions, the comparisons and the interval literals. The few cases where it differs, mostly the `pow` of the namespace `gaol`, are stated where the operations are described, and the tightness of each operation is documented, as the standard requires (Chapter 11).

- **New operations**, which the standard requires or recommends: `exp2`, `exp10`, `log2`, `log10`, `fma`, the integer functions `sign`, `trunc`, `round_ties_to_even` and `round_ties_to_away`, sixteen recommended functions (`expm1`, `log1p`, `hypot`, `compoundm1`, `sinpi`...), the cancellative subtraction and addition, the radius, the comparisons `less` and `strictly_less`, the boolean functions `is_entire` and `is_common_interval`, `overlap()`, the reverse functions of the power and of `atan2` (powRev1, powRev2, atan2Rev1, atan2Rev2), pownRev for a non-positive exponent and mulRevToPair; `atan2`, which GAOL declared, is implemented.

- **The names of the standard.** The namespace `gaol_ieee1788` gives each operation of the standard that GAOL provides the name and the argument order the standard gives it (Chapter 10).

- **The rounding direction is the business of GAOL.** Each operation that computes a bound sets the rounding direction upward when it is not, whichever direction the calling code left; the operations of the AVX-512 path set none, their instructions carrying the direction. The user no longer has to keep it upward (Chapter 4).

- **Exact input and output.** Numbers are read exactly, and intervals written in decimal are rounded outward, whatever the C library; written in hexadecimal, an interval is read back as the same doubles (Chapter 9).

- **Three builds that agree.** GAOL has a CMake build, which is the one to use, and the autotools and meson builds, and the three configure GAOL the same way. Tests compare the bounds GAOL computes with their exact values, and a continuous integration builds and tests GAOL on Linux, macOS and Windows, with the compilers and on the processors listed in Chapter 2.

A program written for GAOL 4 needs a few changes, which Chapter 15 lists: first of all, `gaol/gaol.h` no longer opens the namespace `gaol`.

About this manual

This document is both a manual and a reference to use GAOL. It assumes a prior knowledge of interval arithmetic, whose notions the first chapter recalls in the form GAOL uses them; the books and papers by Goldberg, Neumaier, Moore, Alefeld and Herzberger, and others [2, 8, 10, 1, 11] give a full presentation of floating-point arithmetic, interval arithmetic and the use thereof. The main entry point for interval arithmetic on the Web is the *Interval Computation site* of Vladik Kreinovich, `http://www.cs.utep.edu/interval-comp/`.

The pages of the directory `doc` of the sources give the details of each change GAOL v5 brings, with its measures, and this manual takes their content up; Appendix A lists them. The mark [GAOL v5] follows, throughout the text, what behaves otherwise than in GAOL 4. A clause number in parentheses, as (12.10.3), and a table cited as Table 9.1 or Table 10.7 are those of IEEE 1788-2015 (the tables of this manual are numbered by chapter and linked).

The examples of the reference show what they print, and every output shown was produced by GAOL v5. A script of the sources compiles each example that shows its output under its code, runs it and compares what it prints with the manual (Chapter 14); the outputs shown apart, in a frame Output, are not compared by it. An example that writes a `bool` sets `boolalpha` first: without it, `cout` writes 1 and 0. The examples assume `using namespace gaol;` and `using namespace std;`, except those of Chapter 10, which open `gaol_ieee1788` instead, and each example is a program of its own.

Classes, methods, functions, macros, constants and variables available in the library but not described in this document are likely to change or to be removed. Consequently, they should be used with caution, if at all. Among them are the intervals of floats, `gaol::intervalf` and `gaol::interval2f`, which are unfinished and which none of the builds compiles: only a developer of GAOL does, defining `GAOL_FLOAT_INTERVALS` in `gaol/gaol_config.h`.

Outline

The first chapter recalls the notions this manual relies on: floating-point numbers and their roundings, intervals and the containment property, the dependency problem, what IEEE 1788-2015 fixes, the idea of the relational functions, the mathematical libraries from which correctly rounded elementary functions came, and the accuracy modes of the standard.

Chapters 2 and 3 are the user's guide. The second chapter describes the installation of GAOL: the platforms it supports, the compilers and options it refuses, the three builds and their options, and what the configuration decides by itself. The third chapter shows how a program uses GAOL: a first program, the flags of interval arithmetic the code including the headers of GAOL is compiled with, the two

namespaces and their powers, the tools of interval algorithms, the common errors, and the correspondence with the names of other interval libraries.

Chapter 4 is devoted to the rounding direction and to the floating-point environment, which GAOL v5 takes charge of: what the upward rounding does to the rest of the program, how the direction can be preserved, the AVX-512 path, the NEON intervals of 64-bit ARM processors, the FPU intervals, the modes that flush the subnormal numbers to zero, the floating-point exceptions, and the initialization and the cleanup of GAOL.

Chapters 5 to 9 are the reference of the class `interval`: the constructors and the assignments, the relations, the arithmetic operations, functional and relational, the numeric and integer functions, the splitting methods and the set operations, and the reading and writing of intervals. Chapter 10 gives the names of IEEE 1788-2015, and where the standard and GAOL differ. Chapter 11 states the tightness of each operation, as the standard requires.

Chapter 12 explains how GAOL computes its intervals: the representation of an interval in a register, the SSE2, AVX-512, NEON and FPU kernels, where the time of a program goes, the cost of the rounding direction on each processor, what the compilers give, how GAOL compares with the other interval libraries, with ITF1788, the test suite of the standard, and with the benchmark of Tang et al., and the verdicts on new paths. Chapter 13 describes the floating-point constants and functions, the version information, the errors and the debugging facilities. Chapter 14 presents the tests, the examples, the continuous integration and the scripts of the repository.

The last chapter, Chapter 15, is written for the programs written for GAOL 4: what they have to change to compile with GAOL v5, and what they will see differently. The conclusion sums up what GAOL v5 established and what is left open. The bibliography comes before two appendices, which list the documentation and the references of GAOL, with the way to report a bug, and reproduce the licences of GAOL and of CORE-MATH; an index ends the manual.

Chapter 1

Interval arithmetic and floating-point numbers

A computer holds a real number as a floating-point number, a `double`, and a computation on doubles rounds each of its results: the quotient 1/3, the constant 0.1 and most values of the elementary functions have no exact representation, and a program that computes with them accumulates errors it does not see. Interval arithmetic replaces each real number by two doubles that enclose it, and each operation by an operation on intervals whose result encloses every value the exact operation can take. The errors are then not avoided but bounded: the result of an interval computation is a set that contains the exact result, whatever the roundings along the way. GAOL is a C++ library to perform arithmetic with floating-point intervals, and this chapter gathers the notions on which the rest of the manual relies.

In this chapter, we define the floating-point numbers and their roundings, the floating-point intervals and the operations on them, and the vocabulary of IEEE 1788-2015, the standard for interval arithmetic that GAOL v5 follows. The first section describes the doubles and the roundings downward, upward and to nearest. The second defines the intervals, the empty set, the unbounded intervals and the canonical intervals. The third states the containment property and the outward rounding that enforces it, with the property on which GAOL computes both bounds in a single rounding direction. The fourth presents the dependency problem, which makes an enclosure wider than the range. The fifth gives an overview of IEEE 1788-2015 as GAOL v5 follows it, and the sixth the idea of the relational functions, which the standard calls reverse functions. The seventh explains why the bounds of an elementary function depend on the mathematical library that computes it, from the libm of the systems to CORE-MATH; the eighth defines the three accuracy modes of the standard, tightest, accurate and valid. The chapter ends with a discussion.

1.1 Floating-point numbers and rounding

The numbers GAOL computes with are the `double`s of C++: the floating-point numbers of the double format of IEEE 754 [8, 2], called binary64 since the 2008 revision of the standard. A double is written on 64 bits: a sign, a significand of 53 bits, 52 of them stored, and an exponent of 11 bits; its value is $\pm m \cdot 2^e$ with m an integer below 2^{53}. We write $\mathbb{F}$ for the set of the doubles, the two infinities included, and $\mathbb{R}$ for the set of the real numbers. $\mathbb{F}$ is finite and symmetric: $y \in \mathbb{F}$ if and only if $-y \in \mathbb{F}$.

The doubles are not spread evenly on the real line. Between two consecutive powers of two, 2^e and 2^{e+1} with $-1022 \leqslant e \leqslant 1023$, they are 2^{52} in number, spaced by 2^{e-52}: this spacing is the *unit in the last place* (ulp) of the doubles between these powers. The *normal numbers* are those of magnitude between 2^{-1022}, the smallest normal double, and $\text{MAX} = (2 - 2^{-52}) \cdot 2^{1023} \approx 1.797693 \cdot 10^{308}$, the largest double, which C names `DBL_MAX`. Below 2^{-1022} lie the *subnormal numbers*, the multiples of 2^{-1074}, which fill the gap between the smallest normal double and zero with the same spacing. The bounds GAOL computes may be subnormal, and the modes of the processors that flush the subnormal numbers to zero break them (see Section 4.8, page 104).

Three kinds of values are special. The two zeros, -0 and $+0$, are equal in the comparisons and stand for the same number; IEEE 1788-2015 and GAOL take them for the same bound, and the sign a zero bound gets differs between the builds of GAOL (see Section 3.10.6, page 78). The two infinities, $-\infty$ and $+\infty$, which an overflow or a division by zero produces, are the bounds of the unbounded intervals. NaN (*not a number*) is the result of an invalid operation, as $0/0$ or $\sqrt{-1}$; it compares unequal to everything, itself included, and GAOL holds the empty set as an interval whose two bounds are NaN (Section 1.2, page 7).

A real number that is no double has to be rounded. IEEE 754 defines four rounding directions for the binary formats of the processors: to nearest, toward $-\infty$, toward $+\infty$ and toward zero. The processor holds the direction in effect in a control register, and every operation of its floating-point unit rounds its result in that direction; a program starts in the rounding to nearest. Interval arithmetic uses the first three, which we define now.

Definition 1.1.1 (Round down, round up) *Given $\mathbb{R}$ the set of real numbers and $\mathbb{F}$ the set of floating-point numbers (`double`), we have, for all $x \in \mathbb{R}$:*

$$\downarrow x \downarrow = \max\{y \in \mathbb{F} \mid y \leqslant x\},$$
$$\uparrow x \uparrow = \min\{y \in \mathbb{F} \mid y \geqslant x\}.$$

$\downarrow x \downarrow$ is the rounding of x downward, toward $-\infty$, and $\uparrow x \uparrow$ its rounding upward, toward $+\infty$.

Definition 1.1.2 (Round to nearest) *Given* $\mathbb{R}$ *the set of real numbers and* $\mathbb{F}$ *the set of floating-point numbers (*`double`*),* $\updownarrow x \updownarrow$ *is, for* $x \in \mathbb{R}$*, the* $y \in \mathbb{F}$ *that minimizes* $|x - y|$*:*

$$|x - \updownarrow x \updownarrow| = \min\{|x - y| \mid y \in \mathbb{F}\},$$

the one whose significand is even when two are equally near; a real of magnitude at least $2^{1024} - 2^{970}$ *is rounded to the infinity of its sign, as IEEE 754 does.*

For a double d, the three roundings return d itself: a double is never moved. For a real x that is no double, $\downarrow x \downarrow$ and $\uparrow x \uparrow$ are two consecutive doubles, and $\updownarrow x \updownarrow$ is one of them. Beyond the largest double, the definitions give $\uparrow x \uparrow = +\infty$ and $\downarrow x \downarrow = \mathrm{MAX}$ for $x > \mathrm{MAX}$.

Definition 1.1.3 (Neighbours of a double) *For a double* d*,* $d^+ = \min\{y \in \mathbb{F} \mid y > d\}$ *is the double that follows* d*, and* $d^- = \max\{y \in \mathbb{F} \mid y < d\}$ *the double that precedes it. For a real* x *that is no double,* $\uparrow x \uparrow = (\downarrow x \downarrow)^+$*. The function* `next_float()` *of GAOL computes the successor of a double (Chapter 13).*

Example 1.1.4 *Programming with floating-point numbers is one of the few activities where the compiler has to be considered with distrust. Let us consider the following piece of code:*

Example

```
1  #include <gaol/gaol>  // We do not import the gaol namespace
2
3  using gaol::interval;
4
5  int main(void) {
6    interval one_tenth(0.1); // BEWARE: wrong!
7
8    [some code using one_tenth]
9    gaol::cleanup();
10 }
```

Though a rational perfectly representable in decimal, `0.1` *is not representable in binary, at least not with a finite number of bits, and thus requires rounding. The purpose of the programmer was to define an interval containing this value. However,* `0.1` *is rounded* at compile time*, most certainly to the nearest representable floating-point number* $\updownarrow 0.1 \updownarrow$*. As a consequence,* `one_tenth` *is a degenerate interval containing only* $\updownarrow 0.1 \updownarrow$*, and the containment property (Section 1.3, page 9) is violated. The right way to write such a constant is given in Example 1.2.4.*

1.2 Intervals

Definition 1.2.1 (Floating-point interval) *A floating-point interval* $I = [a, b]$*, with* a *and* b *in* $\mathbb{F}$*,* $a \leqslant b$*,* $a < +\infty$ *and* $b > -\infty$*, is the set of the real numbers between its bounds:*

$$[a, b] = \{x \in \mathbb{R} \mid a \leqslant x \leqslant b\}.$$

$a = \underline{I}$ is its lower bound, or left bound, and $b = \overline{I}$ its upper bound, or right bound. The empty set $\varnothing$ is an interval as well. We write $\mathbb{I}$ for the set of the floating-point intervals.

An interval is a set of real numbers: an infinite bound marks that the interval is unbounded on that side, and is no member of it. The intervals with an infinite bound, as $[1, +\infty]$, are the *unbounded* intervals, intervals as any other; $[-\infty, +\infty]$ is the whole real line, which IEEE 1788-2015 calls entire and GAOL `interval::universe()`. The point intervals $[+\infty, +\infty]$ and $[-\infty, -\infty]$ do not exist: the standard has no such interval, and the constructors of GAOL v5 give the empty set for them (see Section 5.1.5, page 122). [GAOL v5] An interval $[a, a]$ reduced to one double is a *degenerate* interval, or point interval, which the literals of the standard write $[a]$. The empty set is the result of an operation whose argument has no point in the domain of the function (Section 1.5, page 11); GAOL holds it as an interval whose two bounds are NaN, and the method `is_empty()` tells it.

Definition 1.2.2 (Hull) *The hull of a set S of real numbers, written $\square S$, is the smallest floating-point interval containing S: $\square \varnothing = \varnothing$, and for a non-empty S*

$$\square S = [\downarrow \inf S \downarrow, \uparrow \sup S \uparrow],$$

where $\inf S = -\infty$ when S has no lower bound and $\sup S = +\infty$ when it has no upper bound.

Definition 1.2.3 (Canonical interval) *A non-empty interval $I = [a, b]$ is canonical if and only if $a \geqslant b^-$: it contains at most two consecutive floating-point numbers.*

The hull of a real number x that is no double, $\square\{x\} = [\downarrow x \downarrow, \uparrow x \uparrow]$, is canonical: it is the tightest interval of doubles that encloses x, and no element of $\mathbb{I}$ does better.

Example 1.2.4 *The right way to deal with rational constants that might not be perfectly representable as floating-point numbers is to write them as strings, and to read them with* `gaol::textToInterval()` *(see Section 5.1, page 117), so that they are correctly rounded downward and upward at* run time*:*

Example

```
#include <gaol/gaol>

using gaol::interval;

int main(void)
{
  // OK: this is the right way
  interval one_tenth = gaol::textToInterval("0.1");
  // Another possible way
  interval one_tenth2 = gaol::textToInterval("1/10");

  [some code using one_tenth]
  gaol::cleanup();
  return 0;
}
```

Now, `one_tenth` *is the smallest floating-point interval enclosing* 0.1*: a canonical interval. Written in hexadecimal (see Section 9.2.2, page 191), which shows the bounds exactly, the interval of Example 1.1.4 and this one are:*

Output

```
interval(0.1) = [0x1.999999999999ap-4]
one_tenth     = [0x1.9999999999999p-4, 0x1.999999999999ap-4]
```

Width, midpoint and radius. Three numbers describe an interval $[a, b]$: its *width* $b - a$ (wid of IEEE 1788-2015), its *midpoint* $(a + b)/2$ (mid) and its *radius* $(b - a)/2$ (rad), so that $[a, b]$ is the set of the reals at a distance at most the radius from the midpoint. Computed on doubles, none of the three is exact in general: the width GAOL computes is rounded upward, and the midpoint is a double that the computation has to keep inside the interval, for an unbounded interval too [5]. Chapter 8 gives the methods `width()`, `midpoint()` and `rad()`, and their values on the empty set and on the unbounded intervals.

1.3 Interval arithmetic and the containment property

An operation on intervals stands for the operation on every pair of points of its arguments. Its result has to contain all the values the real operation can take and, to be of use, should contain as little else as possible: the smallest floating-point interval that does is the hull.

Definition 1.3.1 (Containment property) *The containment property of floating-point interval arithmetic requires that, for any operation "$\circ$" and any intervals I and J, $I \circ J$ contains $\{i \circ j \mid i \in I,\ j \in J\}$, the pairs on which $\circ$ is not defined being left out, and, for any function f of one variable and any interval I, that $f(I)$ contains $\{f(i) \mid i \in I\}$, the points of I outside the domain of f being left out. The containment principle asks for the smallest interval that does so, the hull:*

$$I \circ J = \square\{i \circ j \mid i \in I,\ j \in J\}, \qquad f(I) = \square\{f(i) \mid i \in I\}.$$

Example 1.3.2 *For the square root, the principle gives $\sqrt{I} = \square\{\sqrt{i} \mid i \in I\}$. From monotonicity considerations, the square root of $[1, 2]$ is then $\sqrt{[1, 2]} = [1, \sqrt{2}]$ on the real numbers; as $\sqrt{2}$ is no double, the interval computed is $[1, \uparrow\sqrt{2}\uparrow]$, whose upper bound is the double above $\sqrt{2}$.*

Fulfilling the containment property with doubles requires *outward rounding*: the lower bound of a result is rounded downward and its upper bound upward. For example, to add the intervals $[a, b]$ and $[c, d]$, we compute $[\downarrow a + c\downarrow, \uparrow b + d\uparrow]$, where $\downarrow r\downarrow$ and $\uparrow r\uparrow$ return the greatest double at most r and the smallest double at least r (Definition 1.1.1). These two operations are performed by switching the

rounding direction of the floating-point unit toward, respectively, $-\infty$ and $+\infty$. With $P = \{ac, ad, bc, bd\}$ and $Q = \{a/c, a/d, b/c, b/d\}$, the four basic operations on bounded intervals become

$$\begin{aligned}
[a, b] + [c, d] &= [\downarrow a + c\downarrow, \uparrow b + d\uparrow], \\
[a, b] - [c, d] &= [\downarrow a - d\downarrow, \uparrow b - c\uparrow], \\
[a, b] \times [c, d] &= [\downarrow \min P\downarrow, \uparrow \max P\uparrow], \\
[a, b] \div [c, d] &= [\downarrow \min Q\downarrow, \uparrow \max Q\uparrow] \quad \text{if } 0 \notin [c, d].
\end{aligned}$$

A divisor that contains zero gives the hull of the quotients by its non-zero points, which IEEE 1788-2015 defines case by case: the empty set when the divisor is $[0, 0]$, as $[1, 2] \div [0, 0] = \varnothing$; otherwise the interval $[0, 0]$ when the dividend is $[0, 0]$, and an unbounded interval when it is another nonempty interval, as $[1, 2] \div [-1, 1] = [-\infty, +\infty]$. The infinite bounds and the zeros of each operation follow the standard as well; Chapter 7 states the rules.

Proposition 1.3.3 *For every real number r, $\downarrow -r\downarrow = -\uparrow r\uparrow$.*

Proof. The set $\mathbb{F}$ is symmetric: $y \in \mathbb{F}$ if and only if $-y \in \mathbb{F}$. Hence $\{y \in \mathbb{F} \mid y \leqslant -r\} = \{-z \mid z \in \mathbb{F},\ z \geqslant r\}$, and the greatest element of the first set is the opposite of the smallest element of the second: $\max\{y \in \mathbb{F} \mid y \leqslant -r\} = -\min\{z \in \mathbb{F} \mid z \geqslant r\}$, that is, $\downarrow -r\downarrow = -\uparrow r\uparrow$. ∎

On most platforms, switching the rounding direction is costly. Proposition 1.3.3 cuts down the number of switches: nearly all downward rounding operations can be replaced by upward rounding ones by negating appropriately twice the operations performed. The lower bound of a sum, $\downarrow a + c\downarrow$, is $-\uparrow(-a) + (-c)\uparrow$: the sum of the negated lower bounds, rounded upward, then negated. GAOL stores the lower bound of an interval negated, as the pair $\langle -a, b\rangle$, and computes both bounds rounding upward: the rounding direction can then be set toward $+\infty$ once and for all. The sum of $\langle -a, b\rangle$ and $\langle -c, d\rangle$ is the pair $\langle \uparrow -a - c\uparrow, \uparrow b + d\uparrow\rangle$, two upward additions, which the kernels of GAOL compute on the two halves of one register (see Section 12.1, page 228). How GAOL sets the rounding direction, and what the upward rounding does to the rest of the program, is the subject of Chapter 4 (see Section 4.1, page 88).

Exact operations. Not every operation needs a rounding. The negation maps $[a, b]$ to $[-b, -a]$, whose bounds are doubles since $\mathbb{F}$ is symmetric; the minimum and the maximum return a bound of an argument, and the absolute value a bound of its argument, its opposite or 0; and the integer functions, floor, ceiling, truncation, the roundings to the nearest integer and the sign, return integers that are doubles, the integer part of a double being a double. These operations are exact: their result is the hull of the exact result with no rounding at all, whatever the rounding direction in effect. The four basic operations, the square root and the elementary functions round: their bounds need a rounding in a chosen direction, and, for the elementary

functions, depend on the mathematical library that computes them (Section 1.7, page 15).

1.4 The dependency problem

The containment principle makes each operation as tight as doubles allow, but a sequence of operations is not: an expression evaluated on intervals gives an interval that encloses the range of the expression, and is in general wider than it. The reason is that each occurrence of a variable ranges over its interval on its own, as if the other occurrences did not exist. This is the *dependency problem* of interval arithmetic [10, 11].

Example 1.4.1 *Let* $x = [0, 1]$. *The expression* $x - x$ *is zero for every value of* x, *but the subtraction of intervals takes its two operands as independent:* $[0, 1] - [0, 1] = [-1, 1]$. *In the same way, with* $x = [-1, 1]$, *the product* $x \times x = [-1, 1]$ *combines the value* -1 *of the first factor with the value* 1 *of the second, which one number cannot take at once, whereas the square, which knows that its two factors are equal, gives* $x^2 = [0, 1]$, *the range; GAOL provides it as* `sqr()` *(Chapter 7). The distributivity is weakened in the same way: for any intervals* x, y *and* z, *the operations being computed exactly on the reals,* $z \times (x + y) \subseteq z \times x + z \times y$, *with equality not in general, as* $[1, 2] \times ([-1, 1] + [3, 4]) = [2, 10]$ *where* $[1, 2] \times [-1, 1] + [1, 2] \times [3, 4] = [1, 10]$. *Computed with doubles and rounded outward, even the inclusion may fail, by a double or two.*

The effect grows with the expression. The first program of Chapter 3 (see Section 3.1, page 61) evaluates the Goldstein-Price function on the two intervals $[-2, 2]$ of its variables, and obtains an enclosure of its range that is 231 times as wide as the range: each occurrence of x and of y in the formula ranges over $[-2, 2]$ on its own, as if the others did not exist.

Cutting x and y into smaller intervals, and taking the hull of the values of f over the pieces, narrows the enclosure, at the price of more evaluations: the program `examples/03_dependency_problem.cpp` of the sources does it. Writing the expression with fewer occurrences of each variable, the square in place of a product of a variable by itself for instance, is the other remedy: the width of an enclosure is a property of the expression evaluated, not of the function alone.

1.5 IEEE 1788-2015

IEEE 1788-2015 [9] is the standard for interval arithmetic. It fixes what the libraries had each decided their own way: what an interval is, what the operations return at the bounds of their domains, how the empty set, the infinite bounds and the comparisons behave, how an interval is written and read, and how tight each operation has to be. GAOL v5 follows the standard in the operations GAOL provides: the

empty set, the infinite bounds, the domains of the functions, the comparisons and the interval literals. The few cases where it differs, mostly the `pow` of the namespace `gaol`, are stated where the operations are described (see Section 3.5, page 70, Chapter 10), and the tightness of each operation is documented, as the standard requires (Chapter 11).

The set-based flavor and the bare intervals. The standard is organized in *flavors*, and defines one, the *set-based flavor*: an interval is a closed connected subset of $\mathbb{R}$, that is, the empty set, a bounded interval $[a, b]$ or an unbounded one, and the exact result of an operation is the set of its values on the points of its arguments that lie in the domain of the function, which the interval returned encloses (Section 1.8, page 20). The intervals of GAOL are those of the set-based flavor (Section 1.2, page 7). The standard also attaches to its intervals a *decoration*, which records whether the function was defined and continuous on the whole of its argument along the computation (Clause 11). GAOL has no decorations: only bare intervals are provided, and the other parts of the standard it does not provide are listed in Chapter 10 (see Section 10.5, page 203).

The operations. The standard lists its operations in tables. Table 9.1 holds the forward elementary functions every implementation provides, with the domain of each: the arithmetic operations, the powers, the exponentials and logarithms, the trigonometric and hyperbolic functions and their inverses, the integer functions, abs, min and max. Table 10.1 of the standard holds the reverse functions (Section 1.6, page 13), Table 10.2 the numeric functions of an interval, Tables 10.3 and 10.4 the comparisons, and Table 10.5 the recommended functions, which an implementation should provide (10.6). GAOL v5 adds the operations of these tables that GAOL 4 lacked, which the standard requires or recommends, and the introduction lists them. [GAOL v5] The relations of Tables 10.3 and 10.4, equal, subset, interior, disjoint, precedes, strictPrecedes, less and strictLess, follow the definitions of the standard for the empty set and for the infinite bounds too (Chapter 6). [GAOL v5] The table of Chapter 10 says which operation of the standard each function of GAOL is.

The conventions. A few conventions of the standard run through the whole manual.

- **The domains.** A function is computed on the part of its argument within its domain, which is that of IEEE 1788-2015 (Tables 9.1 and 10.5): the part outside is left out, and an argument with no point of the domain gives the empty set, as `log`$([-4, 0])$ does. [GAOL v5] Every function gives the empty set for an empty argument. GAOL, which has no decorations, says nothing of the part left out: `sqrt`$([-1, 4])$ is $[0, 2]$.
- **The empty set.** The empty set is an interval, the result of an operation on an argument outside the domain of the function, and an interval whose

bounds are in the wrong order, as `interval(1, 0)`, is empty too. Its numeric functions are those of Table 10.2: inf and sup of the empty set are $+\infty$ and $-\infty$, where `left()` and `right()` of GAOL are NaN, and inf returns -0 for a left bound zero (12.12.8).

- **The infinite bounds.** An interval is a set of real numbers: an infinite bound is no member of it, so that isMember(m, x) is `false` for an infinite m (10.6.3), and the point intervals at $\pm\infty$ do not exist. The relations follow these definitions for the infinite bounds too.

- **The literals.** An interval is written as a *literal* of the standard (12.11): `[l, r]`, `[a]` for a degenerate interval, `[empty]` for the empty set and `[entire]` for the whole line, the case of the letters being ignored; a literal is read back as an interval containing the one written (13.2, 13.3). GAOL writes and reads them (Chapter 9, see Section 9.1.1, page 184); `textToInterval` of the namespace `gaol_ieee1788` reads the names of the functions of the standard and returns the empty set for a string that is no interval (12.1.3), where that of `gaol` reads the names of GAOL and throws (see Section 10.4, page 203). [GAOL v5]

- **The powers.** The pow of Table 9.1 is defined for $x > 0$, and for $x = 0$ when $y > 0$; the power with an integer exponent is pown(x, n), whose n is an integer rather than an interval. The `pow` of the namespace `gaol` takes the integer power for an integer exponent, a negative base included, which is the main point where GAOL and the standard differ (see Section 3.5, page 70, Section 7.2.2).

GAOL names its operations its own way, and the names of the standard differ: the reverse function coshRev(c, x) is `acosh_rel(c, x)`, and roundTiesToEven(x) is `round_ties_to_even(x)`. The type `interval` and the functions of GAOL live in the namespace `gaol`, and the namespace `gaol_ieee1788`, which `gaol/gaol.h` brings along, gives each operation of the standard that GAOL provides the name and the argument order the standard gives it, so that a reader of the standard finds it without looking for its translation (see Section 3.4, page 69, Chapter 10). [GAOL v5]

1.6 Relational (reverse) functions

A function on intervals, as the containment principle defines it, maps an interval of arguments to the hull of its values: this is its *functional* version. A second reading of the same function is of use when the function appears in a relation between variables.

Example 1.6.1 *The functional square root of* $[1, 2]$ *is* $\sqrt{[1, 2]} = [1, \sqrt{2}]$ *(Example 1.3.2). Another interpretation of the square root is*

$$\sqrt{}_r I = \square\{j \in \mathbb{R} \mid \exists i \in I \colon j^2 = i\},$$

the relational square root, which permits obtaining both negative and positive values: $\sqrt{}_r\,[1,\,2] = [-\sqrt{2},\,\sqrt{2}]$. *This operator arises when we consider the relation* $x^2 = y$, *which can alternatively be written* $x = \sqrt{}_r\,y$. *Here, the functional square root is not suitable, since it would confine* x *to* $[0,\,+\infty]$ *and lose the negative solutions of the relation.*

Some applications, mainly in the area of constraint programming, require such operators: a contractor narrows the domains of the variables of a relation, and needs, for the relation $y = f(x)$, the part of the domain of x whose image meets the domain of y. As a consequence, GAOL offers both functional and relational versions of the main arithmetic operators, which IEEE 1788-2015 has since required, as its *reverse functions* (Table 10.1).

A relational function of one argument, as the relational square root, is enough when the preimage of an interval has finitely many pieces. For a periodic function the preimage has infinitely many, and the domain of the variable has to be taken into account.

Example 1.6.2 *Consider the relation* $y = \cos x$, *where* x *and* y *are interval variables. One would like to express this relation in the equivalent way* $x = \operatorname{acos} y$. *However, the function* acos *cannot be used, because its result is always included in the interval* $[0,\,\pi]$. *What is needed is a relational version of* acos*; but since, for any value* y, *there are infinitely many values* x *verifying* $y = \cos x$, *the domain of* x *has to be taken into account. GAOL therefore defines a binary operator* `acos_rel` *as*

$$\texttt{acos_rel}(Y, X) = \square\{x \in X \mid \exists y \in Y : y = \cos x\},$$

to be contrasted with the functional arccosine,

$$\texttt{acos}(Y) = \square\{x \in \mathbb{R} \mid \exists y \in Y : x = \operatorname{acos} y\}.$$

Figure 7.1, in Chapter 7, presents the different results obtained when computing $\texttt{acos}(J)$ *or* $\texttt{acos_rel}(J, I)$, *where* J *and* I *play the parts of* Y *and* X. *An interval* X *that meets no piece of the preimage gives the empty set.* [GAOL v5]

Definition 1.6.3 (Relational function) *Let* f *be a function of one real variable,* Y *an interval of values and* X *an interval of arguments. The relational function of* f *with respect to* X, *its reverse function in the words of IEEE 1788-2015, is*

$$f_{\text{rel}}(Y, X) = \square\{x \in X \mid \exists y \in Y : y = f(x)\},$$

the hull of the part of X *whose image by* f *meets* Y. *The relational function of one argument is the case* $X = \mathbb{R}$.

GAOL names a relational function after the inverse of the function it reverses, as `acos_rel` for the cosine, `asin_rel` for the sine and `sqrt_rel` for the square; the standard names it after the function itself, cosRev, sinRev, sqrRev, and the namespace `gaol_ieee1788` gives these names (Chapter 10). The relational version of a binary operation, as the relational division `div_rel`, which reverses the product, takes one more argument. The reference of the relational arithmetic, with the figure of the relational cosine, is in Chapter 7 (see Section 7.3, page 160).

1.7 The elementary functions and the mathematical libraries

The bounds of the four basic operations and of the square root are the business of the processor: IEEE 754 requires them correctly rounded in the direction in effect, and the roundings of Definition 1.1.1 are what the floating-point unit computes. The elementary functions, exp, log, sin, cos, tan, their inverses and the hyperbolic functions, are computed by software, by a mathematical library, and the tightness of their bounds depends on it. GAOL v5 bounds its elementary functions with CORE-MATH [13], compiled into it from `3rd/math-core`, and takes no elementary function from any other library: the C library gives it only functions that are exact or correctly rounded, `sqrt`, `fma`, the integer roundings `floor`, `ceil`, `trunc` and `round`, `frexp`, `ldexp`, `fmod`, `nextafter` and the functions of `fenv.h`, and the integer powers, the roots of an order other than 2 and 3 and `compoundm1` are computed by GAOL itself. Before that, GAOL was built with one of three: mathlib by default with its `configure`, CRlibm, which its meson build took by default, or the mathematical library of the system; the three builds of GAOL v5 no longer have these options (Chapter 15). [GAOL v5] The four come from four moments of the same history and were written with different goals, which is why they give bounds of different tightness. This section says where each of them comes from and what separates them, and why GAOL v5 ended up with the last.

1.7.1 The table maker's dilemma

A mathematical library evaluates exp, log, sin… in a format, binary64 here, in which the exact result is not representable: it has to round a value it only knows approximately. For some arguments the exact result falls extraordinarily close to the middle of two consecutive doubles, or to a double for the directed roundings, and deciding which way to round then asks for far more than 53 bits of intermediate precision. No bound on that distance follows from the definition of the functions: this is the *table maker's dilemma*, named by W. Kahan. Two ambitions follow from it, and they are the axis along which the four libraries differ.

Definition 1.7.1 (Faithful rounding) *A library is faithfully rounded, or "nearly correct", when its error is below one unit in the last place: it returns one of the two doubles that enclose the exact value, that value itself when it is a double. The libraries of this kind are within about half a unit in practice. This is enough for most numerical computation.*

Definition 1.7.2 (Correct rounding) *A library is correctly rounded when it returns exactly the double the rounding direction in effect gives of the exact result:* $\updownarrow f(x) \updownarrow$, $\downarrow f(x) \downarrow$ *or* $\uparrow f(x) \uparrow$. *It makes a program reproducible from one machine and one library to another, and, for interval arithmetic, it gives the tightest bound a format can hold: a value correctly rounded upward* is *the upper bound, with nothing to add.*

IEEE 754-2008 lists the correctly rounded elementary functions among its recommended operations, which is what the last two libraries below set out to provide.

1.7.2 libm, the mathematical library of the system

libm is not a library but an interface: the mathematical part of the C standard library, with one implementation per system. The line starts with the libm of Unix V7 in the 1970s, then the libm of 4.3BSD in the middle of the 1980s, written around W. Kahan at Berkeley (Coonen, K. C. Ng, Thomas, Tang), where the culture of testing the accuracy of elementary functions was formed. In 1993 Sun published fdlibm, the Freely Distributable libm of K. C. Ng: portable C, no assembly, aiming at less than one unit in the last place. fdlibm is the common ancestor of nearly everything that followed: `msun` of FreeBSD, the libm of musl, openlibm of Julia, and part of that of the glibc.

Its design puts speed first and accuracy second. There is one evaluation phase, an argument reduction and a polynomial or rational approximation chosen by hand for each function; the error bound announced is empirical, established by tests rather than proved; the mathematical properties (monotonicity, symmetry, $\sin^2 x + \cos^2 x \leqslant 1$) are not guaranteed. Rounding to nearest is assumed throughout: nothing is promised in the directed rounding directions, which is precisely what interval arithmetic needs. Beside these portable implementations live the fast and non-portable ones: Intel SVML, AMD libm, the `optimized-routines` of ARM, and the vector variants.

This is why GAOL could not keep the libm of the system as an option: the bounds would not be certified, holding only where that libm stays within about one double of the exact value.

1.7.3 mathlib, the IBM Accurate Portable Mathematical Library

mathlib, also called libultim, was written at IBM Haifa at the end of the 1990s (Abraham Ziv, Moshe Olshansky, Ealan Henis, Anna Reitman). It is the industrial form of the idea Ziv published in 1991 in *ACM TOMS*: the *onion peeling* strategy. The function is evaluated with increasing precision, a fast phase accurate to about half a unit in the last place plus a little, then a second one, then a software multiprecision arithmetic, and the computation stops as soon as a rounding test proves that the rounding can be decided. Since the hard cases are rare, the average cost stays close to that of an ordinary libm, while the result is correctly rounded in practice.

Its two limits are the ones that decided its fate. Nothing bounds the number of iterations: the strategy has no proof and no bound on the worst case, which can cost thousands of cycles. And it rounds to nearest only, so a bound drawn from it has to be moved one double outward, which costs one unit in the last place of tightness:

this is what GAOL 4 did, built with mathlib, for every elementary function but the hyperbolic ones (Section 1.8, page 20).

mathlib was contributed to the glibc around 2001 and provided its `exp`, `log`, `sin`, `cos`, `atan` and `pow` for some fifteen years. In 2018 the glibc 2.28 removed these multiprecision slow paths: the pathological execution times were judged worse than the accuracy was worth. The glibc thereby gave up correct rounding and went back to about half to one unit in the last place with a bounded worst case. The version GAOL built was mathlib 2.1.1, under the GNU LGPL, as Frédéric Goualard distributed it; GAOL v5 carries it no longer. [GAOL v5]

1.7.4 CRlibm, the proofs

CRlibm is a project of the LIP at the ENS de Lyon and then of the INRIA Arénaire team (later AriC), from 2000 to about 2011: Jean-Michel Muller, Florent de Dinechin, Catherine Daramy-Loirat, David Defour, Christoph Lauter, with Guillaume Melquiond for the proof tools. Its last release, 1.0beta4, dates from 2010; it is no longer maintained. CRlibm keeps Ziv's two phases but changes what is guaranteed.

- **The second phase is bounded.** Lefèvre and Muller had computed, by an exhaustive search made tractable by their algorithms, the hardest-to-round cases in double precision: about 120 bits are known to be enough. No unbounded multiprecision is needed any more, a double-double or triple-double arithmetic suffices, and the worst-case execution time is bounded, typically a few times the average.

- **The error bounds are proved**, and in part checked by machine with Gappa. The documentation of CRlibm is as much a proof document as a manual, and this is its main scientific contribution.

- **The four rounding directions are provided explicitly**: `cos_rn`, `cos_rd`, `cos_ru`, `cos_rz`. This is what makes it natural for interval arithmetic, and why GAOL took its functions rounded downward and upward directly, without moving them outward: built with CRlibm, the `exp`, `log`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sinh` and `cosh` of GAOL were the tightest bounds of the value at the bound. CORE-MATH gives the same, in the direction in effect, for every function and on every system (Section 1.8, page 20).

Its price was the work of a specialist for each function, a narrow coverage (a dozen functions, binary64 only: neither `tanh` nor `asinh`, `acosh`, `atanh`), and an LGPL licence, which a libm that asks for a permissive one, as musl's does, cannot take; the glibc, itself under the LGPL, was asked in 2007 by one of the authors of CRlibm whether it wanted it, and has not taken it. Its direct descendant is Metalibm (de Dinechin, Kupriianova, Lauter): generating the code and its proof rather than writing them.

1.7.5 CORE-MATH, correct rounding made adoptable

CORE-MATH [13] started in 2021 at Inria (Caramba team, Nancy), by Paul Zimmermann, also behind MPFR, with Alexei Sibidanov, Tom Hubrecht, Sélène Corbineau and others. It draws the lessons of the three libraries above, and its design is as much a strategy of adoption as a technique.

- **The stated goal is correct rounding at least as fast as the current libms.** It answers the reason mathlib was removed from the glibc: if correct rounding costs nothing, the argument from latency falls.
- **The licence is MIT**, deliberately permissive, so that the code can be taken up by the glibc, the libm of LLVM or musl. It gives what a libm that asks for a permissive licence, musl's for one, wants and the LGPL of CRlibm did not give, and it works: routines of CORE-MATH have been integrated upstream. It is also what lets GAOL v5 take the sources it needs into `3rd/math-core`, with a few changes marked in the code, and compile them into its library.
- **The coverage is wide**: the functions of C23, in binary32, binary64 and binary128, the recent functions included.
- **The validation is graded**: in binary32 it is exhaustive, the 2^{32} arguments being enumerated and compared with MPFR; in binary64 and beyond, a proved fast phase and an accurate phase, with a search for the hard cases.
- **The four rounding directions are honoured**, the one in effect included. This is what GAOL v5 rests on: correctly rounded in the upward rounding GAOL computes in, CORE-MATH gives the tightest bounds without switching the rounding direction: the interval `log` of GAOL v5 takes 20.9 ns where that of GAOL 4.2.3, which calls mathlib, moves its value outward and switches the rounding direction twice, takes 46.8 ns (Intel i7-1185G7, Clang 21.1, 17 October 2026, see Section 12.5, page 237).

A proposal to add `cr_*` functions to the C standard comes from the same team, so that correct rounding can be asked for by the program rather than imposed on everyone. Beside it, and of an orthogonal design, is RLIBM (Rutgers, Jay Lim and Santosh Nagarakatte, about 2019–2022): instead of approximating the real function and then rounding, it looks by linear programming for a polynomial that directly yields the correct rounding for every input. Elegant, but practicable mainly for the short formats (binary32, bfloat16, posits).

1.7.6 What separates them

Table 1.1 sets the four libraries side by side. The line of the story is plain: libm established that a mathematical library had to be portable and fast; mathlib showed that correct rounding was reachable on average, and was rejected on its worst case;

CRlibm showed that it could be proved and bounded, and was not adopted, for want of coverage, of maintenance and, for the libms that ask for a permissive licence, of one; CORE-MATH takes all of it up and treats industrial adoption, that is, performance, MIT licence, C23 coverage and standardization, as a design constraint of the same rank as accuracy.

TABLE 1.1 – The four mathematical libraries GAOL has been built with

	libm (fdlibm, glibc)	mathlib (IBM)	CRlibm	CORE-MATH
Time	1980s, 1993–	about 1997–2001	2000–2011	2021–
Where from	Berkeley, Sun	IBM Haifa	ENS de Lyon, INRIA	Inria, CERN
Goal	speed, below 1 ulp	correct rounding in practice	correct rounding proved	correct rounding and speed
Method	one phase	unbounded onion peeling	two phases, the second bounded by the hard cases	proved phases, exhaustive in binary32
Evidence	tests	tests	Gappa, published proofs	proofs and exhaustive checks
Worst case	bounded	unbounded	bounded	bounded
Rounding	to nearest	to nearest	the four, explicit	the four, the one in effect
Formats	all	binary64	binary64	binary32, 64, 128
Licence	permissive (fdlibm), LGPL (glibc)	LGPL	LGPL	MIT
State	alive	frozen	abandoned	active, being adopted

1.7.7 What it means for the bounds of GAOL

The hierarchy is read directly in the tightness of the bounds, which Chapter 11 states function by function.

- **The mathematical library of the system** gave bounds that were not certified: the value rounded to nearest moved outward by $2 \cdot 2^{-52}$ relatively and by the smallest normal double, several doubles beyond the tightest one.
- **mathlib** was correctly rounded to nearest, hence one double outward: the bounds were accurate within one double.
- **CRlibm and CORE-MATH** are correctly rounded in the direction wanted, hence give the tightest bound of the value at the bound, with nothing to add.

This is also why GAOL v5 takes from CORE-MATH every elementary function CORE-MATH provides, above all the hyperbolic ones (neither mathlib nor CRlibm has them all, and the libm of the system is not accurate enough everywhere for its values moved outward to be bounds), and on every architecture and with every compiler, `gaol/math/gaol_u128.h` giving the accurate phases the 128-bit integer where the compiler has none. The bounds are the same on every architecture and with every compiler, and there is no mathematical library to install, to find or to link along with GAOL: CORE-MATH is compiled into it. [GAOL v5]

Further reading. The onion peeling strategy is in A. Ziv, *Fast evaluation of elementary mathematical functions with correctly rounded last bit*, ACM TOMS 17(3), 1991. The table maker's dilemma, the hard cases and the two-phase evaluation are in J.-M. Muller, *Elementary Functions: Algorithms and Implementation*, and in the *Handbook of Floating-Point Arithmetic*; the floating-point arithmetic itself is presented by Goldberg [2] and in the standard [8]. The documentation distributed with the sources of CRlibm holds the proofs of its error bounds. CORE-MATH is presented in [13] and at `https://core-math.gitlabpages.inria.fr/`; the file `3rd/README.md` of the sources says what GAOL v5 takes from it. The NEWS of the glibc 2.28 records the removal of the multiprecision slow paths inherited from mathlib.

1.8 The accuracy modes

The containment principle asks for the hull of the exact result, which the basic operations reach but which the elementary functions, computed by a library, may not. IEEE 1788-2015 therefore defines three accuracy modes (12.10.1), for an operation f, its exact value f^* and an input box x, and requires an implementation to document the tightness of each of its interval operations (12.10.3).

Definition 1.8.1 (Tightest) *f is tightest when $f(x)$ is the hull, in doubles, of $f^*(x)$: each bound is the nearest double on its side of the exact bound.*

Definition 1.8.2 (Accurate) *f is accurate when $f(x)$ encloses $f^*(x)$ and is within*

$$\mathrm{nextOut}\Big(f_{\mathrm{tightest}}(\mathrm{nextOut}(x))\Big),$$

nextOut *moving each bound one double outward: roughly, $f(x)$ is no wider than the tightest result for an input one double wider, plus one double on each side.*

Definition 1.8.3 (Valid) *f is valid when $f(x)$ encloses $f^*(x)$.*

The standard requires the basic operations (neg, add, sub, mul, div, recip, sqr, sqrt, fma), the integer functions and abs, min and max to be tightest; the other elementary functions, the reverse functions and the recommended functions to be

valid, and recommends them to be accurate (12.10.2). In this manual, "within k doubles" means that each bound is at most k doubles beyond the tightest one.

GAOL bounds its elementary functions with CORE-MATH, which is correctly rounded in the rounding direction in effect. Computing in the upward rounding it keeps, GAOL takes the value at a bound as the upper bound, with nothing to add, and the double below it as the lower one. The upper bound is the tightest one, and so is the lower bound: the double below the upper bound is the tightest lower bound when the exact value is no double, and where the exact value is a double, as for $\log(1) = 0$ and $\sin(0) = 0$, the operations return that value itself rather than the double below it (Remark 1.8.4). Where an algorithm evaluates the function at the bounds of the interval, the bounds of the interval are therefore the tightest ones. Before GAOL v5, built with mathlib, the default of its `configure`, the elementary functions but the hyperbolic ones were correctly rounded to nearest and moved one double outward, which is at most one double beyond the tightest bound but for the sine, computed as $\cos(x - \pi/2)$, and the real power, computed as $\exp(y \log x)$, which were wider: accurate or not, and not tightest. The hyperbolic functions came from the mathematical library of the system, whose values are sometimes further than one double from the exact ones (Remark 11.2.1). [GAOL v5]

Remark 1.8.4 *The double below the correctly rounded upper value is the tightest lower bound only when the exact value is no double. When it is, as for* $\log(1) = 0$*, the value correctly rounded upward is the exact value itself, and an operation that moved it one double down would be accurate, not tightest. The operations of GAOL return the exact value where it happens, and Chapter 11 says where.*

Chapter 11 documents the tightness of every operation of `interval`, from the algorithm of each operation, as the standard requires; the tests of `tests` check each statement on random and special arguments (Chapter 14), and the page `doc/accuracy.md` gives, for each, the algorithm and the largest distance the tests found. [GAOL v5] The statements hold for GAOL as the three builds make it, whatever the rounding direction the calling code left, on every architecture and with every compiler: for the SSE2, the NEON and the FPU intervals, and for the AVX-512 path, whose bounds are those of the SSE2 intervals but for the sign of a zero bound of a product with a double. They hold under the conditions that Chapter 11 states (see Section 11.1, page 206): the code including the headers of GAOL compiled with the flags of interval arithmetic (see Section 3.3, page 64), and the floating-point exceptions masked as a program starts (see Section 4.9, page 107). The intervals of floats are not covered.

1.9 Discussion

This chapter has set the vocabulary of the manual. A floating-point interval is a set of real numbers between two doubles, the empty set and the unbounded intervals

included; the containment property requires every operation to return an interval that encloses the exact result, and outward rounding is what enforces it on doubles. The property $\downarrow -r\downarrow = -\uparrow r\uparrow$ lets GAOL compute both bounds in the upward rounding, with the lower bound stored negated: a choice that spares the switches of the rounding direction, at the price of leaving the direction upward for the rest of the program, which Chapter 4 examines. The dependency problem is the limit of the method: an enclosure is as tight as the expression allows, not as the function does.

Two choices of GAOL v5 follow from the notions of this chapter. The standard IEEE 1788-2015 fixes the special cases, the empty set, the infinite bounds, the domains and the literals, and GAOL v5 follows it in the operations it provides, under two sets of names; it provides bare intervals only, without the decorations of the standard. The elementary functions are bounded with CORE-MATH, whose correct rounding in the direction in effect gives the tightest bounds without a switch of the rounding direction, where the libm of the system gave uncertified bounds and mathlib bounds one double too wide; the three accuracy modes of the standard, tightest, accurate and valid, are the words in which Chapter 11 states what each operation achieves.

The next chapter leaves the concepts for the software: how GAOL v5 is obtained, configured, built and installed on each platform (Chapter 2).

Chapter 2

Installation

GAOL v5 comes as source code, to be compiled for the computer it is used on. The sources of CORE-MATH, which compute the elementary functions, are part of it and are compiled into the library `libgaol` itself: there is no mathematical library to install, to find or to link along with GAOL, and no choice to make. The bounds GAOL computes are the same on every architecture and with every compiler, and they are the tightest ones for the elementary functions of CORE-MATH, and within a few doubles of them for a few other operations (Chapter 11).

GAOL is built with CMake, which is the build to use. The autotools and meson builds of GAOL are kept for those who use them. On a given machine with a given compiler, GAOL behaves the same whichever build configured it: the three give it the same macros and the same flags, and a few options, the same in the three, are all that the user chooses. The rest of the configuration is decided by checks of the compiler and of the machine.

In this chapter, we describe how GAOL v5 is obtained, built, tested and installed. The first sections give the platforms of the continuous integration, where to get the sources, the tools needed, and the compilers and the options that GAOL refuses because they gave bounds that did not enclose the exact results, or cannot be shown to enclose them. Section 2.4 states what the three builds agree on. Sections 2.5 to 2.7 describe each build, its options and the settings of the developers. Section 2.8 measures what the options are worth. The last sections present the targets of the tests and what an installation holds.

2.1 Supported platforms

The continuous integration of GAOL v5 builds it and runs its tests on the platforms below. Its jobs, the configurations it refuses or leaves out, and what each platform showed are described in Chapter 14 (see Section 14.4, page 295).

- **Linux.** Ubuntu 24.04 and 26.04 on x86-64 and 64-bit ARM, with GCC and Clang, and the compilers of Ubuntu 22.04 (GCC 11, Clang 14, the latter

on x86-64 only: the three builds refuse it on 64-bit ARM) on Ubuntu 24.04; Debian 12 and 13 on x86-64, 64-bit ARM and 32-bit ARM (armhf), and Debian 12 on 32-bit x86; manylinux_2_28 on x86-64 and 64-bit ARM; Alpine (musl) on x86-64 and 64-bit ARM; Debian 13 on s390x, ppc64le and riscv64, and Ubuntu 24.04 on POWER9, these four under qemu.

- **macOS.** macOS 15 and 26, on arm64 and x86-64, and macOS 27, on arm64, with AppleClang, the Clang of LLVM and GCC.
- **Windows.** Visual Studio 2022 and 2026 on x86, x64 and arm64, with Visual C++ and with clang-cl, and Clang with the target of Visual C++; MinGW-w64 GCC 14 and 15 on x64, 12 to 15 on x86; MSYS2 (UCRT64 and MINGW32 with GCC, CLANG64 and CLANGARM64 with Clang); Cygwin (GCC, x64).

The bounds GAOL computes are the same on all of them. On 64-bit ARM processors, GAOL computes its intervals with NEON instructions, whatever the compiler: GCC, Clang, AppleClang, clang-cl, Visual C++ and the Clang of MinGW-w64 (see Section 4.5, page 96). [GAOL v5]

2.2 Getting the software

GAOL v5 is available at https://github.com/Jordan08/GAOL: `git clone https://github.com/Jordan08/GAOL.git` gives the sources in the directory `GAOL`, the root directory where the commands of this chapter are run. The GAOL of Frédéric Goualard, which GAOL v5 continues, is available at https://github.com/goualard-f/GAOL, and presented in the GAOL section of https://frederic.goualard.net.

GAOL comes as source code, to be compiled for the computer it is used on. The sources of CORE-MATH are part of it (`3rd/math-core`): there is nothing else to download. The archive of the sources that CPack makes from a checkout, `gaol-`*`version`*`.tar.gz`, holds the same sources (see Section 2.5.5, page 42).

2.3 Prerequisites

2.3.1 Mandatory tools

Two tools are needed to build GAOL:

- a C++11 and C99 compiler: GCC, Clang, AppleClang, Visual C++ (Visual Studio 2022 or later), clang-cl (the Clang of Visual Studio, Clang 16 or later), or GCC and Clang for Windows (MinGW-w64, MSYS2, Cygwin);
- CMake 3.14 or later, with the build tool of its generator (`make`, `ninja`, Visual Studio); or, for the autotools build, a shell and `make`; or, for the meson build, meson and ninja.

No mathematical library is needed: CORE-MATH is compiled into GAOL. Neither flex nor bison is needed: the lexer and the parser they generate are committed with the sources (Section 2.3.3).

2.3.2 Compilers and options refused

Each of the compilers and options below gave bounds not enclosing the exact results, or cannot be shown to enclose them. The three builds refuse Clang for 32-bit ARM processors and the compilers of the second item when configuring, the C compiler, which builds CORE-MATH, as well as the C++ compiler, with a message naming what to use instead. They keep the options away by giving GAOL the flags of interval arithmetic (see Section 3.3, page 64). The header `gaol/gaol_config.h` refuses them again when compiling, for the code including the headers of GAOL too, all but the second one, which no macro of the compiler shows; it alone refuses the other compilers of the list, MinGW-w64, Cygwin without the fused multiply-add instructions, Visual C++ and clang-cl without `/fp:strict`, and the x87 unit, and the SSE2 intervals on 32-bit Windows compiled without `-faligned-new=8`, whose programs crashed rather than give wrong bounds.

- **Clang for 32-bit ARM processors.** It does not honour the rounding direction there: built by Clang 21, 4556 of 16000 random products, squares and cubes did not enclose their exact values. GCC does.

- **A compiler saying of `-frounding-math` "overriding currently unsupported rounding mode on this target"**, as Clang 14 for 64-bit ARM. Bounds of `pow()` and `nth_root()` did not enclose the exact values. Clang 18 honours the rounding direction there.

- **A MinGW-w64 whose `fma()` or `round()` is wrong.** On x64, the mingw-w64 before version 12 (GCC 11 to 13 of Chocolatey), and those of any version linked with `msvcrt.dll` rather than the UCRT (MSYS2 MINGW64, the cross compilers of Debian and Ubuntu): the two functions are those of the math library of mingw-w64, computed in doubles. That `fma()` is not correctly rounded (10.7 % of the error-free products `fma(a, b, -a*b)` are wrong), and CORE-MATH computes with it wherever the compiler has no `-mfma` (`GAOL_FMA` off; GCC for Windows on x64 and on 32-bit x86 takes it by default, and these toolchains stay refused, their `round()` being wrong too); that `round()` depends on the rounding direction. Bounds of `tan()`, `asin()`, `atan()` and others did not enclose the exact values. On 32-bit x86, the refusal covers mingw-w64 before 11 (GCC 11 of WinLibs), whose `fma()` rounds each of its partial sums to a double, and, without `-mfma`, mingw-w64 12 and later linked with `msvcrt.dll` (MSYS2 MINGW32). On ARM it covers, as on x64, mingw-w64 before 12 and any linked with `msvcrt.dll`; the refusal is not tested there. A toolchain of mingw-w64 12 or later linking the UCRT (MinGW-Builds

GCC 14 and 15, MSYS2 UCRT64, CLANG64 and CLANGARM64) takes the functions of `ucrtbase.dll`, which pass the tests, and is built and tested.

- **Cygwin without the fused multiply-add instructions** (`GAOL_FMA` off, or a processor without them). The `fma()` of the C library of Cygwin is not correctly rounded (`fma(1 + 2^-30, 1 - 2^-30, -1)` gives 0 instead of -2^{-60}), and `std::fma` is a call of it without optimization. GAOL calls the instruction instead (`-mfma`, `__builtin_fma`), which every level of optimization keeps, and Cygwin is built and tested with it.

- **`-ffast-math`, `-Ofast` and `/fp:fast`.** The compiler then rounds to nearest and drops the checks of NaN and infinities. `-Ofast` is not refused where `-fno-fast-math` turns its fast-math optimizations off (below).

- **`-ffinite-math-only`**, which `-ffast-math` and `-Ofast` turn on. The compiler then takes NaN and infinities never to occur, in the inline functions of GAOL too: the empty interval has NaN bounds, and `([1, 2] & [3, 4]).is_empty()` is false (GCC 9.4 and 13 at `-O0`, `-O2` and `-O3`; Clang 18 at `-O0`, and at `-O2` and `-O3` when the bounds are `volatile`).

- **Visual C++ without `/fp:strict`**, and clang-cl from Clang 16 without it (`/fp:precise`, their default). They then assume rounding to nearest, and may evaluate or rewrite floating-point operations accordingly; clang-cl compiles with `-fno-rounding-math -ffp-contract=on`. No test gave a wrong bound so, but nothing certifies the bounds. [GAOL v5]

- **Doubles computed on the x87 unit of 32-bit x86 processors**, in extended precision: without `-msse2 -mfpmath=sse`, without `/arch:SSE2` with Visual C++, or without `-msse2` with clang-cl, which computes them on the x87 unit with `/arch:SSE2`. CORE-MATH assumes every operation on doubles rounded to a double. Computed in extended precision, its results rounded to nearest are rounded twice, and 175 arguments gave the other neighbour of the exact value with GCC 12 on Debian 12 i386; with GCC 9, which rounds to nearest at compile time the constants CORE-MATH rounds in the direction in effect, the bounds of `exp2(-1075)`, `expm1(-800)` or `atan2()` of a tiny and a huge number did not enclose the exact values.

- **The SSE2 intervals on 32-bit Windows, compiled without `-faligned-new=8`** (GCC and Clang). The C runtime aligns the memory of `new` on 8 bytes, and a `std::vector` of intervals crashed on its first `movaps` (see Section 2.4, page 30); `gaol.pc` and `gaol::gaol` give the flag.

Remark 2.3.1 *Two reasons for refusing MinGW-w64 are gone with CORE-MATH.* [GAOL v5] *The math library of mingw-w64 older than version 12 gave* `acosh()` *near 1 up to 25 million doubles away from the exact value, and the* `fesetround()` *of mingw-w64 12 runs the instruction* `cpuid` *at each call. GAOL*

takes no elementary function from the math library of the system any more, and no longer changes the rounding direction for its elementary functions: the older MinGW-w64 are refused for the `fma()` *and* `round()` *that CORE-MATH and GAOL still take from the C library. Before mingw-w64 12 they are those of mingw-w64 in every program; from 12 on, only in the library of* `msvcrt.dll`*. The header* `gaol/gaol_config.h` *tells the two runtimes apart by* `_UCRT`*, which the headers of a UCRT toolchain define, with GCC and Clang alike. On 32-bit x86, mingw-w64 computes the two functions on the x87 unit: there its* `round()` *is right, and the* `fma()` *of mingw-w64 11 (MinGW-Builds GCC 12 and 13), called only with* `GAOL_FMA` *off, keeps its partial sums in extended precision and passes the tests, though it is not correctly rounded.*

`-fno-fast-math`, one of the flags of interval arithmetic, turns `-ffast-math` and `-ffinite-math-only` off when it comes after them on the command line, and not when it comes before them: `gaol.pc` and `gaol::gaol` give it, and the code including the headers of GAOL is refused when `-ffast-math` or `-ffinite-math-only` comes after `-fno-fast-math`, where the compilation stops. With Clang, `-ffast-math` and `-Ofast` turn `-frounding-math` off too, which `-fno-fast-math` does not turn on again: the remedy is all the flags of interval arithmetic after the option, not `-fno-fast-math` alone. With GCC, `-fno-fast-math` turns off all the fast-math optimizations of `-Ofast` but `-fcx-limited-range`, which changes the arithmetic of complex numbers alone (GCC 9.4), wherever it stands on the command line, before `-Ofast` too: `-frounding-math` stays, and `-Ofast` is not refused there. The CMake tests `refused_finite_math_only` and `refused_fast_math` compile `tests/refused_options.cpp` with each of the two options, after the flags of interval arithmetic, and check that the header refuses it, and `refused_positive` that it compiles without them (GCC and Clang); `tests/refused_options.sh` does the same in the autotools and meson builds.

No macro of the compiler shows the following options, which `gaol/gaol_config.h` cannot refuse and which give wrong results all the same. The code using GAOL is not to be compiled with them.

- **`-funsafe-math-optimizations`, and `-ffast-math -fno-finite-math-only`, with GCC.** The compiler rewrites the addition `1.0 + tiny == 1.0`, by which GAOL sees the rounding direction, as `tiny == 0.0` (GCC 9.4), and the former probe of the modes, `1.0 + (subnormal + 0.0) == 1.0`, which sees the modes flushing the subnormals to zero too, as `subnormal == 0.0` (GCC 9.4 from `-O0`, and GCC 13), so that an operation does not set the direction upward again after the code using GAOL left it to nearest, and `width()` is below the exact width. Clang 18 does not rewrite the first addition, but its `-funsafe-math-optimizations` implies `-fno-signed-zeros`: at `-O2` and `-O3`, the second addition loses its `+ 0.0` there too. The probe of the modes is now `!(1.0 + (subnormal + 0.0) > 1.0)`, which GCC 15 does not rewrite under `-funsafe-math-optimizations`

(GCC 9 to 14 were not tried with it); GCC 15 and Clang 21 drop its `+ 0.0` under `-fno-signed-zeros` as they did for the earlier form.

- **`-fno-signed-zeros`**, with which GCC (9.4 from `-O0`, and 13) and Clang 18 (at `-O2` and `-O3`) drop the `+ 0.0` of the second addition, which then misses the flush-to-zero mode (see Section 4.8, page 104). [GAOL v5]

- **`-fno-honor-nans` alone, with Clang**, which does to the empty interval what `-ffinite-math-only` does: `__FINITE_MATH_ONLY__` is 1 only with `-fno-honor-infinities` too.

- **`-ffast-math` or `-Ofast` followed by `-fno-fast-math`, with Clang.** The macros show nothing, but `-frounding-math` stays off, and the compiler takes the rounding direction to be to nearest. The test `tests/rounding_direction.cpp`, compiled so by Clang 18, failed 54 of its 292728 checks after `-ffast-math -fno-fast-math` (operations with an empty operand raised the invalid-operation flag), and 126 after `-Ofast -fno-fast-math` (bounds with subnormal results under flush-to-zero too); with the flags of interval arithmetic given again after the option, none. With the FPU intervals, the relations in loops are among what fails (`certainly_le()`, `certainly_leq()`, `certainly_ge()`, `certainly_geq()` and the operators `<`, `<=`, `>`, `>=`): at `-O3 -march=native`, Clang 18 and 20 vectorize them into the signaling comparisons `vcmplepd`, `vcmpnlepd` and `vcmpnltpd`, which they make for the empty operands too, whose bounds are NaNs (120 of 184 runs of such loops raised the flag, none with `-frounding-math`; `-fno-fast-math` or `-ffp-contract=off` alone change nothing).

Linking with `-Ofast`, `-ffast-math` or `-funsafe-math-optimizations`. The compilation of the headers of GAOL refuses `-ffast-math`, but the link of the program is not seen. GCC and Clang link `crtfastmath.o` into a program linked with one of these three options, on x86 and on ARM Linux, whose constructor sets the modes that flush the subnormal numbers to zero: flush-to-zero and denormals-are-zero of the SSE instructions, FZ of ARM (see Section 4.8, page 104). Loading a plug-in or a Python module built with `-Ofast` sets them too (built by Clang 18, or by GCC before 13). With one of them, every operation with a subnormal operand or result gives bounds that miss the exact result: $[10^{-300}, 10^{-300}] \times [10^{-20}, 10^{-20}]$ is $[0, 0]$. The `-fno-fast-math` of `gaol.pc` and `gaol::gaol` does not prevent it: it keeps `crtfastmath.o` out of the link when it comes after `-ffast-math`, but not after `-Ofast` (GCC 9.4 and 13 and Clang 18 link it all the same, and `-fno-fast-math` silences the error of `gaol/gaol_config.h` against `-ffast-math`, which `-Ofast` would raise) nor, with GCC 9.4 and 13, after `-funsafe-math-optimizations`. Linking with them, or loading code built with them, makes the bounds wrong whatever the flags the code using GAOL was compiled with. GAOL v5 defends itself in two ways. [GAOL v5]

- **Each operation clears the modes.** On x86 processors, and on ARM processors with GCC and Clang and with the NEON intervals of Visual C++, each operation that computes bounds clears the modes when it starts, in the check that sets the rounding direction upward, which uses a subnormal number to see them as well as the direction (`round_upward_if_needed()` in `gaol/gaol_fpu.h`). It makes the check before it reads, compares or copies a bound, the test of the empty set aside, the compiler being kept from moving a comparison above it (a compiler barrier after the modes are cleared, and an empty asm statement for a double passed by value), and with `GAOL_PRESERVE_ROUNDING` makes its result before it sets the modes back. It is the only defence against the plug-in, and against the compilers without `-mno-daz-ftz`; its cost, and the functions that make no check, are given in Chapter 4 (see Section 4.8, page 104).

- **`-mno-daz-ftz` is given to the link** (`gaol::gaol`, `gaol.pc`, the tests) where the compiler accepts it, which keeps `crtfastmath.o` out of it. Each build checks the compiler with a link, as the other flags are checked: `check_cxx_source_compiles()` with `CMAKE_REQUIRED_LINK_OPTIONS`, `AC_LINK_IFELSE` with `-Werror` and the `LDFLAGS` of the user, `has_link_argument()`. `gaol::gaol` gives it to a program built by GCC 13 or a later one, or by the compiler that built GAOL, of its major version and not older: GCC has the option from 11.4 and 12.4 in the series 11 and 12 only, and a GAOL built by GCC 11.4 gave it to GCC 12.3, which stops on it. `gaol.pc`, written for that compiler, gives it to every program, and a program linked by a compiler that refuses it (Clang 18, a GCC before 11.4, GCC 12.0 to 12.3, GCC for ARM) is to be linked without it.

The modes of other processors, and of ARM with the FPU intervals of Visual C++, are neither checked nor cleared; GCC links `crtfastmath.o` for none of the other processors of the continuous integration (POWER, s390x, RISC-V). The tests `tests/rounding_direction.cpp` and `tests/fast_math_link.cpp` check the two defences (see Section 14.2, page 274).

2.3.3 Tools for maintainers

GAOL uses code produced by GNU Flex and GNU Bison for parsing the expressions used to initialize an interval. The three builds compile the lexer and the parser committed with the sources of `gaol/parser`, generated by flex 2.6.4 and bison 3.8.2 from `gaol/parser/gaol_interval_lexer.lpp` and `gaol/parser/gaol_interval_parser.ypp`:

- `gaol/parser/gaol_interval_lexer.cpp`;
- `gaol/parser/gaol_interval_parser.cpp`;
- `gaol/parser/gaol_interval_parser.h`.

No build generates them again by itself, and none needs flex nor bison. After a change of one of the two files, the target `parser` of the three builds (`make parser`, `ninja parser`) generates the lexer and the parser again in `gaol/parser`, with the flex and the bison of the system, and they are committed; flex 2.6.4 and bison 3.8.2 give them again byte for byte. The script `gaol/parser/regenerate_parser.sh` runs the same commands without a build (the variables `FLEX` and `BISON` give other programs):

```
% sh gaol/parser/regenerate_parser.sh
```

2.3.4 Optional tools

- `flex` and `bison`, to generate the lexer and the parser again (`make parser`);
- bash, and Python 3 with numpy, for `make perf` (see Section 2.9, page 54);
- `gcovr` and Python 3, for the report of the coverage (`GAOL_COVERAGE`, see Section 2.5.2, page 36);
- `pdflatex`, `bibtex` and `makeindex`, to build this manual.

2.4 The three builds

On a given machine with a given compiler, GAOL behaves the same whichever build configured it: CMake, `configure` and meson give it the same macros and the same flags. The flags and the macros come from the autotools build of GAOL 4, then from its meson build, and the CMake build gives the same, apart from the errors corrected (Chapter 15). In every build, by default:

- **GAOL is compiled in release.** The flags are `-O3` and the optimizations `configure` adds (`-funroll-loops`, `-fomit-frame-pointer` and `-fexpensive-optimizations`, each where the compiler takes it), `-falign-functions=64` for the library alone (not for the tests, the examples and the benchmarks), `NDEBUG`, `-std=c++11` (`-std=c++14` for Clang with the target of Visual C++, whose standard library needs it), hidden visibility (`-fvisibility=hidden -fvisibility-inlines-hidden`) and `-Wall -Wconversion`, which GAOL compiles without warnings, `-Wsign-conversion` of Clang included. Its libraries are compiled in release too where a project brings GAOL in without choosing a build type, as by a `configure` calling GAOL's: a CMake project without `CMAKE_BUILD_TYPE`, with a generator of one configuration (with a generator of several, as Visual Studio, Release comes first among the configurations of GAOL built alone, and a project bringing GAOL in chooses its configuration), or a meson project in the default build type of meson, with its ninja backend. [GAOL v5]

- **With the flags of interval arithmetic** (see Section 3.3, page 64). The code using GAOL, the tests included, is linked with `-mno-daz-ftz` where the compiler accepts it (GCC 13 and later on x86, from 11.4 and 12.4 in the series 11 and 12, and Clang 20 and 21, not Clang 18), in `gaol::gaol` and in `gaol.pc`: each build tests the compiler with a link. [GAOL v5]

- **SSE2 intervals on x86 processors** (`-msse2 -msse3`; no flag with Visual C++ and clang-cl for x64, where SSE2 is the baseline of the processor). On 32-bit Windows the C runtime aligns the memory of `new` on 8 bytes, and an SSE2 interval needs 16: GCC took it to be aligned on 16, and a `std::vector` of SSE2 intervals crashed. GCC and Clang (MinGW-w64, MSYS2) are told what the runtime gives with `-faligned-new=8`, so that they ask for aligned memory for the intervals and for what holds one, and `gaol::gaol` and `gaol.pc` give the flag to the code using GAOL too, as it allocates them; `gaol/gaol_config.h` refuses the SSE2 intervals compiled without it. Visual C++ and clang-cl have no such flag (`/Zc:alignedNew` does nothing in C++14 with `cl`, and breaks the STL of clang-cl, and only C++17 turns the aligned `new` on): their SSE2 intervals keep their two bounds as two doubles, whose alignment is 8, and load and store them with `movupd` (`GAOL_SSE2_UNALIGNED_STORAGE` of `gaol/gaol_config.h`, which derives it from the target), so that they need no aligned memory, and the class allocators that aligned the intervals on 16 bytes are not compiled. [GAOL v5]

- **NEON intervals on 64-bit ARM processors** (`GAOL_SIMD`, `--enable-simd`, `-Denable-simd`; no flag), with every compiler of them: GCC, Clang and AppleClang under Linux and macOS, clang-cl and the Clang of MinGW-w64 and MSYS2, and Visual C++ (`cl`, whose registers are those of `_ReadStatusReg()` and `_WriteStatusReg()`). Each build tests the compiler with the same program (CMake and meson for Visual C++, which `configure` does not build with): the intrinsics of `<arm_neon.h>` and the registers of the rounding mode. Not with 32-bit ARM, whose NEON has no vector of doubles: the FPU computes the intervals there, as before (see Section 4.5, page 96). A universal build of macOS (`CMAKE_OSX_ARCHITECTURES` with several architectures, or `-arch` several times in the flags of `configure` and meson) compiles each source once per architecture with one configuration, and takes the intervals of the architecture being compiled, NEON for arm64 and SSE2 for x86_64, where `GAOL_SIMD` allows them (`GAOL_UNIVERSAL_NEON` and `GAOL_UNIVERSAL_SSE2` of `gaol_configuration.h`). `GAOL_SIMD` chooses between the vector intervals and the FPU on every processor: `GAOL_ARM64`, `--enable-arm64` and `-Denable-arm64` were an option of their own for the NEON intervals, independent of `GAOL_SIMD`, and are gone. [GAOL v5]

- **The fused multiply-add instructions of the processor**, for GAOL and CORE-MATH, where the compiler has a flag for them and the machine building runs a program compiled with them (no option of the user; `-DGAOL_`

`FMA=OFF` of the developers leaves them out, see Section 2.5.4, page 40): `-mfma` on x86; with GCC for Windows, on x86-64 and on 32-bit x86, `-mfma -mprefer-vector-width=128` (see Section 2.8.1, page 50); `/arch:AVX2` with Visual C++ for x64, not for 32-bit x86, where it broke the rounding direction; `-mfpu=neon-vfpv4 -mfloat-abi=hard` on 32-bit ARM. The check that the machine runs them matters even when it is not a cross-compilation: under Rosetta on macOS 14, which has no AVX, the x86_64 tests stopped on an illegal instruction. The exact products of GAOL (`fma_once()`) and the functions of CORE-MATH then compute `fma()` with one instruction rather than with a call to the math library: on an Intel i7-1185G7 with Clang 21.1, `pow(x, 3)` of an interval takes 5.4 ns rather than 17.4, `sin` 1.3 times less and `cos` 2.5 times less (see Section 2.8, page 48). The code using GAOL is given the flag too (`gaol::gaol`, `gaol.pc`), and compiled for the same processor. `-ffp-contract=off` stays, which forbids the compiler to contract a multiplication and an addition into a fused one, as CORE-MATH asks. [GAOL v5]

- **The AVX-512 path** of $x + y$, $x - y$, $x \times y$, x/y, `sqrt(x)`, `sqr(x)` and, with FMA, the integer powers beyond 1 and -1, taken where the rounding direction is preserved: the path spares each of these operations the store of that direction (`GAOL_PREFER_AVX512`, a setting of the developers, asks for the path without `GAOL_PRESERVE_ROUNDING` and refuses it with it), and where the machine building runs the instructions (`GAOL_AVX512_ARITHMETIC`), which is checked as for the fused multiply-add instructions, the path being left out when cross-compiling. The library then needs a processor with them, which `gaol::init()` checks, and the operations test nothing as they run (see Section 4.4, page 94). Never with GCC for Windows (MinGW-w64, Cygwin), which does not align the stack for the AVX-512 registers (GCC bug 54412; nor for the 256-bit ones of AVX, which `-mprefer-vector-width=128` keeps MinGW-w64 GCC out of). No flag to compile: the functions of the path alone carry the target attribute, which Visual C++ has not and needs not, taking the intrinsics of any instruction set in any function. Visual C++ has no `__builtin_cpu_supports()`, and the one of clang-cl needs the runtime library of LLVM: both tell the processor by `cpuid` and `xgetbv`. [GAOL v5]

- **Without the intervals of floats**, `gaol::intervalf` and `gaol::interval2f` (SSE3), unfinished, for which no build has an option: only a developer of GAOL compiles them, defining `GAOL_FLOAT_INTERVALS` in `gaol/gaol_config.h`.

- **With exceptions, the rounding direction left upward, and silent**: no line on the standard error when GAOL initializes and cleans up, unless `GAOL_VERBOSE_MODE` (`-DGAOL_VERBOSE_MODE=ON`, `--enable-verbose-mode`, `-Denable-verbose-mode=true`) is asked for; the `configure` of GAOL 4 wrote it by default. [GAOL v5]

- **The processor and the system read from the compiler.** `GAOL_IX86_MACOSX`, `GAOL_ARM_MACOSX`..., the sizes of the integer types and the byte order are read from the macros of the compiler, in `gaol/gaol_config.h`, rather than from the machine building.
- **The same configuration header.** The three write the same macros into `gaol/gaol_configuration.h`, and nothing else (see Section 2.10.1, page 56). [GAOL v5]
- **The same version file.** The three read the version of GAOL from the file `VERSION.txt`, the only one to change for a new version, by the same rules (see Section 2.5.1, page 34). [GAOL v5]
- **The same shared library.** The shared library is `libgaol.so.5.0.0`, whose soname is `libgaol.so.5`, where the build makes one (`configure`, meson, CMake with `BUILD_SHARED_LIBS`); it was `libgaol-5.0.so.0` with `configure` in the earlier versions of GAOL v5, and meson gave no version. [GAOL v5]

In a Debug build (`CMAKE_BUILD_TYPE=Debug`, `configure --enable-debug`, `meson setup --buildtype=debug`), GAOL is compiled with `-g`, without optimization nor `NDEBUG`, so that `assert()` stays active. In GAOL 4, `configure --enable-debug` alone defined `GAOL_DEBUGGING`, which checked the assertions of GAOL (`GAOL_ASSERT`), adding `-g -ansi -Weffc++ -pedantic` for `g++` alone to the optimizations of `--enable-optimize`; in the earlier versions of GAOL v5, the Debug build of CMake did not define it, and meson defined it with its option `enable-debug`. [GAOL v5] `GAOL_DEBUG` of CMake and `enable-debug` of meson build the libraries of GAOL and CORE-MATH so whatever the build type or the configuration, a project bringing GAOL in included, as `configure --enable-debug` does for a `configure` calling GAOL's: the code of that project, and the tests and the examples, keep their own.

The directory `.github/audit` configures the three builds and compares the macros GAOL sees, the macros each writes into `gaol/gaol_configuration.h`, and the flags GAOL is compiled with, those of its C++ sources and those of a C source of CORE-MATH; the continuous integration runs it on Ubuntu 24.04 (x86-64 and arm64, with GCC and Clang), macOS 15 (arm64 and x86-64), Debian 12 on 32-bit x86, Debian 13 on 32-bit ARM (armhf), and MSYS2 UCRT64 and MINGW32 (see Section 14.4, page 295). [GAOL v5]

2.5 Building and installing with CMake

In the root directory of the sources:

```
% cmake -S . -B build -DCMAKE_INSTALL_PREFIX=prefix \
        -DWITH_TESTS=ON
```

```
% cmake --build build --config Release
% cmake --build build --config Release --target test
% cmake --install build --config Release
```

With Visual Studio and Xcode the target is `RUN_TESTS`: `cmake --build build --config Release --target RUN_TESTS`. On Linux with the static library and the SSE2 intervals, the test step runs 47 tests (the other intervals give three fewer, the shared library one fewer), of which `intervalf` and `interval2f` are skipped, the intervals of floats being compiled only by a developer of GAOL. CMake 3.14 or later is needed. The build compiles the thirty-eight sources of CORE-MATH that GAOL uses into `libgaol`, which is a static library unless `BUILD_SHARED_LIBS` is `ON`: there is nothing else to build and nothing else to install. The build type is `Release` unless another is given. Brought in by a project that gives none (`add_subdirectory`, `FetchContent`), GAOL and CORE-MATH are compiled as in `Release` all the same: `-O3` (`/O2` with Visual C++), `NDEBUG` and the optimizations of `configure`; the earlier versions of GAOL v5 had `-O3` alone, and CORE-MATH, compiled apart, no optimization. With a generator of several configurations (Visual Studio, Xcode, Ninja Multi-Config), GAOL built alone puts `Release` first among them, which Ninja Multi-Config builds by default, unless `CMAKE_CONFIGURATION_TYPES` is given; `cmake --build` with Visual Studio builds `Debug` all the same without `--config`. [GAOL v5] The `Debug` build type builds GAOL for debugging (Section 2.4).

2.5.1 The version of GAOL

The file `VERSION.txt` holds the version of GAOL, three numbers without leading zeros such as `5.0.0`, and nothing else: the three builds read it, and it is the only file to change for a new version. [GAOL v5] It is not named `VERSION`: the root of the sources is on the include path of the three builds, and on file systems that ignore case (macOS, Windows), `#include <version>`, which the headers of libc++ write, would find it instead of the standard header.

- **CMake** reads it before `project()`, which gives it to `PROJECT_VERSION`, with `gaol_read_version()` of `cmake/gaol_version.cmake`; a change of `VERSION.txt` configures again. Where GAOL is the project built, it reads the line `PACKAGE_VERSION=` of `configure` too, and warns when it configures that `configure` was generated for another version than `VERSION.txt` holds, since the archive of the sources that CPack makes holds `configure` as it is committed. It says nothing for a tree without `configure`, or with a `configure` that has no such line.

- **meson** reads it in `project()`, with Python (see Section 2.7, page 46), which works with meson 0.53 as with later ones (`version: files()` needs meson 0.57); a change of `VERSION.txt` configures again.

- **`configure`** reads it when it runs, and `make` runs it again when `VERSION.txt` changes (`CONFIG_STATUS_DEPENDENCIES` of `Makefile.am`). Only what `AC_`

`INIT` writes when autoconf generates `configure` (`configure --version`, `configure --help`, `config.status --version` and the head of `config.log`) keeps the version of the last generation: `configure` is committed, and is to be generated again before a new `VERSION.txt` is committed, with the versions of autoconf, automake and libtool of Section 2.6 and `AUTOHEADER=true autoreconf` (without `--force --install`, the other files stay as they are committed; `configure.ac` includes `VERSION.txt` with `m4_include`, so that autoconf sees its change despite its cache, `autom4te.cache`). Until then, `configure` configures GAOL with the version of `VERSION.txt` and warns, twice, that it was generated for another one, and the continuous integration fails. With `--enable-maintainer-mode`, `make` generates `configure` again itself, and runs no autoheader over `gaol/gaol_configuration.h.in`, which is written by hand.

From `VERSION.txt` come the macros `GAOL_MAJOR_VERSION`, `GAOL_MINOR_VERSION`, `GAOL_MICRO_VERSION` and `GAOL_VERSION` of `gaol/gaol_configuration.h` (`gaol_core::version_major...` of `gaol/gaol_version.h`, see Section 13.3, page 264), the shared library `libgaol.so.`*`major.minor.micro`* and its soname `libgaol.so.`*`major`*, the `Version` of `gaol.pc`, `gaolConfigVersion.cmake` (`find_package(gaol` *`version`*`)` takes a version of the same major number, not older than the one asked for), the names of the archive and of the packages of CPack, and the version printed by the manual (`configure`, meson, and CMake with `WITH_DOC`). Each build refuses a `VERSION.txt` that does not hold three numbers without leading zeros, which the macros write as C integers, and the three read it by the same rules, as autoconf does for `configure --version`. The blanks and empty lines around the version are ignored, the blanks being the six of ASCII (space, tab, line feed, vertical tab, form feed and carriage return, the CR of the line ends of Windows), not those of Unicode such as a no-break space; so is a UTF-8 byte order mark (the bytes EF BB BF) at the start of the file, which some editors of Windows write. The message of a build that refuses the file quotes what it read and gives the first bytes of the file in hexadecimal, which show what the quotation may hide: CMake and `configure` quote a second byte order mark, or a zero-width space, as it is, and it cannot be seen (meson writes it `\ufeff`, `\u200b`). A file of UTF-16 characters, which Windows PowerShell 5 writes for a redirection (`"5.0.0" > VERSION.txt`), is not decoded but refused, whether it starts with its byte order mark (FF FE, FE FF) or not (it then holds NUL bytes, which a text file does not hold): the message gives its bytes and ends with *the file is UTF-16: save it as UTF-8 or ASCII*, or, without the mark, *the file holds a NUL byte as UTF-16 does: save it as UTF-8 or ASCII*. The test `version_file` (`ctest -R version_file`) checks the reading of CMake, and a script of the continuous integration those of `configure`, meson and autoconf (see Section 14.5, page 305). The edition of this manual is its own, set in `configure.ac`, `manual/meson.build` and `CMakeLists.txt`.

2.5.2 Options

The options are given to the first command, as `-Doption=value`. The three builds have the same options. [GAOL v5] Each had options of its own before, and those of the fused multiply-add instructions, of the AVX-512 path, of the 128-bit integer, of the debugging and of the NEON intervals are gone as options: the configuration decides them (Section 2.5.3). The standard options of each tool do the rest. Table 2.1 gives the options under their three names, with their defaults.

TABLE 2.1 – The options of the three builds

CMake	`configure`	meson	Default
`CMAKE_BUILD_TYPE`	`--enable-debug`	`--buildtype=debug`	`Release`
`CMAKE_INSTALL_PREFIX`	`--prefix`	`--prefix`	the system's
`BUILD_SHARED_LIBS`	`--enable-shared`	`-Ddefault_library=shared`	off
`CMAKE_POSITION_INDEPENDENT_CODE`	`--enable-pic` (`--with-pic` too)	`-Db_staticpic=true`	on
`CMAKE_OSX_ARCHITECTURES`	`-arch` several times in `CFLAGS`, `CXXFLAGS` and `LDFLAGS`	`-arch` several times in `c_args`, `cpp_args`, `c_link_args` and `cpp_link_args`	the machine's
`WITH_TESTS`	`--with-tests`	`-Dwith-tests=true`	off
`WITH_EXAMPLES`	`--with-examples`	`-Dwith-examples=true`	off
`WITH_DOC`	`make -C manual pdf` (no option)	`-Dwith-doc=true`	off
`GAOL_SIMD`	`--enable-simd`	`-Denable-simd=true`	on
`GAOL_PRESERVE_ROUNDING`	`--enable-preserve-rounding`	`-Denable-preserve-rounding=true`	off
`GAOL_COVERAGE`	`--enable-coverage`	`-Denable-coverage=true`	off

- **`CMAKE_BUILD_TYPE`.** `Release` by default; `Debug` builds GAOL without optimization, with debugging information and without `NDEBUG`, so that `assert()` stays active, and enables the debugging facilities (see Section 13.6, page 269). The `Release` build is `-O3`, `NDEBUG` and the optimizations of Section 2.4. With `configure`, `--enable-debug` brought into another project with `AC_CONFIG_SUBDIRS` is the `--enable-debug` of the `configure` of that project, which passes on its options, for the tests of GAOL as well.

- **`CMAKE_INSTALL_PREFIX`.** Where `cmake --install` puts GAOL.

- **`BUILD_SHARED_LIBS`.** `OFF` by default: a static library `libgaol.a`; `ON` builds the shared library `libgaol.so.5.0.0`, whose soname is `libgaol.so.5`, as `configure` and meson do (always a static library with Visual C++). Installed in a *prefix* that the dynamic loader does not search, the shared library is found on Linux by a program run with `LD_LIBRARY_PATH` naming the library directory, *`prefix`*`/lib` (or `lib64` or `lib/`*`multiarch`* on the systems whose libraries go there), or linked with `-Wl,-rpath,` and that directory. With

`configure` the static library is built too unless `--disable-static`, as with `-Ddefault_library=both` of meson; `configure` and meson built both by default in GAOL 4. [GAOL v5]

- **`CMAKE_POSITION_INDEPENDENT_CODE`.** `ON` by default: GAOL is compiled as position-independent code (`-fPIC`), so that it can be linked into a shared library, as Python bindings; a project building GAOL with `FetchContent` that sets it is followed. The `configure` of GAOL 4 compiled the static library without it. [GAOL v5]

- **`CMAKE_OSX_ARCHITECTURES`.** A universal build of macOS, one configuration compiled for each architecture: SSE2 intervals for x86_64 and NEON intervals for arm64, where `GAOL_SIMD` allows them, and no fused multiply-add instructions nor AVX-512 path. CMake only is checked on a Mac of ours; `configure` and meson are checked by the continuous integration alone. The generator of Xcode, which builds arm64 and x86_64 by itself when no list is given, is given the architecture of the machine, as the other generators build for it; the list `arm64;x86_64` asks for the universal build. [GAOL v5]

- **`WITH_TESTS`.** `OFF` by default: builds the unit tests of `tests`, which `make test` and `make check` run (see Section 2.9, page 54).

- **`WITH_EXAMPLES`.** `OFF` by default: builds the examples of `examples`, which `make check` runs.

- **`WITH_DOC`.** `OFF` by default: adds the target `pdf`, which builds this manual in `manual/Manual_GAOLv5.pdf` with pdflatex, bibtex and makeindex (Table 2.8). [GAOL v5]

- **`GAOL_SIMD`.** `ON` by default: the intervals are computed with the vector instructions of the processor as soon as it has them, their two bounds in one register: SSE2 instructions on x86 processors (`-msse2 -msse3`; no flag with Visual C++ and clang-cl for x64; on 32-bit Windows with GCC and Clang and `-faligned-new=8`, which the code using GAOL is given too, and with Visual C++ and clang-cl, whose intervals keep their two bounds as two doubles and need no aligned memory), and NEON instructions on 64-bit ARM processors, with every compiler and no flag (see Section 4.5, page 96); the FPU computes them otherwise, and on any other processor (32-bit ARM, POWER, s390x, RISC-V...). A universal build of macOS takes the NEON intervals for its arm64 part and the SSE2 intervals for its x86_64 part. `GAOL_ARM64`, `--enable-arm64` and `enable-arm64` were an option of the NEON intervals, independent of `GAOL_SIMD`, and are gone. [GAOL v5]

- **`GAOL_PRESERVE_ROUNDING`.** `OFF` by default: each operation restores the rounding direction it found, rather than leaving it upward (see Section 4.3, page 93).

- **`GAOL_COVERAGE`.** `OFF` by default: compiles GAOL and its tests with the counters of gcov, and adds the target `coverage`, which runs the tests and writes the report of their coverage, `coverage/coverage.html`, and a page of conclusions (`cmake --build build --target coverage`, `make coverage` of `configure`, `ninja -C build gaol-coverage` of meson, which keeps `coverage` for its own report). GCC and Clang, with the `Debug` build, `gcovr` and Python 3; the tests have to be built.

2.5.3 What the configuration decides

The rest of the configuration has no option. It is decided by checks of the compiler and of the machine, the same with the three builds, each told in the messages of `cmake`, `configure` and `meson setup`. They assume that the compiler, the architecture and the system that configure and compile GAOL are those of the code using it: a program compiled and run by the configuration tells what the machine does, and the headers and `gaol.pc` carry the flags of the code using GAOL. [GAOL v5]

- **The fused multiply-add instructions** (was `GAOL_FMA`, `--enable-fma`, `enable-fma`). GAOL and CORE-MATH are compiled with them whenever the C++ and the C compiler take their flag and the machine building runs a program compiled with them: a program that runs the instruction itself, `__builtin_fma()`, and the intrinsic `_mm_fmadd_sd()` with Visual C++ and clang-cl for x64, which a processor without FMA3 stops on. The flags are `-mfma` on x86 (`-mfma -mprefer-vector-width=128` with GCC for Windows, on x86-64 and on 32-bit x86), `/arch:AVX2` with Visual C++ for x64, `-mfpu=neon-vfpv4 -mfloat-abi=hard` on 32-bit ARM; 64-bit ARM, POWER, s390x and RISC-V have the instructions without a flag. The library then needs a processor with them (on x86, Intel Haswell and AMD Piledriver, 2012–2013, and later; with Visual C++ and clang-cl, whose `/arch:AVX2` also needs AVX2, Intel Haswell and AMD Excavator, 2013–2015, and later), and the code using GAOL is given the flag too, in `gaol::gaol` and `gaol.pc`; `-ffp-contract=off` stays. The elementary functions are much faster with them: `cos` 2.5 times, `log` 1.6 times, `sin` 1.3 times (Intel i7-1185G7, Clang 21.1), and with GCC 16.2 for Windows `cos` 4.7 times, `log` 3.2 times and `sin` 2.0 times (GitHub runner, MSYS2 UCRT64). GCC for Windows does not align its stack on 32 bytes for the 256-bit registers that `-mfma` alone would have it use (GCC bug 54412): `-mprefer-vector-width=128` keeps it to the 128-bit ones, and the objects of GAOL built so by GCC 16.2 have no instruction on a 256-bit register (see Section 2.8.1, page 50). In a cross-compilation, where no program can be run, they are left out, with a message. A universal build of macOS leaves the flag out, its arm64 part refusing `-mfma` while it has the instructions without a flag: the x86_64 part is then built without them, runs on any x86_64 processor, Rosetta 2 included, but CORE-MATH and the exact products of GAOL call the `fma()` of the

system library, and the integer powers beyond 1 and −1 do not take the packed kernel that uses the instruction; a build for x86_64 alone (`CMAKE_OSX_ARCHITECTURES=x86_64`) takes `-mfma` where the machine building runs a program compiled with it.

- **The AVX-512 path** (was `GAOL_PREFER_AVX512`, `--enable-prefer-avx512`, `enable-prefer-avx512`). The four arithmetic operations, the square, the square root and, with FMA, the integer powers beyond 1 and −1 take the AVX-512 instructions, which carry their rounding direction and set none, where `GAOL_PRESERVE_ROUNDING` is on, the compiler takes the instructions (a program that uses them, linked) and the machine building runs them; otherwise the SSE2 intervals compute them. The rule comes from measurements (see Section 4.4, page 94): the path spares the store of the rounding direction that every operation pays with `GAOL_PRESERVE_ROUNDING`, where it gains 2.9 times at the median of seven operations, on an Intel i7-1185G7 (no other processor was measured); without that option the operations take 20 % longer at the median, the direction being written only when it is not upward already, so the path is left out. The library then needs a processor with the instructions, as with the fused multiply-add instructions, and `gaol::init()` stops a program run on another one with a message (see Section 4.4, page 94). In a cross-compilation, the path is left out, with a message.

- **The 128-bit integer** of the accurate phases of CORE-MATH (was `GAOL_U128_EMULATION`): the type of the compiler where it has one, else two 64-bit halves (Visual C++, and GCC for a 32-bit target), which `gaol/math/gaol_u128.h` takes by itself and the configuration says.

- **Clang with the target of Visual C++** (`clang++` for `x86_64-pc-windows-msvc` or `i686-pc-windows-msvc`, with the driver of GCC, not clang-cl). It defines `_MSC_VER`, which the three builds read (`CMAKE_CXX_SIMULATE_ID` of CMake) to compile GAOL in C++14, the standard library of Visual C++ needing it, with the `unistd.h` of `cmake/msvc` for the lexer and without the warnings of the C runtime, and, with CMake and meson, to build the static library alone, as with Visual C++ (a DLL would export nothing); `configure` builds the static library by default. [GAOL v5]

- **The debugging** (was `GAOL_DEBUG`, `GAOL_DEBUGGING`). GAOL and CORE-MATH are built for debugging, without optimization nor `NDEBUG`, with debugging information and `GAOL_DEBUGGING` (see Section 13.6, page 269), for the `Debug` build type (`CMAKE_BUILD_TYPE`, `--enable-debug`, `--buildtype=debug`); there is no target `perf` in the `Debug` build of CMake, nor with `-Denable-debug=true` of meson (`configure --enable-debug` and `--buildtype=debug` of meson keep it). Each configuration keeps its other flags (the C runtime of Visual C++, sanitizers...); the tests and the examples, and the code of a project bringing GAOL in, keep their build type.

- **The memory and the flags of the intervals**: the aligned memory the SSE2 intervals need on 32-bit Windows, the flags of interval arithmetic, the link option `-mno-daz-ftz`, `<fenv.h>`, the visibility, the type of the intervals of a universal build.

2.5.4 The settings of the developers

The checks above decide by themselves. A developer who wants to run a path the machine would not take, or a user who builds a package for any processor, gives one of the settings of Table 2.2. They are not options of the user: `configure --help` lists none of them but `--enable-debug`, which is also the `Debug` build of `configure`, and `meson configure` lists them with the mark *developers*, since meson cannot leave them out. CMake reads them where they apply, without having them in the cache unless they were given. By default, the three builds report errors by exceptions (see Section 13.5.1, page 266); one setting of the table makes GAOL abort instead.

TABLE 2.2 – The settings of the developers

CMake	`configure`	meson	Meaning
`-DGAOL_FMA=OFF`	`--disable-fma`	`-Denable-fma=false`	No fused multiply-add instructions: a library for any processor of the architecture
`-DGAOL_PREFER_AVX512=ON`, `=OFF`	`--enable-prefer-avx512`, `--disable-prefer-avx512`	`-Denable-prefer-avx512=true`, `=false`	The AVX-512 path asked for without `GAOL_PRESERVE_ROUNDING`, or refused with it (a package for any processor); asked for where the compiler cannot take it, an error
`-DGAOL_U128_EMULATION=ON`	`CPPFLAGS=-DGAOL_U128_FORCE_EMULATION=1`	`-Dc_args=...` and `-Dcpp_args=...` with the same macro	The two 64-bit halves where the compiler has a 128-bit type: the code of Visual C++ and of the 32-bit targets on an ordinary machine
`-DGAOL_DEBUG=ON`	`--enable-debug`	`-Denable-debug=true`	GAOL and CORE-MATH built for debugging whatever the build type, a project bringing GAOL in included (`-Dgaol:enable-debug=true` for a subproject of meson); the tests and the examples keep the build type

Table 2.2, continued

CMake	`configure`	meson	Meaning
`-DGAOL_EXCEPTIONS_ENABLED=OFF`	`--disable-exceptions`	`-Denable-exception=false`	GAOL aborts to signal an error instead of raising an exception
`-DGAOL_VERBOSE_MODE=ON`	`--enable-verbose-mode`	`-Denable-verbose-mode=true`	A line on the standard error when GAOL initializes and cleans up; GAOL is silent by default
`-DGAOL_SSE2_UNALIGNED_STORAGE=ON`	`CXXFLAGS=-DGAOL_SSE2_UNALIGNED_STORAGE=1`	`-Dcpp_args=...` with the same macro	The SSE2 intervals with their two bounds as two doubles, as on 32-bit Windows with Visual C++ and clang-cl: that layout on another system. CMake alone writes the macro into the installed `gaol/gaol_configuration.h`: given to `configure` or meson by the flags, it reaches what the build compiles, not a program compiled later with the installed GAOL, which then has to be compiled with the same flag, or its layout differs from the library's; with CMake, a program must not define the macro

Table 2.2, continued

CMake	`configure`	meson	Meaning
`-DCMAKE_CXX_FLAGS= -DGAOL_RELATION_ BOUNDS_FIRST=0`	`CXXFLAGS=-DGAOL_ RELATION_BOUNDS_ FIRST=0`	`-Dcpp_args=...` with the same macro	The relations between intervals (`<`, `<=`, `>`, `>=`) test the empty sets before they compare the bounds, as they do on RISC-V; `=1`, the default elsewhere, compares the bounds first. The answers are the same; the order of the tests changes, which a processor may run faster or slower, and which decided whether GCC 14 raised the invalid-operation flag for an empty operand on RISC-V (the reason for its default there). It is to be defined the same in every file that includes GAOL (`gaol/gaol_port.h`)
`-DGAOL_MACHINE_ RUNS_FMA=1`, `-DGAOL_ MACHINE_RUNS_ AVX512=1`	`gaol_cv_machine_ runs_fma=yes`, `gaol_cv_machine_ runs_avx512=yes`	`-Dmachine-runs- fma=true`, `-Dmachine-runs- avx512=true`	In a cross-compilation, or under an emulator, where no program can be run: the machine running GAOL has the fused multiply-add instructions or runs the AVX-512 ones

Remark 2.5.1 *An option that GAOL had and no longer has is refused or warned of: meson refuses* `-Denable-arm64`*, CMake and* `configure` *warn that* `GAOL_ ARM64=OFF` *and* `--disable-arm64` *are no longer read, and say* `GAOL_SIMD=OFF` *and* `--disable-simd`*. A build directory of CMake configured by an earlier GAOL v5 holds the entries of the options that are gone (*`GAOL_FMA`*,* `GAOL_DEBUG`*...), which stay harmless, and* `GAOL_PREFER_AVX512`*, which the older CMake kept with the value it had derived: the record* `GAOL_PREFER_AVX512_DERIVED` *tells it, and both entries go.*

2.5.5 The archive of the sources and the packages

CPack makes them from the CMake build, in the directory of the build, where `make dist` of the autotools build made the archive of the sources:

```
% cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/usr \
        -DGAOL_FMA=OFF -DGAOL_PREFER_AVX512=OFF
```

```
% cmake --build build --config Release
% cmake --build build --target package_source
% cmake --build build --target package
```

- **package_source** gives gaol-*version*.tar.gz: the sources, in gaol-*version*/, with configure and the Makefile.in, which the three builds build. CPack takes the source tree as it is, less the files of CPACK_SOURCE_IGNORE_FILES (the repository of git, the builds made within it, the objects) and the notes of the work on GAOL v5: made from a clean checkout, the archive holds the other files of the commit, configure as it was generated among them. If that was for another version than VERSION.txt holds, configure --version in the archive gives the old one, and CMake warns of it when it configures (see Section 2.5.1, page 34). The archive takes its name from the version CMake read when it configured. After a change of VERSION.txt, package_source configures the build directory again first, whatever the generator: Ninja does before it runs CPack, and CPack does when VERSION.txt holds another version than the build directory was configured for (cmake/gaol_package_source.cmake), where the Makefile generators ran CPack at once. [GAOL v5]

- **package** gives gaol-*version*-*system*.tar.gz, what cmake --install installs, in gaol-*version*-*system*/: the headers, libgaol.a (or libgaol.so* with BUILD_SHARED_LIBS), the CMake package of GAOL and gaol.pc, to extract anywhere, gaol.pc finding its directories from where it is. On Linux it gives libgaol-dev_*version*_*arch*.deb too, the same files in /usr, for apt install ./libgaol-dev_*version*_*arch*.deb: by default a static library and its headers, hence the name and the section libdevel of the development packages of Debian. -DCMAKE_INSTALL_PREFIX=/usr gives the library directory of Debian and Ubuntu (lib/x86_64-linux-gnu); with another prefix, the .deb puts the files in /usr/lib.

The packages hold libgaol as the build compiled it: where the machine building has the fused multiply-add instructions, only the processors that have them run it, and the code using it is compiled with -mfma; -DGAOL_FMA=OFF makes packages for any processor of the architecture that has SSE3 on x86 (-msse3 is given wherever the compiler takes it). So does -DGAOL_PREFER_AVX512=OFF where GAOL_PRESERVE_ROUNDING is ON, whose AVX-512 path makes a library that only the processors with the AVX-512 instructions run, when the machine building has them (Visual C++ and clang-cl for x64 too). [GAOL v5]

Remark 2.5.2 *On Cygwin,* package_source *did not work in the job of the continuous integration of 23 October 2026 (CMake 4.4.4, libarchive 3.8): CPack, with TGZ, TBZ2 and ZIP alike, failed to add* README*, a symbolic link to* README.md *in the repository, with "Couldn't list extended attributes", so that the target fails there and the test* cpack_stale_configure *says it is skipped. The cause was not found:*

`cmake -E tar`, *GNU* `tar` *and* `git archive` *make an archive of the checked-out tree, and* `listxattr()`, `llistxattr()` *and* `flistxattr()` *find nothing wrong with its link.* `git archive` *makes the archive from the repository.*

2.6 Building and installing with autotools

In the root directory of the sources (for a build outside the sources, `configure` is called by its path from another directory):

```
% ./configure --prefix=prefix --with-tests
% make
% make test
% make install
```

`configure` compiles the thirty-eight sources of CORE-MATH into `libgaol` (`3rd/Makefile.am`, whose convenience library `gaol/Makefile.am` puts into `libgaol`), with the flags it gives to the C code of GAOL: there is nothing else to build, nothing else to install and nothing else to link. There is no `make dist` nor `make distcheck`: the archive of the sources is made with CMake (see Section 2.5.5, page 42).

`configure`, `aclocal.m4`, the `Makefile.in`, `ltmain.sh`, `m4/*.m4` and the scripts `compile`, `config.guess`, `config.sub`, `depcomp`, `install-sh`, `missing` and `test-driver` are committed, so that a checkout builds without the autotools, and `make` never regenerates them (`AM_MAINTAINER_MODE([disable])`). They are generated by autoconf 2.72, automake 1.18.1 and libtool 2.5.4, with `AUTOHEADER=true autoreconf --force --install` followed by `git checkout INSTALL`, which `--install` replaces by the text of automake; `AUTOHEADER=true` keeps `gaol/gaol_configuration.h.in`, written by hand, which autoheader 2.72 overwrites (2.73 leaves it). `config.guess` and `config.sub` are those of 2026-05-17 from GNU config (`https://git.savannah.gnu.org/cgit/config.git`), newer than the ones libtool installs. The versions matter: the files of GAOL 4.2.2 came from libtool 2.4.2, which linked `libgaol` with `-flat_namespace -undefined suppress` on macOS 11 and later, taking them for Mac OS X 10.0, and from a `config.sub` of 2016, which refused `loongarch64`. [GAOL v5] The file `INSTALL` gives the four commands of each of the three builds, and points to `doc/building.md` and `doc/using.md` for the options and the flags.

`configure` accepts the following options, which are those of the three builds (Table 2.1):

- **`--help`.** Displays a list of all options; only those described below are options of the user, with the options of libtool and automake; the settings of the developers are not listed, but `--enable-debug`, which is also the `Debug` build.

- **`--prefix=`*`prefix-dir`*`.`** The root directory where the library is installed. It defaults to `/usr/local`.
- **`--libdir=`*`lib-dir`*`.`** The directory where to put the libraries. It defaults to *`prefix-dir`*`/lib`.
- **`--includedir=`*`include-dir`*`.`** The directory where to put the header files. It defaults to *`prefix-dir`*`/include`.
- **`--enable-debug[=`*`yes/no`*`].`** Builds GAOL as the `Debug` build of CMake: with `-g`, without optimization nor `NDEBUG`, and with the debugging facilities (see Section 13.6, page 269); the `Release` build of CMake otherwise, with `-O3` and `NDEBUG`, and the flags `-funroll-loops`, `-fomit-frame-pointer` and `-fexpensive-optimizations` where the compiler takes them, and `-falign-functions=64` for the library alone. This option defaults to `no`.
- **`--enable-simd[=`*`yes/no`*`].`** The vector intervals, SSE2 on x86 processors and NEON on 64-bit ARM processors, as `GAOL_SIMD` (see Section 2.5.2, page 36). This option defaults to `yes`. [GAOL v5]
- **`--enable-preserve-rounding[=`*`yes/no`*`].`** Each operation restores the rounding direction it found (see Section 4.1, page 88). This option defaults to `no`; [GAOL v5] the `configure` of GAOL 4 had it on by default, although its manual says the contrary.
- **`--enable-coverage[=`*`yes/no`*`].`** Compiles GAOL and its tests with the counters of gcov, and adds the target `make coverage`, as `GAOL_COVERAGE`. This option defaults to `no`. [GAOL v5]
- **`--enable-shared[=`*`yes/no`*`], --enable-static[=`*`yes/no`*`].`** The static library `libgaol.a`, which is all that is built by default, as with CMake; `--enable-shared` builds the shared library `libgaol.so.5.0.0` too, whose soname is `libgaol.so.5`, as CMake (`BUILD_SHARED_LIBS`) and meson name it (it was `libgaol-5.0.so.0` in the earlier versions of GAOL v5, and it was built by default before them); `--enable-pic` (the default, though `configure --help` says *use both*) compiles the code as position-independent code. [GAOL v5]
- **`--with-tests[=`*`yes/no`*`].`** Builds the unit tests of `tests`, which `make test` and `make check` run, as `WITH_TESTS`. This option defaults to `no`.
- **`--with-examples[=`*`yes/no`*`].`** Builds the examples of `examples`, which `make check` runs, as `WITH_EXAMPLES`. This option defaults to `no`.

As usual, `--disable-`*`xxx`* is equivalent to `--enable-`*`xxx`*`=no`, and `--enable-`*`xxx`* to `--enable-`*`xxx`*`=yes`. The fused multiply-add instructions, the AVX-512 path, the 128-bit integer and the NEON intervals are decided by checks of the compiler and of the machine (Section 2.5.3). `--enable-optimize` (the `Release` build,

which is the default, and `-O` with `--disable-optimize`) and `--enable-arm64` (the NEON intervals follow `--enable-simd`) are no longer options; the `--enable-verbose-mode` and `--enable-exceptions` of GAOL 4 are settings of the developers, as `--enable-fma` and `--enable-prefer-avx512`, which `configure --help` does not list (Section 2.5.4); the other options of GAOL 4, `--with-mathlib`, `--with-mathlib-include`, `--with-mathlib-lib`, `--with-cppunit-include`, `--with-cppunit-lib`, `--enable-fast-math`, `--enable-asm` and `--enable-relations`, are gone, and `configure` warns that it does not know them. [GAOL v5]

A universal build of macOS is configured with `-arch` several times in the flags of the compilers: `./configure CFLAGS="-arch arm64 -arch x86_64" CXXFLAGS="-arch arm64 -arch x86_64" LDFLAGS="-arch arm64 -arch x86_64"`; the dependency tracking of automake, which the compilers do not take with several `-arch`, is turned off then. [GAOL v5]

The targets of the `Makefile` in the root directory are `all`, which builds `libgaol.a` (and the shared library `libgaol.so.5.0.0` with `--enable-shared`), `test`, `check`, `perf` and `parser`, those of the three builds (see Section 2.9, page 54), `install`, `clean`, `distclean` and, with `--enable-coverage`, `coverage`. `clean` erases what `make` built, and keeps what `configure` made, so that `make` builds again without it: the `Makefile`, `config.status`, `config.log`, `libtool`, `gaol.pc`, `gaol/gaol_configuration.h`, the `manual/v*/gaol_version.tex`, the `.deps` directories of the dependencies and the `.dirstamp` of the directories of CORE-MATH. `distclean` erases them too, the `.deps` directories included, which it left empty in GAOL 4, and gives the source tree back as git has it, `configure` to be run again; the continuous integration checks it. [GAOL v5] `make -C manual pdf` builds this manual; no build makes the manual of GAOL 4 again. `configure --help` ends with the address for the bug reports, `jordan.ninin@ensta.fr`, and the page of GAOL v5, `https://github.com/Jordan08/GAOL`.

2.7 Building and installing with meson

```
% meson setup build --prefix=prefix -Dwith-tests=true
% meson compile -C build
% meson test -C build
% meson install -C build
```

meson compiles the thirty-eight sources of CORE-MATH into `libgaol` (`gaol/meson.build`), as CMake and `configure` do, and the lexer and the parser committed with the sources: neither flex nor bison is needed. `meson compile` needs meson 0.54; with an older one, as the meson 0.53 of Ubuntu 20.04, `ninja -C build` builds GAOL as well. It builds `libgaol.a`, and `libgaol.so.5.0.0` with `-Ddefault_library=shared` (`both` for the two), whose soname is `libgaol.so.5`, as `configure` with `--enable-shared` and CMake name it; with Visual C++, the

static library alone, as CMake, and `gaol_dep` and `gaol.pc` no longer give the code using GAOL `__GAOL_PUBLIC__` defined empty (see Section 3.3, page 64). [GAOL v5] The options are given to `meson setup`, as `-Doption=value`; the build type gives the optimizations and the debugging facilities, as with CMake (`--buildtype=debug` for the `Debug` build), and the others are those of `configure` (Table 2.3).

TABLE 2.3 – The options of the meson build

Option	Default
`buildtype`	`release`: `-O3`, `NDEBUG` and the optimizations of the `Release` build of CMake and `configure`; `debug` for the `Debug` build, without optimization, with debugging information and without `NDEBUG`. The option `enable-optimize` is gone. Brought in as a subproject, GAOL is built in `release` too where the project leaves the build type to its default
`enable-simd`	`true`: the vector intervals, SSE2 or NEON, as `GAOL_SIMD`
`enable-preserve-rounding`	`false`, as `GAOL_PRESERVE_ROUNDING`
`enable-coverage`	`false`; the target `ninja gaol-coverage`, as `GAOL_COVERAGE`
`default_library`	`static`; `shared` builds the shared library, `both` the two, as `BUILD_SHARED_LIBS` (it was `both`)
`b_staticpic`	`true`: position-independent code, as `CMAKE_POSITION_INDEPENDENT_CODE`
`with-tests`	`false`; builds the unit tests, which `meson test` and `ninja check` run (it was `with-test`)
`with-examples`	`false`; builds the examples, which `ninja check` runs
`with-doc`	`false`; the target `pdf` then builds this manual, `manual/Manual_GAOLv5.pdf`, with pdflatex, bibtex and makeindex, as `WITH_DOC`

These are the options of the three builds. `enable-fma`, `enable-prefer-avx512`, `enable-debug`, `enable-exception`, `enable-verbose-mode`, `machine-runs-fma` and `machine-runs-avx512` are settings of the developers, which `meson configure` lists with the mark *developers* (Section 2.5.4). The options `enable-arm64` (the NEON intervals follow `enable-simd`), `enable-optimize`, `enable-relations`, `with-test` and `check-perf` are gone, and meson refuses them. [GAOL v5] A universal build of macOS is configured with `-arch` several times in `c_args`, `cpp_args`, `c_link_args` and `cpp_link_args`. The summary of `meson setup` gives the address for the bug reports, `jordan.ninin@ensta.fr`, and the page of GAOL v5, `https://github.com/Jordan08/GAOL`.

`meson setup` runs Python once, in `project()`, to read the version of GAOL in `VERSION.txt` (see Section 2.5.1, page 34), and twice more for the message of a file it

refuses: the first of `python3` and `python` that it finds in the `PATH` or, when it finds neither, the Python that runs meson (the case of the `meson.exe` of the Windows installer when no Python is installed). `meson.build` names `python3` first because meson falls back on its own Python for that name alone; the continuous integration checks it on Linux with a `meson setup` whose `PATH` holds no Python.

Remark 2.7.1 *On Windows, none of what follows was checked, but read in the sources of meson. The directory* `WindowsApps` *of the profile of the user (*`%USERPROFILE%\AppData\Local\Microsoft\WindowsApps`*) can hold aliases of Python,* `python3.exe` *and perhaps* `python.exe` *too, which only open the Microsoft Store when Python was not installed from it. meson 0.53.1 and later (the 0.53.2 of Ubuntu 20.04 and the meson of pip among them) leave that directory out of their search for programs, as long as the* `PATH` *names it by that path, not for a profile whose directory differs from* `USERPROFILE`*. meson 0.53.0 and earlier may take such an alias, and* `meson setup` *then stops on the failure of the command that reads* `VERSION.txt`*: the remedy is a later meson (*`pip install meson`*), or to turn off the aliases of Python in the Windows settings (*Manage app execution aliases*).*

2.8 What the options are worth

The options of Table 2.1 that change the code of the operations are the vector intervals (`GAOL_SIMD`) and the preserved rounding direction (`GAOL_PRESERVE_ROUNDING`); the fused multiply-add instructions, which the configuration decides, change it too. Table 2.4 measures the vector intervals and the fused multiply-add instructions with `tests/performance.cpp` on an Intel i7-1185G7 with Clang 21.1, in nanoseconds per operation, the best of nine runs, each program being run in turn with the others.

TABLE 2.4 – The options on x86: nanoseconds per operation, Intel i7-1185G7, Clang 21.1, the best of nine runs

Build	$x+y$	$x\times y$	`sqr`	`pow(x, 3)`	`exp`	`log`	`sin`	`cos`
Default (FMA and `GAOL_SIMD` on)	1.07	2.04	1.80	5.42	18.6	21.7	31.3	36.6
`-DGAOL_FMA=OFF`	1.14	2.06	1.81	17.4	21.6	34.5	41.4	90.3
`-DGAOL_SIMD=OFF`	2.30	4.32	2.46	7.65	19.9	23.6	32.8	39.0

- **The fused multiply-add instructions pay on the elementary functions.** `cos` is 2.5 times faster with them, `pow(x, 3)` 3.2 times, `log` 1.6 times, `sin` 1.3 times and `exp` 1.2 times. CORE-MATH computes with `__builtin_fma` throughout, which is one instruction where the processor has it and a call to the math library where it has not. They are on by default wherever the compiler has a flag for them and the machine building runs a program compiled with it. They do nothing for the arithmetic, which does not multiply and add in one step. [GAOL v5]

- **The SSE2 intervals pay on the products and the additions.** $x \times y$ takes 2.04 ns rather than 4.32, and $x+y$ 1.07 rather than 2.30, the two bounds being computed in one instruction. They are on by default on x86 processors. The elementary functions do not go through them.

On a 64-bit ARM processor, Table 2.5 measures them with `tests/performance.cpp` on an Apple M2 with AppleClang 21.0, the median of nine runs of each program, 6 November 2026: the first two builds in turn in one session, the last two in another.

TABLE 2.5 – The options on 64-bit ARM: nanoseconds per operation, Apple M2, AppleClang 21.0, the median of nine runs; the last column is a line of 13 operations

Build	$x+y$	$x-y$	$x \times y$	x/y	`sqr`	`sqrt`	13 operations
`-DGAOL_SIMD=OFF` (FPU intervals)	1.15	1.15	3.26	3.22	1.78	5.17	50.6
Default (NEON intervals)	0.90	0.90	1.52	2.08	1.27	2.04	16.6
`-DGAOL_SIMD=OFF -DGAOL_PRESERVE_ROUNDING=ON`	37.4	37.7	43.2	43.9	42.6	46.6	519
`-DGAOL_PRESERVE_ROUNDING=ON` (NEON intervals)	26.6	27.1	27.8	35.1	27.2	35.7	370

- **The NEON intervals pay on every operation of the table.** $x-y$ takes 0.90 ns rather than 1.15, $x \times y$ 1.52 rather than 3.26 (of operands of random signs 1.60 rather than 3.32), x/y 2.08 rather than 3.22, `sqrt` 2.04 rather than 5.17, and a line of 13 operations 16.6 ns rather than 50.6. The two bounds are one vector, and the operations take them from one to the next in a register. The elementary functions, which CORE-MATH computes, go through them only for their arguments: no faster, no slower. They are on by default on 64-bit ARM processors, with every compiler of them.

- **The product, the quotient and the square of the FPU intervals take no branch on the signs of the bounds**, and $x-y$ reads its bounds after the check of the rounding direction. [GAOL v5] In the earlier versions of GAOL v5 the row read 3.54 for $x-y$, 3.25 for $x \times y$ (6.39 of operands of random signs), 3.98 for x/y, 2.07 for `sqr` and 49.6 for the line of 13 operations. The product of a line of operands of one sign, which a processor predicts, takes more time than it did (a line of 7 products, 35.0 ns rather than 23.3), the largest of four products waiting for two comparisons. With `GAOL_PRESERVE_ROUNDING` the write of FPCR waits for the result as well, and the FPU row of that option loses 6 to 7 % on operands of one sign ($x \times y$ 42.4 ns rather than 39.7 for the former code, measured in another session than Table 2.5, where it is 43.2) and gains on operands of random signs (42.4 rather than 47.5). These are the figures of this processor, with AppleClang. The processors of

the continuous integration that run the FPU intervals (32-bit ARM, Ubuntu with GCC and Clang, macOS; and, when they were measured, Visual C++ and clang-cl for x86 and for arm64, which have the SSE2 and the NEON intervals now), measured in turn, give other ratios, losses included (the line of 7 products 0.58 with GCC on Ubuntu, 0.57 with clang-cl for arm64); POWER, s390x and RISC-V are not measured (see Section 4.6, page 98).

- **The rounding direction is read with one `mrs` of FPCR** (0.8 ns on this processor) and written with `msr` only when the direction of the program is not upward: `msr` takes 4 ns for the value FPCR holds and 11 ns when it changes the direction. With `GAOL_PRESERVE_ROUNDING` an operation of a program that computes to nearest writes it twice, which is the 27 ns above (37 ns with the FPU intervals, which call `fegetround()` and `fesetround()`); a program that already computes upward writes nothing.

2.8.1 GCC for Windows

With GCC for Windows (MinGW-w64, MSYS2, Cygwin) on x86-64 and on 32-bit x86, the fused multiply-add instructions give `-mfma -mprefer-vector-width=128`; for 32-bit x86 the choice follows the measurements below. [GAOL v5] The earlier versions of GAOL v5 gave nothing, because of GCC bug 54412: `-mfma` turns AVX on, the vectorizer of GCC then writes 256-bit registers (`%ymm`), and GCC takes the stack to be aligned on 32 bytes for those it spills there, where the ABI of Windows x64 aligns it on 16, so that an aligned `vmovaps` to such a slot faults. Without `-mfma`, CORE-MATH called a software `fma()` and the integer powers computed no pairs of products. `-mprefer-vector-width=128`, an option of GCC from GCC 8, keeps the vectorizer to the 128-bit registers. `objdump -d --no-show-raw-insn` found no line with `%ymm` in the objects of GAOL, CORE-MATH and the tests built so by GCC 16.2 (MSYS2 UCRT64), against 30 598 lines with `-mfma` alone, and none without a flag; the whole suite of tests passed with the three. That is a count of instructions, and no proof that the defect cannot occur: the continuous integration makes the count again on the objects it builds, and the check of the fused multiply-add instructions with it (see Section 14.5, page 305). A GCC without the option gets no flag. The code using GAOL is given the two flags (`gaol::gaol`, `gaol.pc`); the 256-bit intrinsics it writes itself would stay exposed to the defect, as with `-mavx`.

Table 2.6 measures the two variants with `tests/performance.cpp` on the Windows Server 2022 runner of GitHub, MSYS2 UCRT64 with GCC 16.2, in nanoseconds per operation, the median of three runs of each variant, alternated in one job (on these lines, the runs of a variant differ by 1.16 times at most). `-mfma` alone gave the same times. The lines of 15 additions and of 7 products did not change (within 1.5 %; the line of 7 divisions took 6.3 % less). The elementary functions and `pow(x, 3)` gain the most, as with Clang above.

For 32-bit x86, Table 2.7 gives the same program with MinGW-w64 for i686 (MinGW-Builds, GCC 15.2 and 14.2, Windows Server 2022 runner of GitHub),

TABLE 2.6 – GCC for Windows on x86-64: nanoseconds per operation, Windows Server 2022 runner of GitHub, MSYS2 UCRT64 with GCC 16.2, the median of three runs of each variant

Flags	$x \times y$	x/y	`pow(x, 3)`	`exp`	`log`	`sin`	`cos`	`floor`
No flag (`-DGAOL_FMA=OFF`, what GCC for Windows had)	3.92	5.04	52.31	46.44	106.43	93.46	260.50	4.24
`-mfma -mprefer-vector-width=128` (default)	3.69	4.72	8.68	32.58	33.63	46.61	55.44	2.19

TABLE 2.7 – GCC for Windows on 32-bit x86: nanoseconds per operation and lines of the objects with `%ymm` and `vfmadd`, MinGW-w64 for i686 (MinGW-Builds GCC 15.2), Windows Server 2022 runner of GitHub

Flags	$x \times y$	`pow(x, 3)`	`log`	`cos`	`%ymm`	`vfmadd`
No flag (what GCC for Windows had)	8.65	70.45	265.68	489.59	0	0
`-mfma`	8.76	12.23	81.24	138.19	4 709	850
`-mfma -mprefer-vector-width=128`	8.66	12.12	81.93	138.33	0	850
`-mfma -mprefer-vector-width=128 -mstackrealign`	8.67	12.14	82.12	139.20	0	850

built, tested (38 tests in every variant, all passing, two skipped) and measured with each set of flags in turn, in nanoseconds per operation (GCC 15.2; an earlier run of the same variants gave times within 1.5 %). A third run on another runner, faster, gave the same ratios (`pow(x, 3)` 56.6 against 10.05 ns, `log` 235.9 against 64.7, `cos` 430.1 against 121.3 ns, with `-mfma -mprefer-vector-width=128`). With GCC 14.2 the ratios are similar (`pow(x, 3)` 67.7 against 12.1 ns, `log` 268.6 against 103.0, `cos` 490.2 against 137.0 in the third run), and `-mfma` alone leaves 600 lines with `%ymm`. GCC 13.2 (MinGW-w64 13.2 for x86) leaves 9 lines with `%ymm` in two test objects even with both options: the check lists them and does not fail on 32-bit x86. $x \times y$, which does not call `fma()`, did not change.

The stack of a thread on 32-bit Windows is aligned on 4 bytes (GCC bug 40838). A program that keeps 16 intervals alive across calls in a loop of $+$, $-$, $\times$, / and `sqr`, then in a loop of `pow(x, 3)`, `pow(x, 5)`, `pow(x, -4)`, `pow(x, 7)`, `exp`, `log`, `sin` and `cos` (which `-mfma` sends through the packed kernel of the integer powers), on the stack of the main thread, of a `CreateThread` thread and of a `std::thread`, gave the same results in the three, without a flag, with `-mfma` and with `-mfma -mprefer-vector-width=128`, with GCC 15.2 and 14.2. That does not prove that a stack aligned on 4 bytes can never meet an aligned access of 16 or 32 bytes. `-mstackrealign` changed no time with GCC 15.2 (1 % at most); `-mincoming-stack-boundary=2` was measured once, with the times of the base. GAOL gives neither. It gives `-mprefer-vector-width=128` with `-mfma` as on x86-64, which leaves no `%ymm` in the objects at the same speed; that bug 54412 acts on 32-bit x86 was not shown (GCC 15.2 for 32-bit Linux realigns the stack itself for the 256-bit spills it makes).

These flags are for GCC for Windows on x86-64 and on 32-bit x86: MinGW-w64, MSYS2 and, since the tests of the three builds name it (`CYGWIN` of CMake, `__CYGWIN__` of `configure` and meson; the AVX-512 path names `__CYGWIN__` too), Cygwin. [GAOL v5] The MSYS environment of MSYS2 (not its MinGW ones) is a Cygwin as far as these tests go, and gets the same flags. If the user gives a `-mprefer-vector-width=` of their own, the builds order it differently from GAOL's: CMake and `configure` put the flags of GAOL after the user's (`CMAKE_CXX_FLAGS`, `CXXFLAGS`), meson puts them before `cpp_args`. The last of two such options is the one that counts (as tried with Clang). `-mprefer-vector-width=128` does not undo what other options of the user turn on. With the Clang that cross-compiles for x86-64, the objects of GAOL, CORE-MATH and the tests built with `-mavx512f -mfma -mprefer-vector-width=128` held 2 lines with `%ymm` and 2 887 with `%zmm`, and those built with `-march=skylake-avx512 -mprefer-vector-width=128` 406 lines with `%ymm`; a second reader counted none, and 4: the gap is not explained, and the `%zmm` may come from the explicit intrinsics of the AVX-512 path rather than from the vectorizer. Nothing was run with GCC but a small loop, which GCC 15.2 compiled with no `%ymm` and no `%zmm` with `-mavx512f -mfma`, with `-march=skylake-avx512` and with `-march=native`, all with `-mprefer-vector-width=128`. The code of GCC itself (`ix86_option_override_internal`, version 14.2, read, not run)

applies the preferred width of an architecture only if none was given, so that `-mprefer-vector-width=128` should win over a `-march`. A program built with such options is exposed to GCC bug 54412, as with `-mavx`.

Remark 2.8.1 *Cygwin x86-64 is built and tested by a job of the continuous integration (GCC 14.4, CMake 4.4, Release and Debug; not measured), whose first runs found what Cygwin needed.* [GAOL v5] *Its* `<stdlib.h>` *does not declare* `posix_memalign()` *under* `-std=c++11` *(the aligned memory is* `_mm_malloc()` *there). Its* `fma()` *is not correctly rounded, and* `std::fma` *is a call of it without optimization: GAOL calls* `__builtin_fma` *(*`fma_once()`*), which is the instruction with* `-mfma` *at* `-O0` *and at* `-O3` *(both checked), and refuses Cygwin without* `-mfma` *(see Section 2.3.2, page 25). Its* `nextafter(-DBL_MAX, -oo)` *is* `-DBL_MAX` *in the rounding direction upward, which* `previous_float()` *and* `next_float()` *now tell apart (the lower bound of* `sinh(-711)` *did not enclose the value).* `is_bisectable()` *and* `bisect()` *call no* `nextafter()`*: they tell the bounds by their bits.* `tests/core_math.cpp` *and* `tests/arithmetic.cpp` *call* `__builtin_fma` *as GAOL does.*

2.8.2 Interprocedural optimization is not offered

Compiled with it (`-flto`), the basic operations of GAOL are 1.1 to 1.5 times faster where the code using GAOL is compiled with it too: an addition of intervals took 0.73 ns rather than 1.07 with Clang 21.1, $x \times y$ 1.71 rather than 2.04, `sqr(x)` 1.27 rather than 1.80. The elementary functions are no faster, and a little slower: `cos` took 39.2 ns rather than 36.6, `sin` 32.6 rather than 31.3, `log` 22.7 rather than 21.7. With Clang 18.1, before the operators took their bounds in a register, the basic operations took two to three times less time with it and the elementary functions two to three times more. The sources of CORE-MATH are compiled apart, without it. The standard options of the tools reach the sources of GAOL, `CMAKE_INTERPROCEDURAL_OPTIMIZATION`, `-Db_lto=true` of meson and `-flto` in `CXXFLAGS`, which the three builds keep away from CORE-MATH alone: they are not to be given with GCC.

With GCC, moreover, it breaks the bounds: GAOL then fails `tests/rounding_direction.cpp`, which checks that an operation gives the same result whatever the rounding direction the calling code left. With `GAOL_PRESERVE_ROUNDING`, `textToInterval("sin(1)+exp(0.1)")` gave an upper bound one double below the right one when the caller left the direction downward or toward zero: the compiler moves floating-point operations across the changes of rounding direction, which `-frounding-math` is meant to forbid and which it does not do across translation units. Compiling the sources of GAOL alone with it is enough for the test to fail. Clang 18.1 passes the tests with it, so this is not a property of the option but of the handling of it by one compiler, which is reason enough not to offer it. An interval that does not enclose the exact value is worse than a slow one: there is no option for this, and `gaol.pc` and `gaol::gaol` carry no such flag.

2.9 Tests, examples, performance and the parser

Built with `WITH_TESTS` (CMake), `--with-tests` (`configure`) or `with-tests` (meson), the programs of `tests` compare the bounds GAOL computes with the exact results of the operations, independently of GAOL and of the floating-point environment: the exact results are computed with integers, or were computed with `mpmath`, with 1500 to 7000 bits of precision. They check, among others, that the basic operations are the tightest enclosures, that the elementary functions are the tightest or within one double of them, that every operation gives the same result whatever the rounding direction the calling code left, that numbers are read exactly and intervals written rounded outward, that the reverse functions are accurate in the sense of IEEE 1788-2015 [9], and that each name of `gaol_ieee1788` is the operation of the standard it names. The unit tests of GAOL 4, which were in `check/` and checked the results of the operations on chosen intervals with CppUnit, are among them, in `tests` now, and need CppUnit no more. [GAOL v5] What each program checks is described in Chapter 14 (see Section 14.2, page 274). The three builds have the same targets (Table 2.8).

Table 2.8 – The targets of the three builds

	CMake	`configure`	meson
The unit tests of `tests`	`make test` (`cmake --build build --target test`; `RUN_TESTS` with Visual Studio), with `WITH_TESTS`	`make test`, with `--with-tests`	`meson test` or `ninja test`, with `with-tests`
The unit tests of `tests` and the examples of `examples`	`make check` (`--target check`), with `WITH_TESTS` and `WITH_EXAMPLES`	`make check`, with `--with-tests` and `--with-examples`	`ninja check` (`meson test --setup check`), with `with-tests` and `with-examples`
GAOL v5 measured on the benchmark of the comparison of the libraries	`make perf`	`make perf`	`ninja perf`
The lexer and the parser regenerated with flex and bison	`make parser`	`make parser`	`ninja parser`
The manual of GAOL v5	`make pdf` (`--target pdf`), with `WITH_DOC`	`make -C manual pdf`	`ninja pdf`, with `with-doc`

- **The tests.** `ctest` runs the tests and the examples of a CMake build; `make test` leaves the examples out (label `example`, CMake 3.17 and later), and runs the compile tests `refused_*` and `nodiscard_*` too. With meson before 0.57, `meson test` runs the examples too, and `meson test --suite unit` the tests alone.

- **The examples** check what they print and fail otherwise (see Section 14.2, page 274).
- **`make perf`** compiles `doc/compare/code/bench_gaol.cpp` with the flags and the library of the build, runs it on the intervals of the benchmark, and writes its times into the tables of `doc/compare/performance.md`, with the times of the other libraries measured by the last whole run, which `doc/compare/code/results.csv` keeps: GAOL 4.2.3, filib++, libieeep1788, PROFIL/BIAS, Solaris Studio and Boost.Interval are not run (see Section 12.5, page 237); it runs the ten long lines of `doc/compare/code/run_lines.sh` too. It needs bash, and Python 3 with numpy; with CMake and meson, Linux and macOS (Linux only before), and with `configure` wherever bash is. There is no target `perf` in the Debug build of CMake, nor with `-Denable-debug=true` of meson. [GAOL v5]
- **`make parser`** regenerates the lexer and the parser of the strings with the flex and the bison of the system, where the CMake build finds them (`find_program`, not required; see Section 2.3.3, page 29).
- **`make pdf`** builds this manual with pdflatex, bibtex and makeindex, by `manual/build-pdf.sh`, which the three builds run: with CMake, after `cmake -S . -B build -DWITH_DOC=ON`, by `cmake --build build --target pdf`, which CMake had not before. [GAOL v5] The PDF is written in `manual` of the sources, `manual/Manual_GAOLv5.pdf`, and not in the build directory, which keeps the files of LaTeX; the manual of GAOL 4 is not built, its PDF, `manual/v4/Manual_GAOLv4.pdf`, was built once; the target is not part of `all`. This manual can also be compiled by hand in `manual/v5`: `pdflatex manual_gaolv5.tex`, then `bibtex manual_gaolv5`, `makeindex manual_gaolv5.idx -o manual_gaolv5.ind` and `pdflatex` twice; without `gaol_version.tex`, which the builds write, the title page takes the version of `VERSION.txt`, the edition 5.0 and today's date.

`tests/find_package` builds the same tests with an installed GAOL, and `tests/fetch_content` with a GAOL brought into a project by `FetchContent`. `gaol_performance` (`tests/performance.cpp`) measures the time per operation of the arithmetic and the elementary functions of GAOL; its figures are those of Section 2.8.

2.10 What is installed

Each build installs:

- **the headers**, in *`prefix`*`/include/gaol`;

- **the library**, `libgaol`, which holds CORE-MATH: static by default with the three builds, or shared with `BUILD_SHARED_LIBS`, `--enable-shared` of `configure` and `-Ddefault_library=shared` of meson (static only with Visual C++). The shared library is `libgaol.so.5.0.0`, whose soname is `libgaol.so.5`, with the three builds;
- `gaol.pc`, for pkg-config, in the `pkgconfig` directory of the library directory (except the CMake build with Visual C++);
- **the CMake package of GAOL**, with CMake, which defines the target `gaol::gaol`.

Both `gaol.pc` and `gaol::gaol` carry the flags that the code including the headers of GAOL has to be compiled with (see Section 3.3, page 64).

2.10.1 The configuration header

Each build writes `gaol/gaol_configuration.h`, which `gaol/gaol_config.h` includes with every compiler: CMake from `cmake/gaol_configuration.h.in`, `configure` from `gaol/gaol_configuration.h.in` (written by hand, not by autoheader), meson from `gaol/gaol_configuration_meson.h.in`. The three define the same macros, and nothing else, but for the switch of a developer that CMake alone writes, `GAOL_SSE2_UNALIGNED_STORAGE` (Table 2.2); the continuous integration checks that they write the same header on Ubuntu 24.04 (x86-64 and arm64, GCC and Clang), macOS 15 (arm64 and x86-64), Debian 12 (i386), Debian 13 (armhf) and MSYS2 (UCRT64 and MINGW32) (Section 2.4). [GAOL v5] The sources of GAOL read all of them but `GAOL_HAVE_AVX512_TARGET` and `GAOL_PREFER_AVX512`, which the three builds use to decide `GAOL_AVX512_ARITHMETIC` and the continuous integration prints, and `GAOL_UNIVERSAL_SSE2` and `GAOL_UNIVERSAL_NEON`, which only the configuration header reads, to define the macros of the architecture being compiled; `GAOL_USING_SSE3_INSTRUCTIONS` is read only by the code of the intervals of floats, which a developer of GAOL compiles.

Remark 2.10.1 *Each macro of Table 2.9 but the version,* `GAOL_UNIVERSAL_SSE2` *and* `GAOL_UNIVERSAL_NEON` *is 1 where the table says, and 0 otherwise; those two are defined only in a universal build of macOS.* [GAOL v5] *A macro that is 0 now was left undefined in GAOL 4, so that a header reading it with* `#if` *warned under* `-Wundef`*, and a program could not include GAOL under* `-Wundef -Werror`*. A program tests these macros with* `#if` *as well:* `#ifdef` *is now true of each of the others.*

TABLE 2.9 – The macros of gaol/gaol_configuration.h

Macro	1 when
GAOL_MAJOR_VERSION, GAOL_MINOR_VERSION, GAOL_MICRO_VERSION, GAOL_VERSION	Always, from VERSION.txt: 5, 0, 0 and "5.0.0" for GAOL 5.0.0 (project() with CMake and meson, the file read when configure runs)
GAOL_DEBUGGING	In a Debug build: GAOL_DEBUG_VERBOSE(level, command) runs its commands (CMAKE_BUILD_TYPE=Debug or GAOL_DEBUG=ON, --enable-debug, --buildtype=debug or enable-debug)
GAOL_EXCEPTIONS_ENABLED	GAOL raises exceptions rather than abort: by default in the three builds (GAOL_EXCEPTIONS_ENABLED=OFF, --disable-exceptions, enable-exception=false turn it off)
GAOL_PRESERVE_ROUNDING	The operations restore the rounding direction they found (GAOL_PRESERVE_ROUNDING, --enable-preserve-rounding, enable-preserve-rounding)
GAOL_PREFER_AVX512	The four arithmetic operations, the square, the square root and, with FMA, the integer powers beyond 1 and −1 are to take the AVX-512 instructions: with GAOL_PRESERVE_ROUNDING, where the compiler takes the path (GAOL_HAVE_AVX512_TARGET); GAOL_PREFER_AVX512=ON or OFF, --enable-prefer-avx512, --disable-prefer-avx512 and enable-prefer-avx512 decide otherwise
GAOL_AVX512_ARITHMETIC	The same operations take the AVX-512 instructions: GAOL_PREFER_AVX512, GAOL_HAVE_AVX512_TARGET, and a machine building that runs them, not when cross-compiling; the library then needs a processor with them (checked by the three builds)
GAOL_VERBOSE_MODE	A line on the standard error when GAOL initializes and cleans up (GAOL_VERBOSE_MODE=ON, --enable-verbose-mode, enable-verbose-mode)
GAOL_USING_SSE2_INSTRUCTIONS	The intervals computed with SSE2 (x86; on 32-bit Windows with GCC and Clang told to ask for aligned memory, -faligned-new=8, and with Visual C++ and clang-cl, whose bounds are two doubles there): GAOL_SIMD, --enable-simd, enable-simd, on by default

Table 2.9, continued

Macro	1 when
GAOL_USING_NEON_INSTRUCTIONS	The intervals computed with NEON (64-bit ARM, with every compiler; not 32-bit ARM): GAOL_SIMD, --enable-simd, enable-simd, on by default (GAOL_ARM64, --enable-arm64 and enable-arm64 were options of their own)
GAOL_USING_SSE3_INSTRUCTIONS	And compiled with -msse3, where the compiler takes it
GAOL_UNIVERSAL_SSE2, GAOL_UNIVERSAL_NEON	A universal build of macOS (several architectures in CMAKE_OSX_ARCHITECTURES, or -arch several times in the flags of configure and meson): the three macros above are then those of the architecture being compiled, GAOL_USING_SSE2_INSTRUCTIONS and GAOL_USING_SSE3_INSTRUCTIONS for x86_64 and GAOL_USING_NEON_INSTRUCTIONS for arm64, where GAOL_SIMD allows them; the fused multiply-add and the AVX-512 path stay off, their tests failing for one of the architectures
GAOL_HAVE_AVX512_TARGET	The compiler takes __attribute__ ((target("avx512f"))) and __builtin_cpu_supports("avx512f"), which the AVX-512 path needs; never GCC for Windows (MinGW-w64, Cygwin), which does not align the stack for the AVX-512 registers (GCC bug 54412); Visual C++ for x64 takes the instructions in any function and tells the processor by cpuid and xgetbv, clang-cl has the attribute and does the same (checked by the three builds)
GAOL_HAVE_VISIBILITY_OPTIONS	The library is compiled with -fvisibility=hidden (checked)
GAOL_HAVE_FENV_H	The compiler has <fenv.h> (checked; required by CMake)

What GAOL needs to know of the processor and the system comes from the compiler, in gaol/gaol_config.h: GAOL_IX86_MACOSX and GAOL_ARM_MACOSX, the sizes of the integer types (GAOL_SIZEOF_INT, GAOL_SIZEOF_LONG_LONG_INT, from <limits.h>) and the order of the bytes (GAOL_WORDS_BIGENDIAN), the names of the systems and the order of the bytes being 1 or 0. GAOL_IX86_LINUX and GAOL_AARCH64_LINUX are defined as well, though no code of GAOL reads them. These macros, and the GAOL_USING_SSE* and GAOL_HAVE_* of the table, were named without GAOL_ before GAOL v5. [GAOL v5]

Remark 2.10.2 *The* `configure` *and meson builds of GAOL 4 defined some fifty other macros, of GAOL 4 or of autoconf, which nothing read, and they are gone:* [GAOL v5] *the checks of C headers (*`HAVE_STDLIB_H`*,* `HAVE_UNISTD_H`*,* `STDC_HEADERS`*...), of functions (*`HAVE_FLOOR`*,* `HAVE_POW`*,* `HAVE_MALLOC`*,* `HAVE_REALLOC`*,* `HAVE_FINITE`*...), of types and sizes (*`HAVE__BOOL`*,* `SIZEOF_LONG`*...),* `const`*,* `inline`*,* `size_t`*,* `malloc` *and* `realloc` *(*`rpl_malloc()` *and* `rpl_realloc()` *of* `AC_FUNC_MALLOC` *and* `AC_FUNC_REALLOC` *in a cross-compilation),* `CLOCK_IN_HEADER`*,* `LT_OBJDIR`*,* `PACKAGE`*,* `VERSION` *and* `PACKAGE_*`*, which* `gaol/gaol_config.h` *undefined, undefining those of the code including GAOL.* `HAVE_LIMITS`*,* `HAVE_CASSERT`*,* `HAVE_CLOCK` *and* `GETRUSAGE_IN_HEADER` *were read, but are true with every compiler of C++11, and* `HAVE_NEXTAFTER` *and* `HAVE_ISNAN` *were read for Visual C++, which has both: the code that read them no longer does.*

2.11 Discussion

This chapter established how GAOL v5 is built: from the sources alone, with CORE-MATH compiled into `libgaol`, by one of three builds that give it the same macros and the same flags, and whose few options are the same under three names. The choices it explained are those of a library whose bounds must be the same everywhere. The compilers and the options that gave bounds not enclosing the exact results, or that nothing certifies, are refused when GAOL is configured or compiled, and, but for Clang 14 for 64-bit ARM, again when the code using it is compiled, and the link with `-Ofast` is countered by the clearing of the flush-to-zero modes and by `-mno-daz-ftz`. The fused multiply-add instructions and the AVX-512 path are decided by what the machine building runs. The elementary functions gain up to 2.5 times with the fused multiply-add instructions (4.7 times with GCC for Windows), and the arithmetic operations 2.9 times with the AVX-512 path where the rounding direction is preserved; the price is a library bound to the processors that have them, and a package for any processor turns them off. Interprocedural optimization, which would gain 1.1 to 1.5 times on the basic operations, is not offered, because GCC then moves operations across the changes of the rounding direction.

Some points are left open. The refusal of the older MinGW-w64 is not tested on ARM; the processors POWER, s390x and RISC-V are not measured; the flush-to-zero modes of these processors are neither checked nor cleared; the Python aliases of Windows were read in the sources of meson, not checked; and the cause of the failure of `package_source` on Cygwin is not known. The code using GAOL still has to be compiled with the flags of interval arithmetic and without the options that no macro shows: the next chapter shows how a program includes the headers, compiles with these flags, from CMake, pkg-config or meson, and uses the library.

Chapter 3

Using GAOL

The previous chapter built and installed GAOL. This chapter shows how a program uses it: what it includes, how it is compiled and linked with the flags of interval arithmetic that the inline operations of GAOL require, and which of the two namespaces it opens. It then reviews what GAOL gives to the algorithms of interval computing, the errors most often made with intervals, and the names of the other interval libraries next to those of GAOL.

In this chapter, we assume that GAOL is installed, and that its library and its headers are accessible to the compiler (Chapter 2). The first section walks through a complete program, from the header it includes to the enclosure it prints. The second section lists the headers a program may include, and the third gives the flags of interval arithmetic and the way to take them from CMake, from pkg-config and from meson. The fourth and the fifth sections present the namespaces `gaol` and `gaol_ieee1788`, and the power, where they differ. The sixth section tells how an integer is taken, and the seventh what the compiler says of a result thrown away. The eighth section gathers the tools of interval algorithms, the ninth summarizes how GAOL reports an error, the tenth reviews the common errors, and the last one gives the names of IBEX, Codac, C-XSC and Boost.Interval next to those of GAOL, and the meaning of the relation symbols in these libraries and in PROFIL/BIAS.

3.1 A first program

Let us consider the Goldstein-Price function

$$f(x,y) = \Big(1 + (x+y+1)^2(19 - 14x + 3x^2 - 14y + 6xy + 3y^2)\Big) \\ \times \Big(30 + (2x-3y)^2(18 - 32x + 12x^2 + 48y - 36xy + 27y^2)\Big)$$

and let us enclose its range for $x \in [-2, 2]$ and $y \in [-2, 2]$.

Example 3.1.1 *The program below computes the formula of f with intervals in place of numbers. The line under it is what it prints.*

Example

```
1  #include <iostream>
2  #include <gaol/gaol.h>
3  using namespace gaol;
4
5  int main(void)
6  {
7    interval
8      x(-2,2),
9      y(-2,2), z;
10
11   z=(1+sqr(x+y+1)*(19-14*x+3*sqr(x)-14*y+6*x*y+3*sqr(y)))*
12     (30+sqr(2*x-3*y)*(18-32*x+12*sqr(x)+
13        48*y-36*x*y+27*sqr(y)));
14
15   std::cout << "z = " << z << std::endl;
16   gaol::cleanup(); // Always, after the last use of GAOL
17   return 0;
18 }
19 ➥ z = [-87881320, 147125080]
```

The program includes the header `gaol/gaol.h`, which gives access to the intervals and their functions in both namespaces; the expressions have headers of their own (see Section 3.2, page 63). The type `interval` and the functions of GAOL are in the namespace `gaol_core`, which a program names through the namespace `gaol`: the `using` directive of Line 3 imports it. The `gaol/gaol.h` header of GAOL 4 imported the namespace itself; that of GAOL v5 no longer does, and a program adds the directive, or names `gaol::interval` and `gaol::sqr` (see Section 3.4, page 69). [GAOL v5]

Nothing has to be called before using GAOL: `gaol::init()` runs by itself when the program starts, before `main()`, with every compiler (see Section 4.10, page 112), and each operation of GAOL sets the rounding direction it needs (see Section 4.1, page 88). [GAOL v5]

Remark 3.1.2 *The program calls* `gaol::cleanup()` *right after its last use of GAOL, at the latest at the end of* `main()`*, whatever the compiler. GAOL also calls it itself, but only when the program ends: without the call, and unless GAOL is built to preserve the rounding direction, the code after the last use of GAOL would compute its doubles rounded upward (Remark 4.10.1 gives the reason).* [GAOL v5]

The function `sqr(x)` is the square of x, and is equivalent to `pow(x,2)`: it is the function that `pow(x,2)` calls, with the same bounds, the sign of a zero bound included. [GAOL v5]

Let `f.cpp` be the name of the file containing the program above. With GAOL installed in *prefix* (Chapter 2), pkg-config is told where `gaol.pc` is, and the program is compiled with the flags it gives (see Section 3.3, page 64):

```
% export PKG_CONFIG_PATH=prefix/lib/pkgconfig
% c++ -std=c++11 -O2 $(pkg-config --cflags gaol) -o f f.cpp \
      $(pkg-config --libs gaol)
```

GAOL is the only library to link: CORE-MATH, which computes its elementary functions, is compiled into it, and the C++ compiler links the mathematical library of the system itself.

Executing `f` prints the line shown under the program. We then know that $f(x, y)$ lies in $[-87881320, 147125080]$ for every x and y in $[-2, 2]$: `z` *encloses* the range of f. It is not the range, which is $[3, 1015690.27\ldots]$: the minimum is $f(0, -1) = 3$, and the maximum is reached on the edge $y = 2$. `z` is 231 times wider. Each occurrence of x and of y in the formula ranges over $[-2, 2]$ on its own, as if the others did not exist: this is the dependency problem (see Section 1.4, page 11), which makes an enclosure wider than the range. Cutting x and y into smaller intervals, and taking the hull of the values of f over the pieces, narrows the enclosure, at the price of more evaluations; `examples/03_dependency_problem.cpp` of the sources does it.

3.2 The headers

A program includes `gaol/gaol` (or `gaol/gaol.h`), and `gaol/parser/gaol_expression.h` for the expressions, with `gaol/parser/gaol_expr_eval.h` to evaluate them. The three builds install these headers and the headers they include. Each of the installed headers compiles alone, but for the four that other headers include in the middle of their code (`gaol/core/gaol_interval_fpu.h`, `gaol/core/gaol_interval_neon.h`, `gaol/core/gaol_interval_sse.h`) or that Visual C++ alone uses (`gaol_fpu_msvc.h`). The headers of the sources of GAOL are not installed: [GAOL v5] `gaol/parser/gaol_interval_parser.h`, which did not compile when included, `gaol_init_cleanup.h`, `gaol/math/gaol_core_math.h`, `gaol/math/gaol_u128.h`, `gaol/math/gaol_compoundm1.h`, `gaol/math/gaol_roundeven.h`, `gaol/math/gaol_ldexp.h`, `gaol/math/gaol_core_math_port.h`, `gaol/math/gaol_double_op.h` (the bounds at doubles, `exp_dn()`..., which are no longer part of the interface), and the intervals of floats, `gaol/float/gaol_intervalf.h` and `gaol/float/gaol_interval2f.h`. Headers of GAOL 4 are gone: `gaol_parameters.h`, which declared nothing, `gaol_double_op_apmathlib.h` and `gaol_double_op_crlibm.h`, of the mathematical libraries GAOL 4 was built with, `gaol_limits.h`, `gaol_flags.h`, `gaol_profile.h`, `gaol_allocator.h`, `gaol_exceptions.h`, `gaol_exact.h` with `sysdeps/`, which `dtoa.c` needed, and `gaol_assert.h`, whose `GAOL_ASSERT` is `assert()` now. The headers of the expressions and of the parser moved to `gaol/parser/`: a program that included `gaol/gaol_expression.h` includes `gaol/parser/gaol_expression.h` (Chapter 15).

The headers of GAOL compile without a warning under `-Wall -Wextra -Wold-style-cast -Wundef`, and put nothing into the global namespace. `gaol/gaol_exceptions.h` declared `string` and `exception` there; the header

is gone, GAOL throws the exceptions of the standard library (see Section 3.9, page 76), and a program names `std::string` and `std::exception`. [GAOL v5] The macros the headers define start with `GAOL_` (or `gaol_`), so as not to meet those of the program or of another library: [GAOL v5] `USING_SSE2_INSTRUCTIONS`, `HAVE_FENV_H`, `WORDS_BIGENDIAN`, `INLINE`, `MEMALIGN`, `__HI...` are now `GAOL_USING_SSE2_INSTRUCTIONS`, `GAOL_HAVE_FENV_H`, `GAOL_WORDS_BIGENDIAN`, `GAOL_INLINE`, `GAOL_MEMALIGN`, `GAOL_HI...` The header `<stdlib.h>` is no longer included with `_XOPEN_SOURCE` defined again, and `nb_fp_numbers()` returns an `unsigned long long`, which GAOL 4 named with the macro `ULONGLONGINT`.

The doubles `pi`, `half_pi`, `two_pi`, `pi_dn`, `pi_up`, `half_pi_dn`, `half_pi_up`, `ln2_dn`, `ln2_up`, `two_power_51` and `two_power_52` of GAOL 4 are no longer declared for the program, which found them with `using namespace gaol`, its own `pi` being then ambiguous. [GAOL v5] The bounds of π are those of `interval::pi()`, and the doubles around π and $\pi/2$ are `gaol_pi_dn`, `gaol_pi_up`, `gaol_half_pi_dn` and `gaol_half_pi_up`, in `gaol`, `gaol_ieee1788` and `gaol_core`.

3.3 Compiling a program using GAOL

Part of the operations of GAOL on intervals is inline, in the headers: the code that includes the headers of GAOL has to be compiled with the *flags of interval arithmetic*, not only GAOL itself. [GAOL v5] With GCC and Clang, each where the compiler takes it, the flags are the following.

- `-frounding-math -fno-fast-math -ffp-contract=off`, so that the compiler rounds each operation in the direction in effect, as written, and neither evaluates it at compile time in the default rounding nor contracts a multiplication and an addition into a fused one.
- `-msse2 -mfpmath=sse` on 32-bit x86, so that doubles are computed in double precision rather than on the x87 unit.
- `-msse2 -msse3` on x86 processors with the SSE2 intervals (`GAOL_SIMD`).
- `-faligned-new=8` with GCC and Clang on 32-bit Windows (MinGW-w64, MSYS2) with the SSE2 intervals, so that the compiler asks for aligned memory for the intervals, which the C runtime would align on 8 bytes instead of 16. [GAOL v5] `gaol::gaol` and `gaol.pc` give it to the code using GAOL, which allocates the intervals, and `gaol/gaol_config.h` checks it. It is an option of C++: GCC warns of a C file compiled with it ("valid for C++/ObjC++ but not for C"), and with `-Werror` the warning is an error. This happens where the flags of `gaol.pc`, or `gaol_dep` of meson, are given to every file of a target, whereas `gaol::gaol` of CMake gives `-faligned-new=8` to the C++ files only: a project that compiles C files gives the option to its C++ files only, pkg-config having no flag for one language. Visual C++ and clang-cl have no

such flag, and need none: their SSE2 intervals on 32-bit Windows keep their two bounds as two doubles, whose alignment is 8 (`GAOL_SSE2_UNALIGNED_STORAGE` of `gaol/gaol_config.h`).

- `-mstackrealign` (or `-mincoming-stack-boundary=2`) with GCC on 32-bit x86, only for code that something calls with the stack aligned on 4 bytes, and not on 16 as GCC takes it (bug 40838 of GCC): a caller that does not follow the ABI of Linux, a hand-written trampoline or a foreign runtime for instance. A function of the program that keeps intervals in locals then stops on a `movaps` at an address that is not aligned (`test_stack_entered_aligned_on_4_bytes` of `tests/misc.cpp`, see Section 14.2, page 274). [GAOL v5] The programs for 32-bit Windows need neither: the system enters its threads and its callbacks with the stack aligned on 4 bytes, but the compilers for Windows realign the stack of a function that needs it, and that test passes without them with GCC for MinGW-w64, with Clang and with clang-cl, in the continuous integration.

- Nothing more on 64-bit ARM processors with the NEON intervals (`GAOL_SIMD`, see Section 4.5, page 96): the first item is all they ask for. [GAOL v5]

- `-ffloat-store` where doubles are still computed in extended precision (`FLT_EVAL_METHOD` not 0); on x86 this is the x87 unit, which `gaol/gaol_config.h` refuses (see Section 2.3.2, page 25).

- `-mfma` on x86, and `-mfpu=neon-vfpv4 -mfloat-abi=hard` on 32-bit ARM, where GAOL is compiled with the fused multiply-add instructions of the processor (`GAOL_FMA`, see Section 2.5.3, page 38): the code using GAOL then runs only on a processor with them, as GAOL itself. With GCC for Windows on x86-64 and on 32-bit x86 the flags are `-mfma -mprefer-vector-width=128`, the second keeping the vectorizer of GCC to the 128-bit registers, which it would otherwise spill to a stack that Windows aligns on 16 bytes, not 32 (GCC bug 54412): the objects of GAOL, CORE-MATH and the tests built so by GCC 16.2 have no instruction on a 256-bit register. A program that writes 256-bit intrinsics itself is exposed to that defect, as with `-mavx`, and so is one built with `-mavx512f` or a `-march` that turns on wider registers, which the option may not always undo. Cygwin and MSYS get these flags as the other GCC for Windows; Cygwin is built and tested by a job of the continuous integration, and refused without them. [GAOL v5]

With Visual C++ and clang-cl, the flags are `/fp:strict`, `/arch:AVX2` for x64 with `GAOL_FMA`, and `-msse2` with clang-cl for 32-bit x86, which computes doubles on the x87 unit with `/arch:SSE2`. CMake installs these flags with GAOL in `gaol::gaol`, and meson in `gaol.pc` and `gaol_dep`. `gaol/gaol_config.h` refuses code compiled by Visual C++, or by clang-cl from Clang 16, without `/fp:strict`. Clang does not tell the code whether `-frounding-math` was given, nor GCC before 15 (GCC 15 defines `__ROUNDING_MATH__`, which `gaol/gaol_config.h` does not read), and neither

tells whether `-ffp-contract=off` was given: there, it only refuses what contradicts them, `-ffast-math`, `-ffinite-math-only` and doubles computed on the x87 unit (see Section 2.3.2, page 25). The order of the options matters. `-fno-fast-math` turns the first two off when it comes after them on the command line, and does nothing when `-ffast-math` or `-ffinite-math-only` comes after it, where the compilation stops. `-Ofast`, which turns `-ffast-math` on, is refused with it, except with GCC when `-fno-fast-math` is anywhere on the command line. With Clang, `-ffast-math` and `-Ofast` also turn `-frounding-math` off, which `-fno-fast-math` does not turn on again: they have to come before the flags of interval arithmetic, not after them.

Remark 3.3.1 *With* `-frounding-math` *on x86, GCC 12.1 to 12.3 and 13.1 to 13.2 (Debian 12 has GCC 12.2) initialize wrongly a double member of a static array of structures given* `std::numeric_limits<double>::denorm_min()`*: they store the long double* 2^{-1074} *there, whose first eight bytes read* -0 *and whose last two overwrite the next member (*`{ denorm_min(), DBL_MAX }` *read* $[-0, \mathrm{NaN}]$*, the empty set, on 32-bit x86). A double or an array of doubles initialized so is right, and so are GCC 12.4, 13.3 and later. A program built with these releases computes such a value at run time, as* `tests/rounding_direction.cpp` *does.* [GAOL v5]

The link of the program takes one option too, where the compiler accepts it (GCC 13 and later on x86, from 11.4 and 12.4 in the series 11 and 12, and Clang 20 and 21, not Clang 18): `-mno-daz-ftz`, which keeps out of the link the file that sets the flush-to-zero and denormals-are-zero modes when a program is linked with `-Ofast` (see Section 4.8, page 104). `gaol::gaol` and `gaol.pc` carry it too, where the compiler that built GAOL accepts it: the three builds test the compiler for it with a link. [GAOL v5]

3.3.1 From CMake

A CMake project takes everything from the target `gaol::gaol` of the package of GAOL:

Example

```
1 find_package(gaol REQUIRED)
2 target_link_libraries(my_target PRIVATE gaol::gaol)
```

A GAOL installed in a *prefix* of its own is found by giving `-DCMAKE_PREFIX_PATH=`*`prefix`* to the first `cmake` of the project. `gaol::gaol` carries the include directory, the flags above, and the link option above for the code built by GCC 13 or a later one, or by the compiler that built GAOL, of its major version and not older: another compiler, as Clang 18, would stop on it, and so would GCC 12.0 to 12.3 with a GAOL built by GCC 11.4, GCC having the option from 11.4 and 12.4 only in the series 11 and 12. It no longer carries, for Visual C++, the definition `__GAOL_PUBLIC__=` of the earlier versions of GAOL v5: GAOL being a static

library there, its declarations carry no `__declspec(dllimport)`, and a definition left in a project changes nothing. [GAOL v5] There is no other library to link: CORE-MATH is compiled into `libgaol` itself. A library whose headers include those of GAOL, as those of Codac do, links `gaol::gaol PUBLIC`, so that its own users get the flags, and its CMake package finds GAOL again (`find_dependency(gaol)`). `tests/find_package` is a project using an installed GAOL this way.

A project can also build GAOL for itself, with the options it wants:

Example

```
include(FetchContent)
FetchContent_Declare(gaol GIT_REPOSITORY
    https://github.com/Jordan08/GAOL.git GIT_TAG master)
FetchContent_MakeAvailable(gaol)
target_link_libraries(my_target PUBLIC gaol::gaol)
```

GAOL is then a target of the project, built with it, and `cmake --install` of the project installs it with it, CMake package and `gaol.pc` included; nothing is downloaded beyond the sources of GAOL. `tests/fetch_content` is a project building GAOL this way. GAOL is built in Release where the project gives no build type, and `-DGAOL_DEBUG=ON` builds it for debugging whatever the build type or the configuration of the project, whose own code keeps them (see Section 2.5.2, page 36). [GAOL v5] GAOL, a static library, is compiled as position-independent code (`-fPIC`), so that `gaol::gaol` can be linked into a shared library, such as Python bindings. A project that sets `CMAKE_POSITION_INDEPENDENT_CODE`, `ON` or `OFF`, before `FetchContent_MakeAvailable(gaol)`, or on the command line, is followed instead.

3.3.2 From pkg-config

Each build installs `gaol.pc` in the `pkgconfig` directory of its library directory (*`prefix`*`/lib/pkgconfig`, or `lib64` or `lib/`*`multiarch`* rather than `lib` on the systems whose libraries go there), except the CMake build with Visual C++. Its `Cflags` carries the flags above with the include directory, and its `Libs` GAOL itself and the link option above, where the compiler that built GAOL accepts it. The variable `PKG_CONFIG_PATH` names the directory to pkg-config:

```
% export PKG_CONFIG_PATH=prefix/lib/pkgconfig
% c++ -std=c++11 -O2 $(pkg-config --cflags gaol) program.cpp \
      $(pkg-config --libs gaol)
```

Built by CMake with its default options on x86-64 Linux, with GCC 15, GAOL gives, on one line:

```
% pkg-config --cflags --libs gaol
-Iprefix/lib/pkgconfig/../../include -frounding-math -fno-fast-math
-ffp-contract=off -msse2 -msse3 -mfma -Lprefix/lib/pkgconfig/../../lib -lgaol -mno-daz-ftz
```

The paths are written from the directory of `gaol.pc`, so that the installed files can be moved. The option `-mno-daz-ftz` comes after `-lgaol` where the compiler that built GAOL takes it, as GCC from 13 and Clang from 20 do: `gaol.pc` is written for that compiler, and a program linked by one that refuses the option, as Clang 18, a GCC before 11.4 or GCC 12.0 to 12.3, stops on it, and is to be linked without it. There is no `-lm`: GAOL still calls functions of the math library of the system (those of `fenv.h`, `sqrt()`, `floor()`, `fma()` where the processor has no such instruction...), which the C++ compiler links itself. [GAOL v5]

3.3.3 From meson

In a meson project, `dependency('gaol')` finds an installed GAOL through pkg-config, in a *prefix* of its own with `PKG_CONFIG_PATH` set as above. A meson project can also build GAOL for itself, as a subproject, the sources of GAOL being in its `subprojects/gaol`: [GAOL v5]

Example

```
gaol_dep = dependency('gaol', fallback: ['gaol', 'gaol_dep'])
executable('program', 'program.cpp', dependencies: gaol_dep)
```

`dependency()` takes an installed GAOL where it finds one, and builds the subproject otherwise; `subproject('gaol').get_variable('gaol_dep')` always builds it. `gaol_dep` carries the include directory, the flags above and the link option above, as `gaol.pc` does: the flags that the `meson.build` of GAOL gives its own targets do not reach those of the project. `tests/meson_subproject` is a project building GAOL this way. GAOL could not be a subproject before GAOL v5: meson refuses the `add_global_arguments()` its `meson.build` called in a subproject.

GAOL is built in `release`, as CMake and `configure` build it brought into another project, where the project leaves the build type to its default, `debug`. meson 1.8.4 and later give a subproject its own default build type, and a `--buildtype=debug` asked for builds GAOL for debugging. Earlier ones give it the build type of the project, which would have built GAOL without optimization, and before meson 1.8 with `GAOL_DEBUGGING`: there, where the options are those of `debug`, GAOL compiles its libraries as in `release` all the same, as it cannot tell the default from a `--buildtype=debug` asked for. `-Dgaol:enable-debug=true` (or `default_options: ['enable-debug=true']` in `subproject()` or `dependency()`) builds GAOL for debugging, with any meson. Two cases keep the build type of the project: Visual C++ with meson before 0.57, whose `debug` gives `/RTC1`, which `cl` refuses with `/O2`, and the Visual Studio backend of meson (`--backend=vs`), which takes the build type of the project for every target; the default backend, ninja, is not one. [GAOL v5]

3.4 The namespaces

The type `interval` of GAOL and its functions are in the namespace `gaol_core`. A program names them through one of two namespaces, which `gaol/gaol.h` declares and does not open: [GAOL v5]

- **`gaol`**, GAOL under its own names, which this manual uses, to stay compatible with GAOL 4: a program written for GAOL 4 keeps most of its names there (Chapter 15 lists those that are gone) and, but for the results that change, its results;
- **`gaol_ieee1788`**, GAOL under the names of IEEE 1788-2015 (Chapter 10), to follow the standard: its operations have the names, the arguments and the results the standard gives them.

The two namespaces hold the same type and compute with the same functions. The main difference between them is the power, `pow` and `pown` (see Section 3.5, page 70), and the names that `textToInterval` reads. A program that opens `gaol` writes the operations under their names:

Example

```
1 #include <gaol/gaol.h>
2 using namespace gaol;
3
4 interval x(1, 2);
5 interval y = pow(x, 3) + sin(x);
```

`gaol/gaol.h` no longer opens `gaol` itself, as GAOL 4 did: a program that relied on it adds `using namespace gaol;`, or names `gaol::interval`. [GAOL v5]

The two namespaces hold `interval`, and the functions that have the same name and the same meaning in both (`sin`, `exp`, `sqrt`, `min`...) are the same functions, those of `gaol_core`. `pow` and `textToInterval` are the ones whose meaning differs: each namespace has its own, and none is in `gaol_core`, the namespace where argument-dependent lookup looks for a call `pow(x, y)` on an interval, so that each namespace finds its own only. A program opens one of the two namespaces, not both: with both open, `pow(x, y)` is ambiguous, and so is `textToInterval(s)`, which reads the names of GAOL in `gaol` and those of the standard in `gaol_ieee1788` (see Section 5.1, page 117).

`gaol` names the type and the functions of `gaol_core` one by one. [GAOL v5] They are the names GAOL 4 had in `gaol` that GAOL v5 still names there (the class `interval`, the expressions with their nodes and visitor, the functions and the operators of intervals, the functions of doubles and the rounding direction, `round_downward()`, `round_upward()`, `round_nearest()`, `previous_float()`, `next_float()`, `is_finite()`, the version of the library, `version_major`, `version_minor`, `version_micro` and `version`, and the doubles around π and $\pi/2$ of

Section 3.2, page 63), the functions on intervals GAOL v5 adds (`exp2`, `log2`, `hypot`, `compoundm1`, `trunc`, `sinpi`, `erf`, `fma`, `cancel_minus`, `round_ties_to_even`... and the nodes of their expressions), `restore_rounding()` and `exact_string()`. `gaol_ieee1788` names the version as `gaol` does.
Some names are in `gaol_core` only, and a program names them there: `gaol_core::rnd_keep`, for instance. [GAOL v5] They are `rnd_keep()` (GAOL v5 adds it; the releases 4.3.0 to 4.3.2 had it in `gaol`, Chapter 15; GAOL 4 has none), `NaN_val`, `eval_stack`, and the reader of strings, `parse_interval()` and `parsing_names`.

What the code of GAOL uses for itself is in `gaol_core` only too, out of the way of the names of a program that opens `gaol`: the functions saving and setting the rounding direction and the flush-to-zero modes (`rounding_state`, `get_rounding()`, `set_rounding()`, and `round_upward_if_needed()` and `gaol_set_rounding_x86()`, which the releases 4.3.0 to 4.3.2 had in `gaol` and GAOL 4 had not, see Chapter 15; and `get_flush_modes()`...), the functions computing the `pow` of `gaol` (below), and the internals GAOL 4 had in `gaol` without its manual documenting them: the functions `gaol_signbit()`, `modulo_k_pi()` and the `round_*_sse()`, the types `uintdouble`, `ullidouble`, `Interval_struct` and `prec_t`, and the variable `the_null_expr`. A program calling one of them names it in `gaol_core`: `gaol_core::get_rounding()`, for instance.

The `pow` of `gaol` takes the integer power for an integer exponent, a negative base included, and the `pow` of IEEE 1788-2015 for any other exponent (see Section 7.2.2, page 151). In `gaol_core`, its versions are named apart: `gaol_pown()`, `gaol_uipow()`, `gaol_pown_ll()`, `gaol_pow_real()`, `gaol_pow_hybrid()`, `gaol_pown_exp()` and `gaol_pow_exp()`, which are not in `gaol`.

3.5 The powers in the two namespaces

The power is where `gaol` and `gaol_ieee1788` differ. [GAOL v5] IEEE 1788-2015 has two powers (Table 9.1): $\mathsf{pown}(x, n)$, the integer power, for an integer n, defined for every x but 0 when $n < 0$, x^0 being 1; and $\mathsf{pow}(x, y)$, the real power $e^{y \log x}$, for an interval y, defined for $x > 0$, and for $x = 0$ when $y > 0$. GAOL 4 had a single `pow`, which took the power of a negative base, and `gaol` keeps it: its `pow` is a hybrid of the two.

- In `gaol_ieee1788`, `pown(x, n)` is the integer power, for an n of any integer type, and `pow(x, y)` the real power: only the part of x in $[0, +\infty]$ counts, whatever y, and `pow(x, p)` with a number p is `pow(x, [p])`, an integer p included, as the standard gives no other meaning to it.
- In `gaol`, `pow(x, y)` takes the integer power where the exponent is an integer: an integer type, a double that is an integer, or a degenerate interval $[n, n]$ holding an integer, beyond the `int`s too; any other exponent takes the real power of the standard. There is no `pown` in `gaol`.

TABLE 3.1 – The power of `gaol` and of `gaol_ieee1788` on some arguments

	`gaol`	`gaol_ieee1788`
`pow([−4, −1], 2)`	[1, 16]	∅ (`pown`: [1, 16])
`pow([−2, 3], 3)`	[−8, 27]	[0, 27]
`pow([0, 0], 0)`	[1, 1]	∅ (`pown`: [1, 1])
`pow([−2, 2], -1)`	[−∞, +∞]	[0.5, +∞]
`pow([−2, −1], [10^10, 10^10])`	[1, +∞]	∅
`pow([−1, 4], 0.5)`	[0, 2]	[0, 2]
`pow([0, 2], -2)`	[0.25, +∞]	[0.25, +∞]
`pow([−4, −1], [1.5, 2.5])`	∅	∅

Table 3.1 shows the two on some arguments. They agree on every exponent that holds no integer alone, and on $x \subseteq [0, +\infty]$ but for 0^0; they differ where x has a negative part and the exponent is an integer, and at 0^0. The `pow` of `gaol` is thus not monotone for the inclusion of its exponent: `pow([−4, −1], [2, 2])` is [1, 16], and `pow([−4, −1], [1.5, 2.5])`, whose exponent holds 2, is empty; the `pow` of the standard is. Both give the tightest bounds, but for the integer power: within the ints it comes from exact products, and is one double beyond the tightest where it is within twice the bound of the error of its products, never for the square; beyond the ints it comes from the `pow` of CORE-MATH, and is the tightest for an exponent that is a double and within three doubles of it otherwise, beyond 2^{53} (Chapter 11). A degenerate integer exponent of `gaol_ieee1788::pow` takes the integer power too, with its bounds: `pow([0x1.fffffffffffffp+1], [7])` has its lower bound one double below the tightest.

The exponent 2 is the square of the intervals: `pow(x, 2)`, `pow(x, [2])`, `gaol_ieee1788::pown(x, 2)` and, on the part of x in $[0, +\infty]$, `gaol_ieee1788::pow(x, [2])` are `sqr(x)` itself, with its bounds and the sign of its zero bounds, where the integer power of the earlier versions of GAOL v5 made its square from exact products, the same bounds in a longer time and, for some intervals holding 0, a zero bound of the other sign. [GAOL v5] The exception flags it raises are those of `sqr(x)` too. For the empty set, `pow(x, 2)`, `pow(x, 2u)`, `pow(x, 2LL)`, `pow(x, 2.0)`, `pow(x, [2])`, `gaol_ieee1788::pown(x, 2)` and `gaol_ieee1788::pow(x, [2])` return as `sqr` does, before any check, and leave the rounding direction as the program had it, and the modes that flush the subnormal numbers to zero as it had them too; so do the real and interval forms of `pow` for any exponent, an empty exponent included, and the `long long` form beyond the unsigned ints. [GAOL v5] The integer powers of an `int` or of an `unsigned int` other than 2 check the direction first and leave it upward: a program does not count on the direction after an operation that computes no bound (see Section 4.1, page 88).

The rest of GAOL follows the same split. The expressions `pow(e, n)` and `pow(e1, e2)` of `gaol` are the hybrid power, `pown(e, n)` and `pow(e1, e2)` of `gaol_`

`ieee1788` the two powers of the standard. The text `"pow([-4,-1],2)"` is read as [1, 16] by the `textToInterval` of `gaol`, and as the empty set by that of `gaol_ieee1788`, which reads `pow` as the real power and `"pown([-4,-1],2)"` as the integer power, a name `gaol` does not read. The reverse functions are those of the standard in both namespaces: `pow_rel` and `pow_exponent_rel` (powRev1 and powRev2) reverse the real power, and `nth_root_rel` (pownRev) the integer power, for every integer exponent, where it took $n \geqslant 1$ only. [GAOL v5]

A program written for GAOL 4, or for IBEX and Codac, whose intervals take the `pow` of GAOL, opens `gaol`; a program that wants the results of the standard, or compares GAOL with other libraries of IEEE 1788-2015, as the comparison of Section 12.5, page 237 does, opens `gaol_ieee1788`.

Example 3.5.1 *The three powers of* $[-4, -1]$ *with the exponent 2, named in their namespaces.*

Example

```
#include <iostream>
#include <gaol/gaol.h>

int main(void)
{
  gaol::interval x(-4, -1);
  std::cout << gaol::pow(x, 2) << ' '
            << gaol_ieee1788::pow(x, 2) << ' '
            << gaol_ieee1788::pown(x, 2) << std::endl;
  gaol::cleanup();
  return 0;
}
➥ [1, 16] [empty] [1, 16]
```

3.6 Integers

An integer given to GAOL is taken as the integer it is. [GAOL v5] `interval(n)` is the tightest interval containing n: $[n, n]$ where n is a double, as every `int` is, and the two doubles around it otherwise. `interval(9007199254740993LL)`, $2^{53}+1$, is $[2^{53}, 2^{53}+2]$, where its conversion to a double gave $[2^{53}, 2^{53}]$ or $[2^{53}+2, 2^{53}+2]$, according to the rounding direction, neither of which contains it, and `interval(ULLONG_MAX)` contains $2^{64}-1$. The same holds for `interval(a, b)` with an integer bound, which is empty for $a > b$ compared as numbers (`interval(9007199254740993LL, 9007199254740992LL)` is empty, though both lie between the same two doubles), for the assignment `x = n` and `x |= n`, the arithmetic operators (`x + n`, `n * x`, `x += n`...), `set_contains(n)`, `expression(n)`, and `numsToInterval()` and `isMember()` in `gaol_ieee1788`; `x &= n` is `interval(n)` where `x` contains n, and the empty set otherwise. The relations and `set_strictly_contains(n)` compare n as the integer it is: `interval(9007199254740992.0) < 9007199254740993LL`, $[2^{53}, 2^{53}] < 2^{53}+1$, is `true`, though no double lies between them. The constructors are detailed in Section 5.1, page 117, and the relations in Section 6.5, page 138.

Every integer type is taken so, `bool` aside, whatever the rounding direction; an unscoped enumeration, and `__int128` outside the GNU modes of GCC and Clang with libstdc++, still convert to a double, and a `long double` bound next to an integer takes `interval(double, double)`. An integer that is a double gives the result the double gives, and for the types of 53 bits or less (`int`, `short`, `char`...) the same code. `interval(n)` and `expression(n)` are explicit, as their constructors from a double; `expression(n)` is `expression(double)` where n is a double.

`pow(x, n)` of `gaol` and `pown(x, n)` of `gaol_ieee1788` take an exponent of any integer type. In the earlier versions of GAOL v5, `pow(x, 5L)` did not compile, a `long` being as far from an `int` as from an `unsigned` and a `double`, and `pown(x, 3000000000u)` converted the exponent to a negative `int`. An exponent within the ints is taken as an `int`, one within the unsigned ints as an `unsigned`, and one beyond them as a `long long` (`gaol_pown_ll()`), whose power comes from the `pow` of CORE-MATH at the bounds of x, as for `gaol::pow(x, [n])` ($[-\infty, +\infty]$ for a nonempty x in the earlier versions of GAOL v5): `pow(interval(1 + std::ldexp(1.0, -30)), 1LL << 33)`, about e^8, is [2980.9579759367944, 2980.9579759367948]. Beyond 64 bits (an `__int128`, in the GNU modes of GCC and Clang with libstdc++), the exponent is taken as $\pm(2^{63} - 1)$ or $\pm(2^{63} - 2)$, of its sign and parity, the power of every double but 0, ± 1 and $\pm\infty$ being beyond the doubles there. The same holds for `nth_root(x, q)` and `rootn(x, q)`, whose order beyond the ints and the unsigned ints gives an enclosure of the roots, computed by `pow` with the exponent $1/q$ ($x^{1/q}$ is within 2^{-21} of 1 for every double $x > 0$ there), for `pownRev(c, x, p)`, which gives the hull of the parts of x in these roots and, for an even p, in their opposites (for a negative p, c is cut at 0 first), and for `pow(e, n)`, `pown(e, n)` and `nth_root(e, n)` of the expressions. In the earlier versions of GAOL v5, `nth_root(x, 3L)` was ambiguous; `rootn(x, 3000000000u)` and `pownRev(c, 3000000000u)` converted the order to a negative `int`, the first giving a wrong interval ([0.99999999839, 0.99999999946] for [2, 8], rather than [1.00000000023, 1.00000000069]) and the second the reverse of a negative power; `rootn(x, 5000000001LL)` and the powers of expressions cut a `long` to an `int`; and `nth_root(e, -2)` took the order 4294967294. Beyond the ints, `pow(e, n)` and `pown(e, n)` of the expressions compute the `pow` of GAOL with the exponent $[n, n]$ where n is a double, as every n up to 2^{53} is, and give the power of the interval; for an n that is no double, no node holding a wider integer, they compute the product of x^h and x^l, $n = h + l$, h a double and $|l| < 2^{11}$, which is wider than the power of the interval where e holds 0 inside; beyond the unsigned ints, `nth_root(e, n)` is the root of the interval but where e contains 0, where it is wider.

3.7 A result thrown away

The functions of GAOL do not change the interval they are given: they return another one. `sqrt(x);` alone leaves `x` as it was, and so do `x.mid();` and

`x.emptyset();`, a static function that returns the empty set. The functions whose result is all they do carry an attribute that makes the compiler warn about such a call: [GAOL v5] the functions and operators on intervals, their predicates, the names of `gaol_ieee1788` and the functions building expressions.

Example

```
sqrt(x);         // warning: the result is thrown away, x is unchanged
x = sqrt(x);     // what was meant
(void)sqrt(x);   // thrown away on purpose: no warning, but with GCC before 7
```

The attribute is `GAOL_NODISCARD`, of `gaol/gaol_config.h`. It is `[[nodiscard]]` in C++17 and later. A program is compiled in the standard its project sets, or else in the one of its compiler, which is C++14 for GCC 6 to 10, Clang 6 to 15 and Visual C++. Before C++17, `GAOL_NODISCARD` is `[[nodiscard]]` too where the compiler takes it there: GCC 7 and later, and Visual C++ 2019 16.4 and later, whose warning is C4834. Clang, which warns about `[[nodiscard]]` before C++17 with `-pedantic`, and GCC before 7 take `__attribute__((warn_unused_result))`, whose warning is `-Wunused-result` (on by default); older Visual C++ takes `_Check_return_` of `<sal.h>`, which only the code analysis reports (`/analyze`, warning C6031), not the compiler. A cast to void (`(void)sqrt(x);`) silences each of them, but the attribute of GCC before 7: the result is kept in a variable there. A program that does not want the warnings defines `GAOL_NODISCARD` empty before including GAOL (`-DGAOL_NODISCARD=`). The compound assignments (`x += y`), which do change `x`, do not carry it.

3.8 Tools of interval algorithms

The algorithms of interval computing, the enclosure of a range by subdivision, the interval Newton method, the contractors, the branch and bound, are written with a few operations beyond the arithmetic, which they otherwise write for themselves. GAOL v5 provides them. [GAOL v5]

- **The width.** `x.width_enclosure()` is the interval `interval(r) - interval(l)` enclosing the exact width $r - l$ of $x = [l, r]$, whose upper bound is `width()`, the width rounded upward; it is [MAX, $+\infty$] for an unbounded x, as `textToInterval()` reads `inf`, and the empty set for the empty set.
- **The inflation.** `x.inflate(r)` is x widened by the absolute radius r, `x + interval(-r, r)`, each bound rounded outward; $r = +\infty$ gives the whole line, and $r < 0$ or a NaN r the empty set.
- **A midpoint and a radius.** `interval::midrad(m, r)` is the interval of midpoint m and radius r, `interval(m) + interval(-r, r)`, which encloses $[m - r, m + r]$, where `interval(m - r, m + r)` does not: both bounds are then computed in the upward rounding of GAOL, the lower one above $m - r$ where it is inexact. An infinite m gives the empty set, as `interval(m)` does.

- **The bisection.** `x.bisect(ratio)` is the pair of the parts of x on either side of a cut point at the given ratio of its width, $0 < \mathit{ratio} < 1$, which share the cut point and cover x; `bisect(0.5)` cuts where `split()` does, at `midpoint()`, and so does every ratio for a half-line or the whole line. A ratio outside $(0, 1)$, or NaN, throws `std::domain_error`; the cut of finite bounds raises no overflow exception, whatever their magnitude. `x.is_bisectable()` tells whether a double lies strictly between the bounds, both parts being then smaller than x: not for two consecutive doubles, nor for [MAX, $+\infty$] (see Section 8.3, page 179).
- **The hull and the intersection.** `hull(x, y)` and `intersect(x, y)` are `x | y` and `x & y` as functions, in `gaol` as in `gaol_core`.

Example 3.8.1 *The program below uses these tools on an enclosure of* 0.1*, on the interval of midpoint* 1 *and radius* 10^{-20}*, and on* [0, 1] *cut at a quarter of its width.*

Example

```
#include <iostream>
#include <gaol/gaol.h>
using namespace gaol;

int main()
{
  const interval x = textToInterval("0.1");        // encloses 0.1
  const interval y = interval::midrad(1.0, 1e-20); // [1 - 1e-20, 1 + 1e-20]
  const std::pair<interval, interval> parts = interval(0.0, 1.0).bisect(0.25);
  std::cout << x.width_enclosure() << ' ' << y << ' ' << parts.first << ' '
            << parts.second << ' ' << hull(x, y) << std::endl;
  gaol::cleanup();
  return 0;
}
```

The reverse functions. A contractor shrinks a box without losing a solution of a constraint: it evaluates the expression forward, from the variables to the result, and goes backward, each node being intersected with what the others allow. The backward operations are the relational functions of GAOL, the `*_rel` functions, which reverse each operation (see Section 7.3, page 160). Their arguments are in one order, the result first and the operand contracted last: `div_rel(c, b, x)` contracts a factor of $z = x \cdot y$, and `sqrt_rel(c, x)` the argument of a square root. The reverse functions of the power follow it: `pow_rel(K, J, I)` is the hull of the x of I with an x^y in K for a y of J, and `pow_exponent_rel(K, H, I)` the hull of the y of I with an x^y in K for an x of H; they are powRev1(b, c, x) and powRev2(a, c, x) of IEEE 1788-2015 (Table 10.1), as `pow_rel(c, b, x)` and `pow_exponent_rel(c, a, x)`, x being $[-\infty, +\infty]$ when left out. [GAOL v5] Both reverse the `pow` of the standard, that of `gaol_ieee1788::pow`, in either namespace: x^y is $e^{y \log x}$ for $x > 0$, 0 for $x = 0$ and $y > 0$, and has no value elsewhere, so that no $x < 0$ is a solution, and `pow_rel([1, 4], [2], [-3, 3])` is [1, 2]. The `pow` of `gaol` takes the integer power for

a degenerate integer exponent $[n, n]$, which has negative bases too: it is not one function of x and y, and the reverse of x^n is `nth_root_rel(K, n, I)`, which takes every integer n (see Section 7.3, page 160).

`max_rel(z, y, x)`, `min_rel(z, y, x)`, `sign_rel(z, x)` and `floor_rel(z, x)` contract x under the constraints $z = \max(x, y)$, $z = \min(x, y)$, $z = \operatorname{sign}(x)$ and $z = \lfloor x \rfloor$. [GAOL v5] Each returns the hull of the x of `x` that the function sends into `z`, with some y of `y` for `max` and `min`: the reverse functions IEEE 1788-2015 defines for every function (10.5.4), with the arguments in the order of the relational functions of GAOL. `max` and `min` being symmetric, `max_rel(z, x, y)` contracts y, and `max(x, y) & z` contracts z. IEEE 1788-2015 names and requires only the reverse functions of its Table 10.1, which these four are not in: `gaol_ieee1788` has no name for them, and a program that opens it calls them as `max_rel(z, y, x)` all the same, argument-dependent lookup finding them in `gaol_core` (see Section 3.4, page 69).

Example

```
interval x(-5.0, 5.0), z(1.0, 2.0);
interval a = max_rel(z, interval(0.0, 1.0), x);   // [-5, 2]: y meets z
interval b = max_rel(z, interval(-1.0, 0.0), x);  // [1, 2]: y below z
interval c = max_rel(z, interval(3.0, 4.0), x);   // empty: y above z
interval s = sign_rel(interval(-1.0), x);         // [-5, 0]
interval f = floor_rel(interval(0.0, 2.0), x);    // [0, 3]
```

The interval Newton method encloses the roots of f in X with the relational division, which solves $(x - m)\, d = -f(m)$ for d in the enclosure of $f'(X)$ where the division `/` does not (Section 3.10, page 77, and `examples/05_interval_newton.cpp`); dividing by the negative and by the positive part of $f'(X)$ apart, two `div_rel`, or `gaol_ieee1788::mulRevToPair(b, c)`, keeps the gap around m. The examples 06 and 07 of `examples/` write a branch and bound and a contractor; they cut their boxes with the `bisect()` of `examples/box.h`, at 0.45 of the width, and 07 contracts with `div_rel()`, `sqrt_rel()` and `asin_rel()`.

3.9 Errors, in short

GAOL reports an error by throwing an exception of the standard library (`<stdexcept>`), where GAOL 4 had classes of its own: [GAOL v5] `std::invalid_argument` for a string that is no interval, given to `gaol::textToInterval()` or read by `operator>>`, and `std::domain_error` for a function called with an argument it does not take, `nb_fp_numbers()` with a NaN, an infinity or $a > b$, `bisect()` with a ratio outside $(0, 1)$, or `nth_root(8, 1.5)` in a string. Both derive from `std::logic_error`, which derives from `std::exception`, so that a handler that knows nothing of GAOL says what went wrong: its `what()` is *`file:line: explanation`*, the name of the source file of GAOL without its directories and the line that threw, then what went wrong. Where memory runs out, GAOL throws the

`std::bad_alloc` of C++, the reader of strings included. A build without `GAOL_EXCEPTIONS_ENABLED` prints a message and aborts instead of throwing; the three builds can be asked to do so (a setting of the developers). The classes of GAOL 4, what became of each, the messages, the warnings and the builds that abort are detailed in Section 13.5.1, page 266.

3.10 Common errors

This section reviews the errors most often made with GAOL, so that the user recognizes them in a program. Most of them come from the two facts that a double is not an interval, and that an interval is not a number.

3.10.1 Floating-point arithmetic and rounding

The constant `0.1` written in a program is rounded when compiling, to the nearest double $\updownarrow 0.1 \updownarrow$, so that `interval one_tenth(0.1)` is a degenerate interval that does not contain 0.1, and the containment property (see Section 1.3, page 9) is violated (Example 1.1.4). The right way to deal with a constant that might not be a double is to write it as text and to read it with `gaol::textToInterval()` (see Section 5.1, page 117), so that it is correctly rounded downward and upward at run time, as in Example 1.2.4: `one_tenth` is then the smallest floating-point interval enclosing 0.1, a canonical interval (see Section 1.2, page 7). Written in hexadecimal (see Section 9.2.2, page 191), which shows the bounds exactly, the two intervals are:

Output

```
interval(0.1) = [0x1.999999999999ap-4]
one_tenth     = [0x1.9999999999999p-4, 0x1.999999999999ap-4]
```

3.10.2 Functions of numbers

A function called with a number is the function of the C library, even where the namespace `gaol` is open: `sqrt(2.0)` is a `double`, rounded in the direction in effect, and no enclosure of $\sqrt{2}$. A `double` no longer becomes an interval by itself, the constructor being explicit, so that `interval r = sqrt(2.0);` does not compile; but `interval r(sqrt(2.0));` does, and gives a point that does not contain $\sqrt{2}$. The enclosure is the function of GAOL on an interval, or the text read by `textToInterval()`: [GAOL v5]

Example

```
1 interval root2 = textToInterval("sqrt(2)");
2 cout << boolalpha << interval(sqrt(2.0)).set_contains(root2) << ' '
3      << sqrt(interval(2.0)).set_contains(root2);
4 ➥ false true
```

For the same reason, `gaol::sin(0.5)` does not compile: it is written `gaol::sin(interval(0.5))`, and `sin(0.5)` alone is the `sin` of C. Writing code inside `namespace gaol {}` has the same effect: the using-declarations of `gaol` are visible there, a call `sqrt(2.0)` finds `gaol::sqrt`, which hides `::sqrt`, and does not compile. With GAOL 4, whose `interval(double)` was implicit, the call compiled and computed `gaol::sqrt(interval(2.0))`.

3.10.3 Midpoint and radius

`interval(m - r, m + r)`, with doubles, is not an interval enclosing $[m-r, m+r]$: the program computes `m - r` and `m + r` itself, in the direction in effect, which is upward until `cleanup()`, so that `m - r` may be above the real $m - r$, and to nearest each bound may be on either side. With `m = 1.0` and `r = 1e-20`, the interval is $[1, 1+2^{-52}]$ while the direction is upward, and does not contain $1-10^{-20}$. `interval::midrad(m, r)` (see Section 5.1, page 117), or `m + interval(-r, r)`, computes both bounds in intervals, and gives $[1-2^{-53}, 1+2^{-52}]$, which does. [GAOL v5]

Example

```
1 double m = 1.0, r = 1e-20;
2 interval wrong(m - r, m + r);  // Does not enclose [m - r, m + r]
3 interval right = interval::midrad(m, r);  // Does
```

3.10.4 The doubles of the program

The rounding direction that GAOL leaves applies to the rest of the program, which computes its own doubles in it: after an operation of GAOL, `1.0 / 3.0` is the double above 1/3, and `printf()` and the streams of the GNU C library write the decimal digits of a double rounded upward too, until `cleanup()`. Section 4.2, page 90 gives what changes, and what to do. [GAOL v5]

3.10.5 Zero

`interval x(0, 0)`, `interval x(0)` and `interval::zero()` are $[0, 0]$. The literal `0` is also a null pointer, and the constructors from one and from two strings of GAOL 4 made these calls ambiguous; GAOL v5 has no constructor from strings. [GAOL v5]

3.10.6 The sign of a zero bound

A bound that is zero has a sign, which differs between the builds: `sqr(interval(-1, 2))` is $[-0, 4]$ with the SSE2 intervals, which the default build uses on x86 processors, and with the NEON intervals, which it uses on 64-bit ARM processors, and $[0, 4]$ with the FPU intervals, those of the other processors and of the builds with `GAOL_SIMD` off (Visual C++ and clang-cl have the SSE2 intervals on x86 and on x64); [GAOL v5] `interval::zero()` is $[-0, 0]$ with the first two and $[0, 0]$ with the third. `pow(x, 2)` is `sqr(x)`, and has the same sign. [GAOL v5] The NEON and the

SSE2 intervals can differ too: the maximum of 64-bit ARM, which the product takes of its four products, ranks +0 above −0, where the one of x86 returns its second operand, so that the lower bound of `interval(-1, 0) * interval(-1, 0)` is −0 with the NEON intervals and +0 with the SSE2 ones (see Section 4.5, page 96); the sign of a zero bound of a product of the FPU intervals is the one the compiler gives (see Section 4.6, page 98). IEEE 1788-2015 and GAOL take the two zeros for the same number: `x.left() == 0.0` is `true` whichever it is, and a point interval at zero is written `[0]`. A program that reads the sign, with `std::signbit(x.left())` or `1.0 / x.left()`, which is $-\infty$ or $+\infty$, gets one answer with a build and the other with the other: it compares the bounds with `==`, `<` and `<=`. [GAOL v5]

3.10.7 The empty set and the domain of a function

A function is computed on the part of its argument that lies in its domain (IEEE 1788-2015), and GAOL, which has no decorations, says nothing of the rest: `sqrt(interval(-1, 4))` is [0, 2], `log(interval(-1, 0))` is empty, and so are `interval(1, 0)` and the interval of one infinite or NaN double. The empty set then passes the tests as if it were a good answer: it has no point that contradicts a certainly relation, and it is included in every interval, so that a box outside the domain of a function is both inside and outside the set a paving looks for:

Example

```
1 interval e = sqrt(interval(-2.0, -1.0)), z(0.0, 1.0);
2 cout << boolalpha << e.is_empty() << ' ' << (e < z) << ' '
3      << (e > z) << ' ' << z.set_contains(e) << ' '
4      << e.set_disjoint(z);
5 ➥ true true true true true
```

A program tests `is_empty()` first, and decides itself what a box outside the domain of its function means, as `examples/08_sivia.cpp` does. [GAOL v5]

3.10.8 Relations, sorting and containers

The symbols `<`, `<=`, `>` and `>=` are the certainly relations of IEEE 1788-2015, which do not tell which of two intervals is the greater, and there is no `==`: `interval(1,3) < interval(2,4)` and `interval(1,3) >= interval(2,4)` are both `false` (see Section 6.5, page 138). In C-XSC and in PROFIL/BIAS, `x <= y` means that `x` is a subset of `y`, and `r <= x` that the number `r` is in `x`: code ported from them compiles, and changes its meaning (see Section 3.11, page 83). None of them is an ordering for the standard library: `std::sort`, `std::set` and `std::max` take the comparator `gaol::lexicographic_less` (see Section 6.6, page 140). [GAOL v5]

3.10.9 Both namespaces

A program opens the namespace `gaol` or the namespace `gaol_ieee1788`, not both (see Section 3.4, page 69):

Example

```
#include <gaol/gaol.h>
using namespace gaol;
using namespace gaol_ieee1788;

gaol::interval x(1,2), y = pow(x,x); // Does not compile
```

The compiler reports that the call of `pow` is ambiguous. Not every call is: with an `int` or an `unsigned int` for exponent, `pow(x, 2)` compiles, and takes the `pow` of `gaol`, silently, where that of `gaol_ieee1788` gives another result for a negative base (see Section 3.5, page 70). Only the calls with another type of exponent, `pow(x, 2.0)`, `pow(x, 2L)` or `pow(x, y)`, are ambiguous: [GAOL v5]

Example

```
#include <iostream>
#include <gaol/gaol.h>
using namespace gaol;
using namespace gaol_ieee1788;

int main(void)
{
  gaol::interval x(-4,-1);
  std::cout << pow(x, 2) << std::endl; // The pow of gaol
  gaol::cleanup();
  return 0;
}
➥ [1, 16]
```

3.10.10 Bisection

A loop that cuts an interval until it is narrow, `while (x.width() > eps)`, never ends on an unbounded interval: `bisect(0.5)`, like `split()`, cuts $[1, +\infty]$ at `DBL_MAX`, and the half [DBL_MAX, $+\infty$] is cut at `DBL_MAX` again, its width staying infinite. What a loop stops on is `is_bisectable()`, which is `false` for this half, and for an interval of two consecutive doubles, the cut of which gives the interval itself as one half (see Section 8.3, page 179): [GAOL v5]

Example

```
interval x(1.0, GAOL_INFINITY);
x = x.bisect(0.5).second;
cout << boolalpha << x << ' ' << x.width() << ' '
     << x.is_bisectable();
➥ [1.797693134862315e+308, inf] inf false
```

3.10.11 Newton's method: the division and the relational division

The step $N(X) = m - f(m)/F'(X)$ of the interval Newton method divides with `/`, the set of the quotients a/b of a point of the numerator by a point other than

0 of the denominator. Where $0 \in F'(X)$ and $f(m) = 0$, the quotient $0/F'(X)$ is $[0, 0]$, so that $N(X) = m$ lies inside X and "proves" that X holds exactly one root, where it holds seven for $\sin x - x/10$ on $[-10, 10]$. A root satisfies a relation, $(x - m)\, d = -f(m)$ for some d in $F'(X)$, which `%`, or `div_rel()`, the reverse of the product (see Section 7.3, page 160), solves: it gives the whole line, rightly, when d and $f(m)$ can both be 0.

Example

```
interval fm(0.0), D(-1.1, 0.9);
cout << fm / D << ' ' << fm % D << ' '
     << div_rel(fm, D, interval(-5.0, 5.0));
➥ [0] [-inf, inf] [-5, 5]
```

The functional division is the right one where $F'(X)$ holds no 0. `examples/05_interval_newton.cpp` shows the trap and the extended step.

3.10.12 Reading text

`gaol::textToInterval()` throws `std::invalid_argument` for a text that is no interval, and `std::domain_error` for a text that is one but calls a function with an argument it does not take, such as `nth_root(8, 1.5)`, whose order is not an integer: a loop that catches the first lets the second escape. Both derive from `std::logic_error` (Chapter 13). [GAOL v5]

Example

```
try {
  cout << textToInterval("nth_root(8, 1.5)");
} catch (const std::invalid_argument&) {
  cout << "std::invalid_argument";
} catch (const std::domain_error&) {
  cout << "std::domain_error";
}
➥ std::domain_error
```

`gaol_ieee1788::textToInterval()` throws neither: it gives the empty set for every text that is no interval, as IEEE 1788-2015 has it, so that a malformed text cannot be told from `"[empty]"`:

Example

```
cout << gaol_ieee1788::textToInterval("[1, 2") << ' '
     << gaol_ieee1788::textToInterval("[empty]");
➥ [empty] [empty]
```

A null pointer is no text, and the `textToInterval()` of both namespaces and `exactToInterval()` throw `std::invalid_argument` for it: none of the three calls below reaches the reader.

Example

```
1 interval a = textToInterval(nullptr);
2 interval b = textToInterval(std::getenv("UNSET"));
3 interval c = gaol_ieee1788::exactToInterval(nullptr);
```

Without the overloads of a C string, which take the null pointer first, their argument would be a `const std::string&` built from the pointer, which the C++ standard leaves undefined (libstdc++ throws `std::logic_error`). [GAOL v5]

Two more texts deserve a word. A sign next to an uncertain number, `10-3.56?1` or `10+3.56?1`, throws `std::invalid_argument`: the lexer reads `-3.56?1` (and `+3.56?1`) as one token, and `10 -3.56?1` throws too. A blank is needed after the sign: `10 - 3.56?1`, or `10- 3.56?1`. Reading a long number takes a time quadratic in its number of digits (see Section 9.1, page 183): a text coming from elsewhere can thus block a program.

3.10.13 Threads and the global settings

The output format of intervals and their number of digits, set by `interval::format()` and `interval::precision()`, are variables of the program, shared by its threads, and not atomic: setting one while another thread writes an interval, with `operator<<`, the conversion to a `std::string` or `gaol_ieee1788::intervalToText()` (which reads the precision only), is a data race, with the undefined behaviour of C++ (ThreadSanitizer reports it). The program that sets them synchronizes with the threads that write: it sets them before starting its threads, or takes a lock of its own around the setting and around each output. Without a setting in between, several threads may write intervals at once, and read strings at once (see Section 9.1, page 183); `exact_string()` reads neither setting. [GAOL v5]

3.10.14 The flags of interval arithmetic

Compiled without the flags of interval arithmetic (see Section 3.3, page 64), the inline operations of GAOL may be evaluated by the compiler at compile time, rounded to nearest, or contracted into fused multiply-adds, and give bounds that do not enclose the exact results. `gaol/gaol_config.h` stops the compilation where it can tell:

Output

```
% c++ -ffast-math -c program.cpp
gaol/gaol_config.h: error: #error "GAOL cannot be
compiled with -ffast-math (nor -Ofast): the bounds it
computes would not enclose the exact results (remove
the option, or put the flags of gaol.pc and gaol::gaol
after it)"
```

It stops `-ffinite-math-only` in the same way, with which the compiler takes NaN, the bounds of the empty interval, and infinities never to occur (see Section 2.3.2,

page 25). Taking the flags from `gaol::gaol` or `gaol.pc` avoids the question. The link is not seen by the header: a program linked with `-Ofast` gets the flush-to-zero mode, which makes the bounds of the operations with subnormal operands and results wrong where GAOL does not clear it (see Section 4.8, page 104). [GAOL v5]

3.11 Coming from another library

The names of interval libraries differ, and the same symbol can mean another relation. Tables 3.2 and 3.3 give, for what an algorithm uses, the name of GAOL v5, the one of `gaol_ieee1788`, and those of IBEX and Codac, which share them, of C-XSC and of Boost.Interval. A dash means that the sources read have no function of the kind, not that the algorithm cannot be written. [GAOL v5]

TABLE 3.2 – The names of GAOL v5, of `gaol_ieee1788`, and of IBEX and Codac

	GAOL v5	`gaol_ieee1788`	IBEX, Codac
Lower bound	`x.left()`	`inf(x)`	`x.lb()`
Upper bound	`x.right()`	`sup(x)`	`x.ub()`
Midpoint, a double	`x.midpoint()`	`mid(x)`	`x.mid()`
Midpoint, an interval	`x.mid()`	—	—
Width	`x.width()`, rounded upward; `x.width_enclosure()`	`wid(x)`	`x.diam()`
Radius	`x.rad()`	`rad(x)`	`x.rad()`
Smallest, greatest absolute value	`x.mig()`, `x.mag()`	`mig(x)`, `mag(x)`	`x.mig()`, `x.mag()`
Hull, intersection	`x | y`, `x & y`; `hull(x, y)`, `intersect(x, y)`	`convexHull(x, y)`, `intersection(x, y)`	`x | y`, `x & y`
Is empty	`x.is_empty()`	`isEmpty(x)`	`x.is_empty()`
Subset	`x.set_leq(y)`	`subset(x, y)`	`x.is_subset(y)`
Interior	`x.set_le(y)`; `y.set_strictly_contains(x)`	`interior(x, y)`	`x.is_interior_subset(y)`
Contains a number	`x.set_contains(d)`	`isMember(d, x)`	`x.contains(d)`
Disjoint	`x.set_disjoint(y)`	`disjoint(x, y)`	`x.is_disjoint(y)`
Equal	`x.set_eq(y)`	`equal(x, y)`	`x == y`
Inflate by `r`	`x.inflate(r)`, a new interval	—	`x.inflate(r)`, which changes `x`
Bisect	`x.bisect(ratio)`	—	`x.bisect(ratio)`, ratio 0.5 (IBEX) or 0.49 (Codac)
Two-piece division	two `div_rel`, or `mulRevToPair(b, c)` of `gaol_ieee1788`	`mulRevToPair(b, c)`	`div2` (IBEX only)

TABLE 3.3 – The names of GAOL v5, of C-XSC and of Boost.Interval

	GAOL v5	C-XSC	Boost.Interval
Lower bound	`x.left()`	`Inf(x)`	`lower(x)`
Upper bound	`x.right()`	`Sup(x)`	`upper(x)`
Midpoint, a double	`x.midpoint()`	`mid(x)`	`median(x)`
Width, rounded upward	`x.width()`	`diam(x)`	`width(x)`
Radius	`x.rad()`	—	—
Smallest, greatest absolute value	`x.mig()`, `x.mag()`	`AbsMin(x)`, `AbsMax(x)`	`norm(x)`, the greatest only
Hull, intersection	`x | y`, `x & y`; `hull(x, y)`, `intersect(x, y)`	`x | y`, `x & y`	`hull(x, y)`, `intersect(x, y)`
Is empty	`x.is_empty()`	`IsEmpty(x)`	`empty(x)`
Subset	`x.set_leq(y)`	`x <= y`	`subset(x, y)`
Interior	`x.set_le(y)`	`x < y`, `in(x, y)`	—
Contains a number	`x.set_contains(d)`	`d <= x`, `in(d, x)`	`in(d, x)`
Disjoint	`x.set_disjoint(y)`	`Disjoint(x, y)`	`!overlap(x, y)`
Equal	`x.set_eq(y)`	`x == y`	`equal(x, y)`
Inflate by `r`	`x.inflate(r)`	`Blow(x, eps)`, relative to the width	`widen(x, r)`
Bisect	`x.bisect(ratio)`	—	`bisect(x)`, at the median only

`overlap(x, y)` of Boost.Interval is a boolean, true when x and y have a point in common: `!x.set_disjoint(y)` in GAOL v5. The `overlap` of GAOL v5 (`x.overlap(y)`, `gaol_ieee1788::overlap(x, y)`) is the function of IEEE 1788-2015 and returns an `overlapping_state`, which does not convert to `bool`: a Boost program that tests `overlap(x, y)` does not compile with it.

What the symbols `<`, `<=` and `==` mean is where code ported from another library compiles and changes meaning (Table 3.4).

TABLE 3.4 – The meaning of the symbols `<`, `<=` and `==` in each library

	`x < y`	`x <= y`	`x == y`
GAOL v5	every point of `x` is below every point of `y` (strictPrecedes)	every point of `x` is at most every point of `y` (precedes)	not defined: `x.set_eq(y)`
IBEX	not defined	not defined	the same bounds, the empty set equal to itself
Codac	a `BoolInterval`: true, false, unknown, or empty if an operand is	not defined	the same bounds, as IBEX
C-XSC	`x` in the interior of `y`	`x` is a subset of `y`; `r <= x`: the number `r` is in `x`	the same bounds

Table 3.4, continued

	`x < y`	`x <= y`	`x == y`
PROFIL/BIAS	`x` in the interior of `y`	`x` is a subset of `y`; `r <= x`: `r` is in `x`	the same bounds
Boost.Interval	true if every point of `x` is below every point of `y`, false if none is, else it throws `comparison_error`	true if every point of `x` is at most every point of `y`, false if none is, else it throws	true if `x` and `y` are the same point, false if they are disjoint, else it throws

So `0.0 <= f` is "0 is in `f`" in C-XSC and in PROFIL/BIAS, and in GAOL v5 the certainly relation "0 is at most every point of `f`": a program ported from them compiles and no longer tests what it did (`f.set_contains(0.0)` does). A Boost.Interval program that catches `comparison_error` to detect an uncertain comparison never sees it with GAOL v5, whose `x < y` is false for overlapping intervals, and the empty set, which Boost refuses, passes every certainly relation (see Section 6.6, page 140). The names are those of IBEX 2.9.1 (`ibex_Interval.h`), Codac 2 (`codac2_Interval.h`), C-XSC 2.5.4 (`interval.hpp` and `interval.inl`), PROFIL/BIAS 2.0.8 (`Interval.h`) and Boost 1.92.0 (`boost/numeric/interval/`), read in their sources.

3.12 Discussion

This chapter has shown what a program using GAOL looks like, and what it owes to the library: one header, the flags of interval arithmetic on every file that includes it, which CMake, pkg-config and meson pass on, and a call to `gaol::cleanup()` after the last use. The flags are the price of inline operations: a compiler left free to round to nearest, to fold a constant or to fuse a product and a sum would break the enclosure, and `gaol/gaol_config.h` refuses the options it can see, not the absence of those it cannot. The two namespaces are the answer to two audiences, the programs written for GAOL 4, IBEX and Codac, and those written to IEEE 1788-2015: the same type and the same functions, under two sets of names, with the power as their main difference, and `textToInterval`, which throws in `gaol` and gives the empty set in `gaol_ieee1788`.

Most of the common errors reviewed here come from the same source: a double is not an interval, and an interval is not a number. A decimal constant, a function of a number, a midpoint and a radius, the sign of a zero bound, the empty set that passes every test, the relations that order nothing, are each a place where the habits of floating-point programming give a wrong program that compiles. The tables of names of the other libraries serve the same purpose for a program ported to GAOL.

The next chapter looks at what this one took for granted: the rounding direction that each operation sets, its effect on the rest of the program, and the initialization and the cleanup of GAOL.

Chapter 4

The rounding direction and the floating-point environment

An interval library computes its bounds with the rounding direction of the processor set outward, and that direction is a state of the processor which the whole program shares. How the library takes charge of this state, and what it leaves to the program, decides both the correctness of the bounds and the behaviour of every double the program computes beside its intervals. GAOL 4 either trusted the user with the direction or restored it after each operation, according to its build. GAOL v5 sets it itself, in each operation, and the choice has consequences the user needs to know: the doubles of the program are rounded upward while GAOL is initialized, the C library converts its numbers in that direction, and the compiler restores nothing.

The rounding direction is not the only state of the floating-point environment that matters to the bounds. The modes that flush the subnormal numbers to zero, which a program linked with `-Ofast` gets without asking, make the bounds of some operations wrong, and the floating-point exceptions, when a program enables them, stop the program in the middle of an operation of GAOL. The chapter describes the three, and how GAOL defends its bounds against them.

In this chapter, we describe the rounding direction as GAOL v5 handles it, and what it does to the rest of the program. The first section states how each operation sets the direction, and which operations leave it alone. The second section shows what the upward direction does to the doubles of the program, and what a program does about it; the third describes the build that preserves the direction of the program. The fourth, fifth and sixth sections describe the AVX-512 path, which carries its rounding direction in its instructions, and the NEON and the FPU intervals; the seventh describes the control registers of the x86 processors. The following two sections treat the flush-to-zero modes and the floating-point exceptions. The chapter ends with the reference of the initialization and the cleanup of GAOL, and of its rounding functions. The design of the kernels, which instructions compute the bounds and what a write of the control register costs on each processor, is the

subject of Chapter 12 (see Section 12.1, page 228); the options of the builds are those of Chapter 2 (see Section 2.5.2, page 36).

4.1 The rounding direction

Floating-point interval arithmetic requires *outward rounding* to fulfil the containment property (see Section 1.3, page 9): the sum of $[a, b]$ and $[c, d]$ is computed as $[\downarrow a + c\downarrow, \uparrow b + d\uparrow]$, where $\downarrow r\downarrow$ is the greatest double of $\mathbb{F}$ not above the real r and $\uparrow r\uparrow$ the smallest one not below it (see Section 1.1, page 6). The processor performs these two roundings when its rounding direction is set toward $-\infty$ and toward $+\infty$ respectively.

On most platforms, switching the rounding direction is costly. Proposition 1.3.3 (see Section 1.3, page 9) cuts down the number of switches: nearly every downward rounding is replaced by an upward one, by negating twice the operation performed. GAOL stores the lower bound of an interval negated, and computes both bounds rounding upward: the rounding direction is set toward $+\infty$ once, and the other directions are needed only where a computation asks for them, as `midpoint()`, `chi()`, and the writing and the reading of an interval do, to nearest, and the writing of a bound that the C library rounds does, downward for a lower bound (see Section 12.1, page 228).

GAOL 4 had two modes. In its *trust rounding mode*, which its manual gives as the default, it left the rounding direction to the user, who had to set it toward $+\infty$ before any computation on intervals; built with `--enable-preserve-rounding`, which its `configure` and its meson build in fact turn on by default, each operation set the direction upward and restored the one it found, on x86-64 in the x87 control word only (see Section 15.3.2, page 320). GAOL v5 takes charge of the direction. [GAOL v5] Each operation that computes a bound checks the direction, and sets it upward when it is not, whatever the build; the operations of the AVX-512 path, which carry their direction in the instruction, are the exception (see Section 4.4, page 94). The direction is then left upward, unless GAOL is built to preserve it (see Section 4.3, page 93). The bounds are therefore right whatever rounding direction the calling code left.

On most processors, the check is an addition, $1+2^{-60}$ or one of its variants below, which is above 1 only when rounded upward, rather than a reading of the rounding direction, which costs far more on some platforms: under Rosetta 2, reading it made an addition of intervals take 800 ns rather than 2.8 ns, and a call of `fegetround()` cost about 80 ns with 32-bit Visual C++; the NEON intervals and the 32-bit x86 processors read the control registers themselves, which is cheap there. The check takes four forms, by processor:

- on x86-64 processors, and on ARM processors with the FPU intervals and GCC or Clang, it is $1 + (2^{-1060} + 0)$, which shows the modes that flush the subnormal numbers to zero as well (see Section 4.8, page 104);

- with the NEON intervals of 64-bit ARM processors (see Section 4.5, page 96), it is a read of the control register FPCR, which holds both the direction and these modes;
- on the other processors (POWER, s390x, RISC-V), and with the FPU intervals of Visual C++ for 64-bit ARM, it is $1 + 2^{-60}$, which does not see the modes that flush the subnormal numbers to zero (see Section 4.8, page 104);
- on 32-bit x86 processors, whose two units that round, the x87 unit and the SSE instructions, each have a direction, it is a read of the control word of the first and of MXCSR, with `fnstcw` and `stmxcsr` (see Section 4.7, page 103). [GAOL v5]

Each function of intervals checks the direction once, and the functions it calls on doubles take it to be upward already. The cost of the check is measured in Section 4.8, and the cost of a write of the control register on each processor in Chapter 12 (see Section 12.3, page 233).

Unless GAOL is built to preserve it, the rounding direction stays upward from the initialization of GAOL on, for the whole program, and `cleanup()` sets back the one that the initialization found: every program calls `gaol::cleanup()` right after its last use of GAOL (see Section 4.10, page 112). [GAOL v5] The direction set back is the one the first `init()` found when GAOL initialized itself: to nearest, as a program starts; on x86, for the x87 unit and for the SSE instructions each; in the thread calling `cleanup()`. The rest of the floating-point environment is left as it is, the exception flags raised in particular.

Some operations set no direction, and leave the one they found, with or without `GAOL_PRESERVE_ROUNDING`: a program that wants a direction after one of them sets it itself. [GAOL v5]

- Those that round nothing and make no check: the negation and the unary plus, `abs()`, `min()`, `max()`, `sign()`, the roundings to an integer (`floor()`, `ceil()`, `trunc()`, `integer()`, `round_ties_to_even()`...), `&` and `|`, `hull()` and `intersect()`, `invabs_rel()`, `mig()` and `mag()`, `nth_root(x, 0)` and `nth_root(x, 1)`, the constructors from bounds, the access to the bounds, `is_empty()`, the relations and the other predicates, `exact_string()`.
- Those that return before their check. Some return the empty set at once for an empty operand, and some do not: `y * empty` and `exp(empty)` leave the direction, where `empty + y` and `sqrt(empty)` set it upward, so that a program counts on neither. And `hausdorff(x, y)` returns $+\infty$ when a bound is infinite in one of the two intervals only, where it sets the direction upward otherwise.
- The operations of the AVX-512 path, which set none (see Section 4.4, page 94).

4.2 What the upward rounding does to the program

The rounding direction is the processor's, and the whole program shares it. The results of GAOL do not depend on it, since each operation sets it upward when it is not, in every thread; the doubles the program computes beside its intervals do. [GAOL v5] Unless GAOL is built with `GAOL_PRESERVE_ROUNDING`, they are rounded upward from the initialization of GAOL, before `main()`, until `cleanup()`, and again after each operation of GAOL that follows it and sets the direction. In a build that takes the AVX-512 path, which without `GAOL_PRESERVE_ROUNDING` only the developers ask for, the arithmetic operations set none, and the direction after one of them is the program's.

The compiler does not undo it. With the flags of `gaol::gaol`, `-frounding-math` in particular, an inexact operation on constants written in a function, `1.0 / 3.0`, is not evaluated when compiling, to nearest, but when the program runs, in the direction in effect. Only what the compiler must evaluate when compiling, a `constexpr` or the constant initializer of a static variable, is still rounded to nearest.

Table 4.1 gives what the program computes, with GCC 9.4 or Clang 18 and the GNU C library 2.31 on x86-64; `examples/13_rounding_environment.cpp` of the sources computes each row and checks it.

The quotient is the double above 1/3, where the program expects the nearest. The C library rounds in the direction in effect too, as the C standard recommends: with the GNU C library, `std::rint(-2.7)` is -2 where it is -3 to nearest, and `strtod()`, `stod()`, `printf()` and the streams round up the decimal digits they read and write. A double written with 17 digits, which reads back to itself to nearest, then comes back one double above, and one more at each round trip. Another C library may convert to nearest whatever the direction: the example reports what the one it runs with does. Sums are biased: each addition rounds up, so 0.1 added ten million times ends $5.5 \cdot 10^{-4}$ above the exact $10^7 \times 0.1$, where to nearest it ends $1.6 \cdot 10^{-4}$ below.

The error-free transformations break. TwoSum, the algorithm that recovers the error of a sum,

Example

```
1 s = a + b;  bb = s - a;  e = (a - (s - bb)) + (b - bb);
```

gives $s + e = a + b$ only to nearest. Rounded upward, the exact error of $10^{20} + 10^{-20}$, which is $10^{-20} - 16384$, is no double: `s` is $10^{20} + 16384$ and `e` is -16383.999999999998, whose sum is not $10^{20} + 10^{-20}$. TwoProd, which recovers the error of a product from Dekker's splitting of the operands, breaks the same way: 1 888 of the 50 000 pairs of the table, drawn in $[1, 2)$, lose their exactness upward, where pairs of widely different magnitudes keep it. It is exact in every direction when it is computed with `std::fma` instead: the error of a rounded product is a double, which `std::fma`

TABLE 4.1 – What the program computes while GAOL is initialized and after `cleanup()`, with GCC 9.4 or Clang 18 and the GNU C library 2.31 on x86-64.

The program computes	While GAOL is initialized	After `cleanup()`
`1.0 / 3.0`	0.3333333333333337	0.3333333333333331
`std::stod("0.3") == 0.3`	false	true
`std::lrint(2.3)`	3	2
`std::printf("%.2f", 2.675)`	2.68	2.67
`std::cout << std::setprecision(3) << 3.14159265358979`	3.15	3.14
0.1 added 10^7 times	1000000.0005483569	999999.99983897537
20 000 doubles written with `%.17g`, read back by `std::strtod`	20 000 come back different	none
`TwoSum(1e20, 1e-20)`, the error of a sum	$s + e \neq a + b$	$s + e = a + b$
`TwoSum(a, b)`, 50 000 pairs of doubles of exponents from -60 to 60	$s + e \neq a + b$ for 10 717 pairs	none
`TwoProd(a, b)`, Dekker's splitting, 50 000 pairs of doubles of $[1, 2)$	$p + e \neq ab$ for 1 888 pairs	none
`TwoProd(a, b)`, `e = std::fma(a, b, -a*b)`, the same pairs	none	none

computes however it rounds, provided it is correctly rounded, by the instruction or by a call of a C library whose `fma()` is (with GCC, `std::fma` without optimization is a call of the C library, whose `fma()` is wrong on Cygwin; `__builtin_fma`, which GAOL calls itself, is the instruction with `-mfma`). Compensated sums, and the double-double arithmetic built on such algorithms, lose their accuracy the same way. Built with `GAOL_PRESERVE_ROUNDING`, the program computes the right-hand column of the table all the time.

The upward direction has other consequences:

- a thread created after the initialization of GAOL starts upward too (with the GNU C library and POSIX threads, a new thread gets the floating-point environment of the thread creating it; on Windows it starts with the default one, to nearest), and `cleanup()` sets back the direction of the calling thread only;
- the libraries the program calls run upward as well: the `exp`, `log` and `pow` of the GNU C library 2.31 go from 0.51 ulp to 1.01 ulp from the exact value, its `cbrt` from 2.8 to 3.8 ulp (20 000 random arguments each, against mpmath), and its `sin`, `cos`, `tan` and `atan` give the same results;
- GCC does not model the rounding direction, even with `-frounding-math` (GCC bug 34678, https://gcc.gnu.org/bugzilla/show_bug.cgi?id=34678), and reuses after `cleanup()` a double computed before it, still rounded upward, where the program computes it again: from `-O1`, GCC 9.4 does it for `a / b` written on both sides of the call, where Clang 18 computes the quotient again. A program cannot rely on either.

What a program does about it:

- **It calls `cleanup()` right after the last use of GAOL**, before it prints, converts or sums what it computed: its doubles are then computed to nearest. A program that goes back to GAOL afterwards calls `restore_rounding()` (or `std::fesetround(FE_TONEAREST)`) after each use, since an operation of GAOL sets the direction upward again and `cleanup()` sets it back once only (see Section 4.10, page 112).
- **It reads and writes numbers through GAOL**, not through the C library: `gaol::textToInterval("0.3")` encloses 3/10 in every direction, where `std::stod("0.3")` does not, and `exact_string()` and `%a` write a number that reads back to the same bits in every direction. GAOL writes its intervals rounded outward whatever the direction.
- **It computes in a block the doubles that need rounding to nearest** (an error-free transformation, `std::lrint()`, a decimal conversion, a call of a library written for that): it saves the direction with `std::fegetround()`, sets `FE_TONEAREST`, and sets the saved one back when the block ends, with no

operation of GAOL inside, which would set the direction upward again. The example of the sources writes such a guard, `nearest_scope`.

- **It does not count on the compiler** to compute a double where the source code does. One that has to be computed on this side of a change of direction is computed from operands read again through `gaol_core::rnd_keep()`, which writes its argument to volatile memory and returns it: `gaol_core::rnd_keep(a) / gaol_core::rnd_keep(b)` after `cleanup()` is the quotient to nearest with GCC too, where `gaol_core::rnd_keep(a / b)` may be the one computed before.

- **Or GAOL is built with `GAOL_PRESERVE_ROUNDING`**, and the program keeps its own direction (see Section 4.3, page 93).

4.3 Preserving the rounding direction

Code that needs its own rounding direction after the operations of GAOL builds GAOL with the option `GAOL_PRESERVE_ROUNDING` of CMake, which is `--enable-preserve-rounding` with autotools and `-Denable-preserve-rounding=true` with meson (see Section 2.5.2, page 36). It is off by default in the three builds. With it, each operation reads the rounding direction it finds, sets it upward, and sets back the one it found when it is done; it also sets back the modes flushing the subnormal numbers to zero that it cleared (see Section 4.8, page 104). The operations of the AVX-512 path set none (see Section 4.4, page 94). Nothing else changes: the bounds are the same, and the program still calls `cleanup()`. [GAOL v5]

The option changes four other things.

- **The AVX-512 path.** With it, the build takes the AVX-512 path (it was the default of an option, `GAOL_PREFER_AVX512`): where the machine building has the AVX-512 instructions, the four arithmetic operations, the square, the square root and, with FMA, the integer powers beyond 1 and −1 take them, which carry their rounding direction, set none and store none. The library then needs a processor with them. [GAOL v5]

- **The initialization.** It leaves the whole floating-point environment as it found it, where the default build sets the default environment: an exception a static object enabled before it stays enabled (see Section 4.9, page 107).

- **`restore_rounding()`.** It does nothing, GAOL leaving the direction of the program alone (see Section 4.10, page 112).

- **`cleanup()`.** It sets nothing back either, the direction of the program being its own.

The direction is written only where it has to be. On the processors whose control registers GAOL writes itself, an operation reads the direction and the flush modes once at its start, writes the register only if the direction is not upward or a flush mode is set, and writes back what it found at its end only if it differs from what the operation left: a program that computes upward already, with the flush modes clear, pays no write. These registers are FPCR on 64-bit ARM with the NEON intervals and FPSCR on 32-bit ARM with GCC (see Section 4.5, page 96), and the control word of the x87 unit and MXCSR on x86 (see Section 4.7, page 103). The FPU intervals of 64-bit ARM (`-DGAOL_SIMD=OFF`) read the direction with `fegetround()` and set it back with `fesetround()`, and write FPCR at the end of every operation for the flush modes, whatever the direction of the program.

The option costs time, less with the AVX-512 path, which it turns on where the machine has the instructions. Table 4.2 gives the factor by which each operation gets longer on an Intel i7-1185G7 with Clang 21.1.8 (`tests/performance.cpp` and the long lines of `doc/compare/code/run_lines.sh`, the median of 9 interleaved runs of 7 November 2026), with the path, and without it (a build for a machine without the instructions, or one that refuses the path with `-DGAOL_PREFER_AVX512=OFF`, a setting of the developers). The arithmetic operations take 1.2 to 1.6 times longer with the path, and several times longer without it; a line of 15 additions of intervals takes 1.9 times longer with the path and 8.9 times without.

Table 4.2 – How many times longer each operation takes with `GAOL_PRESERVE_ROUNDING` than without it, with and without the AVX-512 path (Intel i7-1185G7, Clang 21.1.8, `tests/performance.cpp`, the median of 9 interleaved runs, 7 November 2026, the run of Table 12.2: the time of the preserved build over that of the default one).

Operation	With the AVX-512 path	Without it
`x + y`, `x - y`	1.2	7.2 and 7.1
`x * y`	1.6	4.4
`x / y`	1.6	3.4
`sqr(x)`	1.0	4.3
`sqrt(x)`	0.6 (it is faster)	1.6
`pow(x, 3)`	1.2	1.6
`exp(x)`	1.2	1.2
`log(x)`	1.1	1.1
`sin(x)`, `cos(x)`	1.2	1.2
a line of 15 additions	1.9	8.9

4.4 The AVX-512 path

The AVX-512 instructions of the x86-64 processors carry a rounding direction in the instruction itself, the *embedded rounding*: an operation that takes them reads no direction, sets none and stores none. GAOL takes them in the build that preserves

the rounding direction, in which every operation of the other path saves and restores the direction (see Section 4.3, page 93): the path relieves `+`, `-`, `*`, `/`, `sqrt`, `sqr` and the integer powers of that store. [GAOL v5] The instructions of the kernel are described in Chapter 12 (see Section 12.1, page 228).

When the build takes it. The path is decided when GAOL is configured, as the fused multiply-add instructions are (see Section 2.5.2, page 36): where `GAOL_PRESERVE_ROUNDING` is on, the compiler takes the instructions and the machine building runs them, the macro `GAOL_AVX512_ARITHMETIC` is 1 in `gaol/gaol_configuration.h`; a cross-compilation leaves the path out unless the machine running GAOL is said to run the instructions (`-DGAOL_MACHINE_RUNS_AVX512=1`, `gaol_cv_machine_runs_avx512=yes` of `configure`, `-Dmachine-runs-avx512=true`, settings of the developers), and the operations test nothing as they run. With that option every operation gains, 2.9 times at the median of seven operations, from 1.3 to 6.1 times, on an Intel i7-1185G7, whose AVX-512 units are halved; without it they take 20 % longer at the median, the direction being written only when it is not upward already, so the path is left out. `GAOL_PREFER_AVX512`, `--enable-prefer-avx512` and `-Denable-prefer-avx512` were options of the user, and are now settings of the developers: [GAOL v5] `=ON` asks for the path without the preserved direction, and `-DGAOL_PREFER_AVX512=OFF`, `--disable-prefer-avx512` and `-Denable-prefer-avx512=false` refuse it, for a package for any processor of the architecture. A library built with the path needs a processor with the instructions, as with the fused multiply-add instructions, and `gaol::init()` stops a program run on another one, with a message on the standard error, rather than letting its first operation stop on an illegal instruction (an operation made before GAOL initializes itself, as described in Section 4.10, page 112, still does). GAOL no longer chooses the path as the program runs, in `init()`, with every operation testing the flag it set, in the default build too. [GAOL v5]

The operations. On such a build, `x + y`, `x - y`, `x * y`, `x / y`, `sqrt(x)` and `sqr(x)`, and the operations `+`, `-`, `*` and `/` with a double, are computed by the rounding direction the instructions carry. So are, with the fused multiply-add instructions, `pown(x, n)` and `pow(x, n)` for an integer n of 2 or more ($n = 2$ is `sqr(x)`), or of -2 or less where `x` has no zero, whose two corners are within the range of the exact products of their kernel; the exponents 0, 1 and -1, and the powers beyond the range, which a corner near 0 or a large exponent makes, take the path of the direction. These operations set no rounding direction: the program keeps its own, in the default build as with `GAOL_PRESERVE_ROUNDING`, and the other operations (`%`, the elementary functions, the reading of a number, the integer powers beyond the range) set it upward, and leave it so in the default build, or set back the direction of the program with `GAOL_PRESERVE_ROUNDING`, as they always do.

The flags and the flush modes. The instructions of the path raise no floating-point exception flag, the architecture requiring the suppression of the exceptions with the embedded rounding; the check of the other operations raises the flags described in Section 4.9, page 107. The modes that flush the subnormal numbers to zero, which the embedded rounding honours as the other instructions do (measured on an Intel i7-1185G7: with flush-to-zero set, the difference of `3e-308` and `2.9e-308` is 0), are cleared before the operation reads a bound when a program or a plug-in has set them, as the other operations clear them (see Section 4.8, page 104), and set back after its result with `GAOL_PRESERVE_ROUNDING`: with Clang 18, the difference of `x - y` was once computed after flush-to-zero had been set back, and `[3e-308] - [2.9e-308]` was [0, 0]. [GAOL v5]

The bounds. They are those of the other path, bit for bit, but for the sign of a zero bound in one case.

Remark 4.4.1 `x * d`, `x *= d` *and* `d * x`, *which is computed as* `x * d`, [GAOL v5] *of an interval whose zero bounds have different signs,* [+0, −0], *are* [−0, +0] *with the AVX-512 path and* [+0, −0] *with the SSE2 one,* +0 *and* −0 *being the same number (see Section 3.10.6, page 78). The rounding rule is the same, and the tests of* `tests/`*, which a job of the continuous integration runs with the option on, check it with* `==`*, which does not see the sign of a zero;* `rounding_direction` *checks the bits of that one case in each path, and every result of the operations of intervals and with a double on the intervals of two bounds among 0 of either sign,* ±1, ±2.5, ±3, $\pm 10^{-310}$, $\pm 10^{300}$ *and* $\pm\infty$ *was compared bit for bit.*

The cost. The operations of the path cost more time on a processor that halves its AVX-512 instructions, where they cost less on one that does not. On an Intel i7-1185G7 (halved), with Clang 21.1.8 (`gaol_performance`, the median of 9 runs of 7 November 2026, from Table 12.2), `x * y` takes 4.5 ns rather than 2.8 in the default build, `x / y` 5.0 ns rather than 3.2 and `x + y` 1.5 rather than 1.2, but `sqrt(x)` 4.7 rather than 8.5, the instruction rounding each root where the SSE2 path checks the root of the C library with a product. With `GAOL_PRESERVE_ROUNDING`, where the SSE2 path saves and restores the direction, `x + y` takes 1.5 ns rather than 8.8, `sqr(x)` 2.0 rather than 8.4 and `pown(x, 3)` 7.8 rather than 10.4; the comparison of the libraries (see Section 12.5, page 237) has the preserved column of the other operations. The processors that do not halve them, as the Xeon of the continuous integration, are expected to take less. Which of the paths a GAOL took, `examples/13_rounding_environment.cpp` tells.

4.5 The NEON intervals

On 64-bit ARM processors (AArch64), GAOL computes its intervals with NEON instructions, as it does with SSE2 instructions on x86. [GAOL v5] The two bounds of

an interval are the two lanes of one `float64x2_t`, the lower bound negated as in the `__m128d` of the SSE2 intervals, and `+`, `-`, `*`, `/`, `sqr()` and `sqrt()` are kernels that take and give their bounds in registers (see Section 12.1, page 228 describes what each does).

They are the intervals of GAOL on 64-bit ARM with every compiler (GCC, Clang and AppleClang under Linux and macOS, clang-cl and the Clang of MinGW-w64 and MSYS2, and Visual C++), by default: the option is `GAOL_SIMD` of CMake, `--enable-simd` of `configure` and `enable-simd` of meson, as for the SSE2 intervals (see Section 2.5.2, page 36), and the macro `GAOL_USING_NEON_INSTRUCTIONS` is then 1 in `gaol/gaol_configuration.h`. They need no flag beyond those of interval arithmetic (see Section 3.3, page 64). `GAOL_ARM64`, `--enable-arm64` and `-Denable-arm64` were an option of their own, independent of `GAOL_SIMD`, and are gone. [GAOL v5] 32-bit ARM, whose NEON has no vector of doubles, does not take them: the option is ignored there, and the FPU computes the intervals, as before (see Section 4.6, page 98). A universal build of macOS made by CMake takes them for its arm64 part, and the SSE2 intervals for its x86_64 part.

Visual C++ writes the control register FPCR through the intrinsics `_ReadStatusReg()` and `_WriteStatusReg()`, where GCC and Clang have `mrs` and `msr` in an asm statement; it has no asm statement to order its operations with respect to those writes, and relies on `/fp:strict` to keep them where the source puts them, which `tests/rounding_direction.cpp` checks. With `/W4`, Visual C++ for arm64 warns (C4324, "structure was padded due to alignment specifier") of each structure that holds an interval, its `float64x2_t` being aligned on 16 bytes there, and of `expr_eval` of the installed `gaol/parser/gaol_expr_eval.h`: the warning is harmless, and no job of the continuous integration compiles with `/W4 /WX` for arm64. [GAOL v5]

The control register FPCR. 64-bit ARM has no instruction that carries its own rounding direction, as the AVX-512 path has: the direction is the field RMode of the control register FPCR (bits 23 and 22, upward being `0x400000`), which the scalar and the vector instructions share, and which holds the modes that flush the subnormal numbers to zero as well (FZ, bit 24, and from Armv8.7 FIZ, bit 0). The NEON intervals therefore check the direction by reading FPCR with `mrs`, which raises no exception flag, and write it with `msr` only if it is not upward with the flush modes clear, where the FPU intervals add $1 + (2^{-1060} + 0)$, which raises the inexact flag (see Section 4.9, page 107). `msr` on FPCR is a serializing instruction, slower than `mrs`, which is why it is written when needed only. With `GAOL_PRESERVE_ROUNDING`, each operation reads FPCR once, and writes back the direction and the modes it found at its end only if they differ from what the operation left: a program that computes upward already, with the flush modes clear, pays no write. `round_upward()`, `round_nearest()` and `round_downward()` write FPCR as well, and not through `fesetround()`, and only if the direction changes (see Section 4.11, page 115). `fegetround()` and `fesetround()` of the C library read and write the

same bits (the `FE_*` values of the C libraries of 64-bit ARM are these bits, which a static assertion checks, but for those of Windows, whose `FE_DOWNWARD` is `0x100`), so that a program may use either.

The register FPSCR of 32-bit ARM. The FPU intervals of 32-bit ARM processors with GCC and a floating-point unit of doubles do the same for `round_upward()`, `round_nearest()` and `round_downward()`, and for the direction that `GAOL_PRESERVE_ROUNDING` restores (`GAOL_RND_FPSCR_DIRECT` of `gaol/gaol_config.h`, 1 there; `-DGAOL_RND_FPSCR_DIRECT=0` gives the C library back). The register is FPSCR, read with `vmrs` and written with `vmsr`, with the same field RMode (the `FE_*` values of the C libraries of 32-bit ARM are its bits, which the same assertion checks). The flags of the exceptions and of the comparisons are in it too, so that the functions that write it from the value they read at once (`round_nearest()`, `round_upward()`, the end of an operation) read it again just before, and no flag raised since is lost. With `GAOL_PRESERVE_ROUNDING`, each operation reads FPSCR once at its start, writes it only if the direction is not upward or a flush mode is set, and writes back what it found at its end only if it differs. On the Debian armhf jobs of the continuous integration, which run on the arm64 runners of GitHub, `x + y` took 22.5 ns rather than 49 with `GAOL_PRESERVE_ROUNDING`, a line of 15 additions 318 ns rather than 721, and `midpoint()` 38 ns rather than 63. [GAOL v5] The check of every operation without the option stays the addition: `vmrs` took longer than the two additions there, `x + y` 6.1 ns rather than 4.1 and a line of 15 additions 87 ns rather than 57, and the 32-bit ARM intervals keep their time.

The bounds. They are those of the FPU intervals but for the sign of a zero bound, which the maximum of 64-bit ARM, ranking $+0$ above -0, can set otherwise than the SSE2 and the FPU intervals do, where the `maxpd` of x86 returns its second operand (see Section 3.10.6, page 78).

Remark 4.5.1 *The lower bound of* `[-1, 0] * [-1, 0]` *is* -0 *with the NEON intervals and* $+0$ *with the SSE2 ones (the FPU intervals leave it to the compiler, see Section 4.6, page 98), and* `interval::zero()` *is stored* `<+0, +0>`*, as with SSE2, so that its lower bound is* -0*;* $+0$ *and* -0 *being the same number,* `==` *does not see the difference. The tool* `tests/tools/arith` *compares the NEON and the FPU intervals on a grid of operands, with the calling code in each of the four rounding directions: it found no other difference. The same kind of caveat exists for the AVX-512 path (Remark 4.4.1).*

4.6 The FPU intervals

The intervals of the processors that have neither SSE2 nor NEON keep their two bounds as two doubles, the lower one negated as in the SSE2 and the NEON

intervals, and compute with the scalar floating-point unit. They are the intervals of 32-bit ARM, POWER, s390x, RISC-V and the other processors; of Windows on 32-bit x86 with a compiler that takes no flag asking for the aligned memory the SSE2 intervals need (MinGW-w64 with GCC and Clang takes the SSE2 intervals there, with `-faligned-new=8`, and so do Visual C++ and clang-cl, whose intervals need no aligned memory; 64-bit Windows takes the SSE2 intervals too); and of the builds that ask for them (`-DGAOL_SIMD=OFF`, `--disable-simd` or `-Denable-simd=false`), which leave `GAOL_USING_SSE2_INSTRUCTIONS` and `GAOL_USING_NEON_INSTRUCTIONS` at 0 in `gaol/gaol_configuration.h`.

`x * y`, `x / y` and `x % y` of a divisor without zero, and `sqr(x)` take no branch on the signs of the bounds in the source: [GAOL v5] GAOL 4 took one of nine cases of the signs for a product, and went down a chain of up to eight tests for a quotient, which a processor mispredicts when the signs come at random. The product is the largest of four products for the upper bound and of four others for the lower one, each with a factor negated, a product and its opposite not being rounded the same way where the only direction is upward; the quotient divides each bound of the dividend by the bound of the divisor that its sign chooses, two divisions whatever the signs; the square is the largest of two squares for each bound, the lower one being 0 for an interval that straddles 0. They are the kernels of the SSE2 and the NEON intervals on two doubles each (see Section 12.1, page 228), and give the bounds of the SSE2 ones bit for bit for the product of finite bounds and the quotient by a divisor without zero (checked against a model of their kernels, not by running them), but for the sign of a zero bound of a product (Remark 4.6.1). An interval with an infinite bound, whose 0 times an infinity is no product to form, takes a function that sets both factors of such a product to 0 (the nine cases of the code before the kernels took 12.4 ns for operands with an infinite bound drawn at random, and it takes 9.2), and a divisor with a zero takes the table of the NEON intervals, one division at most. `x - y` reads the bounds of `y` after the check of the rounding direction, as `x + y` does: it took three times as long as the sum, the bounds being kept across the calls of the rare path of the check.

The time of the kernels. Table 4.3 gives the times measured with `gaol_performance` on an Apple M2 with AppleClang 21 (`-DGAOL_SIMD=OFF`), the median of nine runs of each program, in turn with the others, on 6 November 2026, in nanoseconds, with the code before the kernels (the tree of 22 October 2026) and with the kernels of GAOL v5.
The operands of `gaol_performance` are 1 024 intervals that the processor learns by heart, so that a branch on their signs costs less than it does on other operands. On 100 000 intervals centred at random (the benchmark of `doc/compare`, see Section 12.5, page 237), `x * y` took 13.9 ns and takes 3.5, `x / y` 8.5 and 3.2, `sqr(x)` 6.9 and 1.8, `x - y` 3.7 and 2.0, and Shekel 5 took 298 ns and takes 138.

A line of products of operands of one sign, which a processor predicts, takes more time: 7 products in a line took 23.3 ns and take 35.0 (23.7 and 35.2 in the

TABLE 4.3 – The FPU intervals on an Apple M2 with AppleClang 21 (`-DGAOL_SIMD=OFF`): the code before the kernels (the tree of 22 October 2026) and the kernels of GAOL v5, in nanoseconds (`gaol_performance`, the median of nine runs).

	Before the kernels	Kernels
`x - y`	3.54	1.15
`x * y`	3.25	3.25
`x * y`, random signs	6.39	3.31
`x / y`	3.98	3.23
`x / y`, random signs	3.17	3.21
`sqr(x)`, random signs	2.64	1.78
a line of 7 `*`	23.3	35.0
a line of 7 `/`	35.1	32.9
a line of 13 `+ - * /`	49.6	50.4

benchmark of `doc/compare`, `pos_mul8`; Apple M2). Each product waits for the one before it, and the largest of four products waits for its products and for two comparisons, where one product on a predicted branch waits for one. The maximum of 64-bit ARM (`fmax`, one instruction, which ranks +0 above −0) would shorten the wait: it is not used, being a call with GCC for 32-bit ARM and x86. With `GAOL_PRESERVE_ROUNDING` the write of the control register waits for the result, so that the longer wait of the product shows there as well: `x * y` takes 42.4 ns rather than 39.7 and the line of 7 `*` 297 rather than 279, that is 6 to 7 % more (`sqr(x)` 42.4 rather than 39.5; `x / y` 43.6 rather than 43.1; `x * y` of random signs 42.4 rather than 47.5, where the branches of the former code were mispredicted; the median of nine runs of each code, in turn, on 6 November 2026). The operators write their result into the bounds of `*this`, memory of the caller that is also given by address to the functions of the rare cases, which a call or an asm statement with a "memory" clobber that restores the direction finds written: the products they hold are computed before it, with the direction upward. That is how the compilers treat it (GCC 15 for 32-bit ARM with `-DGAOL_RND_FPSCR_DIRECT=0`, where the end of an operation is a call of `fesetround()`: the stores precede `bl fesetround`), not a rule of the language, and no test holds it: `GAOL_RND_KEEP()` would make it one, for 2.6 ns more with `GAOL_PRESERVE_ROUNDING`, and the former product and `x + y` did without.

The processors of the continuous integration. The processors and compilers of the continuous integration that took the FPU intervals were measured in turn (Visual C++ and clang-cl for arm64 take the NEON intervals since), [GAOL v5] in one run of GitHub Actions: each job builds the code before the kernels and after them, and runs `gaol_performance` five times with each, alternately, with the tools of `.github/scripts/performance-ab.sh` and `compare-performance.py` (see Section 14.5, page 305). The compilers are GCC 13.3 and Clang 18.1 on Ubuntu 24.04 for x86-64 (`-DGAOL_SIMD=OFF`), GCC 14.2 on Debian 13 for 32-bit ARM, run natively on an arm64 runner, Visual C++ 19.51 for x86 and arm64, clang-cl 22.1 for x86 and arm64, and AppleClang 21 on macOS 26 (`-DGAOL_SIMD=OFF`). Table 4.4

gives the ratio of the median before the kernels to the median after them, so that a ratio above 1 means that the kernels are faster. These are figures of shared runners, of one run, of the FPU intervals as they were: Visual C++ and clang-cl for x86 have the SSE2 intervals since, [GAOL v5] with their two bounds as two doubles, and Visual C++ and clang-cl for arm64 the NEON intervals: the rows of these four configurations are no longer those of the default build.

TABLE 4.4 – The FPU intervals on the processors of the continuous integration: the ratio of the median time of the code before the kernels to the one of the kernels of GAOL v5 (five runs of `gaol_performance` each, alternately; a ratio above 1 means that the kernels are faster). The columns are `x * y`, `x * y` of random signs, `x / y`, `sqr(x)`, `x - y`, a line of 7 `*`, a line of 7 `/` and a line of 13 `+ - * /`; "preserved" is the build with `GAOL_PRESERVE_ROUNDING`. †: the spread of the five runs exceeds 1.15, and the ratio is noise; in bold, the ratios below 0.9 of a stable row.

	`x*y`	`x*y` rnd	`x/y`	`sqr`	`x-y`	7 `*`	7 `/`	13 ops
Ubuntu, GCC 13	1.29	1.64	1.25	1.09	1.02	**0.58**	1.13	**0.76**
Ubuntu, Clang 18	1.18	1.67	1.47	**0.85**	0.99	0.98	1.05	1.01
armhf, GCC 14	1.41	2.28	1.63	1.17	**0.86**	1.11	1.25	1.04
Visual C++, x86	1.68†	3.47	1.11	1.39	1.06	1.43	1.16	1.22†
Visual C++, x86, preserved	1.18	1.64†	1.02†	1.09†	1.02†	1.18	1.04	**0.53**
Visual C++, arm64	1.53	2.41	**0.82**	2.36	1.08†	1.71	1.05	1.41
clang-cl, x86	1.06	1.23	1.24	1.06	1.04	1.04	1.16	1.05
clang-cl, arm64	**0.77**	1.75	1.29	**0.84**	1.01	**0.57**	1.24	**0.86**
macOS, FPU	0.92†	1.98†	1.13†	1.03†	2.49†	0.60†	1.04†	0.83†

The product wins where the signs are random (1.23 to 3.47 times) and, alone, of positive operands on 32-bit ARM, Visual C++ and Ubuntu. It loses on the lines of products of positive operands with GCC 13 on Ubuntu for x86-64 (0.58 for the line of 7 `*`, that is 1.7 times slower, where the Apple M2 above gives 23.3 and 35.0 ns, 1.5 times) and with clang-cl for arm64 (0.57), and alone with clang-cl for arm64 (0.77): there, the branch of the former code is well predicted and the kernel of eight products adds latency. The macOS runner is too noisy to say (spreads of 1.30 to 2.71). The quotient wins from 1.11 to 1.63 times on six configurations, is within the noise on two (the direction preserved, 1.02, and macOS, 1.13) and loses with Visual C++ for arm64 (0.82, spread 1.04). `sqr(x)` is mixed: 0.84 with clang-cl for arm64 and 0.85 with Clang on Ubuntu, 2.36 with Visual C++ for arm64. `x - y` does not change where the rows are stable (0.86 to 1.06), 0.86 on 32-bit ARM where `x + y`, whose code did not change, gives 0.92 in the same job; the 3.54 to 1.15 ns of the M2 does not show. With the direction preserved on Visual C++ for x86, the line of 13 operations is 1.9 times slower (0.53, spread 1.08), where each operation wins or stays: the cause was not searched. POWER, s390x, RISC-V and MinGW-w64 for arm64 are not measured.

What a compiler makes of the choices of the kernels is its own, and was read in the assembly on Compiler Explorer, not run. The maxima of the product are conditional moves with GCC (13 to 16) for 32-bit ARM, arm64 and x86, with Clang, and with Visual C++ for x86 and arm64, `xsmaxcdp` instructions with GCC 13 for POWER9,

and branches on the products with GCC for POWER8, s390x and RISC-V, which have no conditional move of doubles. The choices of the quotient and of the square are conditional moves with GCC for POWER and with Clang and Visual C++ for arm64, and branches on a sign with GCC for 32-bit ARM at `-O2` and `-O3` (two in the quotient with GCC 13 to 15, four with GCC 16, none in the square; at `-Os`, two in the square with GCC 14 and 15), arm64 (one in the quotient), s390x and RISC-V, and with Visual C++ for x86.

Remark 4.6.1 *The sign of a zero bound of a product is not specified for the FPU intervals: where two products are zeros of opposite signs, +0 and −0, the maximum of the two is a comparison and a selection, and the compiler decides which of the two it returns. The* `maxsd` *of x86 returns its second operand, and the compilers that keep the order written do the same (the lower bound of* `[-1, 0] * [-1, 0]` *was +0 on the jobs of the continuous integration with GCC for 32-bit ARM and for arm64, with Clang for arm64 and with Visual C++ for x86), but GCC 13 for POWER9 compiles* `(a > b) ? a : b` *into* `xsmaxcdp`*, whose result for a +0 and a −0 is its first operand (the lower bound was −0 on the POWER9 jobs under QEMU, and the assembly of Compiler Explorer shows six* `xsmaxcdp` *in the product and no other maximum; not run on a processor). The lower bound of* `[-1, 0] * [-1, 0]` *is therefore +0 with the SSE2 intervals, −0 with the NEON ones, and +0 or −0 with the FPU ones, by the compiler and the target. The upper bound of* `[-1, 0] * [0, 1]` *is such a tie too, but with other signs: it is −0 with the SSE2 intervals. +0 and −0 are the same number:* `==` *and* `<` *do not see it, and a program does not read it (see Section 3.10.6, page 78). The sign of the zero bound of a quotient and of a square does not depend on a tie, and is fixed:* `[0, 1] / [-2, -1]` *has the upper bound −0 (the zero bound of the dividend is divided by a number whose sign the bounds of the divisor choose), and* `sqr(interval(-1, 2))` *has the lower bound +0 with the FPU intervals, which a selection of a constant sets, where the SSE2 and the NEON intervals give −0 (see Section 3.10.6, page 78).*

The other bounds and the flags are those of the former code, as `tests/tools/arith` (the four rounding directions of the caller) and 1.2 million random operands of every kind, with and without the mode that flushes the subnormals to zero, show. `x * y` computes every corner product, as the SSE2 and the NEON intervals do, so that a corner that is not a bound of the result and underflows raises the underflow exception (`FE_UNDERFLOW`), which the former code did not, for 135 808 of the 1.2 million `x * y`, and a corner that overflows raises the overflow exception (`FE_OVERFLOW`) for 23 672 more, whose result has an infinite bound all the same; no operation raises the invalid-operation or the division-by-zero exception it did not (see Section 4.9, page 107).

4.7 The control registers of x86 processors

A 32-bit x86 processor has two units that round, each with a direction of its own: the x87 unit, in the field RC of its control word (bits 11 and 10), and the SSE instructions, which compute the doubles of GAOL, in the field RC of MXCSR (bits 14 and 13), where the modes that flush the subnormals to zero are too (FTZ, bit 15, and DAZ, bit 6). Both have to be upward: the square root and the `fma()` of a C library, and a `long double`, follow the direction of the x87 unit (the elementary functions of CORE-MATH read MXCSR, as on x86-64, after having read the one of the x87 unit with `fegetround()`: the one of glibc, and -1 with the C runtime of Windows when the two differ), and the addition that shows the direction elsewhere sees the SSE instructions only. Each operation therefore reads the two registers, the control word with `fnstcw` and MXCSR with `stmxcsr`, in one asm statement (inline assembly with Visual C++), where it called `fegetround()` and read MXCSR. [GAOL v5] That call took about 80 ns with Visual C++ for x86: `x + y` took 81 to 100 ns, and a line of 15 additions 1 211 to 1 546 ns, in the step of the continuous integration that runs `gaol_performance`.

The registers are written (`fldcw`, `ldmxcsr`) only when the direction of one is not upward or a flush mode is set, and the one that differs only; MXCSR is written once for the direction and the flush modes. With `GAOL_PRESERVE_ROUNDING`, an operation reads them once at its start, and writes back at its end what it found, in the register that differs from what the operation left: a program that computes upward already, with the flush modes clear, pays no write. The SSE2 operations (`+`, `-`, `*`, `/`, `sqr()`...), which use no x87 instruction, keep MXCSR alone with `GAOL_PRESERVE_ROUNDING`, with GCC and Clang, and leave the x87 unit as it is: they write it only if it is not upward with the flush modes clear, and give back what they found only if it was not. With Visual C++ and clang-cl they set and give back both units, as the other operations do: Visual C++ computes on the x87 unit a double that a function returns (`quotient_sat()`, which `div_rel()` calls, divides with `fdiv`). [GAOL v5] A program that computes upward already pays no write there either, `x + y` taking 1.7 ns rather than 8.2 and `x * y` 2.5 rather than 8.3 (GCC 15.2, an Intel i7-1185G7, the best of 15 runs of a loop; a program to nearest pays 0.2 ns more, 8.4 for `x + y`). `round_upward()`, `round_nearest()` and `round_downward()` write the register whose direction is not the one they set (both, as `fesetround()` does, when neither is), and `cleanup()` and `restore_rounding()` set back the direction that `init()` found in each unit (see Section 4.10, page 112). The values are those of the hardware (`0x0800` for upward in the control word, `0x4000` in MXCSR), not the `FE_*` constants of `<fenv.h>`, which are `0x0400` and `0x0800` for downward and upward with glibc and `0x0100` and `0x0200` with the C runtime of Windows and with mingw-w64 13 and later.

An x86-64 processor has the two units too, and GAOL reads and writes their registers itself there, as above, where it restores a direction (`GAOL_PRESERVE_ROUNDING`, `cleanup()`, `restore_rounding()`, and the operations that set the direction to near-

est and back: `midpoint()`, `chi()`, the writing and the reading of an interval), rather than call `fegetround()` and `fesetround()`, [GAOL v5] the latter writing both registers whatever they hold, and then MXCSR once more for the direction of the SSE instructions: three writes and a call, where a program that computes to nearest needs one write and a program that computes upward none. The register that differs is written only, with `fldcw` or `ldmxcsr`, which keep the flags; Visual C++ for x64, which has no inline assembly and uses no x87 unit, reads and writes MXCSR alone. With mingw-w64, the C library gave reasons of its own to read and write the registers directly: from version 13, its `fegetround()` joins the fields RC of the two units by OR, which is the direction of both only where they agree (to nearest and upward give upward, `FE_DOWNWARD | FE_UPWARD` is `FE_TOWARDZERO`), and its `fesetround()` clears the six exception flags of MXCSR when it changes its value, so that the flags of the program were lost in `midpoint()`, in `chi()`, in the writing and the reading of an interval, with `GAOL_PRESERVE_ROUNDING`, and in `cleanup()` and `restore_rounding()` with it or without it.

4.8 Flush-to-zero and denormals-are-zero

Definition 4.8.1 *The subnormal numbers are the nonzero doubles below* 2^{-1022} *in magnitude. Processors have modes that flush them to zero. On x86, the flush-to-zero mode (FTZ) and the denormals-are-zero mode (DAZ) of the SSE instructions, two bits of the control register MXCSR, make every subnormal result a zero (FTZ) and read every subnormal operand as a zero (DAZ), in the comparisons too. On ARM, the mode FZ of the control register (FPCR, FPSCR) does both.*

With one of them set, the bounds of an operation with a subnormal result (FTZ or FZ) or a subnormal operand (DAZ or FZ) can miss the exact result, whatever the rounding direction: with flush-to-zero set, the product `interval(1e-300) * interval(1e-20)`, the square `sqr(interval(1e-160))` and the difference `interval(3e-308) - interval(2.9e-308)` are all [0, 0]. [GAOL v5]

How a program gets them. A program linked with one of the options `-Ofast`, `-ffast-math` and `-funsafe-math-optimizations` gets them: GCC and Clang then link `crtfastmath.o`, on x86 and on ARM Linux, whose constructor sets them for the whole program. GCC links it last, and it runs after GAOL initialized itself; Clang links it first, and the initialization of a static GAOL, which follows, clears the modes again, unless GAOL is built with `GAOL_PRESERVE_ROUNDING`, or is a shared library, initialized before the program. The option `-fno-fast-math` of `gaol.pc` and `gaol::gaol` does not prevent it: it keeps `crtfastmath.o` out of the link when it comes after `-ffast-math`, but not after `-Ofast` (GCC 9.4 and 13 and Clang 18 link it all the same, and `-fno-fast-math` silences the `#error` of `gaol/gaol_config.h` against `-ffast-math`, which `-Ofast` would raise), nor, with GCC 9.4 and 13, after the option `-funsafe-math-optimizations`. Loading a plug-in or a Python module

built with `-Ofast` sets the modes too, when it is loaded, where its compiler linked `crtfastmath.o` into it: Clang 18 does, GCC did before version 13.

Remark 4.8.2 *Linking a program that uses GAOL with* `-Ofast`, `-ffast-math` *or* `-funsafe-math-optimizations`*, or loading code built with them, sets the modes, which makes the bounds wrong unless GAOL defends itself against them, as below. A program that uses GAOL is linked without these options, or with* `-mno-daz-ftz`*, and the code that needs them is compiled apart from it.*

GAOL defends itself in two ways.

- **The check of each operation clears the modes.** On x86 processors, and on ARM processors with GCC and Clang and with the NEON intervals of Visual C++, each operation that computes bounds clears the modes when it starts, in the check that sets the rounding direction upward: the check is an addition of a subnormal number and a comparison, which shows the modes as well as the direction (see Section 4.1, page 88), or, with the NEON intervals, a read of FPCR, which holds both; the modes stay cleared for the rest of the program, as the rounding direction stays upward. The check comes before the operation reads, compares or copies a bound, the test of the empty set aside, and the compiler is kept from making a comparison before it, which it would do, not knowing that the check changes how a subnormal compares: with GCC 13 at `-O3`, `asinpi()` compared the bounds it had read with 0 before its check, and with the modes set `asinpi()` of $[100 \cdot 2^{-1074}]$ was $[32 \cdot 2^{-1074}]$, above the exact value. With `GAOL_PRESERVE_ROUNDING`, each operation sets the modes back as it found them, once it has made its result, its maxima and minima of bounds included. The bounds of every operation that makes the check are then the ones it gives with the modes cleared, and the modes are cleared, or set back, after it; [GAOL v5] `tests/rounding_direction.cpp` checks it for each of them. It is the only defence against a plug-in, which sets the modes at any time, and against the compilers and processors without `-mno-daz-ftz`.

  The check costs little. On an Intel Xeon of the Cascade Lake generation, with Clang 18, an addition with a subnormal operand and result took as long as one of normal numbers (1.2 ns in a chain of dependent additions), as on an Intel i7-1185G7, and the check as long as the one of the direction alone, $1 + 2^{-60}$, in front of the addition of two SSE2 intervals. Through the library (`gaol_performance`, medians of 9 interleaved runs), `x * y` took 4.0 ns rather than 3.4 ns, `sqrt` 11.0 ns rather than 10.3 ns and `pow(x, 3)` 13.7 ns rather than 13.0 ns, and as long with the same check made of a normal number: the cost of its second addition, not of the subnormal; `x + y`, `x - y`, `x / y`, `sqr`, `exp`, `log`, `sin` and `cos` stayed within the noise of the machine. Some x86 processors take a microcode assist, of the order of a hundred cycles, for an operation with a subnormal operand or result, which each operation would

pay there. `gaol_performance` (`tests/performance.cpp`) prints the time of the check and of the same check made of 2^{-60}, a normal double: on the 68 jobs of the continuous integration that run on processors (x86-64, i386, arm64 and armhf, under Linux, Windows and macOS, Rosetta 2 included), the first took 0.35 to 1.43 ns and the second 0.40 to 1.38 ns, the two differing by −0.63 to +0.44 ns, either way from one job to another: none of these processors took such an assist. Under qemu (ppc64le, riscv64, s390x), both took 25 to 83 ns.

- **The link option `-mno-daz-ftz`.** `gaol.pc` and `gaol::gaol` give `-mno-daz-ftz` to the link, where the compiler accepts it: GCC 13 and later on x86, from 11.4 and 12.4 in the series 11 and 12, and Clang 20 and 21 (GCC 9.4, 12.3, GCC for ARM and Clang 18 refuse it, and the builds test the compiler with a link, not with its version). It keeps `crtfastmath.o` out of the link, so that a program linked with `-Ofast` does not get the modes at all, its own code included.

On the other processors (POWER, s390x, RISC-V, MIPS...), whose FPU intervals use the generic `round_upward()` of `gaol/gaol_fpu_fenv.h` (a call to `fesetround(FE_UPWARD)`, whose return value is not read), neither defence exists: [GAOL v5] the mode NI (non-IEEE) of the FPSCR of POWER and the bit FS of the FCSR of MIPS, modes of the kind of FTZ and DAZ that may flush the subnormal numbers, are not cleared, and a program that sets them may get bounds that miss the exact result for subnormal operands and results. What they do depends on the processor, whether the toolchains of these processors set them in a program linked with `-Ofast` was not checked, and nothing of GAOL was tested with them set.

The functions that make no check. The functions that only read, compare or copy the bounds of an interval make no check, which made each of them 3 to 4.5 ns slower in a micro-benchmark on an Intel i7-1185G7, two to three times the constructor or `certainly_le()`: they compare the bounds as bounds, whatever the modes. [GAOL v5] A mode that reads a subnormal operand as a zero (DAZ, FZ) can make two different doubles compare equal, both zero or subnormal, but never puts two doubles in the wrong order: the doubles are compared first, and only where they compare equal are they compared as integers, by their bits, which no mode changes. These functions are the constructor from two bounds, the relations (`set_contains()`, `certainly_le()`, `set_eq()`, `is_zero()`, `lexicographic_less`...), the intersection `&` and the hull `|`, `max()`, `min()`, `abs()`, `sign()`, `floor()`, `ceil()`, `integer()` and the other roundings to an integer, `invabs_rel()`, `mig()`, `mag()`, `is_canonical()`, `feven()`, `nb_fp_numbers()`, and `inf()` and `sup()` of `gaol_ieee1788`; the roundings to an integer round a subnormal bound from its bits, which the `floor()` and `ceil()` of musl would return as it is. They give the results they give with the modes cleared, and leave the modes as they found them, writing nothing to the control register, with `GAOL_PRESERVE_ROUNDING` or without.

On the Intel i7-1185G7, with Clang 18 (`gaol_performance`, medians of 9 interleaved runs), this left the constructor as fast as before, and made `x <= y`, `x.set_contains(y)`, `x &= y` and `x | y` 0.5 to 0.8 ns slower, and `floor()` 1.7 ns. Before, under denormals-are-zero, `max()` of $[3 \cdot 2^{-1074}, 100 \cdot 2^{-1074}]$ and $[200 \cdot 2^{-1074}]$ was the first one, `abs(interval(-1e-309, -1e-310))` stayed negative, `interval(`2^{-1073}`,` 2^{-1074}`)` kept its bounds in the wrong order, and the hull `|` of the same two intervals, inline in the headers of GAOL, depended on how the compiler of the program arranged its comparisons.

`midpoint()`, `split()` and `chi()`, which compute, and the writing of an interval (`operator<<`, `intervalToText()`) make the check, which clears the modes, and set them back as they found them with `GAOL_PRESERVE_ROUNDING` (1.3 ns more for `midpoint()`): the midpoint of $[2 \cdot 2^{-1074}, 4 \cdot 2^{-1074}]$ was 0, outside it, and a subnormal upper bound was written rounded to nearest, below the bound, where the C library writes a subnormal 0 under denormals-are-zero with gdtoa (FreeBSD, macOS) and with the Debug runtime of Visual C++ (`tests/rounding_direction.cpp` checks all of them).

What remains. The reader, which makes no check of the modes, takes the numbers apart by their bits, and compares the two numbers of `<a, b>`, which have to be the same double, by theirs too, [GAOL v5] so that under denormals-are-zero `<1e-310, 1e-309>` is refused, as without it (see Section 9.1.1, page 184). The unary minus makes no check either, but it only exchanges the stored bounds, which the modes do not change, as they do not change the test of the empty set, made before the check of every operation (an operation with an empty operand may return before its check, the modes left as it found them), nor the test by which `hausdorff()` returns $+\infty$ before its check when a bound is infinite in one of its two intervals only. The modes of other processors, and of ARM with the FPU intervals of Visual C++, are neither checked nor cleared, though the functions above compare the bounds right there too, and `operator<<` writes a subnormal bound outward, as 2^{-1022} away from zero and 0 toward it; GCC links `crtfastmath.o` for none of the other processors GAOL is tested on.

4.9 The floating-point exceptions

A program starts with the floating-point exceptions masked, and GAOL leaves them as they are: an operation that raises one only sets its flag while the exception is masked.

Remark 4.9.1 *A program that enables the floating-point exceptions, with* `feenableexcept()` *of the GNU C library,* `_controlfp_s()` *of Visual C++ or* `fesetenv()` *with an environment that traps, disables them while GAOL computes, and enables them again afterwards if it wants them: the operations of GAOL raise*

some, as listed below, and the processor then stops the program (with `SIGFPE` *on Linux) instead of letting the operation give its bounds.* [GAOL v5]

What the operations raise:

- `log(interval(0, 1))`, which is $[-\infty, 0]$, raises the divide-by-zero exception; `interval(1e308)*10`, which is [DBL_MAX, $+\infty$], raises the overflow exception.
- With the AVX-512 path (see Section 4.4, page 94), `+`, `-`, `*`, `/`, `sqr()`, `sqrt()` and the integer powers it takes raise no flag of IEEE 754: `interval(1e308)*10` raises no overflow exception there, and `x + y` no inexact one. Their check of the flush modes, $2^{-1060} + 0$, is exact, but where flush-to-zero alone is set: the sum, flushed to zero, then raises the underflow and the inexact exceptions, as the check of the other operations does. [GAOL v5]
- Reading a number above the largest double (`1e400`, `1.8e308`, `0x1p1024`) raises no overflow exception. The reader used to give it to `strtod()`, which computes $+\infty$ and raises it, so that `1e400*0`, which is [0, 0], raised the flag that stands for an infinite bound, and a program that had enabled the exception died while it read. The interval read is [DBL_MAX, $+\infty$]; an operation on it raises the flag where its own result overflows (`1e400*2`, `exp(1e400)`), as for any other interval. Nor does reading a number below the least normal double (`1e-310`, `1e-400`, `0x1p-1075`) raise the underflow exception, which `strtod()` raises when it rounds such a number to a subnormal or to 0. [GAOL v5]
- Almost every operation, but those of the AVX-512 path, raises the inexact exception: most bounds are rounded results, and where GAOL checks the rounding direction with an addition, $1 + 2^{-60}$ or $1 + (2^{-1060} + 0)$, whose result is inexact (see Section 4.1, page 88), even an operation whose bounds are exact raises it. On x86-64, the addition of the subnormal 2^{-1060} also raises the denormal-operand flag of the SSE instructions, which is none of the five flags of IEEE 754 (not in `FE_ALL_EXCEPT`), and its exception stops the program where the program enabled it; and with flush-to-zero set, the sum $2^{-1060} + 0$, flushed to zero, raises the underflow exception (`FE_UNDERFLOW`) too, before the check clears the mode; with denormals-are-zero set, as `crtfastmath.o` sets it with flush-to-zero, the check is exact and raises no flag at all, the inexact one included. With the NEON intervals the check is a read of FPCR (see Section 4.5, page 96), which raises no flag: `+`, `-`, `/` and `sqrt()` raise the inexact one only when a bound they compute is rounded, `*`, `sqr()` and `pow(x, 2)`, which is `sqr()`, also when a corner product that is not a bound of the result is (`[-a, 128] * [-a, 4]` for $a = 1 + 2^{-52}$ has the exact bounds $-128a$ and 512, and the corner a^2 is rounded). [GAOL v5]
- An empty operand raises no invalid-operation exception, though the bounds of the empty interval are NaN: every operation, relation, function and output of `gaol::interval` and of `gaol_ieee1788` takes it without raising that

exception. `is_empty()` and the relations compare the bounds with the quiet comparisons of `<cmath>` (`std::islessequal()`...), each one instruction, as `<=` is, with GCC and Clang on x86-64, and so does the first comparison of the constructor `interval(l, r)`, which gives the empty set for NaN bounds. With Visual C++, where these are calls to the C library, which made `x * y` and `sqrt()` 34 to 61 % slower on x64, GAOL compiles them itself into the same instruction, `ucomisd`, on x64 and on x86 with SSE2; on arm64 they stay calls. With the invalid-operation exception enabled, `is_empty()` of an empty interval killed the program, and so did about 50 operations with an empty operand, which compared its NaN bounds: `x & y` for an empty `y`, `sqrt`, `exp`, `min`, `max`, `floor`, `set_contains()`, `set_disjoint()`, the output of the empty set...; and `interval::emptyset()` itself in a build without optimization. A NaN the program gives GAOL, the empty set for `interval(d)`, raises it no longer either: `interval(NAN)`, `interval(NAN, 1)`, `x = NAN`, `x += NAN` (and `-=`, `*=`, `/=`, `%=`), `x &= NAN`, `x <= NAN` and `pow(x, NAN)` compared it with `<` or `<=`. An empty operand may still raise the inexact exception: the sum of two empty intervals does where GAOL checks the rounding direction with an addition. The vector comparisons of 64-bit ARM (`fcmgt`, `fcmge`) signal the invalid-operation exception on a NaN: the NEON operations tell the empty set, the infinite bounds and the zeros from the bits of the bounds, and compare no bound as a double before they know it is no NaN. [GAOL v5]

- A compiler may compile a quiet comparison into a signaling one, or compute a comparison that follows a test of the empty set before it, and GAOL is written against the cases seen. GCC turns a choice made on a quiet comparison into a conditional move with a signaling comparison on 32-bit ARM (`vcmpe`, GCC 12 to 14; GCC bug 52258, https://gcc.gnu.org/bugzilla/show_bug.cgi?id=52258, is of this kind) and on POWER9 (`xscmpgedp`, GCC 13 with `-mcpu=power9`): there `is_empty()`, the intersection and the constructors of the FPU intervals tell NaN bounds with `std::isunordered()` first, which stays quiet, so that the choice a program makes on `is_empty()` (`x.is_empty() ? a : b` for an empty `x`), `x &= y` for an empty `y` and `floor()`, `max()`... of an empty interval raise nothing (the armhf jobs of the continuous integration and a POWER9 job under qemu check them, built for size too, where GCC calls the intersection rather than inlining it). GCC 9.4 at `-O3` computed the comparisons of the bounds of `x &= y` before its test of the empty set, in a loop of intersections: they are quiet ones. And GCC vectorizes loops of comparisons of bounds into vector comparisons that raise the exception for an empty element: GCC 9.4 and 13 where AVX is on (`-mavx`, or `-mfma`, which the three builds give where the processor has FMA) and GCC 13 to 15 for 32-bit x86 with SSE2 alone (MinGW-w64) make the quiet comparisons signaling predicates (`vcmpnlepd`, `cmpnlepd`...), GCC 14 for POWER8 uses `xvcmpgedp`, and GCC 11 to 15 for 64-bit ARM compute the second and third comparisons of the constructor for an empty element too. GCC 14 for RISC-

V made the comparison of the relations on the NaN bounds of an empty operand a signaling one, which is why the relations test the empty set first there (`GAOL_RELATION_BOUNDS_FIRST` 0). With GCC and the FPU or the NEON intervals, on every processor, the bounds `is_empty()` compares, those of the relations that do not call it (`straddles_zero()`, `set_contains(d)`...) and the doubles given to the constructors go through an empty asm statement, which keeps GCC from vectorizing such loops: loops of `x.set_contains(d)` and of `x.straddles_zero()` took 0.8 and 1.3 ns per element rather than 0.5 ns with GCC 13 on x86-64, about as long as with GCC 9.4, which did not vectorize them and took as long as before. A choice the program makes on a relation, `x.set_contains(y) ? a : b`, and a loop of relations other than those of `tests/rounding_direction.cpp` are not known to raise the exception, but nothing keeps a compiler from compiling them so. [GAOL v5]

- Nonempty operands raised the invalid-operation exception as well, and the cases found no longer do. With the SSE2 intervals, the products `interval(0) * interval::one_plus_infinity()` and `interval::positive() * interval(0)`, which multiply a zero bound by an infinite one, did (the FPU intervals gave the same products without it), and so did, through them, `pow` of the base `interval(1)` and the exponent `interval::one_plus_infinity()`; `pow` did in every build, through the `pow` of CORE-MATH, for an exponent of extreme magnitude (`pow(interval(1, 2), interval(4.9e-324))`, `pow(interval(1, 2), interval(1e-300))`, an exponent interval with a bound near `DBL_MAX` such as $[10^{300}, 10^{308}]$), which the CORE-MATH of GAOL v5 no longer does, and for a power next to the largest double (`pow(interval(2), interval(1023.9999))`, `pow(interval(0x1.000003f82a1fp+0), 3000000001LL)`), where CORE-MATH multiplied its first approximation by 2^{1024}, an infinity, and computed $\infty - \infty$ (changed in the vendored sources, see `3rd/README.md`); and with the SSE2 intervals, `div_rel(K, J, I)` did where it divided an infinite bound of `K` by an infinite bound of `J` in a half of the register it did not keep ($K = [2, +\infty]$ and $J = [-\infty, 2]$), and raised the division-by-zero exception where it divided a bound by a zero one there ($K = [-2, -2]$ and $J = [-2, 0]$), and so did the division, the relational division and the inverse of a divisor with a zero bound (`interval(1, 2)/interval(0, 2)`, `interval(1, 2)%interval(-2, 0)`, `inverse(interval(0, 2))`, `pow(interval(0, 2), -1)`). This list is not exhaustive. [GAOL v5]

- The overflow and the division-by-zero exceptions are no longer raised for a result that has no infinite bound. Some functions computed beyond the largest double on the way, or divided by zero, and raised them for a finite result, the empty set included: `sqrt(interval(1, DBL_MAX))`, `nth_root(interval(DBL_MAX), 3)` and the roots of order 4 and more, and the relational `sqrt_rel()` and `nth_root_rel()` (the product that proves a root exact is 2^{1024} for the square root of `DBL_MAX`), `sin(interval(-1, DBL_MAX))`

and `cos()` (the width of the interval), `tan(interval(DBL_MAX))` ($x + \pi/2$), `div_rel()` (the quotient by a divisor next to 2^{-1074}), `atan_rel()`, `atan2_rel_x()` and `atan2_rel_y()`, `pow_rel()` and `pow_exponent_rel()` (the powers beyond the largest double that their searches try, and the exponents next to 2^{-1074}), and `atanh_rel()` (atanh at -1 and 1). A quotient, a product or a power that would overflow is the infinity, or the largest double, that the operation gives, without being computed, where it is only an intermediate value: where they are raised, the exceptions stand for an infinite bound of the result, as above; an infinite bound does not always raise them (`interval(1, 2) / interval(0, 2)`, which is $[0.5, +\infty]$, raises none). This list is not exhaustive either, and the tests check these functions and the others of the same kind on operands next to the limits of the doubles (`tests/rounding_direction.cpp`). [GAOL v5]

An empty operand, and the nonempty operands tested, raising no invalid-operation exception therefore does not make that exception safe to leave enabled: every exception stays disabled while GAOL computes. [GAOL v5]

The flags tell nothing of the results: after most operations of GAOL, `fetestexcept(FE_INEXACT)` is raised whatever the result (not after `-x`, `abs()`, `&`, `|`, `floor()`, `max()` or `min()`, which round nothing and check no rounding direction, nor after `+`, `-`, `/` and `sqrt()` with the NEON intervals when their bounds are exact), and `FE_OVERFLOW` or `FE_DIVBYZERO`, where they are raised, stand for an infinite bound of the interval, or for the infinite width of a finite one (`interval(-DBL_MAX, DBL_MAX).width()`), not for an infinite number the program computed: `midpoint()` and `mid()` of [DBL_MAX, DBL_MAX], which overflowed the sum of the bounds, raise none. IEEE 1788-2015 [9] (1.4) leaves the flags of its operations unspecified. A program that reads them for its own computations clears them (`std::feclearexcept(FE_ALL_EXCEPT)`) just before, and reads them before its next operation of GAOL. [GAOL v5]

The initialization of GAOL, which runs before `main()`, sets the default environment (`fesetenv(FE_DFL_ENV)`), which masks every exception, unless GAOL is built with `GAOL_PRESERVE_ROUNDING`: the initialization then leaves the whole environment as it found it, and an exception a static object enabled before it stays enabled. A program enables the exceptions in `main()` or later, whichever the build, not in a static object initialized before GAOL. `cleanup()` leaves the exceptions and the flags as they are, and so does every operation, whichever way GAOL is built: it leaves the exceptions the program enabled enabled, and the flags it raised raised, and adds the flags it raises itself. With `GAOL_PRESERVE_ROUNDING` and the SSE2 intervals, `+`, `-`, `*`, `/`, `%`, `div_rel()`, `sqr()`, `inverse()`, the integer powers and `+= d`, `-= d`, `*= d`, `/= d`, `%= d` wrote the SSE control register whole, every exception masked and every flag cleared: an exception the program had enabled was masked again by the first of these operations, and the flags it had raised were lost (`tests/rounding_direction.cpp` checks it). [GAOL v5]

4.10 Initialization and cleanup

GAOL initializes itself when the program starts, before `main()`: `init()` need not be called. Since Release 1.0 of GAOL, there is an automatic initialization and cleanup, so that the operations of GAOL give their bounds even when the program calls neither function. Since GAOL v5, the initialization is made with every compiler, Visual C++ included: each file including the headers of GAOL holds a static object whose constructor initializes GAOL the first time, as `<iostream>` initializes the standard streams, so that GAOL is initialized before the static objects that such a file defines after its `#include`. Each operation also sets the rounding direction upward itself (see Section 4.1, page 88), so that the program does not need to set it again after changing it. [GAOL v5]

Remark 4.10.1 `cleanup()`*, on the other hand, is to be called by the program right after its last use of GAOL, at the latest at the end of* `main()`*, with every compiler.* [GAOL v5] *Unless GAOL is built to preserve the rounding direction, the initialization leaves the rounding direction upward for the whole program, not only for the operations of GAOL, and* `cleanup()` *sets back the one the program started with. The automatic cleanup does so too, but only when the program ends, once* `main()` *has returned and the static objects are destroyed, or when the library is unloaded: until then, the code after the last use of GAOL (the rest of* `main()`*, the destructors of static objects, the functions registered with* `std::atexit()`*) computes its doubles rounded upward.*

Example 4.10.2 *The program below computes an interval, calls* `cleanup()` *right after its last use of GAOL, and then prints a double of its own, which is computed to nearest. It calls no* `init()`*.*

Example

```
#include <cstdio>
#include <iostream>
#include <gaol/gaol.h>
using namespace gaol;

int main()
{
  // No gaol::init(): GAOL initialized itself before main()
  interval x(1, 2);
  std::cout << pow(x, 3) + sin(x) << std::endl;

  gaol::cleanup();   // always, right after the last use of GAOL
  std::printf("%g\n", 1.0 / 3.0);   // the program's own doubles, to nearest
  return 0;
}
```

Only the first call of `cleanup()` sets the direction back: an operation of GAOL after it sets the direction upward again, and `restore_rounding()` sets it back as

many times as the program needs. [GAOL v5] What the initialization allocated is freed by the automatic cleanup only, after the static objects are destroyed, so that the expressions of `gaol/parser/gaol_expression.h` still alive when the program calls `cleanup()` remain valid.

The direction set back is the one GAOL found when it initialized itself. It is not the one the program started with when an operation of GAOL came first: in the initialization of a static object of a file that uses GAOL without including its headers, through a library of the program for instance, or, with Clang, of an `inline` variable or a static member of a class template, whose initialization C++ does not order with the rest of the file.

In a build that takes the AVX-512 path (see Section 4.4, page 94), the initialization stops the program, with a message on the standard error, when the processor or the system does not run the AVX-512 instructions, even in a program that computes nothing. [GAOL v5]

The constants of GAOL (π, the masks of its SSE2 operations) are initialized when compiling: a static object of the program may compute intervals, before the files of GAOL are initialized, which with the static library of the CMake build come after those of the program, except with MinGW-w64. [GAOL v5]

`bool` **`init`**`(int` *`dbg_lvl`* `= 0)`

> Initializes the variable `debug_level` (see Section 13.6, page 269) to the value of *dbg_lvl*.
>
> Unless GAOL was built to preserve the rounding direction, the first call notes the rounding direction it finds, which `cleanup()` sets back, then sets the floating-point environment to its default one, then the rounding direction toward $+\infty$. In addition, it sets the number of digits to display for interval bounds to 16. In a build that takes the AVX-512 path, the first call stops the program, with a message on the standard error, when the processor or the system does not run the AVX-512 instructions, even in a program that computes nothing. [GAOL v5]
>
> Returns `true` if the library was not already initialized and `false` otherwise. After its first call, the function sets the debugging level to its new value and, unless the rounding direction is preserved, the rounding direction upward again: the C runtime of Windows resets it when the program starts, after the library initialized itself.

`bool` **`cleanup`**`(void)`

Unless GAOL was built to preserve the rounding direction, sets back the rounding direction that the first call of `init()` found: the one of the program when GAOL initialized itself, to nearest when the program starts. On x86 processors, the x87 unit and the SSE instructions each get theirs back. The rest of the floating-point environment is left as it is, the exception flags raised since in particular. The rounding direction being one of each thread, it is set back in the thread calling `cleanup()`. [GAOL v5]

The program calls it right after its last use of GAOL: an operation of GAOL after it sets the rounding direction upward again. The automatic cleanup calls it when the program ends, or when the library is unloaded, if the program did not, and frees what the initialization allocated, after the static objects of the program are destroyed: the expressions of `gaol/parser/gaol_expression.h` still alive when the program calls `cleanup()` remain valid.

Returns `true` the first time it is called and `false` afterwards: only the first call sets back the rounding direction.

Example

```
#include <gaol/gaol.h>

int main(void)
{
  gaol::init(1); // First level of debugging requested;
                 // without it, GAOL initializes itself

  [Some code using interval arithmetic]

  gaol::cleanup(); // Always, right after the last use
                   // of GAOL: sets back the rounding
                   // direction
  return 0;
}
```

`void` **`restore_rounding`**`(void)`

Sets back the rounding direction that the first call of `init()` found, as `cleanup()` does at its first call, but as many times as the program calls it. [GAOL v5] A program that goes back to GAOL after `cleanup()`, whose operations set the direction upward again, calls it after each use. Like `cleanup()`, it sets back the direction of the calling thread only, and leaves the rest of the floating-point environment as it is. With `GAOL_PRESERVE_ROUNDING` it does nothing, GAOL leaving the direction of the program alone (see Section 4.3, page 93).

4.11 The rounding functions of the FPU

GAOL provides functions to set the rounding direction of the floating-point unit. The operations of GAOL do not need them: each sets the rounding direction upward itself (see Section 4.1, page 88).

On x86 processors, with GCC, Clang and Visual C++, they write the control registers of the x87 and SSE units directly (`fnstcw`/`fldcw`, `stmxcsr`/`ldmxcsr`), rather than calling `fesetround()`, which cost 50 to 250 ns per call with the C runtime of Windows; with Visual C++ for x64, the SSE register only, the x87 unit being unused there (see Section 4.7, page 103). [GAOL v5] Elsewhere (ARM, POWER, s390x, RISC-V), they call `fesetround()`, except on 64-bit ARM processors with the NEON intervals, and on 32-bit ARM processors with GCC (see Section 4.5, page 96), where they write the rounding direction field of the control register, FPCR or FPSCR, themselves, and only when the direction changes. [GAOL v5]

`void` **`round_downward`**`(void)`

Sets the rounding direction toward $-\infty$.

`void` **`round_nearest`**`(void)`

Sets the rounding direction to nearest, ties to even.

`void` **`round_upward`**`(void)`

Sets the rounding direction toward $+\infty$.

4.12 Discussion

This chapter has established how GAOL v5 handles the rounding direction: each operation that computes a bound checks it, with an addition or a read of the control register, and sets it upward when it is not, so that the bounds are right whatever direction the calling code left, where GAOL 4 trusted the user to set it, or restored it after each operation but in the x87 control word only on x86-64. The price of this choice falls on the doubles of the program, which are rounded upward from the initialization of GAOL until `cleanup()`: the quotients, the decimal conversions of the C library, the sums and the error-free transformations all change, and the program answers by calling `cleanup()` right after its last use of GAOL, by reading and writing its numbers through GAOL, and by computing to nearest in a guarded block what needs it. The build with `GAOL_PRESERVE_ROUNDING` removes the price from the program and puts it on the operations, 1.2 to 1.6 times slower with the AVX-512 path and several times without it; the control registers are then written

only where they have to be, and the AVX-512 instructions, whose rounding direction is in the instruction, are taken where the machine has them.

The same check defends the bounds against the modes that flush the subnormal numbers to zero, which a program linked with `-Ofast` gets without asking, and which GAOL clears on x86 and on ARM but for the FPU intervals of Visual C++, while `-mno-daz-ftz` keeps them out of the link where the compiler accepts it. The floating-point exceptions, on the other hand, are left to the program, which disables them while GAOL computes: the operations raise the inexact flag almost always, the overflow and division-by-zero flags for some infinite bounds, and no longer the invalid-operation one for an empty operand in the cases found, but nothing keeps a compiler from compiling a quiet comparison into a signaling one. Several points stay open in the sources: the other processors (POWER, s390x, RISC-V, MIPS) have no defence against their flush modes, and nothing was tested with them set; the FPU intervals of POWER, s390x, RISC-V and MinGW-w64 for arm64 are not measured; the slower line of 13 operations with the direction preserved on Visual C++ for x86 has no explanation; and the processors that do not halve their AVX-512 instructions are expected, not measured, to take less time on that path.

With the floating-point environment in place, the next chapters turn to the intervals themselves: Chapter 5 describes how an interval is created, from numbers or from text, and assigned.

Chapter 5

Creating intervals

An interval of GAOL is an object of the class `interval`. A program creates it from its two bounds, from one number, from nothing, from another interval, or from the text that writes it; it can also build it from a midpoint and a radius. Once created, an interval receives new values by assignment, alone or combined with an operation, and a few intervals of constant use, the empty set, the whole real line, the enclosures of π, are provided ready-made by the class.

The way a value enters an interval is not a detail. A `double` is taken as it is, an integer as the integer it is, which may be no double, and a string as the interval it writes, rounded outward. A decimal constant written in the program, as 0.1, is rounded by the compiler before it reaches the constructor, and an interval built from it does not contain the number the programmer wrote. The constructors of GAOL v5 answer every input that makes no interval, bounds in the wrong order, a NaN bound or an infinite point, with the empty set, and they refuse the silent conversions that GAOL 4 accepted, from a `double` and from a string. Each of these differences is marked in the text.

In this chapter, we describe the creation of intervals and their assignment. The first section presents the four constructors, the function `textToInterval()`, the static method `midrad()` and the intervals of one infinite point, which GAOL v5 does not have. The second section covers the straight assignment, the third the assignment combined with an operation (`+=`, `&=`...) and the fourth the interval constants of the class. The format of the strings that `textToInterval()` reads is the subject of Chapter 9.

5.1 Constructors

The class `interval` has four constructors for a program. The SSE2 and NEON intervals have one more, for the code of GAOL (see Section 12.1, page 228): `interval(const __m128d&)` or `interval(const float64x2_t&)`, explicit, which takes the register that holds the stored bounds, the lower bound negated in the first lane, and gives the empty set where a lane holds $-\infty$. Besides the constructors,

a program creates an interval from the text that writes it with the function `textToInterval()`, and from a midpoint and a radius with the static method `interval::midrad()`. We describe the constructors first, then the two functions, and we end with the intervals of one infinite point.

5.1.1 The four constructors

- **From the two bounds.** The left and the right bounds are given:

Example

```
1    interval x(l,r);
```

where l and r are `double`s or of a type that is castable into a `double`. The result is the empty set when $[l, r]$ is not an interval: a left bound $+\infty$, a right bound $-\infty$, bounds in the wrong order ($l > r$) and NaN bounds all give the empty set. [GAOL v5] An integer bound of any integer type but `bool`, the other bound being an integer, a `double` or a `float`, is taken as the integer it is: the left bound is the largest double at most l, the right one the smallest double at least r. Thus `interval(1, 9007199254740993LL)` contains $2^{53}+1$, which no double is, and which its conversion to a double made 2^{53} or $2^{53}+2$ according to the rounding direction. The result is the empty set for $l > r$ compared as numbers: `interval(9007199254740993LL, 9007199254740992LL)` is empty, though both bounds lie between the same two doubles. [GAOL v5]

- **From one number.** A degenerate interval, a point, is built from one bound:

Example

```
1    interval x(v);
```

This is equivalent to `interval x(v,v);`, and gives the empty set for an infinite or NaN v. The constructor is `explicit`: a `double` no longer converts silently to an interval. The statements `interval x = 0.5;`, `return 0.5;` in a function returning an interval, `gaol::sin(0.5)` and `min(x, 1.0)` do not compile, where `interval x(0.5)`, `interval(0.5)`, `sin(interval(0.5))` and `min(x, interval(1.0))` do. The assignment (see Section 5.2, page 123), the operators `&=`, `|=` and the other assignments combined with an operation (see Section 5.3, page 123), the arithmetic operators and the relation symbols (see Section 6.5, page 138) take a `double` as they take `interval(d)`. [GAOL v5] An integer n of any integer type but `bool` gives the tightest interval containing it: $[n, n]$ where n is a double, as every `int` is, and the two doubles around it otherwise. `interval(9007199254740993LL)` is $[2^{53}, 2^{53}+2]$, where the conversion of $2^{53}+1$ to a double gave $[2^{53}, 2^{53}]$ or $[2^{53}+2, 2^{53}+2]$, neither of which contains it, and `interval(ULLONG_MAX)` contains $2^{64}-1$. This constructor is `explicit` too. The same operations take an integer as they take `interval(n)`; the relations compare it as the integer it is (see Section 6.5, page 138), and `&=` gives `interval(n)` where the interval contains n, the empty set otherwise. [GAOL v5]

- **Without any bound.** The default constructor builds the whole real line:

Example

```
1    interval x;
```

This is equivalent to:

Example

```
1    interval x(-GAOL_INFINITY,+GAOL_INFINITY);
```

where `GAOL_INFINITY` is the infinity of the `double` format (see Section 13.1, page 261).

- **By copy.** The copy constructor duplicates an existing interval:

Example

```
1    interval x(-12,12), y=x, z(x);
```

Remark 5.1.1 *The integers are taken as the integers they are whatever the rounding direction, and for every integer type but* `bool`. [GAOL v5] *Three cases still convert to a* `double`*: an unscoped enumeration, an* `__int128` *outside the GNU modes of GCC and Clang with libstdc++, and a* `long double` *bound next to an integer, which takes* `interval(double, double)`*. An integer that is a double gives the result the double gives, and, for the types of 53 bits or less (*`int`*,* `short`*,* `char`*...), through the same code. The constructor* `interval(n)` *is* `explicit`*, as the constructor from a* `double`*. The other operations of GAOL that take an integer are described in Section 3.6, page 72.*

5.1.2 Intervals read from a string

The function `textToInterval()` reads a string that writes an interval in the format of the input (see Section 9.1.1, page 184), and returns that interval. It takes the place of the constructors from strings of GAOL 4.

`interval` **`textToInterval`**`(const std::string& s)`

Returns the interval the string s writes, read with the names of the functions of GAOL (see Section 9.1.1, page 184). It is in the namespace `gaol`, under the name IEEE 1788-2015 gives it.

Example

```
1 interval x = textToInterval("[-23, inf]"),
2          y = textToInterval("[5*0.1+dmin, 89*sinh(2.1)]");
```

If the string does not comply with the expected format, the exception `std::invalid_argument` is thrown, and `std::domain_error` if it calls a function with an argument the function does not take, as `"nth_root(8, 1.5)"`, when the library was compiled with exception support, which is the default of the three builds. Without exception support, the function `gaol_error()` prints an error message on the standard error stream, and the program aborts (Chapter 13). The `textToInterval()` of `gaol_ieee1788` reads the names of the functions of the standard, and returns the empty set instead (Chapter 10). A null pointer is no string: the overload for a `const char*`, which `textToInterval(nullptr)` reaches, throws `std::invalid_argument` in both namespaces, and gives the same error without exception support (a message, then `abort()`). [GAOL v5]

GAOL v5 has no constructor from one string. `textToInterval()` takes the place of the constructor `interval(const char*)` of GAOL 4, which converted any `const char*` implicitly: `interval(0)` was ambiguous, and `x + p` compiled for a null `p`, and crashed. [GAOL v5]

`interval textToInterval(const std::string& sl, const std::string& sr)`

Returns the interval whose left bound is the one of the interval the string *sl* writes, and whose right bound is the one of the interval *sr* writes. The result is the empty set when the lower bound of the first is above the upper bound of the second. The comparison is between the intervals the two strings write, each rounded outward, not between the numbers, where `textToInterval("[l,u]")` orders the numbers.

Example

```
1 // x=[-4,4]
2 interval x = textToInterval("[-5,4]+1","[4,6]-[2,3]");
```

It is in the namespace `gaol`, and throws `std::invalid_argument` as `textToInterval(s)` does, for a null pointer in either place too. It takes the place of the constructor from two strings of GAOL 4, which kept the bounds [2, 1] of `interval("2", "1")`: `is_empty()` was true, but its sum with [0, 1] was [2, 2]. [GAOL v5]

5.1.3 A midpoint and a radius

The static method `interval::midrad()` builds an interval from its midpoint and its radius, both given as `double`s.

`interval interval::midrad(double m, double r)` *static*

■ `midrad`$(m, r) = [\downarrow m - r\downarrow, \uparrow m + r\uparrow]$

Returns the interval of midpoint m and radius r, computed as `interval(m) + interval(-r, r)`. It encloses every real of $[m - r, m + r]$, where `interval(m - r, m + r)` rounds both bounds in the same direction, upward unless GAOL is built with `GAOL_PRESERVE_ROUNDING`. A radius $r = 0$ gives the point m, and $r = +\infty$ the whole line. A negative or NaN r gives the empty set, and so does an infinite or NaN m, as `interval(m)` does. [GAOL v5]

Example

```
1 cout << interval::midrad(1, 0.25) << ' '
2      << interval::midrad(1, -1);
3 ➥ [0.75, 1.25] [empty]
```

5.1.4 Decimal constants

Of the three forms in which a number reaches an interval, a `double`, an integer and a string, the `double` is the usual one, and the one that calls for care.

Remark 5.1.2 *A rational number with a finite expansion in the decimal base may need a rounding in the binary base. The statement*

Example

```
interval x(0.1);
```

does not *create an interval containing* 0.1*. This number has an infinite expansion in base two, so no significand, whatever its size, represents it exactly, and the constant* 0.1 *is rounded, in all likelihood to the nearest floating-point number, when the program is compiled (see Section 1.1, page 6, and Section 3.10, page 77). The interval that contains* 0.1 *is read from a string:*

Example

```
interval x = textToInterval("0.1");
```

5.1.5 Infinite point intervals

The degenerate intervals $[+\infty, +\infty]$ and $[-\infty, -\infty]$ do not exist in GAOL v5. [GAOL v5] IEEE 1788-2015 [9] has no such interval: its constructor fails there and gives the empty set (clauses 10.5.8 and 12.12.7). The constructors of GAOL give the empty set too, as those of IBEX do, for an infinite point and for all the cases listed above:

Example

```
cout << interval(GAOL_INFINITY) << ' '
     << interval(-GAOL_INFINITY,-GAOL_INFINITY) << ' '
     << interval(2,1) << ' '
     << interval(1,GAOL_INFINITY) << endl;
➥ [empty] [empty] [empty] [1, inf]
```

GAOL 4 built them as intervals of one infinite point, which its operators did not support. The intervals with one infinite bound, as $[1, +\infty]$, are intervals as any other.

Read from a string, the constant **inf**, an expression, stands for [MAX, $+\infty$], MAX being the largest double, and **-inf** for $[-\infty, -\text{MAX}]$. The interval literals whose left bound is $+\infty$ or whose right bound is $-\infty$ are the empty set, as numsToInterval of IEEE 1788-2015 has it:

Example

```
cout << textToInterval("inf") << ' ' << textToInterval("[inf]") << ' '
     << textToInterval("[-inf, -inf]") << endl;
➥ [1.797693134862315e+308, inf] [empty] [empty]
```

The string **"<-inf, -inf>"**, which GAOL 4 read as $[-\infty, -\text{MAX}]$, throws `std::invalid_argument`: a bound of `<a, b>` must have a double for its value, and the value of **-inf** is $[-\infty, -\text{MAX}]$, no point (see Section 9.1.1, page 184). [GAOL v5]

5.2 Straight assignment

An existing interval receives a new value from another interval, from a string, from a `double`, or from an integer (below).

- **From another interval.** The assignment copies it:

Example

```
interval x(-12,12),
         y; // Here, y is [−∞, +∞]
y = x; // Now, y is [−12, 12]
```

- **From a string.** The string follows the format expected for the input (see Section 9.1, page 183), and `textToInterval()` reads it:

Example

```
interval x;
x = textToInterval("[-inf, 123]");
```

GAOL 4 assigned the string itself, `x = "[-inf, 123]"`. GAOL v5 refuses this statement, having no constructor from one string. [GAOL v5]

- **From a `double`.** The operator `operator=(double)` assigns it as `interval(d)`, the constructor from one `double` being explicit: [GAOL v5]

Example

```
interval x;
x = 1234.5; // Now, x is [1234.5, 1234.5]
x = GAOL_INFINITY; // Now, x is empty
```

The assignments from a string and from a `double` follow the rules of Section 5.1, page 117: assigning an infinity to an interval empties it. An integer n is assigned as `interval(n)`, the tightest interval containing it. [GAOL v5]

Remark 5.2.1 *The interface has no method for modifying one bound of an interval: an interval is created, assigned and replaced as a whole. (The non-const accessors* `get_xmminterval()` *and* `get_neoninterval()` *of the SSE2 and NEON intervals hand the register of the stored bounds to the code of GAOL; a program does not write it.)*

5.3 Assignment combined with an operation

In this manual, we write `self` for the object to which `this` is a pointer in C++. The assignment operators below combine the value of `self` with the value of their right-hand side, an interval, a `double` or an integer, and store the result in `self`.

`interval& interval::operator&=(const interval& I)`

`interval& interval::operator&=(double d)`

- `self` $\leftarrow$ `self` $\cap\, I$
- `self` $\leftarrow$ `self` $\cap$ `interval`(d)

Assigns to `self` the interval resulting from the intersection of `self` and I (resp. `interval(d)`, empty for an infinite or NaN d), which is the empty set when they are disjoint. [GAOL v5]

Example

```
1 interval x(-12,12);
2 x &= interval(-6,23); // Now, x is [−6, 12]
```

`interval& interval::operator|=(const interval& I)`
`interval& interval::operator|=(double d)`

- `self` $\leftarrow \square($`self` $\cup\, I)$
- `self` $\leftarrow \square($`self` $\cup$ `interval`$(d))$

Assigns to `self` the interval hull of the union of `self` and I (resp. `interval(d)`, which leaves `self` as it is for an infinite or NaN d). [GAOL v5]

Example

```
1 interval x(-12,12);
2 x |= interval(-6,23); // Now, x is [−12, 23]
```

`interval& interval::operator+=(const interval& I)`
`interval& interval::operator+=(double d)`

- `self` $\leftarrow$ `self` $+\, I$
- `self` $\leftarrow$ `self` $+\, d$

Assigns to `self` the interval resulting from adding `self` and I (resp. d).

Example

```
1 interval x(-12,12);
2 x += interval(-6,23); // Now, x is [−18, 35]
```

`interval& interval::operator-=(const interval& I)`
`interval& interval::operator-=(double d)`

- `self` $\leftarrow$ `self` $-\, I$
- `self` $\leftarrow$ `self` $-\, d$

Assigns to `self` the interval resulting from subtracting I (resp. d) from `self`.

Example

```
1 interval x(-12,12);
2 x -= interval(-6,23); // Now, x is [−35, 18]
```

`interval& interval::operator*=(const interval& I)`
`interval& interval::operator*=(double d)`

- `self` $\leftarrow$ `self` $\times I$
- `self` $\leftarrow$ `self` $\times d$

Assigns to `self` the interval resulting from multiplying `self` and I (resp. d).

Example

```
1 interval x(-12,12);
2 x *= interval(-6,23); // Now, x is [−276, 276]
```

`interval& interval::operator/=(const interval& I)`
`interval& interval::operator/=(double d)`

- `self` $\leftarrow$ `self`$/I$
- `self` $\leftarrow$ `self`$/d$

Assigns to `self` the interval resulting from dividing `self` by I (resp. d).

Example

```
1 interval x(-12,12);
2 x /= interval(-6,23);   // Now, x is [−∞, +∞]
3 x /= interval::zero();  // Now, x is ∅
```

`interval& interval::operator%=(const interval& I)`
`interval& interval::operator%=(double d)`

Assigns to `self` the interval resulting from dividing `self` by I (resp. d), using a *relational division* (see Section 7.3, page 160).

Example

```
1 interval x(-12,12);
2 x %= interval(-6,23);   // Now, x is [−∞, +∞]
3 x %= interval::zero();  // Now, x is [−∞, +∞]
```

The arithmetic assignments with a `double` (`+=`, `-=`, `*=`, `/=`, `%=`) give the result of the same operation with `interval(d)`, choosing the case by the sign of d; an infinite or NaN d gives the empty set, as `interval(d)` is empty. [GAOL v5]

5.4 Interval constants

Some useful intervals, and some canonical intervals enclosing real constants, are defined as static methods of the class `interval`. Table 5.1 lists them.
The enclosures of π, 2π and $\pi/2$ are the tightest ones (see Section 1.8, page 20).
The empty set is held as an interval whose two bounds are NaN.

TABLE 5.1 – The interval constants, static methods of the class `interval`

Function	Value
`interval::emptyset()`	$\varnothing$
`interval::half_pi()`	$[\downarrow\frac{\pi}{2}\downarrow, \uparrow\frac{\pi}{2}\uparrow]$
`interval::minus_one_plus_one()`	$[-1, 1]$
`interval::negative()`	$[-\infty, 0]$
`interval::one()`	1
`interval::one_plus_infinity()`	$[1, +\infty]$
`interval::pi()`	$[\downarrow\pi\downarrow, \uparrow\pi\uparrow]$
`interval::positive()`	$[0, +\infty]$
`interval::two_pi()`	$[\downarrow 2\pi\downarrow, \uparrow 2\pi\uparrow]$
`interval::universe()`	$[-\infty, +\infty]$
`interval::zero()`	0

Example

```
cout << interval::emptyset() << ' ' << interval::pi() << endl;
➥ [empty] [3.141592653589793, 3.141592653589794]
```

5.5 Discussion

This chapter described the ways an interval of GAOL comes to exist and to change: four constructors, the function `textToInterval()` for the text of an interval, the static method `midrad()` for a midpoint and a radius, the straight assignment, the assignments combined with an operation, and the constants of the class. One rule runs through the constructors, `midrad()` and the assignments: an input that makes no interval, bounds in the wrong order, a NaN bound or an infinite point, gives the empty set rather than an object the operators do not support. GAOL v5 follows IEEE 1788-2015 and IBEX there, where GAOL 4 built intervals of one infinite point. `textToInterval()` gives the empty set too for a literal such as `"[2, 1]"` or `"[inf]"`, and throws for a string that writes no interval, as `"<-inf, -inf>"`, whose bound `-inf` is no double.

Two choices of GAOL v5 cost the programmer a few characters and remove ambiguities. The constructors from one `double` and from one integer are explicit, so that a number no longer becomes an interval without being asked to. The constructors from strings of GAOL 4 are replaced by `textToInterval()`, so that `interval(0)` is no longer ambiguous and a null pointer no longer compiles into a crash. An integer bound is taken as the integer it is, whatever the rounding direction, which keeps $2^{53} + 1$ inside the interval that encloses it. The decimal constants remain the responsibility of the programmer: the way to an interval containing 0.1 is a string.

The intervals created here are compared in the next chapter, which describes the set relations, the certainly relations and the order of IEEE 1788-2015 on intervals (Chapter 6).

Chapter 6

Interval relations

Comparing two intervals is not comparing two numbers. An interval stands for a set of reals, and a relation between two such sets can be read in two ways: as a relation between the sets themselves, the equality or the inclusion, or as a relation that every point of the first set bears to every point of the second. A program of interval analysis needs both readings. It asks whether a box is included in another one, whether the enclosure of a function lies certainly below a bound before it prunes a branch, or whether an interval contains zero before it divides by it. GAOL v5 provides each reading as a named method of the class `interval`, and gives the relation symbols of C++ one fixed meaning.

The relations of GAOL v5 follow IEEE 1788-2015 [9], which defines the comparisons of intervals, their value for the empty set and for the infinite bounds, and a primitive, overlap, which tells in which of sixteen states two intervals are and from which most of the comparisons can be derived. GAOL 4 had a larger family of relations and a build option that chose the meaning of the symbols. GAOL v5 keeps two groups of relations and no option.

In this chapter, we first define the two groups, the set relations and the certainly relations, and we name the relations of the standard (Section 6.1). Sections 6.2 and 6.3 are the reference of the set relations and of the certainly relations. Section 6.4 covers the order of IEEE 1788-2015 and the overlap state. Section 6.5 gives the meaning of the relation symbols, Section 6.6 explains why they are no ordering for the containers of the standard library and which comparator to give them, and Section 6.7 lists the relations that test one interval: whether it contains zero, is a point, is empty or is bounded.

6.1 Two groups of relations

Let I and J be two intervals. The relations of GAOL v5 fall into two groups, according to what they compare.

Definition 6.1.1 *A set relation compares the intervals I and J as sets of reals. The equality, for instance, is*

$$I = J \Leftrightarrow (\forall x \in I, \exists y \in J : x = y) \wedge (\forall y \in J, \exists x \in I : x = y),$$

so that two nonempty intervals are equal when they have the same bounds, and the empty set is equal to itself alone.

Definition 6.1.2 *A certainly relation holds between I and J when the relation on the reals holds for every pair of values taken in the intervals. For instance,*

$$I \leqslant J \Leftrightarrow (\forall x \in I, \forall y \in J : x \leqslant y),$$

so that I is less than or equal to J when its right bound is at most the left bound of J.

Remark 6.1.3 *GAOL 4 had a third group, the possibly relations, true when at least one pair of values satisfies the relation on the reals, and, among the certainly relations, an equality and an inequality,* `certainly_eq()` *and* `certainly_neq()`*. They are gone from GAOL v5, and so are the operators* `==` *and* `!=` *on intervals.* [GAOL v5] `certainly_neq()` *was the negation of* `certainly_eq()`*, which is "possibly not equal", so that* $[3, 4]$ *was certainly not equal to* $[3, 4]$*. IEEE 1788-2015 has neither. Its equality is* equal*,* `set_eq()`*; that no value of I equals a value of J is* disjoint*,* `set_disjoint()`*; and "possibly equal" is written* `!set_disjoint()`*.*

The relation symbols `<`, `<=`, `>` and `>=` are the certainly relations in every build (see Section 6.5, page 138).

The relations that IEEE 1788-2015 defines (Tables 10.3 and 10.4) follow its definitions, for the empty set and for the infinite bounds too: [GAOL v5] equal, subset, interior, disjoint, precedes, strictPrecedes, less and strictLess. Table 6.1 gives the method of GAOL v5 that each one is, with the predicates of the standard on one interval that this chapter covers; each entry of the reference says which name it bears. The namespace `gaol_ieee1788` gives the methods the names of the standard (Chapter 10).

6.2 Set relations

The set relations compare the intervals as sets of reals: the inclusion, the inclusion in the interior, the disjointness and the equality. They are subset, interior, disjoint and equal of IEEE 1788-2015. Each inclusion has two methods: one where `self`, the interval on which the method is called, is the larger set, `set_contains()` and `set_strictly_contains()`, and one where it is the smaller, `set_leq()` and `set_le()`. `set_geq()` and `set_ge()` are `set_contains()` and `set_strictly_contains()` under the names of an order.

TABLE 6.1 – The relations of IEEE 1788-2015 and the methods of GAOL v5 that compute them, for two intervals I and J and a double d

IEEE 1788-2015	GAOL v5	Symbol
equal(I, J)	`I.set_eq(J)`	
subset(I, J)	`I.set_leq(J)`, `J.set_contains(I)`, `J.set_geq(I)`	
interior(I, J)	`I.set_le(J)`, `J.set_strictly_contains(I)`, `J.set_ge(I)`	
disjoint(I, J)	`I.set_disjoint(J)`	
precedes(I, J)	`I.certainly_leq(J)`	`I <= J`
strictPrecedes(I, J)	`I.certainly_le(J)`	`I < J`
less(I, J)	`I.less(J)`	
strictLess(I, J)	`I.strictly_less(J)`	
overlap(I, J)	`I.overlap(J)`	
isMember(d, I)	`I.set_contains(d)`, for a finite `double` d	
isSingleton(I)	`I.is_a_double()`	
isEmpty(I)	`I.is_empty()`	
isEntire(I)	`I.is_entire()`	
isCommonInterval(I)	`I.is_common_interval()`	

```
bool interval::set_contains(const interval& I) const
bool interval::set_contains(double d) const
```

- $I \subseteq$ `self`
- $d \in$ `self`

Returns `true` if I (resp. $\{d\}$) is included in `self`: subset(I, `self`) of IEEE 1788-2015, and isMember(d, `self`) for a finite d. d shall not be a NaN. An integer of any integer type but `bool` is compared as the integer it is, here and in `set_strictly_contains()` (see Section 3.6, page 72): $[2^{53}, 2^{53}]$ does not contain $2^{53}+1$, which its conversion to a double made 2^{53} in the rounding to nearest, and $[2^{53}, 2^{53}+2]$ contains it strictly. [GAOL v5]

Example

```
1 interval x(-12,34), y(-12,5);
2
3 cout << boolalpha << x.set_contains(y) << ' '
4      << x.set_contains(interval::emptyset()) << ' '
5      << y.set_contains(x) << ' '
6      << interval::emptyset().set_contains(x) << ' '
7      << interval::emptyset().set_contains(interval::emptyset())
8      << endl;
9 ➥ true true false false true
```

`bool interval::set_strictly_contains(const interval& I) const`
`bool interval::set_strictly_contains(double d) const`

- I interior to `self`
- d in the interior of `self`

Returns `true` if I is interior to `self`: interior(I, `self`) of IEEE 1788-2015. Each bound of `self` is then beyond that of I, an infinite bound being beyond the same infinite bound, and the empty set is interior to any interval, itself included. [GAOL v5] The second form returns `true` if d is strictly between the bounds of `self`.

Example

```
interval x(-10,12), y(-10, 11), z, t, u(10.5);

cout << boolalpha
     << x.set_strictly_contains(y) << ' '
     << z.set_strictly_contains(t) << ' '
     << x.set_strictly_contains(u) << ' '
     << interval::emptyset().set_strictly_contains(
                                 interval::emptyset()) << ' '
     << u.set_strictly_contains(interval::emptyset())
     << endl;
➥ false true true true true
```

`bool interval::set_disjoint(const interval& I) const`

- `self` $\cap I = \varnothing$

Returns `true` if the intersection of `self` and I is empty: disjoint of IEEE 1788-2015.

Example

```
interval a(2,4), b(6,8);
cout << boolalpha << a.set_disjoint(b) << ' '
     << interval::emptyset().set_disjoint(interval::emptyset());
➥ true true
```

`bool interval::set_eq(const interval& I) const`

- $\forall x \in$ `self`$, \exists y \in I : x = y \wedge \forall y \in I, \exists x \in$ `self`$: y = x$

Returns `true` if the intervals `self` and I are equal when considered as sets of reals: equal of IEEE 1788-2015.

Example

```
cout << boolalpha << interval(4,8).set_eq(interval(4,8)) << ' '
     << interval::emptyset().set_eq(interval::emptyset()) << endl;
➥ true true
```

`bool interval::set_neq(const interval& I) const`

■ $\neg$(`self` $= I$)

Returns `true` if `self` and I are not equal when considered as sets of reals.

Example

```
cout << boolalpha << interval::universe().set_neq(interval::emptyset());
➥ true
```

`bool interval::set_le(const interval& I) const`

■ `self` interior to I

Returns `true` if `self` is interior to I: `I.set_strictly_contains(self)`, interior(`self`, I) of IEEE 1788-2015. [GAOL v5]

Example

```
interval a(2,GAOL_INFINITY), b(1,GAOL_INFINITY);
cout << boolalpha << interval(-4.5,3).set_le(interval(-10,10)) << ' '
     << interval::emptyset().set_le(interval(5,6)) << ' '
     << interval::emptyset().set_le(interval::emptyset()) << ' '
     << interval::universe().set_le(interval::universe()) << ' '
     << a.set_le(b);
➥ true true true true true
```

`bool interval::set_leq(const interval& I) const`

■ `self` $\subseteq I$

Returns `true` if the real set defined by `self` is included in I: subset(`self`, I) of IEEE 1788-2015.

Example

```
cout << boolalpha << interval(4.5,6).set_leq(interval(4.5,6)) << ' '
     << interval(3.5,9).set_leq(interval(2,6)) << ' '
     << interval::emptyset().set_leq(interval::emptyset());
➥ true false true
```

`bool interval::set_ge(const interval& I) const`

■ I interior to `self`

Returns `true` if I is interior to `self`: `set_strictly_contains(I)`, interior(I, `self`) of IEEE 1788-2015. [GAOL v5]

Example

```
cout << boolalpha << interval(-10,10).set_ge(interval(-4.5,3)) << ' '
     << interval(1,2).set_ge(interval(1,2));
➥ true false
```

`bool interval::set_geq(const interval& I) const`

■ $I \subseteq$ `self`

Returns `true` if the real set defined by `self` contains I: `set_contains(I)`, subset(I, `self`) of IEEE 1788-2015.

6.3 Certainly relations

A certainly relation holds when every pair of points, one in each interval, satisfies the relation on the reals. The empty set has no point to contradict it: the four relations of this section are true when either interval is empty, as precedes and strictPrecedes of IEEE 1788-2015 are. [GAOL v5] The four tests of sign that follow them, `certainly_positive()` and its companions, are the certainly relations with zero, written as the inclusion of `self` in a half line; they are true of the empty set too.

`bool interval::certainly_le(const interval& I) const`

■ $\forall x \in$ `self`, $\forall y \in I$: $x < y$

Returns `true` if `self` is certainly strictly less than I: strictPrecedes of IEEE 1788-2015, true when either interval is empty. [GAOL v5]

Example

```
cout << boolalpha << interval(4,5).certainly_le(interval(6,9)) << ' '
     << interval(4,5).certainly_le(interval(5,9)) << ' '
     << interval::emptyset().certainly_le(interval(4,6)) << ' '
     << interval(4,6).certainly_le(interval::emptyset());
➥ true false true true
```

`bool interval::certainly_leq(const interval& I) const`

■ $\forall x \in$ `self`, $\forall y \in I$: $x \leqslant y$

Returns `true` if `self` is certainly less than or equal to I: precedes of IEEE 1788-2015, true when either interval is empty. [GAOL v5]

Example

```
cout << boolalpha << interval(4,5).certainly_leq(interval(6,9)) << ' '
     << interval(4,5).certainly_leq(interval(5,9)) << ' '
     << interval(5,9).certainly_leq(interval(4,5)) << ' '
     << interval(4,8).certainly_leq(interval(5,9)) << ' '
     << interval::emptyset().certainly_leq(interval(4,6)) << ' '
     << interval(4,6).certainly_leq(interval::emptyset());
➥ true true false false true true
```

`bool interval::certainly_ge(const interval& I) const`

■ $\forall x \in$ `self`, $\forall y \in I$: $x > y$

Returns `true` if `self` is certainly strictly greater than I, or if either interval is empty. [GAOL v5]

Example

```
cout << boolalpha << interval(8,10).certainly_ge(interval(4,8)) << ' '
     << interval(8,10).certainly_ge(interval(4,7)) << ' '
     << interval::emptyset().certainly_ge(interval::emptyset());
➥ false true true
```

`bool interval::certainly_geq(const interval& I) const`

■ $\forall x \in$ `self`, $\forall y \in I$: $x \geqslant y$

Returns `true` if `self` is certainly greater than or equal to I, or if either interval is empty. [GAOL v5]

Example

```
cout << boolalpha << interval(8,10).certainly_geq(interval(4,8)) << ' '
     << interval::emptyset().certainly_geq(interval::emptyset());
➥ true true
```

`bool interval::certainly_positive(void) const`

■ `self` $\subseteq [0, +\infty]$

Returns `true` if the left bound of `self` is greater than or equal to zero, or if `self` is empty.

Example

```
cout << boolalpha << interval::emptyset().certainly_positive() << ' '
     << interval(4,5).certainly_positive() << ' '
     << interval(-0.0,6).certainly_positive() << ' '
     << interval(-6,0).certainly_positive();
➥ true true true false
```

`bool interval::certainly_strictly_positive(void) const`

■ `self` $\subseteq]0, +\infty]$

Returns `true` if the left bound of `self` is strictly greater than zero, or if `self` is empty.

Example

```
cout << boolalpha
     << interval::emptyset().certainly_strictly_positive() << ' '
     << interval(4,5).certainly_strictly_positive() << ' '
     << interval(-0.0,6).certainly_strictly_positive() << ' '
     << interval(-6,0).certainly_strictly_positive();
➥ true true false false
```

`bool interval::certainly_negative(void) const`

■ `self` $\subseteq [-\infty, 0]$

Returns `true` if the right bound of `self` is less than or equal to zero, or if `self` is empty.

Example

```
cout << boolalpha << interval::emptyset().certainly_negative() << ' '
     << interval(4,5).certainly_negative() << ' '
     << interval(-6,0).certainly_negative() << ' '
     << interval(-6,-5).certainly_negative();
➥ true false true true
```

`bool interval::certainly_strictly_negative(void) const`

■ `self` $\subseteq [-\infty, 0[$

Returns `true` if the right bound of `self` is strictly less than zero, or if `self` is empty.

Example

```
cout << boolalpha
     << interval::emptyset().certainly_strictly_negative() << ' '
     << interval(4,5).certainly_strictly_negative() << ' '
     << interval(-6,0).certainly_strictly_negative() << ' '
     << interval(-6,-5).certainly_strictly_negative();
➥ true false false true
```

6.4 The order of IEEE 1788-2015 and the overlap state

Besides the set relations and the certainly relations, IEEE 1788-2015 defines an order on the intervals, less and strictLess (Table 10.3), which compares the bounds on the same side: I is less than J when the left bound of I is at most the left bound of J and the right bound of I is at most the right bound of J. The standard also defines a primitive, overlap (10.6.4), which tells in which of sixteen states two intervals are. A note of the standard says that every comparison it requires can be implemented from it with bit masks, but three of them also need the bounds. [1, 1] and [1, 2] are in the state **`starts`**, as [1, 1.5] and [1, 2], and only the first pair is in precedes. $[-\infty, 1]$ and $[-\infty, 2]$ are in **`starts`** too, and are in interior and in strictLess, where [1, 1.5] and [1, 2] are in neither. Three of the states are for the empty set. The thirteen others are the relations of Allen between two nonempty intervals $a = [\underline{a}, \overline{a}]$ and $b = [\underline{b}, \overline{b}]$, told by their bounds, an infinite bound being equal to the same infinite bound and -0 to $+0$ (Table 6.2). Of the three methods of this section, `less()` and

TABLE 6.2 – The 13 states of Allen for two nonempty intervals a and b, as `a.overlap(b)` returns them (Table 10.7 of IEEE 1788-2015); each state of the left column is the inverse of the one on its right

State	Bounds	State	Bounds
before	$\overline{a} < \underline{b}$	**after**	$\overline{b} < \underline{a}$
meets	$\underline{a} < \overline{a} = \underline{b} < \overline{b}$	**metBy**	$\underline{b} < \overline{b} = \underline{a} < \overline{a}$
overlaps	$\underline{a} < \underline{b} < \overline{a} < \overline{b}$	**overlappedBy**	$\underline{b} < \underline{a} < \overline{b} < \overline{a}$
starts	$\underline{a} = \underline{b},\ \overline{a} < \overline{b}$	**startedBy**	$\underline{a} = \underline{b},\ \overline{b} < \overline{a}$
finishes	$\underline{b} < \underline{a},\ \overline{a} = \overline{b}$	**finishedBy**	$\underline{a} < \underline{b},\ \overline{a} = \overline{b}$
containedBy	$\underline{b} < \underline{a},\ \overline{a} < \overline{b}$	**contains**	$\underline{a} < \underline{b},\ \overline{b} < \overline{a}$
equals	$\underline{a} = \underline{b},\ \overline{a} = \overline{b}$		

`strictly_less()` compare the bounds on the same side, and `overlap()` tells the state.

`bool interval::`**`less`**`(const interval& I) const`

■ $\underline{\texttt{self}} \leqslant \underline{I} \wedge \overline{\texttt{self}} \leqslant \overline{I}$

Returns `true` if each bound of `self` is at most the bound of I on the same side: less of IEEE 1788-2015 (Table 10.3). Two empty sets are in the relation, and an empty set and a nonempty interval are not (Table 10.4). [GAOL v5]

Example

```
interval e = interval::emptyset();
cout << boolalpha << interval(1,2).less(interval(1,3)) << ' '
     << interval(1,4).less(interval(2,3)) << ' '
     << e.less(e) << ' ' << e.less(interval(1,2));
➥ true false true false
```

`bool interval::`**`strictly_less`**`(const interval& I) const`

■ $\underline{\texttt{self}} < \underline{I} \wedge \overline{\texttt{self}} < \overline{I}$

Returns `true` if each bound of `self` is below the bound of I on the same side, an infinite bound counting as below itself: strictLess of IEEE 1788-2015. The empty sets are as for `less()`. [GAOL v5]

Example

```
interval a(-GAOL_INFINITY,1), b(-GAOL_INFINITY,2);
cout << boolalpha << interval(1,2).strictly_less(interval(1,3)) << ' '
     << a.strictly_less(b) << ' '
     << interval(0,1).strictly_less(interval(2,3));
➥ false true true
```

`overlapping_state interval::overlap(const interval& I) const`

Returns the state of Table 10.7 of IEEE 1788-2015 that `self`, $a = [\underline{a}, \overline{a}]$, and I, $b = [\underline{b}, \overline{b}]$, are in: overlap (10.6.4), a primitive from which most of the comparisons can be derived. [GAOL v5] The result is one of the 16 values of the type `overlapping_state` (in `gaol` and in `gaol_ieee1788`), which has the names of Table 10.7, in its order, from **`bothEmpty`**, 0, to **`after`**, 15. The state number s is the bit s of a 16-bit word counted from the left, **`bothEmpty`** being the leftmost bit, as in the note of the standard, where `disjoint` is the mask `0xF001`. A state is written to a stream by its name in the table, as **`meets`**. **`bothEmpty`**, **`firstEmpty`** and **`secondEmpty`** are for an empty a and b, an empty a alone and an empty b alone. The 13 other states are Allen's, for two nonempty intervals, told by their bounds as Table 6.2 says, an infinite bound being equal to the same infinite bound and -0 to $+0$. A point interval on a bound of the other one is **`starts`**, **`finishes`**, **`finishedBy`** or **`startedBy`**, or **`equals`** for the same point, never **`meets`** nor **`metBy`**, as the standard asks. The function compares the bounds as the integers of their bits: it raises no floating-point exception, even for the `NaN` bounds of an empty interval, and the modes that flush the subnormal numbers to zero do not change it.

Example

```
interval a(1,3), b(2,4), c(1,2), e = interval::emptyset();
cout << a.overlap(b) << ' ' << c.overlap(interval(2,3)) << ' '
     << interval(2).overlap(c) << ' ' << e.overlap(c) << ' '
     << static_cast<int>(c.overlap(a));
➥ overlaps meets finishes firstEmpty 6
```

6.5 Relation symbols

The relation symbols `<`, `<=`, `>` and `>=` are the certainly relations, whichever build configured GAOL. The option of GAOL 4 that chose between the set, certainly and possibly relations is gone: `configure` warns that it does not know it (see Section 2.6, page 44), and meson refuses it (see Section 2.7, page 46). [GAOL v5] The operators `==` and `!=` are not defined on intervals: a program compares intervals with `set_eq()`, `set_disjoint()` or `less()`.

An interval and a `double` d compare as the interval and `interval(d)`, which is empty for an infinite or NaN d: `x < 0.0` is `x < interval(0.0)`, and there is still no `==` nor `!=`. For a finite d, these overloads give what the implicit conversion of a `double` to an interval gave in GAOL 4 configured with the certainly relations, its default, before the constructor from one `double` became explicit (see Section 5.1, page 117); for an infinite or NaN d, `x < d` and `x > d` are both `true`, whatever `x`. [GAOL v5]

An interval and an integer n of any integer type but `bool` compare as the interval and the integer it is, where it is no double (see Section 3.6, page 72).

A double is below n if and only if it is at most the largest double below n, so that `interval(9007199254740992.0) < 9007199254740993LL`, that is $[2^{53}, 2^{53}] < 2^{53}+1$, is `true`, where the conversion of $2^{53}+1$ to the double 2^{53}, in the rounding to nearest, made it `false`. An integer that is a double compares as that double. [GAOL v5]

```
bool operator<(const interval& I, double d)
bool operator<(double d, const interval& I)
bool operator<(const interval& I1, const interval& I2)
```

Returns `I1.certainly_le(I2)`: strictPrecedes of IEEE 1788-2015.

```
bool operator<=(const interval& I, double d)
bool operator<=(double d, const interval& I)
bool operator<=(const interval& I1, const interval& I2)
```

Returns `I1.certainly_leq(I2)`: precedes of IEEE 1788-2015.

```
bool operator>(const interval& I, double d)
bool operator>(double d, const interval& I)
bool operator>(const interval& I1, const interval& I2)
```

Returns `I1.certainly_ge(I2)`.

```
bool operator>=(const interval& I, double d)
bool operator>=(double d, const interval& I)
bool operator>=(const interval& I1, const interval& I2)
```

Returns `I1.certainly_geq(I2)`.

Example

```
cout << boolalpha << (interval(1,2) < interval(3,4)) << ' '
     << (interval(1,2) <= interval(2,4)) << ' '
     << (interval(1,3) <= interval(2,4)) << ' '
     << (interval(3,4) > interval(1,2));
➥ true true false true
```

6.6 Intervals in containers and algorithms

The relation symbols do not tell which of two intervals is the greater: `interval(1,3) < interval(2,4)` and `interval(1,3) >= interval(2,4)` are both `false`: `x >= y` is not the negation of `x < y`. The empty set, which has no point to contradict a certainly relation, is less than every interval, and greater, itself included: `e < e` is `true`. A program ported from a library where `x <= y` is the inclusion compiles and changes its meaning (see Section 3.11, page 83).

Remark 6.6.1 *None of the relation symbols is a strict weak ordering, so intervals are not given to the algorithms and the containers of the standard library that compare with* `<` *(*`std::sort`*,* `std::set`*,* `std::map`*,* `std::priority_queue`*,* `std::max`*,* `std::min`*,* `std::clamp`*...). Two overlapping intervals are equivalent for* `<`*, a relation that is not transitive, and the empty set is less than itself. The behaviour of these algorithms is then undefined, and differs with the implementation of the library. A* `std::set<interval>` *that holds* [1, 2] *drops* [1, 3]*: two of the three intervals* [1, 2]*,* [1, 3] *and* [5, 6] *stay.* `std::sort` *of a vector that holds an empty interval can read past the end of the vector: AddressSanitizer stops the program with libstdc++ for 17 intervals or more, at some positions of the empty one.* `std::max`([1, 3], [2, 4]) *is* [1, 3]*, its first argument, where* `gaol::max` *gives* [2, 4].

`gaol::max(x, y)` and `gaol::min(x, y)` are the maximum and the minimum of the standard: the set of the greater (or the lesser) of a point of `x` and a point of `y`, and not the greater (or the lesser) of the two arguments.

`gaol::lexicographic_less` is the comparator to give the containers and the algorithms of the standard library: a strict weak ordering that puts the empty set first and the other intervals by their left bound, then by their right bound, -0 and $+0$ being the same bound. [GAOL v5] Two intervals with the same bounds, or two empty sets, are not less than each other, as a container needs. The comparator is also in `gaol_core`; a program that opens `gaol_ieee1788` writes its name in full. Specializing `std::less` for intervals is left to the program, so the comparator is given explicitly.

Example 6.6.2 *The program below puts four intervals, the empty set among them, in a* `std::set` *and in a sorted* `std::vector` *with the comparator* `lexicographic_less`.

Example

```
1 #include <algorithm>
2 #include <iostream>
3 #include <set>
4 #include <vector>
5 #include <gaol/gaol.h>
6 using namespace gaol;
7
8 int main(void)
```

```
{
  const interval boxes[] = { interval(1,2), interval(0,5),
                             interval::emptyset(), interval(1,3) };
  std::set<interval, lexicographic_less> s(boxes, boxes + 4);
  std::vector<interval> v(boxes, boxes + 4);
  std::sort(v.begin(), v.end(), lexicographic_less());
  for (const interval& x : v) std::cout << x << ' ';
  std::cout << s.size() << std::endl;
  gaol::cleanup();
  return 0;
}
➥ [empty] [0, 5] [1, 2] [1, 3] 4
```

The vector comes out with the empty set first, then $[0, 5]$, $[1, 2]$ *and* $[1, 3]$*, and the set holds the four intervals: none is dropped.*

An algorithm that needs to know which of two boxes to process first, as a best-first search, orders them by a number of its own, the lower bound of an enclosure for instance, as `examples/06_global_optimization.cpp` does.

6.7 Interval-specific relations

The relations of this section test one interval: its position with respect to zero, its shape (a point, an integer, a canonical interval, a symmetric one) and its place among the sets (the empty set, the whole real line, a bounded interval). Four of them are predicates of IEEE 1788-2015: isSingleton, isEmpty, isEntire and isCommonInterval.

`bool interval::straddles_zero(void) const`

■ $0 \in$ `self`

Returns `true` if `self` contains zero. The call `I.straddles_zero()` is equivalent to `I.set_contains(0.0)`.

Example

```
interval x(0,4), y, z(-12,-5);

cout << boolalpha << x.straddles_zero() << ' '
     << y.straddles_zero() << ' '
     << z.straddles_zero() << endl;
➥ true true false
```

`bool interval::strictly_straddles_zero(void) const`

■ $\{0\}$ interior to `self`

Returns `true` if zero is included in the interior of `self`. The call `I.strictly_straddles_zero()` is equivalent to `I.set_strictly_contains(0.0)`.

Example

```
interval x(0,4), y, z(-12,-5);

cout << boolalpha << x.strictly_straddles_zero() << ' '
     << y.strictly_straddles_zero() << ' '
     << z.strictly_straddles_zero() << endl;
➥ false true false
```

`bool interval::is_a_double(void) const`

Returns `true` whenever the left and right bounds of the interval are equal: isSingleton of IEEE 1788-2015.

Example

```
interval x(-12.5), y;

cout << boolalpha << x.is_a_double() << ' '
     << y.is_a_double() << ' '
     << interval::emptyset().is_a_double() << endl;
➥ true false false
```

`bool interval::is_an_int(void) const`

Returns `true` whenever the left and right bounds of the interval are equal to the same integer, within the range of the type `int`.

Example

```
interval x(-12.0), y;

cout << boolalpha << x.is_an_int() << ' ' << y.is_an_int() << ' '
     << interval::emptyset().is_an_int() << ' '
     << interval(3e10).is_an_int() << endl;
➥ true false false false
```

`bool interval::is_canonical(void) const`

Returns `true` if `self` is nonempty and contains at most two floating-point numbers.

Example

```
interval x(0.0), y(-14,9);

cout << boolalpha << x.is_canonical() << ' '
     << y.is_canonical() << ' '
     << interval::emptyset().is_canonical() << ' '
     << interval::pi().is_canonical();
➥ true false false true
```

`bool interval::is_empty(void) const`

■ `self` $= \varnothing$

Returns `true` if `self` is an empty interval: isEmpty of IEEE 1788-2015. It raises no floating-point exception, the invalid-operation one included, though the bounds of the empty set are NaN (see Section 4.9, page 107). [GAOL v5]

Example

```
cout << boolalpha << interval(4,5).is_empty() << ' '
     << interval(5,4).is_empty() << ' '
     << interval::emptyset().is_empty();
➥ false true true
```

`bool interval::is_entire(void) const`

■ `self` $= [-\infty, +\infty]$

Returns `true` if `self` is the whole real line: isEntire of IEEE 1788-2015. [GAOL v5]

Example

```
cout << boolalpha << interval::universe().is_entire() << ' '
     << interval::positive().is_entire() << ' '
     << interval::emptyset().is_entire();
➥ true false false
```

`bool interval::is_common_interval(void) const`

Returns `true` if `self` is nonempty and bounded: isCommonInterval of IEEE 1788-2015. [GAOL v5]

Example

```
cout << boolalpha << interval(1,2).is_common_interval() << ' '
     << interval::positive().is_common_interval() << ' '
     << interval::emptyset().is_common_interval();
➥ true false false
```

`bool interval::is_zero(void) const`

- `self` = [0, 0]

Returns `true` if `self` is equal to the interval containing only 0.

Example

```
cout << boolalpha << interval::zero().is_zero() << ' '
     << interval(0.0,0.0).is_zero() << ' '
     << interval(-0.0,+0.0).is_zero() << ' '
     << interval(0,5).is_zero() << ' '
     << interval::emptyset().is_zero();
➥ true true true false false
```

`bool interval::is_symmetric(void) const`

Returns `true` if the left bound of `self` is the opposite of the right bound. An empty interval is not symmetric.

Example

```
cout << boolalpha << interval(-5,5).is_symmetric() << ' '
     << interval::emptyset().is_symmetric() << endl;
➥ true false
```

`bool interval::is_finite(void) const`

Returns `true` if neither bound is infinite. The empty set, whose bounds are NaN, is finite; `is_common_interval()` is not true of it.

Example

```
cout << boolalpha << interval(4,GAOL_INFINITY).is_finite() << ' '
     << interval::emptyset().is_finite() << ' '
     << interval(5,80).is_finite();
➥ false true true
```

6.8 Discussion

This chapter has fixed the vocabulary of the comparisons of GAOL v5. Two groups remain of the three of GAOL 4: the set relations, which compare the intervals as sets of reals, and the certainly relations, which hold when every pair of points is in the relation. The possibly relations, `certainly_eq()`, `certainly_neq()` and the operators `==` and `!=` are gone: IEEE 1788-2015 defines none of them, and a certain inequality that was the negation of a certain equality made [3, 4] certainly not equal to itself. Every relation that IEEE 1788-2015 defines follows the standard, for the empty set and for the infinite bounds too, and the overlap state tells in which of the

sixteen states of the standard two intervals are, from which most of the comparisons can be derived.

The relation symbols have one meaning in every build, the certainly relations, and compare an interval with a `double` or an integer as with the interval of that number, an integer that is no double being compared as the integer it is. The price of a fixed meaning that follows the standard is that the symbols order nothing: a program that sorts intervals or keeps them in a `std::set` gives the comparator `gaol::lexicographic_less`, and an algorithm that chooses the next box to process orders the boxes by a number of its own. Specializing `std::less` for intervals is left to the program.

The relations tell what two intervals are to each other. The next chapter turns to the operations that build new intervals from intervals: the functional arithmetic, the relational arithmetic and the cancellative operations (Chapter 7).

Chapter 7

Interval arithmetic: the operations

An interval program is written with the operators and the functions of C++, which GAOL overloads for the type `interval`: a sum, a product, a square root or a cosine of intervals is an interval that contains every value the operation takes on the real numbers of its arguments. This chapter is the reference of these operations. The containment property they satisfy, the roundings that give their bounds and the mathematical libraries that compute the elementary functions are the subject of Chapter 1. Here we state, for each operation, what it computes, on which domain, which operation of IEEE 1788-2015 [9] it is, and what differs from GAOL 4.

The operations are of two kinds. A *functional* operation extends a function of real numbers to intervals: the square root of an interval is the hull of the square roots of its points. A *relational* operation inverts a relation: the relational square root of J with respect to I is the hull of the x of I whose square is in J, the negative ones included. The relational operations are the reverse functions of IEEE 1788-2015, and the tools with which the algorithms of constraint programming contract their domains.

The first section recalls the containment principle, the two kinds of operations and the rule of the domains. The second section is the reference of the functional arithmetic: the basic operations, the powers and roots, the exponentials and logarithms, the trigonometric functions, the hyperbolic functions and the other functions of the standard. The third section is the reference of the relational arithmetic, and gives its correspondence with the reverse functions of the standard. The fourth section describes the cancellative addition and subtraction. The tightness of each result is stated in Chapter 11 (see Section 11.2, page 206), and the names of the standard are the subject of Chapter 10.

7.1 Functional and relational operations

The containment principle of floating-point interval arithmetic (see Section 1.3, page 9) requires that, for any operation $\circ$ and any intervals I and J,

$$I \circ J = \square\{i \circ j \mid i \in I, j \in J\}$$

where $\square$ maps any set of real numbers to the smallest floating-point interval that contains it. The interval square root, for instance, is

$$\sqrt{I} = \square\{\sqrt{i} \mid i \in I\}$$

and, the square root being increasing, $\sqrt{[1,\ 2]}$ encloses $[1,\ \sqrt{2}]$: it is $[1,\ \uparrow\sqrt{2}\uparrow]$. This is the *functional square root*: it extends the function $x \mapsto \sqrt{x}$ to intervals. Another reading of the square root gives both the negative and the positive values.

Definition 7.1.1 *The relational square root of an interval I is the hull of the real numbers whose square is in I:*

$$\sqrt[r]{I} = \square\{j \in \mathbb{R} \mid \exists i \in I\colon j^2 = i\}.$$

Hence $\sqrt[r]{[1,\ 2]}$ encloses $[-\sqrt{2},\ \sqrt{2}]$.

Example 7.1.2 *The relational square root arises from the relation $x^2 = y$ between two variables, which a program may want to write $x = \sqrt[r]{y}$ to compute x from y. The functional square root is not suitable there: its result lies in $[0,\ +\infty]$, and the negative values of x would be lost.*

Some applications, mainly in constraint programming, need such operators. GAOL therefore offers both the functional and the relational versions of the main arithmetic operators (see Section 1.6, page 13); a relational operator contracts one operand of a relation from the others, which is how the interval algorithms of Chapter 3 use it (see Section 3.8, page 74).

Domains. A function is computed on the part of its argument that lies within its domain, which is the domain of IEEE 1788-2015 (Tables 9.1 and 10.5): the part outside is left out, and an argument with no point of the domain gives the empty set, as `log`$([-4,\ 0])$ does. [GAOL v5] Every function but `cancel_minus` and `cancel_plus` gives the empty set for an empty argument: these two give $[-\infty,\ +\infty]$ for a nonempty X and an empty Y, or an empty X and an unbounded Y, as IEEE 1788-2015 requires (12.12.5).

7.2 Functional arithmetic

Each operation of this section is the hull of the image, by the function of the same name, of the part of its arguments within its domain. The entries give, for each one, the operation of IEEE 1788-2015 it implements, the domain where it is not the whole real line, and what GAOL v5 changed.

7.2.1 Basic operations

```
interval interval::operator+(void) const
interval operator+(const interval& I, double d)
interval operator+(double d, const interval& I)
interval operator+(const interval& I1, const interval& I2)
```

Addition of two intervals, or of an interval and a double: add of IEEE 1788-2015.

```
interval interval::operator-(void) const
interval operator-(const interval& I, double d)
interval operator-(double d, const interval& I)
interval operator-(const interval& I1, const interval& I2)
```

Negation (neg), or subtraction (sub) of two intervals, or of an interval and a double.

```
interval operator*(const interval& I, double d)
interval operator*(double d, const interval& I)
interval operator*(const interval& I1, const interval& I2)
```

Multiplication of two intervals, or of an interval and a double: mul of IEEE 1788-2015. A zero bound times an infinite bound counts as 0, as the product of a real number of the interval by 0. [GAOL v5] `d * I` is computed as `I * d`, and `n * I` of an integer `n` as `I * n`: the bounds are the same, with the same signs of a zero bound (see Section 3.10.6, page 78). [GAOL v5]

```
interval operator/(const interval& I, double d)
interval operator/(double d, const interval& I)
interval operator/(const interval& I1, const interval& I2)
```

Functional division of two intervals, or of an interval and a double: div of IEEE 1788-2015. The quotient by an interval containing 0 is the hull of the quotients by its nonzero part, and the quotient by [0, 0] is the empty set.

Example

```
1 cout << interval(1,2) * interval(-3,4) << ' '
2      << interval(0,1) * interval(1,GAOL_INFINITY) << ' '
3      << interval(1,2) / interval(0,1) << ' '
4      << interval(1,2) / interval(-1,1) << ' '
5      << interval(1,2) / interval::zero();
6 ➥ [-6, 8] [0, inf] [1, inf] [-inf, inf] [empty]
```

```
interval operator%(const interval& I, double d)
interval operator%(double d, const interval& I)
interval operator%(const interval& I1, const interval& I2)
```

Relational division of two intervals, or of an interval and a double (see Section 7.3, page 160).

```
interval interval::inverse(void) const
interval inverse(const interval& I)
```

■ $1/I$

Inverse of I (of `self` for the method): recip of IEEE 1788-2015. The inverse of an interval containing 0 is the hull of the inverses of its nonzero part.

Example

```
cout << inverse(interval(2,4)) << ' ' << inverse(interval(-1,2));
➥ [0.25, 0.5] [-inf, inf]
```

```
interval sqr(const interval& I)
```

■ I^2

Square of I: sqr of IEEE 1788-2015.

```
interval sqrt(const interval& I)
```

■ $\sqrt{I}$

Functional square root of I, on the part of I in $[0, +\infty]$: sqrt of IEEE 1788-2015. The bounds are the tightest ones, whatever the rounding of the square root of the C library. [GAOL v5]

Example

```
cout << sqrt(interval(-4,9)) << ' ' << sqrt(interval(-4,-1));
➥ [0, 3] [empty]
```

```
interval fma(const interval& X, const interval& Y, const interval&
Z)
```

■ $\square\{xy + z \mid x \in X, y \in Y, z \in Z\}$

Fused multiply-add: fma of IEEE 1788-2015. Each bound is rounded once, and the result is the tightest enclosure. [GAOL v5]

Example

```
cout << fma(interval(1,2), interval(3,4), interval(-1,1));
➥ [2, 9]
```

7.2.2 Powers and roots

The functions of this subsection are those of the namespace `gaol`. The namespace `gaol_ieee1788` has its own `pow`, the `pow` of the standard: the two powers are compared in Chapter 3 (see Section 3.5, page 70).

```
interval pow(const interval& I, int n)
interval pow(const interval& I, unsigned int n)
interval pow(const interval& I, double p)
template <class T>
interval pow(const interval& I, T n)
interval pow(const interval& I, const interval& J)
```

- I^n
- I^p
- I^J

Power function. `pow` is hybrid in GAOL v5. [GAOL v5]

- **An integer exponent** takes the *integer power*, pown of IEEE 1788-2015: I to the power n, the negative part of I included, where $x^0 = 1$ for every x, 0 included, and x^n has no value at 0 for $n < 0$. A double p and a degenerate interval $J = [n, n]$ whose value is an integer take the integer power as well, beyond the `int`s too. [GAOL v5] An exponent n of another integer type T (`long`, `long long`, `short`...; `pow(x, 5L)` was ambiguous in the earlier versions of GAOL v5) gives the same integer power; beyond 64 bits (an `__int128`, which GAOL takes as an integer where `std::is_integral` does: with libstdc++, in the GNU modes of GCC and Clang only, and in an ISO mode such as `-std=c++11`, `pow(x, n)` does not compile), it gives the power of $\pm(2^{63}-1)$ or $\pm(2^{63}-2)$, with the sign and the parity of n, every power of a double but 0, ±1 and $\pm\infty$ being beyond the doubles there. [GAOL v5]
- **Any other exponent** takes the *real power*, pow of IEEE 1788-2015, on the part of I in $[0, +\infty]$, where 0^y is 0 for $y > 0$ and has no value for $y \leqslant 0$. An infinite or NaN exponent p gives the empty set.

Within the `int`s, the integer powers are computed from exact products, x^{-n} included: they are the tightest enclosures, or one double beyond where the power is closer to a double than twice a bound of their error proved beforehand (Chapter 11), and exact where the power is a double. Beyond the `int`s, the integer powers come from the power of CORE-MATH at the bounds of I: they are the tightest enclosures where n is a double, and within three doubles of them otherwise. [GAOL v5] The real power takes the power of CORE-MATH at the corners of the box $I \times J$ where it is the least and the greatest. The upper bound is the tightest one, and so is the lower

bound: it is the power itself where that power is a double, as $4^{0.5}$, $9^{1.5}$ or 2^{-3}, and the double below the value of CORE-MATH otherwise. At a corner where the base is 0 or $+\infty$ or the exponent is infinite, the bound is the limit of x^y there, 0 or $+\infty$ (1 for a base 1 or an exponent 0): `pow`$([4, +\infty]$`, 0.5)` is $[2, +\infty]$. [GAOL v5]

Example

```
cout << pow(interval(-4,-1), 2) << ' '
     << pow(interval(-4,-1), interval(2.0)) << ' '
     << pow(interval(-4,9), interval(0.5)) << ' '
     << pow(interval(-4,-1), interval(1.5, 2.5)) << endl;
➥ [1, 16] [1, 16] [0, 3] [empty]
cout << pow(interval(0.0), interval(0.0)) << ' '
     << pow(interval(-2,2), -2) << ' '
     << pow(interval(4), 0.5) << endl;
➥ [1] [0.25, inf] [2]
```

Remark 7.2.1 *The result of* `pow` *is not monotone for the inclusion of J:* `pow`$([-4, -1]$`,` $[2, 2])$ *is* $[1, 16]$*, and* `pow`$([-4, -1]$`,` $[1.5, 2.5])$ *the empty set. The result for a degenerate integer exponent* $[n]$ *is not within the result for an exponent around* n*, and* `pow` *is not one function of* x *and* y*. GAOL 4 truncated a double exponent* p*, and computed the powers of a negative base on its magnitude.*

```
interval nth_root(const interval& I, unsigned int n)
interval nth_root(const interval& I, long long q)
interval nth_root(const interval& I, unsigned long long q)
interval nth_root(const interval& I, int q)
```

■ $\sqrt[n]{I}$

Computes the nth functional root of I: rootn of IEEE 1788-2015. For an odd n, the root of a negative number is the opposite of the root of its magnitude, and the root is defined on the whole real line; for an even n, only the part of I in $[0, +\infty]$ is considered. [GAOL v5] A negative q gives $1/x^{1/|q|}$, defined on $\mathbb{R} \setminus \{0\}$ for an odd q and on $]0, +\infty[$ for an even one. The function returns the empty set for $n = 0$.

The bounds are the tightest ones, or up to 2 doubles beyond (4 for a negative q): each is proved with integer powers. The cube root is that of CORE-MATH, the tightest one, and the root of a double that is the nth power of a double is that double. An order of another integer type (`long`, `long long`...), with which `nth_root` was ambiguous, is taken as an `int` or an `unsigned int` where it is one. Beyond them, the roots are enclosed by `pow` with the exponent $1/q$ taken as an interval, $x^{1/q}$ being within 2^{-21} of 1 for every double $x > 0$ there, after one check of the rounding direction for a `long long` or an `unsigned long long` order. [GAOL v5]

Example

```
1 cout << nth_root(interval(-8,27), 3) << ' '
2      << nth_root(interval(-4,16), 2) << ' '
3      << nth_root(interval(4,16), -2) << ' '
4      << nth_root(interval(-8,-1), -3);
5 ➥ [-2, 3] [0, 4] [0.25, 0.5] [-1, -0.5]
```

7.2.3 Exponentials and logarithms

The functions of this subsection, and of the next two, are bounded with CORE-MATH at the bounds of the interval, or at the points within it where the function reaches its extrema. CORE-MATH being correctly rounded, the value at the bound where the function is greatest, computed rounding upward, is the upper bound, and the double below the value at the bound where it is least, computed the same way, is the lower bound. These are the tightest bounds, unless the exact value is a double; the functions give that double where they know that it happens ($\exp(0) = 1$, $\log(1) = 0$, $\sin(0) = 0$, $2^3 = 8$...). [GAOL v5]

```
interval exp(const interval& I)
```

Exponential of I.

```
interval exp2(const interval& I)
interval exp10(const interval& I)
```

2^x and 10^x for x in I: exp2 and exp10 of IEEE 1788-2015. [GAOL v5]

```
interval expm1(const interval& I)
interval exp2m1(const interval& I)
interval exp10m1(const interval& I)
```

$e^x - 1$, $2^x - 1$ and $10^x - 1$ for x in I, within $[-1, +\infty]$: expm1, exp2m1 and exp10m1 of IEEE 1788-2015 (Table 10.5). [GAOL v5]

```
interval log(const interval& I)
```

Natural logarithm of I, defined on $]0, +\infty[$. An interval holding no positive number gives the empty set, where GAOL 4 kept its part in $[0, +\infty]$ and gave $[-\infty, -\mathrm{MAX}]$ for `log([-4, 0])`. [GAOL v5]

```
interval log2(const interval& I)
interval log10(const interval& I)
```

Logarithms in base 2 and 10 of I: log2 and log10 of IEEE 1788-2015. [GAOL v5]

```
interval log1p(const interval& I)
interval log2p1(const interval& I)
interval log10p1(const interval& I)
```

$\log(1 + x)$, $\log_2(1 + x)$ and $\log_{10}(1 + x)$ for x in I, defined on $]-1, +\infty[$, $-\infty$ being their limit at -1: logp1, log2p1 and log10p1 of IEEE 1788-2015 (Table 10.5). [GAOL v5]

Example

```
cout << exp(interval(0.0)) << ' ' << log(interval(1)) << ' '
     << log(interval(-4,0.0)) << ' '
     << log(interval(-4,1)) << endl;
➥ [1] [0] [empty] [-inf, 0]
cout << exp2(interval(3)) << ' ' << log2(interval(0.25)) << ' '
     << log10(interval(0.0,1000)) << ' '
     << expm1(interval(-1e300,0.0)) << ' '
     << log1p(interval(-2,0.0)) << endl;
➥ [8] [-2] [-inf, 3] [-1, 0] [-inf, 0]
```

7.2.4 Trigonometric functions

`interval cos(const interval& I)`

Returns the cosine of I, within $[-1, 1]$.

`interval sin(const interval& I)`

Returns the sine of I, within $[-1, 1]$.

`interval tan(const interval& I)`

Returns the tangent of I: $[-\infty, +\infty]$ when I holds a pole or is wider than the double below π. No double is a pole: `M_PI_2`, the double below $\pi/2$, is $6.1 \cdot 10^{-17}$ short of it. The interval $[-$`M_PI_2`, `M_PI_2`$]$ holds no pole and has a finite tangent (the fourth result of the example); its width is the double below π; an interval whose width lies between that double and π and that holds no pole, a theoretical case, gets $[-\infty, +\infty]$ too, which is not the tightest result.

The three functions find the pieces of I on which they are monotonic by dividing its bounds by an enclosure of π. Where the quotients cannot tell, next to an extremum or a pole and at the large magnitudes, they use the signs at the bounds of the derivative, for `cos` and `sin`, and of the cosine, for `tan`, which CORE-MATH gives exactly. Their bounds are the tightest ones at every magnitude where the function is evaluated at the bounds, and -1 and 1 exactly at the extrema (Chapter 11). [GAOL v5] GAOL 4 gave $[-1, 1]$ for `cos`$([2^{60}, 2^{60}])$, and computed the sine as a shifted cosine, which widened narrow intervals.

Example

```
cout << cos(interval(1,4)) << endl
     << sin(interval(1,2)) << endl
     << tan(interval(1,2)) << endl
     // 1.5707963267948966 is M_PI_2, the double below pi/2
     << tan(interval(-1.5707963267948966, 1.5707963267948966)) << endl
     // 1152921504606846976 is 2 to the 60
     << cos(interval(1152921504606846976.0)) << endl;
➥ [-1, 0.5403023058681398]
➥ [0.8414709848078965, 1]
➥ [-inf, inf]
➥ [-1.633123935319537e+16, 1.633123935319537e+16]
➥ [-0.5567960822766418, -0.5567960822766416]
```

`interval acos(const interval& I)`

Returns the arccosine of I, on the part of I in $[-1, 1]$.

`interval asin(const interval& I)`

Returns the arcsine of I, on the part of I in $[-1, 1]$.

`interval atan(const interval& I)`

Returns the arctangent of I.

Example

```
cout << acos(interval(1,3)) << ' ' << asin(interval(1)) << ' '
     << atan(interval(1));
➥ [0] [1.570796326794896, 1.570796326794897]
➥ [0.7853981633974482, 0.7853981633974484]
```

`interval atan2(const interval& Y, const interval& X)`

Returns the hull of the angles in $(-\pi, \pi]$ of the points of ordinate in Y and abscissa in X, the point $(0, 0)$ left out, as the atan2 of IEEE 1788-2015 does: `atan2([0],[0])` is empty. The result is $[-\pi, \pi]$ whenever the box holds points of the half-line $y = 0$, $x < 0$, where the angle is π, and points below it, where the angle is next to $-\pi$. GAOL 4 declared the function, but it raised `unavailable_feature_error`. [GAOL v5]

Example

```
interval x(1,GAOL_INFINITY);
cout << atan2(interval(0.0), interval(0.0)) << endl
     << atan2(interval(-1,1), interval(-1)) << endl
     << atan2(x, x) << endl;
➥ [empty]
➥ [-3.141592653589794, 3.141592653589794]
➥ [0, 1.570796326794897]
```

`interval sinpi(const interval& I)`
`interval cospi(const interval& I)`
`interval tanpi(const interval& I)`

$\sin(\pi x)$, $\cos(\pi x)$ and $\tan(\pi x)$ for x in I: sinPi, cosPi and tanPi of IEEE 1788-2015 (Table 10.5). [GAOL v5] Their extrema and poles are at the multiples of $1/2$, which are doubles, and are found exactly, at every magnitude. `tanpi` has no value at the half-integers, which are left out: a half-integer alone gives the empty set.

`interval asinpi(const interval& I)`
`interval acospi(const interval& I)`
`interval atanpi(const interval& I)`

`interval` **`atan2pi`**`(const interval& Y, const interval& X)`

asin(x)/π, acos(x)/π, atan(x)/π and atan2(y, x)/π: asinPi, acosPi, atanPi and atan2Pi of IEEE 1788-2015 (Table 10.5), within $[-1/2, 1/2]$, $[0, 1]$, $[-1/2, 1/2]$ and $[-1, 1]$. [GAOL v5]

Example

```
cout << sinpi(interval(1e17)) << ' '
     << sin(interval::pi() * 1e17) << endl;
➥ [0] [-1, 1]
cout << tanpi(interval(0.5)) << ' '
     << tanpi(interval(0.25,0.75)) << ' '
     << asinpi(interval(1)) << ' '
     << atan2pi(interval(1), interval(-1)) << endl;
➥ [empty] [-inf, inf] [0.5] [0.75]
```

7.2.5 Hyperbolic functions

`interval` **`cosh`**`(const interval& I)`

Returns the hyperbolic cosine of I.

`interval` **`acosh`**`(const interval& I)`

Returns the hyperbolic arccosine of I, on the part of I in $[1, +\infty]$.

`interval` **`sinh`**`(const interval& I)`

Returns the hyperbolic sine of I.

`interval` **`asinh`**`(const interval& I)`

Returns the hyperbolic arcsine of I.

`interval` **`tanh`**`(const interval& I)`

Returns the hyperbolic tangent of I, within $[-1, 1]$.

`interval atanh(const interval& I)`

Returns the hyperbolic arctangent of I, defined on $]-1, 1[$, $-\infty$ and $+\infty$ being its limits at -1 and 1. The part of I outside is left out, and an interval with no point of it gives the empty set, as `atanh([1, 5])` and `atanh([−1, −1])` do.

Example

```
cout << cosh(interval(-1,2)) << ' '
     << tanh(interval(20,30)) << endl
     << acosh(interval(0,1)) << ' '
     << atanh(interval(-2,2)) << ' '
     << atanh(interval(1,5)) << endl;
➥ [1, 3.762195691083632] [0.9999999999999998, 1]
➥ [0] [-inf, inf] [empty]
```

Remark 7.2.2 *The six hyperbolic functions are those of CORE-MATH. GAOL 4 took them from the mathematical library of the system, which is sometimes further than one double from the exact value.* [GAOL v5]

7.2.6 Other functions

`interval hypot(const interval& X, const interval& Y)`

$\sqrt{x^2 + y^2}$ for x in X and y in Y: hypot of IEEE 1788-2015 (Table 10.5), computed at the points of the box nearest to the origin and farthest from it. [GAOL v5]

`interval rsqrt(const interval& I)`

$1/\sqrt{x}$ for x in I, defined on $]0, +\infty[$, $+\infty$ being its limit at 0: rSqrt of IEEE 1788-2015 (Table 10.5). [GAOL v5]

Example

```
cout << hypot(interval(3), interval(4)) << ' '
     << hypot(interval(-3,1), interval(4,5)) << endl
     << rsqrt(interval(4,16)) << ' '
     << rsqrt(interval(0.0,4)) << endl;
➥ [5] [4, 5.830951894845301]
➥ [0.25, 0.5] [0.5, inf]
```

`interval` **`compoundm1`**`(const interval& X, const interval& Y)`

$(1+x)^y-1$ for x in X and y in Y: compoundm1 of IEEE 1788-2015 (Table 10.5). It keeps the digits of a small x that `pow(1+x, y) - 1` loses, $1+x$ being rounded and the difference cancelling what is left. It is defined for $x > -1$, and for $x = -1$ when $y > 0$, where it is -1. The part of the box outside is left out, and -1 and $+\infty$, the limits at $x = -1$, at $x = +\infty$ and for an infinite y, are bounds. CORE-MATH has no function for it: the value at a corner of the box is computed with integers, 64 bits in a fast phase that settles 71 to 95 % of the pairs with $|y \log(1+x)|$ below 8, and none beyond, 128 bits and a bound of the error in the others. It takes 0.1 μs for each of the one or two corners where the bounds are, where the fast phase settles it, and 0.6 to 0.9 μs elsewhere (Intel i7-1185G7, GCC 15). The bounds are the tightest ones, the double itself where the value is one (2.375 for $x = 0.5$ and $y = 3$, but also $2^m - 1$ for $x = 1$ and an integer $y = m$ with $|m| \leq 53$, and the quotient $-x/(1+x)$ of $y = -1$, divided exactly), but in a case out of 10^{17} or so, where a bound is a double beyond. [GAOL v5]

Example

```
cout << compoundm1(interval(0.5), interval(2)) << ' '
     << compoundm1(interval(1,3), interval(1,2)) << endl
     << compoundm1(interval(-1), interval(0,1)) << ' '
     << compoundm1(interval(-1,0), interval(-1)) << endl
     << compoundm1(interval(1e-20), interval(3)) << endl;
➥ [1.25] [1, 15]
➥ [-1] [0, inf]
➥ [2.999999999999999e-20, 3.000000000000001e-20]
```

`interval` **`erf`**`(const interval& I)`
`interval` **`erfc`**`(const interval& I)`

The error function $\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt$, increasing from -1 to 1, and its complement $\mathrm{erfc}(x) = 1 - \mathrm{erf}(x)$, decreasing from 2 to 0, for x in I: the `erf` and `erfc` of C, which CORE-MATH computes, with the tightest bounds, exact at 0 and at the infinities. $\mathrm{erfc}(x)$ is subnormal from $x = 26.55$ on, and below the least subnormal from $x = 27.22$ on, where the lower bound is 0. IEEE 1788-2015 does not name them, and `gaol_ieee1788` does not hold them: a call `erf(x)` on an interval finds them all the same, by argument-dependent lookup. [GAOL v5]

Example

```
cout << erf(interval(0.0)) << ' ' << erf(interval(-1,1)) << ' '
     << erf(interval(6,7)) << endl
     << erfc(interval(0.0,1)) << ' ' << erfc(interval(28,30)) << endl;
➥ [0] [-0.8427007929497149, 0.8427007929497149] [0.9999999999999998, 1]
➥ [0.1572992070502851, 1] [0, 4.940656458412466e-324]
```

7.3 Relational arithmetic

The relational functions of GAOL are the *reverse functions* of IEEE 1788-2015 (10.5.4, Table 10.1), the arguments of the binary ones and of pownRev in another order: each computes the hull of the x of an interval I whose image by the function is in J. Their correspondence is given at the end of this section (Table 7.1), and the namespace `gaol_ieee1788` provides them under their names (Chapter 10).

```
interval operator%(const interval& K, double d)
interval operator%(double d, const interval& J)
interval operator%(const interval& K, const interval& J)
```

■ $K \,\%\, J = \square\{z \in \mathbb{R} \mid \exists x \in K, \exists y \in J : x = yz\}$

Relational division: `div_rel(K, J,` $[-\infty, +\infty]$`)`.

Example

```
1 cout << interval(1,2) % interval(0,1) << ' '
2      << interval(1,2) % interval(-1,1);
3 ➥ [1, inf] [-inf, inf]
```

7.3.1 $(n+1)$-ary relational functions

Consider the relation $y = \cos x$ between two interval variables x and y. The equivalent form $x = \operatorname{acos} y$ cannot be computed with the function acos, whose result always lies in $[0, \pi]$: a relational version of acos is needed. For any value of y in $[-1, 1]$ there are infinitely many x such that $y = \cos x$, so the domain of x has to be taken into account, and the relational arccosine is a binary operator.

Definition 7.3.1 *The relational arccosine of Y with respect to X is*

$$\texttt{acos_rel}(Y, X) = \square\{x \in X \mid \exists y \in Y : y = \cos x\}.$$

It is to be contrasted with the functional arccosine,

$$\texttt{acos}(Y) = \square\{x \in \mathbb{R} \mid \exists y \in Y : x = \operatorname{acos} y\}.$$

Figure 7.1 shows the results of $\texttt{acos}(J)$ and of $\texttt{acos_rel}(J, I)$ on the same arguments.

Remark 7.3.2 *An interval I that meets no piece of the preimage gives the empty set.* [GAOL v5] *The same holds for every relational function of this section: the result is empty when no x of the interval contracted is in the relation.*

```
interval acos_rel(const interval& J, const interval& I)
```

■ $\texttt{acos_rel}(J, I) = \square\{x \in I \mid \exists y \in J : y = \cos x\}$

Returns the *relational arccosine* of J with respect to I.

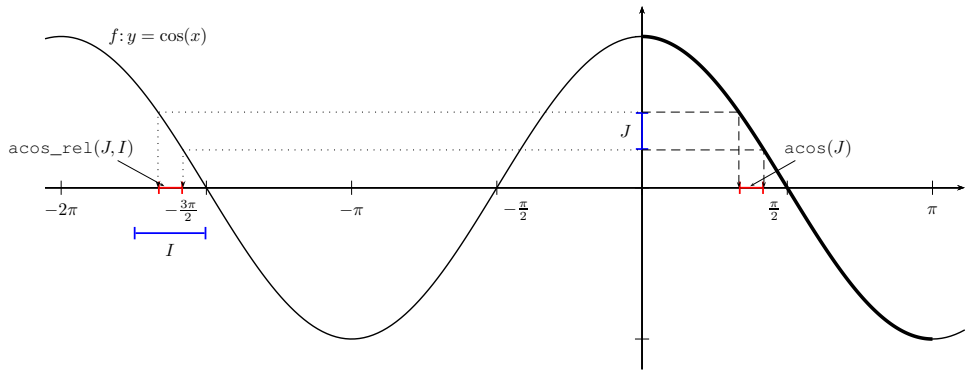


FIGURE 7.1 – The functional arccosine `acos`(J) and the relational arccosine `acos_rel`(J, I)

`interval` **`asin_rel`**`(const interval& J, const interval& I)`

■ `asin_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = \sin x\}$

Returns the *relational arcsine* of J with respect to I.

`interval` **`atan_rel`**`(const interval& J, const interval& I)`

■ `atan_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = \tan x\}$

Returns the *relational arctangent* of J with respect to I.

The three functions enclose the multiples of π of the pieces of the preimage in double-double arithmetic, and add the inverse function of J before rounding the sum: their bounds are within one double of the accurate ones. [GAOL v5]

Example

```
1 cout << asin_rel(interval(0.0), interval(1,7)) << endl
2      << atan_rel(interval(1), interval(0,5)) << endl;
3 ➥ [3.141592653589793, 6.283185307179588]
4 ➥ [0.7853981633974482, 3.926990816987242]
```

`interval` **`acosh_rel`**`(const interval& J, const interval& I)`

■ `acosh_rel`$(J, I) = \square\{x \in I \mid \exists y \in J : y = \cosh x\}$

Returns the *relational hyperbolic arccosine* of J with respect to I.

`interval` **`asinh_rel`**`(const interval& J, const interval& I)`

`interval` **`atanh_rel`**`(const interval& J, const interval& I)`

- `asinh_rel`$(J, I) = \square\{x \in I \mid \exists y \in J \colon y = \sinh x\}$
- `atanh_rel`$(J, I) = \square\{x \in I \mid \exists y \in J \colon y = \tanh x\}$

Return the *relational hyperbolic arcsine* and *arctangent* of J with respect to I, that is, $\operatorname{asinh}(J) \cap I$ and $\operatorname{atanh}(J) \cap I$, sinh and tanh being one-to-one. As tanh takes its values in $]-1, 1[$, `atanh_rel` gives the empty set for a J with no point of it, as $[1,\ 1]$.

Example

```
cout << acosh_rel(interval(1,2), interval::universe()) << endl
     << asinh_rel(interval(0,1), interval::universe()) << endl
     << atanh_rel(interval(1), interval::universe()) << endl;
➥ [-1.316957896924817, 1.316957896924817]
➥ [0, 0.8813735870195431]
➥ [empty]
```

`interval` **`sqrt_rel`**`(const interval& J, const interval& I)`

- `sqrt_rel`$(J, I) = \square\{x \in I \mid \exists y \in J \colon y = x^2\}$

Returns the *relational square root* of J with respect to I (Definition 7.1.1).

Example

```
cout << sqrt_rel(interval(1,4), interval::universe()) << ' '
     << sqrt_rel(interval(1,4), interval(0,10));
➥ [-2, 2] [1, 2]
```

`interval` **`nth_root_rel`**`(const interval& J, unsigned int n, const interval& I)`

`interval nth_root_rel(const interval& J, int p, const interval& I)`

- `nth_root_rel`$(J, n, I) = \square\{x \in I \mid \exists y \in J\colon y = x^n\}$
- `nth_root_rel`$(J, p, I) = \square\{x \in I \mid x^p$ is defined and in $J\}$

Returns the *relational inverse n-th power* of J with respect to I, for $n \geqslant 1$. It takes the roots of `nth_root()`. [GAOL v5] For an `int` p, or an integer of another type, it is the reverse of x^p for every integer p, the pownRev of the standard. x^0 is 1 for every x, 0 included, so that the result is I where $1 \in J$, and the empty set elsewhere: `nth_root_rel`$([-1, 5]$`, 0, I)` is I, and `nth_root_rel`$([2, 3]$`, 0, I)` is empty. For $p < 0$, x^p is $1/x^{|p|}$, which 0 has not. J is then cut at 0, and each part gives its roots: the hull of the roots of the whole of J would keep the part of I between them. The roots reach 0 where J is unbounded, 0 being a limit there and no solution: `nth_root_rel`$([1, +\infty]$`, -2,` $[0, 0]$`)` is empty. The bounds are those of `nth_root(J, p)`, the root and then its inverse: up to four doubles beyond the tightest ones for $p < 0$, but for $p = -1$ and a J below 1/DBL_MAX, about 2^{-1024}, a subnormal number whose inverse lies beyond the largest double: a bound is then the largest double, as `div_rel` gives it, and `nth_root_rel`$([2^{-1074}, 2^{-1074}]$`, -1,` $[0,$ `DBL_MAX])` is [DBL_MAX, DBL_MAX], where the tightest result is empty. An exponent of type `double` or `float` is refused at compile time, as the ambiguous `nth_root(x, 2.0)` is: it was converted to an `unsigned int`, which cut 2.5 to 2.

Example

```
interval J(1,16);
cout << nth_root_rel(J, 4, interval::universe()) << ' '
     << nth_root_rel(J, 4, interval(-10,0.0));
➥ [-2, 2] [-2, -1]
```

Example

```
cout << nth_root_rel(interval(1,8), -3, interval::universe()) << ' '
     << nth_root_rel(interval(-1,2), -3, interval(-0.5,0.5)) << ' '
     << nth_root_rel(interval(1,GAOL_INFINITY), -2, interval(0,3)) << ' '
     << nth_root_rel(interval(1,2), 0, interval(-5,5));
➥ [0.5, 1] [empty] [0, 1] [-5, 5]
```

`interval invabs_rel(const interval& J, const interval& I)`

- `invabs_rel`$(J, I) = \square\{x \in I \mid |x| \in J\}$

Returns the *relational inverse absolute value* of J with respect to I. The negative part of J holds no magnitude: GAOL 4 took it as magnitudes, and gave $[-2, 2]$ where the result is empty. [GAOL v5]

Example

```
cout << invabs_rel(interval(1,2), interval::universe()) << ' '
     << invabs_rel(interval(-2,-1), interval(-10,10));
➥ [-2, 2] [empty]
```

`interval` **`div_rel`**`(const interval& K, const interval& J, const interval& I)`

■ `div_rel`$(K, J, I) = \square\{x \in I \mid \exists z \in K, \exists y \in J \colon z = xy\}$

Returns the *ternary relational division* of K by J with respect to I, which is empty when no x of I is in the relation. [GAOL v5]

Example

```
cout << div_rel(interval(1,2), interval(-1,1),
                interval(0,10)) << ' '
     << div_rel(interval(5,6), interval(1,2),
                interval(10,20));
➥ [1, 10] [empty]
```

`interval` **`pow_rel`**`(const interval& K, const interval& J, const interval& I)`

■ `pow_rel`$(K, J, I) = \square\{x \in I \mid \exists z \in K, \exists y \in J \colon z = x^y\}$

Returns the *relational base* of the power $z = x^y$: the hull of the x of I with an x^y in K for a y of J. It contracts the base of $z = x^y$ as `div_rel(K, J, I)` contracts a factor of $z = xy$, with the arguments in the same order. [GAOL v5] x^y is the `pow` of IEEE 1788-2015 (Table 9.1), that of `gaol_ieee1788::pow`: $e^{y \log x}$ for $x > 0$, 0 for $x = 0$ and $y > 0$, and no value elsewhere, so that no $x < 0$ is a solution, in either namespace. The `pow` of `gaol` takes the integer power for a degenerate integer exponent $[n]$, which has negative bases too (Remark 7.2.1): the reverse of x^n is `nth_root_rel(K, n, I)`.

Example

```
cout << pow_rel(interval(4,9), interval(2), interval::universe()) << ' '
     << pow_rel(interval(2,3), interval(1,GAOL_INFINITY), interval(0,1)) << ' '
     << pow_rel(interval(1,4), interval(2), interval(-3,3));
➥ [2, 3] [empty] [1, 2]
```

$x^y \geqslant 2$ for a $y \geqslant 1$ asks $x > 1$: the second result is empty.

`interval pow_exponent_rel(const interval& K, const interval& H, const interval& I)`

■ `pow_exponent_rel`$(K, H, I) = \square\{y \in I \mid \exists z \in K, \exists x \in H : z = x^y\}$

Returns the *relational exponent* of the power $z = x^y$, the same `pow` as `pow_rel()`: the hull of the y of I with an x^y in K for an x of H. [GAOL v5]

Example

```
cout << pow_exponent_rel(interval(8), interval(2), interval::universe()) << ' '
     << pow_exponent_rel(interval(3), interval(2,4), interval::universe());
➥ [3] [0.792481250360578, 1.584962500721157]
```

The bounds of both functions are the tightest ones, but in one case, given below. Each end of the solutions, $c^{1/y}$ or $\log c / \log x$ for bounds c, x and y of the arguments, is enclosed by two doubles: the largest d whose power d^y (or x^d), computed by the correctly rounded `pow` of CORE-MATH, is proved to lie on one side of c, and the smallest one whose power is proved to lie on the other side. A bound of I where the solutions start between it and the next double is kept when its power is within one double of c, which only knowing whether that power is a double would settle: `pow_rel`$([1 + 2^{-52}, 1 + 2^{-52}]$, $[0.5, 0.5]$, $[0, 1 + 2^{-51}])$ is $[1 + 2^{-51}, 1 + 2^{-51}]$, where $(1 + 2^{-52})^2 = 1 + 2^{-51} + 2^{-104}$ and no x of $[0, 1 + 2^{-51}]$ is a solution: a result within the tightest result for I one double wider, $[1 + 2^{-51}, 1 + 3 \cdot 2^{-52}]$.

Example 7.3.3 *The two reverse functions of the power contract in turn the base and the exponent of the relation $z = x^y$, each from the current domains of the two others.*

Example

```
interval x(0, 10), y(2, 3), z(4, 9);
x = pow_rel(z, y, x);            // the x of x with pow(x, y) in z for a y of y
y = pow_exponent_rel(z, x, y);   // the y of y with pow(x, y) in z for an x of x
cout << x << ' ' << y << endl;   // [1.587401051968199, 3] [2, 3]
gaol::cleanup();
```

The base is contracted to $[1.587401051968199, 3]$*, the cube root of 4 being its least value with a power in* $[4, 9]$*; the exponent* $[2, 3]$ *is not contracted, every y of it having a base in* $[1.587401051968199, 3]$.

`interval max_rel(const interval& Z, const interval& Y, const interval& X)`

`interval` **`min_rel`**`(const interval& Z, const interval& Y, const interval& X)`

- `max_rel`$(Z, Y, X) = \square\{x \in X \mid \exists y \in Y\colon \max(x, y) \in Z\}$
- `min_rel`$(Z, Y, X) = \square\{x \in X \mid \exists y \in Y\colon \min(x, y) \in Z\}$

Return the *relational maximum* and *minimum* of Z given Y with respect to X, the arguments in the order of `div_rel`: the result, the other operand, then the operand contracted. max and min being symmetric, `max_rel(Z, X, Y)` contracts Y, and `max(X, Y) & Z` contracts Z. $\max(x, y)$ is in Z when x and y are at most $\overline{Z}$ and one of them at least $\underline{Z}$: the set is empty when Y lies above Z, $X \cap [-\infty, \overline{Z}]$ when Y meets Z, and $X \cap Z$ when Y lies below Z, the maximum being x; `min_rel` is its mirror image. The bounds are those of X and Z: each result is the tightest enclosure. [GAOL v5]

Example

```
cout << max_rel(interval(1,2), interval(3,4), interval(-5,5)) << ' '
     << max_rel(interval(1,2), interval(0,1), interval(-5,5)) << ' '
     << max_rel(interval(1,2), interval(-1,0), interval(-5,5)) << endl
     << min_rel(interval(1,2), interval(2,3), interval(-5,5)) << endl;
➥ [empty] [-5, 2] [1, 2]
➥ [1, 5]
```

`interval` **`sign_rel`**`(const interval& Z, const interval& X)`

- `sign_rel`$(Z, X) = \square\{x \in X \mid \text{sign}(x) \in Z\}$

Returns the *relational inverse sign* of Z with respect to X: the negative x of X where Z holds -1, 0 where Z and X hold it, and the positive x of X where Z holds 1; the empty set where Z holds none of them, as $[0.5, 0.7]$. The negative x go up to 0, which their tightest enclosure takes although $\text{sign}(0) = 0$, the negative doubles next to 0 being in the set. [GAOL v5]

Example

```
cout << sign_rel(interval(-1), interval(-2,3)) << ' '
     << sign_rel(interval(-1), interval(0,3)) << ' '
     << sign_rel(interval(0,1), interval(-2,3)) << endl;
➥ [-2, 0] [empty] [0, 3]
```

`interval` **`floor_rel`**`(const interval& Z, const interval& X)`

- `floor_rel`$(Z, X) = \square\{x \in X \mid \lfloor x \rfloor \in Z\}$

Returns the *relational inverse floor* of Z with respect to X: the part of X in $\left\lceil \lceil \underline{Z} \rceil, \lfloor \overline{Z} \rfloor + 1 \right\rfloor$, the x whose floor is an integer of Z, and the empty set when Z holds no integer. That interval is open at its upper end, which its tightest closed enclosure takes all the same, the reals just below it being in the set: `floor_rel(interval(0), interval(0.5, 2))` is $[0.5, 1]$, and `floor_rel(interval(0), interval(1, 2))` is empty, X starting at that end. Beyond 2^{53}, where every double is an integer, $\lfloor \overline{Z} \rfloor + 1$ is no double, and the bound is the double above it: `floor_rel` of $[2^{53}, 2^{53}]$ ends at $2^{53} + 2$, and at $+\infty$ for [DBL_MAX, DBL_MAX], where X goes on to it. Each result is the tightest enclosure. [GAOL v5]

Example

```
cout << floor_rel(interval(0,2), interval(-5,5)) << ' '
     << floor_rel(interval(0), interval(1,2)) << ' '
     << floor_rel(interval(0.5,0.7), interval(-5,5)) << endl;
➥ [0, 3] [empty] [empty]
```

`interval` **`atan2_rel_y`**`(const interval& C, const interval& X, const interval& Y)`
`interval` **`atan2_rel_x`**`(const interval& C, const interval& Y, const interval& X)`

- `atan2_rel_y`$(C, X, Y) = \square\{y \in Y \mid \exists x \in X\colon \operatorname{atan2}(y, x) \in C\}$
- `atan2_rel_x`$(C, Y, X) = \square\{x \in X \mid \exists y \in Y\colon \operatorname{atan2}(y, x) \in C\}$

Return the *relational* `atan2` with respect to its first and to its second argument: the hulls of the ordinates and of the abscissas of the points of the box $X \times Y$ whose angle is in C, the point $(0, 0)$, which has no angle, left out. [GAOL v5] As in `div_rel`, the interval contracted comes last, and the other argument of `atan2` before it. The angles are those of `atan2`, in $]-\pi, \pi]$: the half-line $y = 0$, $x < 0$ has the angle π, and a C holding $-\pi$ but not π leaves it out; the angles of C beyond π are no angle of any point. The bounds are within two doubles of the tightest ones where these are not empty; a box within one rounding of the cone of C can keep a point where the tightest result is empty: `atan2_rel_y([0.5, 0.5], [1, 1], [t, t])`, t being the double above $\tan 0.5$, is $[t, t]$.

Example

```
interval c(1,2), x(-1,1), y(0,1);
cout << atan2_rel_y(c, x, y) << ' '
     << atan2_rel_x(c, y, x) << endl
     << atan2_rel_x(interval(-4,-3), interval(0.0), x)
     << endl;
➥ [0, 1] [-0.4576575543602859, 0.6420926159343309]
➥ [empty]
```

7.3.2 The reverse functions of the standard

Table 7.1 gives the correspondence between the reverse functions of IEEE 1788-2015 and the relational functions of GAOL. The last argument x of a function of the standard is $[-\infty, +\infty]$ when it is left out.

TABLE 7.1 – The reverse functions of IEEE 1788-2015 and the relational functions of GAOL

IEEE 1788-2015	GAOL
sqrRev(c, x)	`sqrt_rel(c, x)`
absRev(c, x)	`invabs_rel(c, x)`
pownRev(c, x, p)	`nth_root_rel(c, p, x)`
sinRev, cosRev, tanRev(c, x)	`asin_rel`, `acos_rel`, `atan_rel(c, x)`
coshRev(c, x)	`acosh_rel(c, x)`
mulRev(b, c, x)	`div_rel(c, b, x)`
powRev1(b, c, x)	`pow_rel(c, b, x)` [GAOL v5]
powRev2(a, c, x)	`pow_exponent_rel(c, a, x)` [GAOL v5]
atan2Rev1(b, c, x)	`atan2_rel_y(c, b, x)` [GAOL v5]
atan2Rev2(a, c, x)	`atan2_rel_x(c, a, x)` [GAOL v5]

Remark 7.3.4 *mulRevToPair*(b, c)*, the two parts, on either side of 0, of the set that* `div_rel(c, b, interval::universe())` *encloses, is in the namespace* `gaol_ieee1788` *under its name (Chapter 10).* [GAOL v5] *pownRev takes every integer* p*,* $p \leqslant 0$ *included.* `max_rel`*,* `min_rel`*,* `sign_rel` *and* `floor_rel` *are the reverse functions of* max*,* min*,* sign *and* floor *as the standard defines them (10.5.4), which it neither names nor requires, Table 10.1 not listing them: the namespace* `gaol_ieee1788` *has no name for them. A program that opens that namespace calls them as* `max_rel(z, y, x)` *all the same, argument-dependent lookup finding them in* `gaol_core`*.* `asinh_rel` *and* `atanh_rel`*, which Table 10.1 does not list either,* sinh *and* tanh *being one-to-one, are named* `sinhRev` *and* `tanhRev` *in* `gaol_ieee1788`*, after coshRev.* [GAOL v5]

7.4 Cancellative addition and subtraction

The two functions of this section are the cancelMinus and cancelPlus of IEEE 1788-2015 (10.5.6): `cancel_minus(X, Y)` is the tightest interval that, added to Y, gives an interval containing X, when such an interval exists.

`interval cancel_minus(const interval& X, const interval& Y)`

The tightest interval Z such that $Y + Z$ contains X, $[\underline{X} - \underline{Y}, \overline{X} - \overline{Y}]$, which exists when X and Y are bounded and X is at least as wide as Y; $[-\infty, +\infty]$ otherwise, and the empty set for an empty X and a bounded Y: cancelMinus of IEEE 1788-2015 (10.5.6). Whether X is at least as wide as Y is decided exactly, which rounding the differences of the bounds cannot do. [GAOL v5]

`interval` **`cancel_plus`**`(const interval& X, const interval& Y)`

`cancel_minus(X, -Y)`: cancelPlus of IEEE 1788-2015. [GAOL v5]

Example

```
1 cout << cancel_minus(interval(1,5), interval(0,2)) << ' '
2      << cancel_minus(interval(1,2), interval(0,3)) << ' '
3      << cancel_plus(interval(1,5), interval(0,2));
4 ➥ [1, 3] [-inf, inf] [3, 5]
```

7.5 Discussion

This chapter has given the reference of the operations on intervals. Every functional operation computes the hull of the image of the part of its argument within the domain of IEEE 1788-2015, and gives the empty set for an argument with no point of that domain. The elementary functions take their values from CORE-MATH, which is correctly rounded: their bounds are the tightest ones, where GAOL 4 depended on the mathematical library of the system. The power is hybrid: an integer exponent, whatever its type, takes the integer power pown, with the negative bases, and any other exponent the real power pow, at the price of a result that is not monotone for the inclusion of the exponent; the powers of the two namespaces are compared in Chapter 3.

The relational functions are the reverse functions of the standard with the arguments in the order of GAOL, the result first and the operand contracted last. GAOL v5 adds the reverse functions of the power and of `atan2`, which the standard names, and those of the maximum, the minimum, the sign and the floor, which it defines but does not name, and gives the empty set wherever no point of the operand is in the relation. [GAOL v5] How tight each operation is, and the conditions under which the bounds are the same on every machine, are the subject of Chapter 11; how the bounds are computed, of Chapter 12.

Chapter 8 describes the functions on intervals that are not arithmetic operations: the numeric and the integer functions of an interval, its splitting, and the union and the intersection of intervals.

Chapter 8

Interval functions

An interval algorithm does more than evaluate expressions on intervals. A branch-and-bound method measures the boxes it holds, cuts the widest one in two parts and keeps the parts that its tests do not reject; an interval Newton step starts from the midpoint of the current interval and intersects its result with it; a stopping test compares a width with a threshold. The operations these steps call do not compute the image of a real function: they read the bounds of an interval and return a number, they round its elements to integers, they cut it, or they combine two intervals as sets. This chapter describes them.

The numeric functions of IEEE 1788-2015 (Table 10.2, 12.12.8) are those of this chapter. The namespace `gaol_ieee1788` names them as the standard does, and returns $+\infty$ and $-\infty$ for the bounds of the empty set, where `left()` and `right()` return NaN (Chapter 10).

The first section covers the numeric functions, which return a double read or computed from the bounds of an interval, or the interval enclosing such a double: the radius, the width, the midpoint, the magnitude and the mignitude, the Hausdorff distance and the function `chi`, with the absolute value, the minimum and the maximum of intervals. The second section covers the integer functions, which round the elements of an interval to integers. The third section describes the splitting methods, which cut an interval in two parts, and the fourth the intersection of two intervals and the hull of their union. A discussion closes the chapter.

8.1 Numeric functions

The quantities of this section describe an interval by a real number. GAOL computes each of them on the doubles of the bounds, with the rounding stated in its entry, and gives two of them, the width and the midpoint, in a second form that returns the interval enclosing the exact value.

Definition 8.1.1 *Let $I = [a, b]$ be a non-empty interval. The width of I is $b - a$ and its midpoint is $(a + b)/2$. The magnitude of I is the greatest absolute value*

of its elements, and its mignitude *the smallest one [6, chap. 3]: the mignitude is* 0 *when* $0 \in I$*, a when* I *is strictly positive, and* $-b$ *otherwise. The signed* mignitude *of* I *[14, def. 1.3.28] is its element of smallest absolute value:* 0 *when* $0 \in I$*, a when* I *is strictly positive, and* b *otherwise.*

Remark 8.1.2 *The empty set has no width, no midpoint and no element.* `rad()`*,* `width()`*,* `mig()`*,* `smig()`*,* `mag()`*,* `midpoint()` *and* `chi()` *return NaN for it, as* `left()` *and* `right()` *do, its bounds being NaN;* `hausdorff()` *returns NaN when one of its arguments is empty;* `width_enclosure()`*,* `mid()` *and* `inflate()` *return the empty set. GAOL 4 returned* -1 *for the width of the empty set.* [GAOL v5]

Remark 8.1.3 *An unbounded interval has an infinite width and no midpoint in* $\mathbb{R}$*.* `width()` *and* `rad()` *return* $+\infty$ *for it, and* `width_enclosure()` *the interval* $[MAX, +\infty]$*,* MAX *being the largest double.* `midpoint()` *returns* 0 *for the whole line,* $-MAX$ *for* $[-\infty, b]$ *and* MAX *for* $[a, +\infty]$*: the double at which the splitting methods cut these intervals (see Section 8.3, page 179).*

`double interval::rad(void) const`

■ `rad`$([a, b]) = \min\{r \in \mathbb{F} \mid [m - r, m + r] \supseteq [a, b]\}$

Returns the radius of `self`: the smallest double r such that $[m - r, m + r]$ contains `self`, m being `self.midpoint()` (rad of IEEE 1788-2015). The radius of an unbounded interval is $+\infty$, and the radius of the empty set is NaN. The radius of a point interval of zero is $+0$, whatever the signs of its bounds, $[-0, -0]$ included, as 12.12.8 requires of every numeric function but inf. [GAOL v5]

`void interval::mid_rad(double& m, double& r) const`

Sets m to `self.midpoint()` and r to `self.rad()` (midRad of IEEE 1788-2015). [GAOL v5]

Example

```
1 double m, r;
2 interval(1,4).mid_rad(m, r);
3 cout << m << ' ' << r << ' ' << interval(1,GAOL_INFINITY).rad();
4 ➥ 2.5 1.5 inf
```

`double interval::width(void) const`

■ `width`$([a, b]) = \uparrow b - a \uparrow$

Returns the width of `self`, rounded upward: `wid` of IEEE 1788-2015. The width of the empty set is NaN, as the standard has it, where GAOL 4 returned -1, and that of a point zero is $+0$, whatever the signs of its bounds. [GAOL v5]

Example

```
cout << boolalpha << interval(4,6).width() << ' '
     << (interval(1,next_float(1)).width()
         == std::numeric_limits<double>::epsilon()) << ' '
     << interval::emptyset().width();
➥ 2 true nan
```

`interval interval::width_enclosure(void) const`

■ `width_enclosure`$([a, b]) = [\downarrow b - a \downarrow, \uparrow b - a \uparrow]$

Returns an interval enclosing the exact width of `self`, of which `width()` is the upper bound only. It is `interval(b) - interval(a)`: the point $[0, 0]$ for a point interval, and the empty set for the empty set. The width of an unbounded interval, $+\infty$, is no real number; the result is then [MAX, $+\infty$], MAX being the largest double, as `textToInterval("inf")` is. [GAOL v5]

Example

```
cout << interval(1,3).width_enclosure() << ' '
     << interval(-1,1e16).width_enclosure() << endl
     << interval(1,GAOL_INFINITY).width_enclosure();
➥ [2] [1e+16, 1.0000000000000001e+16]
➥ [1.797693134862315e+308, inf]
```

`interval interval::inflate(double r) const`

■ `inflate`$([a, b], r) = [\downarrow a - r \downarrow, \uparrow b + r \uparrow]$

Returns `self` widened by the absolute radius r, that is `self + interval(-r, r)`. The result encloses every $x + e$ with x in `self` and $|e| \leqslant r$, however small r is next to the bounds. $r = 0$ gives `self`, and $r = +\infty$ the whole line. The empty set stays empty, and a negative or NaN r gives the empty set. [GAOL v5]

Example

```
cout << boolalpha << interval(1,2).inflate(0.5) << ' '
     << interval(1,2).inflate(1e-20).set_eq(interval(1,2));
➥ [0.5, 2.5] false
```

`double interval::mig(void) const`

Returns the mignitude of `self` (Definition 8.1.1): mig of IEEE 1788-2015. The mignitude of the empty set is NaN.

Example

```
cout << interval(4,5).mig() << ' '
     << interval(-6,-3).mig() << ' '
     << interval(-3,8).mig();
➥ 4 3 0
```

`double interval::smig(void) const`

Returns the signed mignitude of `self` (Definition 8.1.1), after Stahl [14, def. 1.3.28]. The signed mignitude of $[a, b]$ is 0 when the interval contains 0, a when it is strictly positive, and b otherwise. The signed mignitude of the empty set is NaN.

Example

```
cout << interval(4,5).smig() << ' '
     << interval(-6,-3).smig() << ' '
     << interval(-3,8).smig();
➥ 4 -3 0
```

`double interval::mag(void) const`

Returns the magnitude of `self`: mag of IEEE 1788-2015. The magnitude of $[a, b]$ is the greatest absolute value of its elements. The magnitude of the empty set is NaN.

Example

```
cout << interval(4,5).mag() << ' '
     << interval(-6,-3).mag() << ' '
     << interval(-10,5).mag();
➥ 5 6 10
```

`double` **`hausdorff`**`(const interval& I1, const interval& I2)`

Returns the Hausdorff distance between the sets *I1* and *I2*,

$$\texttt{hausdorff}(I_1, I_2) = \max(|\underline{I_1} - \underline{I_2}|, |\overline{I_1} - \overline{I_2}|),$$

rounded upward: the result is the tightest upper bound of the distance. [GAOL v5]

Two equal bounds are at distance 0, infinite ones included: the distance of $[1, +\infty]$ to itself is 0, and to $[2, +\infty]$ it is 1, where GAOL 4 returned $+\infty$ for both. The distance is $+\infty$ when a bound is infinite in one of the two intervals only, and NaN when one of them is empty. [GAOL v5]

Example

```
cout << hausdorff(interval(4,8),interval(5,10)) << ' '
     << hausdorff(interval(1,GAOL_INFINITY),interval(2,GAOL_INFINITY)) << ' '
     << hausdorff(interval(1,GAOL_INFINITY),interval(1,2));
➥ 2 1 inf
```

`double interval::`**`midpoint`**`(void) const`

Returns the midpoint of `self`: `mid` of IEEE 1788-2015. For a and b two finite doubles, and MAX the largest positive double, `std::numeric_limits<double>::max()`, the cases are:

$$\begin{cases} \texttt{midpoint}(\varnothing) & = \text{NaN} \\ \texttt{midpoint}([-\infty, +\infty]) & = 0 \\ \texttt{midpoint}([-\infty, b]) & = -\text{MAX} \\ \texttt{midpoint}([a, +\infty]) & = \text{MAX} \\ \texttt{midpoint}([a, b]) & = \updownarrow(a+b)/2\updownarrow \end{cases}$$

The midpoint of $[a, b]$ is rounded to nearest, ties to even. It is the half of $a + b$, or the sum of the halves of a and b where $a + b$ may overflow [5], that is where $|a|$ or $|b|$ is 2^{1023} or more. This case is tested first, so that `midpoint()` of [MAX, MAX] raises no overflow exception. [GAOL v5]

`interval interval::`**`mid`**`(void) const`

Returns the tightest interval enclosing the midpoint of `self`, which is included in `self`. With the notations of `midpoint()`, the cases are:

$$\begin{cases} \texttt{mid}(\varnothing) & = \varnothing \\ \texttt{mid}([-\infty, +\infty]) & = [0, 0] \\ \texttt{mid}([-\infty, b]) & = [-\text{MAX}, -\text{MAX}] \\ \texttt{mid}([a, +\infty]) & = [\text{MAX}, \text{MAX}] \\ \texttt{mid}([a, b]) & = [\downarrow(a+b)/2\downarrow, \uparrow(a+b)/2\uparrow] \end{cases}$$

The midpoint is computed as `midpoint()` computes it, with the rounding outward. GAOL 4 did not enclose the midpoint of some intervals whose bounds are subnormal numbers. [GAOL v5]

`double interval::left(void) const`

■ `left`$([x, y]) = x$

Returns the left bound of `self`: NaN for the empty set, whose bounds are NaN. `gaol_ieee1788::inf()` returns $+\infty$ there, as IEEE 1788-2015 has it.

`double interval::right(void) const`

■ `right`$([x, y]) = y$

Returns the right bound of `self`: NaN for the empty set. `gaol_ieee1788::sup()` returns $-\infty$ there.

`interval abs(const interval& I)`

■ $\text{abs}(I) = \{|x| \in \mathbb{R}^+ \mid x \in I\}$

Returns the absolute value of I.

Example

```
1 cout << abs(interval(-5,6)) << ' '
2      << abs(interval(-4,-2));
3 ➥ [0, 6] [2, 4]
```

`double chi(const interval& I)`

Returns the function `chi` of Ratschek and Rokne [12], which characterizes the degree of symmetry of an interval:

$$\text{for } I = [a, b], \quad \text{chi}(I) = \begin{cases} -1 & \text{if } I = [0, 0] \\ a/b & \text{if } |a| \leqslant |b| \\ b/a & \text{otherwise.} \end{cases}$$

It is NaN for the empty set, 1 for $[-\infty, +\infty]$, and 0 for the other intervals with an infinite bound.

Example

```
1 cout << chi(interval(3,6)) << ' '
2      << chi(interval(-6,3)) << ' '
3      << chi(interval::emptyset()) << ' '
4      << chi(interval::universe()) << ' '
5      << chi(interval(-5,GAOL_INFINITY));
6 ➥ 0.5 -0.5 nan 1 0
```

`interval` **`min`**`(const interval& I, const interval& J)`

■ $\min([a,\, b], [c,\, d]) = [\min(a, c),\, \min(b, d)]$

Returns the minimum of two intervals.

Example

```
cout << min(interval(5,6),interval(3,9)) << ' '
     << min(interval::emptyset(),interval(3,8));
➥ [3, 6] [empty]
```

`interval` **`max`**`(const interval& I, const interval& J)`

■ $\max([a,\, b], [c,\, d]) = [\max(a, c),\, \max(b, d)]$

Returns the maximum of two intervals.

Example

```
cout << max(interval(5,6),interval(3,9)) << ' '
     << max(interval::emptyset(),interval(3,8));
➥ [5, 9] [empty]
```

8.2 Integer functions

The functions of this section round the elements of an interval to integers. They are those of IEEE 1788-2015 (Table 9.1), but `integer()`, which is GAOL's own; their names in the standard are given with the operations of Chapter 10.

Proposition 8.2.1 *Each function of this section but* `integer()` *is non-decreasing, so that the bounds of its result are its values at the bounds of the interval; those of* `integer()` *are the* `ceil` *of the left bound and the* `floor` *of the right one. Each is exact. None of them depends on the rounding direction.*

`interval` **`floor`**`(const interval& I)`

■ [`floor`(I.`left`()), `floor`(I.`right`())]

The elements of I rounded down to an integer.

Example

```
cout << floor(interval(4.5,6.5)) << ' '
     << floor(textToInterval("[-10.4,3.5]"));
➥ [4, 6] [-11, 3]
```

`interval` **`ceil`**`(const interval& I)`

■ [`ceil`(I.`left`()), `ceil`(I.`right`())]

The elements of I rounded up to an integer.

Example

```
cout << ceil(interval(4.5,6.5)) << ' '
     << ceil(textToInterval("[-10.4,3.5]"));
➥ [5, 7] [-10, 4]
```

`interval` **`integer`**`(const interval& I)`

■ [`ceil`(I.`left`()), `floor`(I.`right`())]

Narrows the bounds down to the closest integers: the empty set when I holds no integer. The bounds of the result are still `double` numbers, which an integral type may not represent.

Example

```
cout << integer(interval(4.5,6.5)) << ' '
     << integer(interval(4.2,4.8));
➥ [5, 6] [empty]
```

`interval` **`sign`**`(const interval& I)`

The signs of the elements of I: -1 below 0, 0 at 0, 1 above (sign). [GAOL v5]

`interval` **`trunc`**`(const interval& I)`

The elements of I rounded toward zero (trunc). [GAOL v5]

`interval` **`round_ties_to_even`**`(const interval& I)`
`interval` **`round_ties_to_away`**`(const interval& I)`

The elements of I rounded to the nearest integer, halfway values to the even one, respectively away from zero (roundTiesToEven, roundTiesToAway). [GAOL v5]

Example

```
cout << sign(interval(-2,3)) << ' '
     << trunc(interval(-2.5,3.7)) << ' '
     << round_ties_to_even(interval(0.5,2.5)) << ' '
     << round_ties_to_away(interval(0.5,2.5));
➥ [-1, 1] [-2, 3] [0, 2] [1, 3]
```

8.3 Splitting methods

A branch-and-bound algorithm cuts the intervals it cannot discard. GAOL cuts an interval $[a, b]$ at a double c of the interval that both parts share: the parts $[a, c]$ and $[c, b]$ cover the interval exactly. `split()`, `split_left()` and `split_right()` cut at `midpoint()`; `bisect()` cuts at a ratio of the width chosen by the caller; `is_bisectable()` tells whether a cut gives two parts smaller than the interval.

Definition 8.3.1 *An interval is bisectable when a double lies strictly between its bounds; the two parts of a cut at such a double are then both smaller than the interval. The empty set, a point interval and an interval of two consecutive doubles are not bisectable, nor are the half-lines $[MAX, +\infty]$ and $[-\infty, -MAX]$, MAX being the largest double.*

Remark 8.3.2 *A cut point is a double of the interval. An unbounded interval is therefore cut at the largest double or at its opposite, whatever the ratio, and the whole line at 0; the half $[MAX, +\infty]$ keeps an infinite width, so that a loop which cuts an interval until its width falls under a threshold never ends on it. Such a loop stops on* `is_bisectable()`*, which is* `false` *for this half (see Section 3.10, page 77).* [GAOL v5]

`void interval::split(interval& I1, interval& I2) const`

Splits `self` into two parts at `midpoint()`, and returns the left part in *I1* and the right part in *I2*. *I1* or *I2* may be `self` itself.

Example

```
interval I1a, I2a,
         I3(1.0,next_float(1.0)),
         I1b, I2b;
interval(4,5).split(I1a,I2a);
I3.split(I1b,I2b);
cout << boolalpha << I1a << ' ' << I2a << ' '
     << I1b.set_eq(interval(1.0)) << ' ' << I2b.set_eq(I3) << endl;
➥ [4, 4.5] [4.5, 5] true true
```

`interval interval::split_left(void) const`

■ `split_left`$([a, b]) = [a, m]$

Splits `self` into two parts at $m =$ `midpoint()`, and returns the left part.

Example

```
cout << interval(4,5).split_left();
➥ [4, 4.5]
```

`interval interval::split_right(void) const`

■ `split_right`$([a, b]) = [m, b]$

Splits `self` into two parts at $m =$ `midpoint()`, and returns the right part.

Example

```
cout << interval(4,5).split_right();
➥ [4.5, 5]
```

`std::pair<interval, interval> interval::bisect(double ratio) const`

■ `bisect`$([a, b], \rho) = ([a, c], [c, b])$

Returns the two parts of `self` on either side of the cut point c, a double of $[a, b]$ that both parts share, so that they cover `self` exactly. For $0 < \rho < 1$, c is $a + \rho(b - a)$, computed as $a(1 - \rho) + b\rho$ or, where a bound is 2^{1023} or more in magnitude, as twice $\frac{a}{2}(1 - \rho) + \frac{b}{2}\rho$ kept between $\frac{a}{2}$ and $\frac{b}{2}$, so that no overflow exception is raised for finite bounds, and kept within $[a, b]$; when `is_bisectable()` is true, c is moreover strictly between a and b, so that both parts are smaller than `self`. $\rho = 0.5$ cuts at `midpoint()`, as `split()` does, and so does every ratio for an unbounded interval: a half-line is cut at the largest double or at its opposite, the whole line at 0. The empty set gives two empty sets, and a point interval itself twice. A ratio outside $(0, 1)$, or NaN, throws `std::domain_error` (see Section 13.5.3, page 268). [GAOL v5]

Example

```
std::pair<interval, interval> p = interval(0,4).bisect(0.25);
cout << p.first << ' ' << p.second;
➥ [0, 1] [1, 4]
```

`bool interval::is_bisectable(void) const`

Returns `true` when a double lies strictly between the bounds of `self` (Definition 8.3.1), so that `bisect()` gives two parts, each smaller than `self`. Returns `false` for the empty set, a point interval, two consecutive doubles, and the half-lines [MAX, $+\infty$] and $[-\infty, -\text{MAX}]$, which `split()` cuts at their finite bound. [GAOL v5]

Example

```
cout << boolalpha << interval(1,2).is_bisectable() << ' '
     << interval(1,next_float(1)).is_bisectable();
➥ true false
```

8.4 Union and intersection

Intervals are sets, and two of them can be intersected or joined. The intersection of two intervals is an interval, possibly empty; their union is not always one, and GAOL returns its hull, the smallest interval that contains both. The operators `&` and `|` compute them, and the functions `intersect()` and `hull()` give them a name.

Definition 8.4.1 *Let $I_1 = [a, b]$ and $I_2 = [c, d]$ be two non-empty intervals. Their intersection $I_1 \cap I_2$ is the empty set when $b < c$ or $d < a$, and $[\max(a, c), \min(b, d)]$ otherwise. The hull of their union, $\square(I_1 \cup I_2)$, is $[\min(a, c), \max(b, d)]$.*

Remark 8.4.2 *GAOL 4 kept the greatest left bound and the smallest right bound of two disjoint intervals: $[1, 2] \cap [3, 4]$ was $[3, 2]$, which the operations computing on the bounds did not take for empty. GAOL v5 returns the empty set,* `interval::emptyset()`*, whose bounds are NaN.* [GAOL v5]

`interval` **`operator&`**`(const interval& I1, const interval& I2)`

■ *I1* ∩ *I2*

Returns the intersection of *I1* and *I2*: intersection of IEEE 1788-2015. The intersection of disjoint intervals is the empty set `interval::emptyset()` (Remark 8.4.2).

Example

```
1 cout << (interval(4,6) & interval(5,9)) << ' '
2      << (interval(1,2) & interval(3,4));
3 ➥ [5, 6] [empty]
```

`interval` **`operator|`**`(const interval& I1, const interval& I2)`

■ □(*I1* ∪ *I2*)

Returns the hull of the union of *I1* and *I2*: convexHull of IEEE 1788-2015.

Example

```
1 cout << (interval(3,6) | interval(9,12));
2 ➥ [3, 12]
```

`interval` **`hull`**`(const interval& I1, const interval& I2)`
`interval` **`intersect`**`(const interval& I1, const interval& I2)`

`I1 | I2` and `I1 & I2` under a name, for an algorithm written with names rather than operators, and for `std::accumulate` or a pointer to a function. They are in the namespace `gaol` too. [GAOL v5]

Example

```
1 cout << hull(interval(1,2), interval(4,5)) << ' '
2      << intersect(interval(1,3), interval(2,5));
3 ➥ [1, 5] [2, 3]
```

8.5 Discussion

This chapter has described the functions that read an interval rather than map it: the measures of an interval, the roundings of its elements to integers, its cuts, and the set operations on two intervals. Two conventions run through them. The empty set, whose bounds are NaN, gives NaN to every function that returns a double and itself to every function that returns an interval, but the hull of a union, `hull(interval::emptyset(), x)` being `x`; GAOL 4 returned -1 for the width of the empty set and $[3, 2]$ for the intersection of $[1, 2]$ and $[3, 4]$, values that the operations on the bounds did not take for empty. An unbounded interval gives $+\infty$ to its width and its radius, and the largest double, or its opposite, or 0 for the whole line, to its midpoint and to its cut points, which have to be doubles of the interval.

The roundings follow the containment property (see Section 1.3, page 9). The width is rounded upward, so that it bounds the exact width from above, and `width_enclosure()` encloses the exact width when its lower bound matters; the midpoint is rounded to nearest, with the sum of the halves where the sum of the bounds would overflow, and `mid()` encloses it; `inflate()` rounds outward; the cut point of `bisect()` is computed so that it cannot overflow and stays within the interval. The integer functions are exact in every rounding direction. The union of two intervals is given by its hull, the standard's convexHull, which adds the reals between two disjoint intervals to the result: the intersection is exact, the union is enclosed.

The examples of this chapter print their intervals with the operator `<<`, as `[4, 4.5]` or `[empty]`. The next chapter describes how GAOL reads and writes intervals, with the formats it accepts and the number of digits it writes.

Chapter 9

Input and output

An interval enters a program and leaves it as text: a literal in a source file, a line of a data file, a string typed by the user, a result printed on the screen. A decimal number is seldom a double, and a double is seldom written exactly with a few decimal digits. Reading and writing must therefore preserve the containment property (see Section 1.3, page 9): the text read gives an interval that contains the number or the set it writes, and the text written for an interval in the formats **`bounds`** and **`hexa`**, or by intervalToText, read back, gives an interval that contains it. GAOL v5 does both, whatever the C library and the locale of the program. [GAOL v5]

In this chapter, we describe how GAOL v5 reads and writes intervals. The first section covers the reading: the operator `>>` of the input streams, then the language of the strings that it and `textToInterval()` accept, with its numbers, the uncertain form of IEEE 1788-2015, its functions, its bounds and infinities, the limits of the reader and its errors. The second section covers the writing: the operator `<<`, the exact text of `exact_string()`, the conversion to `std::string`, the five output formats, the number of digits, and a complete program. The functions that read and write with the names of the standard, textToInterval, intervalToText, intervalToExact and exactToInterval, belong to the namespace `gaol_ieee1788` and are described in Chapter 10 (see Section 10.4, page 203).

9.1 Reading intervals

An interval is read from a stream with the operator `>>`, as a number is, and built from a string with `textToInterval()` (Chapter 5). Both read the language of Section 9.1.1.

`istream& operator>>(istream& in, interval& I)`

Reads a line from the input stream *in*, and assigns to I the interval it writes. The blanks before the line, the ends of lines included, are skipped first, as before a number, even under `std::noskipws`, which keeps a number from skipping them: an empty line, or one holding only blanks, is never a line to read. An interval cannot be written on several lines.

If the line is syntactically ill-formed, I is set to the empty set. A `std::invalid_argument` exception is then thrown (see Section 13.5.1, page 266) if the library was compiled with exceptions enabled (see Section 2.5.4, page 40); if they were disabled, an error message is printed to `cerr` and the program aborts. An ill-formed line also sets the `failbit` of *in*, which `in.clear()` resets before reading on. A second argument of `nth_root` that is not an integer, as in `nth_root(8,1.5)`, throws `std::domain_error` instead; a call with a wrong number of arguments throws `std::invalid_argument`, and an argument outside the domain of a function gives the empty set, as `sqrt(-1)` does.

Where no line is left, *in* gets its `failbit`, I is left as it was, and nothing is thrown, as for a number: `while (in >> x)` reads the lines one after the other, the blank ones skipped, and ends with the input, a file ending with an empty line included. [GAOL v5] GAOL 4 read the empty text left, and threw `input_format_error`, as it did on an empty line, so that `in >> d >> x` over the lines `1.5` and `[1, 2]` threw at the end of the first one.

Example

```
std::istringstream in("[1, 2]\n[0.1, 0.2]\n");
interval a, b;
in >> a >> b;
cout << a << ' ' << b;
➥ [1, 2] [0.09999999999999999, 0.2000000000000001]
```

Example

```
std::istringstream in("1.5\n\n[1, 2]\n\n");
double d;
interval x;
in >> d >> x;
cout << d << ' ' << x << ' ';
cout << (in >> x ? "more" : "end");
➥ 1.5 [1, 2] end
```

9.1.1 The input format

A string to be translated into an interval has the syntax below, with the terminals in lower case and the non-terminals in italic upper case.

```
EXPR
        : NUMBER
        | UNCERTAIN_NUMBER // An interval literal, never a bound by itself
        | LITERAL
        | dmin            // Smallest positive normal floating-point number
        | dmax                    // Largest positive floating-point number
        | pi
        | inf     // [MAX, +infinity] as an expression, +infinity as a bound
        | infinity                                                 // The same
        | empty                                                 // Empty interval
        | EXPR + EXPR
        | EXPR - EXPR
        | EXPR * EXPR
        | EXPR / EXPR
        | - EXPR
        | + EXPR
        | FUNCTION_CALL
        | ( EXPR )
        ;

LITERAL
        : [ EXPR ]
        | [ EXPR , EXPR ]
        | [ EXPR , ]                                          // Right bound +∞
        | [ , EXPR ]                                           // Left bound -∞
        | [ , ]                                              // Entire real line
        | [ entire ]                                         // Entire real line
        | [ ]                                                   // Empty interval
        | < EXPR , EXPR >
        ;

FUNCTION_CALL
        : NAME ( EXPR )
        | NAME ( EXPR , EXPR )
        | NAME ( EXPR , EXPR , EXPR )
        ;
```

Every string is an expression, computed with interval arithmetic. The operators * and / bind tighter than + and -, the four associate to the left, and a sign binds tighter than all of them: `"1-2-3"` is -4 and `"2+3*4"` is 14. The literals of intervals, the numbers, the constants and the functions combine with the operators in any order, and a literal stands wherever a number may, in a bound too. [GAOL v5] `1+[1,2]`, `2*cos([0,1])` and `[cos([0,1]), 2]` are read. `[empty]` is the form `[`*EXPR*`]` of the expression `empty`.

Remark 9.1.1 *GAOL 4 read a string with two grammars, one for the intervals between brackets and one for the numbers. It refused* `1+[1,2]` *and* `[1,2]+1+[1,2]`*,*

while it read `[1,2]*2`. *GAOL v5 reads every string with the one grammar above.* [GAOL v5]

Spaces, tabs and newlines are not significant except in numbers, and the case of letters is ignored, as IEEE 1788-2015 has it for its literals: `[Empty]`, `[1, Inf]`, `SIN(1)` and `1E3` are read, whatever the locale of the program. [GAOL v5] The sign "+" before a number and before `inf` is optional.

Numbers. A *NUMBER* is written in decimal, with an optional point and an optional exponent (`12`, `0.1`, `1e-3`), in hexadecimal (`0x1f`, `0x1.3p-1`), in octal (`0o17`) or in binary (`0b101`, `0b1.1p3`); the digits after `0x`, `0o` or `0b` have an optional point and an optional binary exponent after a `p`. [GAOL v5] A number that begins with 0 is decimal: `010` is 10, as IEEE 1788-2015 has it (9.7.2), and octal is written `0o10`, as in Python.

Numbers are read exactly, whatever their number of digits: a number that is a double is read as that double, and any other one is replaced by the tightest interval of doubles containing it. [GAOL v5] GAOL 4 read them with the `strtod()` of the C library, which did not enclose `0.1` where it rounds to nearest in every rounding direction (the C runtime of Windows, musl on 64-bit ARM processors). The point of a number is a point whatever the locale of the program, a locale writing a decimal comma included (as `setlocale(LC_ALL, "")` sets under a French one). [GAOL v5] GAOL 4 read `0.1` as 0 there, `strtod()` stopping at the point. The modes of the processor that flush the subnormal numbers to zero, flush-to-zero and denormals-are-zero, which a program linked with `-Ofast` has (see Section 4.8, page 104), do not change what a number is read as: `1e-310` is read as the tightest interval of doubles containing it, and `0` as 0. [GAOL v5] Reading a long number takes a time quadratic in its number of digits: `"0."` followed by n digits `3` takes 0.011 s for $n = 20\,000$, 0.18 s for 80 000 and 2.8 s for 320 000 (about 28 s expected for a million), and the uncertain form 2.2 to 2.4 times as long (Intel i7-1185G7, GCC 15.2, the best of five runs, 7 November 2026). [GAOL v5]

The uncertain form. An *UNCERTAIN_NUMBER* is written as IEEE 1788-2015 defines it (9.7.4 and 12.11.3).

Definition 9.1.2 *An uncertain number is a decimal number followed by* `?`*, a radius in units of its last digit (half a unit when it is left out, and an infinite radius for a second* `?`*), an optional direction,* `u` *or* `d`*, which keeps only the radius above or below, and an optional exponent.*

`3.56?1` is [3.55, 3.57], `3.56?1e2` is [355, 357], and `-10??u` is $[-10, +\infty]$. [GAOL v5] An uncertain number is an interval literal of its own, not a bound by itself: `"[5?1]"` and `"[1, 3.56?1]"` throw `std::invalid_argument`, as IEEE 1788-2015 has it (12.11.4), while `"3.56?1*2"` is [7.1, 7.14], and a bound computed from one is read, `"[2*5?1]"` being [8, 12]. A sign written next to an uncertain number, as

`10-3.56?1` or `10+3.56?1`, is read as a part of the number, and the text throws `std::invalid_argument`, as does `10 -3.56?1`: a blank is needed after the sign, `10 - 3.56?1`. [GAOL v5]

Functions. A *NAME* is the name of a function of GAOL, with its arguments, whatever the case of its letters. The names of GAOL are read by `gaol::textToInterval()`, from one string or from two, and by `operator>>`; the names of IEEE 1788-2015 are read by `gaol_ieee1788::textToInterval()` (see Section 10.4, page 203), as each namespace has its own `pow`. [GAOL v5] The names of GAOL are the following.

> `sqr`, `sqrt`, `cbrt`, `pow`, `nth_root`, `exp`, `exp2`, `exp10`, `expm1`, `exp2m1`, `exp10m1`, `log`, `log2`, `log10`, `log1p`, `log2p1`, `log10p1`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`, `sinpi`, `cospi`, `tanpi`, `asinpi`, `acospi`, `atanpi`, `atan2pi`, `hypot`, `compoundm1`, `rsqrt`, `sign`, `ceil`, `floor`, `trunc`, `integer`, `round_ties_to_even`, `round_ties_to_away`, `abs`, `min`, `max`, `inverse`, `fma`

Each is the function of GAOL of that name (Chapters 7 and 8); `pow` is the `pow` of the namespace `gaol` (see Section 7.2.2, page 151), and `cbrt(x)` is `nth_root(x,3)`. A name that is not in the list, or a call with a wrong number of arguments, throws `std::invalid_argument`. The second argument of `nth_root` shall be a point interval that can be evaluated as an integer, negative ones included; otherwise, a `std::domain_error` exception is thrown, or an error is reported, depending on whether the exceptions were enabled at configuration time (see Section 2.5.4, page 40). [GAOL v5] The integer may be of any size a double holds: `"nth_root(8,3000000000)"` is `nth_root(8, 3000000000LL)`, where it was refused beyond the `int`s. One of 2^{63} or more in magnitude is taken as $\pm(2^{63}-2)$: the integer powers are the same, and the enclosure of the roots of that order holds those of the larger orders. The second argument of `pown` and `rootn`, with the names of the standard, is read in the same way. GAOL 4 read the trigonometric and hyperbolic functions, `exp`, `log`, `pow`, `sqrt` and `nth_root` only.

Bounds. The notations "n" and "$[n]$" are equivalent, but for `inf` and `-inf`, whose literals `[inf]` and `[-inf]` are empty (see Infinities below), and for an uncertain number, which is no bound. The expressions in the bounds are evaluated with interval arithmetic, intervals included; the left (respectively the right) bound of the result is then used, depending on the side it appears in: `"[cos([0,1]), 2]"` is $[\cos 1, 2]$. A bound left out is infinite.

An interval literal `[l, u]` whose left bound is above its right bound is the empty set, as in IEEE 1788-2015: `"[2,1]"`, and also `"[9007199254740993, 9007199254740992]"` and `"[1.00000000000000001, 1]"`, whose numbers are compared exactly with the doubles around them, a number that is not a double

lying strictly between the two doubles around it. [GAOL v5] GAOL 4 read the last two as the points 2^{53} and 1. Two numbers that are not doubles and lie between the same two doubles are compared exactly too: `"[1.00000000000000002, 1.00000000000000001]"` is the empty set, and `"[1.00000000000000001, 1.00000000000000002]"` is the interval from the double below to the double above them, as is the same number written twice. Three kinds of bounds are not told apart: `pi` and another number lying between the same two doubles; a number far beyond the doubles (about 10^{311} or more, or 10^{-331} or less, in decimal, about 2^{1026} or more, or 2^{-1081} or less, in the other bases); and bounds that are computed, as `1/3`. The literal is then the interval from the lower bound of the first to the upper bound of the second (case (b) of 12.12.7, which GAOL signals by no exception: see Section 10.5, page 203).

The two bounds in the string `<a, b>` must be expressions that evaluate to the same double whatever the rounding direction; a `std::invalid_argument` exception is raised otherwise, as for `"<0.1, 0.1>"`, a number that is not a double being no bound of `<a, b>`. [GAOL v5] GAOL 4 read `"<0.1, 0x1.9999999999999p-4>"`, the double just below 0.1, as the point [`0x1.9999999999999p-4`]. A bound that is computed is a double when its value is a point, as `0.5`, `(0.25+0.25)` and `[0.5]` are; `"<[0.1], 0x1.9999999999999p-4>"` and `"<0x1.5555555555556p-2, 1/3>"`, whose first or second bound has the two doubles around the number for its value, throw `std::invalid_argument` too, where GAOL 4 read them as the point of the other bound. [GAOL v5] The two doubles are compared by their bits, −0 and +0 being the same bound, so that the mode of the processor that reads the subnormal numbers as zero (denormals-are-zero) changes nothing: `"<0x1p-1074, 0x1p-1073>"`, two subnormal doubles, throws under it too, where a comparison of the bounds as doubles would take them for one. [GAOL v5]

Remark 9.1.3 *GAOL 4 wrote the degenerate intervals in the form* `<a, b>`*, and the texts it wrote for those whose digits are exact, as* `<4, 4>`*, are still read.* `operator<<` *writes such an interval* `[a]`*, the literal of IEEE 1788-2015 for a point, and any other with its two bounds (see Section 9.2.2, page 191).*

Infinities. An interval literal whose left bound is $+\infty$ or whose right bound is $-\infty$ is the empty set, as numsToInterval of IEEE 1788-2015 has it: `"[inf]"` and `"[-inf, -inf]"` are empty. [GAOL v5] The expression `"inf"` alone is still [MAX, $+\infty$], and `"-inf"` [$-\infty$, −MAX] (see Section 5.1.5, page 122).

Threads. Several threads may read strings at once, in parallel: the lexer is reentrant and the parser pure, and each reading of a string has a state of its own. [GAOL v5] GAOL 4 kept that state in global variables, and two threads reading a string each crashed.

Long and nested strings. A string may be as long as memory allows: a sum of 100000 terms is read in a time proportional to its length, and takes no more of

the stack of the program than a sum of two terms. [GAOL v5] The reader of GAOL 4 built a tree with a node for each operation, which was evaluated and deleted with one call within another per node: a sum of 100000 terms overflowed the stack and crashed the program. The operations are now computed as they are read, and the expressions that a program builds through `gaol/parser/gaol_expression.h` are evaluated and deleted without recursion too. A token (a number, a name or a run of blanks) holds 2^{29} characters at most, the buffer of the lexer: a longer one throws `std::bad_alloc`. The nesting of a string is limited by the stack of the parser, which holds 10000 entries. An open parenthesis or bracket, or a sign, takes one entry until the expression it opens is complete; the name and the parenthesis of a call take two; an operand and the operator or the comma after it take two, until the operand that follows them is read. About 10000 nested parentheses, brackets or signs, 5000 nested calls of one argument and 3300 levels of `1+(1+(...))` are read; a string nested deeper is refused as any string that is not an expression, and throws `std::invalid_argument`.

A `std::string`, given to `textToInterval()` or read by `operator>>`, is read whole: a NUL character is no character of any token, and a string holding one is not an expression. [GAOL v5] The reader of GAOL 4 read it up to the NUL, so that `"[1,2]"` followed by a NUL and any text was [1, 2], and a string of $2^{32}+3$ characters was read as its first 3, as `"1+1"` for `"1+1+1+..."`: the lexer of flex held the length of a string in an `int`.

Errors. A string that is not an expression throws `std::invalid_argument`. An error found anywhere in the string stops the reading, and no interval is given: `"[nth_root(8,1.5)]+[1,2]"` throws, where the two grammars of GAOL 4 gave $[-\infty, +\infty]$. [GAOL v5] Where memory runs out, the reading throws `std::bad_alloc`, and frees all it allocated. [GAOL v5] In GAOL 4, flex wrote "out of dynamic memory" and ended the program, and the parser, which read a stack it could not grow as a string nested too deep, threw no such exception. Without exceptions, an error that the reader finds as it computes (a wrong call, an uncertain number as a bound, two different bounds in `<a, b>`, or a bound of it that is no double) makes the reading fail, as a syntax error does: `gaol_core::parse_interval()`, of `gaol/parser/gaol_parser.h`, returns `false`, and `gaol::textToInterval()` and `operator>>` report the error and abort, as for any string that is not an expression. [GAOL v5] GAOL 4 reported the error, and aborted, inside the reader.

Example 9.1.4 *The program below reads intervals from strings of the language: a literal with a computed bound, the constants, the form of* `textToInterval()` *with two strings, a literal divided by a literal, the bounds left out, a hexadecimal number, the uncertain form, and a literal in a sum and in a bound. What the printing statements write follows them.*

Example

```
1 interval x = textToInterval("[4, 6*7]");
2 interval y = textToInterval("[-inf, dmax]");
```

```
interval z = textToInterval("[3.14,3.15]/8", "[3.14,3.15]/7");
interval t = textToInterval("[3.14,3.15]/[7,8]");
cout << textToInterval("[1,]") << ' ' << textToInterval("[ ]") << ' '
     << textToInterval("[Entire]") << endl;
➥ [1, inf] [empty] [-inf, inf]
cout << textToInterval("[0x1.3p-1, 2/3]") << ' '
     << textToInterval("3.56?1") << endl;
➥ [0.59375, 0.6666666666666668] [3.549999999999999, 3.570000000000001]
cout << textToInterval("1+[1,2]") << ' '
     << textToInterval("[cos([0,1]), 2]") << endl;
➥ [2, 3] [0.5403023058681396, 2]
```

9.2 Writing intervals

An interval is written to a stream with the operator `<<`, as any primitive type of C++ is. The text depends on the active format (Section 9.2.2); the empty set is written the same way in every format.

`ostream& operator<<(ostream& out, const interval& I)`

Prints the interval I to the output stream *out*. The way the intervals are displayed depends on the active format (Section 9.2.2). Whatever the format, an empty interval is displayed as `[empty]`, as IEEE 1788-2015 writes the empty set (12.11.3). [GAOL v5] The formats **`width`** and **`center`** of GAOL 4 wrote `empty`, which is no interval literal.

In the decimal formats, each bound is rounded outward, whatever the C library: its magnitude is written by the C library, rounded to nearest, then compared exactly with the bound, and its last digit is moved by one when it is on the wrong side. The text read back therefore encloses the interval written. [GAOL v5] GAOL 4 let the C library round each bound in the direction it set, which the C runtime of Windows and musl do not honour.

`std::string exact_string(const interval& I)`

Returns what `operator<<` writes in the format **`hexa`** (Section 9.2.2), without reading nor changing the global output format: `"[empty]"` for the empty set, `"[a]"` for a degenerate interval (`"[0x0p+0]"` for a zero of either sign), and otherwise the two bounds in the hexadecimal-significand form of IEEE 1788-2015 (13.4.1), which `textToInterval()` reads back as the same doubles, a zero degenerate interval as [+0, +0], the same set. This is the exact text representation of the standard, its intervalToExact. [GAOL v5] The text is the same whatever the locale of the program, a locale writing a decimal comma included, and whatever the C library.

Example

```
cout << exact_string(textToInterval("0.1"));
➥ [0x1.9999999999999p-4, 0x1.999999999999ap-4]
```

9.2.1 Converting intervals to strings

The class `interval` provides a conversion operator to the standard type `std::string`, which writes the interval in the active format.

Example

```
interval I(3,4);
string s = "test line embedding " + string(I) + " as a string";
// Now, s is "test line embedding [3, 4] as a string"
```

9.2.2 The output format

Intervals may be displayed in five formats, chosen by the static method `interval::format()` described after them.

- **`agreeing`**. All the digits that are the same in the left and in the right bound are printed once, followed by an interval holding the remaining digits:

  "`3.141~[5926, 6001]`" stands for "[3.1415926, 3.1416001]"

  Each bound is written with the digits of the precision and the flag `showpoint`, in the global locale of the program. The digits are shared only by two bounds that are finite, not zero and of the same sign, written with as many digits before the point and the same exponent, and that share their first digit that is not 0: the other intervals are printed as in the format **`bounds`**. The digits of each bound are kept, but for the zeros that end them, [1.25, 1.2567] being `1.25~[0, 67]`. [GAOL v5] GAOL 4 dropped from both bounds what followed the last character of the left one that is not a zero, and wrote [1.25, 1.2567] `1.25~[, ]`. It shared the digits of every interval whose right bound is at most ten times the left one, [1, 10] being `1~[., 0.]`, [4, 6] `~[4., 6.]`, without a shared digit, and a zero `~[-0., 0.]`, and printed every interval with a negative bound with its bounds. A point interval that the digits write exactly is printed as a single number, without the brackets: `interval(4.0)` is `4.000000000000000`, where the format **`bounds`** prints `[4]`; any other point interval is printed as any interval is, `interval(0.1)` being `0.100000000000000~[0, 1]`. The point zero is printed `[0]`, as in the format **`bounds`** (the formats **`width`** and **`center`** print it `0`, and **`hexa`** `[0x0p+0]`).

  Example

  ```
  interval::format(interval_format::agreeing);
  std::cout << interval(4,6) << ' ' << interval(1.25,1.2567) << ' '
            << interval(-1.2567,-1.25) << std::endl;
  ➥ [4, 6] 1.25~[0, 67] -1.25~[67, 0]
  ```

- **`bounds`**. The bounds are printed between square brackets. A degenerate interval whose value is written exactly by the digits displayed is printed as that value alone between square brackets, the literal of IEEE 1788-2015 for a point (12.11), which `textToInterval()` reads back as the same point:

Example

```
interval::format(interval_format::bounds);
std::cout << interval(3,5) << ' ' << interval(4) << std::endl;
➥ [3, 5] [4]
```

A zero is printed `[0]` whatever the signs of its bounds: $[-0, +0]$, which `interval::zero()` is with the SSE2 and the NEON intervals, $[+0, +0]$ and $[-0, -0]$ are the same set, which `[0]` and `[-0]` are both read back as, and the text is then the same on every build (the format **`hexa`** prints it `[0x0p+0]`). A degenerate interval whose value has more digits than those displayed is printed like any other interval, with its two bounds rounded outward: `interval(0.1)` is printed `[0.1, 0.1000000000000001]`, which is read back, as any other text, as an interval containing it. The bounds are printed with a decimal point, whatever the locale of the stream, so that the reader, whose decimal point is a point whatever the locale, reads the text back: under a locale writing a decimal comma, it would take `[-2,5]` for the interval $[-2, 5]$, and refuses `[0,5, 2,5]`. [GAOL v5] GAOL 4 printed every degenerate interval between angles, the text of its value rounded downward and upward: `interval(4)` was `<4, 4>`, a form of GAOL that is no literal of the standard, and `interval(0.1) <0.1, 0.1000000000000001>`, which its reader refused (`std::invalid_argument`), the angles standing for one double only. The reader still takes the angles (see Section 9.1.1, page 184): a text such as `<4, 4>`, written by GAOL 4, is read back.

- **`width`**. The midpoint and the radius are printed, `midpoint()` and `rad()`, the mid and rad of IEEE 1788-2015 (12.12.8): the interval is contained in the midpoint plus or minus the radius, which is the smallest number that makes it so:

Example

```
interval::format(interval_format::width);
std::cout << interval(4,5) << std::endl;
➥ 4.5 (+/- 0.5)
```

The midpoint is written rounded to nearest and the radius rounded upward, so that a radius that is not zero is never written 0. Rounded upward, a radius may look noisy with the default 16 digits: $[0.1, 0.3]$ is written `0.2 (+/- 0.1000000000000001)`, its radius being the double nearest to 0.1, which is a little above 0.1; this is intended. A degenerate interval has no radius: it is written as its midpoint alone. An unbounded interval has an infinite radius, its midpoint being 0 or plus or minus the largest double (`[3, inf]` is `1.797693134862316e+308 (+/- inf)`). The midpoint and the radius are written with the decimal point of the locale of the stream and its grouping of the digits, the radius without a sign, even with the flag `showpos`, and a midpoint 0 as `0`. [GAOL v5] GAOL 4 wrote $(a+b)/2$ and $(b-a)/2$ rounded to nearest,

which do not contain the interval ($[1,\ 1+2^{-52}]$ was `1 (+/- 1.11e-16)`), a radius 0 for $[0,\ 5 \cdot 10^{-324}]$, and `inf` for the midpoint of $[10^{308},\ 1.7 \cdot 10^{308}]$; its manual and its header described this number as the width.

- **`center`**. The midpoint alone is printed, as the format **`width`** prints it. [GAOL v5]
- **`hexa`**. The left and the right bounds are printed in the hexadecimal-significand form of IEEE 1788-2015 (13.4.1), which is exact and which `textToInterval()` reads back as the same doubles, without the round-off error of a binary-to-decimal conversion: this is the recovery requirement of the standard (13.4), met whatever the precision, the flags and the locale of the stream. As in the format **`bounds`**, a degenerate interval is printed as its value alone between square brackets, `[0x1.8p+0]` for `interval(1.5)`, and a zero `[0x0p+0]` whatever the signs of its bounds, which is read back as $[+0,\ +0]$, the same set. GAOL 4 printed the sixteen hexadecimal digits of each double, which its parser did not read. [GAOL v5]

Example

```
1 interval::format(interval_format::hexa);
2 std::cout << textToInterval("0.1") << std::endl;
3 ➥ [0x1.9999999999999p-4, 0x1.999999999999ap-4]
```

Remark 9.2.1 *The formats* **`bounds`** *and* **`hexa`** *are the ones read back as an input (see Section 9.1.1, page 184), whatever the locale of the stream. The formats* **`width`** *and* **`center`** *are for the eye: the midpoint is written with the digits of the precision, so that the text* `c (+/- w)` *does not necessarily contain the interval, where the text of the formats* **`bounds`**, **`agreeing`** *and* **`hexa`** *does. A radius that took the rounding of the midpoint into account would show the interval* `0.1`*, which is one number, as* `0.1 (+/- 5.6e-18)`*, and would not show that an interval is narrower than the digits of its midpoint, which is what the format is for.*

The format is chosen through the following static methods.

`void interval::format(interval_format::format_t f)` *static*
`interval_format::format_t interval::format(void)` *static*

The first form sets the format used by the subsequent printing of intervals. The second form returns the format in use, a value of type `interval_format::format_t` (see below, and Section 9.2.4 for an example of use). The format is global to the program, not a property of a stream: it is shared by all the threads, and setting it while another thread writes an interval is a data race, one of the common errors listed in Chapter 3 (see Section 3.10, page 77). `exact_string()` gives the hexadecimal text without changing it.

interval_format *struct*

Structure type used to choose the output format of the intervals. It has five possible values of type **interval_format::format_t**:

- **interval_format::agreeing**.
- **interval_format::bounds**.
- **interval_format::width**.
- **interval_format::center**.
- **interval_format::hexa**.

Example 9.2.2 *The program below prints the same interval,* `interval::pi()`*, in the five formats. What each statement prints is given in the comment under it.*

Example

```
interval I(interval::pi());

interval::format(interval_format::agreeing);
cout << I << "\n";
// Prints 3.14159265358979~[3, 4]
// The ~[] part is dropped if the bounds agree on all digits

interval::format(interval_format::bounds);
cout << I << "\n";
// Prints [3.141592653589793, 3.141592653589794]

interval::format(interval_format::width);
cout << I << "\n";
// Prints 3.141592653589793 (+/- 4.440892098500627e-16)

interval::format(interval_format::center);
cout << I << "\n";
// Prints 3.141592653589793

interval::format(interval_format::hexa);
cout << I << "\n";
// Prints [0x1.921fb54442d18p+1, 0x1.921fb54442d19p+1]
```

9.2.3 The number of digits

The number of digits written for each bound is set by the static method `precision()` of the class `interval`. The precision of the stream, which `std::setprecision` sets, is not used for intervals, and is left as it was: the numbers written after an interval keep it. The width of the stream (`std::setw`) and its adjustment apply to the whole interval. With `std::internal`, the fill goes between the sign of the midpoint and

its digits in the formats **width** and **center**, and between the sign and the shared digits in the format **agreeing**, as it does for a number, and before the [in the other formats, as std::right has it. [GAOL v5] GAOL 4 set the precision of the stream to that of the intervals and left it so, and std::setw padded the [of an interval alone.

`std::streamsize interval::`**`precision`**`(void)` *static*

Returns the number of digits used for printing the bounds of intervals, 16 by default. Section 9.2.4 gives an example.

`std::streamsize interval::`**`precision`**`(std::streamsize n)` *static*

Sets the number of digits used for printing the bounds to n, and returns the number of digits previously used. Section 9.2.4 gives an example.

9.2.4 Example

Example 9.2.3 *The program below sets the precision and the format, prints π, reads the format back, and prints π again with sixteen digits in the format* **width**. *Its output is shown under it.*

Example

```
#include <iostream>
#include <gaol/gaol.h>
using namespace gaol;

using std::cout;
using std::endl;

int main(void)
{
  interval::precision(4);
  interval::format(interval_format::bounds);
  cout << interval::pi() << endl;

  if (interval::format() != interval_format::bounds) {
    cout << interval::pi() << endl;
  } else {
    interval::precision(16);
    interval::format(interval_format::width);
    cout << interval::pi() << endl;
  }
  gaol::cleanup();
  return 0;
}
➥ [3.141, 3.142]
➥ 3.141592653589793 (+/- 4.440892098500627e-16)
```

The first call to `interval::format()` is unnecessary, since the default format is **`interval_format::bounds`**.

The constant `interval::pi()` is a predefined canonical interval containing π (see Section 5.4, page 125). Here, its bounds are two consecutive doubles, 2^{-51} apart: the midpoint written is one of them, and the radius is their distance, the smallest that has the interval within the midpoint plus or minus it. [GAOL v5] GAOL 4 wrote half of it.

Remark 9.2.4 *An interval written in decimal and read back gives an interval that encloses it, whatever the precision chosen, the bounds being rounded outward; the fewer the digits, the wider. The format* **`hexa`** *gives the same interval back.*

The format **`width`** is useful whenever the number of digits displayed is insufficient to tell whether the result is a single floating-point number or an interval too narrow for these digits to show: if an interval is not a single number, the radius displayed is different from zero, whatever its size and the number of digits, since the radius is rounded upward. A degenerate interval is displayed without a radius.

9.3 Discussion

This chapter described the two ways an interval crosses the border of a program. On the way in, GAOL v5 reads every string with one grammar, in which the literals, the numbers, the constants and the functions combine with the operators. Its numbers are read exactly, whatever their base, their number of digits, the locale and the C library, and a number that is not a double becomes the tightest interval of doubles containing it. Its reader is reentrant and computes the operations as they are read, so that several threads may read at once and the length of a string is limited by the memory alone. On the way out, the decimal bounds are rounded outward whatever the C library, so that a text read back encloses the interval written, and the format **`hexa`** gives the interval back exactly.

The formats are trade-offs. The decimal formats **`bounds`** and **`agreeing`** are readable and enclose the interval, at the price of a bound moved by one unit of its last digit when the text nearest to it lies on the wrong side; the format **`hexa`** is exact and meets the recovery requirement of the standard; the formats **`width`** and **`center`** are for the eye, and their text does not necessarily contain the interval. The format and the precision are global to the program, where the width and the adjustment belong to the stream. Reading a long number takes a time quadratic in its number of digits.

The next chapter turns to the namespace `gaol_ieee1788`, which offers the operations under the names of IEEE 1788-2015, the reading and the writing of intervals included, and lists where the standard and GAOL differ.

Chapter 10

The names of IEEE 1788-2015

IEEE 1788-2015 [9] names the operations of interval arithmetic, and GAOL names its operations its own way. The reverse function coshRev(c, x) of the standard is `acosh_rel(c, x)` in GAOL, mulRev(b, c, x) is `div_rel(c, b, x)`, with its arguments in another order, and roundTiesToEven(x) is `round_ties_to_even(x)`. A reader of the standard who comes to GAOL has to look for the translation of each name.

The namespace `gaol_ieee1788` spares this search. It gives each operation of the standard that GAOL provides the name and the argument order the standard gives it, so that a reader of the standard finds it without looking for its translation. It is the second of the two namespaces of GAOL v5 (see Section 3.4, page 69); the standard itself is presented in Chapter 1 (see Section 1.5, page 11).

In this chapter, we describe this namespace. The first section presents it and the way a program opens it. The second lists the operations whose meaning differs between the standard and GAOL, and which follow the standard in `gaol_ieee1788`. The third gives the table of the correspondence between the names of the standard and those of GAOL, then details mulRevToPair, which has no name in `gaol`, and the reverse functions of atan2. The fourth describes the reading of intervals with the names of the standard, and the fifth lists what GAOL does not provide of the standard. The names of the other interval libraries, next to those of GAOL and of the standard, are compared in Chapter 3 (see Section 3.11, page 83).

10.1 The namespace of the standard

The namespace `gaol_ieee1788` comes with `gaol/gaol.h`. It is not in `gaol`, and it holds the type `interval` as well: one directive lets a program use GAOL under the names of the standard. The output format is not in it: a program that opens only `gaol_ieee1788` sets it with `interval::format(gaol::interval_format::hexa)`. [GAOL v5]

Example 10.1.1 *The program below opens* `gaol_ieee1788` *and computes with the names of the standard:* `numsToInterval` *builds an interval from its two bounds,*

`mulRev` *is the reverse multiplication, which divides* `x` *by* $[2, 2]$, `sinPi` *and* `rootn` *are two functions of Table 10.5 of the standard,* `entire` *is the interval* $[-\infty, +\infty]$ *and* `strictLess` *the strict order of its Table 10.3. What it prints follows it.*

Example

```
#include <iostream>
#include <gaol/gaol.h>
using namespace gaol_ieee1788;

int main(void)
{
  interval x = numsToInterval(1, 2);
  interval y = mulRev(numsToInterval(2, 2), x);   // x / 2
  interval z = sinPi(x) + rootn(y, 3);
  bool b = strictLess(x, entire());
  std::cout << std::boolalpha
            << x << ' ' << y << ' ' << z << ' ' << b << std::endl;
  gaol::cleanup();
  return 0;
}
➥ [1, 2] [0.5, 1] [-0.2062994740159004, 1] false
```

A program opens `gaol` or `gaol_ieee1788`, not both (see Section 3.4, page 69). The functions that have the same name and the same meaning in both are the same functions; the ones whose meaning differs, `pow` and `textToInterval`, exist in each namespace, and a program that opened both would find each call ambiguous.

Remark 10.1.2 *A name of the program's own that is also a name of the standard, a constant* `inf` *for instance, makes every unqualified use of it ambiguous: the program qualifies both,* `::inf` *and* `gaol_ieee1788::inf(x)`*. The same holds for* `less(x, y)` *next to* `using namespace std;`*, where it is ambiguous with the class template* `std::less`*.*

10.2 Where the standard and GAOL differ

The functions of GAOL that have the name and the meaning the standard gives them (`sin`, `exp`, `sqrt`, `min`...) are the same functions in `gaol_ieee1788`. Where the standard and GAOL differ, the names of `gaol_ieee1788` follow the standard. The differences are the following ones.

- **The power.** `pow(x, y)` is the `pow` of Table 9.1, defined for $x > 0$, and for $x = 0$ when $y > 0$. The `pow` of `gaol` takes the integer power for an integer exponent, a negative base included (see Section 7.2.2, page 151). `pow(x, p)` with a number p is `pow(x, [p])`, an integer p included, `pow(x, 2)` as well, as the standard gives no other meaning to it: the power with an integer exponent is `pown(x, n)` in the standard, whose n is an integer rather than an interval. An integer exponent beyond the `int`s gives the power of CORE-MATH at the

bounds of the part of x in $[0, +\infty]$, as the `pow` of `gaol` does there; the `pow` of `gaol` takes the integer power on the negative part of x too. The results of the two powers are compared in Chapter 3 (see Section 3.5, page 70).

- **The bounds of the empty set.** `inf(x)` and `sup(x)` are $+\infty$ and $-\infty$ for the empty set (Table 10.2), where `left()` and `right()` are NaN. `inf` returns -0 for a left bound 0 (12.12.8).
- **Membership.** `isMember(m, x)` is `false` for an infinite m (10.6.3).
- **Writing.** `intervalToText(x)` writes the bounds of `x` rounded outward, `[l, r]`, or `[a]` for a degenerate interval whose value the digits write exactly, as `[4]`, or `[empty]`, with the digits of `interval::precision()` and a decimal point. This string is an interval literal of the standard (12.11), which `textToInterval` reads back as an interval containing `x` (13.3), whatever the output format that `interval::format()` sets and the locale of the program. `operator<<` writes the current format instead, which is no literal in the formats **`width`** and **`center`**, nor in the format **`agreeing`** where the bounds share digits. [GAOL v5]
- **Reading.** `textToInterval` and `exactToInterval` read the names of the functions of the standard, and return the empty set for a string that is no interval (12.1.3), where `gaol::textToInterval` reads the names of GAOL and throws (see Section 10.4, page 203). A null pointer is no string, and they throw `std::invalid_argument` for it too (see Section 5.1, page 117). [GAOL v5]

Example 10.2.1 *The program below shows these differences. The* `pow` *of the standard is empty on a negative base, where* `pown` *gives the integer power; an integer exponent beyond the* `int`*s gives the power of CORE-MATH at the bounds; the bounds of the empty set are the infinities; an infinity is a member of no interval; and a string that is no interval reads as the empty set.*

Example

```
cout << pow(interval(-4,-1), interval(2.0)) << ' '
     << pown(interval(-4,-1), 2) << ' '
     << pow(interval(2,3), interval(1e10)) << endl;
➥ [empty] [1, 16] [1.797693134862315e+308, inf]
cout << boolalpha << inf(empty()) << ' ' << sup(empty()) << ' '
     << isMember(GAOL_INFINITY, entire()) << ' '
     << textToInterval("[1, 2") << endl;
➥ inf -inf false [empty]
```

10.3 The operations

Table 10.1 gives, for each operation of the standard that GAOL provides, its name in `gaol_ieee1788` and the name GAOL gives it, grouped as the standard groups them, with the clause or the table of the standard that defines them. The last argument of the reverse functions is optional, and $[-\infty, +\infty]$ when it is left out (10.5.4).

TABLE 10.1 – The operations of IEEE 1788-2015 in `gaol_ieee1788`, and their names in GAOL

`gaol_ieee1788`	GAOL
Constants and constructors (10.5.2, 10.5.8)	
`empty()`, `entire()`	`interval::emptyset()`, `interval::universe()`
`numsToInterval(l, u)`	`interval(l, u)`
`textToInterval(s)`	`gaol::textToInterval(s)` with the names of the standard, the empty set for a wrong string
Forward elementary functions (Table 9.1 of the standard)	
`neg`, `add`, `sub`, `mul`, `div`	`-x`, `x + y`, `x - y`, `x * y`, `x / y`
`recip(x)`	`inverse(x)`
`sqr`, `sqrt`, `fma`	the same
`pown(x, p)`	`gaol::pow(x, p)` for an integer p of any type
`pow(x, y)`, `pow(x, p)`	the `pow` of the standard
`exp`, `exp2`, `exp10`, `log`, `log2`, `log10`	the same
`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`	the same
`sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`	the same
`sign`, `ceil`, `floor`, `trunc`	the same
`roundTiesToEven`, `roundTiesToAway`	`round_ties_to_even`, `round_ties_to_away`
`abs`, `min`, `max`	the same
Recommended functions (Table 10.5 of the standard)	
`rootn(x, q)`	`nth_root(x, q)`
`expm1`, `exp2m1`, `exp10m1`	the same
`logp1`, `log2p1`, `log10p1`	`log1p`, `log2p1`, `log10p1`
`hypot`, `rSqrt`	`hypot`, `rsqrt`
`compoundm1`	the same
`sinPi`, `cosPi`, `tanPi`	`sinpi`, `cospi`, `tanpi`
`asinPi`, `acosPi`, `atanPi`, `atan2Pi`	`asinpi`, `acospi`, `atanpi`, `atan2pi`
Reverse functions (Table 10.1 of the standard)	
`sqrRev(c, x)`, `absRev(c, x)`	`sqrt_rel(c, x)`, `invabs_rel(c, x)`
`pownRev(c, x, p)`	`nth_root_rel(c, p, x)`, for every integer p, $p \leqslant 0$ included
`sinRev`, `cosRev`, `tanRev(c, x)`	`asin_rel`, `acos_rel`, `atan_rel(c, x)`
`coshRev(c, x)`	`acosh_rel(c, x)`
`sinhRev`, `tanhRev(c, x)` (not in the standard, named after `coshRev`)	`asinh_rel`, `atanh_rel(c, x)`
`mulRev(b, c, x)`	`div_rel(c, b, x)`

Table 10.1, continued

gaol_ieee1788	GAOL
`powRev1(b, c, x)`, `powRev2(a, c, x)`	`pow_rel(c, b, x)`, `pow_exponent_rel(c, a, x)` [GAOL v5]
`atan2Rev1(x, c, y)`, `atan2Rev2(y, c, x)`	`atan2_rel_y(c, x, y)`, `atan2_rel_x(c, y, x)` [GAOL v5]
`mulRevToPair(b, c)` (10.5.5)	the parts of c/b on either side of 0, each a `div_rel` with `interval::universe()` for its last argument, in a `std::pair`
Cancellative operations and set operations (10.5.6, 10.5.7)	
`cancelMinus`, `cancelPlus`	`cancel_minus`, `cancel_plus`
`intersection(x, y)`, `convexHull(x, y)`	`x & y`, `x | y`
Numeric functions (Table 10.2 of the standard)	
`inf(x)`, `sup(x)`	`x.left()`, `x.right()`, but for the empty set and a zero bound (-0 for `inf`, $+0$ for `sup`)
`mid`, `wid`, `rad`, `mag`, `mig(x)`	`x.midpoint()`, `width()`, `rad()`, `mag()`, `mig()`
`midRad(x, m, r)`	`x.mid_rad(m, r)`
Boolean functions (10.5.10, Tables 10.3 and 10.4 of the standard; 10.6.3)	
`isEmpty(x)`, `isEntire(x)`	`x.is_empty()`, `x.is_entire()`
`equal(a, b)`, `subset(a, b)`	`a.set_eq(b)`, `a.set_leq(b)`
`interior(a, b)`, `disjoint(a, b)`	`a.set_le(b)`, `a.set_disjoint(b)`
`less(a, b)`, `strictLess(a, b)`	`a.less(b)`, `a.strictly_less(b)`
`precedes(a, b)`, `strictPrecedes(a, b)`	`a.certainly_leq(b)`, `a.certainly_le(b)`
`isCommonInterval(x)`, `isSingleton(x)`	`x.is_common_interval()`, `x.is_a_double()`
`isMember(m, x)`	`x.set_contains(m)`, false for an infinite m
`overlap(a, b)` (10.6.4)	`a.overlap(b)`, an `overlapping_state`
Input and output (Clause 13)	
`intervalToText(x)`	the bounds `[l, r]`, `[a]` or `[empty]`, with the current precision, whatever the format and locale
`intervalToExact(x)`	`exact_string(x)`
`exactToInterval(s)`	`textToInterval(s)` of `gaol_ieee1788`
Expressions	
`pown(e, n)`, `pow(e1, e2)`	the expressions of `pown` and of the `pow` of the standard, declared by `gaol/parser/gaol_expression.h`

Two rows of the table call for a word. `overlap(a, b)` is the interval overlapping function of the standard: it returns an `overlapping_state`, one of the sixteen states of Table 10.7 under the names of the table, and the type and the member `a.overlap(b)` are in `gaol` too, which has no other name for it (Chapter 6). [GAOL v5]

The expressions `pown(e, n)` and `pow(e1, e2)` are computed with the `pown` and the `pow` of this namespace; `gaol/parser/gaol_expression.h` declares them with the other expressions, and `gaol/gaol.h` does not include it, as in GAOL 4. The last two parts of this section detail `mulRevToPair` and the reverse functions of `atan2`.

The two-output division. `mulRevToPair(b, c)` returns the set $\{x \mid \exists \beta \in b,\ \beta x \in c\}$ in two intervals, the pair (u, v) of the standard: u before v when the set has two parts, which happens when c does not contain 0 and 0 is strictly between the bounds of b, $(u, \varnothing)$ when it is one interval, and $(\varnothing, \varnothing)$ when it is empty. The convex hull of u and v is `mulRev(b, c)`. Where b is unbounded on both sides, the two parts share 0. [GAOL v5]

Example 10.3.1 *The program below divides* $[1, 2]$ *by* $[-1, 1]$*, whose interior holds 0, and gets two parts; then by* $[2, 4]$*, and gets one part and the empty set.*

Example

```
std::pair<interval, interval> p = mulRevToPair(numsToInterval(-1, 1),
                                               numsToInterval(1, 2));
std::pair<interval, interval> q = mulRevToPair(numsToInterval(2, 4),
                                               numsToInterval(1, 2));
cout << p.first << ' ' << p.second << ' '
     << q.first << ' ' << q.second << endl;
➥ [-inf, -1] [1, inf] [0.25, 1] [empty]
```

The reverse functions of atan2. `atan2Rev1(x, c, y)` is the hull of the ordinates of the points of the box $x \times y$ whose angle $\operatorname{atan2}(y, x)$ is in c, and `atan2Rev2(y, c, x)` the hull of their abscissas; the point $(0, 0)$, which has no angle, is left out. In `gaol`, they are `atan2_rel_y(c, x, y)` and `atan2_rel_x(c, y, x)` (see Section 7.3, page 160). [GAOL v5] As in `mulRev` and `div_rel`, the interval contracted comes last, and the other argument of `atan2` before it, so that the constraint $\theta = \operatorname{atan2}(y, x)$ contracts its three variables with the three lines below.

Example

```
theta &= atan2(y, x);
y = atan2Rev1(x, theta, y);
x = atan2Rev2(y, theta, x);
```

The angles are those of `atan2`, in $]-\pi, \pi]$. The half-line $y = 0$, $x < 0$ has the angle π, not $-\pi$, and a c holding $-\pi$ but not π leaves it out: `atan2Rev2([0], [-4, -3], [-2, 2])` is empty. The angles of c beyond π are no angle of any point: `atan2Rev1([-1], [3, 4], [-1, 1])` is $[0, -\tan 3]$, about $[0, 0.1425]$, the points of angles 3 to π only. A program whose angles go around past π gives the two sides of the jump apart, $[3, \pi]$ and $[-\pi, 4 - 2\pi]$ for $[3, 4]$, and takes the hull of the two results.

10.4 Reading with the names of the standard

`textToInterval(s)` reads a string as `gaol::textToInterval()` does (see Section 9.1.1, page 184), with the names of the functions of IEEE 1788-2015: those of Tables 9.1 and 10.5 that GAOL provides, whatever the case of their letters. [GAOL v5] These names are the following ones.

> `neg`, `add`, `sub`, `mul`, `div`, `recip`, `sqr`, `sqrt`, `fma`, `pown`, `pow`, `exp`, `exp2`, `exp10`, `log`, `log2`, `log10`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`, `sign`, `ceil`, `floor`, `trunc`, `roundTiesToEven`, `roundTiesToAway`, `abs`, `min`, `max`, `rootn`, `expm1`, `exp2m1`, `exp10m1`, `logp1`, `log2p1`, `log10p1`, `hypot`, `compoundm1`, `rSqrt`, `sinPi`, `cosPi`, `tanPi`, `asinPi`, `acosPi`, `atanPi`, `atan2Pi`

`pow` is the pow of Table 9.1, and the second argument of `pown` and `rootn` is an integer. The names of GAOL alone (`nth_root`, `cbrt`, `inverse`, `integer`, `log1p`, `round_ties_to_even`...) are no functions there. They give the empty set, as any string that is no interval: a wrong call (`"pown([2,5],2.5)"`, `"sin(1,2)"`) and a string holding a NUL character included, whether GAOL was configured with exceptions or not. [GAOL v5] Where memory runs out, `textToInterval` throws `std::bad_alloc`.

As `pow`, the function `textToInterval` is in the namespace `gaol` too, where it reads the names of GAOL and throws `std::invalid_argument` for a wrong string, and `std::domain_error` for an order of `nth_root` that is no integer: a program calls the one of the namespace it opens.

Example 10.4.1 *The program below reads three strings with the names of the standard: a power of the standard, a power of a negative base, which the* pow *of the standard leaves empty, and a name of GAOL alone, which is no function there.*

Example

```
1 cout << textToInterval("pown([2,5],5)") << ' '
2      << textToInterval("pow([-4,-1],2)") << ' '
3      << textToInterval("nth_root(8,3)") << endl;
4 ➥ [32, 3125] [empty] [empty]
```

With `using namespace gaol;`, `textToInterval("pow([-4,-1],2)")` *is* $[1, 16]$, *and* `textToInterval("pown([2,5],5)")` *throws.*

10.5 What is not provided

GAOL has no decorations: only bare intervals are provided (Clause 11). Nor does it provide the slope functions (Table 10.6), the reduction operations sum, dot, sumSquare and sumAbs (12.12.12), the exact reduction operations of 12.13.5, the conversion specifier *cs* of intervalToText (13.3), or the exceptions UndefinedOperation and

PossiblyUndefinedOperation that textToInterval and exactToInterval signal (12.12.7, 13.4): for a string that is no interval they return the empty set silently. Where they cannot tell whether $l \leq u$ in a literal `[l, u]`, case (b) of 12.12.7 (the bounds of see Section 9.1, page 183), they return the interval from the lower bound of l to the upper bound of u, without PossiblyUndefinedOperation. That interval holds $[l, u]$ where $l \leq u$; where l is above u it is a false positive that the standard permits, and it can miss l or u when a bound is computed: `"[1/3, 0x1.5555555555555p-2]"` is the point `0x1.5555555555555p-2`, below 1/3. The end of `gaol/gaol_ieee1788.h` keeps this list.

Remark 10.5.1 *In the other direction, a function of GAOL that the standard does not name is not in* `gaol_ieee1788` *either, but for* `sinhRev` *and* `tanhRev`*, which are named after* coshRev *(Table 10.1), and for the operators of intervals, the relational division* `%` *among them, which the namespace names as* `gaol_core` *does.* `erf` *and* `erfc`*, the error function and its complement, are two functions of GAOL that the standard does not name.* [GAOL v5] *A call* `erf(x)` *on an interval finds them all the same in* `gaol_core`*, by argument-dependent lookup, and* `erf(0.5)` *remains the* `erf` *of C.*

10.6 Discussion

This chapter has presented the namespace `gaol_ieee1788`, which lets a program use GAOL under the names and the argument orders of IEEE 1788-2015 with a single directive. The functions that the standard and GAOL name and define alike are shared between the two namespaces. Where they differ, the power, the bounds of the empty set, the membership of an infinity, the writing and the reading of intervals, the namespace follows the standard, and Table 10.1 gives the translation of every other name. Two operations have their details here: `mulRevToPair`, which `gaol` does not name, and the reverse functions of `atan2`, whose arguments `gaol_ieee1788` orders as mulRev does and `gaol` as `div_rel` does.

The choice of two namespaces rather than one keeps the programs written for GAOL 4 and, but for the results that change (Chapter 15), their results, while a program written for the standard, or compared with another library of the standard, opens `gaol_ieee1788`. Its price is stated in Section 10.1, page 197: a program opens one of the two namespaces, never both, and qualifies the names of its own that are also names of the standard. What GAOL does not provide of the standard, the decorations, the slope functions, the reductions, exact or not, the conversion specifier of intervalToText and the exceptions of the reading, is listed rather than approximated (see Section 10.5, page 203), and the header keeps the list.

Whatever the name a program calls an operation by, the bounds it gets are the same. The next chapter states how tight they are: the conditions GAOL guarantees on each bound, and the tightness of each family of functions.

Chapter 11

Accuracy of the operations

The containment property (see Section 1.3, page 9) says that the result of an operation encloses its exact value; it says nothing of how closely. IEEE 1788-2015 [9] requires an implementation to document the *tightness* of each of its interval operations, dividing the possible inputs into ranges and stating the tightness achieved in each (12.10.3). This chapter does so for `interval`, the inf-sup type of GAOL on doubles, from the algorithm of each operation. The tests of `tests` check each statement on random and special arguments (see Section 14.2, page 274), and we give, with each operation, the largest distance they found, or the distance they accept where the text says no more. [GAOL v5]

IEEE 1788-2015 defines three accuracy modes (12.10.1), which Chapter 1 presents (see Section 1.8, page 20). For an operation f, its exact value f^* and an input box x:

- the result is *tightest* when $f(x)$ is the hull, in doubles, of $f^*(x)$: each bound is the nearest double on its side of the exact bound;
- it is *accurate* when $f(x)$ encloses $f^*(x)$ and is within

  $$\text{nextOut}(f_{\text{tightest}}(\text{nextOut}(x))),$$

  nextOut moving each bound one double outward: roughly, $f(x)$ is no wider than the tightest result for an input one double wider, plus one double on each side;
- it is *valid* when $f(x)$ encloses $f^*(x)$.

The standard requires the basic operations (neg, add, sub, mul, div, recip, sqr, sqrt, fma), the integer functions and `abs`, `min` and `max` to be tightest; it requires the other elementary functions, the reverse functions and the recommended functions to be valid, and recommends them to be accurate (12.10.2). Below, "within k doubles" means that each bound is at most k doubles beyond the tightest one.

GAOL bounds its elementary functions with CORE-MATH [13], which is correctly rounded in the rounding direction in effect (see Section 1.7, page 15). Computing in the upward rounding it keeps, GAOL takes the value at a bound as the upper bound, with nothing to add, and the double below it as the lower one. The upper bound is the tightest one, and so is the lower bound unless the exact value is a double, which the operations give exactly where it happens: $\log(1) = 0$ and $\sin(0) = 0$ are exact, and $\arcsin(1)$ is the tightest enclosure of $\pi/2$. Where an algorithm evaluates the function at the bounds of the interval, the bounds of the result are therefore the tightest ones.

The first section of this chapter states the conditions under which these statements hold. The second gives the tightness of each operation in a table, then, family by family, the algorithm of each operation and the largest distance from the tightest bounds that the tests found. The third section explains why the bounds are the same on every machine, and where the sign of a zero bound differs between the kernels.

11.1 The conditions

The statements of this chapter hold for GAOL as the three builds of Chapter 2 make it: with the CORE-MATH of `3rd/math-core`, compiled as the builds compile it, and with the code that includes the headers of GAOL compiled with the flags of interval arithmetic (see Section 3.3, page 64). They hold for the SSE2 intervals, the NEON intervals [GAOL v5] and the FPU intervals alike (see Section 12.1, page 228), and for the AVX-512 path (see Section 4.4, page 94), whose bounds are those of the SSE2 path, but for the sign of a zero bound of `x * d` (see Section 11.3, page 224).

They hold whatever the rounding direction the calling code left (`tests/rounding_direction.cpp`), and with the floating-point exceptions masked, as a program starts: an operation that raises an enabled exception stops the program rather than give its bounds (see Section 4.9, page 107). They hold on every architecture and with every compiler alike, CORE-MATH giving the same bits everywhere: there is no other mathematical library to build GAOL with (see Section 1.7, page 15, and Chapter 15 for what differs from GAOL 4). The intervals of floats, `gaol::intervalf` and `gaol::interval2f`, unfinished, which only a developer of GAOL compiles (`gaol/gaol_config.h`), are not covered.

11.2 The tightness of each operation

Table 11.1 gives the tightness of each operation of `interval`, in the families of IEEE 1788-2015, from its algorithm; the subsections that follow give the algorithm of each operation and what the tests found. The tables numbered 9.1, 10.1, 10.3, 10.4, 10.5 and 10.7 that this chapter names are those of IEEE 1788-2015.

TABLE 11.1 – The tightness of each operation of `interval`

Operations	Tightness
Basic operations (Table 9.1): tightest required	
`-x`, `+`, `-`, `*`, `/`, `inverse`, `sqr`	tightest
`sqrt`	tightest, whatever the rounding of the square root of the C library
`fma`	tightest
Integer and absmax functions: tightest required	
`floor`, `ceil`, `integer`, `sign`, `trunc`, `round_ties_to_even`, `round_ties_to_away`, `abs`, `min`, `max`	tightest, and exact, in every rounding direction
Powers, exponentials and logarithms: valid required, accurate recommended	
`pow(x, n)`, integer n (`pown`)	the tightest bounds, for a base of either sign, or one double beyond where the power is within $2K$ of a double, relatively, K being the bound of the relative error of the power ($K = 2^{-98}$ for $n = 3$, about 2^{-62} at most, at $n = -2^{31}$; no such case for $n = 2$, which is `sqr(x)`); this holds below 2^{-968} and beyond the largest double too; the bounds are exact where the power is a double; the same bounds with the SSE2, the NEON and the FPU intervals. Beyond the `int`s: tightest where n is a double, within three doubles otherwise
`pow(x, y)` (`pow`)	tightest, at the corners of the box and at their limits where the base is 0 or $+\infty$ or the exponent infinite; for a degenerate integer exponent within the `int`s, the integer power of the row above, one double beyond the tightest where it is within twice the error of its products (`pow([0x1.fffffffffffffp+1], [7])` has its lower bound one double below the tightest)
`exp`, `exp2`, `exp10`, `log`, `log2`, `log10`	tightest; exact where the value is a double (exp(0), log(1), 2^n, 10^n for $0 \leqslant n \leqslant 22\ldots$)
Trigonometric and hyperbolic functions: valid required, accurate recommended	
`cos`, `sin`	tightest where the function is evaluated at the bounds, at every magnitude; -1 and 1 exactly at the extrema; $[-1, 1]$ from the width $2\underline{\pi}$ on, $\underline{\pi}$ being the double below π
`tan`	tightest between two poles, at every magnitude; $[-\infty, +\infty]$ exactly when a pole is within x, but for the widths above $\underline{\pi}$ and below π
`asin`, `acos`, `atan`, `atan2`	tightest; exactly 0, or the tightest enclosures of $\pm\pi/4$, $\pm\pi/2$ and π, where the value is one of them

Table 11.1, continued

Operations	Tightness
`sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`	tightest
Recommended functions (Table 10.5)	
`nth_root(x, n)`, $n > 0$	$n = 1$, 2, 3: tightest. Otherwise the tightest bounds or up to 2 doubles beyond, proved with integer powers; beyond the `unsigned int`s, CORE-MATH's `pow` at the corners, within one double
`nth_root(x, q)`, $q < 0$	valid: the root above, then the tightest division, up to 4 doubles beyond the tightest bounds
`expm1`, `exp2m1`, `exp10m1`, `log1p`, `log2p1`, `log10p1`, `hypot`, `rsqrt`	tightest
`compoundm1`	tightest but in a case out of 10^{17} or so, where a bound is a double beyond [GAOL v5]
`sinpi`, `cospi`, `tanpi`, `asinpi`, `acospi`, `atanpi`, `atan2pi`	tightest, at every magnitude
The error functions, which IEEE 1788-2015 does not name	
`erf`, `erfc`	tightest; exact at 0 and at the infinities [GAOL v5]
Reverse functions (Table 10.1): valid required, accurate recommended	
`sqrt_rel`, `acosh_rel`, `asinh_rel`, `atanh_rel`	accurate
`nth_root_rel(c, p, x)`, $p > 0$	the bounds of `nth_root()`: up to 2 doubles beyond the tightest
`nth_root_rel(c, p, x)`, $p < 0$	valid: the root of each part of c and then its inverse, up to four doubles beyond the tightest [GAOL v5]
`invabs_rel`	tightest
`max_rel`, `min_rel`, `sign_rel`, `floor_rel` (not in Table 10.1)	tightest
`asin_rel`, `acos_rel`, `atan_rel`	accurate, or one double beyond; a bound of x beyond 2^{52} is kept
`div_rel`, `%`	accurate; tightest where the bounds of the quotients are doubles
`pow_rel`, `pow_exponent_rel`	tightest, but for a bound of x next to the solutions whose power is within one double of c, which is kept: within the tightest result for x one double wider [GAOL v5]
`atan2_rel_y`, `atan2_rel_x`	accurate; within two doubles of the tightest bounds where these are not empty
`cancel_minus`, `cancel_plus`	tightest

Table 11.1, continued

Operations	Tightness
Set operations, constructors, numeric and boolean functions	
`&`, `|`, `interval(l, u)`	tightest (exact)
`textToInterval(s)`, `operator>>`	tightest for the literals of IEEE 1788-2015; valid for expressions, computed with the operations above
`midpoint()`, `width()`, `rad()`	as 12.12.8 requires
`mag()`, `mig()`, `smig()`	exact
`mid()`, `hausdorff()`	tightest
relations, `overlap()`	exact, as Tables 10.3, 10.4 and 10.7
`operator<<`	`hexa`: exact, read back as the same interval. Decimal formats: valid, and the tightest with the digits asked for where the C library rounds to nearest as it should. `width` and `center`: $[c - w, c + w]$ contains x, w being the smallest radius that does so. `exact_string()`: exact, read back bit for bit

Remark 11.2.1 *Before GAOL v5, the elementary functions but the hyperbolic ones were correctly rounded to nearest and moved one double outward, which is at most one double beyond the tightest bound but for the sine, computed as* $\cos(x - \pi/2)$*, and the real power, computed as* $\exp(y \log x)$*, which were wider: accurate or not, and not tightest. The hyperbolic functions were taken from the mathematical library of the system, whose values are sometimes further than one double from the exact ones, and millions of doubles away for the* `acosh` *of some of them.* [GAOL v5]

11.2.1 Basic operations

The basic operations of Table 9.1 of the standard must be tightest. Each is computed from the bounds, rounded upward once (see Section 12.1, page 228), and each is tightest; the tests find them so.

- **Negation.** `-x` exchanges the bounds: exact.
- **Addition and subtraction.** `x + y`, `x - y`, `+=`, `-=`, and their forms with a double, are the sums of the bounds rounded upward, the lower bound being stored negated: tightest.
- **Multiplication.** `x * y`, `*=`, and the forms with a double, are the products of the bounds chosen by their signs, rounded upward; a zero bound times an infinite one counts as 0: tightest.
- **Division.** `x / y`, `/=`, and the forms with a double, are the quotients of the bounds chosen by their signs, rounded upward; the hull of the quotients when y contains 0, and the empty set when y is [0, 0]: tightest.

- **Inverse.** `inverse(x)` and `x.inverse()` divide 1 by the bounds, rounded upward: tightest.
- **Square.** `sqr(x)` squares the bounds, rounded upward, with 0 as lower bound when x contains 0: tightest.
- **Square root.** `sqrt(x)` works on $x \cap [0, +\infty]$. The upper bound u is the square root rounded upward, from the square root of the C library checked with a product rounded the other way and moved one double when needed; the lower bound is u when $u \cdot u$ is exactly the bound of x, and the double below u otherwise: tightest, whatever the rounding of the square root of the C library.
- **Fused multiply-add.** In `fma(x, y, z)`, the product $x \cdot y$ is bilinear, so it ranges over the hull of the products of the bounds, and $x \cdot y + z$ ranges from the least product plus $\underline{z}$ to the greatest plus $\overline{z}$. Each corner is rounded once by `std::fma`, upward for the upper bound and downward for the lower one, computed as $-\mathrm{fma}(-a, b, -c)$. A bound 0 times an infinite one counts as 0. [GAOL v5] Tightest, over 200 000 boxes against exact rational arithmetic.

Remark 11.2.2 *The result of* `std::fma` *goes through* `rnd_keep()` *before it is negated. Without it, GCC folded* $-\mathrm{fma}(-a, b, -c)$ *into* $\mathrm{fma}(a, b, c)$*, a single instruction rounded upward, and the lower bound came above the exact value, one double above the tightest lower bound.*

11.2.2 Integer and absmax functions

The integer functions, `abs`, `min` and `max` must be tightest. None of them rounds: each returns a double that is an integer, or a bound of its argument, in every rounding direction.

- **Floor and ceiling.** `floor(x)` and `ceil(x)` apply to each bound: exact.
- **Integers of an interval.** `integer(x)`, which the standard does not name, is the hull of the integers in x, $[\lceil \underline{x} \rceil, \lfloor \overline{x} \rfloor]$: exact.
- **Sign, truncation and roundings.** `sign(x)`, `trunc(x)`, `round_ties_to_even(x)` and `round_ties_to_away(x)` take the value at each bound, each function being non-decreasing, as `ceil` and `floor` above, and $\varnothing$ for $\varnothing$, which the four test: GAOL holds the empty interval as two NaN bounds, which `trunc` and the two roundings send to themselves, where `sign` would give $[0, 0]$, a NaN comparing false both to 0 and above it. `trunc` and `round_ties_to_away` are `std::trunc` and `std::round`, which the C++ standard defines without reading the rounding direction, and `round_ties_to_even` reads the bits (`gaol/math/gaol_roundeven.h`), where `std::nearbyint` and `std::rint` would round in the direction in effect, upward in GAOL. [GAOL v5] Tightest and

exact; the tests find the same result in the four rounding directions over 22 407 doubles: the halfway values, the whole numbers and the doubles on either side of them.

- **Absolute value.** `abs(x)` takes the magnitudes of the bounds, and 0 as lower bound when x contains 0: exact.
- **Minimum and maximum.** `min(x, y)` and `max(x, y)` apply to the bounds: exact.

11.2.3 Powers, exponentials and logarithms

These functions must be valid, and the standard recommends them to be accurate. The exponentials and the logarithms are tightest, being CORE-MATH's functions at the bounds; the integer power is tightest but one double beyond in rare cases, and the real power is tightest at the corners of its box.

The integer power. `pow(x, n)` for an integer n of any type, and `pow(x, y)` and `pow(x, d)` for a degenerate integer y or an integer double d, compute `pown` (`gaol_pown()`, `gaol_uipow()`, `gaol_pown_ll()`), as does `gaol_ieee1788::pown(x, n)` (see Section 7.2.2, page 151). Within the `int`s, $n = 2$ is `sqr(x)`, the squares of the bounds rounded outward. [GAOL v5] For any other n, the powers of the two corners of x where x^n is the least and the greatest are computed at once, in the two lanes of an SSE2 register with the FMA instructions or of a NEON register, or of a pair of doubles otherwise (the FPU intervals, and SSE2 without FMA), with the same operations in the same order. [GAOL v5] The power is a binary exponentiation from exact products: each product is rounded, and its rest, which `fma` gives exactly, is carried along, so that the power is $h + l$ within $K\,|h|$ of it, with $K = (8nL + 16) \cdot 2^{-104}$ for an n of L bits, a bound proved beforehand (`gaol/core/gaol_interval_core.cpp`). Each bound is then rounded once, $h+l+K|h|$ upward and $h+l-K|h|$ downward, and K is left out where every rest is 0, x^n being a double. The power x^{-n} is $1/(h + l)$: the inverse of h rounded, corrected with the rest of $1 - q(h + l)$ to the second order and rounded once the same way; x^{-1} is the inverse. A corner whose powers leave $[2^{-968}, 2^{960}]$ is computed alone, scaled to $[1, 2)$, its power kept below 2^{501} by powers of two, then scaled back with one rounding in the direction of the bound, beyond the largest double and among the subnormals too. The bounds 0 and $\pm\infty$ give the limits of x^n: $[\mathrm{mag}(x)^n, +\infty]$ where x holds 0 for an even $n < 0$, $[-\infty, +\infty]$ where it holds 0 inside for an odd one, and $\varnothing$ for $[0, 0]$.

Beyond the `int`s, CORE-MATH's `pow` is called at the corners, as for the real power below, the sign coming from the parity of n; for an n of more than 53 bits that is no double, $x^h \cdot x^l$ is rounded outward, with $n = h+l$, h a double and $|l| < 2^{11}$; beyond 64 bits, n is taken as $\pm(2^{63} - 1)$ or $\pm(2^{63} - 2)$, of its sign and its parity, every power of a double but 0, ±1 and $\pm\infty$ being beyond the doubles there. GAOL 4 gave $[-\infty, +\infty]$ beyond the `unsigned int`s, and beyond the `int`s for `pow(x, y)`; within

them, it rounded $1/x^n$ twice, and rounded the products outward, as it computed every power, where a bound was 0 or infinite or its power beyond the doubles. [GAOL v5]

The result is tightest, or one double beyond where x^n is within $2K$ of a double, relatively: never for $n = 2$, which is `sqr(x)`; K is 2^{-98} at $n = 3$, 2^{-88} at $n = 1000$, 2^{-65} at $n = 2^{31} - 1$ and about 2^{-62} at $n = -2^{31}$, where one bound in about 400 is one double beyond. It is exact where x^n is a double. Beyond the `int`s it is tightest where n is a double, and within three doubles otherwise.

The tests find the tightest bounds at 5 000 doubles drawn at random, with exponents from -30 to 30 and over all the doubles, subnormal results and overflows included, for n from 2 to 9, 16, 31 to 33 and -1 to -4, -7, -16 and -33, and over 5 000 intervals drawn so, for n from 2 to 5, 12, 31 and -1 to -3, -6, -17, x holding 0 included, against the exact powers; the tightest over 4 000 intervals near the ends of the doubles (a bound 0 or infinite, powers below 2^{-1000} or above 2^{1100}); the exact value at the 8 679 powers that are doubles; the tightest at 13 powers for n from $2^{24}+1$ to $2^{32} - 1$, and at 6 corners computed alone, against mpmath. Beyond the `int`s, they find the tightest at the powers of the tests `elementary` and `ieee1788` there (odd and even exponents, negative bases, exponents of 64 and 128 bits), the tightest at 5 powers beyond 2^{53} where n is a double, and within three doubles at 4 where it is none, of intervals and of expressions, against mpmath. They check that x^2 is `sqr(x)` over 400 intervals near 1, 3, 0.75 and 2^{-500}, and bit for bit, the sign of a zero included, in each of its forms (`pow(x, 2)`, `2u`, `2.0`, `[2]`, `gaol_ieee1788::pown(x, 2)` and `pow(x, [2])`), over the special bounds and 90 000 random intervals, in each direction of the calling code and with each flush mode; that $(-c)^n$ is $\pm c^n$, and that the bound from a corner is the same whether the other corner is computed with it or alone, at 1 088 points near 1, 1.5 and 3 and at 12 736 more within 16 doubles of 1. ITF1788, the test suite of Chapter 12, gets the tightest in its 163 cases of `pown`.

The real power. `pow(x, y)` and `pow(x, d)` for a non-integer y (`gaol_pow_hybrid()`, `gaol_pow_real()`), and `gaol_ieee1788::pow(x, y)` for any y, compute `pow` on $x \cap [0, +\infty]$: CORE-MATH's `pow` at the corners of the box $x \times y$ where x^y is the least and the greatest, which the places of the bounds about 1 and 0 give, correctly rounded upward. The upper bound is the value at the upper corner, and the lower bound is the double below the value at the lower corner, unless x^y is a double there. It is one for $x = 2^p$ and an integer $p\,y$ of $[-1074, 1023]$, and for $x = c^{2^k} \cdot 2^{f \cdot 2^k}$ ($c \geqslant 3$ odd) and $y = a/2^k$ ($a > 0$, odd if $k > 0$) where $c^a < 2^{53}$ and $c^a \cdot 2^{f \cdot a}$ is within the doubles, y being then a multiple of $1/32$ in $(0, 33]$; the lower bound is then x^y itself, which a test on x and y proves with integers and exact operations, leaving at once for nearly every y, where calling CORE-MATH downward too would double the cost of the power. [GAOL v5] The result is 1 where a corner has the base 1 or the exponent 0. At a corner where the base is 0 or $+\infty$ or the exponent infinite, it is the limit of x^y there, 0 or $+\infty$, which is the bound itself; GAOL 4 computed $\exp(y \log x)$ there, which gave bounds one to hundreds of doubles beyond. [GAOL v5] 0^y is 0 for $y > 0$, and $\varnothing$ for $y \leqslant 0$. An integer y beyond the `int`s,

in `gaol_ieee1788::pow`, takes the corners at the bounds of x, x^y being monotone in x, where the `pow` of `gaol` takes `pown`; GAOL 4 gave $[-\infty, +\infty]$. [GAOL v5]

The result is tightest, at the corners and at their limits, those at the bounds of x for an integer y beyond the `int`s included. For a degenerate integer y within the `int`s, it is the integer power above, one double beyond the tightest where x^y is within $2K$ of a double.

Example 11.2.3 `pow([4], 0.5)` *is* $[2, 2]$ *and* `pow([4, +oo], 0.5)` *is* $[2, +\infty]$. `gaol_ieee1788::pow([0x1.fffffffffffffp+1], [7])` *has its lower bound,* `0x1.ffffffffffff8p+13`*, one double below the tightest,* `0x1.ffffffffffff9p+13`*: the integer power is within* $2K$ *of a double there.*

The tests find the bounds within 1 double at points and over 385 boxes; the tightest, or one double beyond for one integer power, on the 117 boxes of the test `ieee1788`, the limits at 0 and $+\infty$ included, against mpmath; the tightest on the 1 347 cases of `pow` of ITF1788; the tightest at 5 integer exponents beyond the `int`s; and the tightest, both bounds, at the corners of 84 070 boxes of a base and an exponent (powers of two, perfect powers, midpoints, at the edges of the doubles too, and their neighbours), against x^y computed exactly at the 8 981 where it is rational, 3 211 of them with a power that is a double, and compared exactly at the 12 540 where it is the 2^k-th root of a rational, $k \leqslant 6$.

Exponentials and logarithms. `exp`, `exp2`, `exp10`, `log`, `log2` and `log10` are CORE-MATH's functions at the bounds, correctly rounded upward: the value at the right bound, and the double below the value at the left bound, unless that value is a double. The logarithms work on $x \cap [0, +\infty]$, give $\varnothing$ when x holds no positive number, and $\log(0) = -\infty$; the exponentials have a lower bound at least 0. The values that are doubles are given exactly: $\exp(0) = 1$ and $\log(1) = 0$; for `exp2`, `exp10`, `log2` and `log10`, [GAOL v5] 2^x for an integer x of $[-1074, 1023]$ and 10^x for an integer x of $[0, 22]$ (10^{23} is no double, and no negative power of ten is), $\log_2 x$ for a power of two and $\log_{10} x$ for a power of ten of $[10^0, 10^{22}]$. All six are tightest. The tests find `exp` tightest at about 9 000 doubles of every magnitude, `log` tightest at the 75 doubles of the tests and the million intervals of the comparison with the other libraries (see Section 12.5, page 237), and the other four tightest over 6 483 doubles each, and at the values that are doubles.

11.2.4 Trigonometric and hyperbolic functions

These functions must be valid, and the standard recommends them to be accurate. Every one of them is CORE-MATH's function at the bounds of the pieces where it is monotonic, so each is tightest where the function is evaluated at the bounds; the periodic functions ask first on which piece the interval lies. Every elementary function of this section and of the one before is that of CORE-MATH, whose sources are in `3rd/math-core` (see Section 1.7, page 15).

Cosine and sine. For `cos(x)`, the bounds divided by an enclosure of π, rounded outward, tell whether x lies on one piece where the cosine is monotonic; rounded inward, they tell whether an extremum is within x for sure. Where they cannot tell, a bound being within about $|x| \cdot 2^{-52}$ of a multiple of π, or beyond 2^{52}, the signs of CORE-MATH's `sin` at the bounds do, exactly, x being narrower than 2π (and the sign at the middle of x from π on): signs that differ show one extremum, -1 or 1 according to their order. Then CORE-MATH's `cos` is taken at the bounds, correctly rounded upward (the double below the value at the bound where the cosine is the least), within $[-1, 1]$, and $\cos(0) = 1$ exactly. `sin(x)` proceeds as `cos`, the bounds divided by an enclosure of π minus $1/2$, the signs of CORE-MATH's `cos` where the quotients cannot tell, and $\sin(0) = 0$ exactly.

Both are tightest where the function is evaluated at the bounds, at every magnitude, and give -1 and 1 exactly at the extrema. They give $[-1, 1]$ from the width $2\underline{\pi}$ on, $\underline{\pi}$ being the double below π, and $[-1, 1]$ is then the tightest result: an interval that wide misses an arc shorter than $2.5 \cdot 10^{-16}$, around an extremum, where the sine and the cosine stay within 10^{-31} of ± 1. That the test of a width below $\underline{\pi}$ gives the tightest bounds is measured, not proved: the tests find the tightest bounds of `sin` and `cos` over 29 400 intervals of width about π whose bounds are next to extrema of the sine. They find them tightest at about 9 000 doubles of every magnitude, and over intervals next to the extrema and of width about π and 2π.

Tangent. `tan(x)` is $[-\infty, +\infty]$ when x is wider than $\underline{\pi}$ or holds a pole: $(x + [\pi/2])/[\pi]$ at the bounds, rounded outward, tells that no pole is within x when the integer parts agree, and rounded inward that one is for sure; where they cannot tell, the signs of CORE-MATH's `cos` at the bounds do, exactly, a pole being within x when they differ. Otherwise the result is CORE-MATH's `tan` at the bounds, correctly rounded upward, and $\tan(0) = 0$ exactly. It is tightest between two poles, at every magnitude, and $[-\infty, +\infty]$ exactly when a pole is within x, but for the widths above $\underline{\pi}$ and below π, a theoretical case. The tests find it tightest at about 9 000 doubles of every magnitude, and over intervals next to the poles, the interval from `-M_PI_2` to `M_PI_2` and the intervals of width $\underline{\pi}$ or next to it between two poles included.

Inverse trigonometric functions. `asin(x)` and `acos(x)` are CORE-MATH's functions at the bounds of $x \cap [-1, 1]$, correctly rounded upward; at -1, 0 and 1 they give 0 exactly or the tightest bounds of $\pm\pi/2$ and π. `atan(x)` is CORE-MATH's `atan` at the bounds; at 0, ± 1 and $\pm\infty$ it gives 0 exactly or the tightest bounds of $\pm\pi/4$ and $\pm\pi/2$. `atan2(y, x)` is CORE-MATH's `atan2` at the corners of the box $y \times x$ where the angle is the least and the greatest, which the signs of the bounds give, correctly rounded upward; it gives 0, $\pm\pi/4$, $\pm\pi/2$ and π where a corner is on an axis, on a diagonal of the right half-plane or at an infinity, $[-\pi, \pi]$ when the box has points on the half-line $y = 0$, $x < 0$, where the angle is π, and points below it, and $\varnothing$ for the box $\{(0, 0)\}$. The four are tightest, and exactly 0 or

the tightest enclosures of the multiples of $\pi/4$ named where a bound is one of them. The tests check them within 1 double, and tightest at the special points.

Hyperbolic functions. `sinh(x)`, `tanh(x)` and `asinh(x)` are CORE-MATH's functions at the bounds, correctly rounded upward, `tanh` within $[-1, 1]$; they give 0 at 0 exactly, $\sinh(x)$ beyond the largest double for $|x| \geqslant 711$, and $\tanh(x)$ beyond $\pm(1 - 2^{-53})$ for $|x| \geqslant 20$, where $1 - |\tanh(x)| < 2^{-54}$. `cosh(x)` takes CORE-MATH's `cosh` at the bound of largest magnitude for the upper bound, and, for the lower one, at the bound of smallest magnitude, or 1 when x contains 0, and a value beyond the largest double for $|x| \geqslant 711$. `acosh(x)` and `atanh(x)` proceed as `sinh`, on $x \cap [1, +\infty]$ and $x \cap [-1, 1]$ respectively, with $\operatorname{acosh}(1) = 0$ and $\operatorname{atanh}(0) = 0$ exactly. The hyperbolic arc tangent is defined on $(-1, 1)$, $-\infty$ and $+\infty$ being its limits at -1 and 1, which are bounds but no values: the result is $\varnothing$ when x holds no point of $(-1, 1)$, as for $[1, 1]$, $[1, 5]$, $[-1, -1]$ and $[-5, -1]$. [GAOL v5] The six are tightest, and exact at the points given. The tests check them within 1 double, tightest at 0, at 1, and beyond 711 and 20, and $\varnothing$ for those four intervals.

11.2.5 Recommended functions

The recommended functions of Table 10.5 of the standard must be valid, and the standard recommends them to be accurate. All of them are tightest but the roots of an order other than 1, 2 and 3, which are within 2 doubles of the tightest for a positive order and within 4 for a negative one, and `compoundm1`, which is tightest but in a case out of 10^{17} or so.

Roots. `nth_root(x, n)`, for $n > 0$, computes `rootn`. For $n = 1$ it is x, and for $n = 2$ it is `sqrt`. For $n = 3$ it is CORE-MATH's `cbrt` at the bounds, correctly rounded upward on their magnitudes, the root of a negative number being the opposite of the root of its magnitude: the tightest bounds, and exact where the cube root is a double, which cubing the value tells; it takes 50 ns rather than 189 with the search below, on an Intel i7-1185G7 with Clang 18.1. [GAOL v5] For the other orders, on x (odd n, the root of a negative number being again the opposite of the root of its magnitude) or on $x \cap [0, +\infty]$ (even n), the lower bound is the largest double l with l^n rounded upward at most the bound of x, and the upper bound the smallest double u with u^n rounded downward at least it, which proves them. They are looked for from CORE-MATH's `pow` with the exponent $1/n$ rounded, after a step of Newton's method, by steps that double then by bisection, two powers where the start is next to the root; the roots of 0 and 1 are 0 and 1. The orders 1, 2 and 3 are tightest, and so is any order where the bound of x is an n-th power of a double. Otherwise the bounds are the tightest or up to 2 doubles beyond, whatever the magnitude of x: the $n - 1$ rounded products of l^n move the root by less than 2^{-52} relatively. Two doubles beyond is not accurate where the order is large: `nth_root([0x1.16ef146ba87dap-473], 1000)` has its upper bound two doubles above the tightest one, one above the accurate one. An order beyond the `int`s and the

`unsigned int`s takes CORE-MATH's `pow` at the corners, with the exponent $1/n$ as an interval, within 1 double of the tightest. [GAOL v5] The tests find the roots within 2 doubles at the doubles they draw, 1 double being the largest distance found, among them the x below 2^{-1020}, subnormal ones included, with n up to $2^{32}-1$, against exact arithmetic and mpmath, and the tightest on the values of the test `ieee1788` beyond the `unsigned int`s.

`nth_root(x, q)`, for $q<0$, is $1/x^{1/|q|}$, the inverse of the root above: the domain is then $\mathbb{R}\setminus\{0\}$ for an odd q and $(0,+\infty)$ for an even one, which taking the inverse gives, an interval holding 0 having for inverse the hull of the values away from it; `nth_root(x, 0)` is $\varnothing$. [GAOL v5] It is valid, and within 4 doubles of the tightest bounds: the root above, then the tightest division. It is not always accurate: `nth_root([2.25], -127)` has its upper bound 4 doubles above the tightest one, 3 above the accurate one. The tests find that it encloses the root over 1.5 million values against a reference in `long double`, and that it is the inverse of the positive root over 8 000 values.

Exponentials minus one and logarithms of one plus. `expm1(x)`, `exp2m1(x)` and `exp10m1(x)` are CORE-MATH's functions at the bounds, correctly rounded upward, the double below the value at the left bound unless that value is a double, which b^x-1 is at 0, and at the integers of $[-53,53]$ for $b=2$ and of $[0,15]$ for $b=10$; the result is within $[-1,+\infty]$. [GAOL v5] `log1p(x)`, `log2p1(x)` and `log10p1(x)` are CORE-MATH's functions at the bounds of $x\cap[-1,+\infty]$, the value at -1 being the limit $-\infty$, and x holding no number above -1 giving $\varnothing$; the double below the value at the left bound unless it is a double, which $\log_b(1+x)$ is at 0, at 2^k-1 for $b=2$ (k in $[-53,53]$) and at 10^k-1 for $b=10$ (k in $[0,15]$) only. [GAOL v5] The six are tightest. The tests find the three exponentials tightest over 400 000 points and intervals each, and the three logarithms tightest over 40 000 intervals each, their bounds drawn among those points and their neighbours.

The functions of πx. For `sinpi(x)` and `cospi(x)`, the extrema are at the multiples of 1/2, which are doubles: doubling the bounds, which is exact, and taking their ceiling and floor give exactly the integers $t=2x$ within x, and $\sin(\pi x)$ has its maximum 1 at the t congruent to 1 modulo 4 and its minimum -1 at those congruent to 3 (the cosine at 0 and 2); otherwise the result is CORE-MATH's function at the bounds, exact at the multiples of 1/2. Beyond 2^{61} two distinct doubles are whole periods apart, and the result is $[-1,1]$. [GAOL v5] Both are tightest, at every magnitude: sinpi(10^{17}) is 0, where `sin` of $\pi\cdot x$ gave $[-1,1]$. `tanpi(x)` proceeds as `sinpi`, its poles being at the odd t: $[-\infty,+\infty]$ with a pole within x, $-\infty$ or $+\infty$ for a pole at a bound, and $\varnothing$ for a pole alone, where the function has no value. [GAOL v5] It is tightest. The tests find the three tightest over 400 000 points and intervals each.

`atanpi(x)`, `asinpi(x)` and `acospi(x)` are CORE-MATH's functions at the bounds, `asinpi` and `acospi` on $x\cap[-1,1]$, `acospi` decreasing; they are exact at 0 and ±1 (and at $\pm\infty$ for `atanpi`), the only doubles where the values are doubles (`asinpi`$(\pm1/2)=\pm1/6$, `acospi`$(1/2)=1/3$ and `acospi`$(-1/2)=2/3$ are rational

too, but no doubles). [GAOL v5] `atan2pi(y, x)` is the analysis of the box of `atan2`, the angles divided by π: $[-1, 1]$ for a box crossing the half-line $y = 0$, $x < 0$, and the corners elsewhere. The values at the multiples of 1/4 (0, $\pm 1/4$, $\pm 1/2$, $\pm 3/4$, 1, and -1 as a limit) are exact, and CORE-MATH's `atan2pi` gives the others, which are irrational. [GAOL v5] The four are tightest. The tests find them so over 400 000 points and intervals each (40 000 intervals for `asinpi`, and the arguments next to ± 1 where the accurate phase of CORE-MATH's `asinpi` shifted a 64-bit integer by 65 to 69 bits, which `3rd/README.md` describes), and `atan2pi` over 200 000 boxes.

Hypotenuse and inverse square root. `hypot(x, y)` is CORE-MATH's `hypot` at the point of the box nearest to the origin, $(\operatorname{mig} x, \operatorname{mig} y)$, and at the farthest, $(\operatorname{mag} x, \operatorname{mag} y)$. The lower bound is the value itself where it is a double: writing a, b and $h = \sqrt{a^2 + b^2}$ as odd integers times powers of two, $a^2 + b^2 = h^2$ can only hold where h has the power of two of the one of a, b whose power is the smaller, and $h^2 - b^2 = a^2 \cdot 4^d$ is then compared exactly on 107 bits. [GAOL v5] `rsqrt(x)` is CORE-MATH's `rsqrt` at the bounds of $x \cap [0, +\infty]$, decreasing, the value at 0 being the limit $+\infty$, and x holding no positive number giving $\varnothing$; it is exact at the powers of 4. [GAOL v5] Both are tightest. The tests find `hypot` tightest over 200 000 boxes, among them Pythagorean triples scaled by powers of two and the doubles next to them, and `rsqrt` over 40 000 intervals.

Compound minus one. `compoundm1(x, y)` computes $(1+x)^y - 1$ with the analysis of the box of `pow` for the base $1 + x$: the value at the corner each bound is at, since $(1 + x)^y - 1$ increases with y for $x > 0$ and decreases for $x < 0$, increases with x for $y > 0$ and decreases for $y < 0$, and is 0 on $x = 0$ and on $y = 0$. The limits -1 and $+\infty$ are the bounds at $x = -1$, at $x = +\infty$ and for an infinite y, and the empty set is the result for $x < -1$ and for $x = -1$ with $y \leqslant 0$, where the function has no value. CORE-MATH has no `compoundm1`, and $\exp(y \operatorname{log1p}(x)) - 1$ computed with doubles multiplies the relative error of log1p by $|y \operatorname{log1p}(x)|$: up to 1 100 doubles from the value near the overflow, over 300 draws with the `expm1` and `log1p` of Python. The value at a corner is therefore computed in `gaol/math/gaol_compoundm1.h` with integers only. A fast phase in 64-bit fixed point comes first, whose error bound is proved (27 to 200 units of 2^{-63} relatively, 4.6 to 10 times the worst error found); it settles 95 % of the pairs for $|y \operatorname{log1p}(x)|$ below 0.35, 92 % from 0.35 to 1, 90 % from 1 to 2, 83 % from 2 to 4 and 71 % from 4 to 8, and none past, nor for $|\operatorname{log1p}(x)|$ above 8, $x < -0.99966$ or $x > 2\,980$. The other pairs go on to a 128-bit significand in balls, which add the error of each operation to a bound of the error. Both phases start from the $\operatorname{log1p}(x)$ of CORE-MATH, corrected by one step of Newton's method ($d = (1 + x)\exp(-y_0) - 1$, then $y_0 + d - d^2/2 + d^3/3$), multiply it by y, then take $\exp -1$ by a series of 27 terms after the reduction by a multiple of $\log 2$. The ball is no wider than 2^{-111} relatively for $|y \operatorname{log1p}(x)|$ below 746 (2^{-118} below 10, 2^{-120} below 1), and the bounds are the doubles around it. A corner takes 0.1 μs where the fast phase settles it and 0.6 to 0.9 μs in the balls, on an

Intel i7-1185G7 with GCC 15: 0.12 μs for a point and 0.23 μs for a box of x between -0.3 and 0.5 and y between 0.1 and 3, instead of 2 μs with the balls alone.

The cases where the value is a double are told apart. For $y = -1$ the value is the quotient $-x/(1+x)$ of two integers, which is divided exactly, and the value is the double where the division is exact. The value is the double d where $(1+x)^y = 1+d$, which integers prove for $y = P/2^k$: $s = 1+x$ a power of two with $y \log_2 s$ an integer, or s the 2^k-th power of an integer R times a power of two with $R^P = 1+d$ ($2.375 = 1.5^3 - 1$, 1.25, $0.5 = \sqrt{2.25} - 1$, $2^m - 1$ for $|m| \leqslant 53$); it is $y\,x$ on the side $y(y-1)$ tells where $y\,x$ is a double and $x(|y|+2) \leqslant 1/2$, the ball not telling the side where $x\,|y-1|$ is below 2^{-119}; and $2^m - 1$ is between the double below 2^m and 2^m for m past 53, which the ball cannot tell from $m = 118$ on. [GAOL v5]

The function is tightest, but where the value is within 2^{-111} relatively of a double that is not the value, none of the cases above tells it and the fast phase, which tells apart the doubles that are 2^{-55} to 2^{-58} apart relatively or more, has not settled the pair: about one pair (x, y) in 10^{17} drawn at random for the balls, 5 % of the pairs going to them, and none among the 5 854 pairs of the tests; the result is then one double wider on that side. The contrived values $y = -1$ and $x = \pm(2^e + 2^{2e-1})$ for e from -51 to -40, where $1/(1+x) = 1 - x + x^2 - x^3 \ldots$ has its cubic term cancelled and is within 2^{-155} of a double, are the tightest ones now, with the division. The tests find the function tightest over 5 854 pairs (x, y) of every magnitude, among them the values that are doubles, the tiny x down to the subnormals, $y = -1$ (719 values of x, the contrived ones among them, 633 of them divided), the edges of the fast phase ($|\mathrm{log1p}(x)| = 1/2$ and 8, $y\,\mathrm{log1p}(x) = 8$ and $\log(2)/2$), the thresholds of the reduction and of the cuts, and 850 boxes (the definition read from the hull of the corners), against mpmath at 1 500 to 7 000 bits and fractions; the balls of log1p and exp -1 are checked against mpmath to 256 bits.

11.2.6 The error functions

IEEE 1788-2015 does not name `erf` and `erfc`. GAOL computes them with CORE-MATH's `erf` and `erfc` at the bounds, correctly rounded upward, `erf` being increasing and `erfc` decreasing: the value at one bound, and the double below the value at the other one unless that value is a double, which it is at 0 ($\mathrm{erf}(0) = 0$, $\mathrm{erfc}(0) = 1$) and at $\pm\infty$ (the limits ±1 of erf, 2 and 0 of erfc); the results are within $[-1, 1]$ and $[0, 2]$. $\mathrm{erf}(x)$ is within 2^{-53} of ±1 from $|x| = 5.87$ on, where its bounds are $1 - 2^{-53}$ and 1, or -1 and $-(1 - 2^{-53})$; $\mathrm{erfc}(x)$ is subnormal from $x = 26.55$ on, and below 2^{-1074} from $x = 27.22$ on, where its lower bound is 0. Neither function is known to take a value that is a double at any other double, and the sources of CORE-MATH treat no such case. [GAOL v5] Both are tightest. The tests find them so at the 409 doubles of `elementary_values.h` (tiny arguments, changes of binade, the arguments where the values round to ±1, 2 or 0 or become subnormal, and the hard cases of CORE-MATH's `erfc.c`) and over the intervals between them, against mpmath, and over 9 225 doubles and 80 000 intervals against CORE-MATH called in both directed roundings.

11.2.7 Reverse functions

The relational functions of GAOL compute `f_rel(J, I)`, the hull of the x of I whose image is in J: they are the reverse functions of Table 10.1 of the standard with the arguments in another order (see Section 7.3, page 160). Each computes the preimage of J with the functions above, rounded outward, and intersects it with I: it is valid, which the standard requires. The test `tests/reverse.cpp` checks, on the cases of the minimal tests of libieeep1788 and on random ones, that they are also accurate, which the standard recommends for inf-sup types, the hulls being computed from the definition of each function, apart from GAOL (see Section 14.2, page 274); the test `tests/other_functions.cpp` checks that they keep the values they are given, within the numbers of doubles below.

- **Square root.** `sqrt_rel(c, x)` (sqrRev) is accurate; the tests find it accurate, and within 1 double of the value it keeps.

- **Absolute value.** `invabs_rel(c, x)` (absRev) is tightest: its bounds are those of J, of $-J$ and of I.

- **Integer power, $p > 0$.** `nth_root_rel(c, p, x)` (pownRev) has the bounds of the roots of `nth_root()`, tightest for $p \leqslant 3$ and up to 2 doubles beyond the tightest otherwise, which is not accurate where the order is large; beyond the `unsigned int`s it is the hull of the parts of x in those roots and, for an even p, in their opposites. [GAOL v5] The tests find it within 1 double of the tightest on their cases, and within 2 doubles of the value it keeps.

- **Integer power, $p \leqslant 0$.** [GAOL v5] For $p = 0$, `nth_root_rel(c, 0, x)` is x where 1 is in c, and $\varnothing$ elsewhere: exact. For $p < 0$, c is cut at 0, and the part at least 0 and, for an odd p, the part at most 0 each give the roots `nth_root(part, p)`, the root then the tightest division, up to 4 doubles beyond the tightest bounds; the result is the hull of their parts in x, 0 left out. The hull of the roots of c, which joins its parts, would keep the part of x between them, and the roots reach 0 where c is unbounded, 0 having no power of negative exponent. The result is valid, and the tests find it within 2 doubles of the accurate bounds of 12.10.1, but for $c = [2.25, 2.25]$ and $p = -127$, where it is 3 beyond: it is $\varnothing$ where the hull is $\varnothing$ in the cases written out, and of the 18 052 random cases with $p < 0$ of `tests/reverse_values.py`, 12 495 have a nonempty result with nonempty accurate bounds, 521 of them one double beyond, 25 two, none more. It is at most 2 doubles from the tightest bounds on the table of `tests/reverse.cpp`, 3 on those random cases (60 of them), and 4 at most, found for $c = [2.25, 2.25]$ and $p = -127$. Where $p = -1$ and $1/c$ reaches beyond the largest double, c being below $1/$`DBL_MAX`, a bound saturates at $\pm$`DBL_MAX`, as `div_rel` does: valid, but beyond 4 doubles of the tightest, and not empty where the hull is: the result for $c = [2^{-1074}, 2^{-1074}]$, $p = -1$ and $x =$ [0, `DBL_MAX`] is [`DBL_MAX`, `DBL_MAX`].

- **Trigonometric functions.** `asin_rel`, `acos_rel` and `atan_rel` (sinRev, cosRev, tanRev) are accurate, or one double beyond: the pieces of the preimage are $k\pi$, enclosed within about one double from π in double-double, plus or minus the inverse function of J, which is added before the sum is rounded; beyond 2^{52} a bound of x is kept, and an x of a single double is decided by the image of the function. The tests find them accurate but for one double, with the bounds of x beyond 2^{52}, and within 6 doubles (4, 5 and 3 found) of the value they keep from 1 to 2^{50}, away from the points where the inverse function magnifies the width of the image.

- **Hyperbolic functions.** `acosh_rel(c, x)` (coshRev) is accurate; the tests find it within 16 doubles of the value it keeps. `asinh_rel` and `atanh_rel`, named after coshRev (sinhRev and tanhRev are not in Table 10.1), are accurate; `atanh_rel(J, x)` is $\varnothing$ where J holds no point of $(-1, 1)$, the values of the hyperbolic tangent. [GAOL v5] The tests find them within 10 and 26 doubles of the value they keep, and $\varnothing$ for $J = [1, 1]$ and $[-1, -1]$.

- **Division.** `div_rel(c, b, x)`, and `c % b` for $x = [-\infty, +\infty]$ (mulRev), are accurate, and tightest where the bounds of the quotients are doubles. The tests find them accurate, the tightest over the cases of `%`, and `div_rel` within 2 doubles of the value it keeps.

- **Powers.** [GAOL v5] `pow_rel(c, b, x)` and `pow_exponent_rel(c, a, x)` (powRev1, powRev2) are tightest. The pieces of the solutions over the signs of the other argument ($y > 0$, $y < 0$; $x > 1$, $x < 1$), with $0^y = 0$ for $y > 0$, $x^0 = 1$ and $1^y = 1$, end at $c^{1/y}$ or $\log(c)/\log(x)$, c, x and y being bounds of the arguments; their bounds are the largest double d with d^y (or x^d) proved by CORE-MATH's `pow`, correctly rounded, to be on one side of c, and the smallest one on the other side. A bound of x where the solutions start between it and the next double, and whose power is within one double of c, is kept, which only knowing whether that power is a double would settle: the result is then within the tightest one for x one double wider. The tests find them tightest on the 804 cases of ITF1788 and on 1 244 others, random or whose solutions are doubles, within the tightest result for x one double wider on 966 cases where a bound of x is next to an end of the solutions, and keeping the 54 305 points of 6 000 random boxes whose power is within c.

- **Maximum, minimum, sign and floor.** [GAOL v5] `max_rel(z, y, x)`, `min_rel(z, y, x)`, `sign_rel(z, x)` and `floor_rel(z, x)`, whose reverse functions Table 10.1 does not name, are tightest: their bounds are those of x and z, 0, $\lceil \underline{z} \rceil$ and $\lfloor \overline{z} \rfloor + 1$, the open ends of the sets of `sign_rel` (0) and `floor_rel` ($\lfloor \overline{z} \rfloor + 1$) included, the last rounded upward beyond 2^{53}. The tests check them against the definitions on every interval of half-integer bounds, on cases written out and by sampling, and `floor_rel` at every magnitude, against $\lfloor \overline{z} \rfloor + 1$ computed exactly.

- **Two-argument arc tangent.** [GAOL v5] `atan2_rel_y(c, b, x)` and `atan2_rel_x(c, a, x)` (atan2Rev1, atan2Rev2) are accurate. The angles of c in $(-\pi, \pi]$ are cut at the axes into four cones, whose sides have the slopes $\tan\theta$ at the bounds θ of c (CORE-MATH, correctly rounded, the slope of a subnormal θ being θ itself), and 0 or $+\infty$ at the axes; the bounds of the box, and their products and quotients with the slopes, are rounded outward. The result is within 2 doubles of the tightest where both are nonempty; a box within one rounding of the cone may give a point that the tightest enclosure, the empty set, does not hold. The tests find them within 2 doubles of the tightest over 1 969 cases (1 263 nonempty) against mpmath at 3 000 bits, around the multiples of $\pi/4$, the axes, the half-line $y = 0$, $x < 0$, and with zero, subnormal and infinite bounds, and keeping every point whose angle is in c, over about 600 000 such points of the boxes, among them points of their sides next to the sides of the cones.

Remark 11.2.4 *Where a bound of J is one double from the image of a bound of the preimage, the reverse functions keep a point of I whose image is just outside J:* `sqrt_rel`$([1+2^{-52}, +\infty]$, $[-1, 1+2^{-52}])$ *is* $[-1, 1+2^{-52}]$*, whose tightest enclosure is* $[1, 1+2^{-52}]$*. They are accurate all the same, the widened J of nextOut holding that image.*

mulRevToPair(b, c) (10.5.5) is the pair of the two parts, each a `div_rel` of c by a part of b over $[-\infty, +\infty]$, the part of b at most 0 and the part at least 0: [GAOL v5] as tight as mulRev, so the tightest where the bounds of the quotients are doubles. The test `tests/ieee1788.cpp` checks it against exact quotients, and the 172 cases of ITF1788 pass.

cancelMinus(x, y) and cancelPlus(x, y) (10.5.6) are `cancel_minus(x, y)` and `cancel_plus(x, y)`, the tightest, which 12.12.5 requires. Whether the result exists depends on x being at least as wide as y, which rounding the differences of the bounds cannot decide when they are within a double of each other; each difference is computed exactly, as the sum of its value rounded to nearest and of its error, by the TwoSum of Knuth. The result is $[-\infty, +\infty]$ where there is no value, and $\varnothing$ for an empty x and a bounded y. They are checked over 200 000 pairs against exact rational arithmetic, 11 524 of them of exactly equal widths. [GAOL v5]

11.2.8 Set operations and numeric functions

The set operations, the constructors, the numeric functions and the boolean functions are computed from the bounds, and most of them are exact. The relations are described in Chapter 6, the reading and the writing of intervals in Chapter 9; here are their accuracies.

- **Intersection and hull.** `x & y` and `x | y` (intersection, convexHull) are taken on the bounds: exact.

- **Constructors.** `interval(l, u)` and `interval(d)` (numsToInterval) store the bounds, and the empty set where $l > u$, $l = +\infty$, $u = -\infty$ or a bound is NaN: exact.

- **Reading.** `gaol::textToInterval(s)` and `operator>>` (textToInterval) read the numbers, decimal, hexadecimal, octal or binary, compared exactly with the doubles around them; the rational numbers p/q are divided as intervals; the uncertain form is read as its exact decimal bounds; the literals `[ ]`, `[empty]`, `[entire]`, the infinite and the missing bounds, in any case of letters, are read (see Section 9.1, page 183). GAOL also reads expressions, such as `"sin(1)+0.1"`, computed with the operations above. The reading is tightest for the literals of IEEE 1788-2015, p and q being doubles. The literal `[l, u]` is empty where l is above u, numbers that are not doubles being compared exactly, but for `pi` and another number between the same two doubles, a number far beyond the doubles, or bounds that are computed, which give the interval from the lower bound of l to the upper bound of u. [GAOL v5] It is valid for expressions. The tests find it tightest.

- **Bounds.** `left()` and `right()` (inf, sup) return the stored bounds, and NaN for the empty set, where IEEE 1788-2015 has $+\infty$ and $-\infty$, a zero lower bound of either sign, where inf gives -0, and a zero upper bound of either sign, where sup gives $+0$ (12.12.8), as `gaol_ieee1788::inf()` and `sup()` do: exact.

- **Midpoint.** `midpoint()` (mid) is $(l + u)/2$ rounded to nearest, ties to even, or $l/2 + u/2$ where $l + u$ may overflow, a bound being 2^{1023} or more in magnitude, which is tested first, so that [`DBL_MAX`, `DBL_MAX`] raises no overflow exception; [GAOL v5] it is 0, $-$`DBL_MAX` and `DBL_MAX` for the unbounded intervals, and NaN for the empty set: as 12.12.8 requires. `mid()` is the same midpoint rounded outward: tightest.

- **Width and radius.** `width()` (wid) is $u - l$ rounded upward, NaN for the empty set, $+0$ for a point zero; `rad()` and `mid_rad(m, r)` (rad, midRad) give the greater of $m - l$ and $u - m$ rounded upward, m being `midpoint()`, $+\infty$ for the unbounded intervals, NaN for the empty set and $+0$ for a point zero of either sign: as 12.12.8 requires. `mag()`, `mig()` and `smig()` are taken on the bounds: exact. `hausdorff(x, y)` is the distance rounded upward: tightest.

- **Relations.** `is_empty()`, `is_a_double()` and `set_contains(d)` (isEmpty, isSingleton, isMember), the set relations `set_eq`, `set_contains` and `set_leq`, `set_strictly_contains` and `set_le`, `set_disjoint` (equal, subset, interior, disjoint, as Tables 10.3 and 10.4), the certainly relations `certainly_leq`, `certainly_le`, `certainly_geq` and `certainly_ge` (precedes, strictPrecedes, true when an interval is empty), `is_entire()` and `is_common_interval()` (isEntire, isCommonInterval, a common interval being nonempty and bounded, the NaN bounds of the empty set being no infinities [GAOL v5]) and `less(y)` and `strictly_less(y)` (less, strictLess, as Table 10.3, strictLess counting an

infinite bound as below itself, with the empty cases of Table 10.4 [GAOL v5]) are decided on the bounds: exact (Chapter 6). The tests find them exact, and `less` and `strictly_less` as Tables 10.3 and 10.4 over 20 000 pairs.

- **Overlap.** `x.overlap(y)` and `gaol_ieee1788::overlap(x, y)` (overlap) give the state of Table 10.7 among the 16 of `overlapping_state`: **`bothEmpty`**, **`firstEmpty`** or **`secondEmpty`** for an empty operand, else the order of the four bounds, told as the integers of their bits (`gaol_detail::bound_key()`: −0 and +0 are the same bound, and an infinite bound is the same as itself); a point interval on a bound of the other one is **`starts`**, **`finishes`**, **`finishedBy`**, **`startedBy`** or **`equals`**, never **`meets`** nor **`metBy`**. [GAOL v5] Exact. The tests read the set specification of Table 10.7 with quantifiers over a finite model of the real line, on 31 449 pairs (the empty set, the 104 intervals of a grid of bounds, infinities, zeros of both signs, subnormals and neighbours of a double among them, and 20 000 random pairs whose bounds meet); they check disjoint, equal and subset as masks of the states, the converse state when the operands are exchanged, the 48 cases of ITF1788 without decorations, and the same state with a mode that flushes the subnormals to zero.

- **Writing.** `gaol_ieee1788::intervalToText(x)` writes the bounds `[l, r]`, `[a]` for a point interval whose digits are exact (`[4]`, and `[0]` for a zero of either sign), or `[empty]`, in decimal with the digits of `interval::precision()` and a decimal point, whatever the global format and the locale. [GAOL v5] `operator<<` in the format **`bounds`**, and in **`agreeing`** for the intervals it writes with their bounds (those whose bounds are not both finite, nonzero and of the same sign, or are written with different numbers of digits before the point or different exponents, or share no first digit that is not 0), writes the same, with the digits and the flags of the stream, and a decimal point whatever its locale, so that the reader reads the text back; **`agreeing`** writes the other intervals with the digits their bounds share (see Section 9.2.2, page 191). Each bound is written to nearest by the C library, compared exactly with the bound, and its last digit moved outward when it is on the wrong side. The format **`hexa`** writes the bounds in the hexadecimal-significand form of 13.4.1, `[a]` for a point interval and `[0x0p+0]` for a zero. The decimal formats are valid, and the tightest with the digits asked for where the C library rounds to nearest as it should; the hexadecimal format is exact. The tests find less than one unit of the last digit in decimal and the exact bits in hexadecimal, read the text of intervalToText back as a literal, over every format, precision and a comma locale, and read the text that `operator<<` writes for a point under that locale back as the point, or as an interval containing it.

- **The formats for the eye.** `operator<<` in the formats **`width`** and **`center`** writes the `midpoint()` to nearest and the `rad()` rounded upward, [GAOL v5] as `c (+/- w)`, or `c` for a point interval, and `[empty]` for the empty set; the digits of c are those of the precision, so that the text need not contain the interval.

As doubles, $[c - w, c + w]$ contains x, w being the smallest radius that does so, and a nonzero radius is never written 0. The tests find the radius written at least the exact half-width and `rad()` rounded upward with less than one unit of its last digit, $[c - w, c + w]$ containing x, and the midpoint written to nearest with at most half a unit of its last digit, over the special values, the subnormals, the largest doubles and random intervals, every precision and flag.

- **Recovery.** `exact_string(x)`, which `operator<<` writes with **`hexa`**, and `gaol::textToInterval(s)` (intervalToExact, exactToInterval) meet the recovery requirement of 13.4: an interval written in hexadecimal and read again gives the same bounds, bit for bit, the empty set, the infinite bounds, a bound -0 and the subnormals included, but for a point interval of zero, written `[0x0p+0]` whatever the signs of its bounds and read back as $[+0, +0]$, the same set. [GAOL v5] Exact, over 2 000 random intervals and the values written apart.

11.3 The same bounds on every machine

GAOL is built with CORE-MATH only, whose sources are compiled into the library (see Section 1.7, page 15): the bounds of this chapter are the bounds on every machine. They are therefore those of every build of GAOL, and the test `tests/core_math.cpp` checks them against CORE-MATH called in the downward and the upward rounding, which are the tightest bounds there are.

The SSE2, the NEON and the FPU intervals (see Section 12.1, page 228) compute the integer powers with the same code, from exact products (`pown_products()` of `gaol/core/gaol_interval_core.cpp`), and the square from one product per bound, and give the same bounds. The SSE2 intervals of the earlier versions of GAOL v5 multiplied rounded products from the highest bit of n, and their bounds and those of the FPU intervals differed for about half of the intervals with a bound 0 or infinite or whose power leaves the range of the exact products. [GAOL v5] `pow(x, 2)` is `sqr(x)` in every form of the integer power with the exponent 2 (`pow(x, 2)`, `2u`, `2.0` and `[2]`, `gaol_ieee1788::pown(x, 2)`, and `gaol_ieee1788::pow(x, [2])` on the part of x in $[0, +\infty]$, which is $[0, 0]$ for the part $[0, 0]$): [GAOL v5] the same function, so the same bits, the sign of a zero bound being that of `sqr` in each build. In the earlier versions of GAOL v5, the integer power made its square from exact products, which has the same bounds and gave a zero bound of the other sign for part of the intervals holding 0: `pow(interval(0, 2), 2)` was $[-0, 4]$ with the NEON intervals and `pow(interval(-1, 2), 2)` was $[-0, 4]$ with the FPU ones, where `sqr` gives $[0, 4]$; with the NEON intervals `pow(interval::universe(), 2)` was $[0, +\infty]$ and is $[-0, +\infty]$, as `sqr` has it. The exception flags are those of `sqr` as well (see Section 4.9, page 107): the corner products that are not a bound of the result raise the inexact and the underflow flags too (`pow([-1e-310, 1e154], 2)`

raises underflow for the square of its lower bound, which its lower bound 0 leaves out), and the overflow flag is raised when the upper bound is $+\infty$ only, the smaller of two squares overflowing only with the larger: none of 14 million cases of each build had a finite upper bound with it.

The bounds are the same, but the sign of a zero bound may differ between the kernels, $+0$ and -0 being the same number.

Example 11.3.1 `sqr(interval(-2.0, 3.0))` *is* $[-0, 9]$ *with the SSE2 and the NEON intervals, and* $[0, 9]$ *with the FPU intervals.*

So may the sign of a zero bound of a product or of a quotient. The SSE2 intervals take the larger of four products, and the quotient of the corners their signs choose, with no branch, where GAOL 4 chose one product by the signs of the bounds: `[-1, 0] * [-1, 0]` has the lower bound $+0$ where it had -0. [GAOL v5] The FPU intervals take the larger of four products too (for each bound, the lower one made of products with a factor negated), [GAOL v5] but with a comparison and a selection that the compiler compiles as it chooses, not an instruction: where two products are zeros of opposite signs, the sign of the zero bound of a product is the compiler's and the target's, $+0$ or -0. The lower bound of `[-1, 0] * [-1, 0]` is $+0$ with GCC for 32-bit ARM and for arm64, with Clang and with Visual C++ for x86, and -0 with GCC 13 for POWER9, which compiles the selection into an `xsmaxcdp` that returns its first operand for a $+0$ and a -0. The FPU intervals give the bits of the SSE2 intervals, checked against a model of their kernels and not by running them, for the products of finite bounds but for that sign, and for the quotients by a divisor without zero: `[0, 1] / [-2, -1]` has the upper bound -0, where the FPU intervals gave $+0$ before, the zero bound of the dividend being divided. `div_rel(K, J, I)` of the SSE2 intervals takes the code of the FPU and the NEON intervals where a bound of K is 2^{500} or more in magnitude, or a bound of J that is not 0 is below 2^{-500}, the quotients of the vectors being liable to overflow, [GAOL v5] so that a zero lower bound may have the sign of that code: `div_rel([0, 0x1.e2ab041732196p-5], [2^-1074, 0x1.182p+9], [-DBL_MAX, 0x1.70ef9db22d0e5p+2])` is $[0, 5.764625]$, a zero lower bound of the other sign than the vector code gives: with $J = [1,$ `0x1.182p+9`$]$, which the vector code takes, the result is $[-0,$ `0x1.e2ab041732196p-5`$]$.

The two paths of the SSE2 intervals, with and without the AVX-512 path (see Section 4.4, page 94), give the same bits, but for the sign of a zero bound of `x * d`, `x *= d` and `d * x` (computed as `x * d`) where the zero bounds of x have different signs, $[+0, -0]$: it is $[+0, -0]$ with SSE2 and $[-0, +0]$ with AVX-512. The NEON intervals take the larger of four products too, [GAOL v5] with the maximum of 64-bit ARM, which ranks $+0$ above -0, where the `maxpd` of the SSE2 intervals returns its second operand when the two compare equal: the lower bound of `[-1, 0] * [-1, 0]` is -0 with the NEON intervals, where the SSE2 ones give $+0$ and the FPU ones the compiler's, and so may another zero bound of a product. `interval::zero()` is stored $\langle +0, +0\rangle$ as with SSE2, so that its lower bound is -0 (see Section 12.1, page 228). The tool `tests/tools/arith` compares the NEON and the FPU intervals

on a grid of operands, with the calling code in each of the four rounding directions: it found no other difference.

11.4 Discussion

This chapter documents the tightness of each operation of `interval`, as 12.10.3 of IEEE 1788-2015 requires, from its algorithm, and gives the largest distance the tests found. The basic operations, the integer functions and the absmax functions are tightest, as the standard requires. The elementary functions, the recommended functions and the error functions are tightest too, where the standard requires validity and recommends accuracy, because CORE-MATH is correctly rounded in the direction in effect: the value at a bound and the double below it are the tightest bounds, and the cases where the exact value is a double are told apart with integers. The exceptions are stated with their size: the roots of an order other than 1, 2 and 3 are within 2 doubles, their inverses within 4, the integer power is one double beyond where x^n is within $2K$ of a double, which is rare at random but frequent for a base next to 1, and `compoundm1` in about one pair in 10^{17}. The reverse functions are valid, which the standard requires, and accurate, which it recommends, or within a few doubles of the accurate bounds, as the chapter says of each; the tests measure how far they are from the values they keep.

Several choices trade a little tightness or time for a proof. The real power proves that its lower bound is a double with integers rather than call CORE-MATH downward, which would double its cost; `compoundm1` settles most pairs in a fast phase of 64-bit fixed point whose error bound is proved, and sends the others to balls on 128 bits; the roots of an order above 3 are proved by the power of the double on each side, at the price of up to 2 doubles. The bounds are the same on every machine and in every build, CORE-MATH being compiled into the library and the kernels computing the integer powers with the same code; only the sign of a zero bound differs between the SSE2, the NEON and the FPU intervals, and between the compilers for the FPU ones, which changes no set.

What remains is stated as such: the widths of `tan` between $\underline{\pi}$ and π are a theoretical case where the result is wider than the tightest one, and the intervals of floats are not covered. The next chapter describes how GAOL computes these bounds: the register that holds the negated lower bound, the SSE2, AVX-512, NEON and FPU kernels, the check of the rounding direction and its cost, and the comparison with the other libraries.

Chapter 12

How GAOL computes its intervals

The chapters before this one describe what GAOL computes: the operations, their bounds and their accuracy. This chapter describes how GAOL computes them, and what each part of the computation costs. An interval library spends its time in three places: the arithmetic itself, which is one or a few instructions; the rounding direction, which the processor holds in a control register that is slow to write; and the elementary functions. Which of the three weighs most depends on the processor, on the way GAOL is built and on the program. The measurements of this chapter say how much each weighs on the machines we had, and they are given in the manner of an experimental study: each table of measurements names its machine and, where the source gives them, its compiler and its date, and no number is given that the measurements do not give.

In this chapter, we first present the representation of an interval in a register and the kernels that compute with it on each family of processors, SSE2, AVX-512, NEON and the floating-point unit, with the rule that makes a build take the AVX-512 path (Section 12.1). Section 12.2 profiles the demonstration programs of GAOL to see where the time of a program goes. Section 12.3 gives the cost of a write of the control register on each processor and what each instruction set offers instead, and Section 12.4 what the compilers give, and do not give, to a program that needs a rounding direction. Section 12.5 compares GAOL v5 with GAOL 4.2.3 and with five other libraries of interval arithmetic, on the special cases of the standard, on the enclosure of the elementary functions and on the time of the operations. Section 12.6 reports the run on GAOL v5 of ITF1788, a suite of test cases for IEEE 1788-2015 written outside the standard, and of the benchmark of Tang et al., and Section 12.7 draws the verdicts on the paths that were considered and says what a new one has to show. The chapter closes with a discussion.

12.1 The representation of an interval and the kernels

On most platforms, switching the rounding direction is costly. The number of switches is cut down by Proposition 1.3.3 (see Section 1.3, page 9): GAOL stores the lower bound of an interval negated, and computes both bounds rounding upward, so that the rounding direction can be set toward $+\infty$ once and for all.

Definition 12.1.1 *An interval $[a, b]$ is kept as the pair* `<-a, b>`*, its lower bound negated, in one vector of two doubles: a* `__m128d` *on x86 with the SSE2 intervals, a* `float64x2_t` *on 64-bit ARM with the NEON intervals. The FPU intervals keep the same two numbers in two doubles. The kernel of an operation is the function of the library that computes the two doubles of its result from the two doubles of each operand, which it takes and gives in registers, after checking that the rounding direction is upward and setting it when it is not (the kernels of the AVX-512 path carry the direction in the instruction, set none, and check only the modes that flush the subnormals to zero, see Section 4.4, page 94).*

Example 12.1.2 *Let $x = [a, b]$ and $y = [c, d]$ be kept as* `<-a, b>` *and* `<-c, d>`*. One vector addition rounded upward gives $\langle \uparrow -a - c\uparrow, \uparrow b + d\uparrow \rangle$, which is $\langle -\downarrow a + c\downarrow, \uparrow b + d\uparrow \rangle$ by Proposition 1.3.3: the register of $x + y$. The lower bound is minus the upper bound of the opposite, so that one addition rounded upward gives both bounds (the accuracy of each operation is the subject of Chapter 11).*

Table 12.1 says what each build computes with vector instructions and what stays scalar. The SSE2 intervals are those of x86, the NEON intervals those of 64-bit ARM, both under the option `GAOL_SIMD` of CMake (see Section 4.5, page 96); the AVX-512 path is a variant of the SSE2 intervals (see Section 4.4, page 94); the FPU intervals are those of every other processor (see Section 4.6, page 98).

Remark 12.1.3 *The two bounds of an interval are one vector of two doubles, and* `+`*,* `-`*,* `*`*,* `/`*,* `sqr` *(and* `sqrt` *where it has a kernel) are one or a few vector instructions: a wider register does not help one interval. Only a batch interface, on boxes or arrays of intervals, which GAOL has not, could use 256 or 512 bits.*

12.1.1 The SSE2 and NEON kernels

The NEON intervals compute as the SSE2 intervals do on x86. [GAOL v5] The sum and the difference are one vector addition rounded upward. The product is the maximum of the four products of the bounds, with no branch on their signs. The quotient is one vector division when the divisor has no zero, and single divisions otherwise. With NEON, the square roots of both bounds are one vector square root rounded upward, the lower bound being that root where it is exact, and the double below it otherwise. The compilers that take the NEON intervals, the options that select

TABLE 12.1 – What each build of GAOL computes with vector instructions, and what stays scalar (from `doc/simd.md`).

Build	Vector kernels	Scalar
SSE2 (x86, `GAOL_SIMD`)	`+ - * / sqr`, the operations with a double, `inverse`, `%=`, `-x`, the integer powers with `__FMA__`, the leaves of `div_rel`	`sqrt`, `&=`, `|=`, `mid`, `midpoint`, the relations, the elementary functions, the reverse functions
AVX-512 path	`+ - * / sqr`, the operations with a double and the integer powers (with `__FMA__`), with the rounding direction in the instruction; `sqrt` as two scalar square roots and `/` as two scalar divisions, with embedded rounding	as SSE2 but `sqrt`; `inverse`, `%=` and the leaves of `div_rel` stay the vector kernels of SSE2, which set the direction
NEON (arm64, `GAOL_SIMD`; every compiler)	`+ - * / sqr sqrt`, the operations with a double, `%=`, `inverse`, `&=`, `|=`, `-x`, the integer powers (two lanes with `vfmaq_f64`)	`mid`, `midpoint`, `div_rel`, the relations, the elementary functions, the reverse functions, and `sqrt` of an upper bound above 2^{1000}
FPU (the rest)	none	everything

them and what the user sees of them are described in Chapter 4 (see Section 4.5, page 96).

The products and the quotients were nine cases of the signs of the bounds, and the squares three, told apart by a `switch` in GAOL 4.2.3 and in GAOL v5 at first. When the signs of the operands are random, as those of the benchmark of Section 12.5 are, a mispredicted branch costs more than the operation it chooses. The kernels now compute with no branch on the signs: [GAOL v5] the larger of four products for a product, one division of the two lanes by the denominators the signs choose, by masks, for a quotient, two squares and a mask for a square. The two doubles of each operand are passed in a register to the function of the library that computes an operation, which returns the two doubles of its result in one, where the operators of GAOL 4.2.3 read their operand and wrote their result through memory, the result going back to memory for the operation that followed. Each operation still checks that the rounding direction is upward, with one comparison, and sets it when it is not, where GAOL 4.2.3, in its trust mode, takes it upward on trust (see Section 4.1, page 88).

The NEON intervals check the direction by reading the control register FPCR with `mrs`, which raises no exception flag, and write it with `msr` only if the direction is not upward with the flush modes clear (see Section 4.8, page 104); the FPU intervals add $1+(2^{-1060}+0)$ instead, which raises the inexact flag (see Section 4.9, page 107). `msr` on FPCR is a serializing instruction, slower than `mrs`, which is why it is written when needed only. With `GAOL_PRESERVE_ROUNDING`, each operation reads FPCR once and writes back the direction and the modes it found at its end only if they

differ from what the operation left: a program that computes upward already, with the flush modes clear, pays no write (see Section 4.3, page 93). The bounds of the NEON intervals are those of the FPU intervals but for the sign of a zero bound (see Section 3.10.6, page 78), which the maximum of 64-bit ARM, ranking +0 above −0, can set otherwise than the SSE2 and the FPU intervals do, where the `maxpd` of x86 returns its second operand. The tool `tests/tools/arith` compares the NEON and the FPU intervals on a grid of operands, with the calling code in each of the four rounding directions: it found no other difference.

12.1.2 When the build takes the AVX-512 path

The AVX-512 path is off in the default build. The build takes it only where the option `GAOL_PRESERVE_ROUNDING` is on, where the compiler has the instructions and where the machine building runs them, and leaves it out otherwise; the setting of the developers `-DGAOL_PREFER_AVX512=ON` asks for it without that option, and `=OFF` refuses it (see Section 4.4, page 94). A library built with the path needs a processor with the instructions. The build decides it the same way with the three build systems. [GAOL v5] For `+`, `-`, `*` and `sqr` the path puts the two bounds in two of the eight lanes of a 512-bit register, for the embedded rounding only; `/` and `sqrt` are scalar instructions with the embedded rounding, two of them for the two bounds (see Section 12.7, page 253).

The rule comes from four builds of one source, `GAOL_PRESERVE_ROUNDING` on or off, each with the path (`GAOL_PREFER_AVX512` on) and without it, measured by the script `performance-ab.sh` of `.github/scripts/` with `--runs 9 --lines`: `gaol_performance` and the long lines of `doc/compare/code/run_lines.sh`, the two builds run in turn, the median of 9 runs, in nanoseconds, on an Intel i7-1185G7, whose AVX-512 units are halved, with Clang 21.1.8, `-O3 -mfma`, on 7 November 2026, the machine doing nothing else. Table 12.2 gives the times; the ratio is the time with SSE2 over the time with the path, so that above 1 the path is faster.

The median of the seven operations is 0.83 without `GAOL_PRESERVE_ROUNDING`: the operations take 20 % longer with the path, and the ten long lines 72 % longer (the table gives four of them; the others are in the output of the script); their median ratio is 0.58; only `sqrt` gains. It is 2.87 with it: every operation gains, the ten lines by 1.91 to 4.64, with a median of 3.19. The path pays where each operation would otherwise write the control register twice, and costs where the register is written only if the direction is not upward already, as it is by default: the rule is therefore that of `GAOL_PRESERVE_ROUNDING`. The threshold was a median gain of 5 % over `+ - * / sqr sqrt` and `pown`. One processor was measured, and not one with full-width AVX-512 units: such a processor is expected to lose less without `GAOL_PRESERVE_ROUNDING`, which the rule does not rely on.

TABLE 12.2 – The SSE2 intervals against the AVX-512 path, with and without `GAOL_PRESERVE_ROUNDING`, in nanoseconds per operation or per line (median of 9 runs, Intel i7-1185G7, Clang 21.1.8, 7 November 2026).

	Not preserved			Preserved		
Operation	SSE2	AVX-512	Ratio	SSE2	AVX-512	Ratio
`x + y`	1.22	1.51	0.81	8.82	1.49	5.92
`x - y`	1.33	1.56	0.85	9.38	1.53	6.13
`x * y`	2.75	4.45	0.62	12.14	4.46	2.72
`x / y`	3.21	5.00	0.64	11.05	5.07	2.18
`sqr(x)`	1.97	2.03	0.97	8.42	2.04	4.13
`sqrt(x)`	8.52	4.71	1.81	13.54	4.72	2.87
`pow(x, 3)`	6.61	7.93	0.83	10.42	7.83	1.33
line of 15 `+` (`sum16`)	19.67	40.66	0.48	176.0	37.92	4.64
line of 7 `*` (`mul8`)	23.80	37.42	0.64	86.56	35.14	2.46
line of 7 `/` (`quot8`)	26.50	46.56	0.57	86.73	43.59	1.99
line of 13 `+ - * /` (`mix13`)	27.75	43.23	0.64	152.8	41.49	3.68

12.1.3 CORE-MATH is scalar

CORE-MATH [13], in `3rd/math-core`, is scalar, with no vector variant; GAOL calls 38 of its binary64 functions, once per bound. The fused multiply-add, `GAOL_FMA`, pays in these functions and in the integer powers of GAOL, `cos` being 2.5 times faster and `pow(x, 3)` 3.2 times, and does nothing for the arithmetic; the cost of each build option is given with the installation (see Section 2.8, page 48).

12.2 Where the time of a program goes

To know which part of GAOL a program waits for, the demonstration programs of `examples/` were profiled. The call counts of Table 12.3 are exact, from a NEON build with `-finstrument-functions`. The shares of the time are indicative to about 3 points: one Apple M2 (NEON intervals), the profiler `sample` of macOS at 1 ms during 3 s per program, each sample put to the nearest GAOL function above it, other work running on the machine; the two passes of the profile differ by up to 3 points, and the table gives the second.

- **The solvers.** On the six solvers 06 to 11, the arithmetic is 28 % of the time weighted by the duration of a run (20 % with equal weights), the elementary functions 4 % (20 %), and the code outside GAOL (the priority queue, `malloc` of the boxes, the program itself) 62 % (45 %). The weighted figures are those of 06, which lasts 80 % of the total. Inside GAOL, at equal weights, about 40 % is arithmetic, 34 % the elementary functions, 10 % the reading and writing of text and 7 % `mid()`.

- **The code around CORE-MATH.** The code of GAOL around CORE-MATH weighs 21 to 66 % of the time of an elementary function (median about

TABLE 12.3 – Where the time of the demonstration programs goes: the calls of GAOL and the shares of the time in the arithmetic, in the elementary functions and outside GAOL (Apple M2, NEON intervals, the profiler `sample` of macOS; from `doc/simd.md`).

Program	**Calls** (`+ - * /`, `sqr`, powers)	**Elementary calls**	**Arith.**	**Elem.**	**Outside**
03 dependency problem	0.14 M, 26 k	none	73 %	0	16 %
04 centered form	10 k	cos 10 k, acos 8 k	9 %	82 %	6 %
06 global optimization	25 M, 5 M, 0.49 M	none	31 %	0	63 %
08 SIVIA	1.0 M, 0.5 M	sin 173 k	14 %	12 %	71 %
09 estimation	0.5 M, 68 k	exp 247 k	20 %	42 %	31 %
11 ODE	0.2 M, 8 k, 4.9 k	sin 12 k, cos 3 k, exp 1 k	29 %	34 %	30 %
16 Goldstein-Price	32 M, 6 M powers	none	98 %	0	2 %

42 %): the check of the rounding direction, the reduction of the argument, the construction of the result. In the code of `exp` around the CORE-MATH call, 43 % of the samples fall on the line of the check of the direction, a `mrs` of 0.8 ns: the samples probably land on the instruction that waits for it. The CORE-MATH calls themselves cost 3 to 13 ns on the M2.

- **The other operations.** The relations, `hull`, `intersect`, `min` and `max` are 3.6 % or less in every program; `mid()` is 6 % of the global optimization, and `midpoint()` 11 to 12 % of the Krawczyk iteration (29 ns on the M2: two writes of FPCR).

- **The check of the direction.** The check (one `mrs` of FPCR here), removed in a copy, saves 0.8 % (03), 1.4 % (09), 4.7 % (06) and 8.3 % (16). `GAOL_PRESERVE_ROUNDING` on the M2, for a program that computes to nearest, makes 03 and 16 about 16 times slower, 06 eight times, 09 five times. On x86 with the AVX-512 path the ratio is not measured for these programs; the operations of the benchmark of Section 12.5.1 take 0.6 to 1.5 times as long (Table 12.9).

- **The text.** The text is 41 to 69 % of the five demonstration programs that print (01, 02, 05, 12, 15; 28 to 31 % of 14 and 11 % of 03), at 0.2 to 0.45 μs per bound written. Among the solvers it is 0 to 1 % of 06, 08, 09 and 11, 12 % of 10 and 22 % of 07.

On an Intel i7-1185G7, in the benchmark of Section 12.5.1 (a million operations on arrays of intervals, so partly bound by the memory), the time of an operation is 2.2 ns for `+` and `-`, 2.30 for `*`, 2.52 for `/`, 1.87 for `sqr`, 6.30 for `sqrt` and 5.12 for `pow(x,`

3), the same operation on doubles taking 0.96 ns for +. With GAOL_PRESERVE_ROUNDING and the AVX-512 path: 2.19, 2.16, 3.42, 3.60, 2.04, 3.75 and 6.18. On the M2, measured with tests/performance.cpp (the median of nine runs, Table 2.5), in which the i7-1185G7 takes 1.07 ns for x + y (the best of nine runs, Table 2.4; 1.22 ns, the median of nine runs, in Table 12.2), the NEON intervals take 0.90 ns for +, 1.52 for *, 2.08 for /, 1.27 for sqr and 2.04 for sqrt, against 1.15, 3.26, 3.22, 1.78 and 5.17 with the FPU intervals, whose *, / and sqr take no branch on the signs (1.15, 3.25, 3.98, 2.07 and 4.49 before).

12.3 The rounding direction, by processor

Except on the AVX-512 path, an operation computes upward after a write of the control register of the processor when the direction of the program is not upward already. What a write costs, by family:

- **x86.** LDMXCSR takes 1 to 6 cycles of throughput on the performance cores of Intel and on AMD processors (1.0 on Zen 5, 6.0 on an Arrow Lake P-core) and 8 on the efficiency cores, according to uops.info, which measures one instruction at a time and so does not count the wait for the operations in flight. STMXCSR, the read, takes 1 cycle on most Intel cores but 14 to 16 on Zen 2 to Zen 5: one more reason, with those given in gaol/gaol_fpu.h (800 ns under Rosetta 2, 80 ns with 32-bit Visual C++, a single read reused by Clang 18), why the check of GAOL on x86-64, in the default build, is an addition and not a read. Two writes took 6 ns a sqr (8.3 ns, 4.4 times, in the benchmark of Section 12.5.1) on the SSE2 path with GAOL_PRESERVE_ROUNDING, which the AVX-512 path spares.
- **64-bit ARM.** mrs of FPCR takes 0.8 ns, msr 4 ns for the value FPCR holds and 11 ns when it changes the direction (Apple M2). The reference manuals of the Neoverse N1, N2, N3 and of the Cortex-A78 say that a write that changes the control fields introduces a barrier or a flush.
- **32-bit ARM, POWER, s390x, LoongArch, MIPS, RISC-V.** No figure was found in the references.

Table 12.4 says what each instruction set offers instead of a write, read from its reference manual (the versions are listed at the end of doc/simd.md): whether an instruction can carry its own rounding direction, whether it has a fused multiply-add, which register holds the direction, and what a path of GAOL could be.

TABLE 12.4 – The rounding direction on each processor family: what the instruction set offers, and what a path of GAOL could be (from `doc/simd.md`).

Processor	Direction carried by the instruction	FMA	Register	What a path could be
x86-64 SSE2	no	FMA3, a set of its own: AVX2 does not imply it	MXCSR (RC, FTZ, DAZ)	the SSE2 path of today
x86-64 AVX-512F	yes, `{er}`, for the scalar instructions and the 512-bit ones	yes	MXCSR: RC and the exception flags are ignored, FTZ and DAZ are honoured	the AVX-512 path of today
x86-64 AVX-512VL, AVX10.1, AVX10.2	no at 128 and 256 bits: `VADDPD ymm` has no `{er}` in the Intel manual; AVX10.2 had it for 256 bits in revisions 1 to 3 and removed it in revision 4.0 (8 May 2025), and revision 7.0 (6 April 2026) keeps it for 512 bits only	yes	MXCSR	none; a vector of two doubles has to go through the 512-bit form, as the AVX-512 path does
AArch64, NEON, SVE, SME	no: only `FCVT*`, `FRINT*` and `FCVTX*` name a rounding in the A64 instruction set of March 2026; `FADD`, `FMUL`, `FDIV`, `FSQRT`, `FMLA` use FPCR, scalar, Advanced SIMD and SVE	yes	FPCR (RMode, FZ; FEAT_ AFP adds FIZ, AH and NEP)	the NEON path of today; SVE has 128-bit vectors on the Neoverse N2, N3, V2, V3 and the Cortex-A720, 256 on the V1, and adds no rounding
ARMv7	no; NEON has no double, and always rounds to nearest	VFPv4 only	FPSCR (RMode, FZ)	the FPU intervals
POWER (VSX)	no: `xsadddp` and `xvadddp` use FPSCR.RN; the `RO` bit exists for the quad precision only (round to odd)	yes	FPSCR (RN, NI)	VSX `<-l, r>` with `mffscrni`, needs hardware to measure
s390x	no: `ADB` and `VFA` have no rounding field; the `M3` field of the conversions is not the arithmetic	yes	FPC (BRM)	`VFADB`, needs hardware to measure

Table 12.4, continued

Processor	Direction carried by the instruction	FMA	Register	What a path could be
RISC-V, scalar (D)	yes: the `rm` field of `FADD.D`, `FMUL.D`, `FDIV.D`, `FSQRT.D`, `FMADD.D...`; `rup` is 011	yes	frm (no flush-to-zero mode)	two scalar instructions, no check, no register to restore; inline assembly only
RISC-V, vector (V)	no: all the vector floating-point operations use frm; the `_rm` intrinsics compile to `fsrmi` and `fsrm` around the operation	yes	frm	none
LoongArch	no	yes	FCSR	none
MIPS	no (read from the Linux headers and the compiler output, not from the manual)	not verified	FCSR (RM, FS)	none
WebAssembly 3.0	no: "Non-default directed rounding attributes are not supported"	`relaxed_madd` may not be fused	none	none: only the rounding to nearest

The RISC-V scalar path is the one case of a rounding direction by instruction that GAOL has not. Its interest is that it needs no check and no restore, so that `GAOL_PRESERVE_ROUNDING` would cost nothing, as on the AVX-512 path. No figure exists for it: no RISC-V board is available. The boards read about, in secondary sources, are the SpacemiT K1, which has the vector extension 1.0, the SiFive P550, which has none, and the T-Head C910, which has the draft 0.7.1.

12.4 What the compilers give

No compiler gives a rounding direction by instruction from C or C++: the paths of GAOL are written with intrinsics or inline assembly. What the compilers offer was tried, or read from their output, as follows.

- **The pragma `FENV_ROUND`.** `#pragma STDC FENV_ROUND` (C23) would ask for a static rounding. On the six targets tried (x86-64, aarch64, riscv64, ppc64le, s390x, loongarch64), GCC 14 to 16 and its trunk ignore it, with a warning only under `-Wall` (GCC bug 20785, open since 2005); Clang 18 to its trunk parses it, warns "not supported" and emits the ordinary instruction (the IR carries `round.upward`; no back end of the three tried, x86-64, aarch64 and riscv64, uses it); Visual C++ warns C4068. It does not exist in C++. `-Werror=unknown-pragmas` makes Clang's "not supported" an error, but not its other forms of the same pragma ("expected identifier" with the `FE_UPWARD` of `<fenv.h>`, which is a number; "not supported on this target"), which be-

long to another group of warnings, and `/we4068` should do it for Visual C++: none of these was tried.

- **C++.** P2746R7 (Boehm, January 2025) would replace `fesetround` by a type with an explicit mode, without wording. P4231R0 (May 2026, target C++29) unifies it with P3864; no revision since. Nothing is in C++26, and none proposes a SIMD form.

- **The intrinsics of the embedded rounding.** `_mm_add_round_sd` and `_mm512_add_round_pd` (and the other operations) compile with GCC 14 to its trunk, Clang 18 to its trunk and Visual C++ 19.20 to 19.51. The mode must be a literal, and the compilers do not fold a constant computed with `{ru-sae}`. There is no `_mm_add_round_pd`, and no `_mm256_add_round_pd` with GCC 15 and 16 or Clang 21 and later; Clang 20 with `-mavx10.2` declares it and compiles it to the encoding of the withdrawn revisions of AVX10.2, which the current Intel manual makes `#UD` (never run: no processor with AVX10.2 is on sale, as far as we found). In inline assembly, the GNU `as` of GCC 16.2 on Compiler Explorer assembles `vaddpd {ru-sae}, %ymm1, %ymm0, %ymm2` (bytes `62 f1 f9 58 58 d1`, against `62 f1 fd 58 58 d1` for the 512-bit form): no assembly of that form may be written.

- **`-frounding-math`.** GCC does not honour it across a change of the direction (bugs 34678 and 123482, open): on `fesetround(UP); r = a + b; fesetround(old)`, GCC 14 to its trunk moves the addition after the restore, on x86-64 and on AArch64. Clang keeps it between. The vector intrinsics are ordinary floating-point operations for them: with Apple clang 21 on the M2, at `-O2` with `-frounding-math` or `-ffp-model=strict`, `vaddq_f64` and `vfmaq_f64` are right, `vpaddq_f64`, `vaddvq_f64` and `vmulxq_f64` are computed once, in the first direction, and at `-O2` alone everything is (GAOL uses none of these three). What held on every compiler tried (GCC 14 to its trunk and Clang 18 to its trunk, compiled; Apple clang 21, run on the M2) is an empty assembly statement on each operand after the write of the control register, and on the result, or a copy of it through volatile memory, before the next one. GAOL does that: `reread_bounds()` and `rnd_reread()` are empty `asm` statements, `GAOL_RND_KEEP()` is a copy through volatile memory with SSE2 and the FPU and an empty `asm` with NEON, and `GAOL_RND_BARRIER()` is a memory barrier. A path written with intrinsics needs the same. Visual C++ for arm64 has no `asm` statement: its NEON intervals copy the result through volatile memory, `GAOL_RND_BARRIER()` and the writes of FPCR are `_ReadWriteBarrier()`, and the operands rely on `/fp:strict` (checked by the generated code and by `tests/rounding_direction.cpp`, not by the language).

- **The intrinsics of the other families.** NEON `float64x2_t` with GCC, Clang and Visual C++ for arm64 from 19.26; SVE `<arm_sve.h>` with GCC 14

and Clang 18, and in Visual C++ 19.50 and 19.51 only partially (`svmla_f64_x` becomes an unresolved call without a diagnostic); VSX `vec_add` with `-mvsx`; RVV with GCC from 14.1 and Clang from 17.0.1. `std::experimental::simd` is in libstdc++ (libc++ has the types but no `operator+`), and `std::simd` of C++26 only in GCC 16, for x86-64.

Remark 12.4.1 *A program that relies on* `#pragma STDC FENV_ROUND` *compiles and rounds in the wrong direction, without an error; so do the* `_mm256_add_round_pd` *of Clang 20 and the 256-bit* `{ru-sae}` *assembly. A path of GAOL therefore trusts no rounding direction that the generated code and* `tests/rounding_direction.cpp` *have not checked.*

12.5 GAOL compared with the other libraries

Six implementations of interval arithmetic on doubles are compared in `doc/compare/`: libieeep1788, GAOL v5, filib++, the intervals of Solaris Studio, PROFIL/BIAS and Boost.Interval. Table 12.5 says what each is, in which language it is written, how it computes its bounds and which arithmetic it follows. GAOL v5 gives its intervals under two namespaces, `gaol`, its own names, compatible with GAOL 4, and `gaol_ieee1788`, the names of IEEE 1788-2015 [9]. They compute with the same functions, and differ mainly in the power: the `pow` of `gaol` takes the integer power for an integer exponent, a negative base included, where the `pow` of `gaol_ieee1788` is the real power of the standard, defined for $x > 0$ only, and `pown` the integer power (see Section 3.5, page 70). The directory `doc/compare/code/` holds the scripts and programs, in C++, Fortran, Python and bash, that run the three parts of the comparison again and rewrite the tables of the reports.

TABLE 12.5 – The six implementations compared: what they are, their language, how their bounds are computed and the arithmetic they follow (from `doc/compare/README.md`).

Library	Language	Bounds	Arithmetic
libieeep1788, by Marco Nehmeier (University of Würzburg), last commit in 2015	C++11, header-only	Computed by MPFR, correctly rounded	The set-based flavor of the preliminary IEEE P1788, the prototype of IEEE 1788-2015
GAOL v5, by Jordan Ninin (ENSTA), which continues the GAOL of Frédéric Goualard	C++	Computed with the rounding direction set upward; elementary functions with CORE-MATH, correctly rounded upward	Set-based, following IEEE 1788-2015 in most of its special cases, with the deviations the reports list

Table 12.5, continued

Library	Language	Bounds	Arithmetic
filib++ 3.0.2.2 (University of Wuppertal), as IBEX distributes it, which IBEX computes with by default on Windows	C++, templates and a small library	Computed with the rounding direction set and restored by each operation (`native_switched`); elementary functions of its own	The extended mode of filib++ (`i_mode_extended_flag`, as IBEX uses it), where the empty set and the infinities are handled
Solaris Studio: the `interval(8)` type of Sun's Fortran 95 compiler, `f90 -xia`, in Solaris Studio 12.4 (2014)	Fortran 95, intervals built into the compiler	Computed by `libsunimath`	The containment sets of Sun's interval arithmetic (G. W. Walster), where the infinities are values
PROFIL/BIAS 2.0.8 (2009), by Olaf Knüppel and Christian Keil (TU Hamburg-Harburg)	C++ (PROFIL) over C (BIAS)	Computed by the BIAS routines, which set the rounding direction downward, then upward, then back to nearest; elementary functions from the libm, moved outward	The interval arithmetic before IEEE 1788: no empty set, and an argument outside the domain of a function is an error that aborts the program
Boost.Interval of Boost 1.92.0 (2026), by Hervé Brönnimann, Guillaume Melquiond and Sylvain Pion, the interval library C++ users find first	C++03 templates, header-only	Computed with the rounding direction set upward by each operation, the lower bounds negated, and set back after it (the default policies, `save_state<rounded_arith_opp>`); elementary functions from the C library, called with the rounding direction set downward or upward (`rounded_transc_opp`), a direction glibc does not always honour	The interval arithmetic before IEEE 1788, without empty set: its default checking (`checking_strict`) throws an exception where an operation would create one

The comparison has three parts, each with a report of its own in `doc/compare/`:

- **Special cases.** 290 special cases taken from GAOL's tests (infinities, zeros, NaN, empty sets, divisions by zero, `pow` and `pown`, `+=` and the other operators with doubles, reading from text, midpoints, widths, radii, comparisons and `atan2`), computed by the six libraries and compared with the results of IEEE 1788-2015, GAOL under its own names and under those of the standard (Section 12.5.2).

- **Enclosure.** Whether the elementary functions of GAOL and Boost.Interval enclose their values, at 100 000 random arguments for each function, against mpmath, GAOL under both namespaces (Section 12.5.3).
- **Performance.** The time of a million additions, subtractions, multiplications, divisions, sines, cosines, powers, one-line combinations, evaluations of Shekel 5 and five-line blocks, and the time of long lines of $+$, $-$, $\times$ and $\div$ between 8 intervals, GAOL v5 being called under the names of the standard, as the other libraries of IEEE 1788, and GAOL 4.2.3 under its own (Section 12.5.1).

Table 12.6 sums the three parts up. Its times were measured on 17 October 2026, every library in the same run, compiled by Clang 21.1, on the machine of Section 12.5.1.

TABLE 12.6 – The comparison in short: the special cases, the enclosure of the elementary functions and the time of six operations (17 October 2026, every library in the same run, Clang 21.1, Intel i7-1185G7).

	libieee p1788	**GAOL**	**filib++**	**Solaris Studio**	**PROFIL/ BIAS**	**Boost. Interval**
Special cases with IEEE 1788's result	281 of 281	288 of 288 under `gaol_ieee1788`; 280 under `gaol`	126 of 245	150 of 261	56 of 223	93 of 168
… or an interval enclosing it	0	0; 2	50	41	22	9
… or another result	0	0; 6	69	70	145	66
Cases it has no operation for	8	0	44	28	65	121
Elementary functions at random points: results missing the value	—	0 of 1 600 000	—	—	—	34 % (20 % with `rounded_transc_std`)
`x + y`	137 ns	2.2 ns	5.1 ns	18 ns	14 ns	12 ns
`x * y`	171 ns	2.3 ns	12 ns	21 ns	15 ns	21 ns
`sin(x)`	5.5 μs	43 ns	33 ns	41 ns	115 ns	71 ns
`log(x)`	1.9 μs	21 ns	27 ns	39 ns	10 ns	30 ns
`pown(x, 3)`	240 ns	5.1 ns	18 ns	145 ns	35 ns	20 ns
Shekel 5	9.0 μs	108 ns	397 ns	1.7 μs	1.1 μs	920 ns

- **libieeep1788** gives the result of IEEE 1788 in every case it can compute, as tightly as possible, and is 17 to 135 times slower than GAOL.

- **GAOL** is the fastest on the arithmetic, the integer power and most formulas. Under the names of the standard, `gaol_ieee1788`, it gives IEEE 1788's result in every special case; under its own, `gaol`, in all but 8, all from its hybrid `pow`, which takes `pown` for integer exponents, a negative base included. Its elementary functions, CORE-MATH's, give the tightest bounds; its `exp` and `log` are faster than filib++'s, its `sin` and `cos` slower.

- **filib++** is the fastest on `sin` and `cos`, 1.3 to 1.6 times as fast as GAOL, but 5.3 times as slow on $\times$, 3.5 times on $\div$ and 2.3 times on $+$ and $-$, and its bounds of elementary functions and real powers are up to 36 doubles wider than the tightest. Its extended mode gives other results than IEEE 1788 wherever an infinity or a division by zero is involved, much as Solaris Studio: `interval(+∞)` is [MAX, $+\infty$] and $[0, 0] \times [1, +\infty]$ is $[-\infty, +\infty]$; its `operator>>` rounds the bounds it reads to nearest, and its comparisons with the empty set and the infinities differ from IEEE 1788's.

- **PROFIL/BIAS** is a library of the interval arithmetic before IEEE 1788: it has no empty set, and an operation whose argument leaves the domain of the function, a division by an interval containing zero included, prints an error and aborts the program. Its exponential, logarithm and real power, taken from the libm, are the fastest of the six libraries (9.9, 10.2 and 34 ns), and so is its square root (3.7 ns); its arithmetic is 6.4 times slower than GAOL's on $+$ and $-$, its `sin` and `cos` are the slowest after libieeep1788's (115 and 131 ns), and its integer power, computed as $\exp(n \log x)$, is $1.1 \cdot 10^{-5}$ wider than the tightest on average.

- **Boost.Interval**, the library of Boost, is the interval library C++ programmers find first. Its arithmetic gave the tightest intervals in the benchmark, but its elementary functions are the C library's, called with the rounding direction set downward or upward, which glibc does not always honour: at the points of the test of enclosure, 34 % of its results miss the value of the function with glibc 2.43, 20 % with its other policy, and `atan(`$[-\infty, +\infty]$`)` misses $\pi/2$. With its default checking it has no empty set: an operation that would create one throws an exception. It is 5.6 times slower than GAOL on $+$ and $-$, 1.8 to 11 times on $\times$, $\div$ and the formulas, 3.5 times on the square root, 3.9 times on the integer power and 1.2 to 1.6 times on the other elementary functions.

- **Solaris Studio** is 1.0 to 1.3 times as fast as GAOL on `sin` and `cos`, but slow on integer powers and squares. Its containment sets give other results than IEEE 1788 wherever an infinity, a division by zero or an invalid argument is involved: `interval(+∞)` is [MAX, $+\infty$], $[1, 2]/[0, 1]$ is $[-\infty, +\infty]$ and `interval(2, 1)` is $[-\infty, +\infty]$.

12.5.1 Performance

One million operations of each kind are computed by each library on ordinary intervals: bounded, of widths from 10^{-6} to 1, drawn once and read from the same file by every program. Each operation is timed on the whole array, its results being stored in another one; the time per operation is the best time divided by the million. The same operations on doubles, computed at the midpoints of the intervals without any rounding, give the cost of the arithmetic itself. Table 12.7 lists the operations: a and b are intervals centred in $[-10, 10]$, p in $[1, 10]$, e in $[0.5, 2.5]$, the arguments of Shekel 5 in $[0, 10]$; their widths are 10^u, u uniform in $[-6, 0]$ ($[-6, -1]$ for e).

TABLE 12.7 – The operations of the benchmark of `doc/compare/performance.md`.

Operation	Computes
`add`	$a + b$
`sub`	$a - b$
`mul`	$a \times b$
`div`	a/p
`sqr`	a^2 (`sqr`, `square` in Boost.Interval, `x**2` in Fortran)
`sqrt`	$\sqrt{p}$
`exp`	$\exp(a)$
`log`	$\log(p)$
`sin`	$\sin(a)$
`cos`	$\cos(a)$
`pow_int`	a^3 (`pow(x, int)` in GAOL and Boost.Interval, `pown`, `power(x, int)` in filib++, `x**3`)
`pow_real`	p^e (`pow(x, y)`, `x**y`; $\exp(e \log p)$ with Boost.Interval, which has no real power)
`line_arith`	$(a + b)(a - b)/p$
`line_trig`	$\sin(a)\cos(b) + a^2$
`line_pow`	$\sqrt{p} \cdot a^3 - \exp(b/p)$
`shekel5`	Shekel 5: $-\sum_i 1/(\sum_j (x_j - a_{ij})^2 + c_i)$, 5 terms, 4 variables
`block5`	five lines: $t_1 = ab + p$; $t_2 = \sin(t_1)\cos(b)$; $t_3 = a^2 + t_2/p$; $t_4 = \exp(t_2) - b^3$; $t_3 t_4 + \sqrt{p}$

Everything was built and run again with Clang 21.1, `-O3 -mfma` and the flags of interval arithmetic (see Section 3.3, page 64), so that one compiler answers for every C and C++ library: GAOL v5 (V5.0.0) built by CMake in Release with `GAOL_FMA`, the sources of CORE-MATH compiled into it and the rounding direction not preserved, its default, and called under the names of IEEE 1788-2015, `gaol_ieee1788`, as the other libraries of the standard; the same GAOL built with `GAOL_PRESERVE_ROUNDING`, which takes the AVX-512 path (Section 12.1.2), in a column of the table of the times only; GAOL 4.2.3, the last version of Frédéric Goualard, built by its `configure` from his repository (cd0ee1a), static, the rounding direction not preserved, its verbose mode off, with mathlib 2.1.1 built by Clang 21.1 with `-O3 -mfma -ffp-contract=off`, and called under its own names; libieeep1788 with MPFR 4.2.1 and GMP 6.3.0; filib++ 3.0.2.2, the archive IBEX distributes, as `interval<double, native_switched, i_mode_extended_flag>`; PROFIL/BIAS

2.0.8 in its `x86-64-Linux-compat-gcc` configuration; Boost.Interval of Boost 1.92.0 with its default policies and `rounded_transc_opp` for the elementary functions, glibc 2.43. Only Solaris Studio keeps its own compiler, Sun Fortran 95 8.7 with `-O3 -xia`, by nature. Every library was measured in the same run, on 17 October 2026, and GAOL v5 with the rounding direction preserved again on 18 October, on a laptop with an Intel i7-1185G7 at 3.00 GHz (pinned by `taskset -c 2`), Linux 7.0.0-38-generic, Ubuntu 26.04.1; each time is the best of the runs of three rounds, each program being run in turn with the others. libieeep1788's times, from a single run of each operation in each round, are the least steady. Solaris Studio's and PROFIL/BIAS's operations are calls into libraries, as GAOL v5's `+`, `-`, `*`, `/` and `sqr` are, the bounds in registers, where the compiler inlines GAOL 4.2.3's, filib++'s and Boost.Interval's.

What GAOL v5 changes. GAOL v5 is compared with GAOL 4.2.3, which it continues and which bounds the elementary functions with mathlib (see Section 1.7, page 15). Table 12.8 gives the operations where they differ most. The elementary functions are faster: [GAOL v5] two things go at once with mathlib, the outward move of each bound, and the two changes of rounding direction each function made, to nearest before mathlib and back upward after. The integer power is computed from exact products, both bounds at once in an SSE2 register, with an error bound proved beforehand. The bounds are tighter, not looser: GAOL v5's results are no wider than libieeep1788's, which computes every bound with MPFR, for every one of the seventeen operations (excess 0.0), where GAOL 4.2.3 is wider on the square root, `exp`, `log`, `sin`, `cos`, both powers and the lines that use them.

Table 12.8 – GAOL v5 against GAOL 4.2.3, in nanoseconds per operation (Intel i7-1185G7, Clang 21.1, 17 October 2026).

Operation	GAOL v5	GAOL 4.2.3	Ratio
The elementary functions and the powers			
`log`	20.9 ns	46.8 ns	2.2 times faster
`exp`	18.3	35.1	1.9
`pow(x, y)`	48.2	89.9	1.9
`pow(x, 3)`	5.12	9.07	1.8
line of powers	38.1	61.0	1.6
`sin`	43.2	68.0	1.6
five-line block	155	229	1.5
line of sin and cos	102	148	1.5
`cos`	54.0	62.8	1.2
The arithmetic			
`mul`	2.30 ns	10.0 ns	4.4 times faster
`line_arith`	5.61	18.7	3.3
`sqr`	1.87	5.81	3.1
`div`	2.52	7.50	3.0
`shekel5`	108	189	1.7

The arithmetic is faster than GAOL 4.2.3's, which is what the operators of GAOL v5 are written for (Section 12.1). The sums and the differences, 2.2 ns against 2.4, and the square root, 6.3 against 6.3, take what they took: a sum is one instruction, and what it waits for is memory, the benchmark reading two arrays of 16 MB and writing a third, and the square root is the tightest interval whatever the rounding of the C library's `sqrt`, where GAOL 4.2.3's is wider (excess $1.4 \cdot 10^{-14}$). The check of the direction is what a program needs that changes the rounding direction: the benchmark sums the results rounding to nearest, and GAOL 4.2.3's own `midpoint()` leaves the direction to nearest. Left so, the operations of GAOL 4.2.3 timed next gave bounds narrower than libieeep1788's, which did not enclose the exact results; the benchmark now sets the direction back after its sums. The bounds are the tightest, as before: the zero bound of a product or of a quotient may have the other sign (see Section 3.10.6, page 78).

Preserving the rounding direction. The column of Table 12.9 where the direction is preserved (`GAOL_PRESERVE_ROUNDING` on, with the AVX-512 path) leaves the sums and the differences as they are, 2.2 ns, and costs 1.5 times on the products and 1.4 times on the quotients (3.4 and 3.6 ns), 1.1 times on `sqr` (2.0 ns against 1.9) and 1.2 times on `pow(x, 3)` (6.2 ns against 5.1): these operations take the AVX-512 instructions, whose rounding direction is in the instruction, and set none, so that nothing is saved and restored. The square root, 3.75 ns, is faster than without the preservation (6.3 ns): the instruction rounds each root in its direction, where the SSE2 path checks the root of the C library with a product. The elementary functions read the control register of the direction, set it upward and write it back, once each, and take 1.0 to 1.3 times longer (`exp` 23.2 ns against 18.3, `log` 20.4 against 20.9, `sin`, `cos` and `pow(x, y)` 1.1 times). They paid more, as `sqr` and the integer powers did, before the kernels of Section 12.1: the two writes of the control register, each one waiting for the operations in flight, took 6 ns a `sqr` (8.3 ns, 4.4 times); with the C library's `fegetround()` and `fesetround()` and the writes of the control word of the x87 unit as well, `pow(x, 3)` took 14 ns more (19.0 ns, 3.7 times; 8.4 ns with the writes of the SSE register alone, where the machine has no AVX-512) and the elementary functions 1.2 to 1.7 times longer. Shekel 5 takes 147 ns against 108, and every operation of the table is faster than with GAOL 4.2.3, which does not preserve the direction either (Shekel 5: 189 ns). The results are the same: the sums of the midpoints and of the widths of the million results are those of GAOL v5, to $2 \cdot 10^{-16}$ relatively.

What the timings show. On the arithmetic, GAOL is the fastest on the four operations: $+$ and $-$ take GAOL v5 2.2 ns (2.4 for GAOL 4.2.3), 2.3 times as fast as filib++ (5.1 ns) and 6.4 to 8.0 times as fast as PROFIL/BIAS (14 ns) and Solaris Studio (17.6 ns); $\times$ and $\div$ take GAOL v5 2.3 and 2.5 ns (10.0 and 7.5 for GAOL 4.2.3), against 12.1 and 8.7 ns for filib++, 14.7 and 14.4 ns for PROFIL/BIAS, 21 and 20 ns for Solaris Studio and 21 and 19 ns for Boost.Interval. An addition of

TABLE 12.9 – Time per operation, in nanoseconds: the same operations on doubles (double), libieeep1788 (p1788), GAOL v5 (v5), GAOL v5 with the rounding direction preserved and the AVX-512 path (v5 pres.), GAOL 4.2.3 (v4), filib++, Solaris Studio (Sun), PROFIL/BIAS (BIAS) and Boost.Interval (Boost). One million operations of each kind on the same intervals, the best of 3 rounds; Intel i7-1185G7, Clang 21.1.8, Ubuntu 26.04.1, 17 October 2026 (the preserved column on 18 October).

Operation	double	p1788	v5	v5 pres.	v4	filib++	Sun	BIAS	Boost
`add`	0.96	137	2.23	2.19	2.38	5.10	17.6	14.2	12.4
`sub`	0.92	138	2.21	2.16	2.40	5.12	17.6	14.3	12.3
`mul`	0.90	171	2.30	3.42	10.0	12.1	21.3	14.7	21.5
`div`	0.91	171	2.52	3.60	7.50	8.70	20.3	14.4	18.9
`sqr`	0.59	88.4	1.87	2.04	5.81	8.25	55.5	18.5	16.2
`sqrt`	1.30	107	6.30	3.75	6.26	13.4	26.6	3.73	22.3
`exp`	3.44	1 491	18.3	23.2	35.1	30.1	38.0	9.87	29.8
`log`	3.32	1 919	20.9	20.4	46.8	26.6	38.5	10.2	29.9
`sin`	13.4	5 472	43.2	46.8	68.0	33.4	41.5	115	71.2
`cos`	12.0	4 890	54.0	57.6	62.8	33.9	42.7	131	65.7
`pow_int`	13.0	240	5.12	6.18	9.07	17.8	145	35.1	19.9
`pow_real`	10.3	6 517	48.2	51.4	89.9	64.5	125	33.6	73.2
`line_arith`	1.14	655	5.61	8.24	18.7	27.4	61.6	63.1	61.7
`line_trig`	25.8	10 920	102	113	148	90.8	166	311	187
`line_pow`	19.4	2 401	38.1	43.7	61.0	81.9	253	92.6	115
`shekel5`	2.43	8 993	108	147	189	397	1 720	1 093	920
`block5`	44.5	13 823	155	171	229	197	464	427	343

intervals costs GAOL v5 2.3 times an addition of doubles: one vector addition for both bounds, and the check that the rounding direction is upward, where filib++ sets the rounding direction and restores it, and PROFIL/BIAS calls a function of BIAS that sets it downward, then upward, then back to nearest, which is why its four arithmetic operations all take about the same time. The integer power `pown(x, 3)` takes GAOL v5 5.1 ns and GAOL 4.2.3 9.1 ns: filib++'s `power(x, 3)` takes 18 ns, Boost.Interval's 20 ns, PROFIL/BIAS's 35 ns and Solaris Studio's `x**3` 145 ns; `sqr` takes GAOL v5 1.9 ns, against 8.2 for filib++, 16 for Boost.Interval, 18 for PROFIL/BIAS and 56 for Solaris Studio.

On the elementary functions, PROFIL/BIAS is the fastest on the square root (3.7 ns), `exp` (9.9 ns), `log` (10.2 ns) and the real power `pow(x, y)` (34 ns), which it computes from the libm of the system, moved outward, without correct rounding: its bounds are among the widest, and its `sin` and `cos`, computed by its own argument reduction, are the slowest after libieeep1788's, 115 and 131 ns. GAOL v5 comes next on `exp` and `log`, 18 and 21 ns, faster than Boost.Interval (30 ns each), filib++ (30 and 27 ns) and Solaris Studio (38 and 39 ns), with correctly rounded values, where the others are not. filib++ is the fastest on `sin` and `cos` (33.4 and 33.9 ns) with its own polynomials, then Solaris Studio (41 and 43 ns), then GAOL v5 (43 and 54 ns): GAOL pays there for correct rounding over the whole range, and for dividing the bounds by an interval enclosing π to find where the function is monotonic, asking CORE-MATH for the signs of the derivative where the division cannot tell. GAOL v5's `sin` became faster than its `cos` when CORE-MATH rewrote `sin.c`. On `pow(x, y)` GAOL v5 takes 48 ns, against 65 for filib++, 73 for Boost.Interval, 90 for GAOL 4.2.3 and 125 for Solaris Studio.

On the formulas, GAOL is the fastest on the arithmetic line (5.6 ns for GAOL v5 and 18.7 for GAOL 4.2.3, against 27 for filib++ and 62 to 63 for Solaris Studio, PROFIL/BIAS and Boost.Interval), on the line of powers (38 and 61 ns, against 82 for filib++, 93 for PROFIL/BIAS and 253 for Solaris Studio) and on Shekel 5 (108 and 189 ns, against 397 for filib++, 920 for Boost.Interval, 1 093 for PROFIL/BIAS and 1 720 for Solaris Studio: Shekel 5 is 20 squares, 45 additions and subtractions and 5 divisions, and its squares cost Solaris Studio 56 ns each and PROFIL/BIAS 18). GAOL v5 alone is the fastest on the five-line block (155 ns, against 197 for filib++, 229 for GAOL 4.2.3, 427 for PROFIL/BIAS and 464 for Solaris Studio). filib++ is the fastest on the line of sin and cos (90.8 ns, against 102 for GAOL v5, 148 for GAOL 4.2.3, 166 for Solaris Studio and 311 for PROFIL/BIAS), where its `sin` and `cos` weigh most.

libieeep1788 is 17 to 135 times slower than GAOL v5: every bound is an MPFR computation, about 140 ns for an addition and 5 to 7 μs for `sin`, `cos` and the real power. Its own documentation warns that its focus is correctness, not speed. GAOL v5 gives the same bounds as it does, tightest everywhere, 17 to 135 times faster. Boost.Interval is slower than GAOL v5 on every operation: 5.6 times on $+$ and $-$, 3.5 to 9.3 times on $\times$, $\div$, the square, the square root and the integer power, 1.2 to 1.6 times on `exp`, `log`, `sin`, `cos` and the real power, 11 times on the

arithmetic line and 8.5 times on Shekel 5. Each of its operations reads the rounding direction (`fegetround`), sets it upward (`fesetround`) and, at its end, sets the one it read back: two changes of direction where GAOL checks the direction and changes nothing. Its elementary functions add a change to the downward direction and back for the lower bound, and its `sin` goes through the reduction of $\cos(x - \pi/2)$ by an interval enclosing 2π. Its bounds are those of the C library, which are not correctly rounded, and of which 20 to 34 % miss the value of the function with glibc 2.43 (Section 12.5.3).

The same results. The sums of the midpoints of the million results agree to $4 \cdot 10^{-15}$ relatively, apart from PROFIL/BIAS's and the sines of GAOL 4.2.3 and Boost.Interval, $1.4 \cdot 10^{-13}$ and $1.3 \cdot 10^{-13}$ apart, their sum being small, 373 for a million values, which makes $5 \cdot 10^{-17}$ per result. The arithmetic operations give the tightest intervals in every library, and so do the square roots, except those of GAOL 4.2.3, filib++ and PROFIL/BIAS. Of the elementary functions, only GAOL v5's are the tightest on every operation: on average the others are wider than libieeep1788's by up to $5.9 \cdot 10^{-14}$ relatively with GAOL 4.2.3 (the real power, computed as $\exp(y \log x)$), $2.1 \cdot 10^{-14}$ to $2.4 \cdot 10^{-13}$ with filib++ ($1.4 \cdot 10^{-13}$ for `log`, $2.4 \cdot 10^{-13}$ for the real power), $9.0 \cdot 10^{-15}$ to $1.2 \cdot 10^{-13}$ with PROFIL/BIAS, and at most $4.1 \cdot 10^{-15}$ with Solaris Studio; with Boost.Interval, by $1.4 \cdot 10^{-14}$ and $2.5 \cdot 10^{-14}$ on `cos` and `sin` and $3.7 \cdot 10^{-14}$ on the real power, its `exp` and `log` being as tight as libieeep1788's on average. PROFIL/BIAS's integer power is far wider, $1.1 \cdot 10^{-5}$ on average: it computes `Power(x, n)` as $\exp(n \log x)$, where the others multiply.

Remark 12.5.1 *The arithmetic results of PROFIL/BIAS have midpoints that differ by* $8 \cdot 10^{-15}$ *to* $4 \cdot 10^{-14}$ *relatively while their widths are the tightest: the intervals are the same, but its* `Mid` *computes* $\inf + (\sup - \inf)/2$ *rounded upward, one double above the midpoint for 4 % of the sums of the benchmark, where the others round to nearest. In the same way, the sums of the widths of Boost.Interval's results of* `exp` *and* `log` *equal libieeep1788's, while its upper bound of* `exp` *is one double too low in 2 to 3 % of the results of the test of enclosure: a sum of a million widths cannot show it, and only a check of each bound against the exact value finds it.*

Long lines of arithmetic operators. The operations above are timed one at a time. A program that computes with intervals writes lines of them, an operator after another, where what an operation hands to the next matters as much as what it computes. The lines of Table 12.10 are expressions of $+$, $-$, $\times$ and $\div$ alone, of 7 to 19 operations between 8 intervals of the same million, evaluated for each of them, by every library but Solaris Studio, which has no program for them. The seven first take intervals of both signs, at random; the last three, `pos_sum16`, `pos_mul8` and `pos_quot8`, only p and e, which are positive, so that the branches of an implementation that tells the signs apart, as GAOL 4.2.3's does, are taken the same way from one evaluation to the next. The chains give the next operation the result of the previous

one, the trees (`mul8_tree`, `mix_tree`) several independent ones. Table 12.11 gives the times on the x86 machine of the benchmark, Table 12.12 on an Apple M2.

TABLE 12.10 – The long lines of arithmetic operators between 8 intervals (a, b, p, e as in Table 12.7, s_1 to s_4 in [0, 10]).

Line	Computes	Operations
`sum16`	$a+b+p+e+s_1+s_2+s_3+s_4+a+b+p+e+s_1+s_2+s_3+s_4$	15
`diff16`	$a-b+p-e+s_1-s_2+s_3-s_4-a+b-p+e-s_1+s_2-s_3+s_4$	15
`mul8`	$a \cdot b \cdot p \cdot e \cdot s_1 \cdot s_2 \cdot s_3 \cdot s_4$	7
`mul8_tree`	$((a \cdot b)(p \cdot e))((s_1 \cdot s_2)(s_3 \cdot s_4))$	7
`quot8`	$a/p/e/p/e/p/e/p$	7
`mix13`	$((((((a+b) \cdot p - e) \cdot s_1 + s_2) \cdot p - s_3)/e + a) \cdot b - s_4)/p + e$	13
`mix_tree`	$((a+b)(p-e) + (s_1 - s_2)(s_3 + s_4))/(p+e) + ((a-b)(s_1 + s_2) - (p \cdot e)/(p+e))/(e+p)$	19
`pos_sum16`	$p+e+p+e+p+e+p+e+p+e+p+e+p+e+p+e$	15
`pos_mul8`	$p \cdot e \cdot p \cdot e \cdot p \cdot e \cdot p \cdot e$	7
`pos_quot8`	$p/e/p/e/p/e/p/e$	7

TABLE 12.11 – Time per evaluation of a line, in nanoseconds, with the abbreviations of Table 12.9 (one million evaluations of each line, the best of 3 rounds; Intel i7-1185G7, Clang 21.1.8, 17 October 2026).

Line	double	p1788	v5	v4	filib++	BIAS	Boost
`sum16`	2.45	2 001	22.8	27.4	76.5	216	179
`diff16`	2.44	2 091	23.3	27.1	76.5	215	177
`mul8`	2.42	1 166	17.9	30.9	48.6	104	111
`mul8_tree`	2.37	1 168	15.0	29.4	50.5	105	111
`quot8`	3.36	1 178	20.6	34.0	48.6	102	112
`mix13`	2.38	2 068	23.0	47.6	80.7	192	185
`mix_tree`	2.38	2 985	34.7	64.7	116	278	264
`pos_sum16`	1.03	1 930	23.1	24.5	73.5	214	179
`pos_mul8`	0.89	1 139	17.8	18.9	40.2	100	105
`pos_quot8`	3.27	1 160	20.2	37.6	44.3	101	109

GAOL v5 is the fastest of the libraries on every line, and faster than GAOL 4.2.3 on all ten, 1.06 to 2.1 times as fast: a line of 15 additions takes 22.8 ns, 1.5 ns an addition, against 27.4 ns (1.8 ns), and a line of 7 products 17.9 ns (2.6 ns a product) against 30.9 ns (4.4 ns); the quotients, 20.6 ns for 7 (2.9 ns each) against 34.0 ns (4.9 ns), 1.7 times as fast, and the lines that mix the four operations, `mix13` and `mix_tree`, 23.0 and 34.7 ns against 47.6 and 64.7 ns, 2.1 and 1.9 times. The three lines of intervals of one sign are where GAOL 4.2.3 comes nearest, on the additions and the products, 1.06 times as fast as it, 23.1 ns against 24.5 ns and 17.8 ns against 18.9 ns: with the signs the same from one evaluation to the next, its branches are predicted, and what it still pays is the bounds that go through memory between two operations; its quotients, on the other hand, take 1.9 times as long (37.6 ns against 20.2 ns). An operation of a line costs GAOL v5 less than the same operation timed alone, 1.5 ns an addition against 2.2: the two doubles of the result stay in a register

for the next operation, and the benchmark of a single operation, which reads two arrays and writes a third for each, waits for the memory, where a line reads eight arrays at most for its 7 to 19 operations. Against the other libraries, filib++ takes 2.2 to 3.5 times as long as GAOL v5 on the ten lines, Boost.Interval 5.4 to 8.0 times, PROFIL/BIAS 5.0 to 9.5 times and libieeep1788 57 to 90 times. The bounds are the tightest: the mean widths of the results of GAOL v5 are those of libieeep1788 to $1.1 \cdot 10^{-16}$ relatively on the lines of additions and exactly on the others, where GAOL 4.2.3's differ from them by $2.3 \cdot 10^{-16}$ on the lines of quotients.

Table 12.12 – The same lines on a 64-bit ARM processor, in nanoseconds per evaluation: GAOL v5 with the NEON intervals (v5), the same with the rounding direction preserved (v5 pres.) and GAOL 4.2.3 with the FPU intervals (v4). Apple M2, Apple clang 21.0.0, macOS 27.0.1, 6 November 2026, the best of 3 rounds, the processor not pinned.

Line	**Ops**	v5	v5 pres.	v4	v5 pres. / v5	v4 / v5
`sum16`	15	9.11	385	41.9	42.3	4.6
`diff16`	15	9.30	390	77.6	42.0	8.3
`mul8`	7	10.5	187	28.3	17.8	2.7
`mul8_tree`	7	9.95	190	25.0	19.1	2.5
`quot8`	7	17.6	229	37.3	13.0	2.1
`mix13`	13	14.3	353	61.6	24.7	4.3
`mix_tree`	19	18.5	516	82.2	27.9	4.4
`pos_sum16`	15	8.85	385	39.7	43.5	4.5
`pos_mul8`	7	9.58	187	16.1	19.6	1.7
`pos_quot8`	7	17.6	232	46.5	13.2	2.6

On the Apple M2, GAOL v5 takes 8.85 to 18.5 ns for a line of 7 to 19 operations, 0.6 to 2.5 ns an operation, where GAOL 4.2.3, which has no instructions of 64-bit ARM to compute its FPU intervals with, takes 1.7 to 8.3 times as long (its `configure` was generated again by `autoreconf` and told `--build=arm-apple-darwin`, and mathlib 2.1.1 configured with `--host=aarch64-unknown-linux-gnu`; no line calls it). With the rounding direction preserved the same lines take 187 to 516 ns, 25.7 to 33.1 ns an operation, 13.0 to 43.5 times as long as without: the program computes to nearest, so each operation sets the direction upward and puts nearest back, two `msr` of FPCR of 11 ns each that change it, with the `mrs` of 0.8 ns that reads it. The sums of the midpoints and of the widths of the three agree to $1.1 \cdot 10^{-16}$ relatively. The machine was on mains power, the low power mode off, with a load average of 1.7 at the start and 3.0 at the end (the run took 65 s, the build of the three programs included), and no process above 10 % of a core but the interface of the machine in the readings taken just before and just after the run; the three rounds of a line differ by 1.4 % at the median and by 6.4 % at most, and the best of the three is kept.

12.5.2 The special cases

The special cases of GAOL's tests (`tests/*.cpp`) are computed by the six libraries, GAOL v5 twice, under its own names and under those of IEEE 1788-2015: zeros, infinities and NaN as bounds or operands, empty sets, divisions by intervals containing zero, arguments at the edges of the domains, the powers `pow` and `pown`, the operators `+=`, `-=`, `*=`, `/=` with special doubles, the reading of intervals from text, the numeric functions of intervals, the comparisons and `atan2`. Each result is compared with the result IEEE 1788-2015 defines for the set-based flavor, the tightest interval of doubles, computed with mpmath, and classed as the result of IEEE 1788, as an interval enclosing it, wider, as another result (a set not enclosing it, a set where IEEE 1788 has the empty set, another boolean, or an exception), or as not applicable when the library has no such operation; IEEE 1788 defines no result in two cases (a bare number read as an interval, and an n-th root of order 0). Signed zeros are not told apart. Each case is written once, in `doc/compare/code/cases.py`, and translated for each library; the errors of PROFIL/BIAS, which abort the program, and the exceptions of Boost.Interval are caught. The counts are those of Table 12.6.

libieeep1788 gives the result of IEEE 1788 in each of the 281 cases where IEEE 1788 defines one and it has the operation; it has no n-th root, no operators with doubles and no compound assignments. GAOL under the names of the standard gives the result of IEEE 1788 in each of the 288 cases where IEEE 1788 defines one. [GAOL v5] Under its own names it gives it in 280 cases, an interval enclosing it in 2, and something else in 6, all from the hybrid `pow(x, y)` of `gaol`, which takes the integer power for an integer exponent, where IEEE 1788's `pow` only takes the part of x in $[0, +\infty]$ (see Section 3.5, page 70): `pow([-2], 2.0)` is $[4, 4]$, `pow([0], [0])` is $[1, 1]$, `pow([-1], [2^31-1])` is $[-1, -1]$, `pow([-2, -1], [1e10])`, an integer beyond the ints, is $[1, +\infty]$, and `pow([-2, 0], [-1])` is $[-\infty, -0.5]$, where IEEE 1788 has $\varnothing$; `pow([-2, 3], 3)` is $[-8, 27]$, wider than $[0, 27]$. In GAOL 4, the integer powers beyond the ints were $[-\infty, +\infty]$, which made three more cases wider. [GAOL v5] GAOL reads a bare number, `textToInterval("0.1")`, as the interval enclosing it, an extension of the literals IEEE 1788 allows (see Section 9.1, page 183). Its `sin`, `cos`, `tan`, `acos`, `acosh`, `atan2`, integer powers and n-th roots are the tightest in every one of these cases, the powers being computed from exact products, x^{-n} included, and the roots proved with integer powers; across the half-line $y = 0$, $x < 0$, where the angle jumps from π to $-\pi$, `atan2` is $[-\pi, \pi]$, as in libieeep1788.

The other libraries depart from the standard each in its own way. filib++, in its extended mode, takes the infinities for values, much as Solaris Studio does: the intervals built from an infinity are [MAX, $+\infty$], a product of a zero and an infinite bound is $[-\infty, +\infty]$, `log([0])` is $[-\infty, -\text{MAX}]$ and `atanh([1])` is [MAX, $+\infty$] where IEEE 1788 has $\varnothing$; its `operator>>` reads only the form `[l, u]` and rounds the bounds to nearest, so that `[0.1, 0.3]` does not enclose [0.1, 0.3]; its elementary functions are wider than the tightest by up to 25 doubles and its real powers by up to 36. PROFIL/BIAS, a library of 1993 to 2009, has no empty set: an argument outside the domain of a function, or a division by an interval containing 0, prints

an error and calls `abort()`; the intervals built from an infinity, a NaN or bounds in the wrong order are not empty, `interval(2, 1)` being [1, 2]; its `sin` and `cos` are wider than the tightest by 4 to 14 doubles, its real powers by 3 to 14. Solaris Studio follows the containment sets of Sun's interval arithmetic, where 1/0 is $\{-\infty, +\infty\}$ and $+\infty$ is a point: `interval(2, 1)` and NaN bounds give $[-\infty, +\infty]$ rather than $\varnothing$, a division by an interval containing zero is $[-\infty, +\infty]$, a single number read as an interval carries an uncertainty of one unit of its last digit (`0.1` gives [0, 0.2]), and its powers are often one double wider than the tightest. Boost.Interval, with its default checking, throws `std::runtime_error` wherever the empty set would appear, the empty set given as an argument included; an infinity is a point, a product of a zero and an infinity has a NaN bound, its `median` overflows for [MAX/2, MAX], and `atan([-∞, +∞])` is [−1.5707963267948966, 1.5707963267948966], which does not hold the values of arctan near $\pm\pi/2$: Boost.Interval asks the C library for arctan($+\infty$) rounded upward, and glibc rounds it to nearest, below $\pi/2$, whatever the rounding direction.

12.5.3 The enclosure of the elementary functions

An interval library promises one thing: the interval it returns contains the exact result. The special cases check the promise at the edges of the domains; the test of enclosure checks it in the middle of them. Each of 14 elementary functions, the square root and x^3 is computed on the point intervals $[x, x]$ of 100 000 random arguments x, half of them ordinary, half at the edges of the doubles or of the domain, and each result is compared with $f(x)$, computed by mpmath with 2000 bits (8000 where $f(x)$ is that close to a double, as tanh(709) is to 1). A result $[l, u]$ misses $f(x)$ when $l > f(x)$ or $u < f(x)$. Boost.Interval computes its arithmetic in the rounding direction each bound needs, but its elementary functions are those of the C library, called with the rounding direction set downward for the lower bound and upward for the upper one: they enclose $f(x)$ only if the C library honours that direction, which its documentation says is rarely the case. GAOL v5, whose elementary functions are CORE-MATH's, correctly rounded in each direction, is the control: none of its results may miss. Both are compiled by Clang 21 with the flags of interval arithmetic, Boost 1.92.0 with glibc 2.43 (Ubuntu 26.04.1), under its two policies: `rounded_transc_std`, which sets the direction downward, calls the function for the lower bound, sets it upward and calls it for the upper one, and `rounded_transc_opp`, the policy of the benchmark, which keeps the direction upward and sets it downward only around the call for the lower bound.

GAOL v5 encloses $f(x)$ in every one of the 1 600 000 results, under both namespaces, and gives the tightest interval of doubles around it in all of them (1 232 results of x^3, where the cube is below 2^{-968}, came from products rounded outward and were one or two doubles wider, until these corners were computed apart, scaled; see Section 11.2.3, page 211). Boost.Interval misses $f(x)$ in 34 % of the results with `rounded_transc_opp` and in 20 % with `rounded_transc_std`. Its square root and its x^3, which it computes itself with the rounding of the processor, never miss,

nor do its `asinh` and `atanh`; every other elementary function misses, `log` once, the others in 0.05 % of the results or more, and `asin`, `atan`, `sinh`, `tanh` and `acosh` in 100 % with `rounded_transc_opp`. Three causes were found. The `sin`, `cos`, `tan` and `atan` of glibc ignore the rounding direction: called downward, to nearest and upward, they give the same double for every argument, which misses $f(x)$ unless $f(x)$ is that double; Boost.Interval's `sin`, `cos` and `tan` miss less often, 0.3 to 28 %, because they reduce the argument by subtracting a multiple of an interval enclosing π or 2π, which widens most results by tens to hundreds of doubles. The `exp`, `cosh`, `acos` and `log` of glibc are computed in the direction asked, but not always on the right side of $f(x)$: called upward, `exp` gives a double below $f(x)$ for 2 to 3 % of the arguments, `cosh` for up to 7 %, `acos` misses in 0.09 to 5 %, `log` once in the 100 000. The `asin`, `sinh` and `tanh` of glibc round the wrong way for many arguments, the value computed downward being above the one computed upward, as if the function were computed for $|x|$ and its sign changed afterwards: with `rounded_transc_std` the result is then inverted, its lower bound above its upper one, and the default checking does not see an inverted interval as empty, since it holds that no interval is.

Remark 12.5.2 `rounded_transc_opp` *has a bug of its own. For* `sin`*,* `tan`*,* `asin`*,* `atan`*,* `sinh`*,* `tanh` *and* `acosh` *it computes the lower bound as the opposite of the opposite of* $f(x)$ *computed upward, that is* $f(x)$ *computed upward again: the lower bound is the upper one. The trick of the opposite, which works for the arithmetic, would need* $f_{\downarrow}(x) = -f_{\uparrow}(-x)$*, and only for the odd functions, which* `acosh` *is not. Its results of* `asin`*,* `sinh`*,* `tanh` *and* `acosh` *are therefore single doubles, which miss* $f(x)$ *whenever it is not a double. The results depend on the C library: glibc 2.31, under Ubuntu 20.04, missed in 41 % and 27 % of the results,* `asinh`*,* `atanh` *and* `acosh` *included.*

12.6 ITF1788 and the benchmark of Tang et al.

Two test suites written outside GAOL were run on GAOL v5 under the names of the standard, `gaol_ieee1788`, with the default build (SSE2 intervals on x86, the machine of the tables being an Intel i7-1185G7) and the FPU build. The tests of GAOL itself are the subject of Chapter 14.

ITF1788. ITF1788, the Interval Test Framework for IEEE 1788 of Marco Nehmeier, Maximilian Kiesner and Oliver Heimlich (https://github.com/oheim/ITF1788), holds test cases of the operations of IEEE 1788-2015: those of libieeep1788, of C-XSC, of FI_LIB and of MPFI, which Oliver Heimlich converted, and his own for atan2, absRev, powRev1, powRev2, the constructors and the exceptions. They are written in a format of its own, one case per line of its `.itl` files, such as `add [1.0,2.0] [3.0,4.0] = [4.0,6.0];`. ITF1788 converts them into test programs for several libraries; its plugin for GAOL, written for GAOL 4,

calls GAOL by GAOL's own names and leaves out most of the operations GAOL v5 adds. The generator of `doc/compare/itf1788/` reads the `.itl` files itself and writes a C++ program per file, each case being one call of the operation by the name `gaol_ieee1788` gives it.

The suite holds 9542 cases in 19 files. 7415 pass: GAOL v5 gives the result the case expects, the tightest one, the 1347 cases of pow, the 163 of pown, the 172 of mulRevToPair without decorations, the 48 of overlap without decorations, 802 of the 804 cases of powRev1 and powRev2, and 84 of the 94 cases of pownRev with an exponent $p \leqslant 0$ included. No bug of GAOL v5 was found. The 22 failures are of three kinds: 12 results that are valid but not the tightest, from pownRev, 2 at the largest double and 10 with an exponent $p < 0$: 11 of them 1 double beyond the tightest bound and accurate in the sense of IEEE 1788-2015, and one 2 doubles beyond, valid but not accurate, the standard requiring of this operation a valid result and recommending an accurate one; 8 cases where ITF1788 expects a result that is not the tightest, from the reverse functions sinRev, cosRev and tanRev of libieeep1788's tests and powRev2 of Oliver Heimlich's, where GAOL v5's result is narrower than the expected one and encloses the exact result, which mpmath computes; and 2 cases where ITF1788 expects an interval for a literal `[l, u]` with l above u, `[1.0000000000000002,1.0000000000000001]` and the same bounds in hexadecimal, where GAOL v5 compares the numbers exactly and gives the empty set, as the standard asks of an implementation that decides $l \leqslant u$. 418 cases are not provided: eight trigonometric functions of FI_LIB and MPFI that are not in the standard (`cot`, `sec`, `csc`, their hyperbolic versions, `acot` and `acoth`: 403) and the reduction operations (15). 1687 cases are not applicable: they check the decorated intervals and NaI, the ill-formed interval of the standard (11.3), which GAOL v5 does not have (see Chapter 10). The default and the FPU builds give the same result in every case. The deliberate differences of GAOL v5 from the standard do not show here: the hybrid `pow` is `gaol::pow`, while `gaol_ieee1788::pow` is the `pow` of the standard, and the constructors of intervals with an infinite bound give the empty set where the cases expect it.

The benchmark of Tang et al. The benchmark of interval libraries of Xuan Tang, Zachary Ferguson, Teseo Schneider, Denis Zorin, Shoaib Kamil and Daniele Panozzo, *A Cross-Platform Benchmark for Interval Computation Libraries* [15], compares filib (its C version), filib++, Boost.Interval and BIAS on 132 expressions: 28 of the authors (the four operations, `sqrt`, `exp`, `sin` and `cos`, ten arithmetic expressions of 6 to 10 variables and ten random expressions with `sqrt`, `exp`, `sin` and `cos`) and 104 expressions of FPBench, on the inputs of their preconditions. Each expression is evaluated on random inputs, each input variable being the interval of one double; a result is correct when it contains the exact value of the expression at the input, which the benchmark checks by printing each result as a query with exact rationals. The paper finds every library correct on the arithmetic, Boost.Interval wrong on `exp`, `sin` and `cos`, BIAS wrong on one random expression, and filib++'s

`native_switched` mode wrong on four FPBench expressions; GAOL is cited but not run. A patch of `doc/compare/tang2021/` adds GAOL v5 to it, and a checker replaces the Mathematica script of the paper.

On 100 000 inputs of each of the 132 expressions, 13 200 000 results in each of the default and FPU builds of GAOL (CMake Release by Clang 21.1.8, Intel i7-1185G7, Ubuntu 26.04, 14 October 2026), every interval of GAOL encloses the exact value of the expression the program computes, the eight single operations give the tightest interval every time (800 000 of 800 000), and GAOL's intervals are narrower than those of filib's C version, the paper's reference, at all but 1 488 inputs, where they are as wide. The paper's own criterion fails at 9 884 inputs, in six FPBench expressions: each has a decimal constant that is not a double, which the benchmark's query writes as the decimal while the program computes with the double. These are a defect of the benchmark, not of GAOL; no bug of GAOL was found. The two builds give the same tables, line for line. The speed test of the benchmark, 1000 inputs of each expression each evaluated 10 000 times, on GAOL and filib's C version built by Clang 21 in the SSE2 build, on the same machine and day, with a load of about 1.5, is indicative only: GAOL takes less time than filib's C version on 93 of the 132 expressions, 0.36 to 0.66 times its time on the four operations, 0.40 to 0.61 on the ten arithmetic expressions, 0.55 for the median expression, and 107.9 s in all against 131.7 s; it takes more on the elementary functions, 1.54 times on `exp`, 1.77 on `sqrt`, 1.98 on `sin` and 3.98 on `cos`, whose inputs the speed test draws among $\pm(2^{31}-1)$, where GAOL's bounds are the tightest and filib's far wider on large arguments.

12.7 The verdicts on new paths

The sections above measured what GAOL has. This section draws the verdicts on the tracks that were considered for new paths, lists the cheap gains that need no new instruction set, states what a new path has to show before it is added, and ends with the things to watch and what was not measured. Table 12.13 gives the verdicts.

Table 12.13 – The verdicts on the tracks considered for a path of GAOL (from `doc/simd.md`).

Track	Verdict	Why	What would change it
SSE4.1, AVX, AVX2 (a 256-bit path)	no path	A batch of two intervals per register needs an interface of arrays, and nothing makes one interval faster; `ROUNDPD` rounds to an integer, not as an operation	An interface of boxes or arrays of intervals, and a program bound by the computation and not by the memory

Table 12.13, continued

Track	Verdict	Why	What would change it
FMA3	in use (`GAOL_FMA`)	It pays in the elementary functions and the integer powers, none in the basic operations	The exact transformations that would replace the directed rounding are another study (the rounding direction left to nearest)
AVX-512 path	keep; the division and the square root are scalar instructions with the embedded rounding, two for the two bounds	On Intel the 512-bit division takes 16 cycles of throughput against 4 for the 128-bit one (uops.info), and Zen 5 has full-width units; the square root of the path takes two scalar embedded roots (3.75 ns against 6.30 with SSE2), `x / y` two scalar divisions, and so do `x / d` of a double (2.8 ns against 3.7 for $d > 0$, 3.0 to 3.3 for $d < 0$, on an Intel i7-1185G7 with Clang 21), the divisors with a zero (5.5 ns) and the negative integer powers (`pow(x, -3)` 9.1 ns), which the two scalar divisions, in place of the 512-bit one, left within the noise	A processor where the two scalar divisions lose to the 512-bit one (not seen on the i7-1185G7; no other processor was measured, Zen 5 included)
AVX-512VL, AVX10.2 at 128 and 256 bits	nothing to build	No embedded rounding at these widths	A new revision of the Intel manual restoring it (revision 3.0 of AVX10.2 had it)
NEON (arm64)	keep	Done: 1.3 to 2.5 times faster than the FPU intervals on an M2, 3.2 times on a line of 13 operations	Hardware with FPCR.AH set (below)
SVE, SVE2, SME	no path	The same FPCR, and 128-bit vectors on most of the processors	An interface of arrays, on a 256 or 512-bit implementation; a static rounding in the instruction set
ARMv7	no vector path	NEON has no double	none
POWER (VSX), s390x (vector facility)	measure first	Two doubles per register and a write of the control register, as NEON, but no hardware here and no figure for the write	A machine to run the tests and the benchmark on
RISC-V, scalar	measure on a board, then write it	The only other rounding direction by instruction, by inline assembly	A board, and a check that its cores do not slow the static modes

Table 12.13, continued

Track	Verdict	Why	What would change it
RISC-V vector, LoongArch, MIPS, WebAssembly	no path	No static rounding for the vectors, no user, or the rounding to nearest only	An extension with a static rounding; for WebAssembly, a lifting of its limit

12.7.1 Cheap gains that need no new instruction set

The candidates below were seen in the sources and the profiles; the first and the third are done, and none needs a new path.

- **`pow(x, 2)` through `sqr`** (done). `gaol_pown` took 5.2 ns where `sqr` takes 1.3 on the M2 (NEON intervals; 5.2 and 1.8 with the FPU ones, whose `sqr` has no branch on the signs), and the integer powers are about 40 % of Goldstein-Price (6 million calls). It takes `sqr` now, for $n = 2$ in every form of the integer power: [GAOL v5] 1.6 ns with the NEON intervals and 1.9 with the FPU ones, the call into the library accounting for the 0.3 ns of NEON above `sqr` (0.1 with the FPU ones; the medians of 7 passes alternated with the library before the change, `tests/performance.cpp`, an Apple M2, AppleClang 21). The other powers cost 0.3 to 0.5 ns more with the FPU intervals for $n = 3, 4, 5$ and 10 (+5 %), and less with the NEON ones. On 14 million cases (3 million intervals, the special bounds, the empty set and the universe, in the four directions of the calling code and with flush-to-zero set) the bounds of `pow(x, 2)` are the same as before, the sign of a zero lower bound aside, which is that of `sqr(x)` now.

- **`sqrt` on SSE2** is scalar (6.3 ns on the i7-1185G7, 3.75 with the AVX-512 path and the direction preserved, 2.0 on the M2 with NEON); the NEON kernel ports to `_mm_sqrt_pd`. `sqrt_rel` follows.

- **The integer powers on NEON** (done). The kernel of `pown`, in `gaol/core/gaol_interval_neon.cpp`, computes on two lanes with `vfmaq_f64`, as the SSE2 one of `gaol_interval_sse.cpp` does with `__FMA__`, where it was two scalar lanes with `std::fma`; the generic part is in `gaol_interval_core.cpp`, and each kind of intervals defines the operations of its lanes as a class. [GAOL v5] The bounds are the same bit for bit (4 million powers hashed, and `tests/tools/pow/diffpow.py` on 70 500 lines). The first version of the kernel gained only 1.03 to 1.07 times over the two scalar lanes, far below the 1.5 times that was hoped for: `pow(x, 3)` took 5.7 ns where it took 6.1 (5 passes alternated by `.github/scripts/performance-ab.sh`, an Apple M2, AppleClang 21), and 5.84 ns in the session that found the cause. The kernel was not to blame: it is some 8 of the 125 instructions of a call, and its chain of dependent products is no shorter than that of two scalar lanes (NEON

has no vector `fnmsub`: each $a \times b - c$ is a `vnegq` and a `vfmaq`). The rest was the code before it, which Clang made longer: with `-frounding-math` it turns the generic comparisons `vcgeq_f64`, `vcleq_f64` and `vceqq_f64` of the check of the range into scalar ones, 38 instructions where the same masks with `vcageq_f64` and `vcaleq_f64`, on the absolute values, take 20, and it made the permutation of the corners for a negative exponent (`vextq_f64`) a select through a general register. The check is now written with `vcageq_f64` and `vcaleq_f64`, and the permutation keeps an empty `asm` that leaves it as it is (except with Visual C++): a call of `pow(x, 3)` takes 102 instructions where it took 125 (counted with lldb on the M2). Without `GAOL_PRESERVE_ROUNDING`, in independent calls, it takes 4.42 ns where it took 5.84 (1.32 times), the other exponents measured, from -7 to 100, gain 1.11 to 1.33 times (medians of 21 alternations of the old and the new code), and dependent calls gain 1.04 to 1.07 times. With it, the 27 rows of the micro-benchmark of that session (independent and dependent calls, not in the repository) take 0.95 to 0.99 of their time, a call of `pow(x, 3)` taking 36.4 ns where it took 37.2, under a load of 1.55 to 2.88, above the 1.5 that #216 asks for: only the ratios of one alternation count. The preservation itself adds some 31 ns to an independent call and 18 to a dependent one, whose cause was not looked for. All of this was measured on the M2 with AppleClang 21 only, not with GCC, on the Neoverse N2 or with Visual C++ and clang-cl for arm64. Making the corners from the stored register rather than from two stores and a load had not changed the ratio of the first version.

- `mid()` takes 8.5 to 12 ns on the M2 for an addition at 0.9; a two-lane kernel would keep the rare branches.

- **The argument reduction of `sin`, `cos` and `tan`** (the quotients by π and the `floor` and `ceil`) is part of the code of GAOL around CORE-MATH, which is 42 to 66 % of the time of `sin` in the two programs that call it; no profile says how much of that is the reduction.

- **`div_rel` and `%=`** are trees of branches; the NEON intervals have the cases of the function `divide_by_zero_containing()` as a table for the quotient.

- **Not for SIMD.** `midpoint()` (29 ns on the M2), `bisect()`, `chi()` and `hausdorff()` spend their time in changes of the rounding direction, which is the question of computing without changing it; the elementary functions are limited by CORE-MATH, and a vector version would be another algorithm.

12.7.2 What a new path has to show

Before a path is added, with the rules of GAOL, it has to show:

1. its time against the path it replaces, on the processor and the compiler that matter, with the spread, and the operation or the program that needs it;

2. its bounds: the tightest ones, against references computed apart, in the four rounding directions of the calling code, with and without `GAOL_PRESERVE_ROUNDING`, the subnormals (the flush-to-zero modes cleared), the NaN and the infinities, the empty set without an invalid-operation exception, and the sign of a zero bound documented where it differs;

3. the three build systems alike, a job of the continuous integration that tells which path it took (`.github/scripts/check-intervals.sh`, see Section 14.4, page 295), and the processor it needs written with the platforms (see Section 2.1, page 23), with the path it falls back to.

12.7.3 Things to watch

- **FPCR.AH.** With this bit of FEAT_AFP (Armv8.7, present on the Neoverse V3 of Graviton 5 and absent from the M2) set, `FMAX` and `FMAXP` (the `vmaxq_f64` and `vpmaxq_f64` of the NEON kernels) return the second operand for two zeros of different signs, and when either is a NaN, in which case they also raise the invalid-operation exception. GAOL clears FZ and FIZ and not AH. The kernels never give a NaN to a maximum, and the zeros of the maxima of the product differ only in sign. The square root used to depend on the order of the operands of a maximum, of the interval with its left bound in its natural sign and of $\langle +0, +0\rangle$: a compiler that swapped the operands would have turned `sqrt([-0, r])` into the empty set under AH (Apple clang 21 and GCC 14 to its trunk keep the order, but Clang 21 merges `vmaxq_f64(x, y)` and `vmaxq_f64(y, x)` into one instruction: it takes the intrinsic for commutative). It now clears the lanes whose sign bit is set, a negative number or -0, with a mask, as the part of the interval in $[0, +\infty]$: [GAOL v5] the same bits as the maximum with AH clear, on 20 million pairs of bounds that are no NaN and every pair of the special values, and no dependence on AH or on the order of the operands. The Linux and XNU kernels and the glibc, whose sources were read, do not set AH by default; a program that does (an emulator) is outside what GAOL supports. Modelled in software on 25.8 million results, the bounds are those of the kernels, but for the sign of a zero. `tests/rounding_direction.cpp` runs the operations with AH set where the processor retains the bit, and says so where it does not (the M2 and the emulators QEMU 7.2 and 8.2 do not; QEMU 10.0 does, with `qemu-aarch64 -cpu max`, where the test runs on 441 pairs of operands per operation, and fails 189 checks with the operands of the former maximum exchanged, the square root of `[-0, r]` being then a half-NaN interval).

- **The modes that make a result approximate or flush it** are cleared by GAOL where it has a path that reads the control register: FZ and FIZ on 64-bit ARM (FZ alone on 32-bit ARM), FTZ and DAZ on x86 (the AVX-512 path does not read MXCSR, and clears them where a test shows them set; see Section 4.8, page 104). `NI` of FPSCR on POWER (non-IEEE mode,

where results "may be approximate") and `FS` of the FCSR on MIPS (flush denormalized results to zero) are states of the same kind that GAOL does not clear on the FPU intervals of these processors; a path for them would have to.

- **The pragma `FENV_ROUND`, the `_mm256_add_round_pd` of Clang 20 and the 256-bit `{ru-sae}` assembly** compile and are wrong (Section 12.4).
- **The NEON quotient on a Neoverse N2** (the arm64 runners of GitHub, Azure Cobalt 100; the A/B of `performance.yml` of 7 November 2026, the medians of three runs, the base and the head giving the same figures): `x / y` takes 3.96 ns with Clang, 4.04 with clang-cl, 4.49 with GCC and 4.92 with Visual C++, where the FPU intervals take 4.29 with GCC, so that the one operation the NEON intervals do not gain there is the quotient (with GCC `x * y` takes 2.95 ns against 6.37 and `x + y` 1.26 against 1.87). It took 5.36 ns with clang-cl and 5.40 with Visual C++ before `divide_upward()` told the signs of the divisor on general registers (commit c42f95e, 5.35 to 3.96 ns with Clang 18), where the FPU intervals took 4.35 with clang-cl. It was not the vector division: two scalar divisions in its place (`vdiv_f64` on each lane, the same bounds) took 6.17 and 6.30 ns, and the time was in the rest of the kernel, about 150 instructions in `div_bounds()` of which some 55 were vector ones (counted in the object of GCC 12.2, the common path being a part of them). On the Apple M2 (7 November 2026, mains power, three runs of each build alternated) `x / y` takes 2.07 ns with the NEON intervals against 3.20 with the FPU ones, the line of 7 quotients 18.4 ns against 32.4, and `x * y` 1.52 ns: the N2 takes 1.9 times as long for the product, a ratio that the two vector pipes of the N2 against the four of the M2 would explain (not measured: `perf` is not allowed on the runners and the machine).

12.7.4 What was not measured

No measurement of this chapter on x86 (the SSE2 and AVX-512 paths) was made on another processor than an Intel i7-1185G7, for the times of Section 12.5.1 and the divisions of Table 12.13; no path was timed on RISC-V, POWER, s390x, LoongArch, MIPS or WebAssembly (the tests of GAOL run on POWER, s390x and RISC-V under QEMU in the continuous integration, see Section 14.4, page 295): the figures given for these come from uops.info or the references, and the compilations made to study their instruction sets were made on Compiler Explorer, without running the code. The calls were counted on the NEON build only. GCC never ran a program of this study: the behaviour of the intrinsics and of the pragma under it is read from the assembly. The profile has one machine and one compiler, and `sample` is the macOS profiler. The cost of writing FPSCR (32-bit ARM, POWER), FPC (s390x), FCSR and frm is not in the references read, nor is the manual of MIPS. Whether FPCR.AH changes the kernels was not measured on hardware: the M2 has no FEAT_AFP (the test runs with AH set under QEMU 10.0).

12.8 Discussion

This chapter has shown how GAOL v5 computes an interval: as one vector of two doubles, `<-left, right>`, on which the SSE2 and the NEON kernels compute both bounds with one or a few instructions rounded upward, with no branch on the signs, the bounds staying in registers from one operation to the next. What costs is not the arithmetic but the rounding direction: a write of the control register takes about 3 ns on x86 and 4 to 11 ns on an Apple M2, against one or two nanoseconds for the operation, and the choice of GAOL v5 is to check the direction with one comparison and to write the register only when the direction is not upward already. The measurements bear the choice out. GAOL v5 is faster than GAOL 4.2.3 on every line measured and on every operation but the square root, which takes the same time, 1.06 to 4.4 times on x86 and 1.7 to 8.3 times on the M2, and it is the fastest of the six libraries compared on the arithmetic, the integer power and most formulas, with the bounds of libieeep1788, which computes every bound with MPFR, 17 to 135 times faster. Where it is slower, on `sin` and `cos` against filib++ and Solaris Studio, and on the square root, `exp`, `log` and the real power against PROFIL/BIAS, whose bounds are among the widest, it pays for a correct rounding over the whole range that the others do not give: the test of enclosure found no result of GAOL v5 that misses the value of the function, where 20 to 34 % of Boost.Interval's do.

The trade-offs are those of the rounding direction. Preserving it costs no write of the control register on the AVX-512 path, whose instructions carry their own direction, and that is why a build takes that path with `GAOL_PRESERVE_ROUNDING` only: without the preservation the operations take 20 % longer with the path at the median, and the rule relies on one processor, with halved AVX-512 units. On 64-bit ARM, no instruction carries its direction, and preserving it for a program that computes to nearest makes a line of operations 13.0 to 43.5 times slower on the M2. The paths that remain to try are few: the RISC-V scalar instructions are the only other ones with a rounding direction by instruction, and they need a board; POWER and s390x need hardware to measure; the 128 and 256-bit forms of AVX-512 and the vectors of SVE offer no embedded rounding. The cheap gains left open are a vector square root on SSE2, a two-lane `mid()` and the argument reduction of the trigonometric functions. Whether FPCR.AH changes the NEON kernels could not be measured on hardware, the M2 having no FEAT_AFP; the square root was made independent of it in software, and the other kernels differ under it only in the sign of a zero bound.

These measurements rest on the tests, which check that every path gives the same bounds in the four rounding directions of the calling code, with the subnormals, the infinities and the empty set, and that a change does not move the bounds by a double. The next chapter describes what the floating-point numbers and the errors of GAOL look like to a program; the principles and the programs of the tests, and the continuous integration that runs them on every platform, are described in Chapter 14.

Chapter 13

Floating-point numbers, errors and debugging

An interval library rests on the floating-point numbers of its machine. The bounds of the intervals of GAOL are `double` values, and a program that builds an interval bound by bound, or that examines the result of a computation, needs a few operations on these values: the neighbours of a double, the number of doubles between two of them, a minimum and a maximum that tell the two zeros apart. The first part of this chapter describes the constants and the functions on `double` that GAOL defines beside its intervals, the constants that tell a program the version of the headers it was compiled with, and two utility functions on integers.

The second part is about the errors. GAOL reports an error in one of two ways, by throwing an exception or by printing a message and aborting, and the way depends on how the library was built. GAOL v5 throws the exceptions of the standard library, where GAOL 4 had classes of its own, so that a handler that knows nothing of GAOL says what went wrong. [GAOL v5] We describe the mechanism, the macro with which a program adds its own errors, the two exceptions that GAOL throws and the text of their messages, and the warnings. The last part presents the debugging facilities of the Debug builds.

The chapter is organized as follows. Section 13.1 gives the floating-point constants and Section 13.2 the floating-point functions. Section 13.3 describes the version information and Section 13.4 the additional functions. Section 13.5 presents the error handling, the exceptions and the warnings, and Section 13.6 the debugging facilities. Section 13.7 discusses the choices made.

13.1 Floating-point constants

In addition to the constants available through `numeric_limits<double>`, GAOL defines the two macros of the table below, which expand into `double` values.

Constant (`double`)	Value
`GAOL_NAN`	NaN (quiet)
`GAOL_INFINITY`	$+\infty$

Remark 13.1.1 *GAOL 4 also defined the doubles* `two_pi`, `pi`, `half_pi`, `pi_dn`, `pi_up`, `half_pi_dn`, `half_pi_up`, `ln2_dn`, `ln2_up`, `two_power_51` *and* `two_power_52`, *in the namespace of GAOL. A program that opened it with* `using namespace gaol` *found them among its own names, and a* `pi` *of its own was ambiguous. GAOL v5 no longer declares them for the code that uses it, apart from the four doubles around* π *and* $\pi/2$, *which* `gaol` *and* `gaol_ieee1788` *name* `gaol_pi_dn`, `gaol_pi_up`, `gaol_half_pi_dn` *and* `gaol_half_pi_up`. [GAOL v5] *The bounds of* π, $\pi/2$ *and* 2π *are those of* `interval::pi()`, `interval::half_pi()` *and* `interval::two_pi()` *(see Section 5.4, page 125),* `interval::pi().left()` *being* $\downarrow\pi\downarrow$; *the bounds of* $\ln 2$ *are those of* `log(interval(2.0))`.

13.2 Floating-point functions

GAOL provides the following functions on `double` values. The functions that set the rounding direction of the floating-point unit are described with the rounding direction (see Section 4.11, page 115).

`bool` **`feven`**`(double x)`

Returns `true` whenever x is even. The function is not meant for an infinity or a NaN.

Example

```
1 cout << boolalpha << feven(3.0) << ' ' << feven(4.0) << ' ' << feven(3.5);
2 ➥ false true false
```

`double` **`next_float`**`(double x)`

Returns the smallest `double` greater than x, the neighbour x^+ of x; `next_float(GAOL_INFINITY)` and `next_float(DBL_MAX)` are $+\infty$ with every C library.

`double` **`previous_float`**`(double x)`

Returns the greatest `double` smaller than x, the neighbour x^- of x.

`bool` **`is_signed`**`(double x)`

Returns `true` whenever the sign bit of x is set: for the negative numbers and -0. With `using namespace std;` and `using namespace gaol;`, as the examples of this manual assume, the name is ambiguous with the class template `std::is_signed`: the call is written `gaol::is_signed(x)`. No provision is made for a NaN: a program that wants to test for the negative numbers and -0 alone has to test also whether x is a NaN, with `std::isnan()`.

`double minimum(double x, double y)`

Returns the minimum `double` value of x and y. The function is commutative and returns -0 when comparing -0 and $+0$:

$$\begin{cases} \min(x,y) & = \min(y,x), & \forall x \neq \mathrm{NaN}, \forall y \neq \mathrm{NaN} \\ \min(x,\mathrm{NaN}) & = \min(\mathrm{NaN},x) = \mathrm{NaN}, & \forall x \\ \min(-0,0) & = \min(0,-0) = -0 \end{cases}$$

`double maximum(double x,double y)`

Returns the maximum `double` value of x and y. The function is commutative and returns $+0$ when comparing -0 and $+0$:

$$\begin{cases} \max(x,y) & = \max(y,x), & \forall x \neq \mathrm{NaN}, \forall y \neq \mathrm{NaN} \\ \max(x,\mathrm{NaN}) & = \max(\mathrm{NaN},x) = \mathrm{NaN}, & \forall x \\ \max(-0,0) & = \max(0,-0) = 0 \end{cases}$$

`unsigned long long nb_fp_numbers(double a, double b)`

Returns the number of floating-point numbers in the interval $[a, b]$. In particular:

- `nb_fp_numbers(a,next_float(a)) == 2`;
- `nb_fp_numbers(a,a) == 1`.

The two zeros are the same number: `nb_fp_numbers(-0.0,0.0)` is 1, and `nb_fp_numbers(-0.0,1.0)`, `nb_fp_numbers(-1.0,0.0)` and `nb_fp_numbers(0.0,1.0)` are all 4607182418800017409 (GAOL 4 returned 13830554455654793217 for the first two). [GAOL v5]

The precondition is that a be lower than or equal to b.

The result is an `unsigned long long`. GAOL 4 named its type with the macro `ULONGLONGINT`, which GAOL v5 no longer defines, as it prefixes with `GAOL_` the macros of its headers. [GAOL v5]

The function throws a `std::domain_error` exception (see Section 13.5.1, page 266) if either a or b is a NaN or an infinity, or if a is greater than b; where the exceptions are disabled, it calls `gaol_error` and aborts. It returns no number then, where the earlier editions of this manual gave `numeric_limits<unsigned long long>::max()`. [GAOL v5]

Example

```
1 cout << nb_fp_numbers(1.0, next_float(1.0)) << ' '
2      << nb_fp_numbers(1.0, 2.0);
3 ➥ 2 4503599627370497
```

13.3 Version information

The headers of the library define four constants that give the version of GAOL a program was compiled with, from the macros `GAOL_MAJOR_VERSION`, `GAOL_MINOR_VERSION`, `GAOL_MICRO_VERSION` and `GAOL_VERSION` of `gaol/gaol_configuration.h`, the first three of which a program tests with `#if`, `GAOL_VERSION` being the string `"5.0.0"` (see Section 2.10.1, page 56). The versioning scheme is the one of the Apache Software Foundation, described at http://apr.apache.org/versioning.html. The constants are declared in the namespace `gaol_core` and named in `gaol` and `gaol_ieee1788`. [GAOL v5]

`unsigned int version_major` *const*

Major version of the library: 5 for GAOL v5.

`unsigned int version_minor` *const*

Minor version of the library.

`unsigned int version_micro` *const*

Patch version of the library.

`const char *const version` *const*

Version of the library as a string.

Example

```
1 cout << gaol::version;
2 ➥ 5.0.0
```

13.4 Additional functions

The following functions are utility functions, not necessarily related to intervals or to floating-point numbers.

`template <typename T>`
`bool odd(const T& x)`

Returns `true` if x is odd and `false` otherwise. The type T may be any type that provides the operator `&` ("bitwise `and`") with the same semantics as the one of the `int`s.

`template <typename T>`
`bool even(const T& x)`

Returns `true` if x is even and `false` otherwise. The type T may be any type that provides the operator `&` ("bitwise `and`") with the same semantics as the one of the `int`s.

13.5 Error handling

A program that uses GAOL may see an error reported in two different ways:

- **By an exception.** GAOL throws an exception of the standard library, which the program may catch.
- **By a message.** GAOL prints a message on the standard error, and the program aborts.

The exceptions are those of the standard library, declared in `<stdexcept>`. GAOL 4 had classes of its own, `gaol_exception`, `input_format_error`, `invalid_action_error` and `unavailable_feature_error`, which GAOL v5 no longer has (see Section 13.5.1, page 266). [GAOL v5]

The mechanism in use depends on the way the library is configured (Chapter 2). With the option `--enable-exceptions=yes` of `configure`, the default, every error of an operation is reported by an exception; an AVX-512 build started on a processor without these instructions prints a message and aborts in every build. A build where `GAOL_EXCEPTIONS_ENABLED` is 0, obtained with `--disable-exceptions` of `configure`, `-Denable-exception=false` of meson or `-DGAOL_EXCEPTIONS_ENABLED=OFF` of CMake, prints a message on the standard error and aborts instead of throwing. This is a setting of the developers, for the tests of that path. Relying on exceptions is more in the spirit of C++, though it may incur some overhead.

Remark 13.5.1 *In a build that aborts, the reader of strings is an exception to this rule. An error it finds as it computes, a wrong call (*`nth_root(8, 1.5)`*,* `sin(1, 2)`*), an uncertain number as a bound, two different bounds in* `<a, b>` *or a bound of it that is no double make it fail as a syntax error does:* `parse_interval()` *returns* `false`*, and* `gaol_ieee1788::textToInterval()` *the empty set, as IEEE 1788-2015 has it (12.12.7).* `gaol::textToInterval()` *and the operator* `>>` *print a message and abort then, as for any string that is no interval.* [GAOL v5]

Where memory runs out, GAOL throws the `std::bad_alloc` of C++. The reader of strings does too, and frees what it allocated; in GAOL 4, flex ended the program with `exit(2)` when it could not allocate its buffer, and the parser read a stack it could not grow as a string nested too deep. [GAOL v5]

It is up to the user to comply with this mechanism when adding error reporting code to a program. GAOL defines the following macro, to be used whenever one wants to report an error.

`gaol_ERROR(`*excep*`,`*msg*`)` *macro*

The behaviour of the macro depends on the value chosen for the option `--enable-exceptions`. If the exceptions are enabled, an exception of the class *excep* is thrown with the text "`file:line:` *msg*", `file` and `line` being those of the place where the macro is written (`__FILE__`, cut after its last `/` or `\`, and `__LINE__`; `error_text()` of `gaol/gaol_common.h`): the path of the machine that built GAOL is not in the message. The class *excep* derives from `std::exception`, in practice one of `<stdexcept>`; the compiler refuses any other type, by a `static_assert`, in a build that aborts as in one that throws. Otherwise, the program aborts with the message *msg*. [GAOL v5]

Example

```
1 interval x;
2
3 [Code manipulating x]
4
5 if (x.is_empty()) {
6    gaol_ERROR(std::domain_error,"Emptiness of one interval");
7 }
```

The function `gaol_error()` is defined as follows.

`void gaol_error(const char *`*err*`)`
`void gaol_error(const char *`*file*`, int `*line*`, const char *`*err*`)`

Displays a message on the standard error output. The ternary version should be called with the macro `GAOL_FILE_POS` for its first two parameters (see Section 13.5.1, page 266). A null *file* is written `(unknown file)`, and a null *err* is an empty text. [GAOL v5]

13.5.1 Exceptions

GAOL throws the exceptions of the standard library, and none of its own. [GAOL v5] It throws two of them to report an error:

- `std::invalid_argument`, when a string is no interval (see Section 13.5.2, page 268);
- `std::domain_error`, when a function is called with an argument it does not take (see Section 13.5.3, page 268), as `bisect()` is for a ratio outside $(0, 1)$.

Both derive from `std::logic_error`, which derives from `std::exception`. A single handler of `std::logic_error` catches both, but the handler of one leaves the other alone, and none of them catches the `std::bad_alloc` that GAOL throws where memory runs out.

The `what()` of these exceptions is "`file:line:` *explanation*": the name of the source file of GAOL, without its directories, and the line that threw the exception, then what went wrong, as in `gaol_interval.cpp:48: Syntax error in interval initialization: [1, 2`. For the errors that the reader of strings finds in a call (`nth_root(8, 1.5)`, `sin(1, 2)`), the file is that of the grammar, `gaol_interval_parser.ypp`, and the line one of that file, which the directives `#line` of the generated parser give. A handler of `std::exception`, and the message of an exception that nothing catches, say what went wrong, where GAOL 4 gave `std::exception` whatever the error. The text is valid as long as the exception is.

Example 13.5.2 *The program below gives the reader of strings a text that is no interval, and catches the exception as a* `std::exception`*, knowing nothing of GAOL.*

Example

```
try {
  interval x = gaol::textToInterval("[1, 2");
} catch (const std::exception& e) {
  std::cerr << e.what() << '\n';
}
```

The handler writes `gaol_interval.cpp:48: Syntax error in interval initialization: [1, 2` *on the standard error. Had nothing caught the exception, the program would have ended with the same text; with libstdc++, the output is the following.*

Output

```
terminate called after throwing an instance of 'std::invalid_argument'
  what():  gaol_interval.cpp:48: Syntax error in interval initialization: [1, 2
```

`GAOL_FILE_POS` *macro*

Expands into the first two arguments of `gaol_error()` and `gaol_warning()`, the name of the file and the line:

Example

```
if ([some condition]) {
  gaol_error(GAOL_FILE_POS, "No additional information");
}
```

Remark 13.5.3 *What GAOL 4 had is gone: the base class* `gaol_exception`*, with its accessors* `file()`*,* `line()` *and* `explanation()` *and the operator "*`<<`*" that wrote them, and the classes* `input_format_error`*,* `invalid_action_error` *and* `unavailable_feature_error` *of* `gaol/gaol_exceptions.h`*. The classes become those of the standard library:*

- `gaol::input_format_error` *becomes* `std::invalid_argument`;
- `gaol::invalid_action_error` *becomes* `std::domain_error`.

What `explanation()` *gave follows the file and the line in* `what()`. `unavailable_feature_error` *has no replacement:* `atan2()`, *which threw it in GAOL 4, is implemented.* [GAOL v5]

13.5.2 The `std::invalid_argument` exception

The exception `std::invalid_argument` is thrown whenever a program attempts to create an interval from an invalid string. This situation may occur when reading an interval from a stream with the operator `>>`, or when reading a string with `gaol::textToInterval()`, from one string or from two, a null pointer being no string (see Section 9.1, page 183). The example prints what follows the first colon and the blank of `what()`, the explanation, which the file and the line precede.

Example

```
try {
  interval x = textToInterval("sin(1)+");
} catch (const std::invalid_argument& e) {
  string text = e.what();
  cout << text.substr(text.find(": ") + 2);
}
➥ Syntax error in interval initialization: sin(1)+
```

13.5.3 The `std::domain_error` exception

The exception `std::domain_error` is thrown in three cases only: `nb_fp_numbers()` with a NaN or an infinity as a parameter, or with a greater than b (see Section 13.2, page 262), `bisect()` with a ratio outside $(0, 1)$ (see Section 8.3, page 179), and a string that gives a function an argument it does not take, as `"nth_root(8, 1.5)"` does.

13.5.4 Warnings

GAOL writes no warning itself; the following function writes one, as `gaol_error()` writes an error, on the standard error output `std::cerr`, for the code that uses GAOL.

```
void gaol_warning(const char *warn)
void gaol_warning(const char *file, int line, const char *warn)
```

Prints the message *warn* on the standard error output. The second form should be called with the macro `GAOL_FILE_POS` for its first two parameters. A null *file* is written `(unknown file)`, and a null *warn* is an empty text. [GAOL v5]

13.6 Debugging facilities

The debugging facilities described hereunder are available only when GAOL has been configured with them. The Debug build of each build enables them: `-DCMAKE_BUILD_TYPE=Debug` with CMake, the option `--enable-debug` of `configure` (see Section 2.6, page 44), or `--buildtype=debug` with meson; `-DGAOL_DEBUG=ON` with CMake and `-Denable-debug=true` with meson enable them whatever the build type. [GAOL v5] These builds define `GAOL_DEBUGGING` to 1 in `gaol/gaol_configuration.h`, where it is 0 otherwise, as every option of that file, so that it is read with `#if` (`#ifdef` is true of it in every build). With a generator of several configurations (Visual Studio, Xcode or Ninja Multi-Config) and without `-DGAOL_DEBUG=ON`, the header keeps 0 and `GAOL_DEBUGGING` is 1 only for the sources of GAOL in the configuration Debug: `GAOL_DEBUG_VERBOSE` expands to nothing in the program. The Debug builds compile without `NDEBUG` too, so that `assert()` stays active. [GAOL v5]

`int` **`debug_level`**

Global variable that remembers the current value of the debugging level. The variable is set when the library is initialized, by `init()` (see Section 4.10, page 112). It is declared in the namespace `gaol_core`, and named in `gaol` too.

`GAOL_DEBUG_VERBOSE``(`*`lvl`*`,`*`cmd`*`)` *macro*

Executes *cmd* if *lvl* is lower than or equal to the current debugging level, the variable `debug_level` above, which the macro names unqualified: it is used where `gaol` or `gaol_core` is open. The macro was named `GAOL_DEBUG` before GAOL v5; this name is now the option of CMake that builds GAOL for debugging. [GAOL v5]

The macro expands to nothing when the library was not configured with the debugging facilities.

A possible use of this macro is the following:

Example

```
interval x(-10,10);

[Some code]

GAOL_DEBUG_VERBOSE(1,cout << "The value of x is " << x);
x += interval(3.5,4.5);
GAOL_DEBUG_VERBOSE(2,cout << "Now the value of x is " << x);
```

The first message is displayed whenever the debugging level is at least 1, and the second one only if it is at least 2.

13.7 Discussion

This chapter gathered what GAOL v5 offers around its intervals. The constants and the functions on `double` let a program reason on the bounds themselves: the neighbours of a double, the number of doubles an interval holds, where the two zeros count as one, and a minimum and a maximum that order -0 before $+0$ and propagate a NaN. The doubles of GAOL 4 that named π and $\ln 2$ left the namespace, so that a program opening it keeps a `pi` of its own; the constants of the class `interval` replace them. [GAOL v5] Four constants give the version of the headers a program was compiled with.

On the errors, GAOL v5 made one choice: it throws the exceptions of the standard library, `std::invalid_argument` for a string that is no interval and `std::domain_error` for an argument a function does not take, and none of its own. [GAOL v5] A handler that knows nothing of GAOL reads in `what()` the file and the line of GAOL and what went wrong, and a program written for GAOL 4 changes the classes it catches (Chapter 15). The build that aborts instead of throwing remains available in the three builds, as a setting of the developers (`-DGAOL_EXCEPTIONS_ENABLED=OFF` with CMake); in that build, an error that the reader of strings finds makes the reading fail, so that `gaol_ieee1788::textToInterval()` returns the empty set as IEEE 1788-2015 has it, rather than end the program. The debugging facilities belong to the Debug builds, and `GAOL_DEBUG_VERBOSE` expands to nothing elsewhere.

The next chapter describes the tests of GAOL v5 and its continuous integration, which exercise the exceptions of this chapter among the rest of the library, on every platform GAOL supports.

Chapter 14

Tests and continuous integration

An interval library is trusted for one property: the bounds it computes enclose the exact result of each operation, on every machine it is built for. A test that compares GAOL with itself, or with a floating-point library running in the same environment, cannot establish it, since the reference would share the roundings and the rounding direction of what it checks. The tests of GAOL v5 therefore compare the bounds with results computed apart from the library and from the floating-point environment, with integers, with exact rational arithmetic, or with mpmath at 2000 bits of precision, and they run the library in every state a program may leave the processor in.

In this chapter, we describe how GAOL v5 is tested. The first section gives the principles the whole suite follows: exact references, tests that fail without the change they check, the floating-point exceptions, the four rounding directions, the modes that flush the subnormal numbers, the locales, and the unit tests of GAOL 4. The second section lists the programs of `tests/` and what each of them checks. The third section presents the example programs of `examples/`, which check what they print, and the script that compiles the examples of this manual. The fourth section describes the continuous integration: its platforms, the way it measures the speed of a pull request, the configurations it refuses or leaves out, and what these platforms showed. The fifth section lists the scripts of the repository, and the sixth gives the coverage of the tests. The targets that build and run the tests in the three builds are described with the installation (see Section 2.9, page 54).

14.1 The principles of the tests

Built with `WITH_TESTS` (CMake), `--with-tests` (`configure`) or `with-tests` (meson), all off by default, the programs of `tests/` compare the bounds GAOL computes with the exact results of the operations, independently of GAOL and of the floating-point environment. The exact results are computed with integers, or were computed with 2000 bits of precision by mpmath. The tests follow the rounding tests of Codac. They check, among others, that the basic operations are the tightest

enclosures, that the elementary functions are the tightest or within one double of them, that every operation gives the same result whatever the rounding direction the calling code left, that numbers are read exactly and intervals written rounded outward, that the reverse functions are accurate in the sense of IEEE 1788-2015, and that each name of `gaol_ieee1788` is the operation of the standard it names. `test_ibex_codac` is another kind of test: it checks GAOL against the cases of the unit tests that IBEX and Codac run on their own intervals, which are GAOL's. `make test` runs the tests, and `make check` runs the examples of `examples/` too (see Section 2.9, page 54).

The suite rests on the following principles.

- **References computed apart.** A reference value is never a result of GAOL, nor of the C library running in the same rounding direction. It is an integer, an exact rational, or a value computed by mpmath with 2000 bits (400 to 7000 for some tables) and written into a header that the test includes: `elementary_values.h`, `reverse_values.h`, `atan2_rel_values.h`, `pow_rel_values.h`, `compoundm1_values.h` and `extended_precision_values.h` are generated by the scripts `*_values.py` of `tests/`, run by hand when a table changes (see Section 14.5, page 305). Where a function is correctly rounded by CORE-MATH, the test computes its tightest bounds by calling CORE-MATH itself in the downward and the upward directions, which it sets apart from GAOL.

- **Tests that fail without the fix.** Each check written with a change of GAOL v5 was run against the sources before the change, and the documentation of the tests records how many of its checks fail there: 10 900 of the checks under the modes that flush the subnormal numbers, and, with the SSE2 intervals, 45 of the 48 calls that compared the NaN bounds of an empty operand, which died in a child process that enabled the exception, and 25 of the 45 checks of the operations computed before `main()`. A check that passes with and without the change proves nothing, and the mutation tools of `tests/tools/` build GAOL with a deliberate error that the tests have to catch (see Section 14.5, page 305).

- **The floating-point exceptions.** No operation of GAOL may raise the invalid-operation, the division-by-zero or the overflow flag for a result without an infinite bound, nor lose a flag or a mask the program set (see Section 4.9, page 107). With glibc, each operation checked is also computed in a child process that enabled the exception with `feenableexcept()`, which must not die on `SIGFPE`: the comparison of the NaN bounds of an empty interval once did. Where the processor does not trap, or keeps no flag, the test says so and skips that part.

- **The four rounding directions.** The bounds must not depend on the rounding direction of the calling code: each operation is called with the direction

set upward, to nearest, downward and toward zero (and on x86 with the x87 and the SSE directions differing, six states), and has to give the results it gives when called rounding upward, and to leave the direction upward, or as it found it with `GAOL_PRESERVE_ROUNDING` (see Section 4.1, page 88).

- **The flush-to-zero modes.** Each mode that flushes the subnormal numbers to zero that the processor honours (flush-to-zero and denormals-are-zero on x86, FZ and FIZ on ARM, see Section 4.8, page 104) is set just before an operation, alone and with the others: the operation has to give what it gives with the modes cleared, and to leave them cleared, or set back with `GAOL_PRESERVE_ROUNDING`. A mode the processor keeps without honouring it, as an emulator may, is named and skipped.

- **The locales.** Numbers have to be read as in the C locale, and written back bit for bit, under a locale writing a decimal comma (`fr_FR.UTF-8`, `de_DE.UTF-8`, `French_France.1252`) where one is installed, and the names of the functions have to be read in any case of letters under the Turkish locales of glibc, where `std::tolower('I')` is not `i`. The reading once never ended under a comma locale, and the test, which ctest would otherwise let run with no limit, fails after 5 minutes should it hang again.

- **The unit tests of GAOL 4.** The tests of `check/`, which checked the results of the operations on chosen intervals with CppUnit, are in `tests/` and run with the checks of `unit_tests.h`: CppUnit is needed no more. [GAOL v5] Each suite keeps its class and its tests, a check is named after its test and describes a failure by its line, its expression and the values compared, and a failure no longer ends its test.

Remark 14.1.1 *Under the Debug C runtime of Visual C++, a failed assertion of the runtime or of the checked iterators of its library, and an invalid parameter, are written to the standard error, where the test goes on and fails (*`tests/gaol_tests.h`*). The runtime showed them in a dialog box, which nobody closes on a machine of the continuous integration, and the test hung without output until ctest stopped it after 300 s. That runtime reports such an assertion when it writes a subnormal number under denormals-are-zero, then writes 0:* `operator<<` *clears that mode before it writes, which* `numbers` *checks there too.* [GAOL v5]

Remark 14.1.2 *Built without exceptions (*`--disable-exceptions` *of* `configure`*,* `-Denable-exception=false` *of meson), where an error of GAOL ends the program, the tests leave out the checks of an exception and of a text the reader refuses.* [GAOL v5]

`tests/find_package` builds some of the same tests with an installed GAOL, and `.github/scripts/tests.sh` with a GAOL installed by `configure` or meson. `tests/fetch_content` builds them with a GAOL brought into a project by `FetchContent`, and `tests/meson_subproject` with a GAOL brought in as a meson

subproject, [GAOL v5] which meson refused while `meson.build` called `add_global_arguments()`. Its test `flags` checks that `gaol_dep` gives the compilation the flags of the `Cflags` of `gaol.pc`, without which the tests pass all the same, and the link the options of its `Libs` (`-mno-daz-ftz`, where the compiler takes it); its test `release` checks that GAOL is built in release in the default build type of meson, debug: `tests/release_flags.py` checks that the sources of GAOL and CORE-MATH are compiled with `-O3` and `NDEBUG`, and that `GAOL_DEBUGGING` is 0. With `--debug`, it checks the other way round that `GAOL_DEBUG` of CMake and `enable-debug` of meson build GAOL and CORE-MATH for debugging, GAOL built alone and brought into another project.

14.2 The programs of the test suite

Table 14.1 lists the programs of `tests/` and what each of them checks. The paragraphs that follow give, for each group, the cases and the numbers of checks worth knowing, and the platforms where a check applies.

Table 14.1 – The programs of the test suite

Program	What it checks
`arithmetic`	The basic operations, squares, square roots, integer powers, `fma` and the cancellative operations are the tightest enclosures at every magnitude, from every rounding direction, and the n-th roots are within 2 doubles of them (4 for an inverse root)
`elementary`	The elementary functions at doubles, at intervals and at intervals whose images are known exactly; `pow`, `atan2`, `erf`, `erfc`, the integer functions
`rounding_direction`	The same bounds from every rounding direction and flush mode; the direction left upward or preserved; the control registers; the flags and the masks kept; no `SIGFPE` on an empty operand
`fast_math_link`	A program linked with `-ffast-math`: the flush modes that `crtfastmath.o` sets
`automatic_cleanup`	(Linux) The rounding direction set back to nearest after `main()`
`static_initialization`	About 40 operations computed before `main()` give their bounds of `main()`, bit for bit
`test_ibex_codac`	The cases of the unit tests of IBEX and Codac, against the exact results: 3905 checks
`numbers`	Reading and writing: the tightest reading of every text, the literals of IEEE 1788-2015, the bases 16, 8 and 2, the five output formats, the flags of the streams, the locales, the subnormals under the flush modes

Table 14.1, continued

Program	What it checks
`other_functions`	Midpoints, widths, radii, distances, splitting, `max`, `min`, the tools of interval algorithms, the comparisons of IEEE 1788-2015, the reverse functions of `max`, `min`, `sign` and `floor`
`ieee1788`	`gaol_ieee1788` as a program uses it: `pow`, `pown`, `rootn`, `pownRev`, `overlap`, `intervalToText`, `mulRevToPair`
`core_math`	The bounds against CORE-MATH called in the rounding direction in effect; the `fma()` and `round()` of the C library, and `gaol_ldexp()`; the accurate paths of `sin` and `cos`
`compoundm1`	$(1+x)^y - 1$ from the bottom up: the 256-bit integers, the balls, the fast phase, 5 854 pairs and 850 boxes
`expressions`	The lexer, the parser and the evaluation of the strings: every node, every wrong string, the exceptions, the deep chains, the allocations that fail
`u128`	The two 64-bit halves of the 128-bit integer against the native type of the compiler
`extended_precision`	The arguments the x87 unit rounds wrongly: fails when GAOL is built on it
`reverse`	The relational functions against the reverse functions of IEEE 1788-2015: valid and accurate; `atan2_rel_x`, `atan2_rel_y`, `pow_rel`, `pow_exponent_rel`
`debugging`	The headers compiled with `GAOL_DEBUGGING`, and `GAOL_DEBUG_VERBOSE` at the level given to `gaol::init()`
`version_file`	(CMake) The reading of `VERSION.txt`: byte order mark, line ends of Windows, UTF-16, NUL bytes
`refused_finite_math_only`, `refused_fast_math`, `refused_positive`; `refused_options.sh` (autotools); `refused_options` (meson)	The compilation refused with `-ffinite-math-only` and `-ffast-math`, accepted without; `refused_options.sh` and `refused_options` check as well the three cases of the tests `refused_aligned_new*`
`refused_aligned_new`, `refused_aligned_new_cxx17`, `refused_aligned_new_positive` (CMake)	The SSE2 intervals on 32-bit Windows refused without `-faligned-new=8`
`fp_strict`	(a project of its own, which the Visual Studio jobs configure) Visual C++ and clang-cl refused without `/fp:strict`

Table 14.1, continued

Program	What it checks
`cpack_stale_configure`	(CMake) The warning of CPack when the committed `configure` was generated for another version
`automatic_configuration`	(CMake) The configuration GAOL decides by itself, and the settings of the developers
`arithmetic_operators`, `assignment`, `constants`, `constructor`, `float_functions`, `generic_functions`, `input_output`, `interval_functions`, `misc`, `non_arithmetic`, `relations`, `reverse_mappings`, `trigonometric`	The unit tests of GAOL 4: the results of the operations on chosen intervals; `misc` checks the names of the namespaces and the alignment of the intervals
`intervalf`, `interval2f`	The intervals of floats, where a developer compiles them; skipped otherwise
`nodiscard_discard_cxx11`, `..._cxx14`, `..._cxx17`; `nodiscard_used_cxx11`, `..._cxx14`, `..._cxx17`	A result of GAOL thrown away fails the compilation, in the three standards; a result used compiles
`gaol_performance`	(not a test) The time per operation of GAOL's arithmetic and elementary functions

14.2.1 The arithmetic and the elementary functions

`arithmetic` checks, on doubles and intervals of every magnitude, subnormal doubles and overflows included, that sums, differences, products, quotients, relational divisions, squares, inverses, `abs`, `min`, `max`, `&` and `|` are the tightest enclosures, and so are square roots and integer powers of either sign: [GAOL v5] of doubles for n from 2 to 9, 16 and 31 to 33 and for -1 to -4, -7, -16 and -33, of intervals for n from 2 to 5, 12 and 31 and for -1 to -3, -6 and -17, x^n beyond the largest double or subnormal included, and for $n < 0$ the limits where x holds 0. The suite once allowed up to $2n$ doubles beyond the tightest bounds for $n > 0$, and 8 for $n < 0$. Every interval made of 15 special bounds ($-\infty$, the opposite of the largest double, -2.5, -1, the opposites of the smallest normal and of the smallest subnormal number, -0, $+0$, the smallest subnormal and normal numbers, 0.1, 1, 3, the largest double, $+\infty$), the empty set and the universe, taken two by two, have to give with `+`, `-`, `*`, `/`, `%`, their compound forms (the interval itself as the operand included), with a double on either side where an interval is a point, and for `sqr`, `sqrt` and `inverse`, the bounds of a reference of plain doubles: the least of the corners rounded downward and the greatest rounded upward by the processor, a zero bound times an infinite

one counting as 0. An infinite or NaN double gives the empty set, no operation gives an unexpected empty set, and the bounds do not depend on the rounding direction of the calling code, set to each of the four directions before each operation. Inexact operations on literals the compiler sees (`[0.1] + [0.2]`, `[1] / [3]`, `sqrt([2])`) have to be the tightest enclosures whatever the direction: an operation computing to nearest fails them, and so would a compiler that folded the vectors of the library at compile time, which none does with the `-frounding-math` that `gaol::gaol` gives. The sign of three zero bounds the documentation states, that of `[-1, 0]*[-1, 0]` (SSE2 and NEON intervals only), `sqr([-1, 2])` and `[0, 1]/[-2, -1]`, has to be the one of the build: it tells a tie of the maxima of the products, which no other check did, the others comparing with `==`, which does not see it.

The powers are checked further. x^n has to be the tightest over 4 000 intervals near the ends of the doubles, where the products were once rounded outward and the SSE2 and the FPU intervals gave other bounds for half of them; 8 679 powers that are doubles have to be exact, both bounds (an error bound computed beforehand made `pow([2, 3], 3)` $[8 - 2^{-50}, 27 + 2^{-48}]$). x^2 has to be `sqr(x)`, bit for bit, the sign of a zero bound included, in each of nine forms of the integer power with the exponent 2, over the grid of special bounds and 90 000 random intervals, each with the calling code in the four rounding directions and, for the first 3 000 of each kind, with each flush mode: 4 of the 9 forms fail without one call of `sqr()`, 5 without another. $(-c)^n$ has to be c^n or $-c^n$ at 1 088 points near 1, 1.5 and 3 and 12 736 more. The powers for n from $2^{24} + 1$ to $2^{32} - 1$ have to be the tightest against mpmath: the lower bound was 1962 doubles below the tightest for `pow([1.0000001], 2^32 - 1)`; the same for 46 negative n from -2^{31} to -2^{24}, 44 of which fail without the second-order term of the correction. The n-th roots of x below 2^{-1020}, subnormal included, have to be within 2 doubles of the tightest bounds (4 for an inverse root) for $n = 4$, 5, 33, 65, 129 and 1025 against exact arithmetic and up to $2^{32} - 1$ against mpmath: `nth_root([2^-1074], 33)` was 1983 doubles below. `fma(x, y, z)` has to be the tightest over random boxes, against `std::fma` called downward and upward directly: GCC had folded the negations GAOL rounds its lower bound with. `cancel_minus` and `cancel_plus` have to be the tightest, and `y + cancel_minus(x, y)` to enclose x, over intervals of the same width and one double wider, and at the corners where the differences of the bounds are within a double of each other or beyond the largest double: GCC at `-O3` computed the terms of their TwoSum after the direction was set back to upward, where it is no longer exact. [GAOL v5]

`elementary` checks `exp`, `log`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`, `sqrt` and `pow` at doubles, at intervals, and at intervals whose images are known exactly: extrema, poles, domains, `log` of an interval holding no positive number and `atanh` of one with no point of $(-1, 1)$ being empty, `exp(0)`, `log(1)` and 1^y exact. The functions have to be the tightest where their value is 0, 1, $\pm\pi/4$, $\pm\pi/2$ or π, and `cosh`, `sinh` and `tanh` beyond the largest double and near 1. `pow(x, y)` has to be within one double of the tightest bounds at points, `pow([2], [1023.5])` among them, and over 385 boxes of bases and exponents in every position

about the base 1 and the exponent 0, with the power itself as lower bound where it is a double: `pow([4], 0.5)` is `[2]`. `atan2` has to be within one double over 324 boxes about the axes, the tightest over the boxes with infinite bounds or on an axis, $[-\pi, \pi]$ across the half-line $y = 0$, $x < 0$, and empty at $(0, 0)$. `sin`, `cos` and `tan` have to be within one double at every magnitude, `sin([1e-10])` and `cos([2^60])` included, and over 649 intervals next to their extrema and poles; `tan([-M_PI_2, M_PI_2])` has to be the tightest, $\pm 1.63 \cdot 10^{16}$, its width being the double below π and holding no pole, and `tan` within one double over 158 intervals whose bounds are next to two consecutive poles. [GAOL v5] The integer functions `sign`, `trunc`, `round_ties_to_even` and `round_ties_to_away` have to be exact whatever the rounding direction, which `std::nearbyint` and `std::rint` would not be, over the halfway values, the whole numbers, their neighbours and the magnitudes beyond 2^{52}. `erf` and `erfc` have to be the tightest against mpmath at 409 doubles (tiny and subnormal arguments, changes of binade, the arguments past which $\mathrm{erf}(x)$ is within 2^{-53} of ± 1 and $\mathrm{erfc}(x)$ is subnormal, and hard cases of the `erfc.c` of CORE-MATH) and over the intervals between them. [GAOL v5] The values are in `elementary_values.h`, which `elementary_values.py` generates.

14.2.2 The rounding direction and the floating-point environment

`rounding_direction` calls about 110 operations of GAOL's interface with the rounding direction upward, to nearest, downward and toward zero (and on x86 with the x87 and the SSE directions differing): they have to give the results they give rounding upward, and to leave the direction upward, or as they found it with `GAOL_PRESERVE_ROUNDING`; among them the roots of negative order, the integer powers of large exponents and `modulo_k_pi()` next to multiples of π, which compute after one check of the direction. [GAOL v5] Products and sums have to be the tightest in a loop that changes the direction before each of them. The 30 forms of `+ - * / %`, `sqr`, `sqrt` and `inverse` with intervals, with a double and as compound assignments have to be the tightest enclosures on random operands, in a loop that changes the rounding direction before each of them; on ARM they are run 2^{16} times each, in the four directions in turn, right after the program wrote the control register itself. `pow` with a subnormal result, which CORE-MATH rounds in the direction `fegetround()` gives, has to be the tightest in each state, the x87 unit to nearest and the SSE instructions upward included: the `fegetround()` of glibc reads the x87 unit, and the upper bound was below the exact value. [GAOL v5]

The control registers of the processor are checked apart from the library (see Section 4.1, page 88). What GAOL writes to the control word of the x87 unit and to MXCSR on a 32-bit x86 processor is checked on every platform, with two variables standing for the registers, in the 64 states of the two rounding fields and the two flush modes, with other bits that must not change and exception flags raised meanwhile that must not be lost: only the registers that differ are written, and each is read

once; on such a processor, the registers themselves are checked in the states it can run. The SSE2 operations with `GAOL_PRESERVE_ROUNDING`, which keep MXCSR alone, are checked in its 16 states: MXCSR is written once when the direction is not upward or a mode is set, and never read or written otherwise. On x86-64, `get_rounding()`, `set_rounding()` and `set_rounding_and_flush_modes()` change neither the exception flags nor any other bit, and the type through which the library reads and writes the two registers has to read the words the test reads and write the words it is given, with four sets of the other bits, so that a read that drops the flags or a write that forces the masks is seen. On 64-bit ARM, FPCR is read and written with `mrs` and `msr`, and its bits other than the direction and the flush modes (the default-NaN mode, FZ16, the trap-enable bits, and the AH bit of Armv8.7 where the processor retains it) have to be the program's after each operation. [GAOL v5] With mingw-w64 on x86-64, the control word of the x87 unit is read with `fnstcw` and is part of the state compared, since the `fegetround()` of mingw-w64 13 and later joins the fields of the two units and its `fesetround()` clears the six flags of MXCSR. Every operation has to keep the exception flags the program raised (the five of IEEE 754, and on x86 the six of the SSE control register) and the masks, from each direction, and after `gaol::cleanup()` and `gaol::restore_rounding()`: with `GAOL_PRESERVE_ROUNDING`, the SSE2 operations masked the exceptions again and cleared the flags. [GAOL v5]

The empty set and the special values must raise nothing (see Section 4.9, page 107). `is_empty()` has to be true for six empty sets and false for three intervals, and to raise no flag; with glibc, each is also told empty in a child process that enabled the invalid-operation exception, which must not die: the comparison of the NaN bounds with `<=` did. The operations of the interface with an empty operand on either side, about 220 calls in 66 checks, among them one for each of the 48 calls that compared the NaN bounds and five choices a program makes with `is_empty()` that GCC for 32-bit ARM and for POWER9 made signaling, have to give the result of the empty set, raise no flag and not die in the child process: 45 of these 48 calls died with the SSE2 intervals, and all 48 with the FPU ones. [GAOL v5] So do 18 operations given a NaN double, the products of intervals with bounds in $\{-\infty, -2, -0, 0, 3, +\infty\}$, and the relational divisions of intervals with bounds in $\{-\infty, -2, -0, 0, 4, +\infty\}$ (the SSE2 intervals divided $+\infty$ by $+\infty$, or a bound by a zero one, in the half of a register they did not keep), 13 powers with an exponent of extreme magnitude, 10 powers next to the largest double, where CORE-MATH's `pow` computed $\infty - \infty$, and 65 loops of 64 relations and 4 loops that count the pairs for which a relation holds, which a compiler vectorizes or if-converts: GCC 9.4 and 13 at `-O3` with `-mfma`, GCC 13 to 15 for 32-bit x86, GCC 14 for POWER8 and GCC 11 to 15 for 64-bit ARM made some of them raise the exception with the FPU intervals, and Clang 18 and 20 at `-O3 -march=native` without `-frounding-math`, so that the teeth of these loops depend on the processor the code is compiled for. The midpoints of five intervals with a bound of 2^{1023} or more have to be right and raise no overflow flag, the sum of the bounds having overflowed; 36 calls of

functions that computed beyond the largest double on the way (`sqrt()`, `nth_root()`, `atan()`, `sin()`, `pow()`, the relational functions) are run on 20 000 draws of operands each, and a result without an infinite bound has to come with neither the overflow nor the division-by-zero flag; [GAOL v5] 27 cases such as `sqrt([1, DBL_MAX])`, `tan([DBL_MAX])` and `pow([2^-1074], [1.5 2^1014])` have to give their tightest bounds, computed apart, with neither flag, and the same bounds in a child process that enabled both exceptions.

With a mode that flushes the subnormal numbers set, and the direction upward or not, products, sums, differences, quotients, squares and `exp` with a subnormal operand or result have to be the tightest, computed apart with exact rationals (and mpmath for `exp(-740)`), and the modes have to be cleared after the operation, or restored with `GAOL_PRESERVE_ROUNDING`: a program linked with `-Ofast` gets them from `crtfastmath.o`, and `[1e-300] * [1e-20]` was `[0, 0]`. [GAOL v5] Then each of the 96 operations that check the direction, called with each mode set just before it and each direction, has to give what it gives with the modes cleared and leave them cleared (with GCC 13 at `-O3`, `asinpi` of $[100 \cdot 2^{-1074}, 100 \cdot 2^{-1074}]$ was above the exact value and `sqrt([-1e-310, 4])` the empty set), and each of the 51 functions that make no check has to leave the modes as it found them, its doubles compared by their bits; `midpoint()`, `split()`, `chi()` and the writing of an interval, which make the check, have to clear them: with the sources before that change, 10 900 of these checks fail. [GAOL v5] `bisect(ratio)` and `is_bisectable()` have to give, under each mode and direction, what the bits of the bounds give, told by integers: 2280 of these checks fail with the SSE2 intervals before the change, and 4008 with `GAOL_PRESERVE_ROUNDING`. The AVX-512 path is taken whole where the machine building has the instructions: its bounds are those of the SSE2 path bit for bit, but for the sign of a zero bound of `x * d` where the zero bounds of x differ in sign, and it leaves the direction as the program set it (see Section 4.4, page 94). [GAOL v5] `gaol::cleanup()` has to set back the direction the first `gaol::init()` found, although a static object set it upward before `main()`, and `gaol::restore_rounding()` to set it to nearest again as many times as it is called.

Three programs complete it. `fast_math_link` is linked with `-ffast-math` (its link only, with GCC and Clang), which links `crtfastmath.o` and the flush modes it sets, unless `-mno-daz-ftz` keeps it out: where the build gives the link that option, the modes have to be clear when `main()` starts; otherwise the test sets them, as a plug-in built with `-Ofast` does, and products, a difference and `exp` with subnormal operands have to be the tightest and the modes cleared after them. [GAOL v5] `automatic_cleanup` (Linux only) leaves the rounding direction downward at the end of `main()`, and GAOL's automatic cleanup has to set it back to nearest, or to leave it downward with `GAOL_PRESERVE_ROUNDING`: the direction is read by a function that a constructor of priority 101 registers with `std::atexit()`, before GAOL initializes itself. `static_initialization` computes about 40 operations before `main()`, in the initialization of a static object: the constants (`pi()`, `one()`, `universe()`), products and quotients, the elementary functions and the reader of

strings have to give the intervals they give in `main()`, bit for bit. The static library of the CMake build is initialized after the program, except with MinGW-w64, and GAOL's constants were computed by its dynamic initialization: 25 of the 45 checks failed with the SSE2 intervals, 24 with the others, `pi()` giving `[-0, 0]`. [GAOL v5]

14.2.3 Reading and writing

`numbers` checks that `textToInterval("0.1")` is the tightest interval enclosing the number read, and the number itself when it is a double; that the constants are the tightest enclosures of π, 2π and $\pi/2$; that no `const char*` nor `nullptr` converts to an interval, the constructor from one string being gone; [GAOL v5] that the literals of IEEE 1788-2015 (`[entire]`, `[ ]`, `[1,]`, `3.56?1`, hexadecimal numbers) are read as the tightest intervals in any case of letters, and the numbers in base 16, 8 and 2 (`0x10`, `0xff.8`, `0o17`, `0b1.1p3`) as the tightest intervals enclosing what their digits write, computed from the digits by the test, beyond 2^{64}, around 2^{53} and beyond the doubles, while `0x`, `0o8` and `0b2` are refused (see Section 9.1, page 183). Expressions GAOL cannot read or compute have to throw, and, built with LeakSanitizer, the parser has to free the nodes of all of them. The intervals written in decimal in the general, scientific and fixed formats with 1 to 20 digits have to enclose the intervals, each bound less than one unit of its last digit away, near the powers of ten too; read back, they have to enclose the intervals written. The format of the agreeing digits writes the shared digits only for finite, nonzero bounds of the same sign, with as many digits before the point and the same exponent (`-1.25~[67, 0]`), and the bounds otherwise. A point interval is written `[a]` when the number is the point itself, and `[l, r]` otherwise, neither bound being the point (`[0.1, 0.1000000000000001]` for `interval(0.1)`); the largest doubles and the subnormals, written with all their digits (309 digits, 1074 after the point), are read back as the point. [GAOL v5] In hexadecimal, the bounds have to be written in the form of IEEE 1788-2015 (13.4.1) and read back as the same set, which is the recovery requirement of 13.4. `operator<<` has to leave the precision of the stream as it was, `std::setw` to pad the whole interval, and the texts written under the flags of the stream and a locale of its own have to be those that `operator<<` wrote when it still wrote into a stream of its own, before the text was made in a character string, except that the bounds format writes in the C locale and the width format writes the radius without a sign under `showpos`: 33 cases of four formats. The width and center formats have to write the `midpoint()` and the `rad()` of the standard, the radius rounded upward so that midpoint plus or minus it contains the interval, and never 0 for an interval that is not a point: GAOL wrote $(l + r)/2$ and $(r - l)/2$ rounded to nearest, which do not contain the interval. [GAOL v5] The empty set has to be `[empty]` in the five formats.

`while (in >> x)` has to stop at the end of the input with `failbit` set and the interval unchanged, and a line that is no interval has to set `failbit` and throw `std::invalid_argument`; blank lines have to be skipped, under `std::noskipws` too, and the stream tied to the input flushed before the first character is read, so

that a prompt shows before the user types. [GAOL v5] Numbers with a million zeros after their point and an exponent of 7 digits have to be read exactly: the exponent was cut at 100000. Under a locale writing a decimal comma, numbers of 5000 to 20000 characters, in decimal, in hexadecimal and in the uncertain form, have to be read as the tightest enclosures, known without reading them, and the number of 20000 characters has to take at most 10 times the time it takes under the C locale, and 50 ms more: the reader read the text again for each of its 125 comparisons or so, in a time quadratic in its length, and took 50 times as long. [GAOL v5] The cache variable `GAOL_NUMBERS_TIMEOUT` gives the test more than its 5 minutes on a slower build (600 s with GCC and the sanitizers on macOS x86_64). With each flush mode the processor honours, the numbers from 0 to the least normal double have to be read as the tightest intervals enclosing them, and the doubles among them as themselves: the reader compared each number with the doubles around it as doubles, and denormals-are-zero reads a subnormal as 0, so that `1e-310` was read as the interval from the greatest subnormal to the least normal double, which does not enclose it. [GAOL v5] Under such a mode, `<a, b>` of two different subnormals has to be refused, intervals of subnormals written with 16 digits have to be read back as enclosures once the mode is restored, and the text has to be the one written without the mode, `operator<<` clearing it before it writes. [GAOL v5]

14.2.4 The functions, the standard and the reverse functions

`other_functions` checks the midpoints (of subnormal bounds too), the widths, the radii (that of the point zero, like its width, being +0 whatever the signs of its bounds), the magnitudes and mignitudes, the Hausdorff distances (of intervals with infinite bounds too), `nb_fp_numbers()` across the two zeros, the splitting, the integer parts, and `max(x, y)` and `min(x, y)` of every pair of intervals made of special bounds, with the bits of the zeros and the subnormals, with the flush modes set or not; with the SSE2 and NEON intervals, every pair of stored lanes made of special doubles has to give the bits of the formula of the other kinds, and the quotients of the NEON intervals with a NaN in a lane of an operand the bits of `interval::emptyset()`. The tools of interval algorithms, `width_enclosure()`, `inflate(r)` and `interval::midrad(m, r)`, have to be the tightest enclosures of the exact width, of $[a - r, b + r]$ and of $[m - r, m + r]$, computed with exact rationals; `bisect(ratio)` has to give two parts that cover the interval exactly, cut strictly inside where `is_bisectable()`, for ratios from 10^{-9} to $1 - 10^{-9}$, and to raise no overflow exception for finite bounds next to the largest double, `bisect(0.5)` being `split()`, and to throw `std::domain_error` for a ratio outside $(0, 1)$ or NaN. [GAOL v5] The comparisons of IEEE 1788-2015 have to give the values of its Tables 10.3 and 10.4, `==` and `!=` must not compile on intervals, and each name of `gaol_ieee1788` has to be the operation of the standard it names, which a wrong translation would not show at compilation: the eight comparisons over 20 000 pairs, `inf`, `sup` and the numeric functions against Table 10.2, the reverse functions with the arguments

in the order of the standard, `mulRev(b, c, x)` being `div_rel(c, b, x)`, and `pow` against the pow of Table 9.1, which GAOL's own `pow` is not for a negative base (see Section 3.5, page 70). The reverse functions of `max`, `min`, `sign` and `floor` [GAOL v5] have to be the hulls of their sets, read from the definitions of the functions apart from the code, on every interval with bounds among $-\infty$, -2, -1.5, ..., 2, $+\infty$ and the two zeros, on 66 cases written out, and on 20 000 random intervals of every magnitude, where every sampled double the definition puts in the set has to be in the result.

`ieee1788` uses `gaol_ieee1788` as a program does, under `using namespace gaol_ieee1788;` alone: every name of the standard it provides is called unqualified, which compiles only if none is ambiguous with a function of `gaol_core` (Chapter 10). `pow` has to be the standard's with an interval, an `int` or a `double` exponent, and `pown` and `gaol::pow` the integer power for an exponent of any integer type: [GAOL v5] `pown([2], 3000000000u)` is the overflowing power, which the conversion of the exponent to a negative `int` made a tiny one, and an exponent beyond the long longs or of 128 bits gives the tightest bounds of mpmath where the power is a double and within three doubles of them otherwise. `rootn()`, `gaol::nth_root()` and `pownRev()` take an order of any integer type, and `pownRev(c, x, p)` for $p = 0$ and $p < 0$ has to give the exact roots of the reciprocals of c where they are doubles, with no solution for the part of x in the gap of c. `overlap(a, b)` [GAOL v5] has to be the state the set specification of Table 10.7 gives, whose quantifiers the test reads over a finite model of the real line, on 31 449 pairs: exactly one specification holds for two nonempty intervals, but for meets and metBy with a point interval on a bound of the other one, which the standard gives to starts, finishes, finishedBy or startedBy; `disjoint`, `equal` and `subset` have to be the masks of states 0xF001, 0x8040 and 0xC3C0 of the standard. The bounds of `gaol_ieee1788::pow` and `gaol::pow` on 117 boxes, each reaching a branch of the pow of Table 9.1, have to be the bounds the table gives, checked against the exact power computed with 500 bits by mpmath (`tests/tools/pow/checkrows.py`). `intervalToText(x)` has to be a literal of the standard with the digits of the precision of the intervals, whatever the global format and the locale, which a grammar of the literals of Tables 9.5 and 12.2 checks, and `textToInterval` has to read it back as an interval containing x: under a comma locale the literal `[a]` wrote `[-2,5]`, read as $[-2, 5]$. [GAOL v5] `mulRevToPair(b, c)` has to be the pair of 10.5.5, the part below first, tight on cases whose parts have exact bounds, and every $x = k/4$ of the set on every b and c with integer bounds in $[-3, 3]$ has to be in one of the two parts. [GAOL v5]

`reverse` checks the relational functions against the reverse functions of IEEE 1788-2015 (10.5.4, Table 10.1): `sqrt_rel` (sqrRev), `invabs_rel` (absRev), `nth_root_rel` (pownRev, for every integer p), [GAOL v5] `asin_rel`, `acos_rel`, `atan_rel`, `acosh_rel`, and `div_rel`, `%` and `%=` (mulRev). Over the cases of the minimal tests of libieeep1788 and random ones, each result has to be empty when an argument is, *valid*, enclosing the hull of the x of I whose image is in J, which the test holds the tightest enclosure of, and *accurate*, within nextOut(tightest(nextOut(*arguments*)))

(12.10.1), as the standard recommends for inf-sup types. The bounds of `asin_rel`, `acos_rel` and `atan_rel` are allowed one double further, their reduction modulo π rounding twice, and those of `nth_root_rel` for a negative p two: of the 18 052 random cases with $p < 0$, 12 495 have a nonempty result with nonempty accurate bounds, 521 were one double beyond them, 25 two, none more (see Section 7.3, page 160). The hulls of `reverse_values.h` are computed from the definition of each function with exact rational arithmetic, and mpmath for the periodic ones, not from GAOL nor from the results libieeep1788 expects, which the generator checks are enclosures of them. `atan2_rel_y` and `atan2_rel_x` (atan2Rev1, atan2Rev2), [GAOL v5] which libieeep1788 does not test, have a table of their own, generated with mpmath at 3000 bits: 1 969 cases, 1 263 of them with points, whose hulls are computed from the vertices of the polygon of each convex cone in the box; each result has to be valid, accurate and within 2 doubles of the tightest bounds, and about 600 000 sampled points whose angle is in c have to be in both results. `pow_rel` and `pow_exponent_rel` (powRev1, powRev2) [GAOL v5] are checked on the 804 cases of `pow_rev.itl` of ITF1788, random ones and ones whose solutions are doubles, the hulls computed with mpmath at 2000 bits by cells: the generator found two results of ITF1788 wider than the tightest enclosure, and by sampling over 6 000 random boxes, 54 305 points whose power is within c have to be kept by both functions.

`test_ibex_codac` checks the cases of the unit tests that IBEX and Codac run on their intervals, which are GAOL's, against the exact results: 3905 checks. [GAOL v5] A case is a row of a table, written with `gaol::interval` and the functions of GAOL, named by its origin (`IBEX TestArith::add05`, `Codac Interval_operations:176`); the exact result is computed apart by the scripts of `tests/tools/ibex_codac` with exact rationals and mpmath at 2000 bits, and the bounds of GAOL have to enclose it with no double between them and the tightest bounds (two doubles for `atan2_rel_x` and `atan2_rel_y`). Three sections: the forward tests of IBEX (71 of the 139 tests of `TestInterval` and 182 of the 343 of `TestArith`, and 3 that IBEX never runs; 1393 checks), its backward contractors, its generalized division and its distances (197 cases, 1306 checks), and the tests of Codac (1206 checks; of the 945 checks of its three files, 195 are already in the tests of GAOL). What only IBEX or Codac have, `tan06` of IBEX, whose expectation is wrong, and the distance to the empty set, whose convention differs, are left out. What the projects expect is compared with the exact results by the scripts: 1940 comparisons, no disagreement beyond their tolerances, and 23 expectations of IBEX with an integer bound that is not a bound of the exact result (`tan02` expects $[-1, 1]$ where it is $\pm(1 + 2^{-52})$).

14.2.5 CORE-MATH, compoundm1 and the 128-bit integer

`core_math` checks the bounds of the elementary functions against CORE-MATH itself, which is correctly rounded in the rounding direction in effect: the tightest bounds of f at a double x are the values it gives rounding downward and upward, which the test computes by setting the direction itself. GAOL's bounds have to enclose them, to be equal above and within one double below, and to be the tightest

where the code gives the exact value itself. Each function is tried at the ends of its domain, at the values GAOL treats apart, at the powers of two and their neighbours, at the subnormals and at random doubles. `pow(x, y)` has to be the tightest at the corner of a box, both bounds, over 84 070 pairs; where x^y is rational (8 981 pairs, 3 211 of them a double) the reference is x^y itself, computed with integers apart from GAOL and from CORE-MATH, whose values downward and upward have to be its roundings too, which tells a fault of CORE-MATH from one of GAOL: CORE-MATH's `pow` rounded to nearest the midpoints of its exact phase on 32-bit ARM, and its square roots on a 32-bit x86 Windows. `exp2`, `exp10`, `log2` and `log10` have to be the tightest, and the exact values where they are doubles (`exp10` of 0 to 22, `log10` of a power of ten up to 10^{22}). The functions of Table 10.5 but `compoundm1` have to be the tightest over intervals too, the hull of their image computed from the values at the bounds of the part of the interval in the domain, at the points nearest to and farthest from the origin (`hypot`), and at every multiple of 1/2 within for `sinpi`, `cospi` and `tanpi`. [GAOL v5] `sin` is checked, in the four directions, against the upstream sources of CORE-MATH compiled unchanged at 19 arguments of its accurate path, which computes with the 128-bit integer GAOL ports, and `cos` at 45 arguments of its accurate path against its roundings computed with mpmath at 4 000 bits, which reach the branches of that path. First of all, the test checks the `fma()` and `round()` of the C library, which CORE-MATH calls: `fma()` correctly rounded in the four directions at six triples, and `round()` at ±`0x1.fffffffffffffp-2`, ±0.5, ±2.5 and $2^{52} - 1/2$; the `fma()` and `round()` of mingw-w64's own math library on x64, which GAOL refuses, fail 21 and 6 of these checks under wine. [GAOL v5] The `gaol_ldexp()` CORE-MATH rounds the subnormal values of `erfc` with has to be $x \cdot 2^n$ rounded once in the four directions at 20 000 random pairs. `pow` at the corners where the base is 0 or $+\infty$ or the exponent infinite has to raise neither the division-by-zero nor the invalid-operation exception, and `pow` of a power next to the largest double no invalid-operation exception either: for $y \log x$ from 709.78263 to 709.78267, CORE-MATH's first phase scaled its approximation by 2^{1024} and computed $\infty - \infty$, at 942 of 1 001 exponents of the base 2 from $1024 - 2^{-13}$ in steps of 2^{-24}. [GAOL v5] The reading of a number above the largest double (`1e400`, `0x1p1024`) has to be $[DBL_MAX, +\infty]$ and raise no overflow flag, which `strtod()` raised, and a number below the least normal double no underflow flag. [GAOL v5] CORE-MATH, called without GAOL in each of the six directions of x86, has to round in the direction of MXCSR; on a 32-bit x86 processor its `fegetround()` was the C library's, reading the x87 unit (glibc) or giving −1 (Visual C++) when the two differ. `erf` and `erfc` have to be the tightest over 80 000 intervals, and at 22 hard-to-round arguments of the `erfc.c` of CORE-MATH.

`compoundm1` checks $(1 + x)^y - 1$ of IEEE 1788-2015 (Table 10.5), which CORE-MATH has not and whose bounds GAOL computes with 64-bit fixed point in a fast phase, and where that does not settle them with a 128-bit significand and a bound of the error. [GAOL v5] The test goes from the bottom up: the count of leading zeros, the 256-bit product against the schoolbook one, the integer square root, the doubles

around an integer; the sums and products of 20 000 triples of doubles, whose balls have to hold the exact value; the balls of $\log(1+x)$ for 493 arguments and of $e^t - 1$ for 322, which have to hold the value computed to 256 bits by mpmath and be no wider than 2^{-118} relatively; the fast phase by itself, whose center has to be within 3/10 of the bound of its error over 20 000 pairs from eight domains (the worst error found is 0.22 of it, in 27 million pairs), and which answers 74 to 96 % of the pairs according to the domain; the exact division of $y = -1$ at the 633 points of 719 where $1 + x$ takes 127 bits or less. Then the bounds at 5 854 pairs have to be the greatest double at most the value and the least double at least it, the values computed by mpmath with 1 500 to 7 000 bits, and on 850 boxes the result has to be the hull of the values at the four corners and of 0 where the box crosses $x = 0$ or $y = 0$, with the limits at $x = -1$ and at infinity, read in the script and not in GAOL.

`u128` compiles the two 64-bit halves of `gaol/math/gaol_u128.h`, with which GAOL computes the accurate phases of CORE-MATH where the compiler has no 128-bit integer (Visual C++, and GCC on 32-bit targets), and compares every operation with the native type at the ends of the ranges, at the powers of two, at random values and over every shift count from 0 to 127: the two have to give the same 128 bits. Where the compiler has no native type, the test checks the identities the operations satisfy instead. `extended_precision` checks the results that doubles computed on the x87 unit round wrongly, which is why `gaol/gaol_config.h` refuses that unit (see Section 2.3.2, page 25): `exp(-0x1.74910d52d3051p+9)` gave 0 rather than 2^{-1074} and `tanh(0x1.30fc1931f09c9p+4)` 1 rather than $1 - 2^{-53}$, their results being rounded twice. The values are computed by `extended_precision_values.py` with mpmath at 5000 bits. Built on the x87 unit, the test fails on Debian 12 i386 with GCC 12 (22 checks) and on x86-64 with `-mfpmath=387` and GCC 9 (76 of its 300 checks).

14.2.6 The expressions

`expressions` goes through every node of the tree of `gaol/parser/gaol_expression.h` and every way a string can be wrong: the numbers in every form the lexer takes, the operators and the functions alone and nested, and the strings the parser has to refuse with an exception rather than an interval. The intervals have to be read wherever a number may stand (`1+[1,2]`, `[cos([0,1]), 2]`), which the two grammars of GAOL 4 refused, an error anywhere has to stop the reading, and the names GAOL v5 adds (`exp2`, `log2`, `cbrt`, `sign`, `trunc`, and every function of GAOL) have to be read in any case of letters, the lexer taking the longest name. [GAOL v5] Each value is compared with the same computation written in C++, which the other tests check against the exact results: what is tested is the lexer, the parser and the evaluation, not the operations. What GAOL throws has to be exactly `std::invalid_argument` for what the reader refuses and `std::domain_error` for a function called with an argument it does not take, told apart by `typeid`, with a `what()` that is `file:line: explanation`, the file being the name of the source that threw with no directory (see Section 13.5.1, page 266). [GAOL v5] Long

expressions have to be evaluated and deleted: chains read from a string of 200000 sums or products and of 20000 differences, and chains of 200000 minus signs and 20000 sines, whose evaluation and deletion made one call within another per node and overflowed the stack of 8 MB; a leaf at the bottom of a chain of 20000 nodes has to be visited and deleted less than 64 KiB of stack away from the function that starts the evaluation, where the recursion took 1.9 MB and 0.96 MB (GCC 15.2, x86-64, Release). A string nested 9000 levels has to be read, and one nested 100000 levels refused, and a sum of 10000 terms read with no more than 64 allocations live at once, where the parser built 20000 nodes before the evaluation. [GAOL v5] A string holding a NUL character has to be no expression, a null pointer has to throw `std::invalid_argument` with the names of GAOL and of the standard, and to make `parse_interval()` return `false` without exceptions, where it died of a segmentation fault. `[l, u]` with l above u has to be the empty set, two numbers between the same doubles being compared exactly, as digits, and the bounds of `<a, b>` have to be doubles: `<0.1, 0x1.9999999999999p-4>` is refused. [GAOL v5] Every allocation of a reading has to be able to fail, each in turn: the reading has to throw `std::bad_alloc` and free all it allocated, which the test counts with a replacement of the global `operator new`, where flex ended the program with `exit(2)`. The checks of several threads reading at once are commented out, the tests running no thread.

`debugging` compiles GAOL's headers with `GAOL_DEBUGGING`, which the Debug builds define, and builds and evaluates an expression whose nodes call `GAOL_DEBUG_VERBOSE`, which has to run its command at the level given to `gaol::init()` and not above (see Section 13.6, page 269): `gaol/parser/gaol_expression.h` did not compile with `GAOL_DEBUGGING`, writing on `std::cout` without the header. [GAOL v5]

14.2.7 The builds and the compilers

`version_file` (a script of CMake) checks that the reading of `VERSION.txt` ignores a UTF-8 byte order mark, the line ends of Windows and the blanks around the version, refuses anything but three numbers, and names a UTF-16 file and a NUL byte in its message, whether `file(READ)` cuts the text at the NUL byte (CMake 3.14.7 and 3.16.3) or keeps it (3.28.3, 4.4.3); `.github/scripts/version-file.sh` runs the same cases on `configure`, meson and autoconf in the continuous integration. [GAOL v5] The compile tests `refused_*` compile a program including `<gaol/gaol>` with `-ffinite-math-only` and with `-ffast-math`, put after the flags of interval arithmetic, and the compilation has to fail with the message of `gaol/gaol_config.h`, which the test looks for in the output of the compiler; compiled with neither, it has to compile (see Section 2.3.2, page 25). With `-ffinite-math-only` the empty interval is no longer told empty: `([1, 2] & [3, 4]).is_empty()` was false with GCC 9.4 and 13, and with Clang 18. The CMake build copies and touches the source before each compilation that has to fail (`tests/compile_error.cmake`), since an object compiled once stayed up to date. `refused_aligned_new` and `refused_aligned_new_cxx17` compile `gaol/gaol_`

`config.h` alone with `-D_WIN32 -U_WIN64`, which stand for 32-bit Windows on any system, without the `-faligned-new=8` the flags carry there, in C++11 and in C++17, and have to fail; `refused_aligned_new_positive` compiles it with the option, and has to succeed: the C runtime of 32-bit Windows aligns the memory of `new` on 8 bytes, an SSE2 interval needs 16, and a `std::vector` of SSE2 intervals stopped the program on its first `movaps`. [GAOL v5] `cpack_stale_configure` configures copies of the tree and checks that CMake warns when the committed `configure` says another version than `VERSION.txt`, naming the two versions, and that CPack configures the copy again and names the archive after the new version. `automatic_configuration` configures GAOL several times in one directory, with the defaults, `GAOL_SIMD=OFF`, the AVX-512 path refused and asked for, the Debug build type, the settings of the developers and a cross-compilation, and holds `gaol_configuration.h` and the messages of CMake to what each asks for and to what the checks found; it builds nothing and takes about ten seconds. Only the CMake build has these two scripts: `configure` and meson are checked by the audit of the three builds. The six tests `nodiscard_*` compile a program in each of the three standards with `-Werror=unused-result` and `-Werror=attributes` (`/we4834`, `/we5030` and `/we5051` with Visual C++): throwing away the result of `sqrt(x)` has to fail, and using it has to compile, since `GAOL_NODISCARD` is `[[nodiscard]]` before C++17 with GCC 7 and later and Visual C++ 2019 16.4 and later, and the attribute of Clang with Clang. [GAOL v5]

14.2.8 The unit tests of GAOL 4

`arithmetic_operators`, `assignment`, `constants`, `constructor`, `float_functions`, `generic_functions`, `input_output`, `interval_functions`, `misc`, `non_arithmetic`, `relations`, `reverse_mappings` and `trigonometric` check the results of the operations on chosen intervals, as Frédéric Goualard wrote them in `check/`. `reverse_mappings` draws its random intervals from a seed of its own rather than from the process identifier, so that its checks are the same at each run, and checks the infinite ends of `atanh_rel(J, I)` where J ends at ± 1. [GAOL v5] `input_output` checks that the reader refuses `<-inf, -inf>` and `<inf, inf>`, whose bounds are not one value. `constructor`, `assignment`, `arithmetic_operators` and `relations` also check the integers: `interval(n)` and `interval(a, b)` around 2^{53}, 2^{63} and 2^{64}, for integers of every width and in the four rounding directions, each operator in both orders, and `set_contains()` at $2^{53} + 1$ and $2^{64} - 1$, which the conversion to a double made wrong. [GAOL v5] `relations` checks `overlap()` on each of the 16 states of Table 10.7 and `gaol::lexicographic_less`, a strict total order that sorts a `std::vector` holding an empty interval, which `<` reads past the end of (see Section 6.6, page 140). [GAOL v5] `misc` checks the names of `gaol`, which names those of `gaol_core` one by one, with `static_assert`: the names of GAOL 4 that Codac and IBEX use compile, and what GAOL uses for itself (`get_rounding()`, `gaol_pown()`, `rounding_state`, `the_null_expr`) is not found through `using namespace gaol` (see Section 3.4, page 69); it checks

that the kernels the operators call are in `gaol_detail` and not in `gaol_core`, where they made a program ambiguous (`test_kernels_are_not_in_gaol_core`); that the heap holds a type as aligned as an interval in `std::allocator`, `new`, `std::vector`, `std::list`, `std::deque`, `std::map` and `std::make_shared` (`test_heap_alignment`, which on Linux with a `::operator new` aligning on 8 bytes fails without `-faligned-new=8` with GCC 15 and Clang 21 and passes with it); that intervals at addresses of 8 modulo 16 compute where the SSE2 intervals keep their bounds as two doubles (`test_intervals_aligned_on_8_bytes`); that a function working on local intervals, entered through a trampoline in assembly with the stack at each of the four alignments of 4 bytes, returns with the right bounds on 32-bit Windows with GCC or Clang, where the system calls its threads with the stack aligned on 4 bytes (`test_stack_entered_aligned_on_4_bytes`, which GCC 14 at `-O2` fails on Linux for 32-bit x86 without `-mstackrealign`); and which builds with `GAOL_PRESERVE_ROUNDING` write MXCSR alone (`test_rounding_macros_write_mxcsr_alone`). [GAOL v5] `intervalf` and `interval2f` test the intervals of floats where a developer compiles them (`GAOL_FLOAT_INTERVALS`), and are skipped otherwise (exit status 77). `check/fpu.cpp`, an empty test, `check/essai.cpp` and `check/performances.cpp`, the benchmark of GAOL 4, are gone, `make perf` replacing the last. [GAOL v5]

14.2.9 The benchmark

`gaol_performance` (`tests/performance.cpp`) measures the time per operation of GAOL's arithmetic and elementary functions, of the quotient by a positive double, of the constructor, `floor()`, `x &= y`, `x | y`, `max()`, `min()`, `x % y`, `x.inverse()`, `midpoint()`, `nth_root(x, -2)`, `nth_root(x, 3)` and `asinpi()`, of two relations, of the same operations on doubles, and, in its last rows, of lines of 15 additions, 15 additions and subtractions, 7 products, 7 quotients and 13 operations of the four kinds between 8 and 16 intervals. [GAOL v5] Its title names the intervals GAOL computes with, NEON, SSE2 or FPU, and four rows give the product, the quotient, the square and `pow(x, 2)` of operands of random signs, the other rows having positive operands only, which a processor predicts: the operands are 1 024 intervals that the predictor learns by heart, and the product of random signs of the FPU intervals before their kernels, which chose a branch by the signs as GAOL 4 did, took 6.4 ns on them and 12.4 on 16 384 (Apple M2). The table is Markdown, which the jobs of the continuous integration copy into their summary. After the table, it prints the time of the check each operation makes, $1 + (2^{-1060} + 0)$ not above 1, and of the same check made of 2^{-60}: a processor that takes a microcode assist for a subnormal operand shows it there; with the NEON intervals, the check is the read of FPCR. It is not a test (see Section 14.4.1, page 299). The tools of `tests/tools/pow/` and `tests/tools/arith/`, which compare the bounds of two builds of GAOL and build GAOL with deliberate errors, are not tests either (see Section 14.5, page 305).

14.3 The examples

The directory `examples/` holds 15 programs that use GAOL v5 the way interval arithmetic is used in practice, as IBEX, Codac, C-XSC, INTLAB and the textbooks use it, and the example of GAOL 4, 16 programs in all. `examples/CMakeLists.txt` is a project of its own, which uses an installed GAOL as any user's project would:

```
% cmake -S examples -B build-examples -DCMAKE_PREFIX_PATH=prefix
% cmake --build build-examples
% ctest --test-dir build-examples
```

The three builds of GAOL build them too, with the GAOL they build, where they are asked to (`WITH_EXAMPLES`, `--with-examples`, `with-examples`), and `make check` (`ninja check` with meson) runs them with the unit tests (see Section 2.9, page 54).

Examples 01 to 15 check what they print: [GAOL v5] each enclosure is compared with a value computed apart (mpmath, closed forms, exact fractions), each proof of uniqueness or of emptiness is tested, and each claim of the output ("4 times too wide", "the excess is divided by 100") is proved with interval arithmetic. A failed check prints `FAILED: <claim>` and the program returns a failure, so that ctest runs the examples as tests. The checks test containments and qualitative facts, never digits: they pass on every build. Example 16 checks its output too, with the digits of an enclosure that no operation rounds. All 16 examples compile without a warning (`-Wall -Wextra`, GCC 9.4 and Clang 18) and pass their checks with the four builds tried: the SSE2 intervals, the FPU intervals (`GAOL_SIMD=OFF`), `GAOL_PRESERVE_ROUNDING=ON`, and the library compiled by Clang 18. Each runs in less than 0.3 s with the default build (06 takes 1.3 s with `GAOL_PRESERVE_ROUNDING`). Their outputs are the same on every build, apart from the sign of a zero bound (`[-0, 4]` with SSE2, `[0, 4]` with the FPU code), from the report of the rounding policy in 13, and from a digit or two where a program computes its own doubles (the preconditioner of 10). Every program follows the rules of GAOL v5: `#include <gaol/gaol.h>`, no call to `gaol::init()`, and `gaol::cleanup()` right after the last use of GAOL. Three headers are shared: `box.h`, a vector of intervals with the names IBEX and Codac use for boxes (`widest()`, `max_width()`, `mid()`, `volume()`, `bisect(i, ratio)`, `&`, `|`), since GAOL has no box type and every algorithm on several variables needs one; `dual.h`, a forward-mode automatic differentiation over intervals, with which a function written once as a template gives its natural extension with `interval` and its gradient enclosure with `Dual<N>`; and `affine.h`, the affine forms of example 12. Table 14.2 lists the programs.

TABLE 14.2 – The examples of `examples/`

Program	What it computes and shows
`01_first_steps`	Intervals built every way, their numbers, sets and relations: containment, `interval(0.1)` against `textToInterval("0.1")`, `mid()` against `midpoint()`, no `==`, the certainly relations
`02_decimals_and_rump`	0.1 summed ten times, π by Machin, Rump's polynomial: decimals are not doubles, `sqrt(2.0)` is the C library's, the width as an alarm
`03_dependency_problem`	$x - x$, $x(1-x)$ three ways, $(x-1)^5$, subdivision, Goldstein-Price, rotations: the dependency problem, single-use forms, `sqr` against `x*x`, the linear convergence of subdivision, the wrapping effect
`04_centered_form`	$x \cos x$ and Chebyshev T_5, natural and mean-value forms: automatic differentiation over intervals, the quadratic convergence of the centered form, the monotonicity test
`05_interval_newton`	A root of $\sqrt{x} + (x+1)\cos x$, the 7 roots of $\sin x - x/10$, $\pm\sqrt{2}$: interval Newton, uniqueness proofs, the extended division, the trap of $f(m)/F'(X)$ when $0 \in F'(X)$
`06_global_optimization`	The global minima of Goldstein-Price and of the six-hump camel: Moore-Skelboe branch and bound, a priority queue with an explicit comparator, the cluster effect, mean-value form and monotonicity test
`07_contractors`	IBEX's backward arithmetic, $x^2 + y^2 = 1$ with $y = e^x - 1.5$: forward-backward contraction (HC4Revise) with GAOL's reverse functions, their names in GAOL, IEEE 1788 and IBEX
`08_sivia`	Codac's SIVIA example, IBEX lab2 and lab3, $\sqrt{x} + \sqrt{y} \in [0, 1]$: set inversion, guaranteed area brackets, contractors in pavers, the domain trap of a library without decorations
`09_parameter_estimation`	IBEX lab5, range-only localization from three beacons: bounded-error estimation by set inversion, measurements read as text, a contractor with `sqrt_rel`
`10_krawczyk`	Two circles of IBEX's solver, the roots of $\lambda + e^{-\lambda}$: a branch-and-prune solver with the Krawczyk operator, why IBEX bisects at 45 %, a false proof from a wrong Jacobian
`11_ode_enclosure`	$x' = -x^2$, Codac's Lohner system, $\int \exp(-x^2)$: Picard's a priori enclosure, Taylor steps with a remainder, the mean-value form against the wrapping of $x + hf(x)$, interval Riemann sums

Table 14.2, continued

Program	What it computes and shows
`12_affine_arithmetic`	$x - x$, $x(1-x)$, $(x-1)^5$, $(x-y)^2$, rotations with affine forms: a sound affine arithmetic on GAOL alone, when it beats intervals and when it loses to them
`13_rounding_environment`	The program's own doubles, `printf`, `lrint`, `strtod`, threads, `cleanup()`: what GAOL's rounding direction does to the rest of the program, and how to live with it (see Section 4.2, page 90)
`14_generic_programming`	One template evaluated with `double`, `interval` and `Dual<1>`: argument-dependent lookup, `using gaol::pow`, `T(0.0)`, the standard algorithms and containers with intervals
`15_text_and_ieee1788`	Intervals read and written as text, one contractor step with the names of IEEE 1788: the literals GAOL reads, reading a file, exact output, the namespace `gaol_ieee1788`
`16_Goldstein_Price`	A million evaluations of the Goldstein-Price function, timed: the example of GAOL 4, with the true function; the enclosure contains the range, 231 times wider

Remark 14.3.1 *The example of GAOL 4 dropped the* $+1$ *of the Goldstein-Price function,* $(x+y)^2$ *instead of* $(x+y+1)^2$*, and so did the manual of GAOL 4, which showed the same program and called the enclosure the range of* f*, whose true range, about* $[3, 635255]$*, is about 237 times narrower than the enclosure* $[-56254330, 94177270]$*. Both now compute the true function.* [GAOL v5] *Its enclosure,* $[-87881320, 147125080]$ *over* $[-2, 2]^2$*, contains the range* $[3, 1015690.27\ldots]$ *and is 231 times wider than it, for lack of a link between the occurrences of* x *and* y*: the dependency problem (see Section 1.4, page 11).*

A short tutorial, `examples/tutorial.md`, writes four of the algorithms of the examples in programs of a few dozen lines, the condensed forms of examples 03, 05, 07 and 06: the range of a function by subdivision, the interval Newton method, a contractor, and a branch and bound. Each program is compiled with the flags GAOL gives (see Section 3.3, page 64):

```
% c++ -std=c++11 -O2 $(pkg-config --cflags gaol) prog.cpp -o prog \
      $(pkg-config --libs gaol)
```

Its outputs are the same with the SSE2, the NEON and the FPU intervals. The first program evaluates $x(1-x)$ over $[0.4, 0.6]$, whose range is $[0.24, 0.25]$: the formula written as it is gives an enclosure 20 times too wide, $1/4 - (x - 1/2)^2$, where x occurs once, gives the range itself, and cutting x into 2^k pieces with `bisect(0.5)`, whose images are joined by the hull `|`, narrows the enclosure linearly. The second encloses $\pm\sqrt{2}$ by the interval Newton method, with `div_rel` on the negative and

the positive parts of $f'(X)$, which keeps the gap around the midpoint that the division `/` would fill. The fourth finds the minimum of $x^4 - 4x^2 + x$ over $[-3, 3]$ by a branch and bound with a priority queue on the lower bounds, in 19933 cuts, and prints $[-5.44419306660636, -5.444192066610894]$, which contains the minimum $-5.4441920666108974\ldots$ computed by mpmath; the natural form converges slowly, and example 06 shows that on the six-hump camel function it needs 25 057 bisections to bring the enclosure of the minimum within 10^{-3}, where the mean-value form with the derivative of `dual.h` reaches 10^{-9} in 1 127, and 261 with a monotonicity test. The third program is the contractor of Example 14.3.2.

Example 14.3.2 *The forward-backward contractor of $x^2 + y^2 = 1$ evaluates the expression tree forward, from x and y to the sum, which it intersects with 1; it then goes backward, and each node is intersected with what the others allow. The relational functions of GAOL are the backward operations:* `sqrt_rel(a, x)` *is the hull of the x of X with x^2 in a, both signs included, where* `sqrt(a)` *would lose the solutions with $x < 0$ (see Section 7.3, page 160).*

Example

```
#include <iostream>
#include <gaol/gaol.h>
using namespace gaol;

// The contractor of x*x + y*y = 1: forward-backward evaluation of the tree
// s = a + c, a = sqr(x), c = sqr(y), with s = 1. Returns false when the box
// holds no solution.
bool contract(interval& x, interval& y)
{
  // forward: an enclosure of each node over the box
  interval a = sqr(x);
  interval c = sqr(y);
  interval s = (a + c) & interval(1.0);     // the root must be 1
  if (s.is_empty()) {
    return false;
  }
  // backward: s = a + c gives a = s - c and c = s - a ...
  a &= s - c;
  c &= s - a;
  // ... and a = sqr(x) gives x = +-sqrt(a): sqrt_rel() keeps both signs, and
  // intersects with x itself, where sqrt(a) would keep the positive one only
  x = sqrt_rel(a, x);
  y = sqrt_rel(c, y);
  return !x.is_empty() && !y.is_empty();
}

int main()
{
  interval x(0.8, 3.0), y(-3.0, 3.0);
  std::cout << "box           " << x << ' ' << y << '\n';
  contract(x, y);
  std::cout << "contracted    " << x << ' ' << y << '\n';
  interval u(2.0, 3.0), v(2.0, 3.0);
```

```
34   std::cout << "box           " << u << ' ' << v << '\n'
35             << (contract(u, v) ? "may hold a solution\n"
36                                : "proved: no solution\n");
37   gaol::cleanup();
38   return 0;
39 }
```

The program prints the following lines.

Output

```
box           [0.8, 3] [-3, 3]
contracted    [0.8, 1] [-0.6000000000000001, 0.6000000000000001]
box           [2, 3] [2, 3]
proved: no solution
```

The first box is cut to x in $[0.8, 1]$ and y in $[-0.6, 0.6]$, the bounds being enclosures: the points of the box off this part are not on the circle. The second box is proved empty of solutions, its `s` *being empty. In* `gaol_ieee1788`, `sqrt_rel(c, x)` *is sqrRev(c, x),* `div_rel(c, b, x)` *is mulRev(b, c, x), with its arguments in another order, and* `nth_root_rel(c, n, x)` *is pownRev(c, x, n).*

The examples of this manual are checked as well. `manual/check_examples.py` compiles each example of the files of `manual/v5/chapters/` that shows what it prints, in its lines `@outputs`, with an installed GAOL and the flags of `pkg-config gaol`, runs it, and compares what it prints with the manual, line by line: [GAOL v5] the examples once printed 1 and 0 where the manual showed `true` and `false`, and `chi()` of the empty set printed `-nan` with GCC where it showed `nan`. An example holding a `main()` is compiled as it is; the others are put in a `main()` of their own, after `<gaol/gaol>`, `<gaol/parser/gaol_expression.h>` and six headers of the standard library (`<cmath>`, `<iomanip>`, `<iostream>`, `<limits>`, `<sstream>`, `<string>`), with `using namespace std` and `using namespace gaol`, or `gaol_ieee1788` in the chapter of its names. A line printed that is too long for the page may be shown on several lines of output, each break standing for a blank. Each example is a program of its own: a format or a precision one sets does not reach the next. A compiler error points to the line of the example in its file of `chapters/`, and every output line of the manual has to be in an example the script reads; what the manual shows otherwise, in an `onscreen` block after a program or in a comment, is not checked. The compiler is `CXX`, or `c++`, and `PKG_CONFIG_PATH` has to lead to the `gaol.pc` of the GAOL to check; the exit status is 1 when an example does not compile, fails, or prints something else than the manual shows. It is no test of the builds: the job Ubuntu 24.04 x86_64 GCC of the continuous integration runs it, in C++11, on the GAOL it installs, so that a change of the library that changes one of the outputs shows.

14.4 The continuous integration

The workflows of `.github/workflows/` build GAOL with CMake and run the tests, with `make test` (`cmake --build <build> --target test`, `RUN_TESTS` with Visual Studio). They run for each pull request, and for each push to the branches `master`, `MATH-CORE` and `configure-clean` only: a push to another branch runs nothing until it is proposed, which halves the jobs of a pull request; [GAOL v5] `workflow_dispatch` runs them on any branch. `performance.yml` and `manual.yml` are the exceptions: the first runs for the pull requests that change what it measures only (see Section 14.4.1, page 299), the second for the pushes and pull requests that change `manual/` or itself. The badges of `README.md` show their state on `configure-clean`. Table 14.3 gives the systems, the processors and the compilers they build on; the list of the platforms GAOL supports is in the chapter on the installation (see Section 2.1, page 23).

TABLE 14.3 – The platforms of the continuous integration

System	Processors	Compilers and builds
Ubuntu 24.04, 26.04	x86_64, arm64	GCC, Clang; the sanitizers; `-Wall -Wextra -Werror`; CMake 3.14; GCC 11 and Clang 14 of Ubuntu 22.04; GCC 9; a shared library; autotools, meson
Debian 12, 13 (containers)	amd64, arm64, armhf; Debian 12 on i386	GCC; built for size on Debian 13 armhf; autotools and meson on i386 and armhf
manylinux_2_28	x86_64, aarch64	CMake
Alpine (musl)	x86_64, aarch64	CMake
Debian 13 under qemu	s390x, ppc64le, riscv64	CMake; Ubuntu 24.04 under qemu for POWER9 (GCC 13, `-mcpu=power9`), also built for size
macOS 15, 26, 27	arm64, x86_64 (natively or under Rosetta), a universal build	AppleClang, the Clang of LLVM, GCC; CMake, autotools, meson, the generator of Xcode; the sanitizers
Windows	x86, x64, arm64	Visual Studio 2022 and 2026 (Release and Debug); clang-cl; MinGW-w64 15 and 14 (x86, x64), 13 and 12 (x86); MSYS2 UCRT64 (GCC), CLANG64 and CLANGARM64 (Clang), MINGW32 (autotools, meson); Cygwin (GCC, x86-64); Clang with the target of Visual C++ and the driver of GCC; meson with Visual Studio

Linux. The jobs of Ubuntu build with GCC and Clang, also with the address and undefined behaviour sanitizers, with `-Wall -Wextra -Werror`, with CMake 3.14, with the self-test of the check of the assembly (`tests/tools/asm-order/check_order.py --selftest`, in Python alone), and the check of the library itself with GCC 13.3 and Clang 18, with and without the rounding direction preserved, [GAOL v5] with the compilers of Ubuntu 22.04 (GCC 11 and Clang 14 on x86_64, GCC 11 on arm64), whose images GitHub retires by April 2027, with GCC 9, which has no `__builtin_roundeven()`, and as a shared library (`BUILD_SHARED_LIBS`, `--enable-shared`, `-Ddefault_library=shared`: the static library is the default of the three builds). [GAOL v5] The test `cpack_stale_configure` also runs with `CC="ccache gcc"` and a Ninja outside `PATH`, which it has to pass on to the copies of the tree it configures, and the tests run with the `FE_*` values of Cygwin (newlib, 0 to 3) put over those of glibc for the sources of CORE-MATH. The containers build Debian 12 and 13 on amd64, arm64 and armhf, Debian 12 on i386, manylinux_2_28 and Alpine (musl) on x86_64 and aarch64, Debian 13 under qemu on s390x, ppc64le and riscv64, and Ubuntu 24.04 (GCC 13) under qemu for POWER9; two jobs build for size (`MinSizeRel`, `-Os`) on Debian 13 armhf and for POWER9, where GCC calls the intersection rather than inlining it, and `rounding_direction` checks the choices a program makes on `is_empty()` of an empty interval there too; one job on Debian 13 amd64 tests the empty sets before the bounds in the relations (`-DGAOL_RELATION_BOUNDS_FIRST=0`, the order of RISC-V), where GCC 14 raised the invalid-operation flag for an empty operand.

macOS. The jobs build macOS 15 and 26 on arm64 and x86_64, natively or under Rosetta, macOS 27 on arm64 (on the runner labelled `xcode-27`; there is no job of macOS 27 on x86_64 under Rosetta, GitHub having no runner for it), and a universal build (arm64 and x86_64: the macros of each part, the tests of the arm64 part natively and the test programs of the x86_64 part under Rosetta, and the intervals each part says it has) [GAOL v5] with CMake, with autotools (`-arch` twice in `CFLAGS`, `CXXFLAGS` and `LDFLAGS`) and with meson, on macOS 15 and 26; the generator of Xcode, with the list `arm64;x86_64` and without any, where CMake gives Xcode the architecture of the machine; and the sanitizers with AppleClang, LLVM's Clang and GCC.

Windows. Visual Studio 2022 and 2026 build on x86, x64 and arm64, in Release and Debug. On x64 and x86 the intervals are the SSE2 ones (on x86 their two bounds are two doubles, which need no aligned memory), on arm64 the NEON ones, with Visual C++ and clang-cl, which a step of each job checks. [GAOL v5] A job on x86 and two on arm64 ask for the FPU intervals (`GAOL_SIMD=OFF`), so that their code, and the rounding direction they keep through the x87 unit and MXCSR, stay covered. Three x64 jobs ask for the AVX-512 path (`GAOL_PREFER_AVX512`), two with Visual C++ and one with clang-cl, which they take where the runner has the instructions, with a warning where it has not; since the runners have them or not according to the

machine GitHub gives, five more jobs run the unit tests that compute with intervals under Intel SDE, which emulates a processor with them (Visual Studio 2022 and 2026, Release and Debug, and clang-cl). clang-cl, the Clang of Visual Studio, builds x64 in Release and Debug with Visual Studio 2022, and x86, x64 and arm64 with Visual Studio 2026; on x64 it defines `__x86_64__` and `_WIN32` but not `__WIN32__`, which upstream CORE-MATH tests to read the rounding direction, and on x86 GAOL gives it `-msse2`, without which it computes doubles on the x87 unit. MinGW-w64 15 builds x86 and x64, Release and Debug, with the SSE2 intervals on both and `-faligned-new=8` on x86, which a step checks; 14 builds x86 and x64, 12 and 13 x86; on x86, GCC 15.2 builds with `GAOL_PRESERVE_ROUNDING` and `-mfma`, and GCC 13.2 with `GAOL_FMA=OFF`, the `fma()` of mingw-w64 11 on the x87 unit. MSYS2 builds UCRT64 (GCC, and GCC again with `GAOL_PRESERVE_ROUNDING`, with and without `GAOL_FMA`: with `-mfma`, the rounding direction goes through MXCSR alone, which Linux, clang-cl and Visual C++ x64 with `/arch:AVX2` run), CLANG64 (Clang, and Clang again with the direction preserved, which takes the AVX-512 path where the runner has the instructions, GCC for Windows never taking it) and CLANGARM64 on the arm64 runners (the NEON intervals). Cygwin builds GCC for x86-64, Release and Debug, with `-mfma -mprefer-vector-width=128`, all the tests passing but those skipped there (`intervalf` and `interval2f`, which no build compiles, and `cpack_stale_configure`, the CPack of Cygwin making no archive), and a configuration with `GAOL_FMA=OFF` that `gaol/gaol_config.h` has to refuse. [GAOL v5] Clang with the target of Visual C++ and the driver of GCC (`clang++`, not clang-cl) builds in the environment of Visual Studio with CMake and meson for x64 and x86, and with autotools for x64, the library in C++14 there, the standard library of Visual C++ needing it. With GCC on x64 and x86 in Release, a step reads the objects with `objdump` (`.github/scripts/check-gcc-windows-fma.sh`): `gaol_interval_core.cpp` and the `cos` of CORE-MATH have to hold fused multiply-add instructions, and no object an instruction on a `%ymm` or `%zmm` register on x64, which GCC would spill to a stack Windows x64 aligns on 16 bytes (GCC bug 54412); on x86 such lines are listed and noticed, not an error.

The three builds. The workflows also build GAOL with autotools and meson on Ubuntu (x86_64, arm64), Debian (i386, armhf), macOS (arm64, x86_64), MSYS2 (UCRT64 for x64, MINGW32 for x86) and Cygwin, with meson and Visual Studio 2022 (x64, x86) and 2026 (x64, arm64), and with meson 0.53.2, the oldest the meson build takes; run the tests, and build them again with the GAOL they install, which is the only library installed, CORE-MATH being compiled into it; build GAOL as a part of another project brought in by `FetchContent` and as a subproject of a meson project, whose code compiled with `gaol_dep` has to get the flags of `gaol.pc`, and check that GAOL and CORE-MATH are compiled as in Release where the project gives no build type, and that `GAOL_DEBUG` and `enable-debug` build them for debugging; [GAOL v5] check that the three builds agree on Ubuntu (x86_64 and arm64, GCC and Clang), macOS (arm64, x86_64), Debian (i386, armhf) and MSYS2 (UCRT64, MINGW32) (the audit of `.github/audit`, Section 14.5,

page 305); check that `configure`, meson and autoconf read a `VERSION.txt` that starts with a byte order mark or has the line ends of Windows, and that meson reads it with no Python on `PATH`; check that `make distclean` gives the source tree back as git has it after the autotools have built, installed and tested GAOL in it; and check the refusals of Section 14.4.2, page 301. Jobs of each build restore the rounding direction (`GAOL_PRESERVE_ROUNDING`): Ubuntu x86_64 GCC and Clang (which take the AVX-512 path where the runner has it) and arm64 GCC and Clang, Debian i386 and armhf, macOS arm64, Visual Studio x86, x64 and arm64, clang-cl x64, MinGW-w64 x86, MSYS2 UCRT64, CLANG64 and CLANGARM64, autotools and meson. Jobs of autotools and of meson on Ubuntu x86_64 build GAOL without exceptions and run the examples and the tests, which leave out there the checks of an exception. The jobs built in Release print the time per operation in their summary, a table that no other run compares.

Which intervals. On 64-bit ARM processors the intervals are the NEON ones by default (`GAOL_SIMD`; `GAOL_ARM64`, `--enable-arm64` and `-Denable-arm64` were an option of their own), [GAOL v5] so that the jobs of Ubuntu arm64, macOS arm64, the aarch64 containers, and those of autotools and meson on arm64 test them (see Section 4.5, page 96). A few jobs build the FPU intervals instead, so that their code stays covered, the preserved rounding direction included: the GCC and the Clang of Ubuntu 24.04 arm64, both with the direction preserved too, macOS 15 arm64, and autotools and meson on Ubuntu 24.04 arm64. The GCC and the Clang of Ubuntu 24.04 x86-64 build them too, GCC without and with the direction preserved and Clang with it: the compound products and quotients of the FPU intervals hold the direction only by the order of their stores, which the checks of `rounding_direction` with the direction preserved make a rule. [GAOL v5] A build that fell back to the FPU intervals without saying so would keep every test green, so the jobs check which intervals the configuration took, in a step "The intervals" (`.github/scripts/check-intervals.sh`, which reads `GAOL_USING_NEON_INSTRUCTIONS` and `GAOL_USING_SSE2_INSTRUCTIONS` in `gaol_configuration.h` and fails if they are not the expected ones): the NEON intervals on 64-bit ARM, the SSE2 intervals on x86-64 and on 32-bit x86 (Visual C++, clang-cl, MinGW-w64, MSYS2 MINGW32, Clang for Visual C++ and Debian 12 i386), and the FPU intervals where the option is off and on armhf, s390x, ppc64le and riscv64; two jobs give the SSE2 intervals of Linux the form they have on x86 with Visual C++ (`-DGAOL_SSE2_UNALIGNED_STORAGE=ON`).

Warnings and headers. Five jobs build the library, the sources of CORE-MATH included, the tests and the examples with `-Wall -Wextra -Werror`: four with the GCC and the Clang of Ubuntu 26.04 on x86_64 and on arm64, and one with the AppleClang of macOS 26 arm64. Four jobs do so with `/W4 /WX`, with the Visual C++ of Visual Studio 2026: x64 in Release, and again asking for the AVX-512 path (compiled where the runner has the instructions), x64 in Debug, and x86 in

Release. On arm64, the check of the installed headers below compiles the header of the NEON intervals, which `gaol/core/gaol_interval_core.h` includes, with the warnings of the headers (`-Wold-style-cast`, `-Wundef`) that the intrinsics of `<arm_neon.h>` have to stay clear of. [GAOL v5] The first five check the headers GAOL installs with `.github/scripts/headers.sh`: each header compiles alone with `-std=c++11`, the oldest standard GAOL takes, and `-Wall -Wextra -Wold-style-cast -Wundef -Werror`, but those that other headers include in the middle of their code or for Visual C++ only; and the macros that `gaol/gaol` and the two headers of the parser define, the include guards apart, start with `GAOL_` or `gaol_`. `-Werror` and `/WX` go to the targets only (`CMAKE_COMPILE_WARNING_AS_ERROR`), so that no check of CMake fails on a warning of its test program. The library is also compiled with `-Wall -Wconversion` in every build, which fails no job.

The locales and the manual. The jobs on Ubuntu and in the Debian containers generate `fr_FR.UTF-8` before the tests with `localedef` (`.github/scripts/comma-locale.sh`), their systems having no locale writing a decimal comma; macOS has `fr_FR.UTF-8`, and Windows `French_France.1252`. After the tests, the jobs of Ubuntu, of the Debian containers, of macOS (but the universal builds and the generator of Xcode), of Visual Studio, of MinGW-w64 and of MSYS2 fail if the output of `numbers` holds no check under such a locale, read in `Testing/Temporary/LastTest.log`, `tests/numbers.log` with the autotools or `meson-logs/testlog.txt` with meson. [GAOL v5] Alpine and manylinux have no such locale to generate, and check nothing of the kind. `numbers` has 300 s (`GAOL_NUMBERS_TIMEOUT`), 400 in the four Linux jobs with the sanitizers, where it took 109 to 287 s and once more than 300, and 600 in the job of macOS 15 x86_64 GCC with the sanitizers, where it took 146 to 367 s on the same code, and once more than 400. The manual of GAOL v5 is built with the LaTeX of Ubuntu 24.04, by the autotools, the CMake and the meson builds, when `manual/` changes (`manual.yml`), and its PDF, `manual/Manual_GAOLv5.pdf`, is an artefact of the run; the manual of GAOL 4 is not built; the examples of this manual that show what they print are compiled by `manual/check_examples.py` in the job Ubuntu 24.04 x86_64 GCC at each run (see Section 14.3, page 290).

14.4.1 The speed of a pull request

The times of the continuous integration cannot be compared from one run to the next. The runners are virtual machines shared with other jobs, whose speed changes by several times between two runs of the same code: `x * y` with Visual C++ x64 took 7.9 ns and then 16.3, and `sqrt` with MSYS2 and GCC 16 6.8 ns and then 170. The step “Performance” of the jobs prints the table of `gaol_performance` in their summary, and never fails a job: it tells what a platform costs at that moment, and nothing else. Showing the best of several passes would not help: a pull request is to be compared with its base, which is not measured at the same moment.

`performance.yml` measures two builds in turn, in the same job, so that what the runner does between two moments reaches both. [GAOL v5] It runs for a pull request that changes what is measured, not for one that changes the documentation: `gaol/`, `3rd/`, `tests/performance.cpp`, the files of the benchmark of `doc/compare/code/`, `CMakeLists.txt` and `cmake/`, and the scripts and the workflow themselves. A job:

1. checks out the merge commit of the pull request and its first parent, the base it is merged into: the difference between the two is the pull request alone, whatever the base branch has gained since the pull request was started;

2. configures both with CMake in Release and builds `gaol_performance` alone, with the options of the job; the programs of the base are kept from one run to the next, keyed by the job, the image of the runner, the model of the processor, whether it has the AVX-512 instructions (processors of one model do not all have them) and the commit of the base, so that the next push of the same pull request builds only the pull request, and the two builds come from the same tools and the same choices;

3. runs `gaol_performance` three times with each build, the base first in the first and third runs and the pull request first in the second (`.github/scripts/performance-ab.sh`);

4. writes in the summary of the job the table of the median of each row for the base and for the pull request, their ratio (base over pull request: above 1, the pull request is faster) and the spread of the runs, the largest ratio of the slowest run to the fastest of either build (`.github/scripts/compare-performance.py`).

The jobs are Ubuntu 24.04 x86_64 with GCC, with Clang, with GCC and the rounding direction preserved, and with GCC and with Clang and the FPU intervals; Ubuntu 24.04 arm64 with GCC and with Clang (the NEON intervals) and with GCC and the FPU intervals; macOS 26 arm64 and macOS 15 x86_64; Visual Studio 2026 x64, x86, arm64, x64 with the direction preserved, and x64 with the direction preserved and the AVX-512 path left out, which measures the macros that write MXCSR alone, and clang-cl for x64, x86 and arm64; MSYS2 UCRT64 and CLANG64; and Debian 13 containers for i386 and armhf, with no cache. The other platforms (POWER, s390x, RISC-V, MinGW-w64 of Chocolatey, MSYS2 CLANGARM64 and MINGW32) are not compared, nor are the autotools and meson builds: the three builds give the same flags, which the audit checks.

The table never fails a job because of a time. Only a large difference, on rows that share their code, means something on a shared runner (the same code has given a ratio of up to 1.17), and a ratio within the spread of the runs is noise: under the table, the rows whose ratio is 1.5 or more, or 2/3 or less, and beyond the spread of the runs are listed, or none is. A job fails when a build or a program does, and when the tables cannot be compared: an output with no row, or no row common

to the base and the pull request. The comparison has its test, which needs Python alone and runs first in each job (`.github/scripts/test-performance-tools.py`): it fails if the median is replaced by the mean or the threshold is inverted. On Linux, macOS and with Visual Studio the job also runs the ten long lines of the benchmark of the comparison with the other libraries (see Section 12.5, page 237) with `gaol_bench`, the program `make perf` builds too: 200 000 intervals drawn by `bench.py`, 3 repeats of each line, 3 runs of each build, and a second table under the first; the lines are left out, with a warning, where one of the builds has no `gaol_bench` that builds or numpy cannot be installed. The job `control-register-performance` of the containers (Debian 13 armhf and arm64) uses the same scripts to compare two builds of one tree that differ by a CMake option: the control register of the rounding direction written directly with `GAOL_PRESERVE_ROUNDING`, or the functions of the C library, on the rows that the register can change. The same comparison runs anywhere with CMake, a C++ compiler and, for the long lines, Python 3 with numpy:

```
% sh .github/scripts/performance-ab.sh --before-src ../base --after-src . --lines
% python3 .github/scripts/compare-performance.py base perf-before-1.md \
    perf-before-2.md perf-before-3.md -- PR perf-after-1.md perf-after-2.md \
    perf-after-3.md
```

`--options` and `--before-options` give CMake options to both builds or to the base alone; the script describes them all.

14.4.2 Configurations refused or left out

Table 14.4 gives the configurations GAOL v5 refuses, and those the continuous integration leaves out. The builds refuse the compilers among them when configuring, or `gaol/gaol_config.h` when compiling (see Section 2.3.2, page 25), and the last column says which job checks the refusal. What a configuration gives with GAOL v5 was measured on 21 September 2026, building it past its refusal.

Table 14.4 – The configurations refused or left out

Configuration	Why	What GAOL v5 does
Clang for 32-bit ARM (Debian armhf)	Wrong bounds: Clang does not honour the rounding direction there. Built by Clang 19 (Debian 13) and Clang 21 (Debian sid), GAOL v5 fails `arithmetic`, `elementary`, `reverse`, `other_functions` and `rounding_direction`; the products are not the tightest in any direction.	Refused by the three builds and by `gaol/gaol_config.h`; a job checks that CMake, `configure` and meson refuse it, for the C compiler alone as for the C++ compiler alone. GCC builds the armhf jobs.

Table 14.4, continued

Configuration	Why	What GAOL v5 does
Clang 14 on 64-bit ARM, and any compiler saying of `-frounding-math` "overriding currently unsupported rounding mode on this target"	Wrong bounds: Clang 14 does not honour the rounding direction on 64-bit ARM, and says so. GAOL v5 fails `arithmetic`, `elementary`, `reverse` and `other_functions` in Release, and `elementary` in Debug. Clang 18 honours it.	Refused by the three builds; a job checks it for the C compiler alone, the C++ compiler alone and both. Ubuntu 22.04 arm64 is built with GCC only.
MinGW-w64 whose `fma()` or `round()` is wrong: on x64, GCC 11 to 13 of Chocolatey (mingw-w64 before 12) and the mingw-w64 linked with `msvcrt.dll` (MSYS2 MINGW64, the cross compilers of Debian and Ubuntu); on 32-bit x86, GCC 11 of WinLibs (mingw-w64 9) and, without `-mfma`, mingw-w64 12 or later linked with `msvcrt.dll` (MSYS2 MINGW32); on ARM, as on x64	Wrong bounds. On x64 the `fma()` of mingw-w64's own math library adds the products of the halves of its arguments with four roundings (10.7 % of the error-free products and 25 to 73 % of other triples wrong), and CORE-MATH computes with it wherever the compiler has no `-mfma`; its `round()` depends on the direction. GAOL v5 fails `elementary`, `reverse` and `core_math`: bounds of `tan()`, `asin()`, `atan()`, `sinh()`, `cosh()`, `atanh()` and `atan2()` of small arguments do not enclose the exact values. From mingw-w64 12 on, a toolchain linking the UCRT takes the `fma()` and `round()` of `ucrtbase.dll`.	Refused by `gaol/gaol_config.h`; seven jobs check the refusal, three of them MSYS2. MinGW-w64 GCC 14 and 15 on x64, 12 to 15 on 32-bit x86 (13 also with `GAOL_FMA=OFF`), MSYS2 UCRT64 and CLANG64 are built and tested; `core_math` checks `fma()` and `round()` first.
Visual C++, and clang-cl from Clang 16, without `/fp:strict` (`/fp:precise`, their default)	Bounds not certified: they then assume rounding to nearest, and may evaluate or rewrite floating-point operations accordingly, while GAOL computes rounding upward.	Refused by `gaol/gaol_config.h`; `gaol::gaol` gives `/fp:strict` to the code linking it, and each Visual Studio job checks the refusal (`tests/fp_strict`), those of clang-cl included.

Table 14.4, continued

Configuration	Why	What GAOL v5 does
The SSE2 intervals on 32-bit Windows, compiled without aligned `new`	Crash: GCC takes the memory of `new` to be aligned on 16 bytes, which the C runtime of 32-bit Windows aligns on 8, and a `std::vector` of SSE2 intervals crashes on its first `movaps`; with MinGW-w64 14.2 and 15.2 for x86, `arithmetic`, `other_functions`, `reverse` and `rounding_direction` crash. Visual C++ and clang-cl have no such flag: only `/std:c++17` turns the aligned `new` on.	The three builds give `-faligned-new=8` to the library and to the code using it (`gaol::gaol`, `gaol.pc`, `gaol_dep`), and `gaol/gaol_config.h` refuses the intervals compiled without (`refused_aligned_new*`). Visual C++ and clang-cl keep the two bounds as two doubles of alignment 8 (`GAOL_SSE2_UNALIGNED_STORAGE`), which `test_intervals_aligned_on_8_bytes` exercises.
The NEON intervals on 32-bit ARM	Not offered: they need the vectors of two doubles of 64-bit ARM, and write the rounding direction into FPCR with `mrs` and `msr`.	The FPU computes the intervals, as before, and `gaol/gaol_config.h` stops the compilation of NEON intervals elsewhere; the Visual Studio and MSYS2 CLANGARM64 jobs check the NEON intervals on arm64, and the armhf jobs the FPU ones.
Cygwin without the fused multiply-add instructions (`GAOL_FMA=OFF`, or a processor without them)	Bounds not certified: the `fma()` of the C library of Cygwin is not correctly rounded, `fma(1 + 2^-30, 1 - 2^-30, -1)` giving 0 instead of -2^{-60}, and CORE-MATH computes with `fma()`.	Refused by `gaol/gaol_config.h`; the Cygwin job checks the refusal, and builds and tests Cygwin with `-mfma` (GCC 14.4, Release, three tests skipped).

Table 14.4, continued

Configuration	Why	What GAOL v5 does
Doubles computed on the x87 unit of 32-bit x86	Wrong results rounded to nearest, and wrong bounds: the x87 unit computes with 64-bit significands, and CORE-MATH assumes every operation on doubles rounded to a double. With GCC 12 on Debian 12 i386, CORE-MATH rounded to nearest gives the other neighbour of the exact value at 175 of the arguments compared with SSE2; with GCC 9 on x86-64 (`-mfpmath=387`), the bounds of `exp2(-1075)`, `exp10(-400)`, `expm1(-800)` and of `atan2()` of a tiny and a huge number do not enclose the exact values.	Refused by `gaol/gaol_config.h`; the builds compile with `-msse2 -mfpmath=sse` on 32-bit x86, and `extended_precision` fails on the x87 unit.
Debian 11 Bullseye (amd64, arm64, armhf)	Cannot be installed: out of support since August 2026, its security repository lists packages it no longer serves, so that `g++` cannot be installed in the container. GAOL v5 builds and passes its tests there, with GCC 10.2 and CMake 3.18.4 installed from snapshot.debian.org.	Left out of the continuous integration. Debian 12 and 13 are built.
The intervals of floats (`gaol::intervalf`, `gaol::interval2f`)	Unfinished: `sqrt(intervalf)` returns its argument and `interval2f::inverse()` aborts; neither IBEX nor Codac uses them.	Compiled by none of the three builds, which have no option for them: only a developer of GAOL defines `GAOL_FLOAT_INTERVALS`. The audit still compares it between the builds.

14.4.3 What these platforms showed

Building and testing GAOL on these platforms, while the workflows were written and since their first run on 13 September 2026, showed the following, which GAOL v5 changed; the register of the differences gives each change with its measures (Chapter 15).

- **Bounds that were wrong on some platforms only.** The hyperbolic functions, taken from the libm of the system: the libms of glibc 2.31, musl and MinGW-w64 are sometimes more than one double from the exact value; they are now those of CORE-MATH, correctly rounded, on every platform. [GAOL v5]

The numbers read from text: the C runtime of Windows and musl on 64-bit ARM round `strtod()` to nearest in every direction, and `interval("0.1")` did not enclose 1/10 there; numbers are now read exactly, and intervals written in decimal are rounded outward by GAOL rather than by the C library. [GAOL v5] `mid()` with Visual C++ (`/O2 /fp:strict`), which rewrote $(-0.5)x + y$ into $y - 0.5x$: `mid([2^-1074, MAX])` was wrong in the seven Release jobs of Visual Studio. `mid()` on 64-bit ARM with autotools and meson, which did not give `-ffp-contract=off`: the compilers fused its multiplications and additions, and it returned a single double; the three builds now give the same flags, which the audit jobs check on each kind of machine. Square roots with Visual C++ for 32-bit x86, whose `sqrt` rounds to nearest in every rounding direction.

- **Operations that were slow on some platforms only**, which the table of `gaol_performance` in the summary of each Release job showed. Reading the rounding direction before each operation made `x + y` take 800 ns under Rosetta 2 and 82 to 98 ns with 32-bit Visual C++: an addition shows whether it is upward instead. `fesetround()` cost 130 ns with mingw-w64 13, and 50 to 250 ns with Visual C++: on x86 processors the control registers are written directly. `-ffloat-store`, given to GCC on every target, made `x * y` take 18.3 ns rather than 4.9 on x86-64: it is given only where doubles are still computed on the x87 unit.

- **Crashes and builds that failed.** `configure` and meson did not install the headers for MinGW and Visual C++, and meson did not compile on Linux. The agreement jobs passed while a build had not configured in the containers, a pipe hiding the failure: they now fail, and print the logs. `configure --enable-debug` did not compile, the Debug jobs of CMake now define `GAOL_DEBUGGING` too, and the test `debugging` compiles that code. `minimum()` and `maximum()` of two zeros of different signs gave $+0$ and -0 with Visual C++ in Release, rather than -0 and $+0$, which the unit test `float_functions` checks, built with Visual C++ since the tests of `check/` are in `tests/`. A shared `libgaol` built by `configure` or meson did not export the classes and functions of the expressions, and a program building one did not link with it: the shared library jobs of the three builds, whose tests link with the installed `libgaol.so`, build `tests/expressions.cpp`.

- **Reports of the sanitizers that were suppressed at first**, and are fixed now: the nodes the parser did not free. Nothing is suppressed any more.

14.5 The scripts of the repository

Every script of the repository starts with a comment that says how to run it and what it needs. Tables 14.5 to 14.7 give what each one is for, so that the right one is found without opening them all; the scripts of the comparisons with the other libraries

(doc/compare/code/, doc/compare/itf1788/, doc/compare/tang2021/) are described with the comparisons (see Section 12.5, page 237 and Section 12.6, page 251). The files taken from other projects, which GAOL does not change nor document one by one, are the scripts of the autotools at the root (config.guess, config.sub, configure, depcomp, compile, install-sh, ltmain.sh, missing, test-driver) and the tools and the CI of CORE-MATH under 3rd/math-core/.

TABLE 14.5 – The scripts of the generated files, the manual and the coverage

Script	What it is for
gaol/parser/regenerate_parser.sh	Regenerates the lexer and the parser from the .lpp and .ypp sources (flex 2.6.4, bison 3.8.2), after any change of them (make parser)
gaol/float/gaol_generator_mul_i2f.py	Writes the 256 cases of the multiplication of interval2f by the signs of the operands: a tool for the developers (Python 3 and ply), which no build runs
manual/build-pdf.sh	Builds the manual v5 with pdflatex, in manual/Manual_GAOLv5.pdf; make pdf and the target pdf of CMake and meson call it
manual/check_examples.py	Compiles and runs each example of the manual v5 with an installed GAOL, and compares what it prints with what the manual shows
coverage/coverage_summary.py	Reads the JSON of gcovr and writes coverage/README.md: lines and branches by file, GAOL apart from CORE-MATH, and the lines never run
cmake/gaol_version.cmake	gaol_read_version() reads VERSION.txt before project(), as configure.ac and meson.build do
cmake/gaol_package_source.cmake	Included by CPack: brings the name and the version of the archive of the sources up to date after a change of VERSION.txt

TABLE 14.6 – The audit of the three builds (.github/audit/) and the checks of the continuous integration (.github/scripts/)

Script	What it is for
audit/audit.sh	Configures GAOL with CMake, autotools and meson, and writes a JSON file of what each sees (with probe_build.py)
audit/probe_build.py, audit/make_probe.py	Read the compiler flags and the configuration GAOL sees in the three builds, for a C++ source and for a C source of CORE-MATH, through a file that shows the configuration once preprocessed

Table 14.6, continued

Script	What it is for
`audit/compare.py`	Compares the macros and the flags of the three builds, and with `--check` fails if they differ
`audit/logs.sh`	When the audit fails, prints the end of the logs of the three builds, then exits with the same status
`scripts/tests.sh`	Builds the tests with a GAOL installed by autotools or meson (static or shared), then runs them
`scripts/headers.sh`	Checks that each installed header compiles alone in C++11 with `-Wall -Wextra -Wold-style-cast -Wundef -Werror`
`scripts/check-intervals.sh`	Fails unless the build computes with the expected intervals (NEON, SSE2 or FPU)
`scripts/check-gcc-windows-fma.sh`	Checks that the objects of `gaol_interval_core.cpp` and of the `cos` of CORE-MATH built by GCC for Windows hold fused multiply-add instructions, and that no object names a 256-bit or 512-bit register (an error on x64, GCC bug 54412; a notice on x86)
`scripts/comma-locale.sh`	Generates `fr_FR.UTF-8` with `localedef` before the tests (`generate`), and checks after them that `numbers` ran checks under a locale writing a decimal comma (`check`), so that `numbers` does not pass without having tried a locale writing a decimal comma
`scripts/sde-tests.sh`	Runs the tests that compute with intervals (all but `numbers`, `core_math` and `u128`) under Intel SDE, which emulates AVX-512: the runners of GitHub for Windows may have none
`scripts/version-file.sh`	Checks how `configure`, meson and autoconf read `VERSION.txt` written in several ways
`scripts/performance-ab.sh`	Builds and measures two builds in turn in the same job: the base and the head of a pull request, or one source with two options
`scripts/compare-performance.py`	Compares two series of measures (median, ratio, spread) and writes a Markdown table for the summary of a job
`scripts/test-performance-tools.py`	Tests `compare-performance.py` and the name of the processor, in Python alone and without a build

TABLE 14.7 – The scripts of the tests (`tests/`) and their tools (`tests/tools/`)

Script	What it is for
`elementary_values.py`	Writes `elementary_values.h`: for each $f(x)$, the greatest double below and the least double above, computed with mpmath, for `elementary`
`extended_precision_values.py`	Writes `extended_precision_values.h`: the bounding values and the nearest double, to compare CORE-MATH and GAOL
`compoundm1_values.py`	Writes `compoundm1_values.h`: the doubles around $(1+x)^y - 1$ at pairs, the hull over boxes, and $\log(1+x)$ and $e^t - 1$ to 256 bits, with mpmath and fractions
`reverse_values.py`, `atan2_rel_values.py`, `pow_rel_values.py`	Write the cases of the reverse functions of IEEE 1788-2015, of atan2Rev1 and atan2Rev2, and of powRev1 and powRev2, for `reverse`
`refused_options.sh`	Checks that autotools and meson refuse `-ffast-math` and `-ffinite-math-only`, and the SSE2 intervals of 32-bit Windows without `-faligned-new=8`, as the tests `refused_*` and `refused_aligned_new*` of CMake do
`release_flags.py`	Checks that a project which brings GAOL in without a build type compiles it in release (`-O3`, `NDEBUG`, `GAOL_DEBUGGING` at 0)
`meson_subproject/check_flags.py`	Checks that a meson project which takes GAOL as a subproject gets the flags of its `gaol.pc`
`compile_error.cmake`	A `cmake -P` script that compiles a source whose compilation has to fail (`refused_*`, `nodiscard_discard_cxx*`)
`cpack_stale_configure.cmake`, `automatic_configuration.cmake`, `version_file.cmake`	The three tests of the CMake build that are scripts (see Section 14.2, page 274)
`tools/arith/diffarith.py`	Differential test of the arithmetic: the bounds and the exception flags of `+ - * / %`, `sqr`, `sqrt`, `inverse`, `div_rel` and `mid` on a grid of boxes, under the four rounding directions, with two builds of GAOL
`tools/pow/diffpow.py`	Differential test of `pow`: the bounds of `gaol_ieee1788::pow` and `gaol::pow` with two builds, on 70 500 boxes and under the four directions
`tools/pow/build.py`	Compiles a program of `tests/tools/pow` against a build of GAOL, with the flags that build gives its own tests

Table 14.7, continued

Script	What it is for
`tools/pow/negative_power_rows.py`	Writes the rows of the table of large negative powers of `tests/arithmetic.cpp`: the tightest bounds of x^n for n from $-2^{31}+1$ to -2^{27}, computed with mpmath on 2000 bits
`tools/pow/gen_table.py`, `tools/pow/checkrows.py`	Read and rewrite the table of boxes of `ieee1788`, and check its bounds against the exact power (mpmath, 500 bits)
`tools/pow/mutate.py`, `tools/cos/mutate.py`, `tools/x86-csr/mutants.py`	Mutation testing: one deliberate error in `pow` and the integer powers, in the port of `cos.c` to the 128-bit integer, or in the writes of the control registers of x86, and the tests have to fail
`tools/pow/pown_bound.py`	Checks with rationals the bound of the error of the integer powers computed beforehand
`tools/pow/bench.cpp`, `tools/pow/pownhash.cpp`, `tools/pow/pownedge.cpp`	The time per call of each path of `pow`, and a hash of the bounds of four million integer powers, the sign of a zero included
`tools/cos/accurate_values.py`	Computes the roundings of $\cos x$ with mpmath and checks the table of the accurate path of `core_math`
`tools/asm-order/check_order.py`	Reads the assembly of the library (x86-64, GCC and Clang) for an instruction that rounds on the wrong side of a write of the control register (GCC bug 34678), and for the conversion of a subnormal constant through the x87 unit
`tools/ibex_codac/*.py`	The cases of the tables of `test_ibex_codac`, their exact results (rationals, mpmath), and the check that the rows of the test are those the cases give
`tools/x86-csr/amalgamate.py`	One source file from three units of GAOL, to compile with a compiler that takes one file, such as those of Compiler Explorer

The workflows are read by actionlint, which runs `shellcheck` on the steps that have a script, and reports nothing; the scripts of `.github/` and of `doc/compare/code/` are read by `shellcheck`, run from the root of the repository, which reports nothing either: the lists of flags that the shell has to split carry a comment with the reason. [GAOL v5]

14.6 Coverage

The option `GAOL_COVERAGE` of CMake (`--enable-coverage` of `configure`, `enable-coverage` of meson), off by default, compiles GAOL and its tests with the counters of gcov, and adds the target that runs the tests and writes the report of their coverage, `coverage/coverage.html` and the page of conclusions

`coverage/README.md`: `cmake --build <build> --target coverage`, `make coverage`, or `ninja gaol-coverage`, meson keeping `coverage` for its own report (see Section 2.5.2, page 36). [GAOL v5] It needs GCC or Clang, gcovr and Python 3, and the tests (`WITH_TESTS`); it should be the Debug build: with the Release build, the optimizations move the code around, the counts no longer follow the source, and the build warns. The sources of GAOL (`gaol/`) are counted apart from those of CORE-MATH (`3rd/math-core/`), whose accurate phases are reached by a handful of arguments in a million: counting them together would say nothing of either. Table 14.8 gives the report of the run of 4 November 2026.

Table 14.8 – The coverage of the tests on 4 November 2026

	Lines	Covered	Not run	%
GAOL (`gaol/`)	5466	5153	313	94.3
CORE-MATH (`3rd/math-core/`)	8172	6732	1440	82.4

The tests run 94.3 % of the lines of GAOL, above the 80 % GAOL v5 aims at, and 66.5 % of its branches are taken both ways. The lines of CORE-MATH that are never run are its accurate phases, which a handful of arguments in a million reach, and the paths of the architectures the machine is not: they are counted among the lines not run, and are checked otherwise: the accurate path of `sin` is compared with the upstream sources compiled unchanged, and that of `cos` with its roundings computed by mpmath (see Section 14.2, page 274), and the other accurate phases are reached by the tests of their bounds only. File by file, the least covered are `gaol/parser/gaol_expr_visitor.h` (9 of 99 lines, 9.1 %), `gaol/parser/gaol_expr_eval.h` (87.3 %), `gaol/parser/gaol_expression.cpp` (89.6 %) and `gaol/core/gaol_interval_sse.cpp` (93.9 %); `gaol/core/gaol_interval_core.cpp`, the largest file, with 2492 lines counted, is covered at 97.0 %, and 18 files of the 26 at 100 %. The page lists the lines the tests never run, file by file.

14.7 Discussion

The tests of GAOL v5 compare its bounds with references that owe nothing to the library nor to the floating-point environment, and run every operation in every state a program may leave the processor in: the four rounding directions, the modes that flush the subnormal numbers, the exceptions enabled, a locale with a decimal comma, and the initialization of static objects before `main()`. The programs of `tests/` hold hundreds of thousands of such checks, and the documentation records, for many of the changes, how many of the checks written with them fail without them, and the example programs check what they print, as do the examples of this manual. The continuous integration runs the suite on three operating systems, seven processor architectures and the compilers of each, with the three builds, and refuses

the configurations whose bounds it found wrong or uncertified, most of the refusals being checked by a job of its own.

Three choices of the tests and of the continuous integration deserve a word. The teeth of some checks depend on the processor the code is compiled for: a loop that a compiler vectorizes raises the invalid-operation flag with one set of instructions and not with another, so that the jobs on many processors are the test, not one of them. The speed of a pull request cannot be read from the times of a shared runner, which change by several times between two runs of the same code: it is measured by building the base and the head in the same job and running them in turn, and only a ratio of 1.5 or more, or of 2/3 or less, beyond the spread of the runs, means something. The coverage report counts the lines of GAOL apart from those of CORE-MATH, whose accurate phases a handful of arguments in a million reach; `core_math` checks the accurate paths of `sin` and `cos` against values computed apart instead.

Some checks are deferred or left out, for the reasons the documentation gives: the reading of a text of 512 MiB, which found a 32-bit exponent overflow, has no test, since the text is read through two more copies of it; the checks of several threads reading or writing at once are commented out, the tests running no thread; the refusal of the mingw-w64 of ARM whose `fma()` is wrong is not tested; Debian 11 is left out of the continuous integration, and the intervals of floats are compiled by none of the builds. The next chapter turns to the programs written for GAOL 4: how to compile and link them with GAOL v5, which results change, and the register of the differences by theme.

Chapter 15

Moving a program from GAOL 4 to GAOL v5

GAOL v5 continues the GAOL 4.2.2 of Frédéric Goualard, and most programs written for GAOL 4 compile and run with it once five things are done: the namespace is opened, the strings they gave to `interval(const char*)` are read by `textToInterval()`, the relations `==` and `!=` on intervals are rewritten with `set_eq()`, `set_disjoint()` or `!set_disjoint()`, the handlers of the exceptions of GAOL 4 are retargeted on those of the standard library, and the conversions of a `double` or an integer to an interval, explicit now, are written out. The other changes concern what a program sees: tighter bounds, the empty set where GAOL 4 built an interval, the comparisons of IEEE 1788-2015 [9], and texts read and written otherwise. Each change was made for a reason, and most of them were measured.

In this chapter, we gather what a program may have to change and what it sees differently. The first section lists what concerns the compilation and the link, the second the results that change. The third section is the register of the differences, theme by theme: for each change, what GAOL 4 did, what GAOL v5 does, why, and what a program sees, with the measurements that justified it and the chapter of this manual that describes the new behaviour. The last section discusses the choices made.

A word on the versions. "GAOL 4" is the 4.2.2, which GAOL v5 continues; the benchmarks of Chapter 12 use the 4.2.3, the last version of Frédéric Goualard, which came after it. The releases 4.3.0 to 4.3.2 of the repository of GAOL v5 came next and are not GAOL 4: they already had `rnd_keep()`, `get_rounding()` and `MEMFREE()` (`GAOL_MEMFREE()` now), which neither the 4.2.2 nor the 4.2.3 has. The meson build that this chapter counts with GAOL 4 is the one added to the repository of the 4.2.3 in November 2025, after its last release: the 4.2.2 had `configure` only. GAOL 4 also had classes of exceptions of its own, `input_format_error`, `invalid_action_error` and `unavailable_feature_error`, derived from `gaol_exception`. GAOL v5 throws the `std::invalid_argument` and the `std::domain_error` of the standard library where GAOL 4 threw the first two, and has none of these classes

(see Section 13.5.1, page 266). Where this chapter says what GAOL 4 threw, it gives the class of GAOL 4.

15.1 Compiling and linking

The changes below are the ones a compiler or a linker reports, and those of the initialization and the cleanup, which a program makes in its text. Most of them are described in the section the cross-reference names.

- **The namespace.** `gaol/gaol.h` no longer opens the namespace `gaol`: a program adds `using namespace gaol;`, or names `gaol::interval` (see Section 3.4, page 69). The type and the functions are in `gaol_core`, and the symbols of the library name it: a program compiled against the headers of GAOL 4 has to be compiled again.

- **The names GAOL keeps for itself.** `gaol` holds the names GAOL 4 had there that GAOL v5 still has, except those GAOL uses for itself, which a program names in `gaol_core` (see Section 3.4, page 69): `gaol_signbit()`, `uintdouble`, `ullidouble`, `Interval_struct`, `the_null_expr`, `prec_t`, `modulo_k_pi()` and the `round_*_sse()`, all of them internals that the manual of GAOL 4 did not document, and, of the 4.3 releases of GAOL v5, `rounding_state`, `get_rounding()`, `set_rounding()`, `round_upward_if_needed()` and `gaol_set_rounding_x86()`.

- **Visual C++.** The code using GAOL need no longer define `__GAOL_PUBLIC_` empty: GAOL is a static library there, and its declarations carry no `__declspec(dllimport)`. A definition left in a project changes nothing. The macro is `GAOL_PUBLIC` now.

- **The allocator.** The allocator of containers `aligned_allocator` is gone, with its header, `gaol/gaol_allocator.h`.

- **One library to link.** GAOL is the only library to link: the mathematical library GAOL 4 was built with is no longer used, CORE-MATH being compiled into GAOL, and the options of the builds that chose it are gone.

- **The flags of interval arithmetic.** The code including the headers of GAOL is compiled with the flags of interval arithmetic, which `gaol::gaol` and `gaol.pc` give (see Section 3.3, page 64); `gaol/gaol_config.h` refuses the options and the compilers that give wrong bounds (see Section 2.3.2, page 25).

- **The intervals of floats.** `gaol::intervalf` and `gaol::interval2f` are compiled by none of the builds, which have no option for them: only a developer of GAOL compiles them, defining `GAOL_FLOAT_INTERVALS` in `gaol/gaol_config.h`.

- **The bounds at doubles.** The functions of doubles rounded downward and upward, `exp_dn()`, `sin_up()`, `ipow_up()`, `ipow_dn()`, `pow_up()`, `pow_dn()` and the others of `gaol/gaol_double_op.h`, are no longer part of the interface: the header is no longer installed. `uipow_upup()` and `uipow_dnup()` are no longer declared.

- **The static constants.** The static intervals `interval::cst_one`, `cst_pi` and the other `cst_` members are gone: `interval::one()`, `interval::pi()` and the other functions give these constants, right even in the initialization of a static object (see Section 4.10, page 112).

- **The initialization.** `gaol::init()` need not be called again after the program changed the rounding direction: each operation sets it upward itself (see Section 4.1, page 88). It need not be called at all, GAOL initializing itself with every compiler, Visual C++ included (see Section 4.10, page 112).

- **The cleanup.** The program calls `gaol::cleanup()` right after its last use of GAOL: unless the rounding direction is preserved, it sets back the rounding direction that the initialization found, which GAOL 4 left upward when a static object of the program had computed before its initialization (see Section 4.10, page 112). It no longer frees what the initialization allocated, which the automatic cleanup frees when the program ends.

- **The relations.** The operators `==` and `!=` on intervals are gone, with the possibly relations (`possibly_eq()`, `possibly_le()`...), `certainly_eq()` and `certainly_neq()`, and so is the option `--enable-relations` of `configure` and meson: a program compares intervals with `set_eq()`, `set_disjoint()` and `!set_disjoint()`, and `<`, `<=`, `>` and `>=` are the certainly relations (Chapter 6). The new relations do not give the results of the old operators: `==` was `certainly_eq()` in every build by default, true for two empty sets and for a point interval equal to the upper bound of the other interval (`[4] == [3, 4]` was true, `[3, 4] == [4]` false), and `!=` was its negation, so that `==` of $[3, 4]$ and $[3, 4]$ was false, where `set_eq()` is true, and `!=` was true, where `set_disjoint()` is false. A program chooses the relation it means.

- **The constructors from strings.** The constructor from one string, `interval(const char*)`, is gone, and with it the conversion of a string wherever an interval is expected: `textToInterval("0.1")` reads a string in its place, and throws `std::invalid_argument` where the constructor threw `input_format_error`; `x = "[1, 2]"` becomes `x = textToInterval("[1, 2]")`, and `x + "0.1"` becomes `x + textToInterval("0.1")` (see Section 5.1, page 117). The constructor from two strings is gone too: `interval("0.1", "0.3")` becomes `textToInterval("0.1", "0.3")`, which gives the empty set for bounds in the wrong order. `interval(0)` and `interval(0, 0)`, which these constructors made ambiguous, compile.

- **The constructor from a double.** `interval(double)` is explicit, and so is the constructor from an integer: `interval x = 0.5;`, `interval x = 5;`, `return 0.5;` in a function returning an interval, `min(x, 0.5)` and `interval r = sqrt(2.0);` do not compile, where GAOL 4 converted the double without a word. `x = 0.5` and `x < 0.0` still compile (see Section 5.1, page 117).

- **The exceptions.** The classes `gaol_exception`, `input_format_error`, `invalid_action_error` and `unavailable_feature_error` are gone, with `gaol/gaol_exceptions.h`: a handler names `std::invalid_argument` or `std::domain_error` (see Section 13.5.1, page 266).

- **The headers.** `gaol/gaol_profile.h`, `gaol/gaol_parameters.h`, `gaol/gaol_limits.h`, `gaol/gaol_assert.h`, `gaol/gaol_exact.h` and `gaol/gaol_flags.h` are gone, and so are `gaol/gaol_config_msvc.h`, `gaol/gaol_config_mingw.h`, `gaol/gaol_version_msvc.h`, `gaol/gaol_version_mingw.h` and `gaol/gaol_flt_output.h` (the source `gaol/dllmain.cpp` too); `gaol/gaol_init_cleanup.h` is no longer installed, and a program that builds expressions includes `gaol/parser/gaol_expression.h`, with `gaol/parser/gaol_expr_eval.h` to evaluate them, where it included `gaol/gaol_expression.h` (see Section 15.3.12, page 339).

- **The expressions.** `eval_stack` and `expr_eval` are no longer copyable, the stack of an evaluation growing in the heap (see Section 15.3.10, page 336).

- **The debugging macro.** `GAOL_DEBUG(level, command)` of the debug build is `GAOL_DEBUG_VERBOSE`: the old name is not kept (see Section 13.6, page 269).

- **The options of the builds.** The option `with-test` of meson is `with-tests`. `--enable-asm` of `configure` and `enable-asm` of meson are gone, GAOL v5 no longer calling the assembly they chose: `configure` warns of an option it does not know, and `meson setup -Denable-asm=true` stops with an error (see Section 15.3.13, page 340).

15.2 Results that change

The changes below are the ones a program sees at run time without any change of its text. The third section gives their reasons.

- **Static objects.** The intervals computed in the initialization of the static objects of a program linked with the static library GAOL are right: `interval::pi()` was $[-0,0]$ there, and $[1,2]/[-1,1]$ was $[-0,0]$ rather than $[-\infty,+\infty]$, the constants of GAOL being computed after these objects, except with MinGW-w64 (see Section 4.10, page 112).

- **The bounds are tighter.** The elementary functions give the tightest bounds, rather than one double beyond, and are exact where their value is a double; the integer powers are computed from exact products, and the nth roots are proved with powers rounded upward and downward; `sin`, `cos` and `tan` are within one double of the tightest bounds at every magnitude (Chapter 11).

- **The empty set.** The constructors give it for $[+\infty, +\infty]$, $[-\infty, -\infty]$, bounds in the wrong order and NaN bounds, and assigning an infinity to an interval empties it (see Section 5.1, page 117); the intersection of disjoint intervals is it, with NaN bounds; so are `log` of an interval holding no positive number, `invabs_rel(J, I)` for a negative J and `div_rel` with no solution in I. The operators of an interval and an infinite `double` (`x * INFINITY`, `x + INFINITY`) give it too. `width()` of the empty set is NaN rather than -1.

- **Powers.** `pow(x, p)` for a double p no longer truncates p; `pow(x, y)` keeps the negative part of x for an integer exponent only, and takes the `pow` of IEEE 1788-2015 otherwise (see Section 7.2.2, page 151). `nth_root` of an odd order is defined on the whole real line, and takes a negative order.

- **The comparisons** follow IEEE 1788-2015 for the empty set and the infinite bounds: `certainly_leq()` and its companions are true when either interval is empty, and `set_strictly_contains()` takes an infinite bound as beyond the same infinite bound (Chapter 6).

- **Input and output.** Numbers are read exactly, and the literals of IEEE 1788-2015 are read, in any case of letters; `"[inf]"` is the empty set. Every string is an expression, where an interval may stand wherever a number may (`"1+[1,2]"`), an uncertain number being no bound (`"[5?1]"` is refused) and needing a blank after a sign next to it (`"10-3.56?1"` is refused, `"10 - 3.56?1"` is read), and an error anywhere stops the reading. The reader knows every function of GAOL, and `gaol_ieee1788::textToInterval` reads the names of IEEE 1788-2015 (`pown`, `rootn`...). The format `hexa` writes interval literals, which are read back bit for bit; the decimal formats round the bounds outward whatever the C library, and the format `agreeing` keeps the digits of each bound (Chapter 9).

- **New operations.** `atan2()` is implemented, and the functions of IEEE 1788-2015 GAOL did not provide are added (`exp2`, `hypot`, `sign`, `trunc`..., Chapter 7).

15.3 The differences, by theme

This section condenses the register of the differences between GAOL 4 and GAOL v5, theme by theme. The themes are those of the register, regrouped: the elementary functions first, then the rounding direction, the floating-point exceptions, the

constants, the kernels, the intervals and their functions, the powers and the roots, the other functions, the relations, the reading and the writing, the errors, the interface and the builds. Some of the defects told below were those of the earlier versions of GAOL v5 only, and the text says so. Each measurement is given with the machine and the compiler it was made on wherever the register gives them. The timings are those of the step that made each change, which may be an intermediate one; those of the release against GAOL 4.2.3 are in Chapter 12.

15.3.1 The elementary functions and CORE-MATH

Every elementary function is bounded with CORE-MATH [13], on every architecture and with every compiler, and it is the only library GAOL takes its elementary functions from. [GAOL v5] The other mathematical libraries GAOL 4 could be built with are gone, with their sources, their headers and the scripts that installed them, and so are the options of the builds that chose one: there is nothing to choose, to find, to build apart or to link along with GAOL, whose library holds the functions (see Section 1.7, page 15). No build names `-lm`: the C library still gives GAOL `sqrt()`, `floor()`, the functions of `<fenv.h>` and, where the processor has no fused multiply-add instruction, `fma()`, but `g++` and `clang++` link the math library themselves, so that the autotools build (whose `LDADD` was `-lm` in GAOL 4), meson and the `gaol.pc` the builds install do not name it, as the CMake build never did.

The bounds are the tightest ones rather than one double beyond. CORE-MATH is correctly rounded in the rounding direction in effect, so GAOL takes the value at a bound, computed in the upward rounding it keeps, as the upper bound, and the double below it as the lower one. GAOL 4 took values correctly rounded to nearest only, so each bound had to be moved one double outward, and the direction had to be set to nearest and back around every call. On an Intel i7-1185G7 with Clang 18, the bounds of `log` took 30 ns rather than 62. The accurate phases of `log`, `sin`, `cos`, `tan`, `atan2` and `pow` compute with a 128-bit integer, which Visual C++ has on no architecture and GCC has on no 32-bit target: `gaol/math/gaol_u128.h` gives them the type of the compiler where it has one and a structure of two 64-bit halves where it has none. The two paths were compared with the pristine upstream sources over about 40 million arguments in the four rounding directions, and give the same bits; a job of the continuous integration builds the whole of GAOL with the halves forced (`-DGAOL_U128_EMULATION=ON`, in `linux.yml`; see Section 14.4, page 295).

MinGW-w64 is no longer refused for its version alone, both former reasons being gone: the hyperbolic functions of its math library, which GAOL no longer uses, and the cost of its `fesetround()`, which GAOL no longer calls for its elementary functions. [GAOL v5] Still refused are the MinGW-w64 whose `fma()` or `round()` is wrong, those of its own math library, computed in doubles: that `fma()` is not correctly rounded (10.7 % of the error-free products `fma(a, b, -a*b)` were wrong with mingw-w64 11 on x86-64), and CORE-MATH computes with it wherever the

compiler has no `-mfma`; that `round()` depends on the rounding direction (see Section 2.3.2, page 25).

The functions of IEEE 1788-2015 that GAOL 4 did not provide are added. [GAOL v5] `exp2`, `exp10`, `log2` and `log10`, required among the forward elementary functions (Table 9.1 of the standard), are bounded with CORE-MATH's, the tightest bounds, and the value that is a double is kept as a bound rather than moved: $2^3 = 8$, 10^{22}, $\log_2(1/4) = -2$ and $\log_{10}(100) = 2$ are exact. Sixteen functions of Table 10.5, which the standard recommends, follow: `expm1`, `exp2m1`, `exp10m1`, `log1p`, `log2p1`, `log10p1`, `hypot`, `rsqrt`, `sinpi`, `cospi`, `tanpi`, `asinpi`, `acospi`, `atanpi`, `atan2pi` and `compoundm1`. `sinpi`, `cospi` and `tanpi` find their extrema and poles exactly, at the multiples of 1/2, which are doubles: `sinpi([1e17])` is 0, where `sin(pi*x)` gave $[-1, 1]$, the width of π times 10^{17} being more than a period. $\mathrm{compoundm1}(x, y) = (1+x)^y - 1$ is the one CORE-MATH has no binary64 version of, and $\exp(y \log1p(x)) - 1$ computed with doubles is up to 1 100 doubles from the value: it is computed with integers only, in 64-bit fixed point where that proves the bounds (95 % of the pairs for $|y \log1p(x)|$ below 0.35, in 0.1 μs a corner) and with a 128-bit significand in balls otherwise (0.6 to 0.9 μs). fma, cancelMinus and cancelPlus (`cancel_minus` and `cancel_plus`), which the standard requires the tightest, and less, strictLess, isEntire and isCommonInterval are provided too. The error function `erf` and its complement `erfc`, which the standard does not name, are bounded with CORE-MATH's as well, exact at 0 and at the infinities. The integer functions `sign`, `trunc`, `round_ties_to_even` and `round_ties_to_away` are exact, each being non-decreasing; they take nothing from CORE-MATH, and `round_ties_to_even` reads the bits so as to ignore the rounding direction, which `std::nearbyint` and `std::rint` read (Chapters 7 and 8). The namespace `gaol_ieee1788` gives the operations of the standard GAOL provides, under the names and in the argument order of the standard (Chapter 10).

CORE-MATH reads the rounding direction as GAOL sets it. [GAOL v5] GAOL 4 had no CORE-MATH, and the defects below were found and fixed in the earlier versions of GAOL v5 that took it. Some sources of CORE-MATH round a result themselves, in the direction they read, and two of the ways they read it gave another direction than that of the doubles GAOL computes, the one of MXCSR. `pow` rounds a subnormal result in the direction `fegetround()` gives, which the C library of Linux reads from the x87 unit: with the x87 unit to nearest and MXCSR upward, the upper bound of `pow` was below the exact value for about half of the arguments with a subnormal result. GAOL gives the sources a `fegetround()` that reads MXCSR on x86. On 32-bit x86 that spares `cbrt` and `asinpi` a call of the C library at each bound, about 80 ns a call with Visual C++ for x86: on the runners of the continuous integration, `nth_root(x, 3)` takes 179 ns rather than 355 and `asinpi(x)` 794 ns rather than 1005 with Visual C++ for x86, and 120 ns rather than 324 and 255 ns rather than 438 with clang-cl for x86. `cbrt`, `rsqrt` and `asinpi` read the rounding field of MXCSR and turned it into the `FE_*` value of the C library by a shift chosen from the macros of the platform; with clang-cl and with Cygwin on x86-64, whose newlib gives `FE_*` the values 0 to 3, the shift was wrong for the three, and the

directed roundings were taken for toward zero (`asinpi` was wrong for about half of its arguments with clang-cl). The field is 0 to nearest, 1 downward, 2 upward and 3 toward zero on every x86 processor, and the three functions now take it so, whatever the values of the C library.

CORE-MATH's `pow` is correctly rounded in every direction. A test of `pow` where x^y is a double found three defects in the earlier versions of GAOL v5. With Visual C++, `NAN` and `INFINITY`, which the UCRT writes for C as products of `1e+300`, were computed when the program runs, in the rounding direction in effect, so that `pow` rounded downward was -0 from 2^{-1075} to about 2^{-947}; they are read from their bits now. On 32-bit x86 the square root of the C libraries of Windows rounds to nearest in every rounding direction: CORE-MATH takes its square root from the SSE2 instructions there. On 32-bit ARM, the conversion of a 54-bit integer rounded to nearest, so that the upper bound fell below x^y at a midpoint. Moreover, for $y \log x$ from 709.78263 to 709.78267, a power between $1.7975 \cdot 10^{308}$ and $1.7976 \cdot 10^{308}$, the first phase of `pow` computed $\infty - \infty$ before its second phase gave the right result, and a program that had enabled the invalid-operation exception died on SIGFPE: the vendored source sends those arguments to the second phase. The source of `cospi` shifted a signed integer out of its range, undefined behaviour that the tests run with UBSan reported; it shifts it unsigned now.

The cube root `nth_root(x, 3)` is CORE-MATH's `cbrt`, the tightest bounds, exact where the cube root is a double, which cubing the value tells: it takes 41 ns, where the search by bisection the other roots use takes 89 (the medians of three runs, Intel i7-1185G7, Clang 18.1; 24 ns against 77 on an Apple M2, Apple clang 21). [GAOL v5] `nth_root(x, q)` takes a negative q, which the standard recommends (`rootn` over $\mathbb{Z} \setminus \{0\}$): the reader of strings converted the exponent to an unsigned int, and `nth_root(16, -2)` read from a string was the 4294967294-th root of 16 rather than [0.25, 0.25]. The second argument of `nth_root`, `pown` and `rootn` read in a string may be any integer a double holds: `nth_root(8,3000000000)` threw `invalid_action_error`, and `rootn(8,3000000000)` read with the names of the standard was the empty set in the earlier versions of GAOL v5, where the same call from C++ is computed. The reader of strings also takes the new names `exp2`, `log2`, `cbrt`, `sign` and `trunc`, which gave a syntax error before (see Section 9.1.1, page 184).

15.3.2 The rounding direction and the initialization

The rounding direction is set upward by each operation that computes a bound when it is not, in every build; the operations of the AVX-512 path set none, their instructions carrying their rounding. [GAOL v5] An operation that rounds nothing (`-x`, `abs()`, `min()`, `floor()`, the predicates) or returns before its check (`y * empty`, `exp(empty)`) leaves the direction as it found it (see Section 4.1, page 88). GAOL 4 has no CMake build; the one of the 4.3 releases of GAOL v5, which derives from that of IBEX, set it once, in `gaol::init()`, and computed in the direction the

calling code left: rounding to nearest, downward or toward zero, 30 operations gave other bounds, and sums and products did not enclose their exact values (3199 of the 13825 checks of the test of the rounding direction failed with GCC 9.4 on x86-64). Built with `configure` or meson, whose option `--enable-preserve-rounding` is on by default in GAOL 4 although its manual says the contrary, GAOL 4 restored the rounding direction after each operation, but only the x87 control word on x86-64, where GAOL computes with SSE (43 operations left the SSE instructions rounding to nearest or upward, whatever direction they found), and without the rounding bits of the FPCR register on arm64.

With the option `GAOL_PRESERVE_ROUNDING` of the three builds (`--enable-preserve-rounding` of `configure` and `-Denable-preserve-rounding` of meson, which GAOL 4 had), GAOL saves and restores the direction by reading and writing the control registers itself, a register only if it differs from what is set back: the x87 control word and MXCSR of an x86 processor, FPCR with the NEON intervals, FPSCR with the FPU intervals of 32-bit ARM processors and GCC; the other processors keep `fegetround()` and `fesetround()`. Each value computed before the rounding direction changes is written to a `volatile` variable: GCC does not implement `#pragma STDC FENV_ACCESS`, and moved the computation of the midpoint after `fesetround()` (GCC bug 34678). [GAOL v5] Whether the direction is upward is shown by an addition, $1 + 2^{-60}$ with 2^{-60} read from `volatile` memory, wherever doubles are computed in double precision; on x86-64 processors and, with GCC and Clang, on ARM ones it is $1 + (2^{-1060} + 0)$, which shows the modes that flush the subnormal numbers to zero as well. The NEON intervals of 64-bit ARM processors read the register FPCR instead, and a 32-bit x86 processor the control registers of the two units that round. Reading the direction with the SSE register and `fegetround()` rather than by an addition made `x + y` take 800 ns rather than 2.8 ns under Rosetta 2, 82 to 98 ns rather than 2.6 to 4 ns with 32-bit Visual C++, and 48 ns rather than 6.4 ns with 32-bit MinGW-w64 15.2; and Clang 18 read MXCSR once for a whole loop that changed the rounding direction.

The direction is checked once per operation, but for two cases told below. [GAOL v5] The bounds of the elementary functions at doubles are computed by functions that take the direction to be upward already, as it is after the check of the function of intervals calling them: in the earlier versions of GAOL v5, `exp()` of an interval checked the direction three times, `sin()` and `cos()` up to four, the other elementary functions and `pow(x, y)` two or three. `exp()` takes 5.7 % less time, `sin()` 3.6 %, `pow(x, y)` 3.4 % (Clang 18, Intel i7-1185G7). The operations calling others check once too: in those versions, `tan()` checked six times, calling `width()`, `+` and `/` on intervals after its own check, and `acos_rel()`, `asin_rel()` and `atan_rel()` six times too; they compute on doubles after their one check, and give the same bounds, bit for bit, but for the quotients of `modulo_k_pi()`, now computed on doubles. `tan()` takes 22 % less time, `acos_rel()` 24 %, `sqrt_rel()` 29 %, `pow(x, [3])` 7 %, `nth_root(x, -2)` 13 %, and `modulo_k_pi()` 10 ns rather than 22; with `GAOL_PRESERVE_ROUNDING`, where each check saves and restores the direction, `tan()`

74 %, `acos_rel()` 83 %, `pow(x, [-3])` 20 % and `nth_root(x, -3)` 15 % (Clang 18, Intel i7-1185G7). Still checking more than once are `nth_root(x, q)` for an order beyond 64 bits, which only an `__int128` gives, and `nth_root(x, -2)` where the square root has a vector path of its own (the AVX-512 path, taken with `GAOL_PRESERVE_ROUNDING`, and the NEON intervals without it), which takes `sqrt()` and then `inverse()`, each checking, faster than one check before the scalar root. With that option, `GAOL_RND_KEEP()` of an interval keeps its two bounds through two volatile doubles with the SSE2 and FPU intervals and in the register itself with the NEON ones, where the 4.3 releases copied the interval a byte at a time into volatile memory: `pow(x, [-3])` and `nth_root_rel()` take 26 to 27 % less time, `asinh_rel()` and `atanh_rel()` 21 % (Clang 18, Intel i7-1185G7). `tan()` and `pow(x, y)` keep their bounds before setting the direction back, which they did not, so that GCC, which does not model the rounding direction, could compute them after; and the functions of doubles `exp_dn()`, `sin_up()` and the thirty others saved the direction they found but left it upward, a defect the compilers only showed by warning that a variable was set but not used. They no longer save the direction, and their header, `gaol/math/gaol_double_op.h`, is no longer installed.

The initialization and the cleanup change. [GAOL v5] GAOL initializes itself with every compiler, before the static objects of the program: each file including `gaol/gaol_common.h` holds a static `gaol_core::gaol_initializer`, whose constructor initializes GAOL the first time, as `std::ios_base::Init` does the standard streams. GAOL 4 declared `gaol_init()` with `__attribute__((constructor))`, for GCC and Clang only: Visual C++ had no automatic initialization, and the constructor of the static library ran after those of the program (except with MinGW-w64, which runs the constructors from the last linked to the first), so that the static objects of the program computing intervals set the rounding direction upward before the first `gaol::init()` read it, and `gaol::cleanup()` left it upward. `gaol_init()` is gone. `gaol::cleanup()` sets back the rounding direction that the first initialization found, unless it is preserved: GAOL 4 left it upward after its use when a static object of the program had computed before the initialization, even after its automatic cleanup, and a program computing to nearest after it had to set it back itself. `gaol::cleanup()` does so at its first call only, and an operation of GAOL after it sets the direction upward again; `gaol::restore_rounding()` sets it back as many times as the program needs. Only the rounding direction is set back, not the rest of the environment: the exception flags raised since are the program's. `gaol::cleanup()` no longer frees what the initialization allocated: it deleted the node of the empty expression while expressions still referred to it, a local of `main()` destroyed after the call or a static object, whose destructors then wrote into freed memory. The automatic cleanup frees it when the program ends or the library is unloaded (see Section 4.10, page 112). Following the patch IBEX applies to GAOL, the FPU is initialized with `fesetenv(FE_DFL_ENV)` and `round_upward()`, rather than by writing an x87 control word into `fenv_t`, which on ARM64 left the rounding to nearest and crashed an ARM64 Mac; and `interval::midpoint()` and

`operator<<` set the rounding direction back upward, which they did not when the rounding direction is not preserved.

The rounding direction is set on x86 processors by writing the control registers of the x87 and SSE units (`fnstcw`/`fldcw`, `stmxcsr`/`ldmxcsr`) rather than through `fesetround()`, which cost 130 ns per call with mingw-w64 13, 50 ns with the C runtime of Visual C++ for x64 and 250 ns for x86, 8.5 ns with glibc. [GAOL v5] GAOL 4 changed the direction four times for each elementary function of an interval, where the math library itself took about 10 ns; GAOL v5 no longer changes it there, CORE-MATH computing upward. On the runners of the continuous integration, `exp()`, `log()`, `sin()` and `cos()` of an interval took 535 to 630 ns with MinGW-w64 and MSYS2, and take 48 to 209 ns; 250 to 340 ns with Visual C++ for x64, and 45 to 160 ns; 1050 to 1750 ns with Visual C++ for x86, and 150 to 650 ns. A register is written only if its direction differs from the one set, and no other bit changes. With mingw-w64 13 and later there is a second reason: its `fesetround()` clears the six exception flags of MXCSR when it changes the register, and its `fegetround()` joins by OR the rounding fields of the two units. On a 32-bit x86 processor GAOL also reads the two registers in line, in one asm statement, rather than through `fegetround()`, about 80 ns with Visual C++ for x86. With `GAOL_PRESERVE_ROUNDING`, `x.midpoint()` takes 17.5 ns rather than 27.9 (GCC 15.2, Intel i7-1185G7), and 52 ns rather than 131 with Visual C++ for x64 on the runners of the continuous integration; the bounds are the same. The option also writes the registers only if they have to change: an operation of the SSE2 intervals read MXCSR and wrote it twice whatever it held. In a program that computes upward, `x + y` takes 1.7 ns rather than 8.2 and `x * y` 2.5 ns rather than 8.3; in a program that computes to nearest, 0.2 ns more (GCC 15.2, Intel i7-1185G7, the best of 15 runs of a loop). Visual C++ for x64 writes MXCSR alone with the option, as GCC, Clang and clang-cl do with the fused multiply-add instructions: in the earlier versions of GAOL v5, the operations that start with `GAOL_RND_ENTER()` called the C runtime, because the macros asked for `__FMA__`, which `cl` does not define; they take `__AVX2__` too now, and `pow(x, 3)` takes 29 ns rather than 92, `sin(x)` 71 ns rather than 135 on the runners of the continuous integration (AMD EPYC, Visual Studio 2026).

On ARM processors GAOL writes the control register itself too. [GAOL v5] With the NEON intervals of 64-bit ARM processors the direction is the field RMode of FPCR, read with `mrs` and written with `msr` (see Section 4.5, page 96). On 32-bit ARM processors with GCC and a floating-point unit of doubles it is the field RMode of FPSCR, read with `vmrs` and written with `vmsr`: `x + y` takes 22.5 ns rather than 49 with `GAOL_PRESERVE_ROUNDING` (the Debian armhf jobs of the continuous integration, which run on the arm64 runners of GitHub). FPSCR holds the flags of the exceptions too, so every write but the one at the start of an operation reads the register again just before, and no flag raised since is lost. Without the option nothing changes there: the check of each operation stays the addition, `vmrs` costing more than its two additions (`x + y` 6.1 ns rather than 4.1). Elsewhere (POWER,

s390x, RISC-V, MIPS, the FPU intervals of 64-bit ARM, and 32-bit ARM without GCC or without a floating-point unit of doubles), `fesetround()` still.

15.3.3 The floating-point exceptions and the flush-to-zero modes

Operations raise no invalid-operation exception for the empty set, nor for a NaN the program gives, nor for a zero bound times an infinite one in an SSE2 register. [GAOL v5] The bounds of the empty set are NaN, and `is_empty()` was `!(left() <= right())`, as `set_contains()`, `set_disjoint()` and the other relations compared them with `<`, `<=`, `>` and `>=`: comparisons that signal. With the exception enabled, `is_empty()` of an empty interval and the operations that call it stopped the program with SIGFPE, and with it masked they raised the flag. The SSE2 product computed $0 \cdot \infty$ and cleared the NaN afterwards, and the SSE2 `div_rel` divided an infinite bound by an infinite one in the half of a register it throws away. The bounds of an empty operand are now compared with the quiet comparisons of `<cmath>` (`std::islessequal()`...), one instruction with GCC and Clang on x86-64, as is the first comparison of the constructor, which gives the empty set for NaN bounds: every operation, relation, function and output takes the empty set without the exception, and so does a NaN the program gives (`interval(NAN)`, `x += NAN`, `x <= NAN`, `pow(x, NAN)`). The SSE2 divisions, `%`, `inverse()` and `div_rel()` raise no division-by-zero exception for a divisor with a zero bound (`1/[0, 1]`, `inverse([0, 2])`), which they raised in GAOL 4, and `midpoint()` of `[DBL_MAX]` raises no overflow exception, a bound of 2^{1023} or more in magnitude telling the case of the sum of the halves before the bounds are added. What stays: the inexact exception is raised by almost every operation, and `log([0, 1])` and `[1e308] * 10`, whose bounds are infinite, still raise the division-by-zero and the overflow ones. A program that enables an exception has to disable it while GAOL computes (see Section 4.9, page 107).

A result with no infinite bound raises no overflow nor division-by-zero exception. [GAOL v5] Some functions computed beyond the largest double on the way, or divided by zero, and raised `FE_OVERFLOW` or `FE_DIVBYZERO` for a finite result, the empty set included: `sqrt([1, DBL_MAX])`, `nth_root([DBL_MAX], 3)` and the roots of order 4 and more, `sqrt_rel()`, `nth_root_rel()`, `sin([-1, DBL_MAX])` and `cos()` (the width of the interval overflowed), `tan([DBL_MAX])`, `div_rel()`, `atan_rel()`, the reverse functions of `atan2` and of the power, `atanh_rel()` and, with Visual C++, `atan([DBL_MAX])`. A quotient, a product or a power that would overflow is replaced by what the operation gives, the infinity or the largest double, without being computed; the roots of an argument above 2^{1000} are proved on the argument and the root scaled by powers of 2; `sin()` and `cos()` compare the half of the width with π. A probe of 131 functions and forms on operands next to the limits of the format found 34 that raised them for a finite result; none does with the SSE2 intervals, the FPU intervals and the AVX-512 path. The bounds are the same, bit for bit, up to the sign of a zero bound of `div_rel()` with the SSE2 intervals.

The modes that flush subnormals to zero are cleared by each operation that computes bounds. [GAOL v5] On x86 the flush-to-zero and denormals-are-zero modes of MXCSR, on ARM the mode FZ of FPCR (FPSCR), make every subnormal result a zero and read every subnormal operand as a zero. With one of them set, the bounds of every operation with a subnormal operand or result miss the exact result whatever the rounding direction: `[1e-300] * [1e-20]` is `[0, 0]`. A program linked with `-Ofast`, `-ffast-math` or `-funsafe-math-optimizations` has them (`crtfastmath.o`), and so has one that loads a plug-in built so. Each operation that computes bounds computes $1 + (2^{-1060} + 0)$ (x86-64, and ARM with GCC and Clang), which shows the modes as well as the direction, and clears them with the direction; a 32-bit x86 processor and the NEON intervals of 64-bit ARM processors read the control registers instead, which hold both. With `GAOL_PRESERVE_ROUNDING` they are set back as they were, and the result, maxima and minima of bounds included, is made before. The check comes before the operation reads, compares or copies a bound, and the compiler is kept from moving a comparison before it: in the earlier versions of GAOL v5, with GCC 13 at `-O3`, `asinpi([100*2^-1074])` was above the exact value, and `sqrt([-1e-310, 4])` and `atan2` gave the empty set. At the link, `gaol.pc` and `gaol::gaol` give `-mno-daz-ftz` to the code using GAOL where the compiler accepts it (GCC 13 and later on x86, from 11.4 and 12.4 in the series 11 and 12, and Clang 20 and 21, not Clang 18), which keeps `crtfastmath.o` out of the program; the check of the operations is the only defence against a plug-in, and the only one on the other compilers; on POWER, s390x, RISC-V and MIPS neither defence exists. The cost is that of the second addition of the check: `x * y`, `sqrt` and `pow(x, 3)` take 0.6 to 0.7 ns more than with the direction alone (Intel Xeon Cascade Lake, Clang 18: 4.0 ns rather than 3.4, 11.0 rather than 10.3, 13.7 rather than 13.0). The check of the rounding direction alone, $1 + 2^{-60}$, is the one left where GAOL cannot clear the modes (POWER, s390x, RISC-V, MIPS, and the FPU intervals of Visual C++ for ARM) (see Section 4.8, page 104).

The functions that only read, compare or copy bounds give right results under these modes. With denormals-are-zero a comparison reads a subnormal as a zero, and the functions that make no check gave wrong results: `interval(2^-1073, 2^-1074)` kept its bounds in the wrong order, `abs([-1e-309, -1e-310])` stayed negative, `floor([-1e-310])` was `[-0]`, the midpoint of $[2, 4] \cdot 2^{-1074}$ was 0 and `split()` made an empty half of it. The constructor from two bounds, the relations, `&`, `|`, `max`, `min`, `abs`, `sign`, the roundings to an integer, `mig`, `mag`, `is_canonical`, `feven`, `nb_fp_numbers`, and `inf` and `sup` of `gaol_ieee1788` now compare the bounds as doubles first and, where these are equal, by their bits, which no mode changes, and round a subnormal bound from its bits. `bisect(ratio)` and `is_bisectable()` compare the bounds by their bits too, the cut point included: $[3 \cdot 2^{-1074}, 5 \cdot 2^{-1074}]$ was not bisectable. The AVX-512 path reads its operands once the modes are cleared, and makes its results before they are set back: Clang 18 computed `x - y` after, and `[3e-308] - [2.9e-308]` was `[0, 0]` (see Section 4.4, page 94).

15.3.4 The constants and the static objects

The constants of GAOL are initialized when compiling, so that the intervals computed in the initialization of the static objects of a program are right. [GAOL v5] They were computed by the dynamic initialization of GAOL's files, which, with the static library of the CMake build, comes after the static objects of the program, except with MinGW-w64: an interval computed there found them 0, and `interval::pi()` was $[-0, 0]$, $[1, 2]/[-1, 1]$ was $[-0, 0]$ rather than $[-\infty, +\infty]$ and `sin([0, 4])` was $[-0, 0]$, without any warning (25 of the 45 checks of the test of the static initialization failed with the SSE2 intervals, 24 with the others). The doubles around π and $\pi/2$, `gaol_pi_dn`, `gaol_pi_up`, `gaol_half_pi_dn` and `gaol_half_pi_up`, are written in decimal, exactly, rather than read from the unions that give their bits, which is not a constant expression; they are in `gaol_core` (GAOL 4 had them in `gaol` as `pi_dn`...), and `gaol` and `gaol_ieee1788` name them too. `interval::one()`, `pi()`, `universe()` and the other constants build their interval from these doubles, which the compiler folds, rather than copy the static intervals `interval::cst_one`, `cst_pi`..., which are gone (see Section 5.4, page 125). The masks of the SSE2 intervals are written as vectors, which the compilers put in read-only memory (GCC and Clang) or in initialized data (Visual C++), rather than computed with `_mm_set_pd()` and `_mm_cmpeq_pd()`.

`GAOL_INFINITY` is `std::numeric_limits<double>::infinity()`, a constant of the compiler, where GAOL 4 took the `HUGE_VAL` of the C library. [GAOL v5] The UCRT of Windows (SDK 10.0.26100) writes `HUGE_VAL` as `((double)(float)1e+300)`: Visual C++ makes the conversion when compiling, but clang-cl under `/fp:strict` makes it when the program runs, in the rounding direction of the moment, and got `FLT_MAX` when that direction was downward or toward zero, with the overflow flag. `interval()` was then $[-\text{FLT_MAX}, \text{FLT_MAX}]$, `interval(1e300)` was empty and $[1]/[-1, 1]$ did not contain 10^{300} (see Section 13.1, page 261).

15.3.5 The kernels: SSE2, NEON, AVX-512 and the FPU

On 64-bit ARM processors the intervals are computed with NEON, with every compiler of them: Visual C++, clang-cl and the Clang of MinGW-w64 and MSYS2 as well as GCC, Clang and AppleClang under Linux and macOS. [GAOL v5] GAOL 4 computed the intervals of every ARM processor with the FPU, and 32-bit ARM, whose NEON has no vector of doubles, still does. The two bounds are the two lanes of a `float64x2_t`, as the SSE2 intervals keep them in a `__m128d`, and `+`, `-`, `*`, `/`, `sqr()` and `sqrt()` are kernels that take and return them in registers. On an Apple M2 (AppleClang 21.0), `x + y` takes 0.90 ns rather than 1.15, `x * y` 1.52 ns rather than 3.26, `x / y` 2.08 ns rather than 3.22, `sqrt(x)` 2.04 ns rather than 5.17 and a line of 13 operations 16.6 ns rather than 50.6, the second figures being those of the FPU intervals. The bounds are the same, but for the sign of a zero bound, which the maximum of 64-bit ARM, ranking $+0$ above -0, can set otherwise: the lower bound of $[-1, 0] \cdot [-1, 0]$ is -0 with NEON. Each operation checks the rounding

direction with one `mrs` (0.8 ns on the M2), which raises no exception flag; with `GAOL_PRESERVE_ROUNDING`, an operation takes 27 ns rather than 37 (see Section 4.5, page 96).

The AVX-512 path carries the rounding direction in the instruction. [GAOL v5] `x + y`, `x - y`, `x * y`, `x / y`, `sqrt(x)`, `sqr(x)`, the operators of an interval and a double, and the integer powers beyond 1 and −1 are computed with the rounding that the AVX-512 instructions carry in themselves, on x86-64 where the compiler takes them, never GCC for Windows, which does not align the stack for the AVX-512 registers (GCC bug 54412). They set no rounding direction and raise no exception flag. The bounds are those of the SSE2 intervals bit for bit, but for the sign of a zero bound of `x * d` where the zero bounds of the interval differ in sign. The three builds take the path where `GAOL_PRESERVE_ROUNDING` is on and the machine building runs the instructions, where it computes 2.9 times faster at the median on an Intel i7-1185G7, and leave it out without the option, where the operations take 20 % longer; the library then needs a processor with them, and the initialization stops a program run on another one with a message (see Section 4.4, page 94). Visual C++ and clang-cl for x64 take the SSE2 intervals and the AVX-512 path, as GCC and Clang do, the kernels being `__vectorcall` there so that their `__m128d` arguments go in registers.

`*`, `/` and `sqr()` of the SSE2 intervals compute with no branch on the signs of the bounds, and the basic operations hand the bounds over in registers. [GAOL v5] GAOL 4 chose the product, the quotient and the square by `switch`es and nested branches on the signs of the bounds, and a branch predicted wrong costs more than the operation it chooses; its operators read their operands and wrote their result through memory. `x * y` is now the larger of four products, each rounded upward, in the two lanes of an SSE2 register; `x / y` one division of the two lanes by the denominators that masks drawn from the signs choose; `sqr(x)` two products and a mask. Only an empty operand, an infinite bound or a divisor holding 0 takes a branch. On an Intel i7-1185G7 with Clang 21, `x * y` takes 2.3 ns rather than 10.0 with GAOL 4.2.3, `x / y` 2.5 ns rather than 7.5, and Shekel 5 108 ns rather than 189 (see Section 12.5, page 237). The sign of a zero bound of a product or of a quotient may differ from GAOL 4's, the value never: `maxpd` chooses between a +0 and a −0 that compare equal, where GAOL 4 chose one product by the signs; $[-1, 0] \cdot [-1, 0]$ has the lower bound +0, where GAOL 4 had −0. The FPU intervals multiply, divide and square without a branch too: GAOL 4 took one of nine cases of the signs for a product, and went down a chain of up to eight tests for a quotient. On an Apple M2 (AppleClang 21, `GAOL_SIMD=OFF`), over the random intervals of the benchmark, `x * y` takes 3.5 ns rather than 13.9, `x / y` 3.2 rather than 8.5, `sqr(x)` 1.8 rather than 6.9, and Shekel 5 138 ns rather than 298. A line of products of operands of one sign, which a processor predicts, takes more time (seven `*` in a row: 35.0 ns rather than 23.3, and 1.7 times longer with GCC 13 on x86-64 in the continuous integration), and with `GAOL_PRESERVE_ROUNDING` the kernels lose 7 % on operands of one sign (`x * y` 42.4 ns rather than 39.7) and gain 11 % on operands of random signs (42.4

rather than 47.5). The sign of a zero bound of such a product is not specified: GCC 13 for POWER9 compiles the maximum as an `xsmaxcdp`, which returns its first operand. `x * y` forms every corner product, so that a subnormal corner that is no bound of the result raises `FE_UNDERFLOW`, and a corner that overflows `FE_OVERFLOW` (see Section 12.1, page 228).

Several operations of the kernels were corrected or sped up. [GAOL v5] `x -= x`, `x /= x` and `x %= x` enclose their results in the FPU intervals: GAOL 4 wrote one bound before reading it, and `x -= x` was $[a-b, 2b-a]$ for $x = [a, b]$, `[0.1] /= [0.1]` was empty, and normalizing a row by its pivot, `row[j] /= row[i]` down to $j = i$, did it. A zero bound times an infinite bound counts as 0 in the products of the FPU intervals, as in those of the SSE2 intervals: $[0, 1] \cdot [1, +\infty]$ is $[0, +\infty]$ in every build, where the FPU intervals gave the empty set. The operators of an SSE2 interval with a double compute by the sign of the double rather than through `interval(d)`: on an Intel i7-1185G7 (GCC 9.4, CMake Release), `x += d` takes 3.9 ns rather than 13.6, `x *= d` 6.3 ns rather than 25.2 and `x /= d` 4.1 ns rather than 18.0, giving the same sets. `max(x, y)` and `min(x, y)` of the SSE2 and NEON intervals take both bounds of the result from one comparison of the stored bounds: on an Intel i7-1185G7, `max(x, y)` takes 2.3 ns rather than 8.5 with GCC 15.2 and 2.0 ns rather than 5.6 with Clang 21; on 64-bit ARM they are 1.5 to 2.4 times faster (a Neoverse N2, an Apple M2). The constructors from a register, `interval(const __m128d&)` and `interval(const float64x2_t&)`, give the empty set when a lane is $-\infty$, which is $l = +\infty$ or $u = -\infty$, the lower bound being stored negated; both are explicit now. `minimum()` and `maximum()` of two zeros give -0 and $+0$ with every compiler: Visual C++ compiled `(b <= a) ? b : a` as `minsd`, which gives its second operand for two zeros.

Square roots are the tightest intervals, as IEEE 1788-2015 requires of its basic operations: the lower bound is the square root rounded upward when its square is the argument, the double below it otherwise. [GAOL v5] GAOL 4 computed it as $x/\sqrt{x}$ rounded downward, one double below the tightest bound for half of the doubles; `sqrt()` takes the same time as before (9.0 ns on an Intel i7-1185G7, GCC 9.4), and is bounded whatever the rounding of the C library's `sqrt`, which Visual C++ for 32-bit x86 rounds to nearest in every rounding direction. `sqrt()` is no longer slowed down by GCC on Intel processors: GAOL took the smallest subnormal from `std::numeric_limits<double>::denorm_min()`, a `long double` constant whose conversion GCC does not fold with `-frounding-math`, and the x87 unit converted it with a microcode assist at each use, about 100 ns on an i7-1185G7. `sqrt()` took 124 ns rather than 7.8, and `nth_root(x, 2)` and `nth_root(x, -2)` 113 and 118 ns rather than 6.8 and 10.2 (GCC 15.2, Intel i7-1185G7); the constant is a literal now, and the bounds are the same, bit for bit. `-ffloat-store` is added only where doubles are still computed on the x87 unit: CMake gave it to GCC on every target, and GCC then stored every double variable in memory. On an Intel i7-1185G7 (GCC 9.4, CMake Release), `x + y` takes 3.0 ns rather than 9.1, `x * y` 4.9 ns rather than 18.3, `sqrt(x)` 8.3 ns rather than 41.3, and `log(x)` 73 ns rather than 110.

The SSE2 intervals are computed on 32-bit Windows with GCC, Clang, Visual C++ and clang-cl. [GAOL v5] An SSE2 interval holds a `__m128d` and has to be aligned on 16 bytes, and the C runtime of 32-bit Windows aligns the memory of `malloc()` and `new` on 8: a `std::vector` of intervals crashed on its first `movaps` (MinGW-w64 14.2 and 15.2 for x86). GCC and Clang compile the code using GAOL with `-faligned-new=8`, which the three builds give to the library and to the code that uses it, and `gaol/gaol_config.h` stops the compilation of the SSE2 intervals on 32-bit Windows without it. Visual C++ and clang-cl have no such flag: their SSE2 intervals keep the two bounds as two doubles of alignment 8, loaded with `movupd`, so that a program gives no flag. The memory of the SSE2 intervals is released with the function matching the one that allocated it (`GAOL_MEMFREE()`): `_aligned_malloc()` and `_aligned_free()` on Windows, so that the SSE2 intervals can be used with MinGW-w64 and MSYS2 on x64. The placement `delete` of `interval`, called when a constructor throws, freed the caller's memory; it now leaves it alone.

15.3.6 The constructors, the empty set and the functions of intervals

The constructors give the empty set where their arguments are not an interval, as IBEX does: `interval(+oo)`, `interval(-oo)`, `interval(+oo, +oo)`, `interval(-oo, -oo)`, `interval(2, 1)` and bounds that are NaN are all the empty set, and assigning an infinity to an interval empties it. [GAOL v5] IEEE 1788-2015 has no interval $[+\infty, +\infty]$ nor $[-\infty, -\infty]$: its constructor fails there, and gives the empty set (10.5.8, 12.12.7). GAOL 4 built them as intervals of one infinite point, whose midpoint, `realmax`, was outside them. The intervals with one infinite bound are unchanged. On an Intel i7-1185G7 (GCC 9.4, CMake Release), `interval(a, b)` takes 1.28 ns rather than 1.24 ns: its bounds are set in a register and behind a branch, since a pair written to memory or conditional moves cost several nanoseconds per construction (see Section 5.1, page 117). The constructor from one `double`, `interval(double)`, is explicit, and so are `expression(double)` and `expression(const interval&)`: `interval x = 0.5;`, `return 0.5;` in a function returning an interval, `min(x, 0.5)` and `interval r = sqrt(2.0);` do not compile, where GAOL 4 took the double for a point interval without saying so, the `sqrt(2.0)` of the C library among them. The assignment `x = d`, `x &= d` and `x |= d`, the relations between an interval and a `double`, the arithmetic operators of an interval and a `double` and `set_contains(d)` have overloads of their own, which take `interval(d)` as the conversion did: `x = 0.5` and `x < 0.0` still compile, and give the same results. `interval(double, double)` is not explicit.

Integers are taken as the integers they are. [GAOL v5] An integer given where GAOL takes a `double` was converted to a `double`, in the rounding direction in effect, which an integer beyond 2^{53} is not: `interval(9007199254740993LL)`, $2^{53}+1$, was $[2^{53}, 2^{53}]$ or $[2^{53}+2, 2^{53}+2]$, neither containing it. An integer of any type but `bool` is now taken as the integer it is by the constructors, the assignments, the arithmetic

operators, the relations, `set_contains()`, `expression(n)`, and `numsToInterval()` and `isMember()` of `gaol_ieee1788`: `interval(n)` is the tightest interval containing n, the two doubles around it where n is no double. The arithmetic operators enclose the exact result, but not always with the tightest bounds, for an n that is no double: $[1, 2] + (2^{53} + 1)$ is $[2^{53}, 2^{53} + 4]$. The exponents of `pow` and `pown`, and the orders of `nth_root`, `rootn` and `pownRev`, take an integer of any type too: GAOL 4 had `pow(x, int)` only, so that `pow(x, 3000000000u)` was `pow(x, -1294967296)`, and converted the order of `nth_root` to an `unsigned`, so that `nth_root(e, -2)` of an expression was the order 4294967294.

The constructors from strings are gone. [GAOL v5] `interval(const char*)` took any `const char*` and converted it implicitly: `interval(0)` was ambiguous, 0 being a null pointer as well as a number, and `x + nullptr` compiled, and crashed. `gaol::textToInterval()` reads the string instead, with the same names of functions, and throws `std::invalid_argument` for a string that is no interval, and for a null pointer, through an overload for `const char*`, where the overload for `std::string` would build a `std::string` from the pointer, which the standard leaves undefined. `interval(const char*, const char*)` kept the bounds $[2, 1]$ of `interval("2", "1")`, which `is_empty()` called empty but whose sum with $[0, 1]$ was $[2, 2]$; `textToInterval("0.1", "0.3")` takes the left bound of the first string and the right bound of the second, and gives the empty set for bounds in the wrong order. A result thrown away gives a warning: `sqrt(x);` alone leaves `x` as it was, and the functions whose result is all they do carry `GAOL_NODISCARD`, `[[nodiscard]]` in C++17 and later and the attribute of the compiler before; a cast to void silences it, and a program that does not want it defines `GAOL_NODISCARD` empty (see Section 3.7, page 73).

The functions of intervals change on the empty set and on their rounding. [GAOL v5] The intersection of disjoint intervals is the empty set of `interval::emptyset()`, whose bounds are NaN: GAOL 4 kept the largest left bound and the smallest right bound, $[3, 2]$ for $[1, 2] \cap [3, 4]$, which `is_empty()` took for empty but the operations on the bounds did not, and $([1, 2] \cap [3, 4]) + [0, 1]$ was $[3, 3]$. `width()` of the empty set is NaN, as wid of IEEE 1788-2015, rather than -1, and `chi()` of the empty set is the positive NaN, printed `nan` with every compiler. `hausdorff()` returns the tightest upper bound of the distance, and 0 for equal bounds, infinite ones included: GAOL 4 computed `fabs(a - c)` in the rounding direction of the caller and returned $+\infty$ as soon as a bound was infinite, so that `hausdorff([1, +oo], [1, +oo])` was $+\infty$ and a fixed-point loop on a box with an unbounded side never stopped. An infinite bound in one interval only still puts them at $+\infty$, as in GAOL 4 (`hausdorff([1, +oo], [1, 2])`), and the function then returns before the rounding direction is checked, 2.5 to 4.4 times as fast as through the computation of the distance (Intel i7-1185G7, GCC 9.4 and Clang 18). `nb_fp_numbers()` numbers the doubles by the bits of the magnitudes, -0 and $+0$ being one number: `nb_fp_numbers(-0.0, 1.0)` was 13830554455654793217, and is 4607182418800017409, the number of the doubles from 0 to 1. `mid()` returns the tightest interval enclosing the midpoint, computed as

`midpoint()` computes it with the rounding outward: it computed $a/2+b/2$ rounded upward and subtracted the half of b for its lower bound, which is not exact when $|b| < 2^{-1021}$, so that `mid([0, 2^-1074])` was $[2^{-1074}, 2^{-1074}]$ and 45150 of the 180901 intervals whose bounds are multiples of 2^{-1074} from -300 to 300 times it did not enclose their midpoints. `rad()` and `mid_rad()`, rad and midRad of the standard, are new (Chapter 8).

The tools that algorithms of intervals write for themselves are new: [GAOL v5] `width_enclosure()`, `inflate(r)`, `interval::midrad(m, r)`, `bisect(ratio)`, `is_bisectable()`, and `hull(a, b)` and `intersect(a, b)`, the functions of `a | b` and `a & b`. The first three compute with the operations of intervals, each bound rounded outward: `interval::midrad(m, r)` encloses $[m-r, m+r]$, which `interval(m - r, m + r)` does not, both of its bounds being computed upward; `width_enclosure()` is the enclosure of the exact width that `width()` bounds from above. `bisect(ratio)` cuts at $l(1 - \mathit{ratio}) + r \cdot \mathit{ratio}$, which does not overflow (for a bound of 2^{1023} or more in magnitude it is computed from the halves of the bounds) and lies strictly between the bounds when `is_bisectable()`, and throws `std::domain_error` for a ratio outside $(0, 1)$ or a NaN; `is_bisectable()` is true when a double lies strictly between the bounds (see Section 3.8, page 74).

15.3.7 The powers and the roots

The powers with a real exponent were corrected, after the fix of Codac. [GAOL v5] `pow(I, e)` for a floating-point e called `pow(I, int)`, which truncated the exponent: `pow([4], 0.5)` returned [1]. It now computes an integer e with `pow(I, int)` and any other e with `pow(I, J)`. `pow(I, J)` computed the powers of the negative part of I on its magnitude: `pow([-4,-1], [0.5])` returned $[-1, 2]$. It is now hybrid: a degenerate integer exponent $[n]$ takes `pow(I, int)`, the pown of the standard, which keeps the negative part of I and gives $0^0 = 1$; any other exponent takes the pow of the standard, on the part of I in $[0, +\infty]$, where 0^y is 0 for $y > 0$ and has no value for $y \leqslant 0$: `pow([0], [0.5])` was $[0, 4.9 \cdot 10^{-324}]$ and `pow([0], [-0.5])` was $[\mathrm{MAX}, +\infty]$ (see Section 3.5, page 70).

`pow(I, J)` takes the tightest bounds at the corners of the box. [GAOL v5] x^y increases with y for $x > 1$ and decreases for $x < 1$, increases with x for $y > 0$ and decreases for $y < 0$: over a box of bases above 0 its extrema are at corners, and the pow of CORE-MATH, correctly rounded upward, is taken there, as the upper bound, and, as the lower bound, the double below it, or x^y itself where that is a double, which is proved from the bits of x and y: `pow([4], 0.5)` is [2], and `pow([3, 4], [2, 3])` is $[9, 64]$. At a corner where x is 0 or $+\infty$, or y infinite, the bound is the limit of x^y, and CORE-MATH is not called: `pow([4, +oo], 0.5)` is $[2, +\infty]$. GAOL 4 computed $\exp(J \log I)$, which multiplied the relative width of $\log I$, a few 2^{-52}, by $|y \log x|$: `pow([2], [1023.5])` was 1425 doubles below the exact value and 748 above, and `pow([0, 2^-1000], [-oo, -1])` had a lower bound 617 doubles below 2^{1000}. Over the million powers of the benchmark of `doc/compare`, its intervals

are no wider than those of libieeep1788, which computes each bound with MPFR, where $\exp(J \log I)$ made them $5.9 \cdot 10^{-14}$ wider relatively. With Clang 18 on an Intel i7-1185G7, over 100 000 boxes, `gaol::pow(x, J)` with $x = [1/b, b]$ takes 84 ns (GAOL 4: 138 ns), with $x = [a, +\infty]$ 55 ns (GAOL 4: 99 ns) and with $J = [a, +\infty]$ 54 ns (GAOL 4: 126 ns).

Integer powers are the tightest bounds, of either sign. [GAOL v5] `pow(x, n)` of `gaol`, `pown(x, n)` of `gaol_ieee1788` and the integer exponents of `pow(x, y)` and `pow(x, p)` give the tightest bounds of x^n, or one double beyond where x^n is within twice the error bound of a double (2^{-98} relatively at $n = 3$ and about 2^{-62} at the most; at $n = 3$, one bound in 2^{46} is one double beyond for an x at random), and exact where x^n is a double: `pow([2, 3], 3)` is $[8, 27]$. GAOL 4 rounded each product of its binary exponentiation outward and was up to $n + 1$ doubles from the tightest bounds, 2, 3, 5, 6 and 8 doubles for $n = 3$ to 7, and took x^{-n} for $1/x^n$, two roundings. Both bounds are computed at once, in the two lanes of an SSE2 or a NEON register with the FMA instructions, or in a pair of doubles otherwise: each product is rounded, `fma` gives its rest exactly, and the power is carried as a sum of two doubles whose error is bounded beforehand, so that each bound is rounded once. A corner whose powers leave $[2^{-968}, 2^{960}]$ is computed alone, scaled by powers of two: GAOL 4's $1/x^n$ overflowed before the inversion, and `pow([10], -400)` was $[0, 5.6 \cdot 10^{-309}]$ rather than $[0, 2^{-1074}]$. `pow(x, 3)` takes 5.1 ns against 9.1 with GAOL 4.2.3 (Intel i7-1185G7, Clang 21). The exponent 2 is `sqr(x)` in every form, the squares of the bounds rounded outward: in the earlier versions of GAOL v5, with the exact products, `pow(x, 2)` took 5.3 ns with the NEON intervals and 5.2 with the FPU intervals, and it takes 1.6 and 1.9 now (Apple M2, AppleClang 21); the 6 million calls of `pow(x, 2)` of the Goldstein-Price example take 50 ms rather than 77 with the NEON intervals. The integer powers beyond the ints are computed from the pow of CORE-MATH at the corners of x, the sign from the parity of n: the tightest where n is a double, and within three doubles of it beyond 2^{53}. `pow(x, 2)` in all its forms, the powers with a real or an interval exponent (an empty exponent included), with a `long long` beyond the `unsigned` ints, and `gaol_ieee1788::pow` return the empty set for an empty operand before they check the rounding direction, as `sqr` does, and leave the direction and the flush modes as the program had them; the other integer powers, of an `int`, an `unsigned` or a `long long` within their range, check it first and leave it upward (see Section 7.2.2, page 151).

`nth_root(I, n)` is the `rootn` of IEEE 1788-2015: for an odd n, it is defined on the whole real line, the root of a negative number being the opposite of the root of its magnitude, and `nth_root([-8, 27], 3)` encloses $[-2, 3]$; for an even n, it takes the roots of the part of I in $[0, +\infty]$, as before. [GAOL v5] GAOL 4 took that part for every n, and gave $[-2^{-1074}, 3.000000000000001]$, or the empty set for `nth_root([-8, -1], 3)`; the root of 0 is 0, rather than $[-2^{-1074}, 2^{-1074}]$. The nth roots are proved with integer powers, and are the tightest bounds or up to 2 doubles beyond (4 for `rootn` with $q < 0$, the inverse of a root), for every n and every double, the subnormal ones included. GAOL 4 took the power of the mathematical library

with the exponent $1/n$ rounded, moved one double outward: the bounds were up to 8 doubles from the tightest between 2^{-30} and 2^{30}, and 234 over all the doubles, and `nth_root([27], 3)` was two doubles wide where it is [3] now. A bound l is below the root when l^n, rounded upward, is at most x, and u above it when u^n, rounded downward, is at least x; the search starts from the pow of CORE-MATH brought next to the root by a step of Newton's method, and goes by steps that double, then by bisection. For an x below 2^{-1020} the proofs take scaled powers, the plain proof losing bits where a power is subnormal: `nth_root([2^-1074], 33)` was 1983 doubles below the tightest lower bound and 28785 above the tightest upper bound. `nth_root_rel()` takes the roots of `nth_root()`: it kept a value within 23 doubles, now 2. A point interval takes 152 ns rather than 130, its only root being looked for once; an interval 194 ns rather than 136 (Intel i7-1185G7, GCC 9.4). `pownRev(c, x, p)` for $p \leqslant 0$, the last reverse operation of the standard GAOL lacked, is `nth_root_rel(J, p, I)`, which takes any integer p now: a negative `int` was converted to a large `unsigned int`, a root of order about $4{\cdot}10^9$. For $p < 0$ the bounds are up to 4 doubles beyond the tightest ones. `uipow()`, the `pown` for an unsigned exponent, is `pow(I, n)` for an `unsigned` n in `gaol`, exported with the SSE2 intervals too, where it was inline and did not link; it gives [1] for an exponent 0, and the empty set for an empty interval.

15.3.8 The trigonometric, hyperbolic and other functions

`sin()` is computed as `cos()` is: the bounds of the interval divided by an enclosure of π, minus 1/2, tell the pieces where the sine is monotonic, and the sine of CORE-MATH is taken at the bounds. [GAOL v5] GAOL 4 computed `cos(x - [pi/2])`, whose subtraction widened the argument by about $2^{-52}\max(2, |x|)$: `sin([1e-10])` was $4.4 \cdot 10^{-16}$ wide, billions of doubles, and `sin()` was not accurate in the sense of IEEE 1788-2015. On an Intel i7-1185G7 (GCC 9.4), `sin()` takes 121 ns rather than 130. `sin()`, `cos()` and `tan()` are within one double of the tightest bounds at every magnitude. The bounds divided by an enclosure of π only tell on which pieces the interval lies while the quotients are not within their rounding errors of an integer: a bound within about $|x| \cdot 2^{-52}$ of an extremum was taken as reaching it, `cos([2^60])` was $[-1, 1]$, and `tan()` gave $[-\infty, +\infty]$ next to a pole and beyond 2^{52}. CORE-MATH being correctly rounded, and no double but 0 being a multiple of $\pi/2$, the signs of its sine and cosine at the bounds are the exact ones, and tell whether an extremum or a pole is within the interval; they are only asked for next to an extremum or a pole, and at the large magnitudes. The time is unchanged in the benchmark (113 ns for `sin()` and `cos()`, Intel i7-1185G7, GCC 9.4), 125 ns rather than 121 for intervals holding an extremum; at magnitudes from 10^8 to 10^{15}, `cos()` of a narrow interval takes 250 ns rather than 224, and `tan()` 260 rather than 217. `tan()` is finite over an interval of doubles that holds no pole and whose width, rounded upward, is at most the double below π: `tan([-M_PI_2, M_PI_2])` was $[-\infty, +\infty]$, and is about $\pm 1.63 \cdot 10^{16}$ (Chapter 11).

`atan2(y, x)` is implemented, as the atan2 of the standard, defined on the plane but $(0,0)$ with values in $(-\pi,\pi]$: GAOL 4 declared it, and its parser read it, but it raised `unavailable_feature_error`. [GAOL v5] In each quadrant the angle is monotonic in y and in x, and the atan2 of CORE-MATH is taken at two corners of the box; a box with points on the half-line $y=0$, $x<0$ and points below it gives $[-\pi,\pi]$, and `atan2([0], [0])` is empty. It takes 100 ns on an Intel i7-1185G7 (GCC 9.4) for boxes within a quadrant, against 68 ns for `atan()`. `acos_rel()`, `asin_rel()` and `atan_rel()` enclose the multiples of π of the pieces of the preimage with π in double-double, one rounding at the magnitude of the result rather than two: `atan_rel()` was up to 49 doubles from the tightest bounds over random cases where it is now within 2, and `acos_rel()` up to 34. An I of a single double is decided at every magnitude by whether $f(I)$ meets J, where a bound beyond 2^{52} was kept as it was (see Section 7.3, page 160).

The elementary functions are exact where their value is a double, and the tightest where it is $\pi/4$, $\pi/2$ or π. [GAOL v5] $\exp(0)=1$ and $\log(1)=0$ exactly, the bounds moved one double outward giving `exp([0])` and `log([1])` a width: `pow([1], [-oo, +oo])` was $[0,+\infty]$ rather than $[1]$. `log()` takes 1.5 ns more (72.5 ns rather than 71) and `pow(x, y)` 2 ns more (145 ns rather than 143) on an Intel i7-1185G7 (GCC 9.4). `sin(0)`, `tan(0)`, `asin(0)`, `atan(0)`, `sinh(0)`, `tanh(0)`, `asinh(0)` and `atanh(0)` are 0, `cos(0)` and `cosh(0)` 1, `acos(1)` and `acosh(1)` 0, and `asin(±1)`, `acos(0)`, `acos(-1)`, `atan(±1)` and `atan(±oo)` are the constants of GAOL: `acos([1, 3])` was $[-2^{-1074},2^{-1074}]$, and `acosh([1])` with the libm was $[-3\cdot2^{-1074},3\cdot2^{-1074}]$. The hyperbolic functions `sinh`, `cosh`, `tanh`, `asinh`, `acosh` and `atanh` are those of CORE-MATH, the tightest bounds on every system: GAOL 4 took them from the libm of the system and moved their values one double outward, while the libms of glibc 2.31, musl and MinGW-w64 are sometimes a double further, and the `acosh` of the MinGW-w64 older than version 12 is millions of doubles away next to 1. `cosh()` and `sinh()` are $[\mathrm{MAX},+\infty]$ from 711 on, and `tanh()` has the double below 1 for lower bound from 20 on; the functions take the same time as before, within 1.5 ns (Intel i7-1185G7, GCC 9.4).

The domains follow the standard. [GAOL v5] `log()` gives the empty set for an interval holding no positive number, `log` being defined on $(0,+\infty)$: GAOL 4 gave $[-\infty,-\mathrm{MAX}]$ for `log([-4, 0])` and `log([0])`. `atanh()` gives the empty set for an interval holding no point of $(-1,1)$: `atanh([1])` and `atanh([1, +oo])` are empty, and the limits stay the bounds of an interval holding other points, `atanh([0, 1])` being $[0,+\infty]$. `div_rel()` rounds the lower bound of the quotients downward where K is below 0 and J straddles it, the one branch left without SSE code: `div_rel([-1], [-3, 1], [0, 10])` was one double above 1/3, and the numbers from 1/3 to the double above it were lost, where mulRev has to be valid. `invabs_rel()` takes no magnitude from the negative part of J: `invabs_rel([-2, -1], [-10, 10])` was $[-2,2]$ rather than the empty set. The reverse functions of the power, `atan2`, `max`, `min`, `sign` and `floor` are new, their arguments in the order of `div_rel(K, J, I)`: `pow_rel(K, J, I)`, `pow_exponent_rel(K, H, I)`, `atan2_`

`rel_y(C, X, Y)`, `atan2_rel_x(C, Y, X)`, `max_rel(Z, Y, X)`, `min_rel(Z, Y, X)`, `sign_rel(Z, X)` and `floor_rel(Z, X)`; `gaol_ieee1788` names the first four powRev1, powRev2, atan2Rev1 and atan2Rev2, and adds mulRevToPair. All the reverse operations of the standard are provided (Chapter 7).

15.3.9 The relations

The comparisons with the empty set and the infinities follow IEEE 1788-2015 (Tables 10.3 and 10.4). [GAOL v5] `certainly_leq()` and `certainly_le()`, precedes and strictPrecedes, and `certainly_geq()` and `certainly_ge()`, are true when either interval is empty: GAOL 4 gave false when the second one only was. `set_strictly_contains()` and `set_le()`, interior, take an infinite bound as beyond the same infinite bound: `interior(Entire, Entire)` and $[2, +\infty]$ interior to $[1, +\infty]$ were false, as `set_le()` of the empty set in the empty set.

The possibly relations, `certainly_eq()`, `certainly_neq()`, `==` and `!=` are removed, and so is the option that chose what the relation symbols mean. [GAOL v5] `certainly_neq()` was `!certainly_eq()`, which is "possibly not equal": $[3, 4]$ was certainly not equal to $[3, 4]$, and so was `!=` with the certainly relations, the default of every build. IEEE 1788-2015 has neither the possibly relations nor a certain equality: its equality is equal, `set_eq()`, and "for all x and y, $x \neq y$" is disjoint, `set_disjoint()`; `possibly_eq(y)` was `!set_disjoint(y)`. `<`, `<=`, `>` and `>=` are the certainly relations in every build, strictPrecedes, precedes and their converses. The set relations of the option were never taken, `configure` and meson defining `GAOL_SET_RELATIONS` where the header read `GAOL_SET_RELATION`: they gave the possibly relations (see Section 6.5, page 138).

`gaol::lexicographic_less` is a total order on the intervals, for the containers and the algorithms of the standard library that compare with `<` (`std::set`, `std::map`, `std::sort`). [GAOL v5] `<` serves none of them: it is strictPrecedes, true as soon as either interval is empty, and no strict weak ordering. A `std::set<interval>` that holds $[1, 2]$ drops $[1, 3]$, and `std::sort` of a vector that holds an empty interval can read past its end. With the comparator, the empty set comes first, then the intervals by their left bound and by their right bound, -0 and $+0$ being the same bound (see Section 6.6, page 140). `x.overlap(y)` and `gaol_ieee1788::overlap(x, y)`, the overlap of IEEE 1788-2015 (10.6.4), are new, with the type `overlapping_state`: each returns one of the 16 states of Table 10.7, which GAOL 4 had no function for. The state is read from the bounds as integers in the order of the doubles, so that no floating-point exception is raised, whatever the flush-to-zero modes. A point interval on a bound of the other one is never **`meets`** nor **`metBy`**, as the sentence of 10.6.4 has it (Chapter 6).

15.3.10 Reading and writing intervals

Numbers are read exactly, whatever the locale, the flush-to-zero modes and the C library. [GAOL v5] GAOL 4 read them with `strtod()` rounding downward and upward, and relied on the inexact flag: the C runtime of Windows and musl on 64-bit ARM processors round to nearest in every direction and raise no flag, so `interval("0.1")` did not enclose 1/10 there. Each number is now compared exactly with the doubles around it, whatever the number of digits of its significand and of its exponent; the doubles around the number are bracketed by steps doubling from the double `strtod()` gives, then found by bisection on their bits, two comparisons when `strtod()` gives the nearest double, and at most about 125 whatever it gives. The time to read a number is quadratic in the number of its digits (see Section 9.1, page 183), so that a text from elsewhere can block a program. `strtod()` reads the decimal point of the C locale: under a locale writing a decimal comma, it stopped at the `.` of `0.1`, and `interval("0.1")` was $[0, 0]$; the text is taken apart once, and a number of 20000 characters, which took over 2 s under such a locale, takes about 0.02 s. Reading raises no overflow or underflow exception: with glibc, `strtod()` raises them for `1e400` or `1e-310`, and a program that had enabled the exception died while it read; a number that may be beyond the largest double or below 2^{-1022} is compared with them first, and is not given to `strtod()`. With denormals-are-zero the reader took every subnormal for 0, and `textToInterval("1e-310")` did not contain 10^{-310}; the doubles are taken apart by their bits, which no mode changes (see Section 9.1, page 183).

Intervals are written in decimal rounded outward, whatever the C library. [GAOL v5] GAOL 4 set the rounding direction downward, then upward, and let the C library write each bound: the C runtime of Windows rounds the magnitude, and wrote $-2/3$ rounded downward `-0.6666`, and musl wrote `1.235e+05` for the lower bound of 123456.789; read back, such a text did not enclose the interval written. The magnitude of each bound is now written rounding to nearest, compared exactly with the bound, and its last digit is moved by one when it is on the wrong side; with glibc, which rounds as asked, the text is the one the C library wrote: 1 077 546 bounds written in the general, scientific and fixed formats with 1 to 18 digits were the same, character for character. A point interval whose double the digits write exactly is written $[a]$, the literal of the standard, and a zero of either sign $[0]$: GAOL 4 wrote every point interval `<a, b>`, the text of its double rounded downward and upward, which the reader refused for `interval(0.1)`, throwing `input_format_error`, because it takes `<a, b>` for two numbers that are the same double, so that reading back the text of a point interval failed for about 61 % of 11 788 point intervals tried; the reader still reads `<a, b>`, with the numbers compared by their bits. The format **`width`** writes the midpoint and the radius of the standard, `midpoint()` rounded to nearest and `rad()` rounded upward, which is $+0$ for a point zero of either sign, and **`center`** the midpoint: GAOL 4 wrote $(l + r)/2$ and $(r - l)/2$, both rounded to nearest, so that $[1, 1 + 2^{-52}]$ was written `1 (+/- 1.110223024625157e-16)`, which stops short of the upper bound, and $[0, 5 \cdot 10^{-324}]$

had a radius 0. The empty set is `[empty]` in every format. The format **`bounds`** is written in the C locale whatever the locale of the stream, so that the reader reads the text again: GAOL 4 wrote `[0,25 0,5]` under a comma locale. The format **`agreeing`** keeps the digits of each bound: GAOL 4 dropped from both bounds what followed the last character of the left one that is not a zero, and $[1.25, 1.2567]$ was written `1.25~[ ]`, which says nothing of the right bound; it is `1.25~[0 67]` now, and $[1, 10]$ is written with its bounds, where it was `1~[. 0]`. The hexadecimal format writes an interval literal, `[0x1.999999999999ap-4 ...]`, read back bit for bit, where GAOL 4 wrote the sixteen hexadecimal digits of each double, which no parser read; `exact_string(I)` gives that text without the global output format, and `gaol_ieee1788::intervalToText()` writes a literal with the digits of `interval::precision()`, whatever the format and the locale. `operator<<` leaves the stream as it found it: GAOL 4 set its precision and left it so, the doubles written afterwards getting 16 digits, and wrote the `[` apart, so that `std::setw(8)` padded it alone. The text is made in a character string, taking 30 to 45 % less time than the `std::ostringstream`s of the earlier versions of GAOL v5 (Clang 18, Release) (see Section 9.2, page 190).

`operator>>` ends with the input, and skips blank lines. [GAOL v5] GAOL 4 read the rest of the line where the previous value stopped, so that `in >> d >> x` over two lines threw `input_format_error`, and so did `while (in >> x)` over a file ending with an empty line: such a loop always ended with an exception. Where no line is left, the reader sets the `failbit` of the stream, leaves the interval as it was and throws nothing, as the reading of a double does; a line that is no interval sets the `failbit` too, empties the interval, and `std::invalid_argument` is thrown. A program that reads on after it calls `in.clear()` first.

One grammar reads every string as an expression. [GAOL v5] GAOL 4 read a string with two grammars: the numbers, the constants and the functions of them built a tree, while the intervals written between brackets, and the operations on them, were computed at once; Bison chose between the two at each token (124 shift/reduce and 14 reduce/reduce conflicts), and once a number had started the tree no interval could come, so that `1+[1,2]` and `2*cos([0,1])` were refused while `[1,2]*2` was read. The literals of intervals, the uncertain numbers, `empty` and `<a,b>` are now leaves, computed when they are read, as are the calls of functions and the binary operations; the grammar has no conflict, and an error found in an action stops the reading at once. An uncertain number is still no bound of a literal: `[5?1]` throws `std::invalid_argument`, the standard giving it as a string that is not an interval literal. The literals of IEEE 1788-2015 are read, whatever the case of their letters: `[ ]`, `[entire]`, the bounds left out (`[1,]`, `[,]`), `infinity`, the hexadecimal numbers and the uncertain form (`3.56?1`, `-10??u`), as the tightest intervals of doubles enclosing them; `[inf]` and `[inf, inf]` are the empty set, where GAOL 4 gave $[\mathrm{MAX}, +\infty]$. Numbers in base 16, 8 and 2 are read, `0x10`, `0o17`, `0b101` and `0b1.1p3`, a number that begins with 0 being decimal. $[l, u]$ with l above u is the empty set also where the bounds are numbers that are not doubles: GAOL 4 read

`[1.0000000000000001, 1]` as [1]. A bound of `<a, b>` has to be a double, and a number that is not one, `pi` included, is refused. The reader knows every function of GAOL on intervals, which GAOL 4 did not read (`exp10`, `hypot`, `sqr`, `abs`, `fma`...), and `gaol_ieee1788::textToInterval()` reads the names of the standard, whose `pow` is the pow of the standard: in the earlier versions of GAOL v5 it read those of GAOL, and `pown` gave the empty set. Function names are read in any case of letters, compared in ASCII and not by `std::tolower()`, whose result depends on the locale. `pow(x, n)` read in a string is the `pow` of `gaol` for an integer exponent too: the parser took $1/x^{|n|}$ for $n < 0$, whose $x^{|n|}$ overflowed, and 1 for $n = 0$, which is no value for an empty x (see Section 9.1.1, page 184).

The reader is robust. [GAOL v5] The parser frees the nodes of the expressions it reads: it kept a reference to each node, and the nodes of every `interval("...")` were never freed; the nodes left by a syntax error are freed too, and reading `nth_root(8, 1.5)`, then the same text plus 1, crashed, where both now throw `std::domain_error`. Several threads can read strings at once: the lexer of flex kept its buffer in globals, and two threads each building an interval from a string crashed; the lexer is reentrant and the parser pure, and four threads reading strings at once, which crashed each time with the globals, show nothing under ThreadSanitizer. Long expressions no longer overflow the stack: a sum of 100000 terms crashed the program, and is read in a time proportional to its length; the stack of the parser, 10000 entries, bounds the nesting. A `std::string` is read whole, where the lexer stopped at the first NUL character. An allocation that fails throws `std::bad_alloc`, where flex ended the program with `exit(2)`. Built without exceptions (a setting of the developers), a wrong call in a string fails the reading as a syntax error does, where an earlier version of GAOL v5 ended the program inside the reader: `gaol_ieee1788::textToInterval()` gives the empty set, `parse_interval()` returns false, and `gaol::textToInterval()` and `operator>>` report the error and abort. `parse_interval(nullptr)` returns false, where the program died of a segmentation fault. A newline is a space, where it fell to the default rule of flex, which wrote it on the standard output. Expressions are printed as written: a division with `/`, where GAOL 4 wrote `*`, and a negative number within the parentheses a power of it needs.

15.3.11 The errors

GAOL throws `std::invalid_argument` and `std::domain_error`, and `what()` says what went wrong. [GAOL v5] GAOL 4 had a class of exceptions of its own, `gaol_exception`, and `input_format_error`, `invalid_action_error` and `unavailable_feature_error` derived from it, in `gaol/gaol_exceptions.h`. They are gone, without aliases. A string that is no interval, or a null pointer, throws `std::invalid_argument`; an argument that a function does not take (`nb_fp_numbers()` with a NaN, `bisect()` with a ratio outside $(0, 1)$, `nth_root(8, 1.5)` in a string) throws `std::domain_error`; no operation of GAOL v5 throws `unavailable_feature_error`, which only the `atan2` of GAOL 4 did. A handler of `gaol::input_format_`

`error` becomes a handler of `std::invalid_argument`, one of `gaol::invalid_action_error` a handler of `std::domain_error`, and `std::logic_error` catches both. GAOL 4 left the `what()` of `std::exception`, the text `std::exception` with libstdc++ and libc++ whatever the error; it is `file:line: explanation` now, the file without its directories. `file()`, `line()`, `explanation()` and `operator<<` of the exceptions are gone with the classes. A null file name is "(unknown file)" in the message of an exception, in `gaol_warning()` and in `gaol_error()`: GAOL 4 wrote the null pointer to the stream, which is undefined, and libstdc++ put the stream in the state `bad`, so that the diagnostics written after it were lost (see Section 13.5.1, page 266).

15.3.12 The namespaces, the headers and the interface

The type `interval`, GAOL's functions and its expressions are in `gaol_core`; `gaol` names them as GAOL 4 did, and `gaol_ieee1788` as IEEE 1788-2015 does. [GAOL v5] `gaol` names them one by one, with using-declarations, rather than opening `gaol_core` whole: the names GAOL 4 had in `gaol` that GAOL v5 still has, the functions on intervals GAOL v5 adds, `restore_rounding()` and `exact_string()`. The two namespaces differ mainly in the power, the standard having two where GAOL 4 had one: the `pow` of `gaol` takes the integer power for an integer exponent, a negative base included, and the real power otherwise, while in `gaol_ieee1788` `pow` is the real power, on the part of x in $[0, +\infty]$, and `pown` the integer power. `pow` and `textToInterval`, whose meaning differs between the two, are in each of them and not in `gaol_core`, so that each namespace finds its own only. `gaol` does not name the exceptions, `NaN_val`, `eval_stack` nor `parse_interval()`, which GAOL 4 had in `gaol`, nor what GAOL uses for itself; a program names them in `gaol_core`. `f_negate()`, `f_negate_simple()`, the macro `opposite()`, `get_fpu_cw()` and `reset_fpu_cw()`, which GAOL v5 no longer calls, are gone. `gaol/gaol.h` no longer opens `gaol`: a program adds `using namespace gaol;` or `using namespace gaol_ieee1788;`, one of the two, `pow(x, y)` being ambiguous with both (see Section 3.4, page 69). The `double` constants of `gaol/gaol_port.h`, `two_pi`, `pi`, `half_pi`, `pi_dn`, `ln2_dn`..., are no longer declared for the code using GAOL: a program that opened `gaol` found them among its own names, and a `pi` of its own was ambiguous; the bounds of π are those of `interval::pi()`, and those of $\ln 2$ those of `log(interval(2.0))`.

The expressions were repaired. [GAOL v5] `pow(e, n)` on an expression links: the library defined it with an `unsigned int` exponent, where the header declares an `int`. `expression::operator/=` links: it was declared and not defined. Empty expressions can be created in several threads: they all point to one node, whose count of references, a plain `unsigned int`, lost updates, and two threads each creating and destroying a million empty expressions crashed each time; the references to that node are no longer counted, and it lives from the initialization of GAOL to its automatic cleanup. Other expressions still share the nodes of their copies, whose counts are not atomic: such an expression is not copied or destroyed in two threads at once.

The headers GAOL installs are the ones a program includes. [GAOL v5] The three builds install the same list: `gaol/gaol`, `gaol/gaol.h`, the headers they include, and `gaol/parser/gaol_expression.h` and `gaol/parser/gaol_expr_eval.h` for the expressions; a program that builds expressions includes `gaol/parser/gaol_expression.h`, where it included `gaol/gaol_expression.h`. They install no header of GAOL's sources: neither the header of the parser, which did not compile when included, nor `gaol/gaol_init_cleanup.h`, both of which GAOL 4 installed, nor the headers of `gaol/math/`, among them `gaol_double_op.h`, whose functions of doubles `exp_dn()`, `sin_up()` and the thirty others leave the interface. Each installed header, but the four of Section 3.2, compiles alone with `-std=c++11` and the flags of `gaol.pc`, and without a warning under `-Wall -Wextra` and `-Wold-style-cast -Wundef`: with `-Wextra`, every assignment of intervals warned that the implicit assignment was deprecated. The class `interval` is declared in `gaol/core/gaol_interval_core.h`, and the headers of the SSE2, NEON and FPU intervals are installed in `include/gaol/core/`; `gaol/gaol_interval.h` stays and includes it, so that a program that includes it, or `gaol/gaol`, is not touched by the move. Gone are the timing of GAOL 4 (`gaol/gaol_profile.h`, the class `timepiece` and `get_time()`, `elapsed_time()`...; a program measures the time with the clocks of the C++ standard library), `gaol/gaol_parameters.h` and `gaol/gaol_limits.h` (a program includes `<limits>` itself), `gaol/gaol_assert.h` and `GAOL_ASSERT` (a program writes `assert()`), `gaol/gaol_exact.h` with `get_inexact()` and `clear_inexact()`, which the lexer called to know whether `strtod()` had rounded, `gaol/gaol_flags.h` and `s_nextafter()`, and `gaol/gaol_allocator.h` with `gaol::aligned_allocator`, which neither GAOL, Codac nor IBEX used. The `operator new` of the SSE2 intervals still aligns their memory.

`__GAOL_PUBLIC__` is `GAOL_PUBLIC`, and is empty with Visual C++ and clang-cl, with which GAOL is always a static library. [GAOL v5] GAOL 4 declared its functions `__declspec(dllimport)` there, so that the code using GAOL had to define `__GAOL_PUBLIC__` empty; no header reads it any longer. No header of GAOL declares `string` and `exception` in the global namespace, and the macros of GAOL's headers start with `GAOL_`: `INLINE`, `MEMALIGN`, `__HI`, `WORDS_BIGENDIAN`... are `GAOL_INLINE`, `GAOL_MEMALIGN`, `GAOL_HI`, `GAOL_WORDS_BIGENDIAN`... `_XOPEN_SOURCE` is no longer redefined to 600, and `nb_fp_numbers()` returns an `unsigned long long`. The macros that the installed headers read with `#if` are 1 or 0, never undefined, so that they compile under `-Wundef -Werror`: a program has to test them with `#if` as well, `#ifdef GAOL_PRESERVE_ROUNDING` being true whatever the build.

15.3.13 The builds, the compilers and the options

The three builds agree. [GAOL v5] Before, each had its own idea (GAOL 4 had the first two, and the CMake build is that of the 4.3 releases of GAOL v5): `configure` optimized only when the compiler was named `g++` exactly, computed the doubles of 32-bit x86 on the x87 unit, read the processor of the machine building rather than the target of the compiler, and refused macOS, Windows, s390x and riscv64;

meson did not optimize, compiled with the assertions of libstdc++, refused i686 and armv7l and linked CRlibm by default; CMake left out `-msse3`, the hidden visibility and `-Wconversion`. `-fno-fast-math`, `-ffp-contract=off` and the refused compilers were in CMake only: without `-ffp-contract=off`, the compilers fused the multiplications and additions of `mid()`, which returned a single double, not enclosing the midpoint of 9735 of the 10000 random intervals of a test, with meson on macOS arm64 and with autotools on Ubuntu arm64. The three builds now write the same macros in `gaol/gaol_configuration.h`, offer the same options (`GAOL_PRESERVE_ROUNDING`, `GAOL_SIMD`, `GAOL_COVERAGE`, `WITH_TESTS`, `WITH_EXAMPLES`, `BUILD_SHARED_LIBS` and the build type, with their counterparts in `configure` and meson), and decide the rest by the configuration: the fused multiply-add instructions and the AVX-512 path by checks that run a program, the 128-bit integer by a check that compiles. `GAOL_FMA`, `GAOL_PREFER_AVX512`, `GAOL_U128_EMULATION`, `GAOL_DEBUG`, `GAOL_EXCEPTIONS_ENABLED` and `GAOL_VERBOSE_MODE` are settings of the developers, no longer options of the user. The Debug build of each compiles GAOL with `-g`, without optimization nor `NDEBUG`; `--enable-optimize` of `configure` and `-Denable-optimize` of meson are gone, the Release build giving the optimizations. The C sources of CORE-MATH are compiled with `-std=gnu99`. The shared library is `libgaol.so.5.0.0` with the three builds, made when asked, the static one alone by default. GAOL can be a subproject of meson, and CMake without a build type compiles as in Release, where, in the earlier versions of GAOL v5, `libgaol` had `-O3` alone and CORE-MATH no optimization. The unit tests of GAOL 4, which were in `check/` and needed CppUnit, are in `tests/` and run without it; `make test` runs them, `make check` the tests and the examples, `make perf` the benchmark and `make parser` regenerates the lexer and the parser (Chapter 2, see Section 2.9, page 54).

The compilers and the options that give wrong bounds are refused. [GAOL v5] Visual C++ and clang-cl without `/fp:strict` are refused, as `/fp:fast` is: Visual C++ then assumes rounding to nearest, and nothing certifies the bounds. clang-cl for 32-bit x86 is given `-msse2`: clang-cl 18 computes the doubles on the x87 unit there, even with `/arch:SSE2`, and CORE-MATH rounds some results twice. `-ffinite-math-only` is refused, as `-ffast-math` is; GAOL 4 refused none of them. `-ffast-math` and `-Ofast` turn it on, and the compiler then takes NaN and infinities never to occur, in the inline functions of the headers too: the empty interval has NaN bounds, and $([1,2]\cap[3,4])$.`is_empty()` is false once the compiler has folded the test away (GCC 9.4 and 13 at `-O0`, `-O2` and `-O3`; Clang 18 at `-O0`, and at `-O2` and `-O3` when the bounds are `volatile`). With Clang, `-ffast-math` and `-Ofast` turn `-frounding-math` off too, which `-fno-fast-math` does not turn on again: Clang 18 compiled GAOL's operations as if rounding to nearest (54 of the 292728 checks of the test of the rounding direction failed after `-ffast-math -fno-fast-math`, 126 after `-Ofast -fno-fast-math`, none with all the flags after the option). What no macro shows is not refused, and the code using GAOL is not to be compiled with it: `-funsafe-math-optimizations` and `-fno-signed-zeros`, which rewrite

the addition by which GAOL tells the rounding direction and the modes. The compilers that do not honour the rounding direction are refused, the C compiler as well as the C++ compiler: Clang for 32-bit ARM processors, and any compiler that says of `-frounding-math` that the rounding mode is unsupported on its target, as Clang 14 does for 64-bit ARM. GAOL 4 accepted them; built by Clang 21 for 32-bit ARM, GAOL v5 gave 4556 of 16000 random products, squares and cubes that did not enclose their exact values (see Section 2.3.2, page 25).

GCC for Windows is given the fused multiply-add instructions (MinGW-w64, MSYS2 and Cygwin): the three builds give it `-mfma -mprefer-vector-width=128`. [GAOL v5] It had no flag: `-mfma` turns AVX on, GCC then vectorizes with 256-bit registers and spills them to a stack it takes to be aligned on 32 bytes, where the ABI of Windows x64 guarantees 16 (GCC bug 54412, which crashed the MinGW builds of Codac and is fixed for GCC 17, commit 8aaa5f6e6207 of 25 September 2026), so that CORE-MATH called a software `fma()`. With the second flag `objdump` finds no `%ymm` in the objects, against 30598 lines with `-mfma` alone (GCC 16.2, MSYS2 UCRT64). On the runner of GitHub, `pow(x, 3)` takes 8.7 ns rather than 52.3, `log(x)` 33.6 rather than 106.4 and `cos(x)` 55.4 rather than 260.5; with MinGW-w64 15.2 for 32-bit x86, 12.1 ns rather than 70.5, 81.9 rather than 265.7 and 138.3 rather than 489.6. The library then needs a processor with the FMA and AVX instructions (Intel Haswell, AMD Piledriver, 2012–2013, and later), as with the other x86 compilers. Cygwin on x86-64 is built and tested, and refused without the instructions, the `fma()` of its C library not being correctly rounded; with it, `previous_float()` and `next_float()` give $-\infty$ and $+\infty$ at $\pm$`DBL_MAX` with every C library, where the `nextafter()` of Cygwin computed the overflow as $x + x$ in the rounding direction of the program, so that the lower bound of `sinh(-711)` did not enclose its value (see Section 2.1, page 23).

The intervals of floats, `gaol::intervalf` and `gaol::interval2f`, are compiled by a developer of GAOL only. [GAOL v5] Every build compiled and installed them, and `gaol/gaol` included their headers, while neither IBEX nor Codac uses them, the manual leaves them undocumented, and they are unfinished: `sqrt(intervalf)` returns its argument, and `interval2f::inverse()` aborts. Their code is within `#ifdef GAOL_FLOAT_INTERVALS`, a macro no build has an option for. The archive of the sources and the packages are made by CMake: `cmake --build <build> --target package_source` gives `gaol-<version>.tar.gz`, and `--target package` an archive of what is installed and, on Linux, a `.deb`; `make dist` and `make rpm` of the autotools build are gone, `make dist` stopping on a file it could not copy. The version is in `VERSION.txt` alone, where GAOL 4 wrote it in `configure.ac`, in `meson.build` and in two headers, and `configure --help` gives the address for the bug reports. `is_finite()` is `std::isfinite()` in every build, `finite()` of the C library not being declared by every one. `--enable-asm` and `HAVE_ROUNDING_MATH_OPTION` are gone, the first choosing an assembly GAOL v5 no longer calls. CORE-MATH is in the sources, so that GAOL can be built as a part of another project, brought in by FetchContent, with no network access beyond its

own sources. The CMake build derives from the CMake build of GAOL in IBEX, with the compilation flags of the fork of GAOL by Fabrice Le Bars, and the tests of `tests/` follow the rounding tests of Codac (Chapter 14). The Goldstein-Price function of the manual and of the examples had the factor $(x + y)^2$ where the function has $(x + y + 1)^2$, as in GAOL 4; both now compute the true function, whose natural extension over $[-2, 2]^2$ is $[-87881320, 147125080]$, 231 times as wide as the range, which the example says it encloses.

15.4 Discussion

The move from GAOL 4 to GAOL v5 costs a program little: the namespace to open, the strings to read with `textToInterval()`, the relations `==` and `!=` to rewrite with `set_eq()`, `set_disjoint()` or `!set_disjoint()`, the handlers of the exceptions of GAOL 4 to retarget on those of the standard library, and the conversions of a double or an integer to an interval, explicit now, to write out. What the program then sees is decided by three choices that run through the register of this chapter. The first is CORE-MATH as the only source of the elementary functions, which gave the tightest bounds and removed the mathematical library from the link, at the price of sources vendored into GAOL and ported to the compilers that lack a 128-bit integer. The second is the rounding direction taken in charge by each operation, which made the bounds right whatever the direction the program computes in, at the price of a check whose cost was measured on each processor, and whose variants (the control registers written directly, the AVX-512 path, the NEON register) each removed a part of it. The third is IEEE 1788-2015 as the arbiter wherever GAOL 4 had its own convention: the empty set for the constructors, the comparisons, the literals read and written, the domains of `log`, `atanh` and the roots.

Some differences are deliberate losses: the possibly relations and `certainly_neq()`, which was one of them under another name, the timing of GAOL 4, the intervals of floats, the headers of the sources. Others are left as they are, and the register says so: the sign of a zero bound of a product, which depends on the kernel, the quadratic time of the reading of a number of many digits, the inexact exception that almost every operation raises, and the processors where neither the flush-to-zero modes can be cleared nor `-mno-daz-ftz` exists. A program that depended on one of the former behaviours finds here what replaced it, and in the chapters this chapter refers to how the new behaviour is defined. The appendices, after the conclusion, give where the documentation of GAOL v5 lives and how to report what this chapter does not cover.

Conclusion

This manual described GAOL v5, the continuation of the GAOL of Frédéric Goualard, as a user's guide and as a reference. We recalled the notions it rests on, from the roundings of floating-point numbers to the accuracy modes of IEEE 1788-2015; we described its installation with the three builds, the flags the code using it is compiled with, the two namespaces under which its operations are named, and the rounding direction and the floating-point environment that GAOL v5 takes charge of; we gave each operation of the class `interval` with its examples, the names of the standard, and the tightness of every operation; we explained how the intervals are computed and measured, how GAOL is tested, and what a program of GAOL 4 has to change.

Three facts sum up what GAOL v5 established. First, the bounds it computes are the same on every architecture and with every compiler it supports: every elementary function that CORE-MATH provides is bounded with it, correctly rounded in the direction in effect, and the integer powers, the roots and `compoundm1` are the tightest bounds or within a few doubles of them; the tests compare the bounds with their exact values, computed apart, on every platform of the continuous integration. Second, the operations follow IEEE 1788-2015 for the empty set, the infinite bounds, the domains of the functions, the comparisons and the literals; under the names of the standard, GAOL v5 gives the result of the standard in each of the 288 special cases of the comparison with the other libraries where the standard defines one (Chapter 12), and under its own names in all but those of its hybrid `pow`, kept for the programs of GAOL 4. Third, the rounding direction is no longer left to the user: each operation checks it and sets it upward when it is not, and GAOL can be built to restore the direction of the program after each operation; the operations of the AVX-512 path, which that build takes where the machine has the instructions, set no direction, their instructions carrying it, at a cost that the AVX-512 path, where the machine has the instructions, brings back to a few tens of percent.

What GAOL v5 does not provide is stated where it would be expected. GAOL has no decorations: only bare intervals are provided, and a program that needs to know that a box lies outside the domain of a function tests the empty set itself, as Chapter 3 explains. The slope functions, the reduction operations and the exact reduction operations of the standard are not provided either. The intervals of floats are unfinished, and no build compiles them. The `pow` of the namespace `gaol` is a hybrid of the two powers of the standard, which keeps the results of GAOL 4 for the

programs written for it and for IBEX and Codac; a program that wants the results of the standard opens `gaol_ieee1788`.

What is left to measure concerns the processors on which GAOL v5 could not be timed. Of the instruction sets read for Chapter 12, the scalar instructions of RISC-V are the only ones, beside the AVX-512 path of x86, that carry their rounding direction; a path written with them would need neither a check nor a restore of the direction, and waits for a board to be measured on. The vector units of POWER and s390x hold two doubles per register as NEON does, and wait for hardware as well. Until then, the FPU intervals compute on these processors, with the same bounds, as the continuous integration checks under emulation.

Bibliography

[1] Götz Alefeld and Jürgen Herzberger. *Introduction to Interval Computations*. Academic Press Inc., New York, USA, 1983. Translated by Jon Rokne from the German original *Einführung in die Intervallrechnung*.

[2] David Marc Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, March 1991.

[3] Frédéric Goualard. Interval extensions of multivalued inverse functions. Technical Report hal-00288457, HAL, 2008.

[4] Frédéric Goualard. Fast and correct SIMD algorithms for interval arithmetic. In *PARA '08*, volume 6126–6127 of *Lecture Notes in Computer Science*. Springer, 2012.

[5] Frédéric Goualard. How do you compute the midpoint of an interval? *ACM Transactions on Mathematical Software*, 40(2), 2014.

[6] Eldon Robert Hansen. *Global Optimization Using Interval Analysis*. Pure and Applied Mathematics. Marcel Dekker Inc., 1992.

[7] Timothy J. Hickey, Qun Ju, and Maarten H. van Emden. Interval arithmetic: from principles to implementation. *Journal of the ACM*, 48(5):1038–1068, 2001.

[8] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Institute of Electrical and Electronics Engineers, 2008. IEEE Std 754-2008, https://doi.org/10.1109/IEEESTD.2008.4610935.

[9] IEEE. *IEEE Standard for Interval Arithmetic*. Institute of Electrical and Electronics Engineers, 2015. IEEE Std 1788-2015, https://doi.org/10.1109/IEEESTD.2015.7140721.

[10] Ramon Edgar Moore. *Interval Analysis*. Prentice-Hall, Englewood Cliffs, N. J., 1966.

[11] Arnold Neumaier. *Interval Methods for Systems of Equations*, volume 37 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 1990.

[12] Helmut Ratschek and Jon Rokne. Interval methods. In *Handbook of Global Optimization*, pages 751–828. Kluwer Academic, 1995.

[13] Alexei Sibidanov, Paul Zimmermann, and Stéphane Glondu. The CORE-MATH project. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*, pages 26–34, 2022.

[14] Volker Stahl. *Interval Methods for Bounding the Range of Polynomials and Solving Systems of Nonlinear Equations*. PhD thesis, Johannes Kepler Universität, Linz, September 1995.

[15] Xuan Tang, Zachary Ferguson, Teseo Schneider, Denis Zorin, Shoaib Kamil, and Daniele Panozzo. A cross-platform benchmark for interval computation libraries. In *Parallel Processing and Applied Mathematics (PPAM 2022)*, volume 13827 of *Lecture Notes in Computer Science*. Springer, 2023.

Appendix A

Documentation and references

A.1 Documentation on GAOL

The primary reference is this manual. The directory `doc` of the sources holds the pages that give the details of GAOL v5, with the measures behind its choices, whose content the chapters of this manual take up:

- `doc/building.md`: the CMake, autotools and meson builds, and their options (Chapter 2);
- `doc/using.md`: the flags of interval arithmetic, GAOL from CMake and from pkg-config, the namespaces, the rounding direction and the floating-point environment (Chapters 3 and 4);
- `doc/three-builds.md`: what the three builds agree on, what the options are worth, and the compilers and options they refuse (Chapter 2);
- `doc/tests.md`: what the programs of `tests` check (Chapter 14);
- `doc/accuracy.md`: the tightness of each operation, with its algorithm and what the tests found (Chapter 11);
- `doc/differences.md`: each change GAOL v5 brings to GAOL 4.2.2, and where it comes from (Chapter 15);
- `doc/continuous-integration.md`: the systems, processors and compilers GAOL is built and tested on (Chapter 14);
- `doc/simd.md`: what GAOL computes with vector instructions and with the fused multiply-add, where the time of a program goes, and the rounding direction by processor (Chapter 12);
- `doc/math-libraries.md`: the mathematical libraries libm, mathlib, CRlibm and CORE-MATH, and what separates them (Chapter 1);

- `doc/tools.md`: the scripts of the repository and what each is for (Chapter 14);
- `doc/compare/README.md`: GAOL compared with libieeep1788, filib++, PROFIL/BIAS, Solaris Studio and Boost.Interval, their special cases against IEEE 1788-2015, the enclosure of the elementary functions of GAOL and Boost.Interval, their performance, and the test suites ITF1788 and of Tang et al. (Chapter 12);
- `examples/examples.md` and `examples/tutorial.md`: the example programs, a review of GAOL v5 and a comparison with other interval libraries, and a short tutorial (Chapter 14);
- `3rd/README.md`: the version of CORE-MATH GAOL takes, what differs from it, and the fixes to propose upstream.

The manual of GAOL 4, by Frédéric Goualard, is kept in `manual/v4`; its PDF, `manual/v4/Manual_GAOLv4.pdf`, was built once, and no build makes it again.

A.2 References

The following articles, books and standards have inspired in some way or another the devising of the GAOL library and the writing of this manual.

- *IEEE Standard for Interval Arithmetic*. IEEE Std 1788-2015 [9].
- *The CORE-MATH Project*. Alexei Sibidanov, Paul Zimmermann and Stéphane Glondu [13], and https://core-math.gitlabpages.inria.fr/.
- *How do you compute the midpoint of an interval?* Frédéric Goualard [5].
- *Fast and Correct SIMD Algorithms for Interval Arithmetic*. Frédéric Goualard [4].
- *Interval Extensions of Multivalued Inverse Functions*. Frédéric Goualard [3].
- *Interval Arithmetic Specification*. Dmitri Chiriaev and G. William Walster. Draft revised May 1998.
- *The Extended Real Interval System*. G. William Walster. April 1998.
- *C++ Interval Arithmetic Programming Reference*. Sun Microsystems, Inc. October 2000, revision A.
- *Interval Arithmetic: From Principles to Implementation*. T. Hickey, Q. Ju, and M. H. van Emden [7].

A.3 Reporting bugs

Bugs and suggestions for improvement of GAOL v5 are submitted as issues of its repository:

https://github.com/Jordan08/GAOL/issues

A report is most useful with the program that shows the problem, the interval it gives and the one expected, the system, the processor, the compiler and the options GAOL was built with.

Appendix B

Library copying

GAOL is distributed under the GNU Library General Public License, version 2, reproduced below. The copyright for the initial version, named `cell` (year 2001), is owned by the *Swiss Federal Institute of Technology*, Lausanne, Switzerland. The copyright for the following versions (from 2002 onward) is owned by the *Laboratoire d'Informatique de Nantes-Atlantique*, then by the *Laboratoire des Sciences du Numérique de Nantes*, France. The files GAOL v5 adds carry the copyright of ENSTA, France, and are distributed under the same licence as GAOL; the files of GAOL that GAOL v5 changes keep the copyright of their authors.

GAOL v5 computes its elementary functions with CORE-MATH [13], whose sources are distributed with GAOL (in `3rd/math-core`) and compiled into its library. CORE-MATH is distributed under the MIT licence, reproduced after the licence of GAOL.

B.1 The licence of GAOL

GNU LIBRARY GENERAL PUBLIC LICENSE

Version 2, June 1991

Copyright © 1991 Free Software Foundation, Inc.
59 Temple Place - Suite 330
Boston, MA 02111-1307, USA.

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

[This is the first released version of the library GPL. It is numbered 2 because it goes with version 2 of the ordinary GPL.]

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licenses are intended to guarantee your freedom to share and change free software—to make sure the software is free for all its users.

This license, the Library General Public License, applies to some specially designated Free Software Foundation software, and to any other libraries whose authors decide to use it. You can use it for your libraries, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library, or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link a program with the library, you must provide complete object files to the recipients so that they can relink them with the library, after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

Our method of protecting your rights has two steps: (1) copyright the library, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the library.

Also, for each distributor's protection, we want to make certain that everyone understands that there is no warranty for this free library. If the library is modified by someone else and passed on, we want its recipients to know that what they have is not the original version, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that companies distributing free software will individually obtain patent licenses, thus in effect transforming the program into proprietary software. To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License, which was designed for utility programs. This license, the GNU Library General Public License, applies to certain designated libraries. This license is quite different from the ordinary one; be sure to read it in full, and don't assume that anything in it is the same as in the ordinary license.

The reason we have a separate public license for some libraries is that they blur the distinction we usually make between modifying or adding to a program and simply using it. Linking a program with a library, without changing the library, is in some sense simply using the library, and is analogous to running a utility program or application program. However, in a textual and legal sense, the linked executable is a combined work, a derivative of the original library, and the ordinary General Public License treats it as such.

Because of this blurred distinction, using the ordinary General Public License for libraries did not effectively promote software sharing, because most developers did not use the libraries. We concluded that weaker conditions might promote sharing better.

However, unrestricted linking of non-free programs would deprive the users of those programs of all benefit from the free status of the libraries themselves. This Library General Public License is intended to permit developers of non-free programs to use free libraries, while preserving your freedom as a user of such programs to change the free libraries that are incorporated in them. (We have not seen how to achieve this as regards changes in header files, but we have achieved it as regards changes in the actual functions of the Library.) The hope is that this will lead to faster development of free libraries.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a "work based on the library" and a "work that uses the library". The former contains code derived from the library, while the latter only works together with the library.

Note that it is possible for a library to be covered by the ordinary General Public License rather than by this special one.

Terms and Conditions for Copying, Distribution and Modification

0. This License Agreement applies to any software library which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Library General Public License (also called "this License"). Each licensee is addressed as "you".

   A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

   The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

   "Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

   Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only

if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

   You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

   a. The modified work must itself be a software library.

   b. You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.

   c. You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.

   d. If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

      (For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

   These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

   Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

   This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

   If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

   However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

   When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is

especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also compile or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

   You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

   a. Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)

   b. Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.

   c. If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.

   d. Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

   a. Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.

   b. Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise

of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

    If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

    It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

    This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

13. The Free Software Foundation may publish revised and/or new versions of the Library General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

    Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library

does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

15. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Libraries

If you develop a new library, and you want it to be of the greatest possible use to the public, we recommend making it free software that everyone can redistribute

and change. You can do so by permitting redistribution under these terms (or, alternatively, under the terms of the ordinary General Public License).

To apply these terms, attach the following notices to the library. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

> *one line to give the library's name and a brief idea of what it does.*
> Copyright (C) *year name of author*
>
> This library is free software; you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.
>
> This library is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public License for more details.
>
> You should have received a copy of the GNU Lesser General Public License along with this library; if not, write to the Free Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA

Also add information on how to contact you by electronic and paper mail.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the library, if necessary. Here is a sample; alter the names:

> Yoyodyne, Inc., hereby disclaims all copyright interest in the library 'Frob' (a library for tweaking knobs) written by James Random Hacker.
>
> *signature of Ty Coon*, 1 April 1990
> Ty Coon, President of Vice

That's all there is to it!

B.2 The licence of CORE-MATH

The sources of CORE-MATH, in `3rd/math-core`, which GAOL compiles into its library, carry the copyright notices of their authors, as each file states (Inria, Paul Zimmermann, Alexei Sibidanov, Stéphane Glondu and others), and are distributed under the following licence (MIT licence), whose copyright notice is theirs:

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Index

-Ofast, 104
-frounding-math, 235

↓↓, *see* downward rounding
↑↑, *see* upward rounding
abs, **176**
accuracy
    conditions, 206
    modes, 20
    of the operations, 205
    table of the operations, 206
    the same bounds on every machine, 224
accurate, 20
acos, **155**
acos_rel, **160**
acosh, **157**
acosh_rel, **161**
acospi, **156**
Allen's relations, 136
asin, **156**
asin_rel, **161**
asinh, **157**
asinh_rel, **161**
asinpi, **156**
assignment, 123
atan, **156**
atan2, **156**
atan2_rel_x, **167**
atan2_rel_y, **167**
atan2pi, **156**
atan2Rev1, 202
atan2Rev2, 202
atan_rel, **161**
atanh, **158**
atanh_rel, **161**
atanpi, **156**
AVX-512, 32, 39, 94, 228

bare interval, 12
benchmark of Tang et al., 251
benchmark of the libraries, 241
bisect, 74, **180**
bisection, 179
    of an unbounded interval, 80
BISON, 30
bison, 29
Boost.Interval, 83, 237
builds
    differences with GAOL 4, 340

C++
    standard, 24
C-XSC, 83
cancel_minus, **168**
cancel_plus, **169**
cancellative operations, 168
canonical interval, **8**, 77
ceil, **178**
cell, 353
certainly_ge, **135**
certainly_geq, **135**
certainly_le, **134**
certainly_leq, **134**
certainly_negative, **136**
certainly_positive, **135**
certainly_strictly_negative, **136**
certainly_strictly_positive, **135**
CFLAGS, 36, 296
check_examples.py, 294
chi, **176**
cleanup, 112

`cleanup`, **113**
CMake, 33
    using gaol from, 66
Codac, 83
common errors, 77
comparison of libraries, 237
compiler
    refused, 25
compiling
    a file using gaol, 62, 64
`compoundm1`, **158**, 217
`configure`, 44
constants
    differences with GAOL 4, 326
constructors
    differences with GAOL 4, 329
containment property, 9, 88
continuous integration, 271
    configurations refused, 301
CORE-MATH, 15, 153, 353
    accuracy, 205
    differences with GAOL 4, 318
    scalar, 231
correct rounding, 15
`cos`, **155**
`cosh`, **157**
`cospi`, **156**
coverage, 38, 309
CRlibm, 15
`crtfastmath.o`, 104
`CXX`, 294
`CXXFLAGS`, 36, 52, 53, 296

`debug_level`, **269**
debugging, 269
    option, 45
decorations, 12, 203
denormals-are-zero, 104
dependency problem, 11, 63
`div_rel`, **164**
domain of a function, 12, 79, 148
double, 6
    neighbours, 7
downward rounding ($\downarrow\downarrow$), 6, 88

elementary functions, 15
    differences with GAOL 4, 333
empty
    interval: constructing, 118
    interval: printing, 190
    interval: representation, 125
`[empty]`, 190
empty set, 8
    differences with GAOL 4, 329
    in tests, 79
enclosure, 11
enclosure test, 250
`erf`, **159**
`erfc`, **159**
error handling, 265
errors, 265
    overview, 76
`even`, **264**
exact operations, 11
`exact_string`, **190**
examples, 290
exceptions, 266
    differences with GAOL 4, 338
    floating-point, 107
`exp`, **153**
`exp10`, **154**
`exp10m1`, **154**
`exp2`, **154**
`exp2m1`, **154**
`expm1`, **154**
exponential, 153

$\mathbb{F}$, 6, 88
faithful rounding, 15
`FENV_ROUND`, 235
`feven`, **262**
filib++, 237
first program, 61
flags
    floating-point, 107
flags of interval arithmetic, 64
`FLEX`, 30
`flex`, 29
floating-point constants, 261

floating-point exceptions, 107
differences with GAOL 4, 324
floating-point functions, 262
floating-point numbers, 5
`floor`, **177**
`floor_rel`, **166**
flush-to-zero, 28, 104
`fma`, **150**
format
of the output, 191
`format`, **193**
FPCR, 96, 233
FPSCR, 96
FPU intervals, 98
function
undocumented, 3
functional arithmetic, 147
fused multiply-add, 32, 39, 231

gaol
namespace, 62
pronunciation, 1
web page, 24
GAOL v5, 1
`gaol_configuration.h`, 56
`GAOL_DEBUG_VERBOSE`, **269**
`GAOL_DEBUGGING`, 269
`gaol_ERROR`, **265**
`gaol_error`, **266**
`GAOL_FILE_POS`, **267**
`GAOL_FMA`, 231
`GAOL_INFINITY`, 261
`GAOL_NAN`, 261
`GAOL_NODISCARD`, 73
`gaol_performance`, 289
`GAOL_PREFER_AVX512`, 230
`GAOL_PRESERVE_ROUNDING`, 93, 230
`gaol_warning`, **268**
GAOL 4
compiling a program of, 314
differences with, 317
moving from, 313
results that change, 316

`hausdorff`, **174**
Hausdorff distance, 171
header
and namespace, *see* namespace
headers, 63
differences with GAOL 4, 339
help
on configuration, 44
hexadecimal
form, 193
hull, 8
of a union, 181
`hull`, **181**
hyperbolic functions, 157
`hypot`, **158**

$\mathbb{I}$, 7
IBEX, 83
IEEE 1788-2015, 1, 11
differences with GAOL, 198
names, 13, 197
operations, 199
operations not provided, 203
order, 136
reading, 203
relations, 12, 130
infinity, 6, 261
`inflate`, 74, **173**
`init`, **113**
initialization, 112
input
differences with GAOL 4, 336
input format, 184
integer
as a bound, 119
`integer`, **178**
integer functions, 177
integers
given to GAOL, 72
`intersect`, **181**
intersection, 181
interval
canonical, *see* canonical interval
computation site (web), 3
constants, 125

constructors, 117
conversion to `string`, 191
degenerate, 8
floating-point, 7
midpoint and radius, 9, 78
reading, 183
unbounded, 8
width, 9
with infinite bounds, 122
writing, 190
zero, 78
interval arithmetic, 5
interval literal, 13
`interval::format`
threads, 82
`interval_format`, **193**
`invabs_rel`, **163**
`inverse`, **150**
`is_a_double`, **142**
`is_an_int`, **142**
`is_bisectable`, 74, **180**
`is_canonical`, **142**
`is_common_interval`, **143**
`is_empty`, **143**
`is_entire`, **143**
`is_finite`, **144**
`is_signed`, **262**
`is_symmetric`, **144**
`is_zero`, **144**
ITF1788, 251

kernel, 228
kernels
differences with GAOL 4, 326

`LD_LIBRARY_PATH`, 36
`LDFLAGS`, 29, 36, 296
`left`, **176**
`less`, **137**
`lexicographic_less`, 140
libieeep1788, 237
libm, 15
libraries
names in other, 83
licence
of CORE-MATH, 353
of GAOL, 353
`log`, **154**
`log10`, **154**
`log10p1`, **154**
`log1p`, **154**
`log2`, **154**
`log2p1`, **154**
logarithm, 153
lower bound
stored negated, 10

`mag`, **174**
magnitude, 171
mathematical library, 15
mathlib, 15
`max`, **177**
`max_rel`, **165**
`maximum`, **263**
meson, 46
using gaol from, 68
`mid`, **175**
`mid_rad`, **172**
midpoint, 171
cut at the, 179
`midpoint`, **175**
`midrad`, 74, **121**
`mig`, **173**
mignitude, 171
migration from GAOL 4, 313
`min`, **176**
`min_rel`, **165**
`minimum`, **263**
`mulRevToPair`, 202
MXCSR, 103, 233

namespace
gaol, 62, 69
gaol_core, 69
gaol_ieee1788, 13, 69, 197
namespaces
differences with GAOL 4, 339
NaN, 6, 261
`nb_fp_numbers`, **263**
`NDEBUG`, 269

NEON, 31, 37, 96, 228
Newton's method
    division, 80
next_float, **262**
nth_root, **153**, 215
nth_root_rel, **162**
number
    reading, 186
numeric functions, 171

odd, **264**
operator
    relational, 1, 13
operator*, **149**
operator*=, **125**
operator+, **149**
operator+=, **124**
operator-, **149**
operator-=, **124**
operator/, **149**
operator/=, **125**
operator<, **139**
operator<=, **139**
operator<<, **190**
operator>, **139**
operator>=, **139**
operator>>, **183**
operator%, **150**, **160**
operator%=, **125**
operator&, **181**
operator&=, **123**
operator|, **181**
operator|=, **124**
optimization
    configuration option, 45
    interprocedural, 53
output
    differences with GAOL 4, 336
output format, 191
outward rounding, 9, 88
overlap, 136
overlap, **138**

package, 42
PATH, 48, 296, 298
paths of GAOL
    verdicts, 253
pkg-config, 67
PKG_CONFIG_PATH, 67, 68, 294
platforms, 23
pow
    in the two namespaces, 70
pow, **151**, 211
pow_exponent_rel, **164**
pow_rel, **164**
power, 151
powers
    differences with GAOL 4, 331
precision
    of the output, 195
precision, **195**
previous_float, **262**
PROFIL/BIAS, 83, 237
profile of a program, 231

$\mathbb{R}$, 6, 88
rad, **172**
radius, 171
references
    on interval arithmetic, 3
register of an interval, 228
relation
    certainly, 129
    interval-specific, 141
    possibly (removed), 130
    set, 129
    symbols, 138
        pitfalls, 79
relational arithmetic, 147, 160
relational function, *see* reverse functions
relational operator, 1, 13
relations
    differences with GAOL 4, 335
restore_rounding, **114**
result thrown away, 73
reverse functions, 13, 160
    accuracy, 219
right, **176**

RISC-V, 253
root, 151
round_downward, **115**
round_nearest, **115**
round_ties_to_away, **178**
round_ties_to_even, **178**
round_upward, **115**
rounding
  direction, 6, 10, 88
  downward, **6**, 88
  effects on the program, 90
  functions, 115
  left as found, 89
  outward, *see* outward rounding
  preserving, 37, 45, 89, 93
  to an integer, 177
  to nearest, **7**, 77
  upward, **6**, 88
rounding direction
  cost of a write, 233
  differences with GAOL 4, 320
rsqrt, **158**

scripts, 305
self, 123
set-based flavor, 12
set_contains, **130**, **131**
set_disjoint, **132**
set_eq, **132**
set_ge, **133**
set_geq, **134**
set_le, **133**
set_leq, **133**
set_neq, **133**
set_strictly_contains, **131**, **132**
sign, **178**
sign_rel, **166**
sin, **155**
sinh, **157**
sinpi, **156**
smig, **174**
Solaris Studio, 237
soname, 33
sorting intervals, 140
special cases
  comparison, 249
split, **179**
split_left, **179**
split_right, **180**
splitting, 179
sqr, **150**
sqrt, **150**
sqrt of a number, 77
sqrt_rel, **162**
SSE2, 31, 37, 228
std::bad_alloc, 266
std::domain_error, 266
std::invalid_argument, 266
std::logic_error, 266
std::set, 140
straddles_zero, **141**
strictly_less, **137**
strictly_straddles_zero, **141**
string
  converting interval to, *see* interval
subnormal numbers, 6, 104
SVE, 253

table maker's dilemma, 15
tan, **155**
tanh, **157**
tanpi, **156**
tests, 54, 271
  principles, 271
  programs, 274
textToInterval, **119**, **120**, 203
  errors, 81
this, 123
threads
  output of intervals, 82
tightest, 20
tightness, 20, 205
tool
  mandatory to compile GAOL, 24
tools of interval algorithms, 74
trigonometric functions, 155
trunc, **178**
trust rounding mode, 88

uncertain form, 186
undocumented functions, 3
union of intervals, 181
unit in the last place, 6
universal build, 31
upward rounding (↑ ↑), 6, 88
USERPROFILE, 48

valid, 20
version, **264**
version of the library, 264
VERSION.txt, 34
version_major, **264**
version_micro, **264**
version_minor, **264**

warnings, 268
width
    of an interval, 171
width, **172**
width_enclosure, 74, **173**

x87 control word, 103

zero
    sign of a bound, 6, 78
    sign of a zero bound, 224